TURING 图灵计算机科学丛书

PEARSON Addison Wesley

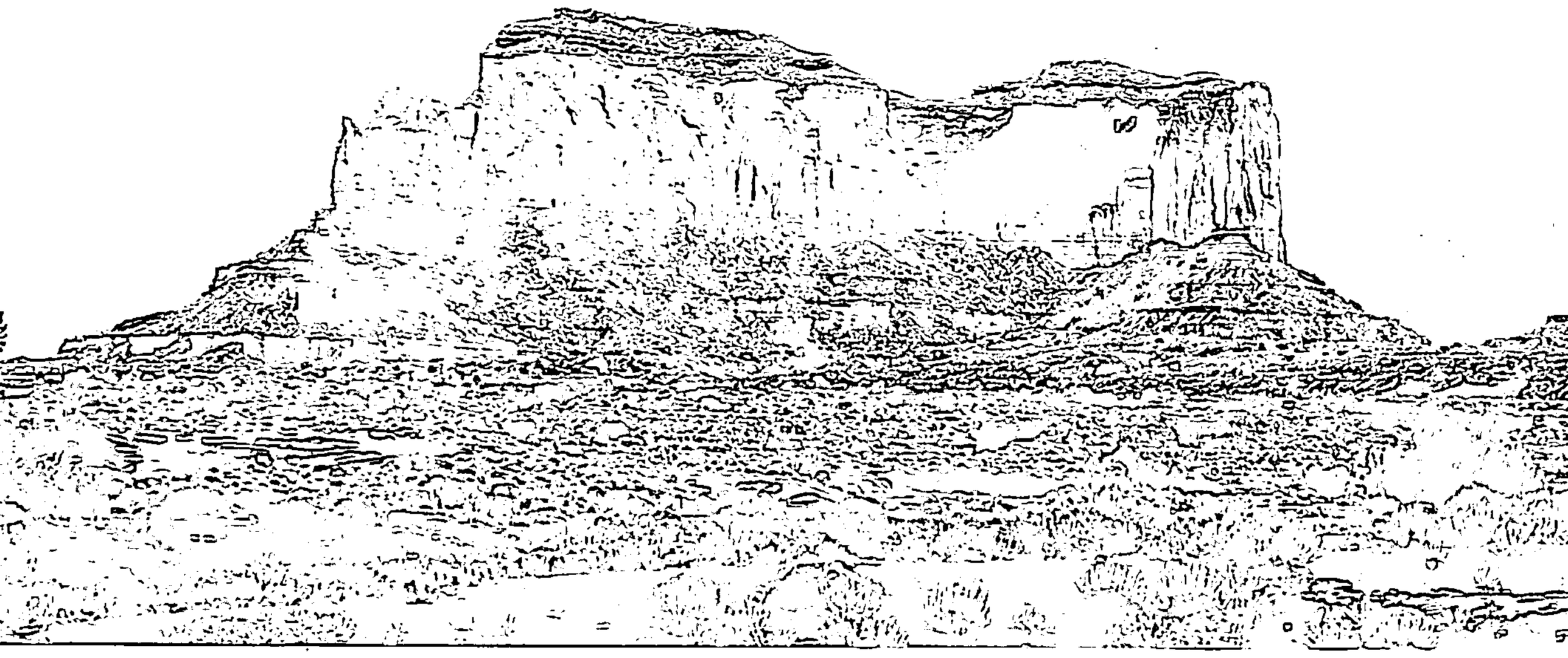
数据库系统基础

高级篇（第5版）

Fundamentals of Database Systems

Fifth Edition

[美] Ramez Elmasri Shamkant B. Navathe 著
邵佩英 徐俊刚 王文杰 等译



人民邮电出版社
POSTS & TELECOM PRESS

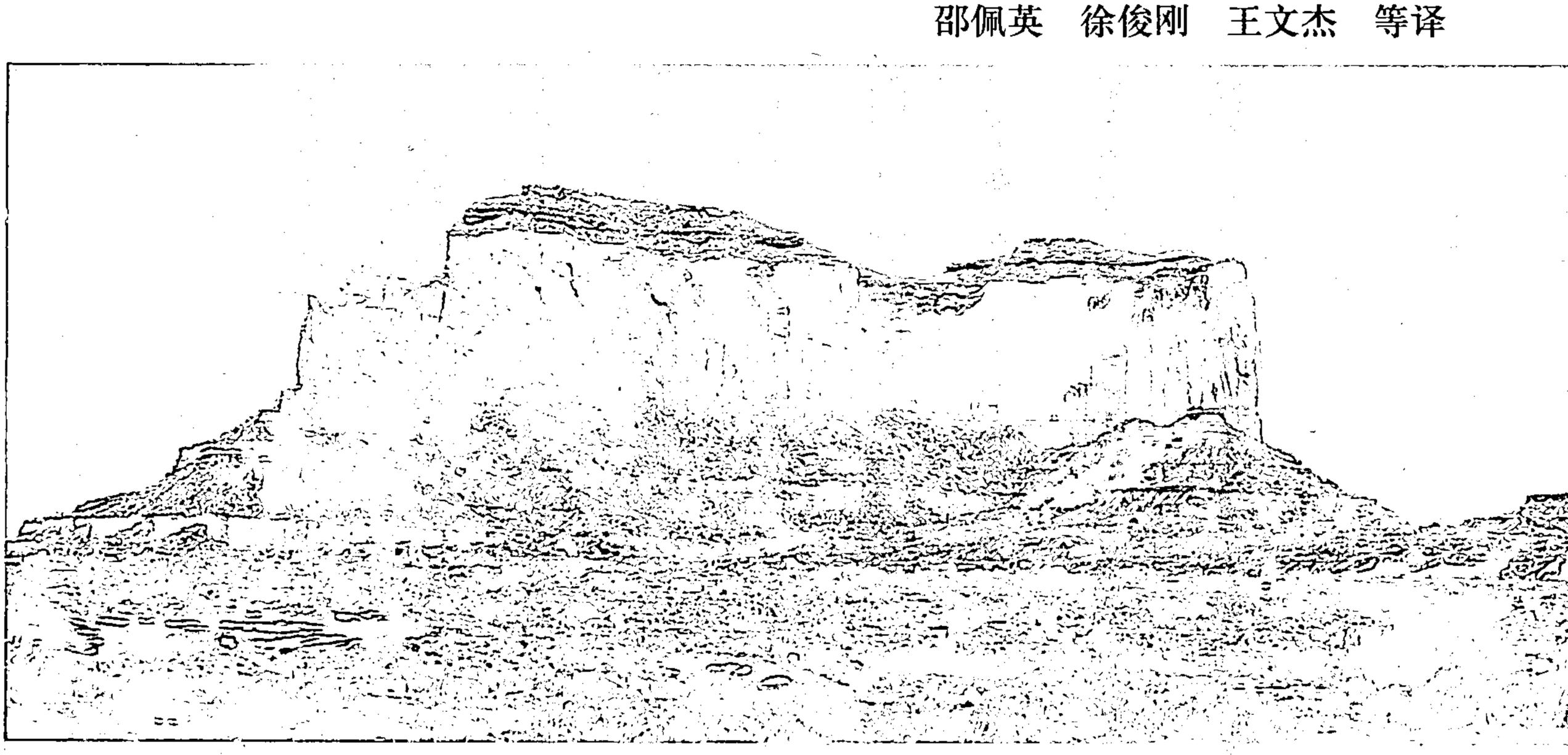
TP311.13
457
仁高

TURING 图灵计算机科学丛书

数据库系统基础
高级篇（第5版）

Fundamentals of Database Systems
Fifth Edition

[美] Ramez Elmasri Shamkant B. Navathe 著
邵佩英 徐俊刚 王文杰 等译



人民邮电出版社
北 京

图书在版编目（CIP）数据

数据库系统基础：第5版．高级篇／（美）埃尔姆斯里（Elmasri, R. E.），（美）内瓦西（Navathe, S. B.）著；邵佩英等译．—北京：人民邮电出版社，2008.1
（图灵计算机科学丛书）
ISBN 978-7-115-17160-3

Ⅰ.数… Ⅱ.①埃…②内…③邵… Ⅲ.数据库系统-理论-教材 Ⅳ.TP311.13

中国版本图书馆CIP数据核字（2007）第174735号

内 容 提 要

本书是一本讲述数据库系统原理的经典教材，重点强调了数据库的设计与实现。全书分为4部分。第一部分讲述事务处理的概念、并发控制技术和数据库恢复技术；第二部分讲述面向数据库的概念、语言、设计及对象-关系和扩展-关系系统；第三部分主要描述了数据库安全和客户-服务器体系结构；第四部分简要介绍了XML、数据挖掘和数据仓库等几种新兴技术。

本书是高等院校计算机及相关专业本科生、研究生数据库系统课程的理想教材，也是对相关技术人员非常有价值的参考书。

图灵计算机科学丛书

数据库系统基础：高级篇（第5版）

◆ 著 [美]Ramez Elmasri Shamkant B.Navathe
译 邵佩英 徐俊刚 王文杰等
责任编辑 杨海玲 刘 静

◆ 人民邮电出版社出版发行 北京市崇文区夕照寺街14号
邮编 100061 电子函件 315@ptpress.com.cn
网址 http://www.ptpress.com.cn
北京艺辉印刷有限公司印刷
新华书店总店北京发行所经销

◆ 开本：787×1092 1/16
印张：19.75
字数：585千字 2008年1月第1版
印数：1-4 000册 2008年1月北京第1次印刷

著作权合同登记号 图字：01-2007-1969号

ISBN 978-7-115-17160-3/TP

定价：49.00元

读者服务热线：(010)88593802 印装质量热线：(010)67129223
反盗版热线：(010)67171154

版 权 声 明

Authorized translation from the English language edition, entitled *Fundamentals of Database Systems, Fifth Edition*, 9780321369574 by Ramez Elmasri and Shamkant B. Navathe, published by Pearson Education, Inc., publishing as Addison Wesley, Copyright © 2007 by Ramez Elmasri and Shamkant B. Navathe.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc.

CHINESE SIMPLIFIED language edition published by PEARSON EDUCATION ASIA LTD. and POSTS & TELECOM PRESS Copyright © 2007.

本书中文简体字版由 Pearson Education Asia Ltd.授权人民邮电出版社独家出版。未经出版者书面许可，不得以任何方式复制或抄袭本书内容。

本书封面贴有 Pearson Education（培生教育出版集团）激光防伪标签，无标签者不得销售。

版权所有，侵权必究。

译 者 序

近几年来，许多出版社影印和翻译了多种数据库系统方面的教材，我们也曾与中国人民大学张坤龙等老师一起翻译了《数据库系统基础》的第3版。凭借多年来从事数据库课程教学的实践经验，我们认为Elmasri和Navathe的《数据库系统基础》是一本出色的、系统的、先进的、实用的数据库课程教科书。它清晰地阐述了数据库系统理论和设计问题，内容极为充实，广泛涵盖了所有涉及数据库系统设计、实现和管理的诸多专题。全书贯穿两个实例COMPANY和UNIVERSITY，引入了最新的现代数据库技术，使得读者能够使用同一个应用来比较不同的方法。我们初步阅读Elmasri和Navathe的《数据库系统基础》第5版时，发现它对全书又做了完善性修改和更新，适时地反映了数据库技术和应用开发的最新发展趋势，而且它新增了一些应用实例使读者可以参与其中，章后练习题和新的实验题又可以让读者获得基于现代数据库的技术，如Oracle、MySQL和SQL Server，以及了解使用PHP和XML的Web数据库编程，获得建造数据库应用系统的实践经验。《数据库系统基础》是从技术角度介绍数据库系统的权威教材，它既适合于研究生使用又适合于本科生使用，更为相关教师提供了足够丰富的数据库课程教学素材。当然，《数据库系统基础》一书也是从事数据库技术研究和应用开发人员的良师益友。

这就是为什么我们要花费大量精力和时间来翻译《数据库系统基础》一书的原因。

《数据库系统基础》一书包括8个部分和5个附录。从第一部分开始，介绍了为更好地掌握数据库设计与实现所需的基本概念，以及数据库系统中用到的概念建模技术。第二部分到第四部分深入讲解了数据库系统基础中最重要的几个方面，包括关系模型的概念、约束、语言、设计和编程，数据库设计理论和方法学，数据存储、索引、查询处理和物理设计。第五部分和第六部分是数据库系统的高级专题，包括：事务处理、并发控制、恢复技术，以及对象和对象-关系数据库，并给出了如何用SQL实现这些概念。第七部分和第八部分作为结束，对诸如数据库安全性，适应高级应用的增强数据模型，分布式数据库及三层客户-服务器体系结构，以及XML、Web数据库、数据挖掘、数据仓库、移动数据库、多媒体数据库、地理信息系统、基因数据管理等新兴技术作了介绍。附录A给出许多可选用的显示ER或EER模式的图形记号，附录B给出磁盘的一些重要物理参数，附录C给出最早的图形化查询语言之一——示例查询（Query-By-Example，QBE）语言的概述，附录D和附录E介绍基于层次数据模型和网状数据模型的传统数据库系统（http://www.aw.com/elmasri）。

《数据库系统基础》一书分为“初级篇”和“高级篇”两册：“初级篇”包括第一部分到第四部分及附录，可以作为“数据库导论课程”的教材；“高级篇”包括第五部分到第八部分及附录，可以作为“数据库高级课程”的教材。当然，正如在前言中所述，讲授一门数据库课程有许多种不同的方式，各位教师可根据课程侧重点的不同，以及根据学生的背景和需求组织教学内容。可以跳过那些选择性的章，也可以添加书中其余部分的章节以便充实课程。

本书的翻译和审校由中国科学院研究生院信息学院邵佩英教授、徐俊刚副教授和王文杰副教授共同组织完成。参加翻译的还有（按姓氏笔画为序）裴莹、陈明、丁杰、林欢欢、余清、张坤、张

颖和周良等。全书由邵佩英教授负责统一定稿。由于水平有限，翻译难免存在不当之处，恳请读者批评指正。

本书翻译过程中曾得到中国科学院高能物理所李伯民研究员的关心和指导，得到我院研究生王辉、祝孔强、阚劲松的帮助，在此向他们表示衷心的感谢。

译者
中国科学院研究生院信息学院
2007年10月

前　言

*Fundamentals of Database Systems, Fifth Edition*一书[①]介绍了在设计、使用和实现数据库系统及应用时所需的基本概念，重点强调了数据库建模与设计的基础、数据库管理系统提供的语言和工具，以及系统实现技术。本书主要阐述数据库的高级主题，旨在作为研究生的数据库系统实现课程的教材或教学参考书，故要求阅读本书的读者具备数据库系统的基本概念、术语，数据库概念、建模原则，关系数据模型和关系型DBMS，数据库设计，数据库系统中使用的物理文件结构和存取方法等方面的基础知识。此外，本书假定读者已熟悉基本的程序设计和数据结构的概念，并对基本的计算机组成知识有所了解。

本书从第一部分和第二部分深入讲解了事务处理以及对象和对象-关系数据库的内容，第三部分和第四部分对数据挖掘、XML、安全、Web数据库等新兴技术进行介绍。

以下是第5版的主要特色：

- 本书内容的组织遵循独立性和灵活性原则，可以根据个人需要进行取舍。
- 对每章末尾的习题集进行了更新和扩充。
- 更新了安全、移动数据库、GIS和生物信息学数据管理方面的内容。
- 提供了一个辅助网站（http://www.aw.com/elmasri），其中包括可以装载到各种类型关系数据库中的数据，以便学生更好地进行实验题。
- 对补充资料进行了重要的修订，包括为教师和学生准备的一组资料，例如PowerPoint幻灯片和书中的图表，以及带有答案的教师指导手册。

与第 4 版的主要区别

第5版对全书内容的组织结构改动较少，主要改动集中于对单独某一章的改进上。主要改动如下：

- 更新了第7章关于安全的部分。
- 新增了第10章，介绍使用PHP脚本语言的Web数据库编程。
- 修订了第14章中关于新兴数据库技术和应用的内容，以反映移动数据库、地理信息系统和基因数据管理方面的新成就。
- 给出了一种新设计，这种设计增强了书中插图的视觉效果，并对各种属性和实体类型使用了特殊字体，从而增强了可读性和可理解性。

第 5 版的内容

第一部分讨论事务处理、并发控制和恢复技术，还讨论了如何用SQL实现这些概念。

第二部分全面介绍对象数据库和对象-关系系统。第4章讨论面向对象的概念；第5章详细介绍

---

① 在本书翻译出版过程中，为考虑与国内数据库教学内容的接轨，将*Fundamentals of Database Systems, Fifth Edition*分两册翻译出版，一本是您正在阅读的本书，另一本是本书的姊妹篇《数据库系统基础：初级篇》，后者主要对数据库的DBMS的概念、术语和体系结构，ER模型和ER图的概念，数据抽象和语义数据建模的概念，UML类图表示法，基本关系模型，关系代数和关系演算，SQL，规范化，磁盘上组织记录文件的主要方法，文件的索引技术，查询处理与优化，以及物理数据库的设计与调优进行了介绍。——编者注

ODMG对象模型，以及与之相关的ODL和OQL；第6章描述如何扩展关系数据库，使其能够引入面向对象的概念，同时介绍对象-关系系统的特性，并回顾SQL 3标准的一些特性以及嵌套关系数据模型。

第三部分和第四部分讨论了一些高级主题。第7章给出了数据库安全的概述，其中包括使用SQL命令进行授权（GRANT）和取消权限（REVOKE）的自主访问控制模型，以及涉及用户分类和多实例化的强制访问控制模型，还介绍扩展的安全控制措施，如访问控制、推理控制、信息流控制和数据加密，还有隐私，以及尚未解决的安全问题；第8章讨论一些用于高级应用的增强型数据库模型，其中包括主动数据库与触发器，还有时态数据库、空间数据库、多媒体数据库以及演绎数据库；第9章介绍分布式数据库以及三层客户-服务器体系结构；第10章是新增的一章，介绍使用PHP的Web数据库编程；第11章介绍可扩展标记语言（eXtensible Markup Language, XML），其中说明了XML的概念，并将XML模型与传统数据库模型进行了比较；第12章介绍数据挖掘，概述了数据挖掘和知识发现的过程，讨论了关联规则挖掘、分类及聚类算法，并简要介绍了其他方法及商业工具；第13章介绍数据仓库的概念；最后，第14章介绍移动数据库、多媒体数据库、地理信息系统（Geographic Information Systems, GIS）以及生物信息学中的基因（Genome）数据管理。

附录A给出了展示基本的ER或EER模式时还可选用的其他一些图示表示法。如果教师认为必要，可以用这些表示法替代书中所用的表示法。附录B给出了关于磁盘的一些重要的物理参数。附录C概述了QBE图形查询语言。附录D和附录E[①]（在本书的辅助网站http://www.aw.com/elmasris 中也可找到）讨论了基于分层和网状数据模型的遗留数据库系统。这些已经使用了30多年的数据库系统是现今许多商业数据库应用和事务处理系统的基础，要完全取代它们还需要数十年。我们认为，对于学习数据库管理的学生来说，了解这些存在已久的方法也是很重要的。

补充资料

本书的所有使用者均可获得辅助资料。

- 在网站http://www.aw.com/cssupport上提供了PowerPoint幻灯片和图。
- 得到确认的教师可以获得习题答案。请访问Addison-Wesley 的教师资源中心（http://www.aw.com/irc），联系当地的Addison-Wesley销售代表或者发送电子邮件到computing@aw.com以获取如何获得习题答案的信息。

致谢

非常荣幸能够向这么多人致谢，感谢他们对本书的帮助与贡献。首先要感谢本书的编辑Matt Goldstein及Katherine Harutunian。特别要感谢第5版的首席编辑Matt Goldstein所付出的努力及给予的帮助。我们还要感谢Addison-Wesley的那些为第5版做出贡献的人们：Michelle Brown、Gillian Hall、Patty Mahtani、Maite Suarez-Rivas、Bethany Tidd和Joyce Cosentino Wells。我们感谢Gillian Hall完成了本书的内部设计，包括版式、字体和艺术设计方面非常细致的工作，正是她周全的考虑，我们才有了如此美观的书。

我们感谢为本书作出贡献的审稿人：

Hani Abu-Salem，迪保罗大学

Jamal R. Alsabbagh，大河谷州立大学

Ramzi Bualuan，圣母大学

Soon Chung，赖特州立大学

Sumali Conlon，密西西比大学

① 附录D和附录E请在图灵网站（www.turingbook.com）本书主页下载。——编者注

Hasan Davulcu，亚利桑那州立大学
James Geller，新泽西理工学院
Le Gruenwald，俄克拉何马大学
Latifur Khan，得克萨斯大学达拉斯分校
Herman Lam，佛罗里达大学
Byung S.Lee，佛蒙特大学
Donald Sanderson，东田纳西州立大学
Jamil Saquer，西南密苏里州立大学
Costas Tsatsoulis，堪萨斯大学
Jack C. Wileden，马萨诸塞大学阿默斯特分校

感谢Raj Sunderraman与我们一起完成了本书的实验部分并设计了实验题。旧金山大学的Salman Azar也贡献了部分习题。

Sham Navathe要向他佐治亚理工学院的学生们致谢：Saurav Sahay、Liora Sahar、Fariborz Farahmand、Nalini Polavarapu、Wanxia Xie、Ying Liu以及Gaurav Bhatia。Ed Omiecinski也曾提供了有价值的建议和修正。

我们还要再次感谢曾经审阅本书以及为本书前几版做出过贡献的人们。

- 第1版。Alan Apt（编辑）、Don Batory、Scott Downing、Dennis Heimbinger、Julia Hodges、Yannis Ioannidis、Jim Larson、Dennis McLeod、Per-Ake Larson、Rahul Patel、Nicholas Roussopoulos、David Stemple、Michael Stonebraker、Frank Tompa和Kyu-Young Whang。
- 第2版。Dan Joraanstad（编辑）、Rafi Ahmed、Antonio Albano、David Beech、Jose Blakeley、Panos Chrysanthis、Suzanne Dietrich、Vic Ghorpadey、Goets Graefe、Eric Hanson、Junguk L.Kim、Roger King、Vram Kouramajian、Vijay Kumar、John Lowther、Sanjay Manchanda、Toshimi Minoura、Inderpal Mumick、Ed Omiecinski、Girish Pathak、Raghu Ramakrishnan、Ed Robertson、Eugene Sheng、David Stotts、Marianne Winslett以及Stan Zdonick。
- 第3版。Maite Suarez-Rivas和Katherine Harutunian（编辑）、Suzanne Dietrich、Ed Omiecinski、Rafi Ahmed、Francois Bancilhon、Jose Blakeley、Rick Cattell、Ann Chervenak、David W.Embley、Henry A. Etlinger、Leonidas Fegaras、Dan Forsyth、Farshad Fotouhi、Michael Franklin、Sreejith Gopinath、Goetz Craefe、Richard Hull、Sushil Jajodia、Ramesh K. Karne、Harish Kotbagi、Vijay Kumar、Tarcisio Lima、Ramon A. Mata-Toledo、Jack McCaw、Dennis McLeod、Rokia Missaoui、Magdi Morsi、M.Narayanaswamy、Carlos Ordonez、Joan Peckham、Betty Salzberg、Ming-Chien Shan、Junping Sun、Rajshekhar Sunderraman、Aravindan Veerasamy以及Emilia E.Villareal。
- 第4版。Maite Suarez-Rivas、Katherine Harutunian、Daniel Rausch以及Juliet Silveri（编辑）；Phil Bernhard、Zhengxin Chen、Jan Chomicki、Hakan Ferhatosmanoglu、Len Fisk、William Hankley、Ali R.Hurson、Vijay Kumar、Peretz Shoval、Jason T.L. Wang（审稿人）；Ed Omiecinski（协助完成了第27章）；得克萨斯大学阿灵顿分校的贡献者有Hyoil Han、Babak Hojabri、Jack Fu、Charley Li、Ande Swathi以及Steven Wu；佐治亚理工学院的贡献者有Dan Forsythe、Weimin Feng、Angshuman Guin、Abrar Ul-Haque、Bin Liu、Ying Liu、Wanxia Xie以及Waigen Yee。

最后，我们真诚地感谢我们的家人所给予的支持、鼓励和耐心。

R.E.
S.B.N.

目　录

第一部分　事务处理的概念

第 1 章　事务处理的概念与理论简介 …… 2

1.1　事务处理简介 …… 2
1.1.1　单用户系统与多用户系统 …… 2
1.1.2　事务、读写操作和DBMS缓冲区 … 3
1.1.3　并发控制的必要性 …… 4
1.1.4　恢复的必要性 …… 6
1.2　事务和系统概念 …… 6
1.2.1　事务状态及附加操作 …… 7
1.2.2　系统日志 …… 7
1.2.3　事务的提交点 …… 8
1.3　事务的描述特性 …… 8
1.4　描述基于可恢复性调度的特征 …… 9
1.4.1　事务的调度（历史） …… 9
1.4.2　描述基于可恢复性调度的特征 … 10
1.5　描述基于可串行性调度的特征 …… 11
1.5.1　串行、非串行和冲突-可串行化调度 …… 11
1.5.2　测试调度的冲突可串行性 …… 14
1.5.3　可串行性的用途 …… 16
1.5.4　视图等价和视图可串行性 …… 17
1.5.5　调度等价性的其他类型 …… 18
1.6　SQL中的事务支持 …… 18
小结 …… 19
复习题 …… 20
练习题 …… 20
参考文献 …… 21

第 2 章　并发控制技术 …… 22

2.1　并发控制的两阶段加锁技术 …… 22
2.1.1　锁类型与系统锁表 …… 22
2.1.2　通过两阶段加锁保证可串行性 … 26
2.1.3　死锁和饥饿处理 …… 27
2.2　基于时间戳顺序的并发控制 …… 29
2.2.1　时间戳 …… 29
2.2.2　时间戳排序算法 …… 30
2.3　多版本并发控制技术 …… 31
2.3.1　基于时间戳排序的多版本技术 … 31
2.3.2　使用验证锁的多版本两阶段加锁 …… 32
2.4　确认（乐观的）并发控制技术 …… 32
2.5　数据项粒度和多粒度加锁 …… 33
2.5.1　加锁的粒度级别考虑 …… 34
2.5.2　多粒度级别加锁 …… 34
2.6　索引中使用加锁进行并发控制 …… 36
2.7　并发控制的其他问题 …… 37
2.7.1　插入、删除以及幻像记录 …… 37
2.7.2　交互式事务 …… 38
2.7.3　锁存器 …… 38
小结 …… 38
复习题 …… 38
练习题 …… 39
参考文献 …… 39

第 3 章　数据库恢复技术 …… 41

3.1　恢复概念 …… 41
3.1.1　恢复概述和恢复算法的分类 …… 41
3.1.2　磁盘块的高速缓存（缓冲） …… 42
3.1.3　先写日志、潜入/非潜入和强制/非强制 …… 42
3.1.4　系统日志中的检查点和模糊检查点 …… 43
3.1.5　事务回滚 …… 44
3.2　基于延迟更新的恢复技术 …… 45
3.2.1　单用户环境下使用延迟更新的恢复 …… 46
3.2.2　多用户环境下并发执行的延迟更新 …… 46
3.2.3　不影响数据库的事务活动 …… 48
3.3　基于即时更新的恢复技术 …… 48
3.3.1　单用户环境下基于即时更新的UNDO/REDO恢复 …… 49
3.3.2　多用户环境下基于即时更新的UNDO/REDO恢复 …… 49
3.4　镜像分页 …… 49
3.5　ARIES恢复算法 …… 50
3.6　多数据库系统中的恢复 …… 53

3.7 数据库备份和灾难性故障恢复……53
小结 ……54
复习题……54
练习题……55
参考文献……57

第二部分 对象和对象-关系数据库

第 4 章 面向对象数据库的概念……60

4.1 面向对象的概念……61
4.2 对象标识、对象结构和类型构造器……63
4.2.1 对象标识……63
4.2.2 对象结构……63
4.2.3 类型构造器……66
4.3 操作、方法和持久性的封装……67
4.3.1 通过类操作指定对象行为……67
4.3.2 通过命名和可达性指定对象的持久性……69
4.4 类型、类层次和继承……70
4.4.1 类型层次和继承……70
4.4.2 对应于类型层次的外延约束……72
4.5 复杂对象……72
4.5.1 非结构化复杂对象和类型可扩展性……73
4.5.2 结构化复杂对象……73
4.6 其他面向对象的概念……74
4.6.1 多态性（操作符重载）……74
4.6.2 多重继承与选择性继承……75
4.6.3 版本和配置……75
小结 ……76
复习题……76
练习题……77
参考文献……77

第 5 章 对象数据库标准、语言和设计……78

5.1 ODMG对象模型概述……79
5.1.1 对象和文字……79
5.1.2 汇集对象的内置接口……83
5.1.3 原子（用户定义）对象……85
5.1.4 接口、类和继承……86
5.1.5 类外延、码和工厂对象……86
5.2 对象定义语言（ODL）……88
5.3 对象查询语言（OQL）……92
5.3.1 简单OQL查询、数据库入口点和迭代变量……92
5.3.2 查询结果和路径表达式……93
5.3.3 OQL的其他特性……95
5.4 C++语言绑定概述……98
5.5 对象数据库概念设计……99
5.5.1 ODB与RDB概念设计的区别……99
5.5.2 EER模式到ODB模式的映射……100
小结 ……101
复习题……102
练习题……102
参考文献……102

第 6 章 对象-关系和扩展-关系系统……104

6.1 SQL及其对象-关系特性概述……104
6.1.1 SQL标准及其组件……104
6.1.2 SQL-99中的对象-关系支持……105
6.1.3 SQL中一些新操作和特性……109
6.2 数据模型的演变和数据库技术的当前发展趋势……109
6.3 Informix Universal Server……110
6.3.1 可扩展数据类型……111
6.3.2 支持用户定义例程……112
6.3.3 支持继承……113
6.3.4 支持索引扩展……115
6.3.5 支持外部数据源……115
6.3.6 支持数据刀片应用编程接口……115
6.4 Oracle 8的对象-关系特性……118
6.4.1 Oracle对象-关系特性的一些示例……118
6.4.2 管理大对象以及其他存储特性……119
6.5 扩展类型系统的实现和相关问题……120
6.6 嵌套关系数据模型……120
小结 ……122
参考文献……122

第三部分 高级主题：安全性、高级建模和分布式

第 7 章 数据库的安全性……126

7.1 数据库安全性问题简介……126
7.1.1 安全性的类型……126
7.1.2 控制措施……127
7.1.3 数据库安全性和DBA……128
7.1.4 访问保护、用户账户和数据库审计……128
7.2 基于授予和收回权限的自主访问控制……129
7.2.1 自主性权限的类型……129
7.2.2 使用视图指定权限……130

7.2.3 收回权限 …… 130
7.2.4 使用GRANT OPTION选项传播权限 …… 130
7.2.5 一个示例 …… 130
7.2.6 指定权限传播的限定 …… 131
7.3 多级安全性的强制访问控制和基于角色访问控制 …… 132
7.3.1 自主访问控制与强制访问控制的比较 …… 134
7.3.2 基于角色的访问控制 …… 134
7.3.3 XML访问控制 …… 135
7.3.4 电子商务和Web的访问控制策略 …… 135
7.4 统计数据库的安全性简介 …… 136
7.5 流控制简介 …… 137
7.6 加密和公钥构架 …… 138
7.6.1 数据加密标准和高级加密标准 …… 138
7.6.2 公钥加密 …… 138
7.6.3 数字签名 …… 139
7.7 保密性问题和保护 …… 140
7.8 数据库安全性所面临的挑战 …… 140
7.8.1 数据质量 …… 140
7.8.2 知识产权 …… 140
7.8.3 数据库生存能力 …… 140
小结 …… 141
复习题 …… 141
练习题 …… 142
参考文献 …… 142

第 8 章 面向高级应用的增强数据模型 …… 144

8.1 主动数据库概念和触发器 …… 145
8.1.1 主动数据库和Oracle触发器的一般模型 …… 145
8.1.2 主动数据库的设计和实现 …… 148
8.1.3 STARBURST系统中使用语句级主动规则的示例 …… 150
8.1.4 主动数据库的潜在应用 …… 151
8.1.5 SQL-99中的触发器 …… 152
8.2 时态数据库概念 …… 152
8.2.1 时间表示法、日历和时间维 …… 153
8.2.2 使用元组版本化在关系数据库中加入时间 …… 154
8.2.3 使用属性版本化在面向对象数据库中加入时间 …… 159
8.2.4 时态查询构造与TSQL2语言 …… 160
8.2.5 时间序列数据 …… 161
8.3 空间和多媒体数据库 …… 162
8.3.1 空间数据库概念介绍 …… 162
8.3.2 多媒体数据库概念介绍 …… 163
8.4 演绎数据库介绍 …… 164
8.4.1 演绎数据库概述 …… 164
8.4.2 Prolog/Datalog表示法 …… 165
8.4.3 Datalog表示法 …… 166
8.4.4 子句形式与Horn子句 …… 167
8.4.5 规则解释 …… 168
8.4.6 Datalog程序及其安全性 …… 169
8.4.7 关系操作的使用 …… 171
8.4.8 非递归Datalog查询的估算 …… 172
小结 …… 173
复习题 …… 174
练习题 …… 174
参考文献 …… 177

第 9 章 分布式数据库和客户-服务器体系结构 …… 178

9.1 分布式数据库概念 …… 178
9.1.1 并行与分布式技术 …… 179
9.1.2 分布式数据库的优势 …… 180
9.1.3 分布式数据库的附加功能 …… 181
9.2 分布式数据库设计的数据分片、复制和分配技术 …… 182
9.2.1 数据分片 …… 182
9.2.2 数据复制和分配 …… 184
9.2.3 分片、分配和复制示例 …… 185
9.3 分布式数据库系统的类型 …… 187
9.4 分布式数据库中的查询处理 …… 189
9.4.1 分布式查询处理的数据传输代价 …… 189
9.4.2 使用半连接的分布式查询处理 …… 191
9.4.3 查询和更新分解 …… 191
9.5 分布式数据库中并发控制和恢复概述 …… 193
9.5.1 基于识别数据项副本的分布式并发控制 …… 194
9.5.2 基于投票方法的分布式并发控制 …… 195
9.5.3 分布式恢复 …… 195
9.6 三层客户-服务器体系结构概述 …… 195
9.7 Oracle分布式数据库 …… 197
小结 …… 199
复习题 …… 199
练习题 …… 200
参考文献 …… 201

第四部分 新 兴 技 术

第 10 章 使用 PHP 进行 Web 数据库程序设计……204
10.1 结构化、半结构化以及非结构化数据……204
10.2 一个简单的PHP示例……207
10.3 PHP基本特性概述……209
10.3.1 PHP变量、数据类型和程序设计结构……209
10.3.2 PHP数组……210
10.3.3 PHP函数……211
10.3.4 PHP服务器变量和表单……213
10.4 PHP数据库程序设计概述……214
10.4.1 连接数据库……214
10.4.2 从表单中汇集数据并插入记录……215
10.4.3 数据库表的检索查询……216
小结……217
复习题……217
练习题……217
实验练习题……217
参考文献……218

第 11 章 XML：可扩展标记语言……219
11.1 XML层次（树）数据模型……219
11.2 XML文档、DTD和XML模式……221
11.2.1 合式、有效XML文档及XML DTD……221
11.2.2 XML模式……222
11.3 XML文档和数据库……226
11.3.1 存储XML文档的方法……226
11.3.2 从关系数据库中抽取XML文档……227
11.3.3 断开环以实现从图到树的转换……230
11.3.4 从数据库抽取XML文档的其他步骤……230
11.4 XML查询……231
11.4.1 XPath：在XML中指路径表达式……231
11.4.2 XQuery：在XML中指定查询……232
小结……232
复习题……233
练习题……233
参考文献……233

第 12 章 数据挖掘概念……234
12.1 数据挖掘技术概述……234
12.1.1 数据挖掘和数据仓库……234
12.1.2 数据挖掘是知识发现过程的一部分……234
12.1.3 数据挖掘和知识发现的目标……235
12.1.4 数据挖掘期间知识发现的类型……236
12.2 关联规则……237
12.2.1 市场购物篮模型、支持度与可信度……237
12.2.2 Apriori算法……238
12.2.3 采样算法……239
12.2.4 频繁模式树算法……239
12.2.5 分区算法……241
12.2.6 其他类型的关联规则……242
12.2.7 关联规则的附加考虑……244
12.3 分类……244
12.4 聚类……246
12.5 其他数据挖掘问题的方法……248
12.5.1 序列模式的发现……248
12.5.2 时间序列中模式的发现……248
12.5.3 回归……249
12.5.4 神经网络……249
12.5.5 遗传算法……249
12.6 数据挖掘的应用……250
12.7 商业化数据挖掘工具……250
12.7.1 用户界面……251
12.7.2 应用编程接口……251
12.7.3 未来的方向……251
小结……252
复习题……252
练习题……252
参考文献……254

第 13 章 数据仓库和 OLAP 概述……255
13.1 概述、定义和术语……255
13.2 数据仓库的特性……256
13.3 数据仓库的数据模型……257
13.4 构建数据仓库……261
13.5 数据仓库的典型功能……263
13.6 数据仓库和视图……263
13.7 数据仓库中的难题和开放问题……264
13.7.1 数据仓库实现中的困难……264
13.7.2 数据仓库的开放问题……264
小结……265
复习题……265

参考文献 …… 265
第 14 章　新兴的数据库技术及应用 …… 266
14.1　移动数据库 …… 266
14.1.1　移动计算体系结构 …… 266
14.1.2　移动环境的特征 …… 269
14.1.3　数据管理问题 …… 270
14.1.4　应用：间歇同步数据库 …… 270
14.2　多媒体数据库 …… 271
14.2.1　多媒体数据和应用的特征 …… 271
14.2.2　数据管理问题 …… 272
14.2.3　开放研究问题 …… 273
14.2.4　多媒体数据库应用 …… 274
14.3　地理信息系统（GIS） …… 275
14.3.1　GIS系统组件 …… 276
14.3.2　GIS中数据的特征 …… 276
14.3.3　GIS概念数据模型 …… 278
14.3.4　GIS的DBMS增强功能 …… 278
14.3.5　GIS标准和操作 …… 279
14.3.6　GIS应用和软件 …… 281
14.3.7　GIS未来的工作 …… 282
14.4　基因组数据管理 …… 283
14.4.1　生物科学和遗传学 …… 283
14.4.2　生物数据的特征 …… 284
14.4.3　人类基因组项目和现有的生物数据库 …… 286
参考文献 …… 289
致谢 …… 291
附录 A　ER 模型的可选图形表示法 …… 292
附录 B　磁盘参数 …… 294
附录 C　QBE 语言概述 …… 296

第一部分

Part 1

事务处理的概念

本部分内容

- 第 1 章　事务处理的概念与理论简介
- 第 2 章　并发控制技术
- 第 3 章　数据库恢复技术

第1章

事务处理的概念与理论简介

事务的概念为描述数据库处理的逻辑单元提供了一种机制。**事务处理系统**（transaction processing system）是拥有大型数据库以及允许数百个用户同时执行数据库事务的系统。这些系统包括：机票预订、银行业务、信用卡处理、股票市场、超市收款等。事务处理系统需要为数以百计的并发用户提供高度的可用性和快速的响应时间。本章将给出事务处理系统中所需要的概念，定义事务的概念，即它是用来表示数据库处理的一个逻辑单元，数据库处理必须完整地被执行以保证正确性。然后讨论并发控制问题，这是为了避免各个用户提交的多个事务，以某种方式相互影响而产生错误的结果。另外，还将讨论事务故障的恢复问题。

1.1节简单地讨论了数据库系统中并发控制和恢复的必要性。1.2节介绍事务的概念，并讨论数据库系统中事务处理所涉及的其他概念。1.3节讨论事务的ACID特性，即原子性、一致性、隔离性和持续性或永久性。1.4节介绍执行事务调度（或历史）的概念和调度的可恢复性特征。1.5节讨论并发事务执行的可串行化概念，借助可串行化概念我们可以定义并发事务的正确执行序列（或调度）。1.6节介绍SQL中支持事务概念的工具。

接下来的两章将更为详细地探讨支持事务处理的技术。第2章描述基本的并发控制技术，第3章概要地介绍恢复技术。

1.1 事务处理简介

本节将简要介绍事务的并发执行和从故障事务中恢复的概念。1.1.1节对单用户数据库系统和多用户数据库系统进行比较，并说明在多用户系统中事务如何并发执行。1.1.2节定义事务的概念并描述一个关于事务执行的简单模型，它是基于数据库读写操作的，用于形式化描述并发控制和恢复的概念。1.1.3节借助常见的示例说明多用户系统中并发控制的必要性。最后，1.1.4节讨论事务在执行过程中导致故障的不同方式，以及允许从事务故障中恢复技术的必要性。

1.1.1 单用户系统与多用户系统

对数据库系统进行分类的一个标准是根据能够**并发地**（concurrently）使用系统的用户数量进行划分。如果在同一时刻只有一个用户能够使用DBMS，那么这个系统就是**单用户**（single-user）系统；如果多个用户可以同时使用系统，并对数据库进行访问，那么这个系统就是**多用户**（multiuser）系统。单用户DBMS多数仅限于用在一些个人计算机系统中；而其他大多数DBMS都是多用户的。如同时会有数以百计的旅行代理商和机票预订员同时使用某个机票预订系统。在银行、保险代理、股票交易、超市中使用的系统以及与之类似的其他系统中，通常都会有很多用户并发地向系统提交事务。

多道程序设计（multiprogramming）技术使得多个用户可以使用计算机系统同时对数据库进行访问，它允许计算机在同一时间里可执行多个程序或**进程**（process）。如果在系统中只有一个中央处理单元（CPU），那么实际上在同一时间它至多只能执行一个进程。因此，**多道程序设计操作系统**（multiprogramming operating systems）在执行完一个进程的某些命令后就将此进程挂起，然后再执行

下一个进程的一些命令，依次类推。一旦轮到一个进程再次使用CPU时，它便从挂起点开始恢复执行。因此，进程的并发执行实际上是**交替的**（interleaved），如同在图1-1中所说明的那样，其中展示的两个进程A和B就是以交替方式并发执行的。当一个进程要进行输入或输出（I/O）操作，如从磁盘中读取一个块时，进程交替可以使CPU始终处于繁忙状态。在等待I/O时间内，CPU将切换到执行另一个进程，而不是处于空闲状态。另外，交替执行也可防止执行时间过长的进程延迟其他进程。

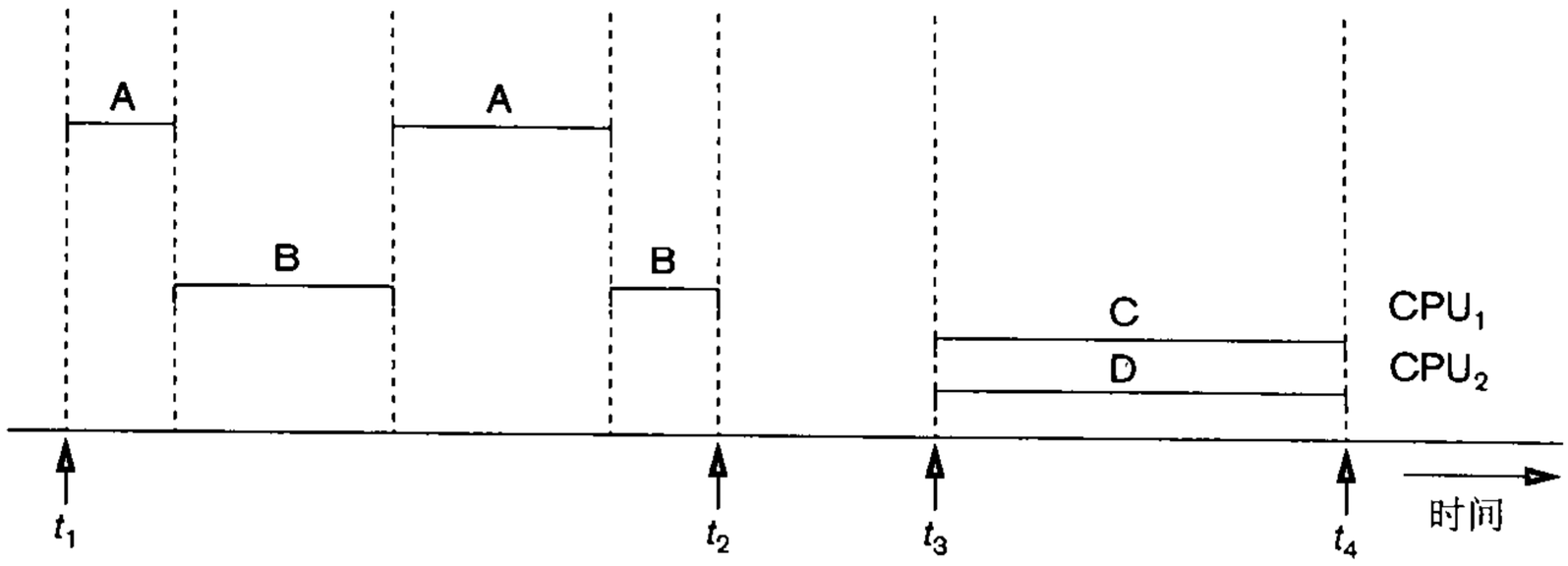


图1-1　并发事务的交替处理与并行处理

如果计算机系统拥有多个硬件处理器（CPU），则可以实现多进程的**并行处理**（parallel processing），如图1-1所示的两个进程C和D。在数据库中大多数与并发控制相关的理论是根据**交替并发**（interleaved concurrency）理论开发的，所以我们在本章后续内容中将假定为这种模型。在多用户DBMS中，所存储的数据项是可以被交互式用户或应用程序并发访问的主要资源，这些用户及应用程序会频繁地检索信息和修改数据库。

1.1.2　事务、读写操作和 DBMS 缓冲区

事务（transaction）是一个构成数据库处理逻辑单元的可执行程序。一个事务包括一个或多个数据库访问操作，这些操作包括插入、删除、修改或检索操作。事务所涉及的数据库操作既可以嵌入到应用程序中，也可以通过诸如SQL的高级查询语言交互地指定。指定事务边界的一种方法是在应用程序中使用**开始事务**（begin transaction）和**结束事务**（end transaction）语句显式地指定。在这种情况下，开始事务和结束事务语句间的所有数据库访问操作就可看作是构成了一个事务。如果在一个应用程序中出现多个事务边界，那么该应用就包含多个事务。如果某个事务涉及的数据库操作仅仅对数据进行检索，而不更新数据库，那么这个事务称为**只读事务**（read-only transaction）。

用来解释事务处理概念的数据库模型已经被大大简化。**数据库**基本上可以看作是**命名数据项**（named data items）的一个集合。数据项的大小称为它的**粒度**（granularity）。数据项可以是数据库中某个记录的一个字段，也可以是更大的单元（如一条记录或甚至整个磁盘块），但是这里所讨论的概念与数据项的粒度无关。使用这个简化的数据库模型，事务可能包含的基本数据库访问操作如下：

- read_item(*X*)：将名为*X*的数据项读入程序变量中。为简化表示，假定程序变量也被命名为*X*。
- write_item(*X*)：将程序变量*X*的值写入名为*X*的数据项中。

从磁盘向主存传送的基本数据单元是一个块（block）。执行read_item(*X*)命令包括如下步骤：

(1) 找到包含数据项*X*的磁盘块地址。
(2) 将该磁盘块复制到主存中的一个缓冲区（如果该磁盘块不在主存的某个缓冲区内）。
(3) 将数据项*X*从缓冲区复制到名为*X*的程序变量中。

执行write_item(*X*)命令包括如下步骤：

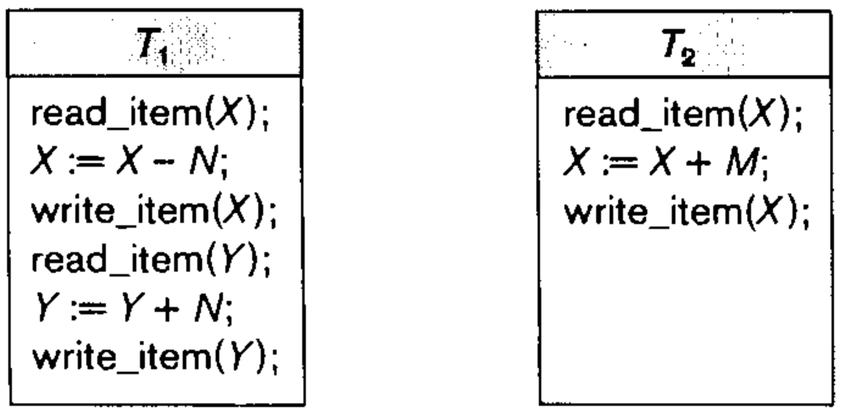
(1) 找到包含数据项*X*的磁盘块地址。
(2) 将该磁盘块复制到主存中的一个缓冲区（如果该磁盘块不在主存的某个缓冲区内）。

(3) 从名为X的程序变量复制项X到它在缓冲区中的正确位置。

(4) 将更新后的块从缓冲区存储回磁盘（立即执行或稍后某个时刻执行）。

第4步才会真正更新磁盘上的数据库。在某些情况下，该缓冲区并不立即写回磁盘，因为还可能对该缓冲区进行另外的修改。通常由DBMS的恢复管理器和底层操作系统协同控制，以决定何时将主存缓冲区中的数据更新写回磁盘。DBMS通常会在主存中维护大量**缓冲区**（buffer），以保存包含正被处理的数据库项的数据库磁盘块。当这些缓冲区都被占用，而且还要把另外的数据库磁盘块复制到主存时，就必须借助某种缓冲区替换策略来选择当前哪个缓冲区要被替换掉。如果所选中的缓冲区已被修改，那么在它被复用前，必须先将其写回磁盘[①]。

包括read_item和write_item操作的事务用于访问和更新数据库。图1-2展示了两个简单事务的示例。事务的**读集**（read-set）是事务读取的所有数据项的集合，事务的**写集**（write-set）是事务写入的所有数据项的集合。如图1-2中事务T_1的读集是{X，Y}，写集也是{X，Y}。



(a) 事务T_1　　(b) 事务T_2

图1-2　两个事务示例

并发控制和恢复机制主要与事务中的数据库访问命令相关。由不同用户提交的事务可以并发地执行，并对相同的数据项进行访问和更新。如果不对并发执行进行适当的控制，将会引发诸多问题，比如数据库不一致。下一节我们将简要介绍可能产生的一些问题。

1.1.3　并发控制的必要性

当并发事务以非受控的方式执行时可能会产生一些问题。下面以一个相当简化的机票预订数据库为例说明其中的一些问题。在数据库中每个航班拥有一个记录，每个记录包括该航班中座位的预定数量，它作为命名数据项与其他信息一起存储在记录中。图1-2a所示的事务T_1将N个预订座位从一个航班（其预订座位的数量存储在数据项X中）转移到另一个航班（其预订座位的数量存储在数据库项Y中）。图1-2b中表示的是一个更为简单的事务T_2，它预订的是事务T_1中第一个航班（X）中的M个座位[②]。为了简化这个示例，这里没有给出事务的其他部分，如在预订其他座位前检查该航班是否有足够的空闲座位。

当编写数据库访问程序时，把航班号、航班日期以及要预订的座位个数作为该程序的参数；因此可能会出现使用同样的程序执行多个事务的情况，而每一个事务都有不同的航班和座位个数被预订。为了实现并发控制，可以把事务看作是关于指定日期、指定航班和指定座位个数的程序的一个特定执行。在图1-2a和图1-2b中，事务T_1和T_2就是关于特定航班程序的特定执行，其中特定航班的座位数量被存储在数据库的数据项X和Y中。下面讨论这两个事务并发执行时可能遇到的几类问题。

1. 更新丢失问题

当访问相同数据库项的两个事务的操作，以某种方式交替执行，从而导致某些数据库项的值不正确时，就会发生更新丢失问题。假设事务T_1和T_2几乎同时提交，并且其操作如图1-3a所示交替执行；

① 我们在这里不讨论缓冲区替换策略，这部分内容一般是在操作系统类的教材中讨论的。

② 一个类似的、更常见的例子是假定一个银行数据库，它具有两个事务，一个是将账户X的存款转移到账户Y上，另一个是在账户X上存款。

那么数据项X的最终值是错误的，因为T_2在T_1改变数据库中的X值之前便读取了它的值，因此T_1产生的更新值被丢失。例如，如果开始时X=80（起初在航班中有80个预订座位），N=5（T_1从与X对应的航班中转移了5个座位到与Y对应的航班中），M=4（T_2在X上预订了4个座位），那么最终的结果应该是X=79；但在图1-3a所示的交替执行中，最终结果是X=84，这是由于T_1从X中移走5个座位的更新操作丢失了。

2. 暂时更新（或脏读）问题

事务更新了某个数据库项，接着由于某种原因（见1.1.4节）事务故障了，然而所更新的项在恢复到原值之前，另一个事务便读取该项。在图1-3b所示的例子中，T_1更新了数据项X，然而在此更新完成之前故障了，这样系统必须将X的值恢复为原值。但在系统进行恢复操作之前，事务T_2读取了X的暂时值，而由于T_1的故障使得该值不会被永久记录在数据库中。T_2读取的X值便被称为脏数据，因为它是由一个没有完成或没有提交的事务产生的；该问题也被称为脏读问题。

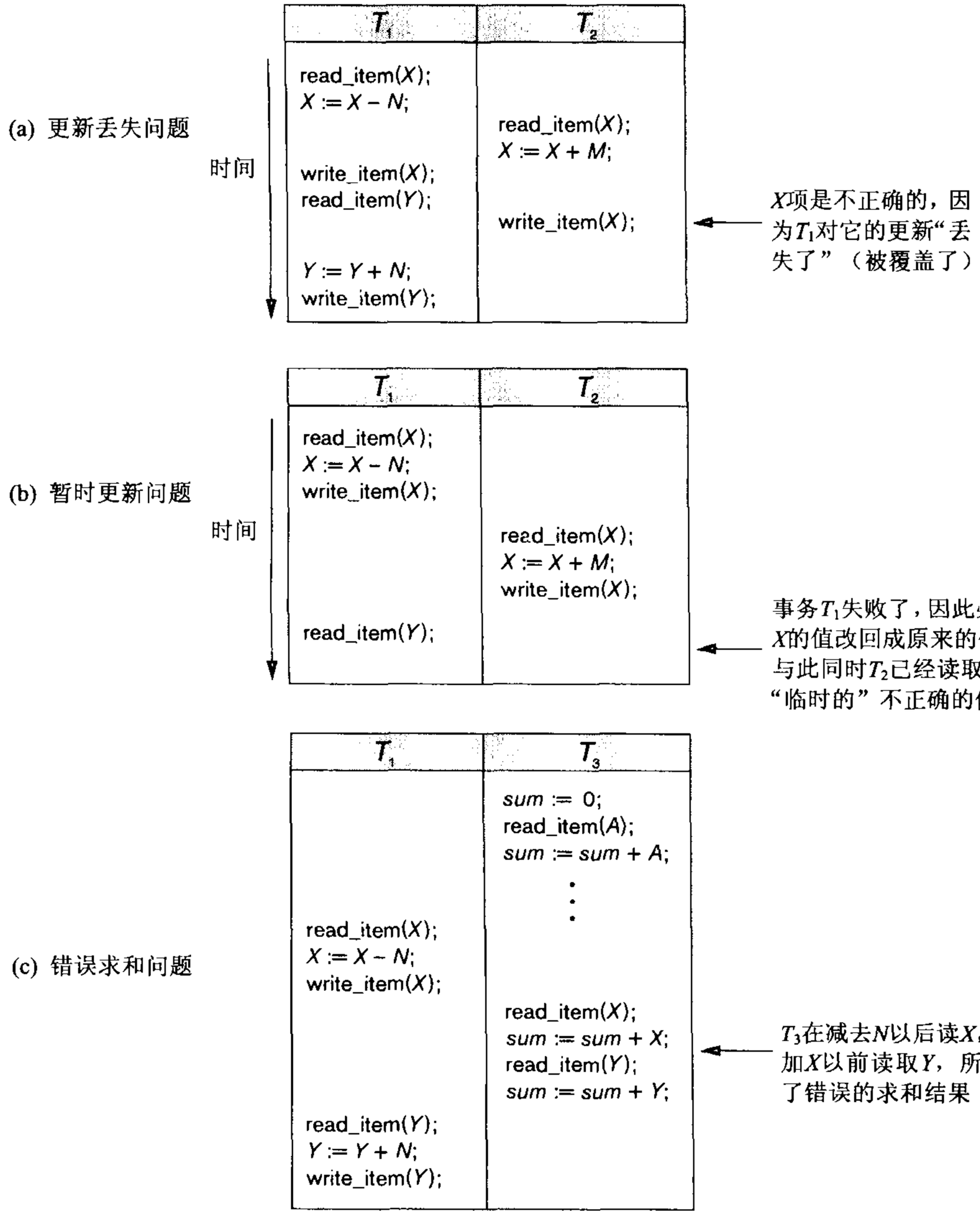


图1-3　当并发执行不受控制时会发生的一些问题

3. 错误求和问题

如果一个事务在若干记录上计算一个聚集求和函数，而另一些事务正在更新其中的某些记录，则聚集函数可能计算了一些更新之前的值，和一些更新之后的值。如假设事务T_3正在计算所有航班上的预订座位总数；与此同时事务T_1也在执行。如果按照图1-3c所示的交替操作执行，那么T_3的结果中就会少N个座位，因为T_3是在X减去N个座位之后读取X值的，并且是在N个座位被加到Y之前读取Y值的。

另一个可能发生的问题是**不可重复读**（unrepeatable read）。事务T需要两次读取同一个数据项，但是在两次读取操作的间隔中，另一个事务T'改变了该数据项的值。因此，T在两次读取同一个数据项时却读出了不同的值。如在机票预订事务执行期间，某个顾客正在查询几个航班上的可用座位。当该顾客确定了某个特定的班机后，该事务在完成预订前会第二次读取该班机的座位数。

1.1.4　恢复的必要性

无论事务何时被提交给DBMS执行，系统必须负责保证：成功完成事务中的所有操作，并且这些操作所产生的影响永久记录在数据库中；或者该事务没有对数据库或其他事务造成任何影响。DBMS必须防止事务T的某些操作已被应用到数据库，而事务T的另外一些操作却没有被应用到数据库。如果事务在执行了一些操作，但还没有完成所有操作之前发生**故障**（fail），那么就有可能出现上述情况。

故障的类型

通常将故障（failure）分为事务故障、系统故障和介质故障。事务在执行期间发生故障可能有如下的几个原因。

(1) **计算机故障（系统崩溃）**：在事务执行过程中，计算机系统可能会出现硬件、软件或网络错误。硬件崩溃通常都是介质故障（如主存故障）。

(2) **事务或系统错误**：事务中的一些操作可能导致事务故障，如整数溢出或除零。错误的参数值或者逻辑程序设计错误[①]也可能导致事务故障。另外，用户也可能在事务执行期间中断事务。

(3) **事务检测到的局部错误或异常条件**：在事务执行期间，可能会产生必须撤销事务的特定条件。如事务所需的数据没有找到。需要注意的是，某个异常条件[②]（如银行系统数据库中账户余额不足）可能会导致另一个事务的撤销（如资金提取）。事务本身应该对这种异常进行错误处理，因此这并不认为是事务故障。

(4) **并发控制的实施**：并发控制（见第2章）可能会因为某事务违反了可串行性（见1.5节）或者几个事务处于死锁状态而决定中止该事务，并在稍后重启该事务。

(5) **磁盘故障**：一些磁盘块可能因为读或写时发生故障，或者因为磁盘的读/写头损坏而丢失数据。那么在事务读或写操作期间就有可能发生磁盘故障。

(6) **物理问题和灾难性事故**：这包括众多的问题，如电源或空调失效、火灾、盗窃、蓄意破坏、由于疏忽而覆写了磁盘或磁带，以及操作员安装了一个错误的磁带卷等。

前4类故障比后2类更为常见。如果发生前4类故障中的任意一类，系统都必须保留足够的信息从故障中进行恢复。后2种故障并不经常发生，一旦遇到的话，那么恢复就将成为一个艰巨的任务。这将在第3章详细讨论从故障中恢复的问题。

事务的概念是许多并发控制技术和从故障中恢复技术的基础。

1.2　事务和系统概念

本节将讨论与事务处理相关的其他概念。1.2.1节介绍事务的不同状态及事务处理所需的其他相

① 通常，应该彻底测试事务以确保它没有程序错误（逻辑程序设计错误）。

② 如果编程正确，那么异常条件不会导致事务失败。

关操作。1.2.2节讨论系统日志，它保存了系统恢复所需要的信息。1.2.3节描述事务提交点的概念，以及它们在事务处理中的重要性。

1.2.1　事务状态及附加操作

事务是工作的一个原子单元，它作为一个整体要么全都执行，要么全都不执行。为保证可恢复性，系统需要保存事务的起始、终止、提交或撤销的记录（见1.2.3节）。因此，恢复管理器要对下列操作进行跟踪记录：

- BEGIN_TRANSACTION：标记事务开始执行。
- READ或WRITE：指明事务对某个数据项进行读或写操作。
- END_TRANSACTION：指明事务的READ或WRITE操作已经结束，并标记事务执行结束。但是在这一点上，有必要检查该事务引起的修改能否永久写入数据库（即提交），或者是否由于违反可串行性（参见1.5节）或某种其他原因而必须异常中止该事务。
- COMMIT_TRANSACTION：表示事务已经成功结束，因此事务执行的任何改变（更新）可以安全**提交**到数据库并且不会被撤销。
- ROLLBACK（或ABORT）：表示事务没有成功结束，因此必须回滚或撤销该事务对数据库所做的任何改变或影响。

图1-4所示的是一个状态转换图，它描述了事务在执行过程中的状态变化。事务在开始执行后立即进入**活动状态**（active state），此时，事务可以进行READ、WRITE操作。事务结束时进入**部分提交状态**（partially committed state）。此时，恢复协议需要确保系统故障后不再记录事务所产生的改变（这些改变通常存储在系统日志中，下一节将对此进行详细讨论）[①]。在此前提下，事务到达提交点从而进入**提交状态**（committed state）。关于提交点的内容将在1.2.3节详细讨论。一旦事务被提交，就必须在日志中记录事务已经成功完成的信息，并把它所做的所有改变永久地记录在数据库中。

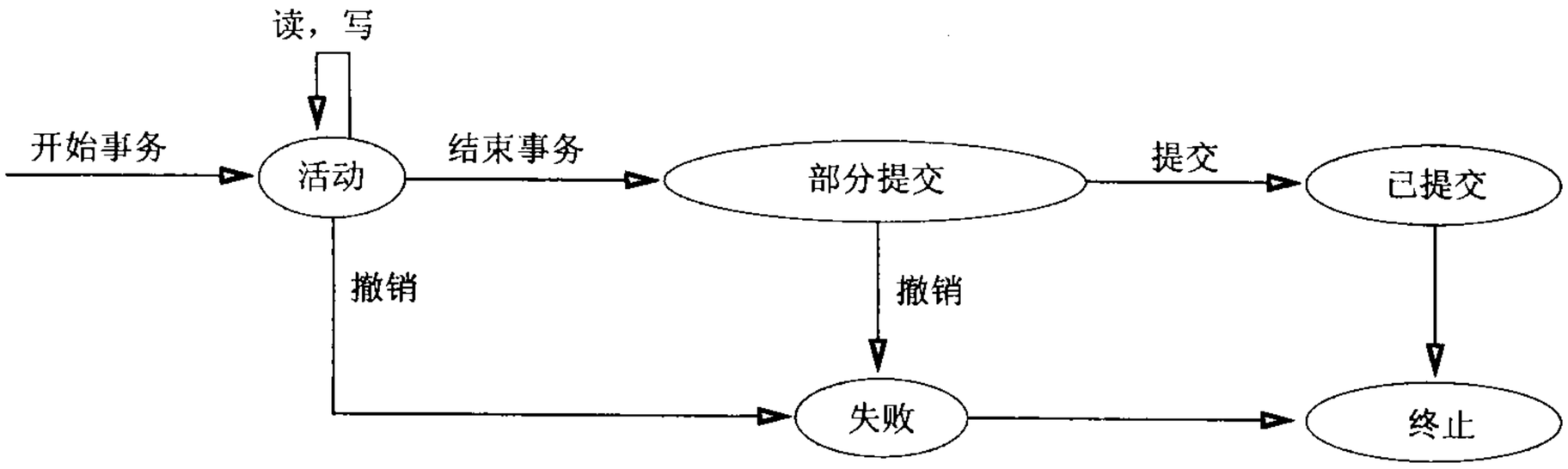


图1-4　状态转换图，说明事务执行的状态

然而，如果有一个校检失败或者事务在活动状态期间被撤销，那么事务就将进入**失败状态**（failed state）。这时，需要将事务回滚，以消除它的WRITE操作对数据库的影响。**终止状态**（terminated state）表示事务已经离开系统。在事务执行时，相关信息保存在系统表中；当事务终止时，事务信息同时被清除。失败或被撤销的事务可能会在稍后作为一个新的事务被重启——自动或由用户重新启动。

1.2.2　系统日志

为了能够从影响事务的故障中恢复，系统维护一个**日志**[②]（log）来记录所有影响数据库项的值的事务操作。当允许从故障中恢复时可能需要这些信息。日志保存在磁盘上，这样除了磁盘故障和灾难性故障外它不会受到其他任何类型故障的影响。另外日志会被定期备份到归档存储设备（磁带）

① 优化的并发控制（见2.4节）同时要求在这一点上进行检查以确保事务不会干扰其他正在执行的事务。

② 日志有时也被称作DBMS日记。

中，以预防磁盘故障和灾难性故障。下面列出的是被写到日志中的条目类型，称为**日志记录**（log record），以及每个类型涉及的相关动作。在条目中，*T*所表示的是唯一**事务标识**（transaction-id），用于标识每个事务，通常由系统自动生成：

(1) [start_transaction, ***T***]：表示事务*T*开始执行。

(2) [write_item, ***T***, ***X***, ***old_value***, ***new_value***]：表示事务*T*已将数据项*X*的值从*old_value*改为*new_value*。

(3) [read_item, ***T***, ***X***]：表示事务*T*已读取数据项*X*的值。

(4) [commit, ***T***]：表示事务*T*已成功完成，其结果已被提交（永久记录）给数据库。

(5) [abort, ***T***]：表示事务*T*已被撤销。

对于避免级联回滚（见1.4.2节）的恢复协议（包括几乎所有的实际协议），不要求将READ操作写入系统日志。但如果日志还用于其他目的，如审计（记录所有数据库操作），那么就需要在日志中包括READ操作的条目。另外，某些恢复协议只要求简单的WRITE条目，那样就不必包括new_value（见1.4.2节）。

需要注意的是：这里我们假定所有对数据库的永久改变都发生在事务内部，所以对事务故障进行恢复的概念实际上就是取消或者重新逐个地执行日志中的事务操作。如果系统崩溃，就可以通过检查日志，并使用第3章所描述的恢复技术，将数据库恢复到某个一致的状态。因为日志包含了每个改变数据项值的WRITE操作记录，因此可通过向后跟踪日志将事务*T*的WRITE操作所改变的所有项值恢复为old_values，从而可以**取消**（undo）事务*T*的WRITE操作对数据库产生的影响。如果事务的所有更新都已记录在日志中，但在我们确认所有这些new_values已被永久记录在实际数据库之前，事务发生故障，那么就有可能需要**重做**（redoing）某些操作[①]。通过向前追溯日志并将*T*的WRITE操作所改变的所有项，置为其new_values，即可重做事务*T*的操作。

1.2.3　事务的提交点

当事务*T*所有的数据库访问操作都已成功执行，并且所有操作对数据库的影响都已记录在日志中时，该事务*T*就到达**提交点**（committed point）。提交点后的事务就成为**已提交**的事务，并且假定其结果永久地记录在数据库中。然后事务在日志中写入一条提交记录[commit, *T*]。在系统发生故障时，需要扫描日志，检查那些已在日志中写入[start_transaction, *T*]，但没有写入[commit, *T*]的所有事务*T*；恢复时必须回滚这些事务以取消它们对数据库的影响。此外，日志中已经提交的事务还必须把其所有的WRITE操作也记录在日志中，这样它们对数据库的影响才可根据这些记录进行重做。

需要注意的是：必须将日志文件保存在磁盘上。我们都知道更新一个磁盘文件时必须先将文件的相应块，从磁盘拷贝到主存的缓冲区，然后在主存的缓冲区中进行更新，最后将缓冲区内容写回磁盘。所以通常的做法是在主存缓冲区中保留一个或多个日志文件块，直到它们都被写满，再将它们一次性写回到磁盘，而不是每增加一个日志条目就写一次磁盘。这将节省对同一日志文件块进行多次写磁盘的开销。当系统崩溃时主存中的内容可能丢失，所以恢复时只能考虑已写回磁盘的日志内容。因此，在事务到达提交点以前，还未写到磁盘的日志的任何部分，必须被写入磁盘。这称为事务提交前**强制写**（force writing）日志。

1.3　事务的描述特性

事务应当有一些特性，通常称为**ACID特性**（ACID properties），这些特性将由DBMS的并发控制和恢复方法来保证执行。ACID特性如下：

(1) **原子性**（atomicity）：事务是处理的一个原子单位；它或者全都执行或者都不执行。

① 取消（undo）和重做（redo）将在第3章中详细讨论。

(2) **一致性保持**（consistency preservation）：如果事务的执行将数据库从一个一致性状态转换到另一个一致性状态，那么该事务是一致性保持。

(3) **隔离性**（isolation）：事务的执行表现为与其他事务相隔离，即事务的执行不应受到其他并发事务执行的干扰。

(4) **持久性或永久性**（durability or permanency）：已提交事务对数据库的改变在数据库中应该是持续存在的。这些改变不会因为任何故障而发生丢失。

原子性要求事务必须被完整地执行。确保完整性是DBMS事务恢复子系统的责任。如果由于某个原因事务的执行没有完成（如在事务执行期间发生系统崩溃），那么恢复时必须取消事务对数据库的任何影响。

保持一致性通常被认为是程序员的责任，或者是执行完整性约束的DBMS模块的责任。我们知道，**数据库状态**（database state）是指在某个给定时间点上，数据库中存储的所有数据项（值）的集合。数据库的**一致性状态**（consistent state）应该满足模式所指定的约束，以及数据库必须满足的任何其他约束。在编写数据库应用程序时，应该保证如果执行事务前数据库处于一致性状态，那么在完整执行该事务后数据库仍然处于一致性状态，这里假定不会发生其他事务的干扰。

隔离性是由DBMS的并发控制子系统实现的[①]。如果每个事务的更新在它被提交之前，对其他事务都是不可见的，则实现了一种隔离性，由此可以解决临时更新和消除级联回滚问题（见第3章）。下面定义事务的**隔离级别**。如果一个事务不会重写更高级别事务的脏读，那么该事务具有0（零）级隔离性；1级隔离性事务不会发生更新丢失；2级隔离性指事务不会发生更新丢失和脏读；3级隔离性（也称为真隔离性）除具备2级隔离性性质外，还支持重复读。

最后，持久性是DBMS恢复子系统的任务。我们将在第3章中讨论恢复协议是如何保证事务的持久性和原子性的。

1.4　描述基于可恢复性调度的特征

当事务以交替方式并发地执行时，我们把不同事务的操作执行顺序称为**调度**（schedule或**历史**，history）。在本节中，首先定义调度的概念，然后描述调度类型的特征，当出现故障时调度有助于进行恢复。在1.5节中，将根据参与事务之间的干扰来刻画调度的特征，并由此引出可串行性和可串行化调度的概念。

1.4.1　事务的调度（历史）

n个事务T_1，T_2，…，T_n的**调度**（或**历史**）S是这n个事务的一个操作顺序，这些事务服从下述约束，即：S中每个事务T_i中的各个操作的顺序，必须与该T_i中这些操作出现的顺序相同。但需要注意的是，来自其他事务T_j的操作可以同S中T_i的操作交错执行。虽然理论上我们可以处理某些操作形成的偏序调度（将在后面讨论），但目前我们假定S中操作的的顺序是全序的。

对于恢复和并发控制来说，我们关心的主要是事务中的read_item和write_item操作，以及commit和abort操作。使用记号r、w、c和a分别表示read_item、write_item、commit和abort操作，并将事务标识（事务号）作为下标加到上述符号下方。在这种符号表示法中，读或写数据项X跟在r和w操作符后的括号内。如使用这种符号表示法表示图1-3a中的调度写作如下的S_a：

$$S_a : r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); w_1(Y);$$

同样，假定图1-3b中事务T_1在read_item(Y)后撤销，那么其对应的调度S_b如下：

$$S_b : r_1(X); w_1(X); r_2(X); w_2(X); r_1(Y); a_1;$$

① 我们将在第2章中讨论并发控制协议。

如果某个调度中的两个操作同时满足如下3个条件，我们就说这两个操作是**冲突的**（conflict）：(1)它们属于不同事务，(2)它们访问的是同一个项X，(3)两个操作中至少有一个是write_item(X)。如在调度S_a中，操作$r_1(X)$和$w_2(X)$是冲突的，操作$r_2(X)$和$w_1(X)$以及$w_1(X)$和$w_2(X)$也是冲突的。但是操作$r_1(X)$和$r_2(X)$不是冲突的，因为它们都是读操作；操作$w_2(X)$和$w_1(Y)$也不是冲突的，因为它们所操作的数据项X和Y不同；操作$r_1(X)$和$w_1(X)$也不是冲突的，因为它们属于相同的事务。

如果下述条件成立，我们说n个事务T_1，T_2，…，T_n的某个调度S是**完备调度**（complete schedule）：

(1) 调度S中的操作却好是T_1，T_2，…，T_n中的操作，S中还包括和每个事务对应的提交或撤销操作。

(2) 对于来自同一个事务T_i的任何操作对，它们在S中的出现顺序与它们在T_i中的出现顺序相同。

(3) 在该调度S中，对于任何两个冲突的操作，其中一个必须在另一个之前出现[①]。

上述第3个条件允许在调度中出现两个非冲突操作，而无需定义它们的执行顺序，这引出调度中关于n个事务的操作的**偏序**（partial order）定义[②]。但是，对于任意一个冲突操作对（条件3）或者来自相同事务的任意一个操作对（条件2），必须在调度中指定为一个全序。条件1简单陈述了在事务中出现的所有操作，必须也出现在完备调度中。因为每个事务或者已经提交，或者已经撤销，所以一个完备调度不会在其末端包含任何还在活动的事务。

一般来说，在事务处理系统中很难出现完备调度，因为新的事务总是连续不断地被提交到系统中来。因此，定义调度S的**已提交投影**（committed projection）$C(S)$的概念会很有用，它只包含S中属于已提交事务的操作，已提交事务T_i的提交操作c_i在S中。

1.4.2　描述基于可恢复性调度的特征

对于某些调度，从事务故障中恢复是很容易的。然而对于另外一些调度，恢复处理却可能十分复杂。因此，对于那些可进行恢复的和恢复起来相对简单的调度类型，描述其特征是很重要的。这些特征并不提供实际的恢复算法，而仅仅是试图在理论上描述不同调度类型的特征。

首先，我们要保证一旦事务T被提交，它就永不需要再回滚。从理论上满足这个条件的调度被称为可恢复的调度，而那些不满足这个条件的调度就称为**不可恢复的**（nonrecoverable）调度，因而也是不被允许的。如果在调度S中的事务T，直到所有写入T读取的数据项的事务T'已经提交成功后，事务T才被提交，则调度S是可恢复的。在调度S中，一个事务T从另一个事务T'中**读**（read），是指：如果某个数据项X首先由T'写入，而后由T读取。另外，T'不应在T读取X前被撤销，并且也不该再有事务在T'写入X后与在T读取X前，写X（如果存在的话，这样的事务要在T读取X前被撤销）。

正如我们将看到的那样，可恢复的调度要求一个复杂的恢复处理，但如果（在日志中）保存了足够的信息，那么可以设计一个恢复算法。在上一节中介绍的（偏）调度S_a和S_b都是可恢复的，因为它们都满足上述定义。考虑下面给出的调度S_a'，除了已经加入到S_a中的两个提交操作外它与调度S_a是相同的：

$$S_a' : r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); c_2; w_1(Y); c_1;$$

S_a'是可恢复的，即使它遇到更新丢失的问题。但是，考虑下面的二个（偏）调度S_c和S_d：

$$S_c : r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); c_2; a_1;$$

$$S_d : r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); c_1; c_2;$$

$$S_e : r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); a_1; a_2;$$

S_c是不可恢复的，因为T_2读取了T_1中的数据项X，然后T_2又在T_1提交前执行了提交。如果S_c中T_1

① 理论上并不需要确定非冲突操作对之间的顺序。

② 实际上大多数调度都有操作的一个全序。如果使用并行处理，那么在理论上带有偏序的非冲突操作的调度就是可能的。

在C_2操作后撤销，那么T_2读取的X值就不再是正确的，因此T_2必须在它已经提交后再撤销，这便导致了一个不可恢复的调度。若希望该调度成为可恢复的，S_c中的C_2操作必须被推迟提交，直至T_1提交成功后再提交，如S_d所示。如果T_1撤销而不是提交，那么T_2也就应该撤销，如S_e所示，因为T_2读取的X值不再是正确的。

在一个可恢复的调度中，永远不需要对已提交事务进行回滚。但是，可能会有**级联回滚**（cascading rollback）或**级联撤销**（cascading abort）现象发生。这里，因为该事务读取了故障事务的数据，所以不得不对这个未提交事务进行回滚。调度S_e是一个很好的例子，其中事务T_2必须被回滚，因为它读取了T_1中的X，但随后T_1被撤销了。

级联回滚是非常耗时的，因为可能有大量的事务要被回滚（见第3章），所以描述保证不发生级联回滚现象的调度的特征是很重要的。如果调度中的每个事务仅读取已提交事务写入的数据，那么该调度就称为是**无级联的**（cascadeless）或**避免级联回滚的**（avoid cascading rollback）。在这种情况下，所有已读数据都不会发生丢弃，从而不会发生级联回滚。为了满足这个准则，调度S_e中的$r_2(X)$命令必须推迟到T_1已经提交（或撤销）之后，这样虽然延迟了T_2，但可以确保如果T_1撤销，则不会发生级联回滚。

最后，第三个具有更多限制的调度类型称为**严格调度**（strict schedule）。在这种调度中，事务既不读取也不写入数据项X，直到最后写入X的事务都成功提交（或撤销）。严格调度简化了恢复处理。在严格调度中，撤销某个已撤销事务的write_item(X)操作的处理，只需简单恢复数据项X的**前映像**（before image）（old_value或BFIM）。这个简单的恢复过程总是能准确地应用于严格调度，但可能无法适用于可恢复的或无级联的调度。如考虑调度S_f：

$$S_f: w_1(X,5); w_2(X,8); a_1;$$

假定X的值原本是9，作为 $w_1(X,5)$ 操作的前映像存储在系统日志中。如果S_f中的T_1撤销，那么即使X已被事务T_2更新为8，在恢复时也可借助X的前映像将X恢复为9，这样将导致潜在的不正确值。虽然调度S_f是无级联的，但它并不是一个严格调度，因为即使最后写入X的事务T_1还没有提交（或撤销），它仍然允许T_2写入X，这在严格调度中是不会发生的。

现在我们已经根据如下条件描述了调度的特征：(1)可恢复性，(2)避免级联回滚，(3)严格性。调度的这3个性质的严格性条件依次递增。因此条件2隐含了条件1，条件3隐含了条件2和1。所以，所有的严格调度都是无级联的，所有的无级联调度都是可恢复的。

1.5　描述基于可串行性调度的特征

在前一节中，我们描述了基于可恢复性性质的调度的特征。现在再来描述另外一些调度类型的特征，即并发事务执行时认为是正确的调度类型。假定有两个用户（两个机票预订员），同时向DBMS提交如图1-2所示的事务T_1和T_2。如果不允许交替操作，那么只有两个可能的结果：

(1) 在（依次）执行事务T_2的所有操作后再（依次）执行事务T_1的所有操作。

(2) 在（依次）执行事务T_1的所有操作后再（依次）执行事务T_2的所有操作。

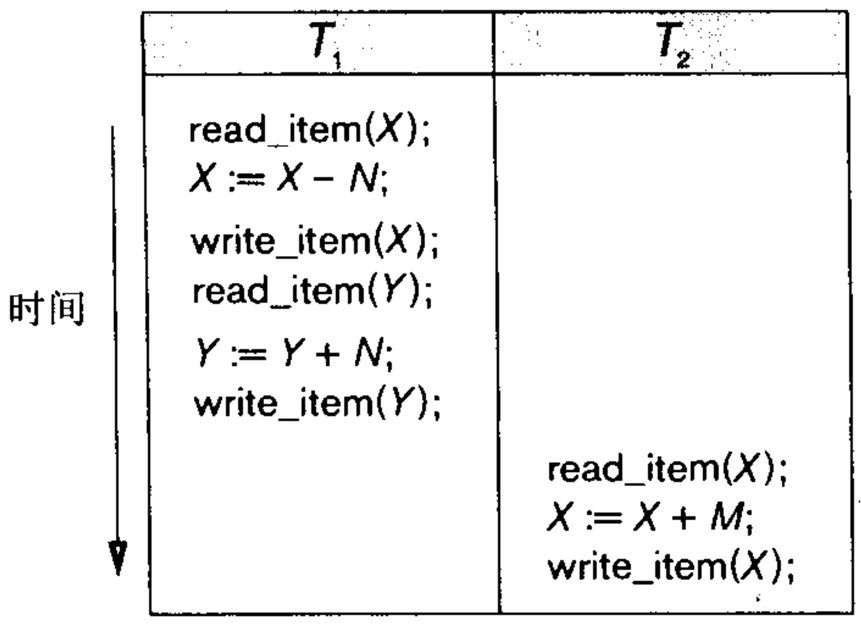
上述两个调度如图1-5a和图1-5b所示。如果允许交替操作，那么系统将会以多种可能的顺序执行事务的相关操作。其中两个可能的调度如图1-5c所示。使用**调度的可串行性**（serializability of schedules）概念来表示：当调度中事务的操作交替执行时，该调度也是正确的。本节将定义可串行性概念，并讨论其实际应用。

1.5.1　串行、非串行和冲突-可串行化调度

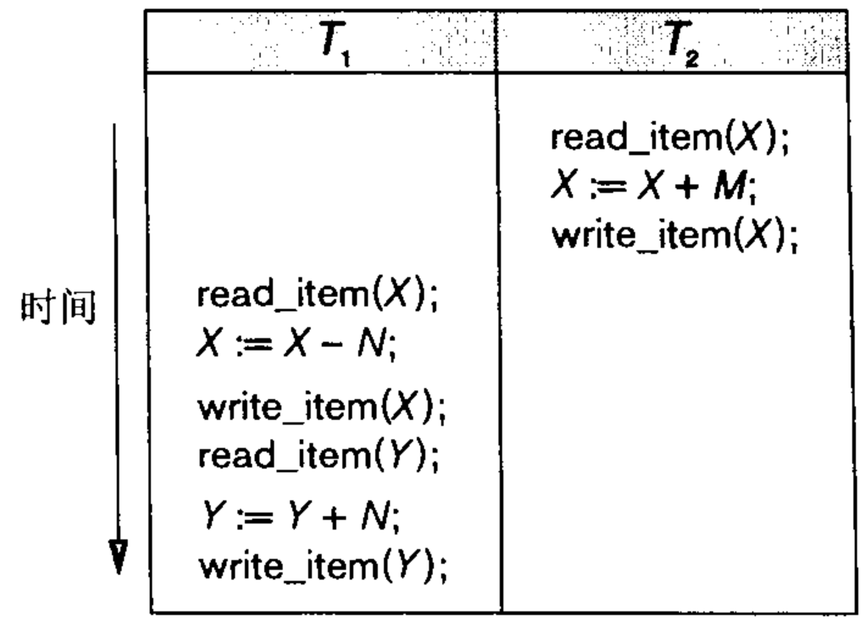
图1-5a和图1-5b中的调度A和B被称为串行调度，因为每个事务的操作都是连续执行的，没有任何来自其他事务的交替操作。在串行调度中，全部事务都以串行顺序执行：图1-5a中T_2跟随在T_1后面，

图1-5b中T_1跟随在T_2后面。图1-5c中的调度C和D称为非串行调度，因为每个序列都包含来自两个事务操作的交替执行。

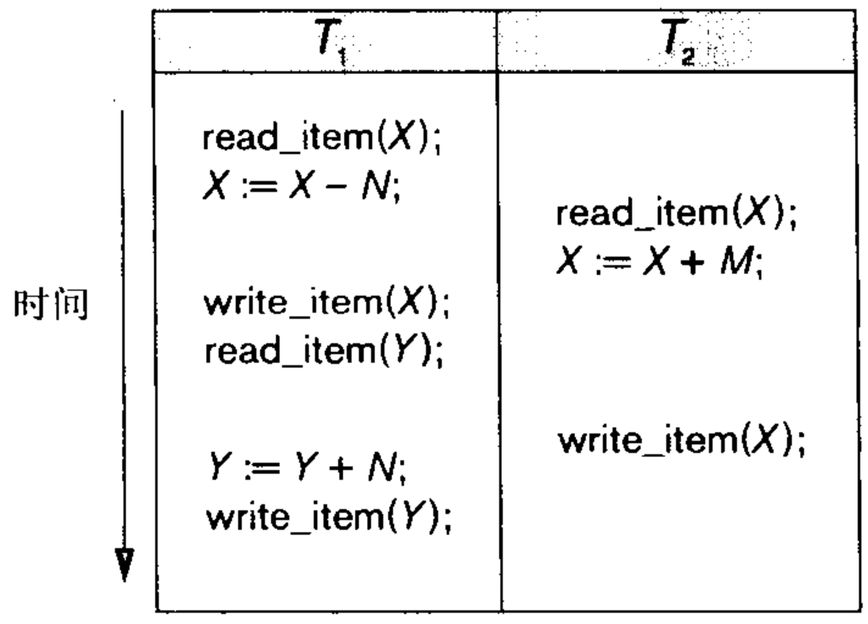
上述定义的形式化叙述如下。对于调度S中的每个事务T，如果T中所有的操作在调度中都是连续执行的，那么就称调度S是**串行的**（serial）；否则，称调度S就是**非串行的**（nonserial）。因此在串行调度中，同一时刻只有一个事务处于活动状态，当前事务的提交（或撤销）将会启动下一个事务的执行。串行调度中不会发生不同事务的交替操作。如果我们认为事务是相互独立的，那么就可以做一个合理的假定，即认为每个串行的调度都是正确的。我们之所以可以做出这样的假定，是因为独立执行的每个事务都被假定是正确的（根据1.3节的一致性特性）。因此，哪一个事务先执行是无关紧要的。只要每个事务从开始到结束的被执行期间，都没有来自其他事务的任何操作的干扰，这样我们总可以得到数据库上正确的最终结果。使用串行调度的问题是它们限制了操作的并发或交替。在串行调度中，如果事务在等待完成某个I/O操作时，CPU处理器无法切换到另一个事务，这样就会浪费宝贵的CPU时间。另外，如果某事务T过长，那么其他事务在开始执行前不得不等待T完成所有操作。因此一般说来，串行调度很少在实际中应用。

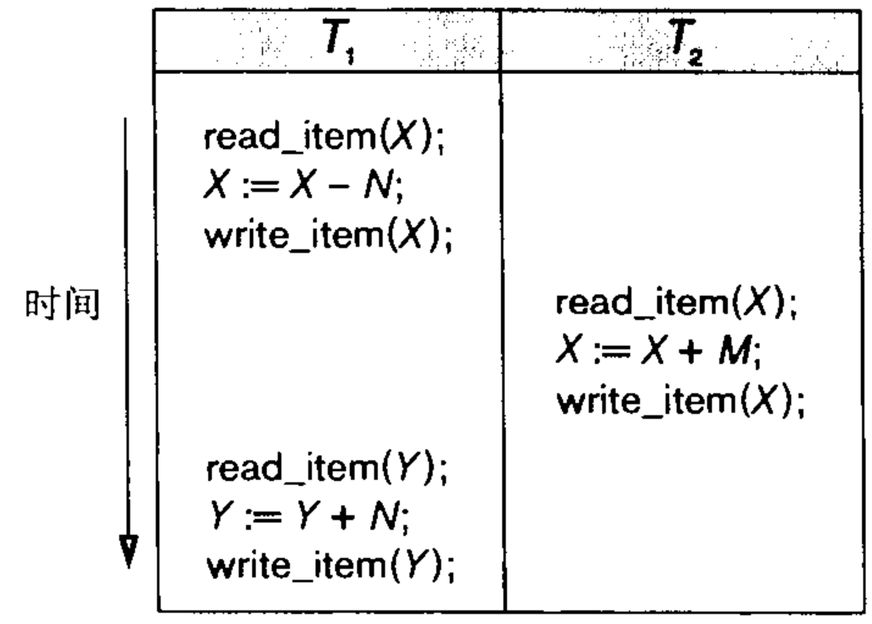


(a) 串行调度A：T_2跟随T_1



(b) 串行调度B：T_1跟随T_2





(c) 两个带有交替操作的非串行调度C和D

图1-5　包含事务T_1和T_2的串行和非串行调度示例

以图1-5中的调度为例，假定最初数据项的值分别为X=90，Y=90，N=3，M=2。在事务T_1和T_2执行完成后，数据项X和Y的值应分别为89和93。按照事务的含义，毫无疑问串行调度A或B都会得到正确的结果。现在考虑非串行调度C和D。调度C（同图1-3a相同）得到的结果是X=92，Y=93，其中X的值是错误的，而调度D将会得到正确的结果。

调度C得到错误结果的原因在于1.1.3节所讨论的更新丢失问题。事务T_2是在事务T_1改变X前对X

值进行读取的，所以在数据库中只有T_2在X上的结果受到了影响。因为T_2对X进行了重写，所以T_1对X的更新丢失了，这导致X值出现错误。但是，另外一些非串行调度得到的是所期望的正确结果，如调度D。我们需要确定哪些非串行调度总能得到正确结果，而哪些将得到错误结果。要以这种方式描述调度的特征，就涉及调度的可串行性概念。

如果具有n个事务的调度S等价于某个具有相同n个事务的串行调度，那么S就是**可串行化的**（serializable）。我们首先简单定义调度等价的概念。n个事务可能有$n!$个串行调度，而可能的非串行调度就更多了。可将非串行调度划分为两个不相交的组：那些等价于一个（或多个）串行调度的非串行调度分为一组，是可串行化的；那些不等价于任何串行调度的非串行调度分为另一组，是不可串行化的。

称非串行调度S是可串行化的，也就等价于说它是正确的，因为它等价于一个被认为是正确的串行调度。现在有待解决的问题是：在什么情况下两个调度被认为是“等价的”。目前有一些方法来定义调度的等价。最简单的、但也是最不令人满意的调度等价定义是，在数据库中比较调度结果，如果两个调度在数据库上产生同样的最终状态，那么就称之为**结果等价**（result equivalent）。然而，两个不同的调度也可能偶然产生相同的最终状态。如在图1-6中，如果调度S_1和S_2都以初始值X=100开始执行，那么它们将产生相同的数据库状态；但是对于其他的初始值X，这两个调度就不是结果等价的。另外，这两个调度执行的是不同的事务，所以可以肯定它们是不等价的，因此单独的结果等价并不能用于定义调度等价。定义调度等价最安全和最通用的方法是：对包含在事务内的操作类型不做任何假定。对于两个等价的调度，作用于受调度影响的数据项上的操作，在这两个调度中都应以相同的顺序作用在该数据项上。常用的两个调度等价定义是：冲突等价和视图等价。下面讨论冲突等价，这是更常用的一种定义。

S_1	S_2

read_item(X);
$X := X + 10$;
write_item(X);
	read_item(X);
	$X := X * 1.1$;
	write_item (X);

图1-6　两个调度对于初始值X=100是结果等价的，然而大部分情况下它们不是结果等价的

如果在两个调度中，任何两个冲突操作的顺序都是相同的，那么这两个调度就是**冲突等价的**（conflict equivalent）。回忆一下1.4.1节中的定义，如果一个调度中的两个操作属于不同的事务，并且访问相同的数据项，而且至少有一个操作是write_item操作，那么就称这两个操作是冲突的。如果在两个调度中以不同顺序应用这两个冲突操作，那么数据库的结果或调度中其他事务的结果就可能是不同的，因此这两个调度就不是冲突等价的。如假设一个读操作和一个写操作在调度S_1中以$r_1(X)$、$w_2(X)$的顺序执行，而在调度S_2中以相反的顺序$w_2(X)$、$r_1(X)$执行，那么$r_1(X)$读取的值在两个调度中就可能是不同的。同样，如果两个写操作在S_1中以$w_1(X)$、$w_2(X)$的顺序执行，而在S_2中以相反的顺序$w_2(X)$、$w_1(X)$执行，那么这两个调度中的下一个$r(X)$操作就有可能读到不同的值；或者如果这两个操作是调度中最后写入X项的操作，那么在数据库中X项的最终值也将是不同的。

使用冲突等价的表示法，如果一个调度S（冲突）等价于某串行调度S'，定义该调度S是**冲突可串行化的**（conflict serializable）[①]。在这种情况下，可对S中的非冲突操作重新排序，直到形成等价串行调度S'。按照该定义，图1-5c中的调度D就等价于图1-5a中的串行调度A。在这两个调度中，T_2的read_item(X)读取的都是由T_1写入的X值，而其他read_item操作读取的都是来自初始数据库状态的数

① 我们将使用可串行化来表示冲突可串行化。在实际中使用的另外一个可串行化定义（见1.6节）具有可重复读、无脏读、无幻像记录的性质（见2.7.1节关于幻像的讨论）。

据库值。另外在这两个调度中，T_1都是最后要写Y的事务，而T_2是两个调度中最后要写X的事务。因为A是串行调度，而调度D等价于A，所以D就是一个可串行化调度。注意调度中$r_1(Y)$和$w_1(Y)$操作并不与$r_2(X)$和$w_2(X)$操作冲突，因为它们访问的是不同的数据项。因此我们可将$r_1(Y)$、$w_1(Y)$操作移到$r_2(X)$、$w_2(X)$之前，从而得到一个等价的串行调度T_1、T_2。

图1-5c中的调度C不等价于串行调度A和B中的任何一个，因此它是不可串行化的。对调度C中的操作重新排序，也仍旧无法得到等价的串行调度，因为$r_2(X)$和$w_1(X)$是冲突的，这意味着不能将$r_2(X)$移到下面以得到等价的串行调度T_1、T_2。同样的，因为$w_1(X)$和$w_2(X)$是冲突的，所以也无法将$w_1(X)$移到下面，以得到等价的串行调度T_2、T_1。

另一个更为复杂的等价定义称为视图等价，它引出视图可串行化的概念。这将在1.5.4节中讨论。

1.5.2　测试调度的冲突可串行性

确定调度的冲突可串行性有一个简单的算法。大多数并发控制方法并不真正测试可串行性，而协议或规则是用于保证一个调度是可串行化的。下面讨论测试调度的冲突可串行性的算法，以更好地理解第2章将要讨论的并发控制协议。

算法1.1可用于测试调度的冲突可串行性。该算法只查看调度中的read_item和write_item操作，并构造**优先图**（precedence graph）或**串行化图**（serialization graph），这是一个**有向图**（directed graph）$G=(N, E)$，它包含一组结点$N=\{T_1, T_2, \cdots, T_n\}$和一组有向边$E=\{e_1, e_2, \cdots, e_m\}$。图中结点表示调度中的每个事务$T_i$。图中的每条边$e_i$形如$(T_j \rightarrow T_k)$，$1 \leqslant j \leqslant n$，$1 \leqslant k \leqslant n$，其中$T_j$是$e_i$的**起始结点**（starting node），T_k是e_i的**终止结点**（ending node）。如果调度中T_j的一个操作出现在T_k的某个冲突操作前，那么就创建这样的一条边。

算法1.1　测试调度S的冲突可串行性。

(1) 对于调度S中的每个事务T_i，在优先图中创建一个结点T_i

(2) 对于每一种这样的情形：如果S中T_j在T_i执行write_item(X)操作后执行read_item(X)操作。那么在优先图中创建一条边$(T_i \rightarrow T_j)$。

(3) 对于每一种这样的情形：如果S中T_j在T_i执行了read_item(X)操作后执行write_item(X)操作。那么在优先图中创建一条边$(T_i \rightarrow T_j)$。

(4) 对于每一种这样的情形：如果S中T_j在T_i执行write_item(X)操作后执行write_item(X)操作，那么在优先图中创建一条边$(T_i \rightarrow T_j)$。

(5) 当且仅当优先图中没有环路时，调度S是可串行化的。

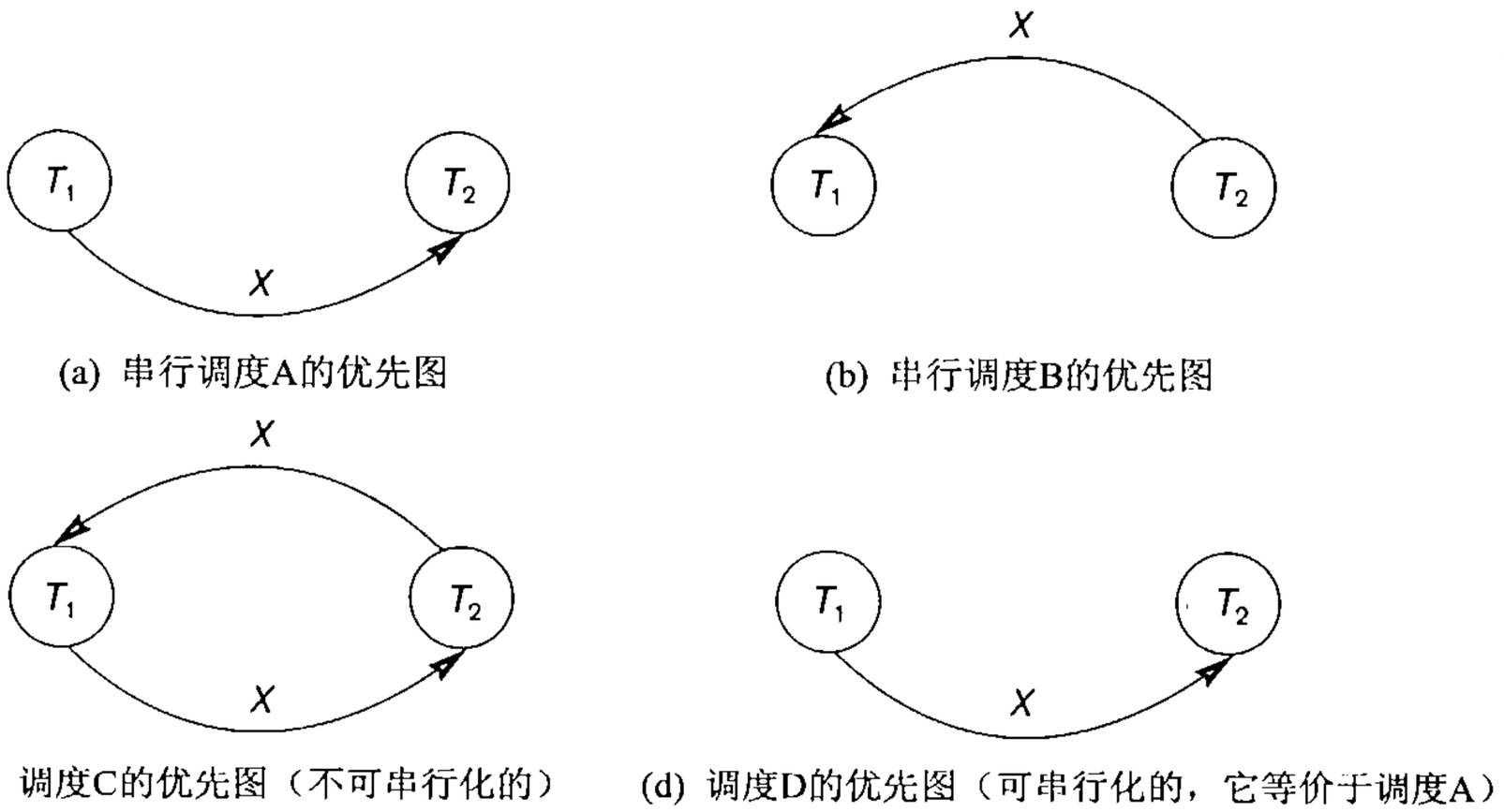
可以依据算法1.1构造优先图。如果在优先图中存在环路，那么调度S就是不可（冲突）串行化的；否则，S就是可串行化的。**环路**（cycle）是指有向图中的一个**边序列**（sequence of edge）$C=((T_j \rightarrow T_k),(T_k \rightarrow T_p), \cdots (T_i \rightarrow T_j))$，其中每条边的起始结点（除第一条边）都与前一条边的终止结点相同，而第一条边的起始结点与最后一条边的终止结点相同（序列是以同一个结点作为开始和结束的）。

在优先图中，一条从T_i到T_j的边意味着事务T_i在任何与S等价的串行调度中必须出现在事务T_j之前，因为两个冲突的操作要按那个顺序出现。如果优先图中不存在环路，就可以创建与S等价的**等价串行调度**（equivalent serial schedule）S'，并按如下方式对S中的事务进行排序：只要在优先图中存在从T_i到T_j的边，则在等价串行调度S'中T_i就必须出现在T_j之前[①]。注意，可以在优先图中的边$(T_i \rightarrow T_j)$上加标注，即以导致创建此边的数据项的名作标注。图1-7显示了边上的这种标注。

通常，如果S的优先序图不存在环路，那么就可有若干个与S等价的串行调度。但如果优先图存

① 这种无回路图的结点排序过程也被称作拓扑排序。

在环路，就不可能创建任何等价的串行调度，从而*S*是不可串行的。图1-7a到图1-7d是和图1-5中的调度A到D相对应的优先图。调度C的优先图存在环路，所以它是不可串行化的。调度D的优先图中没有环路，因此调度D是可串行化的，其等价的串行调度是T_1，T_2。调度A和B的优先图如预期一样没有出现环路，因为它们是串行的也是可串行化的。



(a) 串行调度A的优先图　(b) 串行调度B的优先图

(c) 调度C的优先图（不可串行化的）　(d) 调度D的优先图（可串行化的，它等价于调度A）

图1-7　构造图1-5中调度A到D的优先图，用来测试冲突可串行性

3个事务参与的另一个示例如图1-8所示，图1-8a表示的是每个事务的read_item和write_item操作。这3个事务的两个调度E和F相应表示在图1-8b和图1-8c中，调度E和F的优先图表示在图1-8d和图1-8e中。调度E是不可串行化的，因为其对应的优先图中存在环路。调度F是可串行化的，与F等价的串行调度如图1-8e。虽然这里只表示出F的一个等价串行调度，但一个可串行化调度其等价串行调度通常不止一个。图1-8f中是具有两个串行调度的优先图。

事务T_1
read_item(*X*);
write_item(*X*);
read_item(*Y*);
write_item(*Y*);

事务T_2
read_item(*Z*);
read_item(*Y*);
write_item(*Y*);
read_item(*X*);
write_item(*X*);

事务T_3
read_item(*Y*);
read_item(*Z*);
write_item(*Y*);
write_item(*Z*);

(a) 3个事务T_1，T_2和T_3的读和写操作

事务T_1	事务T_2	事务T_3
	read_item(*Z*);	
	read_item(*Y*);	
	write_item(*Y*);	
		read_item(*Y*);
		read_item(*Z*);
read_item(*X*);		
write_item(*X*);		
		write_item(*Y*);
		write_item(*Z*);
	read_item(*X*);	
read_item(*Y*);		
write_item(*Y*);		
	write_item(*X*);	

时间

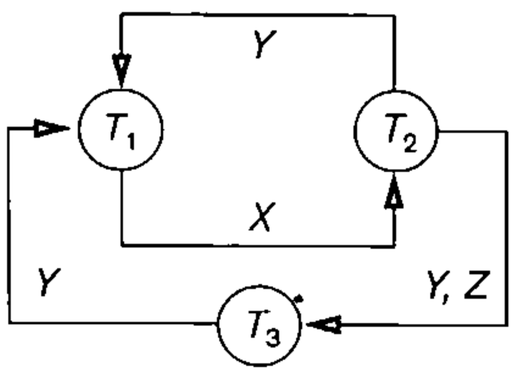
(b) 调度E

图1-8　可串行性测试的另一个示例

事务T_1	事务T_2	事务T_3
		read_item(Y);
read_item(X);		read_item(Z);
write_item(X);		
		write_item(Y);
		write_item(Z);
	read_item(Z);	
read_item(Y);		
write_item(Y);	read_item(Y);	
	write_item(Y);	
	read_item(X);	
	write_item(X);	

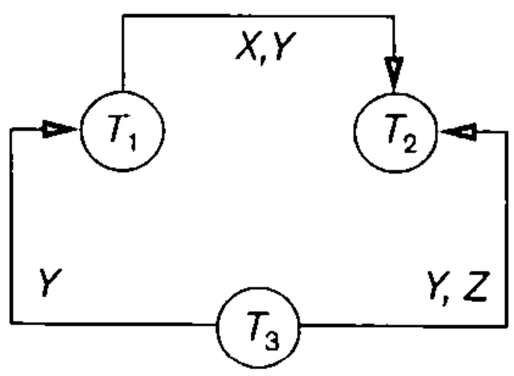
时间

(c) 调度F



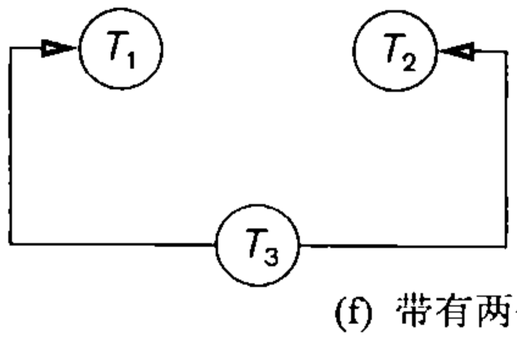
等价串行调度	原因
没有	环 $X(T_1 \rightarrow T_2)$, $Y(T_2 \rightarrow T_1)$ 环 $X(T_1 \rightarrow T_2)$, $YZ(T_2 \rightarrow T_3)$, $Y(T_3 \rightarrow T_1)$

(d) 调度E的优先图



等价串行调度
$T_3 \rightarrow T_1 \rightarrow T_2$

(e) 调度F的优先图



等价串行调度
$T_3 \rightarrow T_1 \rightarrow T_2$
$T_3 \rightarrow T_2 \rightarrow T_1$

(f) 带有两个等价串行调度的优先图

图1-8（续）

1.5.3　可串行性的用途

正如我们前面讨论的那样：S是（冲突）等价于一个串行调度，就等价于S是正确的，那么称调度S是冲突可串行化的。但是，可串行化调度不同于串行调度。串行调度处理的效率比较低，因为它不允许不同事务的操作交替。当事务等待磁盘I/O，或等待另一个事务终止时，就会导致CPU资源的浪费，从而大大降低处理效率。可串行化调度支持事务的并发，并且可以保证执行的正确性。在实际应用中，测试调度的可串行性是非常困难的。因为并发事务操作的交替通常是由操作系统调度程序来实现的，它为所有进程分配资源，操作系统通常把并发事务当作进程来执行。如系统装载、事务提交的时间、进程的优先级等因素将有助于对调度中的操作排序。因此，很难为确保可串行性而事先确定调度的操作如何交替。

如果随意执行事务，然后再测试结果调度的可串行性，那么一旦它产生的结果是不可串行的就

必须取消这个调度的影响。这个严重的问题使这种方法不切实际。因此，在大多数实际系统中采取的方法是确定方法以保证可串行性，而不必测试调度本身。在大多数商业DBMS中采取的方法是设计**协议**（protocol，规则的集合）。如果每个单独事务都遵循协议，或者协议被一个DBMS并发控制子系统执行，那么将确保事务参与的所有调度都是可串行性的。

这里出现的另一个问题是：当不断地向系统提交事务时，很难确定调度何时开始以及何时结束。仅需考虑调度S的已提交**投影**（projection），就可使可串行性理论适用于处理这个问题。回忆1.4.1节中的调度S的已提交投影$C(S)$，它只包含S中已提交事务的操作。我们可以从理论上定义，如果调度S的已提交投影$C(S)$等价于某串行调度，那么调度S是可串行化的，因为DBMS保证的仅是已提交事务。

在第2章中，我们将讨论多种保证可串行性的并发控制协议。其中最常用的技术是两阶段加锁（two-phase locking），它是基于对数据项进行锁定以阻止并发事务相互干扰，并且执行了一个附加条件来确保可串行性。大多数商业DBMS使用的都是这种技术。还有很多其他协议[①]，例如时间戳排序（timestamp ordering），其中每个事务被指定一个唯一的时间戳，协议确保按照事务时间戳的顺序执行任何冲突操作；多版本协议（multiversion protocol）对数据项的多个版本进行维护；最优化（optimistic，也称为验证或确认）协议，是在事务终止后但在事务被允许提交前，检查是否存在可串行性违反情况。

1.5.4　视图等价和视图可串行性

在1.5.1节中，我们定义了调度冲突等价和冲突可串行性的概念。另一个具有较少限制的调度等价定义称为视图等价。它引出另一个可串行性的定义称为视图可串行性。如果调度满足下列3个条件，那么就说两个调度S和S'是**视图等价**（view equivalent）的：

(1) S和S'包括相同的参与事务集合，并且S和S'包含这些事务的相同操作。

(2) 对于S中T_i的任何操作$r_i(X)$，如果由该操作读取的X值已由T_j中的操作$w_j(X)$写入（或者如果它是在调度开始前X的原始值），那么被S'中T_i的操作$r_i(X)$所读取的X值必须保持相同的条件。

(3) 如果T_k的操作$w_k(Y)$是S中最后一个写入Y的操作，那么T_k的$w_k(Y)$必须也是S'中最后写入Y的操作。

视图等价的背景思想是，只要事务的每个读操作读取的是两个调度中同一个写操作的结果，那么每个事务的写操作必须产生相同的结果。因此读操作也可以说成是在两个调度中看到了相同的视图。条件3保证了两个调度中最后写操作访问的每个数据项都是相同的，因此数据库状态在两个调度的结束处也应该是相同的。如果调度S视图等价于一个串行调度，那么就称S是**视图可串行化的**（view serializable）。

如果调度中所有事务都满足称为**约束写假定**（constrained write assumption）的条件，那么冲突可串行化的定义同视图可串行化的定义就是类似的。约束写假定指任何T_i中的写操作$w_i(X)$都跟在T_i的一个$r_i(X)$之后，并且T_i中$w_i(X)$写入的值只依赖于$r_i(X)$读取的X值。它假定新X值的计算是一个基于从数据库中读取的旧X值的函数$f(X)$。但是，在**无约束写假定**（unconstrained write assumption）下，即T_i的操作$w_i(X)$写入的值，可以独立于数据库中的旧值，那么对于视图可串行化的定义，它的限制就要少于冲突可串行化的定义。这种情况称为**盲目写**（blind write），例如，下面的调度S_g说明了盲目写，它具有3个事务：$T_1: r_1(X); w_1(X); T_2: w_2(X); T_3: w_3(X)$。

$$S_g: r_1(X); w_2(X); w_1(X); w_3(X); c_1; c_2; c_3;$$

因为T_2和T_3都没有读取X的值，所以在S_g中操作$w_2(X)$和$w_3(X)$就是盲目写入的。又因为调度S_g

---

① 迄今为止在实际应用中并没有大量使用这些协议；大多数系统使用的都是两阶段加锁协议的某个变种。

视图等价于串行调度T_1，T_2，T_3，所以S_g是视图可串行化的。但S_g并不是冲突可串行化的，因为它并不冲突等价于任何串行调度。任何冲突可串行化的调度都是视图可串行化的，但是反之并不一定成立，上面的这个示例便说明了这一点。有一个算法用于测试调度S是否是视图可串行化的。但是，测试视图可串行性的问题已经被证明是非线性-难(NP-hard)的，这就意味着要找到用于这个问题的有效多项式时间算法几乎是不可能的。

1.5.5　调度等价性的其他类型

作为一个保证并发执行正确性的条件，调度的可串行性有时被认为过于严格。某些应用程序通过满足一些比冲突可串行性或视图可串行性严格程度低一些的条件，也可以产生正确的调度。**借贷事务**（debit-credit transaction）就是这样的一个示例。例如，对一个数据项应用取款和存款操作，数据项的值表示一个银行账户的当前余额。借贷操作的语义是通过对数据项X进行减或加来更新该数据项的值。因为加减操作是可交换的，即它们可以以任何顺序被应用，所以可以生成非可串行化的正确的调度。如下面的两个事务，每个事务都用于在两个银行账户间划拨适量资金：

$$T_1 : r_1(X); X := X - 10; w_1(X); r_1(Y); Y := Y + 10; w_1(Y);$$

$$T_2 : r_2(Y); Y := Y - 20; w_2(Y); r_2(X); X := X + 20; w_2(X);$$

考虑下面的不可串行化调度S_h：

$$S_h : r_1(X); w_1(X); r_2(Y); w_2(Y); r_1(Y); w_1(Y); r_2(X); w_2(X);$$

$r_i(I)$和$w_i(I)$之间的操作是可交换的，在了解这个额外的知识或**语义**后，只要特定事务T_i在特定项I上形成的（读、更新、写）序列不被冲突操作所中断，我们就可以知道执行这些由（读、更新、写）组成的序列的顺序并不重要。因此，虽然调度S_h是不可串行化的，但它是正确的。研究人员已经致力于寻找扩展的并发控制理论以处理某些情况，如可串行性作为调度正确性的条件被认为是过于严格时。

1.6　SQL 中的事务支持

SQL事务的定义类似于我们已经定义过的事务的概念。即，它是一个逻辑工作单元并且保证原子性。单一的SQL语句总被认为是原子的，因为它要么无差错地完成执行过程，要么发生故障并且未对数据库造成任何改变。

SQL中没有显式的Begin_Transaction语句。当处理一个特定的SQL语句时，将隐式地执行事务的初始化。但每个事务必须有显式的结束语句，它可以是COMMIT或ROLLBACK。每个事务都有某些属于它的特征。这些特征由SQL的SET TRANSACTION语句指定，包括存取方式、诊断区域范围、隔离级别。

存取方式（access mode）可以指定为READ ONLY或READ WRITE。默认值是READ WRITE，除非隔离级别READ UNCOMMITTED被指定（详见下面的叙述），在这种情况下存取方式是READ ONLY。READ WRITE方式允许执行选择、更新、插入、删除和创建等命令。READ ONLY方式只允许数据检索，如同它的名字的含义一样。

诊断区域范围（diagnostic area size）选项表示为DIAGNOSTIC SIZE n，它通过指定一个整数n来表示在诊断区域中可以同时保存的状态数目。这些状态向用户或程序提供最近执行的SQL语句的反馈信息（错误或异常）。

使用语句ISOLATION LEVEL<isolation>定义**隔离级别**（isolation level），其中< isolation >的值可以是READ UNCOMMITTED、READ COMMITTED、REPEATABLE READ或者SERIALIZABLE[①]。默认的

① 它们类似于1.3节结尾所简要讨论的隔离级别。

隔离级别是SERIALIZABLE，还有一些系统使用READ COMMITTED作为默认隔离级别。这里，使用SERIALIZABLE是用来防止脏读、不可重复读和幻像的发生[①]，因此它与1.5节中定义的可串行性的方法不完全相同。如果一个事务是在比SERIALIZABLE低的隔离级别上执行，那么有可能发生下述一个或多个违反情况：

(1) **脏读**（dirty read）：事务T_1可能读到事务T_2更新后的值，但是T_2还没有提交。如果T_2因故障而被撤销了，那么T_1就将读到一个并不存在的错误值。

(2) **不可重复读**（nonrepeatable read）：事务T_1可能从某个表中读到一个特定值。如果稍后另一事务T_2更新了该值，而T_1又读取该值，则T_1将读到一个不同的值。

(3) **幻像**（phantom）：事务T_1可能从某表读取一行数据，该行是基于某个在SQL WHERE子句中指定的条件。现在假定事务T_2在T_1用到的表中插入一个新行，它也满足T_1使用的WHERE子句的条件。那么如果T_1被重复执行，它就会看到一个幻像，即以前并不存在的行。

表1-1总结了不同隔离级别可能发生的违反情况。表中“是”表示可能发生违反，“否”表示不可能发生违反。其中，READ UNCOMMITTED级别是最宽松的，而SERIALIZABLE级别是最严格的，其避免了上面提到的所有3种问题。

表1-1　基于SQL中所定义隔离级别的可能违反类型

隔离级	违反类型		
	脏读	不可重复读	幻像
READ UNCOMMITTED	是	是	是
READ COMMITTED	否	是	是
REPEATABLE READ	否	否	是
SERIALIZABLE	否	否	否

如下是SQL事务的一个示例：

```
EXEC SQL WHENEVER SQLERROR GOTO UNDO;
EXEC SQL SET TRANSACTION
     READ WRITE
     DIAGNOSTIC SIZE 5
     ISOLATION LEVEL SERIALIZABLE;
EXEC SQL INSERT INTO EMPLOYEE (Fname, Lname , Ssn , Dno, Salary)
     VALUES ('Robert', 'Smith', '991004321', 2, 35000);
EXEC SQL UPDATE EMPLOYEE
     SET Salary = Salary * 1.1 WHERE Dno = 2;
EXEC SQL COMMIT;
GOTO THE_END;
UNDO: EXEC SQL ROLLBACK;
THE_END: ...;
```

上述事务首先在EMPLOYEE表中插入一个新行，然后更新所有在部门2工作的雇员的工资。如果SQL语句中的任何一句发生错误，那么将导致整个事务的撤销。这意味着任何（由这个事务）被更新的工资值都将被恢复为原值，同时新插入的行将被删除。

如同我们所看到的那样，SQL提供许多面向事务的特性。如果某些事务应用可以接受放宽可串行性的限制，那么DBA或者程序员就可以利用这些选项，通过放宽可串行性来尝试提高事务的性能。

小结

本章讨论了DBMS事务处理的概念。介绍了数据库事务的概念以及与事务处理相关的操作。然

① 错读和不可重复读的问题在1.1.3节中已经讨论过。幻像将在2.1.6节中讨论。

后对单用户系统和多用户系统进行了比较，并举例说明多用户系统中并发事务的无控制执行将导致不正确的结果和数据库值。同时我们也讨论了在事务执行过程中可能发生的各种类型的故障。

接下来我们介绍了事务在执行过程中所经历的典型状态，并讨论了恢复和并发控制中用到的一些概念。系统在日志中保留了数据库的访问记录，并借助这些信息从故障中恢复。事务成功执行并到达提交点，或者因故障而必须被回滚，已提交的事务将其修改永久地记录在数据库中。我们还介绍了事务的ACID特性，即原子性、一致性、隔离性和持久性。

随后定义了调度（或历史）的概念，调度是若干事务的操作的一个执行序列，其中存在交替操作。另外描述了基于可恢复性调度的特征。可恢复的调度保证在事务提交成功后，永远无需对其进行撤销。无级联的调度增加了额外的条件，以保证不会发生被撤销的事务要求级联撤销其他事务的情况。严格的调度具有更强的约束条件，通过一个简单的恢复方式，可以将包括撤销事务所修改的数据项恢复为其原值。

本章定义了调度等价的概念，并说明可串行化调度等价于某个串行调度。另外还定义了冲突等价和视图等价的概念，由此引出了冲突可串行化和视图可串行化的概念。可串行化的调度被认为是正确的。然后介绍了测试调度（冲突）可串行化的算法，并讨论了为什么在现实系统中实施可串行化测试是不现实的，尽管这种方法可用于定义和检验并发控制协议。然后简要提到了另外一些具有较少限制的调度等价定义。最后简要说明了事务概念如何在SQL中实际使用。

在第2章将讨论并发控制协议，第3章讨论恢复协议。

复习题

1.1　在一个多用户系统中数据库事务的并发执行意味着什么？讨论为什么需要并发控制，并给出一个常见的示例。

1.2　讨论故障的不同类型。灾难性故障意味着什么？

1.3　讨论数据库上read_item和write_item操作所采取的动作。

1.4　画一张状态图，讨论事务在执行期间所经历的典型状态。

1.5　系统日志的作用是什么？系统日志内的典型记录类型是什么？什么是事务提交点，它们为什么重要？

1.6　讨论数据库事务的原子性、持续性、隔离性和一致性保持的性质。

1.7　什么是调度（历史）？给出可恢复的、无级联的和严格的调度的概念定义，并对它们在可恢复性方面进行比较。

1.8　讨论事务等价的不同标准。冲突等价和视图等价之间的区别是什么？

1.9　什么是串行调度？什么是可串行化调度？为什么认为串行调度是正确的？为什么认为可串行化调度是正确的？

1.10　约束写假定和无约束写假定之间的区别是什么？哪一个更现实？

1.11　讨论可串行性如何用于支持数据库系统中的并发控制。为什么有时候认为可串行性作为检验数据库正确性的方法过于严格？

1.12　描述SQL中的4个隔离级别。

1.13　定义由脏读、不可重复读和幻像所引起的违反类型。

练习题

1.14　对图1-2b中的事务T_2进行改动：

read_item(X);

X:=X+M;

if X>90 then exit

else write_item(X);

针对下面提出的问题，讨论图1-3a和图1-3b中不同调度的最终结果，其中M=2, N=2。加入上述条件改变最终结果吗？结果服从隐式的一致性规则（X的容量是90）吗？

1.15　重复练习题1.14，在T_1中加入约束，使得Y不能超过90。

1.16　在图1-2中事务T_1和T_2的结尾处加入提交操作；然后对修改后的事务列出所有可能的调度。确定哪些调度是可恢复的，哪些是无级联的，哪些是严格的。

1.17　列出图1-2中事务T_1和T_2所有可能的调度，确定哪一个是冲突可串行化的（正确的）？哪一个不是冲突可串行化的？

1.18　对于图1-8a中的3个事务存在多少个串行调度？可能的调度总数是多少？

1.19　编写一个程序，生成图1-8a中3个事务所有可能的调度，并确定其中哪些调度是冲突可串行化的，哪些不是。对于每个冲突可串行化的调度，在程序中要输出调度和列出所有等价的串行调度。

1.20　为什么SQL中需要一个显式的事务结束语句，却不需要一个显式的起始语句？

1.21　描述不同隔离级可以用于事务处理的情况。

1.22　下面的调度哪一个是（冲突）可串行化的？对每个可串行化的调度，确定其等价的串行调度。

a. $r_1(X);r_3(X);w_1(X);r_2(X);w_3(X);$

b. $r_1(X);r_3(X);w_3(X);w_1(X);r_2(X);$

c. $r_3(X);r_2(X);w_3(X);r_1(X);w_1(X);$

d. $r_3(X);r_2(X);r_1(X);w_3(X);w_1(X);$

1.23　考虑下面的3个事务T_1，T_2，T_3，以及给定的调度S_1和S_2。画出S_1和S_2的可串行性（优先）图，说明每个调度是否是可串行化的。如果调度是可串行化的，写出其等价的串行调度。

$T_1: r_1(X);r_1(Z);w_1(X);$

$T_2: r_2(Z);r_2(Y);w_2(Z);w_2(Y);$

$T_3: r_3(X);r_3(Y);w_3(Y);$

$S_1: r_1(X);r_2(Z);r_1(Z);r_3(X);r_3(Y);w_1(X);w_3(Y);r_2(Y);w_2(Z);w_2(Y);$

$S_2: r_1(X);r_2(Z);r_3(X);r_1(Z);r_2(Y);r_3(Y);w_1(X);w_2(Z);w_3(Y);w_2(Y);$

1.24　考虑下面的调度S_3，S_4，S_5。确定每个调度是否是严格的、无级联的、可恢复的或是不可恢复的。指出不可串行化的调度。

$S_3: r_1(X);r_2(Z);r_1(Z);r_3(X);r_3(Y);w_1(X);c_1;w_3(Y);c_3;r_2(Y);w_2(Z);w_2(Y);c_2;$

$S_4: r_1(X);r_2(Z);r_1(Z);r_3(X);r_3(Y);w_1(X);w_3(Y);r_2(Y);w_2(Z);w_2(Y);c_1;c_2;c_3;$

$S_5: r_1(X);r_2(Z);r_3(X);r_1(Z);r_2(Y);r_3(Y);w_1(X);c_1;w_2(Z);w_3(Y);w_2(Y);c_3;c_2;$

参考文献

（Gray et al，1975）引入了可串行化的概念。（Gray，1981）首次论述了数据库事务的概念。Gray因为其在数据库事务上的工作以及在关系数据库系统上实现事务而获得了1998年的ACM图灵奖。（Bernstein，Hadzilacos & Goodman，1987）将焦点集中在集中式系统和分布式系统中的并发控制和恢复技术上；这是一本极好的参考书。（Papadimitriou，1986）提供了更加理论化的观点。由Gray和Reuter所著（Gray & Reuter，1993）的超过1000页的大型参考书，提供了更加实用的事务处理概念和技术方面的观点。（Elmagarmid，1992）和（Bhargava，1989）提供了关于事务处理的研究论文集。（Date & Darwen，1993）描述了SQL中的事务支持。（Yannakakis，1984）定义了视图可串行化的概念。（Hadzilacos，1983,1988）讨论了调度的可恢复性。

第2章

并发控制技术

本章将讨论几种并发控制技术，这些技术用于确保并发执行的事务不相互干扰或相互具有隔离性的，通过运用保证可串行性的**协议**（即规则集）来保证调度的可串行性（见1.5节）。其中一个重要的协议集采用了数据项**加锁**技术来防止多个事务并发访问数据项。2.1节描述了大部分商业DBMS可以使用的加锁协议。另一个并发控制协议集使用了**时间戳**（timestamp），时间戳是系统为每个事务生成的唯一的标识符。2.2节描述了使用时间戳排序来保证可串行性的并发控制协议。2.3节讨论了**多版本**（multiversion）并发控制协议，该协议使用了一个数据项的多个版本。2.4节给出了一种基于事务执行操作之后的**确认**或者**验证**概念的协议，有时也称之为**乐观协议**。

影响并发控制的另一个因素是数据项的**粒度**（granularity），也就是一个数据项代表的是数据库的哪一部分。数据项可以小到一个属性（字段）值，也可以大到一个磁盘块，甚至可以是整个文件或整个数据库。数据项粒度将在2.5节中进行讨论。2.6节讨论了使用索引处理事务时引起的并发控制问题。最后，在2.7节中讨论其他一些并发控制问题。

如果重点是要介绍实际中最常用的并发控制技术，那么只需要阅读2.1节、2.5节、2.6节以及2.7节的内容，或许再包括2.3.2节就足够了。其他技术主要是理论研究。

2.1 并发控制的两阶段加锁技术

用于控制事务并发执行的主要技术都是基于数据项加锁的概念。**锁**（lock）是与数据项相关的一个变量，它描述了数据项的状态，也表示了可能应用于该数据项上的操作。通常，数据库中的每个数据项都有一个锁。锁是同步并发事务访问数据库中数据项的一种方法。2.1.1节讨论了锁的性质和类型；2.1.2节给出了使用加锁技术保证事务调度可串行性的协议；最后，在2.1.3节中讨论了与锁的使用相关的两个问题，即死锁（deadlock）和饥饿（starvation），并介绍了相应的解决方法。

2.1.1 锁类型与系统锁表

并发控制中使用的锁有若干种类型。为了逐步介绍加锁的概念，首先来讨论二进制锁。它比较简单，但有一定局限性，所以并没有在实际中应用；接下来讨论共享/排他锁，它具备了更通用的加锁能力，并能应用于实际的数据库加锁模式中。2.3.2节介绍了验证锁（certify lock），并描述了如何应用该锁提高加锁协议的性能。

1. 二进制锁

二进制锁（binary lock）有两个**状态**或**值**：已加锁（locked）和未加锁（unlocked）（或简单地记作1和0）。不同的数据库项X与一个不同的锁相关联。如果X的锁的值为1，则请求访问该项X的数据库操作不能访问X；如果X的锁的值是0，则请求访问该项X的数据库操作就可以访问X。我们把与数据项X相关联的锁的当前值（或状态）记作lock(X)。

二进制锁使用两个操作：lock_item和unlock_item。事务首先发出lock_item(X)操作来请求访问数据项X。如果LOCK(X) = 1，则该事务被迫等待；如果LOCK(X) = 0，则将LOCK(X)设置为1（该事务

锁定了该数据项），并且允许该事务访问*X*。当该事务完成对*X*的使用后，它要发出unlock_item(*X*)操作，这个操作将LOCK(*X*)置为0（对该数据项**解锁**），这样其他的事务就可以访问*X*了。因此，二进制锁在数据项上实现了**相互排斥**（mutual exclusion）。lock_item(*X*)与unlock_item(*X*)操作的描述如图2-1所示。

需要注意的是：lock_item与unlock_item操作必须实现为不可分割的单元（如同操作系统中的**临界段**（critical section）。也就是说，一旦加锁或解锁操作开始，只有等到操作结束或者事务等待时才允许操作交叉。图2-1中，lock_item(*X*)操作中的等待命令通常是这样实现的：把事务放入对应*X*的一个等待队列中，直到数据项*X*被解锁并准予该事务访问为止。其他需要访问*X*的事务也要放在同一个队列中。因此，可以认为等待命令在lock_item操作之外。

```
lock_item(X):

B: if LOCK(X) = 0            (* 项X未被加锁 *)
       then LOCK(X) ← 1      (* 加锁该项 *)
   else
       begin
       wait (until LOCK(X) = 0
           and the lock manager wakes up the transaction);
       go to B
       end;

unlock_item(X):

   LOCK(X) ← 0;              (* 解锁该项 *)
   if any transactions are waiting
       then wakeup one of the waiting transactions;
```

图2-1　二进制锁的加锁和解锁操作

需要注意，实现二进制锁非常简单，只需要一个与数据库中各数据项*X*相关联的二值变量LOCK。如果采用最简单的形式，每个锁可以是一个包含3个字段的记录：<Data_item_name，LOCK，Locking_transaction>，再加上一个指向等待访问该数据项的事务队列的指针。系统只需为当前在**锁表**（lock table）中已加锁的数据项保存相应记录，锁表可以组织成一个散列文件。不在锁表中的数据项就认为是未加锁的。DBMS使用一个**锁管理器子系统**（lock manager subsystem）来跟踪和控制对锁的访问。

如果要使用在此所述的简单二进制加锁模式，每个事务必须遵守以下规则：

(1) 事务*T*在完成任何read_item(*X*)或write_item(*X*)操作之前，*T*必须发出lock_item(*X*)操作。

(2) 事务*T*在所有read_item(*X*)和write_item(*X*)操作结束之后，*T*必须发出unlock_item(*X*)操作。

(3) 如果事务*T*已经持有数据项*X*的锁，则不发出lock_item(*X*)操作[①]。

(4) 如果事务*T*没有持有数据项*X*的锁，则不发出unlock_item(*X*)操作。

这些规则可以由DBMS的锁管理器模块执行。事务*T*在lock_item(*X*)和unlock_item(*X*)操作之间的过程，称*T***持有数据项***X***的锁**（hold the lock）。最多只有一个事务能够持有一个特定数据项的锁。因此两个事务不能并发访问同一个数据项。

2. 共享/排他锁（或读/写锁）

对于数据库项来说，上面提到的二进制加锁模式限制太多，因为最多只能有一个事务可以持有一个给定数据项的锁。如果多个事务访问*X*的目的只是为了读，那么应允许这些事务访问同一个数据

① 如果修改图2-1中的lock_item(*X*)操作，即如果该数据项当前由请求它的事务锁定，则同意这个加锁请求，就可以删除这条规则。

项X。但如果一个事务要写X项，则必须互斥访问X。出于这个原因，可以使用一种不同类型的锁，称为**多方式锁**（multiple-mode lock）。这种模式被称为**共享/排他锁**（shared/exclusive lock）或**读/写锁**（read/write lock）。其中有3种锁操作，分别是：read_lock(X)、write_lock(X)和unlock(X)。因此与数据项X相关联的锁LOCK（X）就有3种可能的状态：读锁定、写锁定和未锁定。**读锁定数据项**（read-locked item）也被称为**共享锁定**（share-locked），因为允许其他事务读取此项；而**写锁定数据项**（write-locked item）被称为**排他锁定**（exclusive-locked），因为仅有一个事务排他地持有该数据项的锁。

要实现读/写锁的上述3种操作，一种方法是跟踪锁表中持有某数据项共享（读）锁的事务的数量。锁表中的每条记录有4个字段：<Data_item_name，LOCK，No_of_reads，Locking_transaction>。同样，为了节省空间，系统只需在锁表中维护被锁定数据项的锁记录。采用适当编码，LOCK的值（状态）可以是读锁定（read_locked）或写锁定（write_locked），这里假设在锁表中没有为未锁定的数据项保存记录。如果LOCK(X)的值为写锁定，加锁事务的值是持有数据项X的排他（写）锁的单个事务。如果LOCK(X)的值为读锁定，则加锁事务的值是持有数据项X共享（读）锁的一个或多个事务的列表。read_lock(X)、write_lock(X)和unlock(X)这3种操作[①]如图2-2所示。如前所述，这3种操作中的每一个都应当认为是不可分割的。一旦一个操作开始就不允许操作交叉，直到因允许加锁而终止操作或者该事务被置于该项的等待队列中为止。

采用共享/排他锁模式时，系统必须遵循如下规则：

(1) 事务T在完成任何read_item(X)操作之前，T必须发出read_lock(X)操作或write_lock(X)操作。

(2) 事务T在完成任何write_item(X)操作之前，T必须发出write_lock(X)操作。

(3) 事务T中所有read_item(X)操作和write_item(X)操作完成之后，T必须发出unlock(X)操作[②]。

(4) 如果事务T已经持有一个数据项X的读（共享）锁或写（排他）锁，则T不会再发出read_lock(X)操作。随后我们将会讨论到，可以放宽这条规则。

(5) 如果事务T已经持有一个数据项X的读（共享）锁或者写（排他）锁，则T不会再发出write_lock(X)操作。随后我们就会讨论到，也可以放宽这条规则。

(6) 如果事务T没有持有数据项X上的共享（读）锁或者排他（写）锁，则T不会发出unlock(X)操作。

3．锁变换

有时为了允许**锁变换**（lock conversion），要放宽以上所列的条件4和条件5。在特定的条件下，允许已经对数据项X持有锁的事务把锁从一种锁定状态**变换**（convert）到另一种状态。例如事务T，它可能先发出read_lock(X)，稍后发出write_lock(X)操作来**升级**锁。如果T发出write_lock(X)操作时，它是持有X的读锁的唯一事务，则可以升级锁；否则，T必须等待。也可能T先发出write_lock(X)操作，稍后又发出read_lock(X)操作来**降级**锁。当锁升级和降级时，锁表在记录结构中必须为每个锁包含一个事务标识符（置于locking_transaction字段中），以存储对该数据项持有锁的事务的有关信息。所以必须适当地修改图2-2中对read_lock(X)和write_lock(X)操作的描述，我们将此作为练习留给读者完成。

正如前面所述，在事务中使用二进制锁或读/写锁并不能保证事务调度自身的可串行性。图2-3给出了一个示例，它遵守了前面的加锁规则，但是仍可能得到非可串行化的调度。这是因为在图2-3a中，T_1中的数据项Y和T_2中的数据项X解锁过早。这就可能导致图2-3c所示的调度出现，此调度不是可串行化的，因此会得出错误的结果。为了确保可串行性，必须遵循一个附加协议，这个附加协议与每个事务中的加锁和解锁操作的位置有关。在下一节中就将描述这个著名的两阶段加锁协议。

---

① 本节后面会提到，这些算法不允许锁升级或降级。读者可以扩展这些算法，以允许这些额外的操作。

② 可以放宽这条规则以允许事务先对数据项解锁，然后再加锁。

```
read_lock(X):

B: if LOCK(X) = "unlocked"
       then  begin LOCK(X) ← "read-locked";
             no_of_reads(X) ← 1
             end
   else if LOCK(X) = "read-locked"
      then no_of_reads(X) ← no_of_reads(X) + 1
   else  begin
         wait (until LOCK(X) = "unlocked"
            and the lock manager wakes up the transaction);
         go to B
         end;

write_lock(X):

B: if LOCK(X) = "unlocked"
      then LOCK(X) ← "write-locked"
   else  begin
         wait (until LOCK(X) = "unlocked"
            and the lock manager wakes up the transaction);
         go to B
         end;

unlock (X):

   if LOCK(X) = "write-locked"
      then  begin LOCK(X) ← "unlocked";
            wakeup one of the waiting transactions, if any
            end
   else it LOCK(X) = "read-locked"
      then  begin
            no_of_reads(X) ← no_of_reads(X) − 1;
            if no_of_reads(X) = 0
               then  begin LOCK(X) = "unlocked";
                     wakeup one of the waiting transactions, if any
                     end
            end;
```

图2-2　两种模式（读/写或共享/排他）锁的加锁和解锁操作

T_1	T_2

read_lock(Y);
read_item(Y);
unlock(Y);
write_lock(X);
read_item(X);
$X := X + Y$;
write_item(X);
unlock(X);
	read_lock(X);
	read_item(X);
	unlock(X);
	write_lock(Y);
	read_item(Y);
	$Y := X + Y$;
	write_item(Y);
	unlock(Y);

(a) 两个事务T_1和T_2

初始值：X=20，Y=30
串行调度T_1，T_2（T_2在T_1之后进行）的结果：X=50，Y=80
串行调度T_2，T_1（T_1在T_2之后进行）的结果：X=70，Y=50

(b) T_1和T_2可能的串行调度的结果

图2-3　不遵循两阶段加锁的事务

T_1	T_2
read_lock(Y);	
read_item(Y);	
unlock(Y);	
	read_lock(X);
	read_item(X);
	unlock(X);
	write_lock(Y);
	read_item(Y);
	$Y := X + Y$;
	write_item(Y);
	unlock(Y);
write_lock(X);	
read_item(X);	
$X := X + Y$;	
write_item(X);	
unlock(X);	

时间

调度S的结果：X=50，Y=50
（不可串行化）

(c) 使用锁的一个不可串行化调度S

图2-3（续）

2.1.2　通过两阶段加锁保证可串行性

如果一个事务中的所有加锁操作（read_lock，write_lock）都先于第一个解锁操作，则称该事务**遵循两阶段加锁协议**（two-phase locking protocol）[①]。此时一个事务可以分为两个阶段：**扩展**或**增长**阶段（第一阶段）和**收缩**阶段（第二阶段）。在第一阶段可以获得数据项的新锁，但是不能释放任何已持有的锁；在第二阶段可以释放已持有的锁，但是不能获得新锁。如果允许锁变换，那么锁升级（从读锁定到写锁定）必须在扩展阶段完成，锁降级（从写锁定到读锁定）必须在收缩阶段完成。因此，对已经持有X的写锁降级的read_lock(X)操作只能出现在收缩阶段。

图2-3a中的事务T_1和T_2未遵循两阶段加锁协议。这是因为，在T_1中，write_lock(X)操作是在unlock(Y)操作之后出现的；在T_2中，write_lock(Y)操作也是在unlock(X)操作之后出现。如果保证两阶段加锁，则两个事务可以重写为T_1'和T_2'，如图2-4中所示。现在，根据2.1.1节所示的加锁规则，对于T_1'和T_2'来说（基于其修改后的加锁和解锁操作顺序）图2-3c所示的调度是不允许的。这是因为，T_1'将在它对Y解锁之前发出write_lock(X)操作；因此，当T_2'发出其read_lock(X)操作时，它必须等待直到T_1'在调度中发出unlock(X)释放锁为止。

T_1'	T_2'
read_lock(Y);	read_lock(X);
read_item(Y);	read_item(X);
write_lock(X);	write_lock(Y);
unlock(Y)	unlock(X)
read_item(X);	read_item(Y);
$X := X + Y$;	$Y := X + Y$;
write_item(X);	write_item(Y);
unlock(X);	unlock(Y);

图2-4　事务T_1'和T_2'与图2-3中的事务T_1和T_2相同，不同的是它们遵守了两阶段加锁协议（注意，它们可能产生死锁）

① 这与分布式数据库（参见第9章）中恢复技术的两段提交协议无关。

可以证明，如果调度中的每个事务都遵循两阶段加锁协议，调度就可以保证是可串行化的，无需再测试调度的可串行性。通过保证两阶段加锁规则，加锁机制也可以保证可串行性。

两阶段加锁可能会限制调度中可以发生的并发量。因为如果事务T稍后必须锁定另一个数据项Y，那么它可能在使用完数据项X之后不能释放X；或者T在需要数据项Y之前就必须锁定Y，这样它才可以释放X。因此，在事务T所需读写的所有数据项被锁定之前，X必须保持由T锁定；然后，T才能释放X。同时，另一个希望访问X的事务可能会被迫等待，即使T已经使用完X。相反，如果Y在需要之前就已经被锁定，那么即使T尚未使用Y，另一个希望访问Y的事务也会被迫等待。这就是不检查调度本身而保证所有调度可串行性的代价。

基本、保守、严格、精确的两阶段加锁

两阶段加锁（2PL）有多个变种。以上所述的技术称为**基本2PL**。**保守2PL**或**静态2PL**要求一个事务在开始执行之前要**预声明**其读集合和写集合，以锁定它需要访问的所有数据项。回顾在1.1.2节中指出过的：事务的**读集合**（read-set）是该事务要读取的所有数据项的集合；**写集合**（write-set）是该事务要写的所有数据项的集合。如果所需的任意一个预声明数据项无法锁定，则事务对所有数据项都不能加锁，而是要等到可以为所有的数据项加锁为止。在2.1.3节讨论死锁问题时就可以看到，保守2PL是一个免死锁的协议。但由于保守2PL需要预先声明读集合和写集合，而这在大部分情况下是不可能的，所以它难以在实际中应用。

在实际中，使用最多的2PL变种是**严格2PL**，它保证了严格调度（见1.4节）。采用此变种，事务T直到提交或者异常终止之后才释放其排他（写）锁。因此除非T已经提交，否则任何其他事务都不能读或写T所写的数据项，从而产生了可恢复性的严格调度。严格2PL不是免死锁的。一个限制性更强的严格2PL是**精确2PL**，它也保证了严格调度。在精确2PL中，事务T直到提交或者异常终止之后才释放锁（排他或共享），所以比起严格2PL更容易实现。注意，保守2PL与精确2PL之间的区别是：前者必须在开始之前锁定所有的数据项，所以一旦事务开始它就处于收缩阶段；而后者直到终止（提交或者异常终止）后才释放锁，所以事务一直处于扩展阶段，直到结束。

在许多情况下，**并发控制子系统**（concurrency control subsystem）本身负责生成read_lock和write_lock请求。假设系统要执行严格2PL协议，无论什么时候事务T发出read_item(X)操作，系统都要代表T调用read_lock(X)操作。如果LOCK(X)的状态被其他某个事务T'写锁定，系统则把T置于数据项X的等待队列中；否则系统准予read_lock(X)的请求并允许执行T的read_item(X)操作。另一方面，如果事务T发出write_item(X)，系统则代表T调用write_lock(X)操作。如果LOCK(X)的状态是由另一个事务T'持有读锁或写锁，则系统会把T放入项X的等待队列中；如果LOCK(X)的状态是读锁定，并且T本身是唯一持有X读锁的事务，则系统将该锁升级为写锁定，并且允许执行T的write_item(X)操作；最后，如果LOCK(X)的状态是未锁定，则系统准予write_lock(X)请求，并允许执行write_item(X)操作。在每个动作之后，系统必须适当地更新锁表。

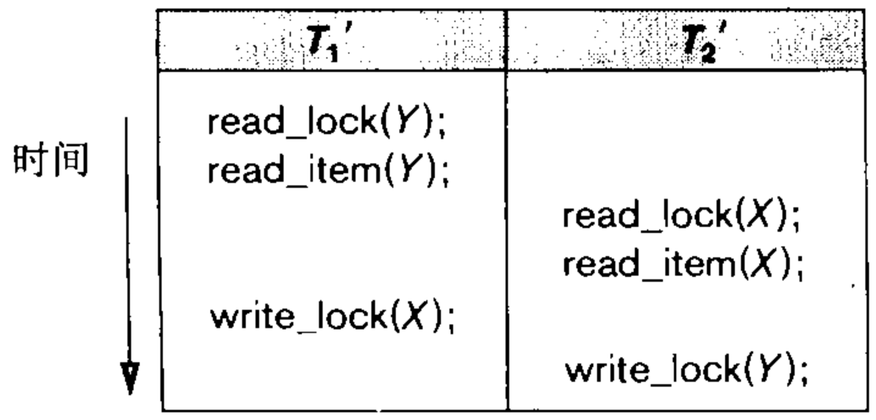
虽然两阶段加锁协议保证了可串行性（即所允许的每个调度都是可串行化的），但它并不一定允许所有可能的可串行化调度（即协议会禁止某些可串行化调度）。另外，使用锁会引起另外两个问题：死锁和饥饿。这两个问题及其解决方法将在下一节中讨论。

2.1.3　死锁和饥饿处理

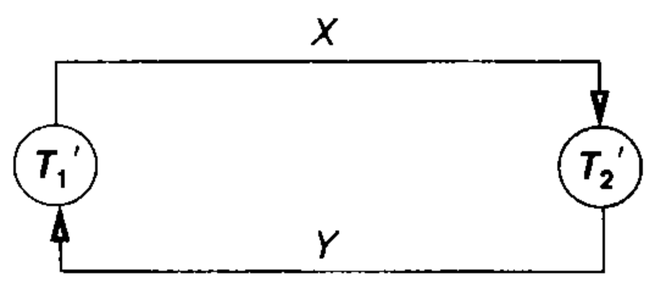
在包含两个或多个事务的一个集合中，当其中的每个事务T都在等待该集合中被其他事务T'锁定的数据项时，就会发生死锁（deadlock）。如此一来，集合中的每个事务都处于等待队列中，等待集合中另外一个事务释放对某个数据项的锁。图2-5a给出了一个简单的示例，其中事务T_1'与T_2'在一部分调度中就是死锁的；T_1'在X的等待队列中等待T_2'锁定的X，同时T_2'在Y的等待队列等待T_1'锁定的Y。此时，T_1'和T_2'以及其他任何事务都不能访问数据项X与Y。

1. 预防死锁协议

预防死锁的一种方法是采用**预防死锁协议**（deadlock prevention protocol）[①]。保守两阶段加锁协议中使用的预防死锁协议要求每个事务预先锁定它需要的所有数据项(这通常是一个不切实际的假定)，如果其中任意一个不能获得，则所有的数据项都不能锁定。如果这样，事务将等待，然后再次尝试锁定它需要的所有数据项。显然这种方案进一步限制了并发性。另一个协议要对数据库中的所有数据项进行排序，并确保需要多个数据项的事务会按照这个顺序锁定所需的项。此协议也限制了并发性。这要求程序员（或系统）清楚数据项的选择顺序，这在数据库环境中也是不切实际的。



(a) T_1'和T_2'的部分调度，处于死锁状态



(b) a中部分调度的等待图

图2-5　说明死锁问题

目前已经提出了多种其他的死锁预防模式，它们决定在可能的死锁情况下如何处理事务：应该阻塞事务并使其等待还是应该异常终止这个事务，或者该事务是否应抢占并异常终止另一个事务？这些技术使用了**事务时间戳**（transaction timestamp）TS(T)的概念，事务时间戳是分配给每个事务的唯一标识符。时间戳主要基于事务开始的顺序，因此，如果事务T_1在事务T_2之前开始，那么TS(T_1) < TS(T_2)。需要注意，较早开始的事务时间戳值较小。预防死锁有两种模式，分别称为等待-死亡和受伤-等待。假定事务T_i试图锁定数据项X，但是由于其他某个事务T_j已经使用冲突锁锁定了X，T_i无法对X加锁。这两种模式遵循的规则如下：

- **等待-死亡**（wait-die）：如果TS(T_i) < TS(T_j)，那么（T_i比T_j早）允许T_i等待；否则（T_i比T_j晚）异常终止T_i（T_i死亡），并在稍后使用相同的时间戳重新启动T_i。
- **受伤-等待**（wound-wait）：如果TS(T_i) < TS(T_j)，那么（T_i比T_j早）异常终止T_j（T_i伤害T_j），并在稍后使用相同的时间戳重新启动T_j；否则（T_i比T_j晚）允许T_i等待。

在等待-死亡中，允许较早的事务等待较晚的事务；但当较晚的事务请求访问较早事务所持有数据项时，该较晚事务会被异常终止并将被重启。受伤-等待的模式则相反：允许较晚的事务等待较早的事务；但当较早的事务请求访问较晚事务所持有数据项时，该较早事务会通过异常终止来抢占（preempt）较晚事务。两种模式都是异常终止两个事务中可能会卷入死锁的较晚事务。由此可见这两种技术都是避免死锁的，在等待-死亡中事务只等待较晚的事务，所以不会产生环（cycle）；在受伤-等待中，只等待较早的事务，所以也不会产生环。但这两种技术都可能造成不必要的异常终止和重启某些事务，即使这些事务实际上从不会引起死锁。

还有另外一组预防死锁协议不需要使用时间戳，这包括无等待和谨慎等待算法。在**无等待算法**中，如果事务不能获取锁，则立即异常终止该事务，并在一定的延迟时间之后重启，而不检查是否真会产生死锁。由于无等待算法可能会引起不必要的事务异常终止/重启，所以提出了**谨慎等待算法**。假定事务T_i试图锁定数据项X，却由于其他某个事务T_j使用冲突锁锁定了X使得T_i无法锁定X。谨慎等待规则如下：

---

① 这些协议在实际中不常用，或者是因为不切实际的假设，或者是因为可能的开销过大。死锁检测和超时（稍后介绍）更为实用。

- **谨慎等待**：如果T_j没有被阻塞（没有等待某个已加锁的数据项），那么阻塞T_i并允许T_i等待；否则异常终止T_i。

考虑每个阻塞事务T被阻塞的时间为$b(T)$。可以证明谨慎等待是避免死锁的。如果以上两个事务T_i和T_j都被阻塞，T_i正在等待T_j，那么$b(T_i) < b(T_j)$，这是因为T_i只能在T_j未阻塞时等待T_j。因此，阻塞时间对于所有被阻塞事务上构成了一个全序，所以不会出现引起死锁的环。

2．死锁检测

另一种更实用的死锁处理方法是**死锁检测**，这里，系统检测是否确实存在死锁状态。如果知道事务将很少有干扰，即不同的事务很少在同一时间访问同一个数据项，那么这种解决方案很具吸引力。如果事务很短并且每个事务只锁定为数不多的数据项，或事务负载很轻，就属于这种情况。另一方面，如果事务很长而且每个事务需要使用多个数据项，或事务的负载非常重时，采用预防死锁模式也许更好。

检测死锁状态的一种简单方法是为系统构造并维护一个**等待图**。在等待图中，要为当前执行的每个事务创建一个结点。只要事务T_i在等待一个数据项X，而X目前被事务T_j锁定，则在等待图中建立一条有向边（$T_i \rightarrow T_j$）。当T_j释放了T_i等待的数据项上的锁时，则在等待图中删除这条有向边。当且仅当等待图中有环时才存在死锁状态。这种方法的一个问题是要确定系统应该**何时**检查死锁。可以使用以下准则：如并发执行的事务的数量，或者多个事务等待锁定数据项的时间间隔。图2-5b展示了在图2-5a中（部分）调度的等待图。如果系统处于死锁状态，就必须异常终止一些导致死锁的事务。选择异常终止哪些事务称为**牺牲选择**。牺牲选择算法通常应该避免选择那些已经运行很长时间以及已经完成多个更新的事务，而应该尝试选择尚未对数据项做太多修改的事务。

3．超时

另一种处理死锁的简单方法是使用**超时**。由于其低开销和简单性，超时方法很实用。采用超时方法是：如果事务等待的时间超过了系统定义的超时时间，则系统会认为该事务可能死锁，并将它异常终止，而不管此时它是否确实存在死锁。

4．饥饿

使用加锁时可能产生的另一个问题是**饥饿**。当一个事务在不确定的时间段内不能继续执行，而系统中其他的事务却可以正常进行时，就会产生饥饿。如果对于锁定数据项的等待模式是不公平的，某些事务的优先级高于其他事务时，就可能产生这种情况。解决饥饿问题的一种方法是使用公平的等待模式，例如使用**先来先服务**队列，事务就能按照请求锁的顺序来为数据项加锁。另一种方法是允许某些事务拥有高于其他事务的优先级，但是要提高等待时间较长的事务的优先级，直到事务最终得到最高优先级并运行为止。如果算法重复地选择同一个事务作为牺牲者，也可能会由于牺牲选择而产生饥饿，这会导致该事务异常终止而且永远不会结束执行。对于多次被异常终止的事务，算法中可以使用较高的优先级，从而避免这个问题。前面讨论的等待-死亡和受伤-等待模式可以避免饥饿。

2.2　基于时间戳顺序的并发控制

使用锁并结合2PL协议，可以保证调度的可串行性。基于正在执行的事务获得的数据项的加锁顺序，由2PL产生的可串行化调度有与它们等价的串行调度。如果一个事务需要的数据项已被加锁，它可能被迫等待，直到这些数据项被释放为止。另一种保证可串行性的方法是：使用事务时间戳，从而为等价串行调度确定事务的执行顺序。我们将在2.2.1节讨论时间戳，以及讨论如何基于时间戳对事务进行排序，以保证可串行性。

2.2.1　时间戳

回顾一下，**时间戳**（timestamp）是DBMS所创建的用于标识事务的唯一标识。通常会按事务提交给系统的顺序来为时间戳赋值，所以可以认为时间戳是事务的起始时间。把事务T的时间戳记

作TS(*T*)。基于时间戳排序的并发控制技术不使用锁，所以不会产生死锁。

时间戳可以通过多种方式生成。一种可能的方式是使用计数器，每次把计数器的值赋给一个事务时，计数器的值就增加1。采用这种模式，事务的时间戳编号为1、2、3、…计算机的计数器有一个有限的最大值，因此当某个较短的时间段内没有事务执行时，系统必须周期性地重置计数器为0。实现时间戳的另一种方法是，使用系统时钟的当前日期/时间值，并且保证在时钟的同一步长内不会产生两个时间戳。

2.2.2　时间戳排序算法

这个模式的思想是：基于事务的时间戳对事务进行排序。因此，事务所参与的调度是可串行化的，等价的串行调度中事务按其时间戳值排序。这被称为**时间戳排序**（timestamp ordering，TO）。需要注意，时间戳排序不同于2PL，在2PL中调度的可串行化是通过等价于加锁协议允许的某个串行调度达到的。在时间戳排序中，调度则等价于与事务时间戳顺序相对应的特定的串行顺序。算法必须保证对于调度中冲突操作访问的各个数据项，访问数据项的顺序不能违反可串行性顺序。为了做到这一点，算法为每个数据项*X*关联了两个时间戳（TS）值：

(1) read_TS(*X*)：数据项*X*的**读时间戳**（read timestamp）。这是在成功读取数据项*X*的所有事务的时间戳中的最大时间戳，即read_TS(*X*) = TS(*T*)，其中*T*是成功读取*X*的最晚事务。

(2) write_TS(*X*)：数据项*X*的**写时间戳**（write timestamp）。这是在成功写入数据项*X*的所有事务的时间戳中的最大时间戳，即write_TS(*X*) = TS(*T*)，其中*T*是成功写入*X*的最晚事务。

1．基本时间戳排序（TO）

只要某个事务*T*试图发出read_item(*X*)或write_item(*X*)操作，**基本时间戳排序**算法就会对*T*的时间戳与read_TS(*X*)、write_TS(*X*)加以比较，以保证未违反事务执行的时间戳顺序。如果违反了这个顺序，那么异常终止事务*T*，并将*T*作为一个使用新的时间戳的新事务重新提交给系统。如果*T*被异常终止并被回滚，那么可能使用了*T*所写数据项值的任何事务T_1也必须被回滚；类似地，可能使用了T_1所写数据项值的任何事务T_2也必须被回滚，依此类推。其影响就是**级联回滚**，这是与基本TO相关的问题之一，因为不能保证所产生的调度是可恢复的。为了确保调度是可恢复、无级联的或严格的，必须执行另外一个协议。我们在此首先描述基本TO算法。在下面的两种情况下，并发控制算法必须检查冲突操作是否违反了时间戳顺序：

(1) 事务*T*发出write_item(*X*)操作：

a．如果read_TS(*X*) > TS(*T*)或者write_TS(*X*) > TS(*T*)，那么异常终止并回滚*T*，且拒绝该操作。这样做是因为时间戳大于TS(*T*)的某个较晚事务（相应的，在时间戳顺序中*T*后面的事务）在*T*得到机会写数据项*X*之前已经读或写了*X*的值，所以这违反了时间戳顺序。

b．如果没有发生（a）中的条件，那么执行*T*的write_item(*X*)操作，并将write_TS(*X*)的值置为TS(*T*)。

(2) 事务*T*发出read_item(*X*)操作：

a．如果write_TS(*X*) > TS(*T*)，那么异常终止并回滚*T*，且拒绝该操作。这样做是因为时间戳值大于TS(*T*)的某个较晚事务（相应地，时间戳顺序中*T*之后的事务）在*T*有机会读数据项*X*之前已经写了*X*的值。

b．如果write_TS(*X*) ≤ TS(*T*)，那么执行*T*的read_item(*X*)操作，并将read_TS(*X*)的值置为TS(*T*)与当前read_TS(*X*)中较大的值。

因此，只要基本TO算法检测出以错误顺序发生的两个冲突操作，它就会通过异常终止发出这两个操作中较晚操作的事务来拒绝该操作。由此可以保证基本TO产生的调度是冲突可串行化的，这与2PL协议类似。但是，在一个协议下允许的一些调度在另一个协议下可能是不允许的。因此，这两个协议都未能允许所有可能的可串行化调度。如前所述，采用时间戳顺序不会发生死锁。但如果不断地异常终止和重启事务则会发生循环重启（相应地带来饥饿）。

2．严格时间戳排序

基本TO的一个变种称为**严格TO**，它保证调度既是**严格**的（易于恢复）又是（冲突）可串行化的。采用这个变种，若事务T发出read_item(X)或write_item(X)操作，从而TS(T) > write_TS(X)，则T要延迟其读或写操作直到写X值的事务T'提交或异常终止（相应地，TS(T') = write_TS(X)）。为了实现这个算法，有必要对事务T'所写数据项X加锁，直到T'提交或者异常终止。该算法不会导致死锁，因为只有TS(T) > TS(T')时，T才会等待T'。

3．Thomas写规则

基本TO的一个修正算法称为**Thomas写规则**，它不保证冲突可串行性；但是通过如下修改对write_item(X)操作的检查，可以拒绝较少的写操作：

(1) 如果read_TS(X) > TS(T)，则异常终止并回滚T，拒绝该操作。

(2) 如果write_TS(X) > TS(T)，则不执行写操作，但继续执行T。这是因为，时间戳大于TS(T)的某个事务（相应地，时间戳顺序中排在T之后的事务）已经写了X的值，因此，我们必须忽略T的write_item(X)操作，因为它已经过期且过时了。注意，这种情况引起的任何冲突都会在情况1中检测到。

(3) 如果1、2中的条件都不发生，则执行T的write_item(X)操作，并置write_TS(X)为TS(T)。

2.3　多版本并发控制技术

并发控制的另外一些协议在更新数据项时保留了数据项的旧值（old value）。这些协议被称为**多版本并发控制**（multiversion concurrency control），因为它们维护了一个数据项的多个版本（值）。当事务需要访问一个数据项时，可以选择一个适当的版本来保持当前执行的调度的可串行性。其思想是：对于其他技术可能拒绝的某些读操作，在这里则允许通过读取数据项的较早版本以保持可串行性。当一个事务写数据项时会写一个新版本，并保留该数据项的旧版本。有些多版本并发控制算法使用了视图可串行性而不是冲突可串行性的概念。

多版本技术的一个显著缺点是：需要更大的存储空间来保存数据项的多个版本。不过，出于某些原因（如为了恢复），可能必须维护较早的版本。另外，一些数据库应用需要保存较早的版本来维护数据项值的发展历史。一个极端的例子就是时态数据库（temporal database）（参见第8章），它要跟踪所有的修改以及修改发生的时间。在这种情况下，多版本技术并没有额外的存储损失，因为已经保存了较早的版本。

目前已经提出了多种多版本并发控制模式。我们在此讨论其中的两种：一种基于时间戳排序，另一种基于2PL。

2.3.1　基于时间戳排序的多版本技术

采用这种方法，需要维护每个数据项X的多个版本X_1, X_2, …, X_k。对于每个版本，保留了版本X_i的值和以下两种时间戳：

(1) read_TS(X_i)：X_i的**读时间戳**，这是在成功读取版本X_i的所有事务的时间戳中最大的时间戳。

(2) write_TS(X_i)：X_i的**写时间戳**，这是写版本X_i值的事务的时间戳。

只要允许事务T执行write_item(X)操作，就会创建数据项X的一个新版本X_{k+1}，并且将write_TS(X_{k+1})和read_TS(X_{k+1})均设置为TS(T)。相应地，当允许事务T读版本X_i的值时，read_TS(X_i)的值设置为当前read_TS(X_i)和TS(T)中较大的那一个。

为了确保可串行性，可以采用如下两条规则：

(1) 如果事务T发出write_item(X)操作，X的版本i在X的所有版本中有最高的write_TS(X_i)，但仍然小于等于TS(T)，且read_TS(X_i) > TS(T)，则异常终止并回滚T；否则创建X的一个新版本X_j，其read_TS(X_j) = write_TS(X_j) = TS(T)。

(2) 如果事务T发出read_item(X)操作，则查找X所有版本中有最高的write_TS(X_i)的X版本i（小于等于TS(T)）；然后向事务T返回X_i的值，并且将read_TS(X_i)的值置为TS(T)和当前read_TS(X_i)中较大的那一个。

从规则2中可以看到，read_item(X)总会成功，因为它基于现有不同版本X的write_TS来查找要读取的正确版本X_i。但是在规则1中，事务T则可能被异常终止并被回滚。发生这种情况是因为如果事务T试图写的X版本应由时间戳为read_TS(X_i)的另一个事务T'读取，但是T'已经读取了版本X_i，X_i是由时间戳等于write_TS(X_i)的事务所写的。如果出现了这种冲突，则T被回滚；否则，会创建由事务T所写的X的一个新版本。注意，如果T被回滚，可能会发生级联回滚。因此为了保证可恢复性，在所有写了T所读的某版本的事务提交之前不允许事务T提交。

2.3.2　使用验证锁的多版本两阶段加锁

在多方式的加锁模式中，数据项有3种加锁方式：读、写和验证，而不只是前面讨论的两种方式（读、写）。因此数据项X的状态LOCK(X)可以是读锁定、写锁定、验证锁定或未锁定。在只有读锁和写锁的标准加锁模式（见2.1.1节）中，写锁是排他锁。我们可以使用图2-6a所示的**锁相容性表**（lock compatibility table）来描述标准模式中读锁与写锁的关系。是表示：如果事务T对数据项X持有列标题所指定类型的锁，而事务T'对同一数据项X请求了行标题所指定类型的锁，则T'可以获得其请求的锁，因为加锁方式是相容的。另一方面，否表示锁是不相容的，所以T'必须等待直到T释放锁。

在标准加锁模式中，一旦事务获得某数据项的写锁，则其他事务就不能访问该数据项。多版本2PL的基本思想是，当只有单个事务T持有X的写锁时，允许其他事务T'读该数据项。这可以通过允许每个数据项X有两个版本来实现：一个版本必须总是由某个已提交的事务写入，当事务T获得该数据项的写锁时，创建第二个版本X'。T持有写锁的同时，其他事务可以继续读X的已提交版本。事务T在需要时可以写X'的值，而不会影响已提交的X版本的值。但是一旦T准备提交，在可以提交之前必须获得它当前持有写锁的所有数据项的**验证锁**。验证锁和读锁不相容，所以该事务可能必须延迟提交，直到其持有写锁定的所有数据项由某些读事务释放从而获得验证锁为止。一旦获得了验证锁，验证锁为排他锁，则把该数据项的已提交版本X设置为版本X'的值，版本X'被丢弃，然后释放验证锁。这种模式的锁的相容性表如图2-6b所示。

	读	写
读	是	否
写	否	否

(a) 读/写加锁模式的相容性表

	读	写	验证
读	是	是	否
写	是	否	否
验证	否	否	否

(b) 读/写/验证加锁模式的相容性表

图2-6　锁相容性表

在这个多版本2PL模式中，读操作能够与一个写操作并发进行，这在标准2PL模式中是不允许的。其代价是，事务可能不得不延迟提交，直到获得其更新的所有数据项的排他验证锁为止。可以证明这个模式能够避免级联异常终止，因为事务只能读取由已提交事务所写的版本X。如果允许把读锁升级为写锁则可能会出现死锁，这必须由该技术的变种来处理，该变种在2.1.3节中已被讨论。

2.4　确认（乐观的）并发控制技术

在迄今为止所讨论的并发控制技术中，执行数据库操作之前都进行了一定程度的检查。如在加锁

技术中，需要检查被访问的数据项是否已经被锁定；在时间戳排序中，要根据数据项的读/写时间戳检查事务时间戳。在事务执行过程中，这些检查都会带来开销，而其影响就是降低事务处理的速度。

乐观并发控制技术，也被称为**确认技术**或**验证技术**。采用这种技术，事务执行时不需要进行检查。目前提出的许多并发控制方法中都使用了确认技术，在此只介绍其中的一种。在这种模式中，事务中的更新并不直接应用于数据库项，直到事务结束为止。事务执行过程中，所有的更新都应用于为事务保留的数据项*局部副本*上[①]。事务执行结束后，**确认阶段**会检查事务的更新是否违反了可串行性。系统必须保留确认阶段需要的特定信息。如果没有违反可串行性则提交事务，便以局部副本更新数据库；否则中止该事务并在稍后重启。

这种并发控制协议有3个阶段：

(1) **读阶段**（read phase）：事务可以从数据库中读取已提交的数据项的值。但是，更新只应用于保存在事务工作区的数据项的局部副本（版本）。

(2) **确认阶段**（validation phase）：执行检查，以确保事务更新应用到数据库时不会违反可串行性。

(3) **写阶段**（write phase）：如果确认阶段成功，则事务更新被应用到数据库；否则，丢弃更新并重启事务。

乐观并发控制的基本思想是：一次完成所有的检查。因此，事务执行过程能以最低开销进行，直到确认阶段。如果在事务之间很少有干扰，大多数事务都将成功确认。但是，如果事务之间干扰较多的话，许多事务执行到最后将丢弃其结果，并且稍后必须重启。在这种情况下，乐观技术不是行之有效的方法。之所以称为乐观技术，原因就在于假设事务之间很少有干扰，因此事务执行过程中无需进行检查。

描述乐观协议使用了事务时间戳，还要求系统保存事务的write_sets和read_sets。另外，需要为每个事务保留3个阶段中每个阶段的开始和结束时间。事务的write_sets是事务所写的数据项的集合；read_sets是事务所读数据项的集合。在事务T_i的确认阶段，协议确认T_i未与任何已提交的事务产生干扰，也未干扰其他正处于确认阶段的事务。T_i的确认阶段要检查对于每个已提交或者已经处于确认阶段的事务T_j，以下的条件之一是否成立：

(1) 在事务T_i开始其读阶段之前，事务T_j已完成其写阶段。

(2) 在事务T_j完成其写阶段之后，事务T_i开始其写阶段。并且T_i的read_sets与T_j的write_sets没有共同的数据项。

(3) T_i的read_sets和write_sets与T_j的write_sets均无共同的数据项，而且在T_i完成其读阶段之前T_j已完成其读阶段。

由于1是最容易检查的条件，所以在确认事务T_i时，首先要对每个事务T_j检查第一个条件。仅当条件1不成立时检查条件2，仅当条件2不成立时检查条件3，条件3计算起来最为复杂。如果上述3个条件中任一个成立就认为没有干扰，T_i成功确认。如果3个条件均不满足，则事务T_i确认失败，异常终止事务T_i并稍后重启，因为可能发生了干扰。

2.5　数据项粒度和多粒度加锁

所有的并发控制技术都假设数据库由大量命名数据项组成。数据库项可以选择如下之一：

- 一个数据库记录。
- 一个数据库记录的一个字段值。
- 一个磁盘块。
- 整个文件。

① 注意，可以看作是保存数据项的多个版本。

- 整个数据库。

粒度会影响到并发控制和恢复的性能。在2.5.1节中，将讨论关于选择加锁粒度级别的一些权衡；在2.5.2节中，将讨论多粒度加锁模式，其中粒度级别（数据项的大小）是可以动态改变的。

2.5.1　加锁的粒度级别考虑

数据项的大小通常称为**数据项粒度**（data item granularity）。细粒度（fine granularity）指数据项较小，粗粒度（coarse granularity）指数据项较大。在选择数据项的大小时必须权衡考虑。我们将讨论加锁环境中的数据项大小问题，虽然类似的论点也同样适用于其他并发控制技术。

首先需要注意的是：数据项越大，允许的并发程度就越低。假设数据项大小是一个磁盘块，事务T要对记录B加锁时，就必须锁定包含B的整个磁盘块X，因为锁是与整个数据项（磁盘块）相关联的。现在，如果另一个事务S需要为另一个不同的记录C加锁，而C恰好也处于冲突锁方式的同一个磁盘块X中，那么事务S被迫等待。如果数据项大小是一条记录，事务S则可以进行，因为它将对不同的数据项（记录）加锁。

另一方面，数据项越小数据库中的数据项就越多。因为每个数据项都与一个锁关联，系统中的锁管理器就要处理更多数量的活动锁，执行更多加锁和解锁的操作，从而导致开销增大。另外，也需要更大的空间来存储锁表。对于时间戳，每个数据项都要存储read_TS和write_TS，而且处理大量的数据项也存在类似的开销。

思考以上情况，就会提出一个显然的问题：数据项粒度多大最为适宜？答案是：这要取决于相关事务的类型。如果典型事务只是访问几条记录，那么数据项粒度为一条记录比较合适；如果事务通常要访问一个文件中的多条记录，那么以块或文件作为粒度更为合适，这样事务会把这些记录考虑为一个（或几个）数据项。

2.5.2　多粒度级别加锁

既然最佳的粒度大小取决于给定的事务，那么数据库系统支持多级别的粒度看上去就很合适，其中不同事务的粒度级别也可以不同。图2-7给出了一个包含两个文件的数据库的简单粒度层次结构，其中每个文件包含多个页，每页包括多个记录。这个图可以用来说明**多粒度级别**（multiple granularity level）2PL协议，其中可以以任意粒度级别请求加锁。但是，要高效地支持此协议，还需要使用其他类型的锁。

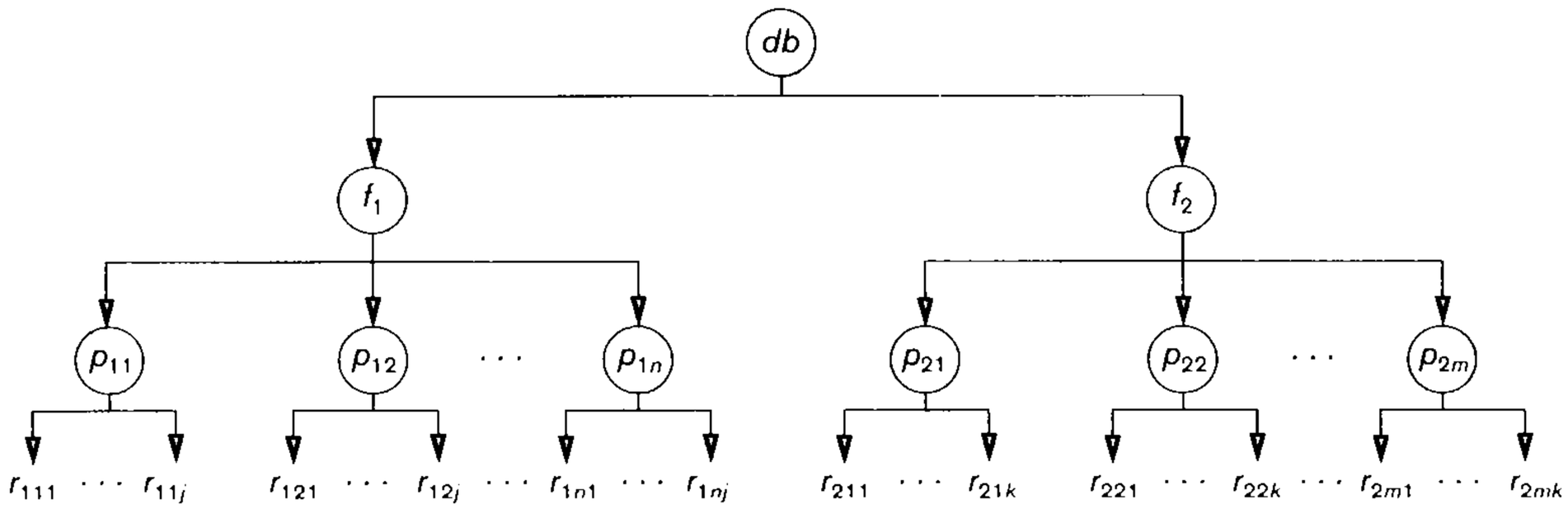


图2-7　说明多粒度级别加锁的粒度层次结构

考虑以下场景（在此引用了图2-7的示例）其中只有共享锁和排他锁。假定事务T_1要更新文件f_1中的所有记录，T_1请求f_1上的一个排他锁，并获得批准。这样f_1的所有页（从p_{11}到p_{1n}）以及包含在这些页中的所有记录都以排他方式被锁定。这对T_1来说是有益的，因为相对于设置n个页锁或者必须逐记录加锁来说，设置一个文件锁更为高效。现在假定另一个事务T_2只需要从文件f_1的p_{1n}页中读记录

r_{1nj}，那么T_2将请求r_{1nj}上的一个共享记录锁。但是数据库系统（即事务管理器，或者更确切地说是锁管理器）必须验证请求的锁与已持有锁的相容性。进行验证的一个途径就是从叶结点r_{1nj}到p_{1n}到文件f_1再到数据库db遍历整个树。如果某一时刻这些数据项上持有冲突锁，则拒绝对r_{1nj}的加锁请求，T_2被阻塞并且必须等待。这种遍历相当有效。

但是，如果事务T_2的请求在事务T_1的请求之前到达，那会怎么样？在这种情况下，对r_{1nj}的共享记录锁授予T_2，但是当请求T_1的文件锁时，锁管理器将很难对结点f_1的所有子结点（页和记录）进行检查以找出锁冲突。其效率会非常低，而且会违背多粒度级别加锁的初衷。

为了使多粒度级别加锁真正实用，还需要采用其他类型的锁，称为**意想锁**（intention lock）。意想锁的思想是：对于一个事务，沿着从根结点到目标结点的路径，指出该结点的某个子结点需要得到什么类型的锁（共享或排他）。有3种类型的意想锁：

(1) 意想共享锁（IS），表示请求某个（些）子结点上的共享锁。

(2) 意想排他锁（IX），表示请求某个（些）子结点上的排他锁。

(3) 共享意想排他锁（SIX），表示以共享模式锁定了当前结点，但是要请求某个（些）子结点上的排他锁。

这3种意想锁以及共享/排他锁的相容性表如图2-8所示。除了引入这3种意想锁外，还必须使用适当的加锁协议。**多粒度加锁**（multiple granularity locking，MGL）协议包括如下规则：

(1) 必须遵循锁相容性（基于图2-8）。

(2) 必须首先以某种方式锁定树的根结点。

(3) 仅当结点N的父结点已经被事务T以IS或IX方式锁定时，结点N才能被事务T以S或IS方式锁定。

(4) 仅当结点N的父结点已经被事务T以IX或SIX方式锁定时，结点N才能被事务T以X、IX或SIX方式锁定。

(5) 仅当事务T未对任何结点解锁时（以保证2PL协议），该事务才能对一个结点加锁。

(6) 仅当结点N的所有子结点当前都没有被事务T锁定时，T才能对N解锁。

	IS	IX	S	SIX	X
IS	是	是	是	是	否
IX	是	是	否	否	否
S	是	否	是	否	否
SIX	是	否	否	否	否
X	否	否	否	否	否

图2-8　多粒度加锁的锁相容性矩阵

规则1只是指出了不允许冲突锁。规则2、规则3、规则4指出事务以某种加锁方式锁定一个给定结点的条件。MGL协议的规则5和规则6保证了2PL规则，以产生可串行化调度。为了说明MGL协议，使用图2-7中的数据库层次结构，考虑如下3个事务：

(1) T_1要更新记录r_{111}和记录r_{211}。

(2) T_2要更新页p_{12}上的所有记录。

(3) T_3要读取记录r_{11j}和整个f_2文件。

图2-9显示了这3个事务的1种可能的可串行化调度。这里只给出了其中的加锁解锁操作。

<lock_type>(<item>)用以显示调度中的加锁操作。

多粒度级别协议特别适合处理包含以下事务的混合事务：(1)只访问少数的数据项（记录或字段）的短事务，(2)访问整个文件的长事务。在这种环境中，与单粒度加锁方法相比这种协议产生的事务阻塞和加锁开销都较少。

T_1	T_2	T_3
IX(*db*)		
IX(f_1)		
	IX(*db*)	
		IS(*db*)
		IS(f_1)
		IS(p_{11})
IX(p_{11})		
X(r_{111})		
	IX(f_1)	
	X(p_{12})	
		S(r_{11j})
IX(f_2)		
IX(p_{21})		
X(p_{211})		
unlock(r_{211})		
unlock(p_{21})		
unlock(f_2)		
		S(f_2)
	unlock(p_{12})	
	unlock(f_1)	
	unlock(*db*)	
unlock(r_{111})		
unlock(p_{11})		
unlock(f_1)		
unlock(*db*)		
		unlock(r_{11j})
		unlock(p_{11})
		unlock(f_1)
		unlock(f_2)
		unlock(*db*)

图2-9　说明可串行化调度的加锁操作

2.6　索引中使用加锁进行并发控制

两阶段加锁也可应用于索引（见《数据库系统基础：初级篇》的第14章），其中索引结点对应于磁盘页。但如果一直持有索引页的锁直到2PL的收缩阶段，可能导致过多的事务阻塞。这是因为索引的搜索总是从根结点开始，如果一个事务要插入一个记录（写操作），就会以排他方式锁定根结点，因此有关这个索引的所有其他冲突锁请求都必须等待，直到该事务进入收缩阶段。这会阻塞其他所有事务访问索引，所以在实际中必须使用其他的方法来为索引加锁。

在开发并发控制模式时，可以利用索引的树结构优点。例如，执行索引搜索（读操作）时，要从根到叶遍历树中的路径。一旦访问了路径中较低层次的结点，就不再使用该路径中较高层次的结点。所以一旦获得了子结点上的排他锁，父结点上的锁就可以释放。其次，对一个叶结点应用插入操作时（即插入一个码和一个指针），必须以排他方式锁定特定的叶结点。但是，如果结点未满，此插入操作不会引起较高层次索引结点的变化，就不需要以排他方式加以锁定。

一个保守的插入方法是以排他方式锁定根结点，然后访问该根结点的适当的子结点。如果此子结点未满，那么可以释放根结点上的锁。此方法可以在树中一直应用到叶结点，从根到叶通常有3层或4层。尽管会持有排他锁，但很快就会将其释放。另一种更加**乐观的方法**是：对于通向叶结点的结点，请求并持有其共享锁，同时持有叶结点的排他锁。如果插入引起叶结点分裂，插入操作将会传播到较高层次的结点，这样较高层结点上的锁可以升级为排他锁。

索引加锁的另一个方法是使用B^+树的变种，称为**B链接树**（B-link tree）。在B链接树中的每一层次上，同层的兄弟结点被链接在一起。这样在请求页时就可以使用共享锁，并要求在访问子结点前释放锁。对于插入操作，一个结点的共享锁可能会升级为排他锁。如果出现分裂，必须以排他方式重新对父结点加锁。这里的一个难点是搜索操作与更新操作并发执行。假定一个并发的更新操作与搜索操作使用同一条路径，并在叶结点中插入一个新的条目，同时假定这个插入操作导致了该叶结点的分裂。当执行插入时，搜索过程继续，即沿着指针到达所需的叶结点，却发现所查找的码不复存在。因为拆分已经把这个码移到了一个新的叶结点上，这将是原叶结点的*右兄弟*。不过，如果搜索过程沿着原叶结点的指针（链接）到达其右兄弟结点（所需的码移至此结点），搜索过程仍能成功。

删除情况的处理也是B链接树并发协议的一部分，其中将会合并索引树的两个或多个结点。在这种情况下，要持有待合并结点上的锁，还要持有待合并的两个结点的父结点的锁。

2.7　并发控制的其他问题

本节将讨论与并发控制相关的其他一些问题。在2.7.1节中，将讨论与插入、删除记录相关的问题，以及所谓的幻像问题，幻像问题可能在插入记录时发生。这个问题在1.6节中作为需要并发控制度量的一个潜在问题已经有所介绍。随后，在2.7.2节中，将讨论以下情况中可能出现的一些问题：事务在提交前向监视器输出一些数据，稍后该事务被异常终止。

2.7.1　插入、删除以及幻像记录

在数据库中**插入**一个新数据项时，很显然在该数据项被创建并完成该插入操作之前，不可以访问该数据项。在加锁环境中，可以创建该数据项的锁并设置为排他（写）方式，基于所用的并发控制协议，可以在释放其他写锁的同时释放这个锁。对于基于时间戳的协议，新数据项的读时间戳与写时间戳可以设置为创建该数据项的事务的时间戳。

接下来，考虑删除已存在数据项的**删除操作**。对于加锁协议，在事务删除数据项之前也必须获得排他（写）锁。对于时间戳排序协议来说，在允许删除数据项之前，协议必须确保较晚的事务不能已经读过或写过该数据项。

当事务T插入的一个新记录满足另一个事务T'所要访问的记录集的限制条件时，就可能产生**幻像问题**（phantom problem）。如假定事务T正在插入一个Dno = 5的EMPLOYEE新记录，同时事务T'正在访问所有Dno = 5的EMPLOYEE记录（比如要对部门号为5的部门汇总其员工的全部Salary值来计算人事预算）。如果等价的串行顺序是T，T'，那么T'必须读取新的EMPLOYEE记录，并在总计中包含其Salary。对于等价的串行顺序T'，T，新工资则不会包括在内。注意，虽然事务在逻辑上冲突，但在后一种情况下两个事务之间确实没有共同的记录（数据项），因为T'可能在T插入新记录之前已经锁定了所有Dno = 5的记录。这是因为引起冲突的记录是**幻像记录**（phantom record），即在插入时突然出现在数据库中的记录。如果两个事务中的其他操作有冲突，则并发控制协议可能无法识别出由幻像记录引起的冲突。

幻像记录问题的一种解决方法是使用**索引加锁**，如2.6节所讨论的那样。《数据库系统基础：初级篇》第14章曾经指出：索引包括条目，而条目中带有属性值以及一组指针，这些指针指向文件中具

有该值的所有记录。如EMPLOYEE的Dno上的索引对于每个不同的Dno值会包括一个条目，并带有一组指针，这些指针指向具有该值的所有EMPLOYEE记录。如果在访问记录本身之前锁定了索引条目，就可以检测到幻像记录上的冲突。这是因为在对实际的记录加锁之前，事务*T'*会请求Dno = 5的索引条目的读锁，而*T*会请求同一个索引条目的写锁。因为索引锁冲突，就可以检测到幻像冲突。

一种更常用的技术称为**谓词加锁**（predicate locking），它以类似的方式对满足一个任意谓词（条件）的所有记录的访问加以锁定。但是已经证明，谓词加锁难以高效实现。

2.7.2　交互式事务

交互式事务在提交之前，从一个交互设备（比如一个监视器屏幕）读入和向该设备写出时，就会出现另一个问题。即用户可能向事务*T*输入一个数据项的值，而该值可能要基于事务*T'*写到屏幕上的某个值。但*T'*可能还没有提交该值。*T*与*T'*之间的这种依赖性无法由系统并发控制方法模拟，它仅基于用户与两个事务的交互。

处理这个问题的一个方法是延期事务到屏幕的输出，直到事务都被提交为止。

2.7.3　锁存器

短时间内持有的锁通常被称为**锁存器**（latch）。锁存器不遵循一般的并发控制协议（如两阶段加锁协议）。例如，当把页从缓冲区写到磁盘上时，可以用锁存器来保证该页的物理完整性。应该为页获得一个锁存器，将页写到磁盘，然后再释放锁存器。

小结

本章介绍了DBMS并发控制技术。首先讨论了基于锁的协议，这是迄今为止在实际中最常用的技术。我们描述了两阶段加锁（2PL）协议以及它的若干变种：基本2PL、严格2PL、保守2PL和精确2PL。严格2PL和精确2PL更常用一些，因为它们具有更好的可恢复性。随后介绍了共享（读）锁和排他（写）锁的概念，并展示了当结合两阶段加锁规则时加锁如何保证可串行性。还给出了处理死锁问题的几种技术，在使用加锁时可能会出现死锁。在实际中，通常使用的方法是超时和死锁检测（等待图）。

本章介绍了其他的并发控制协议，这些协议在实际中并不常用，但是对于解决这个问题它们具有重要的理论意义。这些协议包括时间戳排序协议，它基于事务的时间戳顺序来确保可串行性。时间戳是系统产生的唯一的事务标识符。其中还简述了Thomas写规则，它可以提高性能，但不能保证冲突可串行性。随后给出了严格的时间戳排序协议。本章还讨论了两种多版本协议，这里假设数据库中保留了数据项的旧版本。一种技术称为多版本两阶段加锁（它在实际中已经被使用），假设每个数据项都存在两个版本，并试图使读锁和写锁相容（其代价是引入额外的验证锁方式）来增加并发性。这里还提出了一个基于时间戳排序的多版本协议，并给出了一个乐观协议的示例，乐观协议也称为验证协议或确认协议。

随后，我们把注意力转向数据项粒度这个很重要的实际问题上来。介绍了多粒度加锁协议，它允许基于当前的混合事务来改变粒度（数据项大小），其目的是提高并发控制的性能。然后提出了一个重要的实际问题，这就是开发索引加锁协议，使得索引不会成为并发访问的障碍。最后，介绍了幻像问题和交互式事务问题，并且简要描述了锁存器的概念以及它与锁有何不同。

下一章将概要介绍恢复技术。

复习题

2.1　什么是两阶段加锁协议？它如何保证可串行性？

2.2　两阶段加锁协议有哪些变种？为什么通常倾向于选择严格或精确两阶段加锁协议？
2.3　讨论死锁和饥饿问题，以及处理它们的不同方法。
2.4　比较二进制锁和排他/共享锁。为什么后者更可取？
2.5　描述防死锁的等待-死亡协议和受伤-等待协议。
2.6　描述防死锁的谨慎等待、无等待和超时协议。
2.7　什么是时间戳？系统如何产生时间戳？
2.8　讨论并发控制的时间戳排序协议。严格时间戳排序和基本时间戳排序有哪些不同？
2.9　讨论并发控制的两种多版本技术。
2.10　什么是验证锁？使用验证锁有哪些优点和缺点？
2.11　乐观并发控制技术和其他的并发控制技术有什么不同？为什么也被称为确认或者验证技术？讨论乐观并发控制方法的典型阶段。
2.12　数据项粒度怎样影响并发控制的性能？哪些因素会影响数据项粒度大小的选择？
2.13　插入和删除操作需要什么类型的锁？
2.14　什么是多粒度加锁？在什么情况下使用多粒度加锁？
2.15　什么是意想锁？
2.16　什么时候使用锁存器？
2.17　什么是幻像记录？讨论幻像记录引起的并发控制问题。
2.18　索引加锁如何解决幻像问题？
2.19　什么是谓词锁？

练习题

2.20　证明：基本两阶段加锁协议能够保证调度的冲突可串行性。（提示：说明如果调度的可串行性图中有环，那么该调度中至少有一个事务没有遵守两阶段加锁协议。）
2.21　修改对于多方式锁的数据结构以及read_lock(*X*)、write_lock(*X*)和unlock(*X*)的算法，从而允许对锁升级或降级。（提示：若存在持有锁的事务，锁需要检查持有该锁的事务id。）
2.22　证明：严格两阶段加锁可以保证严格调度。
2.23　证明：等待-死亡协议和受伤-等待协议能够避免死锁和饥饿。
2.24　证明：谨慎等待可以避免死锁。
2.25　将时间戳排序算法应用于图1-8b和图1-8c中的调度，确定该算法是否允许这两个调度执行。
2.26　使用多版本时间戳排序方法重做练习题2.25。
2.27　对于诸如B^+树的索引，为什么不能使用两阶段加锁作为其并发控制方法？
2.28　图2-8的相容性表说明了IS和IX锁是相容的。请解释其原因。
2.29　MGL协议规定，仅当事务*T*未锁定结点*N*的任何子结点时，*T*才能对*N*解锁。如果没有这个条件，MGL协议将是错误的。请对此加以说明。

参考文献

两阶段加锁协议以及谓词锁的概念首先由（Eswaran et al.，1976）提出。（Bernstein et al.，1987）、（Gray & Reuter，1993）以及（Papadimitriou，1986）重点讨论了并发控制和恢复问题。而（Kumar，1996）则集中讨论了并发控制方法的性能。（Gray et al.，1975）、（Lien & Weinberger，1978）、（Kedem & Silbershatz，1980）以及（Korth，1983）中都讨论了加锁。（Holt，1972）对死锁和等待图进行了形式化的描述，（Rosenkrantz et al.，1978）提出了等待-受伤和受伤-死亡模式。（Hsu et al.，1992）讨论了谨慎等待。（Helal et al.，1993）对各种加锁方法进行了比较。（Bernstein & Goodman，1980）以及（Reed，1983）讨论了基于时间戳的并发控制技术。（Kung & Robinson，1981）以及（Bassiouni，1988）讨论了乐观并发控制技术。（Papadimitriou & Kanellakis，1979）以及（Bernstein & Goodman，1983）讨论了多版本技术。（Reed，1978，1983）提出

了多版本时间戳排序，（Lai & Wilkinson，1984）讨论了多版本两阶段加锁。（Gray et al.，1975）提出了多粒度加锁的一种方法，（Ries & Stonebraker，1977）分析了加锁粒度的影响。（Bhargava & Reidl，1988）提出了一种在不同并发控制和恢复方法中动态选择的方法。（Lehman & Yao，1981）以及（Shasha & Goodman，1988）提出了对于索引并发控制方法。（Srinivasan & Carey，1991）给出了各种B^+树并发控制算法的性能研究。

其他有关并发控制的近期研究包括：基于语义的并发控制（Badrinath & Ramamritham，1992）、长时间运行活动的事务模型（Dayal et al.，1991）以及多层事务管理（Hasse & Weikum，1991）。

第3章

数据库恢复技术

本章将讨论一些用于数据库从故障中恢复的技术。在1.1.4节中已经讨论过导致系统故障的多种原因，例如系统崩溃和事务错误。1.2节还阐述了恢复过程涉及的一些概念，例如系统日志和提交点。

3.1节首先将概略介绍典型的恢复过程以及恢复算法的分类，然后讨论一些关于恢复的概念，包括预写日志，原位更新与镜像更新，以及回滚（取消）未完成事务或故障事务的影响。3.2节将介绍基于延迟更新的恢复技术，也称为NO-UNDO/REDO技术。3.3节讨论基于即时更新的恢复技术，包括取消/重做（UNDO/REDO）算法和取消/不重做（UNDO/NO-REDO）算法。3.4节将介绍称为镜像或镜像分页的技术，可以将其归类于不取消 / 不重做（NO-UNDO/NO-REDO）算法。3.5节将介绍实际DBMS恢复模式的一个实例，称为ARIES。在3.6节将简要介绍多数据库的恢复。最后在3.7节讨论灾难性故障的恢复技术。

本章的重点在于概念性地描述几种不同的恢复方法。至于特定系统的恢复特性，读者可以阅读“参考文献”和相应系统的用户手册。恢复技术通常和并发控制机制结合在一起。特定的恢复技术最好与某种特定的并发控制方法一起使用。在本章中，将尽量独立于并发控制机制来讨论恢复的概念，但是也将讨论在何种情况下特定的恢复机制与哪一种特定并发控制协议一起使用最好。

3.1 恢复概念

3.1.1 恢复概述和恢复算法的分类

从事务故障中恢复，通常意味着要把数据库恢复至故障前最近的一致性状态。为此，系统必须维护关于事务更新的数据项信息。在1.2.2节讨论过，这些信息通常保存在**系统日志**（system log）中。典型的恢复策略可以简要地概括如下：

(1) 如果灾难性故障（如磁盘崩溃）导致大范围数据库遭到严重破坏，必须使用归档存储设备（通常是磁带）上的数据库备份进行恢复，并从备份日志重新应用或者重做已提交事务的操作来重构故障前数据库的最新状态。

(2) 如果数据库没有遭到物理破坏，只是由于1.1.4节阐述的1～4种非灾难性故障破坏了数据库的一致性，该策略通过逆向地取消某些操作来还原导致不一致性的任何修改。在后面的示例中我们还将阐述，为使数据库恢复到某个一致性状态，可能还需要重做某些操作。在这种情况下不需要对数据库进行完整的归档备份，但在恢复时需要用到联机系统日志。

从概念上说，非灾难性事务故障的恢复技术主要有两种：延迟更新和即时更新。**延迟更新**（deferred update）技术是在事务到达提交点后才真正更新磁盘上的数据库。在到达提交点前，所有事务的更新都记录在局部事务的工作区（或缓冲区）当中。提交时，更新首先永久记录在日志中，然后才被写入数据库。如果事务在提交点前发生故障，故障事务将不会以任何方式对数据库进行改变，所以在恢复时无需使用UNDO。但可能要使用REDO重做日志中已提交事务的某些操作，因为相应的改变可能还没有记录到磁盘。因此，延迟更新也被称作NO-UNDO/REDO**算法**。我们将在3.2节对此进行讨论。

在**即时更新**（immediate update）技术中，事务的某些操作可能在事务到达提交点前被写入数据

库。但通过强制写日志，通常这些操作的更新在应用于数据库之前已被写入磁盘上的日志中，这就为故障的恢复提供了可能性。如果事务在数据库更新后但未到达提交点前发生故障，则必须对该事务执行取消操作，即回滚该事务。在常见的即时更新情况下，恢复时可能同时用到取消（undo）和重做（redo）。这种UNDO/REDO算法在实际的使用中是最为广泛的。在该算法的变种UNDO/NO-REDO算法中，事务被提交之前所有更新已记录到数据库，因此恢复时只需执行取消操作。我们将在3.3节对此进行进一步讨论。

3.1.2　磁盘块的高速缓存（缓冲）

通常恢复处理过程和操作系统的功能是紧密相关的，尤其是主存中磁盘页的缓冲和高速缓存。通常把要更新的一个或多个包含数据项的磁盘数据页高速缓存到内存缓冲区中，在内存中对数据进行更新，然后再写回磁盘。对磁盘页面进行高速缓存通常是由操作系统来完成的，但因为它对恢复效率有着重要的影响，所以DBMS通过调用底层操作系统例程对其进行管理。

总的来说按照数据库磁盘页（块）来考虑恢复是方便的。通常内存中缓冲区的汇集，被称为**DBMS高速缓存**，由DBMS进行控制，以便保持这些缓冲区。通过使用缓存**目录**来跟踪哪些数据库项在缓冲区中[①]。缓存目录的记录格式形如<Disk_page_address，Buffer_location>的项。当DBMS对某个数据项发出请求时，首先检查缓存目录以确定该数据项所处页面是否在高速缓存内。如果不在，就在磁盘上对其进行定位，然后将相应页面读入缓存中。有时为给新的数据项提供空间，可能需要**替换**或**刷新**高速缓冲区中的某些页。在选择被替换的页面时，可以使用操作系统提供的一些淘汰策略，如最近最少使用策略（LRU）或先进先出策略（FIFO）。

高速缓存中的每个缓冲区都与一个**脏位**（*dirty bit*）相关联，它可能包含在目录项中用来指示该缓冲区是否有所修改。当一个页面首次从磁盘读入高速缓存的缓冲区时，使用新的磁盘页地址更新缓存目录，同时脏位被置为0。一旦该缓冲区被修改，相应目录项的脏位就被置为1。当高速缓存的缓冲区中的内容被替换（刷新）时，必须首先将脏位为1的缓冲区内容写回到相应的磁盘。另外还需要一位，称为**钉住-拔去**（pin-unpin）位，即如果缓冲中的页到目前为止还不能写回到磁盘，则称相应的页被**钉住**（该位的值置为1）。

将修改过的缓冲区刷新到磁盘时，通常使用的策略有两种。第一种称为**原位更新**（in-place update），即把缓冲区写回磁盘原来的位置上，因而会覆盖被修改数据项在磁盘上的旧值[②]。因此，系统只维护数据库磁盘块的单个副本。第二种策略是**镜像**（shadowing）更新，即把更新后的缓冲区写到磁盘的不同位置，所以可以保存数据项的多个版本。

通常数据项更新前的旧值称作**前映像**（BFIM），更新后的新值称为**后映像**（AFIM）。在镜像更新中，BFIM和AFIM都保存在磁盘上，因此并不一定要为恢复而维护日志。我们将在3.4节中简单讨论基于镜像的恢复技术。

3.1.3　先写日志、潜入/非潜入和强制/非强制

基于原位更新的恢复必须使用日志（见1.2.2节）。在这种情况下，恢复机制必须保证数据项的BFIM已经保存在相应的日志项中，并且在AFIM覆盖BFIM之前日志已被刷新到磁盘。这个过程称为**先写日志**。在介绍先写日志协议之前，需要区分对应于写命令的两类日志项信息：UNDO所需的信息和REDO所需的信息。**REDO型日志项**中是被修改数据项的**新值**（即AFIM），因为根据日志重做某些事务时需要用到这些新值（通过把数据项的值改为它的AFIM）。**UNDO型日志项**的内容是数据项的**旧值**（即BFIM），因为根据日志取消某些事务时需要用到这些旧值（通过把数据项的值改为它的BFIM）。在UNDO/REDO算法中，这两类日志项被结合。除此之外在进行级联回滚时，日志中的读数

① 这有些类似于操作系统使用的页表（page table）概念。

② 实际中大部分系统都使用原位更新。

据项（read_item）被认为是UNDO型记录（见3.1.5节）。

如前所述，DBMS的高速缓存内保存着被缓存的数据库磁盘块，它不仅包括数据块，还包括索引块和日志块。当对日志进行写操作时，日志记录存储于DBMS高速缓存的当前日志块中。日志只是一个顺序的（只可追加）磁盘文件，而DBMS的高速缓存可能包括多个将被写回磁盘的日志块（如最后n个日志块）。当对DBMS高速缓存中的数据块进行更新时，相关的日志记录被写到DBMS高速缓存的最后一个日志块中。采用先写日志方法，在数据块本身写回磁盘之前，必须先把包含相关日志记录（对应于特定数据块更新）的日志块写到磁盘中。

标准的DBMS恢复术语包括**潜入/非潜入**（steal/no-steal）和**强制/非强制**（force/no-force），它们确定了何时把数据页从高速缓存写回到磁盘：

(1) 如果缓存中被事务更新的某个页在事务提交之前不能写回磁盘，称为**非潜入方法**，钉住-拔去位指明该页能否被写回磁盘。反之，如果协议允许在事务提交之前把已更新的缓冲区写回磁盘，则该方法被称为**潜入方法**。当DBMS缓存（缓冲）管理器需要为另一个事务提供缓冲区时，缓冲区管理器将会替换某个已被更新但相应事务尚未提交的页，这时用到潜入方法。

(2) 如果事务所有的已更新的页在事务提交时被立即写回磁盘，则称为**强制方法**。否则，称为**非强制方法**。

在3.2节中阐述的延迟更新恢复策略采用的是非潜入方法。但典型的数据库系统一般都使用潜入/非强制策略。潜入方法的优点是：避免使用一个大的缓存空间存储内存中所有的更新页。非强制方法的优点是：当另一个事务需要更新已提交事务的某个更新页时，该页可能仍在缓冲区内，从而避免再次把该页从磁盘上读入缓存中。当多个事务频繁地更新某个页面时，该方法极大地降低了I/O开销。

对使用原位更新的系统进行恢复时，在把更新写入数据库之前必须将恢复所需的某些项永久记录在磁盘日志中。如考虑以下**先写日志**（WAL）协议，它用于同时需要UNDO和REDO操作的恢复算法：

(1) 磁盘上数据项的前映像不能被其后映像所覆盖，除非更新事务的所有UNDO型日志记录已被强制写回磁盘（直至到达该时间点）。

(2) 一个事务不能完成提交操作，除非相应事务的所有REDO型和UNDO型日志记录已被强制写回磁盘。

为了方便恢复过程，DBMS 恢复子系统可能需要维护与系统中所处理事务相关的一些列表。其中包括**活动事务**列表，活动事务是指那些已经开始执行但还未提交的事务。可能还包括自最后一个检查点（见下一节）以来的所有**已提交**（committed）和**已撤销**（aborted）事务的列表。维护上述列表将使得恢复处理更加有效。

3.1.4　系统日志中的检查点和模糊检查点

日志中的另外一类记录项称为**检查点**（checkpoint）[①]。当系统周期性地把所有被修改的DBMS缓冲区写回磁盘时，相应地在日志中写入[checkpoint]记录。因此，在系统发生崩溃时，无需对日志中在[checkpoint]记录项之前已经有[commit,T]记录项的所有事务的WRITE操作进行重做，因为检查点期间它们的所有更新已被写回数据库磁盘。

DBMS的恢复管理器必须确定执行检查点的时间间隔。可以使用时间（如每m分钟）或自上一个检查点之后已提交事务的数目t来表示检查点的间隔，在这里m或t都是系统参数。执行检查点操作包括下列动作：

(1) 暂时挂起事务的执行。

(2) 把主存中被修改的所有缓冲区强制写入磁盘。

(3) 在日志中写入[checkpoint]记录，并把日志强制写入磁盘。

---

① 在某些系统（例如DB2）中，术语检查点用来描述一种限制性更强的情况。它也被用于文学上去描述完全不同的概念。

(4) 重新开始执行事务。

根据第2步的动作，日志中的检查点记录可能还包括一些附加的信息，如活动事务id的列表、每个活动事务在日志中最早和最新（最后）记录的位置（地址）。在必须回滚某些事务时，这些信息有助于对事务进行取消操作。

在第1步中，把所有被修改的缓冲区强制写入磁盘需要一定的时间，由此可能导致事务处理的延迟。为减少延迟，在实际操作中常常用到被称为**模糊检查点**（fuzzy checkpointing）的技术。在这种技术中，系统在[checkpoing]记录写入日志后，就可以重新开始事务的处理，而不必等到第2步结束。但在第2步完成之前，原来的[checkpoint]记录应该仍然有效。因此系统需要维护一个指向有效检查点的指针，该指针始终指向日志中前面的[checkpoint]记录。一旦第2步结束，该指针立即指向日志中新的检查点。

3.1.5　事务回滚

如果在数据库更新后发生事务故障，可能需要对某些事务进行**回滚**（roll back）。如果事务修改了某些数据项的值，并且已经写入了数据库，则必须要把相应数据项恢复为原值（BFIM）。我们使用取消型日志对必须回滚的数据项进行恢复。

如果事务T被回滚，而事务S在这期间读取了T所写的某个数据项X的值，则事务S也必须被回滚。同样，任何读取S所写数据项Y值的所有事务R也必须被回滚，依此类推。将其称为**级联回滚**。如果恢复协议确保调度是可恢复的，但不是严格的和非级联的（见1.4.2节），此时可能发生级联回滚。级联回滚过程非常复杂而且耗时，因此几乎所有的恢复机制都尽量避免使用级联回滚。

图3-1所示的是级联回滚的一个示例。图3-1a中是3个事务的读和写操作。图3-1b是事务的某个执行调度在发生崩溃时的系统日志。事务用到的数据项A、B、C和D的值显示在日志记录的右边。假设数据项的原值分别为A=30、B=15、C=40、D=20。在系统发生故障时，事务T_3尚未执行完毕，因此必须回滚T_3。在对事务进行回滚时，T_3的WRITE操作将被回滚，在图3-1b中用一个*号表示。图3-1c表示了不同事务在不同时间点所进行的操作。

T_1
read_item(A)
read_item(D)
write_item(D)

T_2
read_item(B)
write_item(B)
read_item(D)
write_item(D)

T_3
read_item(C)
write_item(B)
read_item(A)
write_item(A)

(a) 3个事务的读写操作

		A	B	C	D
		30	15	40	20
	[start_transaction,T_3]				
	[read_item,T_3,C]				
*	[write_item,T_3,B,15,12]		12		
	[start_transaction,T_2]				
	[read_item,T_2,B]				
**	[write_item,T_2,B,12,18]		18		
	[start_transaction,T_1]				
	[read_item,T_1,A]				
	[read_item,T_1,D]				
	[write_item,T_1,D,20,25]				25
	[read_item,T_2,D]				
**	[write_item,T_2,D,25,26]				26
	[read_item,T_3,A]				

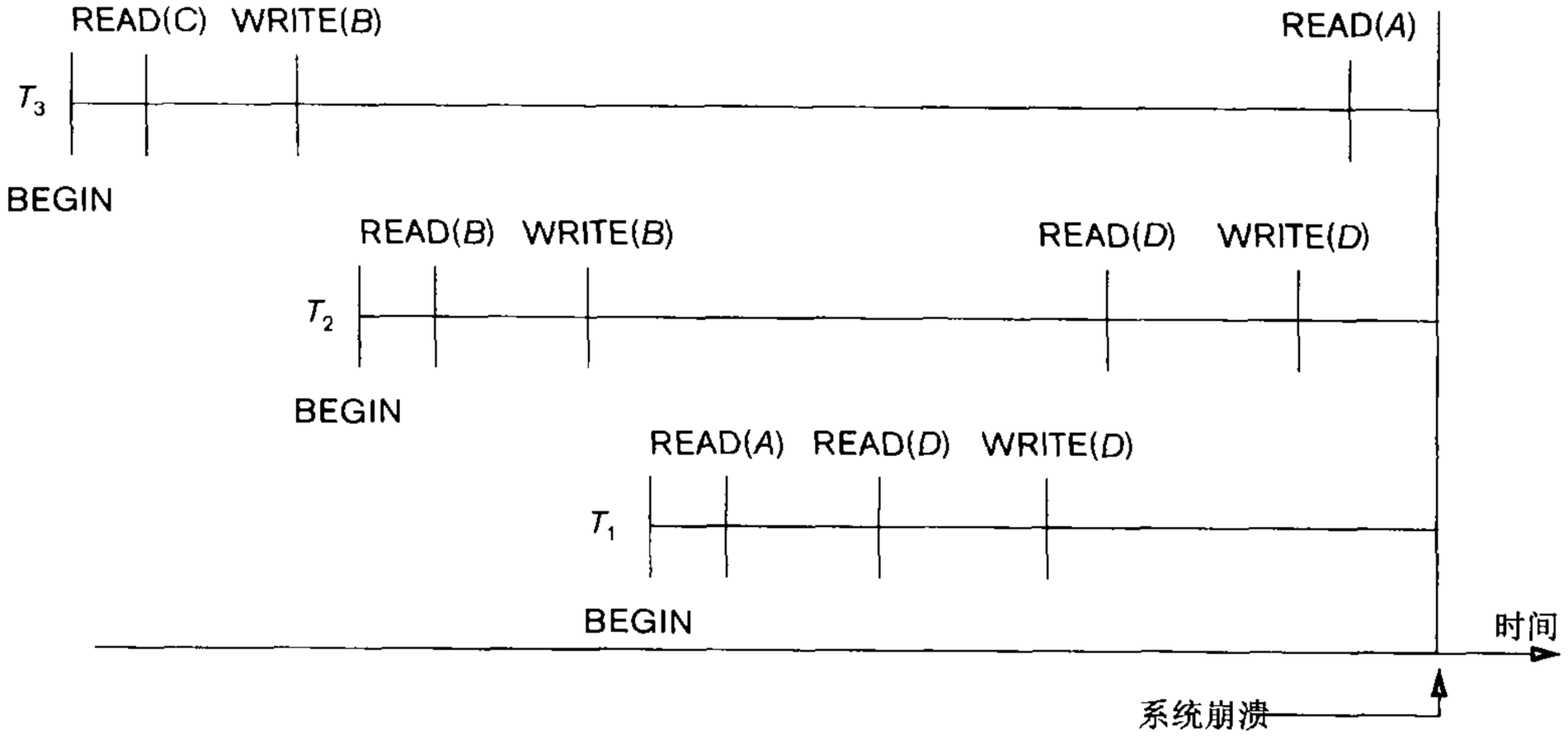
←——— 系统崩溃

* 由于尚未到达提交点，T_3被回滚。

** 由于T_2读了T_3写的B值，T_2也被回滚。

(b) 崩溃点的系统日志

图3-1　说明级联回滚（在严格或非级联调度中不会发生）



(c) 崩溃前的操作

图3-1（续）

现在必须检查级联回滚。从图3-1c中可以看出，事务T_2读取了事务T_3写的B值；也可通过检查日志确定级联回滚。因为T_3被回滚，所以T_2也必须被回滚。T_2的WRITE操作要被取消，这些操作在日志中用**表示。需要注意的是，当回滚事务时只有相应的write_item操作被取消，而日志中的read_item操作只用来确定是否需要级联回滚。

在具体应用中，系统的恢复机制可以保证调度是非级联的或是严格的，因此从不需要使用级联回滚。因此，也不用在日志中记录任何read_item操作，因为只有在确定级联回滚时才会需要这些操作。

3.2　基于延迟更新的恢复技术

延迟更新技术的策略是：把对数据库的任何实际更新延迟或推迟到事务成功完成执行并到达其提交点之后进行[①]。

在事务执行期间，更新只是被记录到日志和高速缓存的缓冲区中。当事务到达提交点并把日志强制写回磁盘时，更新才被记录到数据库中。如果事务在到达提交点之前发生故障，则无需取消该事务进行的任何操作，因为该事务并没有对磁盘数据库产生任何的影响。虽然这会简化恢复，但只有在事务较短，并且修改的数据项较少时才可使用延迟更新。对于其他类型的事务，由于到达提交点之前缓存的缓冲区必须一直保存事务的修改，这样可能导致缓冲区空间开销过大。

典型的延迟更新协议叙述如下：

(1) 仅当一个事务已经到达提交点，该事务才可以更改在磁盘上的数据库。

(2) 仅当一个事务已经把所有更新操作都记录到日志中，并将日志强制写到磁盘中，该事务才能到达提交点。

需要注意的是：步骤2是对先写日志协议（WAL）的另外一种陈述。因为直到事务提交之后磁盘上的数据库才会被更新，所以无需UNDO任何操作。因此，将其称为NO-UNDO/REDO**恢复算法**。如果系统故障发生在事务提交之后，但所有更新还未被写入磁盘，此时需要REDO某些操作。在这种情况下，就要根据日志记录重做事务的某些操作。

通常在多用户系统中，故障时的恢复与并发控制机制密切相关。我们首先讨论单用户系统中的恢复，其中不需要并发控制，所以可以独立于任何并发控制方法来理解恢复过程。然后再对影响恢

① 因此，延迟更新也是一个非潜入方法。

复处理的并发控制进行讨论。

3.2.1　单用户环境下使用延迟更新的恢复

在单用户环境下的恢复算法是相当简单的。算法RDU_S（单用户环境下基于延迟更新的恢复）使用REDO过程对某些write_item操作进行重做。其工作顺序如下：

RDU_S 过程。使用两个事务列表：自上一个检查点以来所有已提交事务列表和当前活动事务列表（因为是单用户系统，所以最多只有一个活动事务被填入）。根据日志记录的顺序，对已提交事务的所有write_item操作进行REDO。然后重新开始活动事务。

REDO过程定义如下：

REDO（WRITE_OP）过程。重做一个write_item操作的WRITE_OP涉及检查相应的日志项[write_item, T, X, new_value]，并把数据库中项X的值置为new_value，即置为后映像（AFIM）。

REDO操作必须是**幂等的**（idempotent），即重复执行多次的效果和只执行一次的效果是等价的。事实上，全部恢复过程应当也是幂等的。这是因为，如果系统在恢复过程中发生故障，再进行恢复时可能会REDO对上一次恢复中已经重做的write_item操作再次重做。在系统恢复期间，从系统崩溃中恢复的结果应当和没有发生崩溃的恢复结果相同！

需要注意的是：由于使用延迟更新协议，因此活动列表中的事务不会对数据库产生任何影响，所以恢复过程可以完全忽略这些事务。但是必须要重新启动这些事务，或由恢复过程自动进行或由用户手工启动。

图3-2展示了一个单用户环境下的恢复示例。其中，在事务T_2的执行过程中系统发生首次故障，如图3-2b所示。恢复过程通过把D值重新置为20（D的新值），对日志中的[write_item, T_1, D, 20]项执行重做。由于T_2尚未提交，因此恢复过程将忽略日志中的[write, T_2, …]项。如果在第一个故障的恢复期间又发生了第二次故障，则从头到尾重复相同的恢复过程，并产生相同的结果。

T_1
read_item(A)
read_item(D)
write_item(D)

T_2
read_item(B)
write_item(B)
read_item(D)
write_item(D)

(a) 两个事务的READ和WRITE操作

[start_transaction, T_1]
[write_item, T_1, D, 20]
[commit, T_1]
[start_transaction, T_2]
[write_item, T_2, B, 10]
[write_item, T_2, D, 25]

← 系统崩溃

T_1的[write_item…]操作被重作。
恢复过程忽略T_2的日志记录。

(b) 系统崩溃时的系统日志

图3-2　单用户环境下基于延迟更新的恢复示例

3.2.2　多用户环境下并发执行的延迟更新

对于具有并发控制的多用户系统，恢复过程可能更为复杂，这取决于用于并发控制的协议。在许多情况中，并发控制和恢复处理是相关的。通常，系统所能达到的并发程度越高，恢复所使用的

时间也越长。

考虑一个使用严格的两阶段加锁协议的并发控制系统，事务到达提交点之前数据项上的锁一直保持有效。事务提交之后，锁才被释放。这就保证了严格且可串行化的调度。假定日志中包括了[checkpoint]项，下面给出称为RDU_M的多用户环境下基于延迟更新的恢复算法。该过程中用到了上面定义的REDO过程。

RDU_M过程（带有检查点）。使用系统维护的两个事务列表：自最后一次检查点以来，所有已提交事务T的列表（**提交列表**）和当前活动事务T'的列表（**活动列表**）。根据日志记录的顺序，对已提交事务的WRITE操作执行REDO。活动事务和尚未提交的事务立即删除，并必须重新提交。

图3-3所示是一个可能的执行事务的调度。当在t_1时刻执行检查点操作时，事务T_1已经提交，但T_3和T_4尚未提交。系统在t_2时刻发生崩溃，此时T_3和T_2已提交，但T_4和T_5尚未提交。根据RDU_M方法，无需重做事务T_1或其他在最后一次检查点t_1前已被提交事务的write_item操作。但必须重做T_2和T_3的write_item操作，因为这两个事务在最后一次检查点之后才到达提交点。我们在前面已经陈述过，事务提交之前先要强制写日志。事务T_4和T_5将被忽略，因为使用延迟更新协议时，T_4和T_5的write_item操作不会对数据库产生任何影响，因此删除或回滚这两个事务。稍后还将用图3-3来说明其他恢复协议。

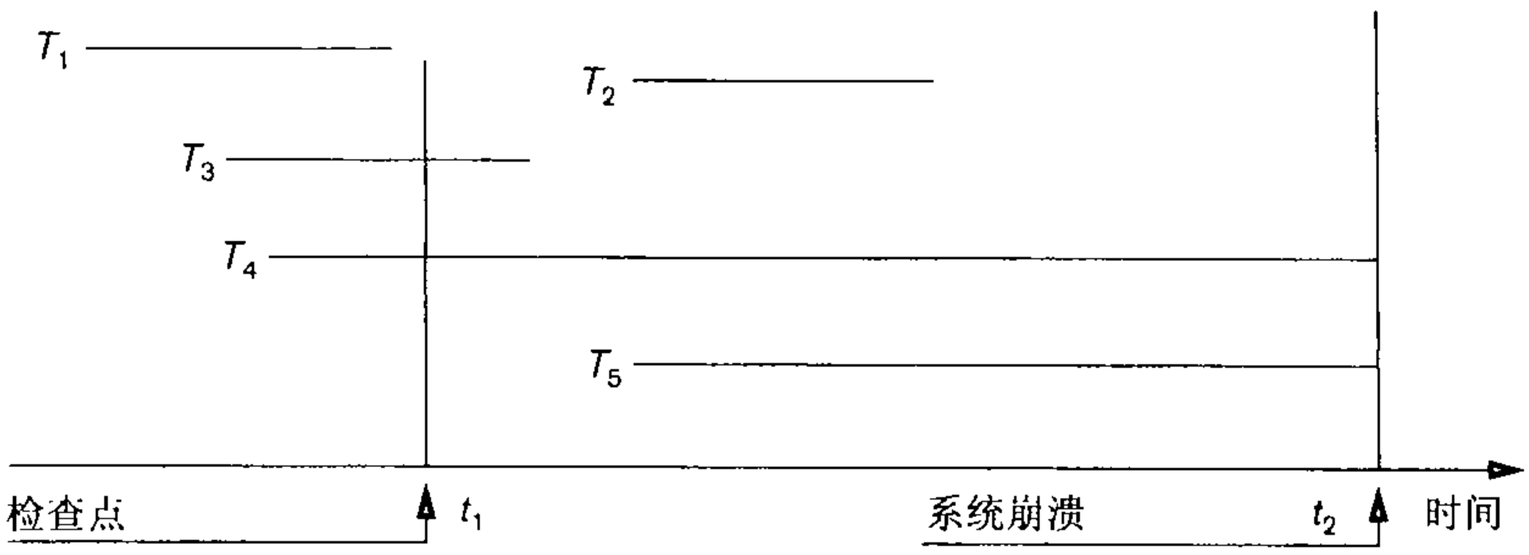


图3-3　多用户环境下的恢复示例

通过以下方法可以使NO-UNDO/REDO恢复算法更加高效：如果自上个检查点之后，数据项X被已提交事务更新了多次，那么在恢复时只需对X的最后一次更新进行重做。无论何种情况，X的其他更新都将被最后的REDO所覆盖。这时，可以从日志的尾部开始，只要某个数据项被重做，就将它加入到重做列表中。在对每个数据执行REDO之前，检查这个列表，如果该数据项出现在列表中，则无需对它再次执行重做，因为它的最新值已被恢复。

如果因为某个原因（譬如死锁检测方法）导致事务撤销，由于该事务尚未对磁盘上的数据造成任何影响，因此只需简单地重新提交该事务。这个方法的缺点是限制了事务的并发执行，因为在事务到达提交点之前，所有的数据项都处于锁定状态。另外，由于事务提交之前所有的更新数据项都保留在缓冲区中，由此导致缓冲区空间开销过大。该方法的主要优点是无需对事务操作进行UNDO，这有如下两个原因：

(1) 事务到达提交点时，即该事务成功完成了它的执行，才把它对数据库的更改写回磁盘。因此，在事务执行过程中发生故障时，无需对事务进行回滚。

(2) 事务不会读取由另一个未提交事务所写的数据值，因为在事务未被提交时相应的数据项处于锁定状态。这样就不会发生级联回滚。

图3-4显示了基于上述恢复和并发控制方法的多用户系统下的恢复示例。

T_1
read_item(*A*)
read_item(*D*)
write_item(*D*)

T_2
read_item(*B*)
write_item(*B*)
read_item(*D*)
write_item(*D*)

T_3
read_item(*A*)
write_item(*A*)
read_item(*C*)
write_item(*C*)

T_4
read_item(*B*)
write_item(*B*)
read_item(*A*)
write_item(*A*)

(a) 4个事务的READ和WRITE操作

[start_transaction,T_1]
[write_item, T_1, *D*, 20]
[commit, T_1]
[checkpoint]
[start_transaction, T_4]
[write_item, T_4, *B*, 15]
[write_item, T_4, *A*, 20]
[commit, T_4]
[start_transaction, T_2]
[write_item, T_2, *B*, 12]
[start_transaction, T_3]
[write_item, T_3, *A*, 30]
[write_item,T_2, *D*, 25] ← 系统崩溃

T_2和T_3没有到达提交点，因此被忽略；
T_4的提交点在上一次检查点之后，因此被重做。

(b) 崩溃点的系统日志

图3-4　使用延迟更新的并发事务恢复示例

3.2.3　不影响数据库的事务活动

通常，一个事务的某些活动将不会对数据库产生影响，如根据数据库检索的信息生成消息和打印报表。如果事务在结束前发生故障，用户可能无法获得所需的报表。如果已生成了一个错误的报表，用户可能会基于该报表的信息采取一些影响数据库的行为，所以恢复过程必须通知用户该报表出错。因此，这类报告只能在事务到达其提交点之后生成。处理这种活动的常用方法是发出生成报告的命令，但把它们保存为批作业（batch job），只有在事务到达其提交点之后才执行这些批作业。如果事务发生故障，则这些批作业被删除。

3.3　基于即时更新的恢复技术

在即时更新恢复技术中，当事务发出更新命令时数据库可以立即更新，而无需等待事务到达提交点。但是，在这些技术中，更新操作应用到数据库之前，仍然必须使用先写日志协议记录到日志中（在磁盘上），这样可以在故障发生时加以恢复。

对于取消故障事务已经应用到数据库的更新操作的影响，必须做一些规定，即回滚相应的事务，并取消事务的write_item操作对数据的影响。从理论上讲，可以区分两类主要的即时更新算法。如果恢复技术确保事务的全部更新在事务提交之前都能够记录到磁盘上的数据库中，就无需REDO已提交事务的任何操作。我们称之为UNDO/NO-REDO**恢复算法**。另一方面，最常见的情况是所有更新被写回数据库之前允许提交事务，那么称之为UNDO/REDO**恢复算法**。这是最为复杂的恢复技术。下面将要讨论两个UNDO/REDO算法的示例，读者可根据该算法思考UNDO/NO-REDO恢复算法。在3.5节中将给出一个更为实用的方法，称为ARIES恢复技术。

3.3.1　单用户环境下基于即时更新的 UNDO/REDO 恢复

如果在单用户系统中发生故障，故障时的执行（活动）事务可能已把更新写回数据库。恢复时必须对所有这样的操作执行取消操作。下面的恢复算法RIU_S（单用户环境下使用即时更新的恢复）用到了前面定义过的REDO过程以及下面要定义的UNDO过程。

RIU_S过程。

(1) 使用系统维护的两个事务列表：自最后一个检查点之后的所有已提交事务列表和活动事务列表（最多有一个事务被填入此表，因为是在单用户系统中）。

(2) 使用下面描述的UNDO过程，对日志中所有活动事务的write_item操作执行取消操作。

(3) 使用上面描述的REDO过程，根据日志记录的顺序重做所有已提交事务的write_item操作。

UNDO过程定义如下：

UNDO（WRITE_OP）过程。

取消一个write_item操作的write_op包括检查相应的日志项[write_item, T, X, old_value, new_value]，并把数据库中X的值置为old_value，即X的前映像（BFIM）。取消多个write_item操作的顺序必须和它们在日志中被写入的顺序相反。

3.3.2　多用户环境下基于即时更新的 UNDO/REDO 恢复

当允许并发执行时，恢复过程将依赖于所使用的并发控制协议。RIU_M（多用户环境下基于即时更新的恢复）过程概括了基于即时更新的并发事务的恢复算法。假设日志使用检查点，而且并发控制协议能够保证调度的严格性，譬如严格的两阶段加锁协议。我们知道，一个事务将不能对某个数据项进行读或写，除非最后写该数据项的事务已被提交（或已被撤销或已被回滚）。但在严格的两阶段加锁中可能发生死锁，因此需要撤销并UNDO某些事务。对于严格调度，UNDO操作需要把数据项改回到其旧值（BFIM）。

RIU_M过程。

(1) 使用系统维护的两个事务列表：自最后一个检查点之后的所有已提交事务列表和活动事务列表。

(2) 使用UNDO过程取消活动（未提交）事务的所有write_item操作。根据日志记录的相反顺序取消上述操作。

(3) 根据日志记录的顺序重做所有已提交事务的write_item操作。

我们在3.2.2节中讨论过，如果从日志末尾开始，并且只对每个数据项X的最后一次更新进行重做，这将会提高步骤3的执行效率。只要某个数据项被重做，就将它加入到已重做列表中，并且不再对其进行重做。可以设计一个类似的过程来提高步骤2的效率。

3.4　镜像分页

单用户环境下的恢复模式无需使用日志。在多用户环境下，对于并发控制方法可能需要日志。为了便于恢复，镜像分页认为数据库是由大量固定大小（例如 n）的磁盘页（或磁盘块）所构成的。创建一个具有n个项的**目录**[①]，其中第i个项指向磁盘上第i个数据库页面。如果目录不是很大就将其保存在主存中，所有对磁盘上数据页的读或写都必须通过目录进行。当开始执行某个事务时，**当前目录**被复制到**镜像目录**中，当前目录项指向最新的或当前的磁盘数据库页面。然后把镜像目录保存在磁盘中，而事务使用当前目录。

在事务执行期间，不允许对镜像目录进行修改。当执行write_item操作时，创建被修改数据库页

① 该目录类似于操作系统为每个进程维护的页表（page table）。

的一个新副本，但该数据页的旧副本不可以被覆盖。新的页写到空闲的磁盘块中，然后对当前目录进行修改以使其指向新的磁盘块，但镜像目录保持不变，并且继续指向旧的未被修改的磁盘块。图3-5说明了当前目录和镜像目录的概念。因此，事务所更新的页面的两个版本被保存下来。旧版本由镜像目录引用，新版本则由当前目录引用。

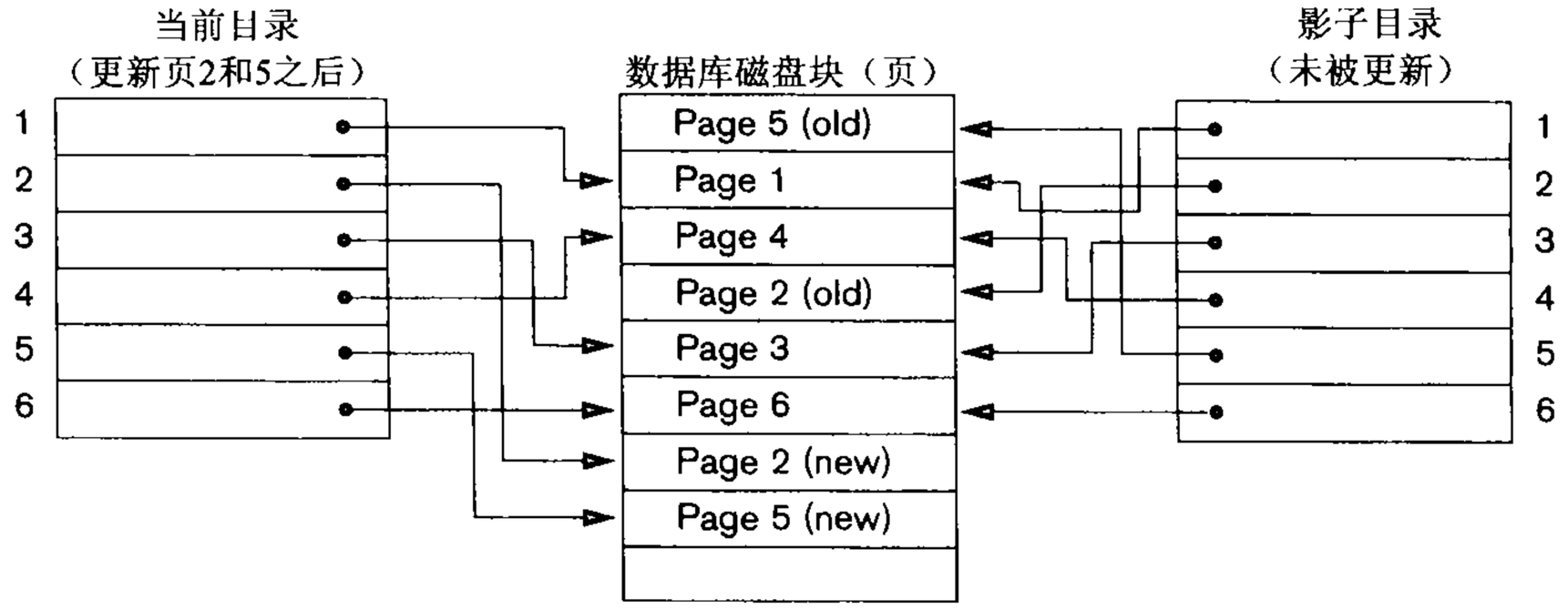


图3-5　镜像分页的示例

为了在事务执行过程中从故障中恢复，只需释放更新的数据库页并删除当前目录。镜像目录提供了事务开始执行前的数据库状态，通过恢复镜像目录可以实现数据库的恢复。这样，数据库可以恢复到系统发生崩溃时正在执行的事务的开始执行之前的状态，并丢弃所有已修改的页。提交事务对应于丢弃前一个镜像目录。由于恢复既不涉及取消对数据项的操作也不涉及重做对数据项的操作，因此，这是一种NO-UNDO/NO-REDO恢复技术。

在并发事务的多用户环境中，必须在镜像分页技术中结合日志和检查点。镜像分页的缺点是，更新的数据页改变了在磁盘中的位置。这使得不借助复杂的存储管理策略的前提下，很难将相关的数据页存储在一起。此外，如果目录尺寸过大，在事务提交时，镜像目录写入磁盘的开销也将逐渐增大。另外一个复杂的问题是，提交事务时如何处理**垃圾回收**（garbage collection）的问题。镜像目录指向的被更新的旧页面必须被释放，并且要把这些页面加入到空闲列表中。这些页面在该事务被提交后就不再需要了。另外，需要注意的是：在当前目录和镜像目录之间的转移必须作为一个原子操作来执行。

3.5　ARIES 恢复算法

现在我们以ARIES算法为例，描述用于数据库系统的一种恢复算法。ARIES使用的是潜入/非强制方法，它基于下面的3个概念：先写日志，在重做时重复历史，以及取消时改变日志。3.1.3节已经讨论了第一个概念先写日志。第二个概念**重复历史**（repeating history）指的是，为从系统崩溃中恢复数据库状态，ARIES将追溯崩溃前数据库系统的所有动作。崩溃时尚未提交的事务（活动事务）被取消。第三个概念**取消期间记日志**（logging during undo）指的是，如果在恢复时又发生故障（这将导致恢复过程的重启），ARIES可以避免重复已完成的取消操作。

ARIES恢复过程由下面3个主要步骤组成：分析、REDO和UNDO。**分析步骤**确定缓冲区[①]中的脏（已更新的）页及崩溃时的活动事务集合。同时确定日志中开始REDO操作的起点。**REDO阶段**实际上是把日志中的更新重新应用到数据库。通常，只对已提交事务进行REDO操作。但在ARIES中则并非如此。ARIES日志中的特定信息为REDO操作提供了起点，REDO操作从起点开始一直进行到日志

① 实际的缓冲区在系统崩溃时可能会丢失，因为它们驻留在主存中。使用检查点时存于日志中的附加表（脏页表、事务表）ARIES可以确定这些信息（见下页）。

末尾。此外，基于AREIS存储的信息和数据页中的信息可以确定已重做的操作是否已被真正应用于数据库，从而不必再次更新数据库。这样，恢复时只用到必要的一些REDO操作。最后，在**UNDO阶段**，反向扫描日志并以逆序取消崩溃时活动事务的操作。ARIES恢复所需的信息涉及日志、事务表及脏页表。此外还会用到检查点。事务管理器对上述两个表进行维护，在检查点时将其写入日志。

在ARIES中，每个日志记录都赋有一个**日志序列号**（log sequence number，LSN），LSN指出日志记录在磁盘上的地址，它随记录的增加而单调递增。每个LSN和某个事务特定的更新（行为）相对应。另外，每个数据页存储对应于该页某个更新的最后的日志记录的LSN。下述任何动作都将写入日志：更新某页、提交事务、撤销事务、取消更新或结束事务。日志记录中需要包括前3个动作的必要性在前面已经讨论过，但需要对后两个动作做一些解释。当取消某个更新时，需要在日志中写入补偿日志记录。当事务结束时（无论是提交还是撤销），都要向日志中写入结束日志记录。

在所有日志记录中的公共字段包括：事务的前一个LSN，事务ID，以及日志记录的类型。前一个LSN是非常重要的，因为它把每个事务的日志记录链接在一起（以相反的顺序）。对于更新（WRITE）动作，在日志记录中的附加字段包括：该数据项所在页的ID，更新数据项的长度，页偏移量，该数据项的前映像和后映像。

除日志外，为提高恢复的效率还需要用到两个表：**事务表**（transaction table）和**脏页表**（dirty page table），由事务管理器对这两个表进行维护。发生故障时，在恢复的分析阶段重建这两个表。事务表包括与每个活动事务相对应的表项，具体信息包括事务ID，事务状态，以及事务最新日志记录的LSN。脏页表中的记录是和缓冲区中每个脏页相对应的一些内容，具体包括页ID和对应于该页最早更新的LSN。

ARIES 中**建立检查点**（checkpointing）包括以下步骤：在日志中写入begin_checkpoint记录，在日志中写入end_checkpoint记录，以及把begin_checkpoint记录的LSN写入某个专用文件中。在恢复时要定位最后的检查点信息就要访问这个专用的文件。使用end_checkpoint记录，把事务表和脏页表的内容添加到日志末端。为降低成本，DBMS使用了**模糊检查点**，以便在检查点时继续事务的执行（见3.1.4节）。此外，在检查点时不必将DBMS缓存中的数据刷新到磁盘，因为事务表和脏页表已被添加到日志中，这两个表包含恢复所需的必要信息。需要注意的是：如果在检查点期间发生崩溃，上面提到的专用文件将指向前一个检查点，系统将借助该文件进行恢复。

在某次崩溃之后，ARIES恢复管理器接管系统。先通过使用上述的专用文件访问最后一个检查点的有关信息。**分析阶段**从begin_checkpoint记录开始并进行到日志尾部。当遇到end_checkpoint记录时，开始对事务表和脏页表进行访问（这两个表在检查点时被写入日志）。在分析阶段，被分析的日志记录可能会导致对上述两个表的修改。例如，若在事务表中遇到事务T的结束日志记录，则从表中删除T的记录。如果遇到事务T'的其他类型的日志记录，若T'的相应记录项尚不存在，则要在事务表中插入一个记录项，并修改最后的LSN字段。如果日志记录和页面P的某个更新相对应，则创建关于页P的一个记录（如果表中不存在该记录），并修改相关的LSN字段。当分析阶段结束时，REDO和UNDO所需的必要信息都已经完全在表中了。

接下来是**REDO阶段**。为减少不必要的工作量，ARIES从日志中的某个点开始重做，并（确实地）知道该点以前脏页的所有变化已被写入磁盘上的数据库。通过在脏页表的所有脏页中查找最小的LSN和M，ARIES可确定日志中开始重做的记录位置。对于可重做的事务，所有对应于LSN<M的更改必定已被写入磁盘或在缓冲区中被覆盖；否则，具有该LSN的脏页将会出现在缓冲区中（和脏页表中）。因此，从LSN=M的日志记录开始REDO，并向前扫描直至全部日志。对于日志记录的每个更改，REDO算法将验证是否应用该更改。例如，若日志所记录的某个更改与页P（不在脏页表中）有关，则该更改已经在磁盘上，无需对其进行重做。或者说，如果日志所记录的某个更改（譬如，其LSN=N）和页P有关，并且脏页表包含P的表项其LSN大于N，则此更改已被写入数据库。如果上述两个条件都不成立，则从磁盘读取页P并将LSN存储于该页中。然后，将LSN（P）和N进行比较，如果N<LSN（P），则此更改已被应用并且无需将该页写回磁盘。

一旦REDO阶段结束，数据库就被恢复到崩溃发生时的状态。在分析阶段，我们已经在事务表中确定了活动事务集合，称为undo_set。现在，**UNDO阶段**通过从日志末尾反向扫描以对某些动作进行取消。对于每个被取消的动作，要向日志中相应地写入一条补偿记录。UNDO从后向前扫描日志，直到undo_set中事务集合的每个动作都被取消。当这些操作完成后，恢复过程结束，并重新开始正常的处理工作。

下面来分析图3-6中所示的恢复示例。图中共有3个事务：T_1、T_2和T_3。T_1更新页C，T_2更新页B和页C，T_3更新页A。图3-6a中是日志的部分内容，图3-6b中是事务表和脏页表的内容。假设这时发生崩溃，由于已经执行了检查点，系统检索到相关的begin_checkpoint记录的地址为location 4。分析阶段从location 4开始直到日志结束。图3-6b中，end_checkpoint记录包括事务表和脏页表，在分析阶段将会进一步重建这些表。当分析阶段遇到日志记录6时，则在事务表中为事务T_3创建一个新的项，在脏页表中为页A创建一个新的项。在结束对日志记录8的分析后，事务表中事务T_2的状态被更改为已提交（committed）状态。图3-6c是分析阶段结束后的事务表和脏页表。

Lsn	Last_lsn	Tran_id	Type	Page_id	Other_information
1	0	T_1	更新	C	···
2	0	T_2	更新	B	···
3	1	T_1	提交		···
4	开始检查点				
5	结束检查点				
6	0	T_3	更新	A	···
7	2	T_2	更新	C	···
8	7	T_2	提交		···

(a) 崩溃点时的日志

事务表

Transaction_id	Last_lsn	Status
T_1	3	提交
T_2	2	进行中

脏页表

Page_id	Lsn
C	1
B	2

(b) 检查点时的事务表和脏页表

事务表

Transaction_id	Last_lsn	Status
T_1	3	提交
T_2	8	提交
T_3	6	进行中

脏页表

Page_id	Lsn
C	1
B	2
A	6

(c) 分析阶段后的事务表和脏页表

图3-6　ARIES的恢复示例

对于REDO阶段，脏页表中最小LSN为1。因此，REDO将从日志记录1开始，并继续对更新进行REDO。对于页C、B、A和C的更新分别对应的LSN{1, 2, 6, 7}并不比那些脏页表中的页的LSN小。因此再次读这些页，并根据日志进行REDO（假设这些数据页中的实际LSN小于相应的日志项）。这样，REDO阶段结束而UNDO阶段开始。在图3-6c所示的事务表中，只对活动事务T_3进行UNDO。UNDO阶段从日志记录6（T_3的最后一个更新）开始并自后向前进行。事务T_3由后向前的更新依次被取消。

3.6　多数据库系统中的恢复

到目前为止，我们一直假定事务访问的是单个数据库。在某些情况下，单个事务可能需要访问多个数据库，这样的事务被称为**多数据库事务**（multidatabase transaction）。这些数据库甚至可能被置于不同类型的DBMS中。例如，有些DBMS是关系类型的，而有些DBMS是面向对象的、层次的或网状类型的。在这种情况下，多数据库事务涉及的每个DBMS可能具有自己的恢复技术，并且这个事务的DBMS恢复管理器和其他DBMS的恢复管理器相分离。这种情形有些类似于分布式数据库管理系统（见第9章），在分布式数据库管理系统中，数据库的各个部分驻留在不同的站点，各站点通过通讯网络相互连接。

为维护多数据库事务的原子性，有必要采取两级恢复机制。除本地恢复管理器及其所维护的信息外，还需要一个**全局恢复管理器**（global recovery manager）或称为**协调器**（coordinator），来维护恢复所需的必要信息（日志、表）。协调器通常遵循**两阶段提交协议**（two-phase commit protocol）。协议的两个阶段分别为：

- **阶段1**。当所有参与数据库通知协调器，多数据库事务涉及的各部分都已结束时，协调器向每个参与者发送准备提交指令，并准备提交事务。收到指令的每个参与数据库把所有的日志记录和局部恢复所需的信息强制写入磁盘，并向协调器发送提交准备完毕或*OK*信号。如果强制写磁盘发生故障或由于某个原因导致局部事务不能提交，那么该参与数据库向协调器发送不能提交或*not OK*信号。如果协调器在某个时间间隔内没有收到来自参与数据库的响应，则假定该数据库*not OK*。
- **阶段2**。如果所有参与数据库的响应都是*OK*，并且协调器的表决也是*OK*，则该事务成功完成，协调器将向每个参与数据库发送提交信号。由于该事务的局部动作及局部恢复所需的信息都已写入参与数据库的日志，一旦发生故障，可以借助上述信息进行恢复。每个参与数据库在日志中写入[commit]项完成该事务的提交，并且如果需要则永久地更新数据库。另一方面，如果有一个或多个参与数据库（或协调器）的响应是*not OK*，则该事务发生故障，协调器向每个参与数据库发送回滚或UNDO该局部事务的指令，各参与数据库根据日志进行取消事务操作。

两阶段提交协议使得所有的参与数据库要么都提交，要么都不提交。若任何参与者或协调器发生故障，总可以把数据库恢复到某个状态，则在这个状态中该事务或提交或回滚。如果在阶段1开始前或进行中发生故障，则事务需要回滚；但如果在阶段2发生故障，则意味着可以成功地恢复或提交事务。

3.7　数据库备份和灾难性故障恢复

到目前为止，我们讨论过的所有技术都只适用于非灾难性故障的恢复。其中一个关键的假设是，系统日志保存在磁盘上，并且不会因故障而丢失。类似地，当使用镜像分页恢复时，镜像目录必须保存在磁盘上。前面所讨论的恢复技术都使用系统日志或镜像目录的内容把数据库恢复到某个一致性状态。

DBMS的恢复管理器还可以处理灾难性故障，比如磁盘崩溃。处理灾难性故障的主要技术是**数据库备份**（database backup）。全部数据库和日志要周期性地备份在廉价存储介质上，如磁带。当系统发生灾难性故障时，可以从最近的一次备份中重新装入磁盘，并重新启动系统。

来自关键应用，（如银行、保险、股票市场及其他的数据库）的数据，为了保护其完整性需要周期性地备份，并移动到安全的物理存储空间。隐蔽的储存室被用于保护遭遇洪水、风暴、地震或火灾等破坏的数据。近期的一些事件，如2001年发生在纽约的9·11恐怖袭击事件，以及2005年发生在

新奥尔良的卡特里娜飓风，都为建立灾难性恢复业务应急数据库敲响了警钟。

为了避免丢失自上次备份以来所有已被执行事务的结果，通常要以更高的频率将系统日志备份到磁带。由于系统日志比数据库小得多，因此可以更加频繁地进行备份。这样，用户就不会丢失最后一次备份之后所有已被完成的事务。对于已备份到磁带的系统日志中所记录的所有已提交事务，都会对重做数据库产生影响。每次数据库备份后都要创立一个新的日志。因此，为了从磁盘故障中恢复，首先利用磁带上最近一次的备份在磁盘上重建数据库，随后根据记录在系统日志备份副本中已提交事务的操作，来重构这些事务操作的结果。

小结

本章讨论了事务故障的恢复技术。恢复的主要目的是保证事务的原子性。如果事务在执行过程中发生故障，恢复机制必须保证该事务未对数据库造成影响。本章首先非正式地给出了恢复过程，然后讨论了有关恢复的一些系统概念，包括缓存、原位更新、镜像更新、数据的前映像和后映像、UNDO操作、REDO操作、潜入/非潜入策略、强制/非强制策略、系统检查点及先写日志协议。

接着讨论了恢复的两种不同方法：延迟更新和即时更新。延迟更新技术在事务到达提交点时更新才被真正写入磁盘。在把更新写入数据库之前，首先要把日志强制写入磁盘。当结合某种并发控制方法使用时，这种方法避免了对事务的回滚，恢复操作只涉及根据日志重做上一个检查点之后的所有已提交事务。该方法的缺点是所需的缓冲区空间过大，因为更新要保存在缓存区中，且在事务提交前不能写入磁盘。根据延迟更新方法可以得到NO-UNDO/REDO恢复算法。即时更新技术在事务成功结束之前便将更新写入磁盘数据库，但对数据库的任何改变必须先被记录在日志中并强制写到磁盘。这样，必要时可对这些操作执行取消。此外还给出了一个使用即时更新的UNDO/REDO恢复算法。如果在提交之前所有事务更新已被写入数据库，还可以使用基于即时更新的UNDO/NO-REDO算法。

此外还讨论了镜像分页技术，它通过使用镜像目录对旧的数据库页进行记录。这实际上是一种NO-UNDO/NO-REDO算法，在单用户系统中不需要使用日志，但在多用户系统中还是要用到。另外还阐述了ARIES，这是专门在IBM的某些关系数据库产品中使用的恢复策略。然后讨论了两阶段提交协议，该协议用于多数据库事务的故障恢复。最后，讨论了灾难性故障的恢复问题，典型的做法是把数据库和日志备份到磁带上。对日志的备份可以比数据库的备份更为频繁。备份的日志可以用于重做自最近一次的数据库备份以来进行的操作。

复习题

3.1　讨论不同类型的事务故障。什么是灾难性故障？

3.2　讨论数据库中read_item和write_item操作的动作。

3.3　系统日志的用途是什么？典型的系统日志记录项包括什么内容？什么是检查点？为什么说它们很重要？什么是事务提交点？为什么说它们很重要？

3.4　恢复子系统如何使用缓冲和高速缓存技术？

3.5　什么是数据项的前映像（BFIM）和后映像（AFIM）？原位更新和镜像更新在处理BFIM和AFIM方面有何不同？

3.6　什么是UNDO型日志？什么是REDO型日志？

3.7　描述先写日志协议。

3.8　区别被恢复子系统维护的三类事务列表。

3.9　什么是事务回滚？什么是级联回滚？为什么实用的恢复方法不允许级联回滚？哪一种恢复技术不需要回滚？

3.10　讨论UNDO和REDO操作，以及使用UNDO、REDO操作的恢复技术。

3.11　讨论恢复的延迟更新技术。这种技术的优缺点是什么？为什么又被称为NO-UNDO/REDO方法？

3.12　恢复时如何处理那些对数据库没有影响的事务操作？如被事务打印的报表。

3.13　讨论单用户环境和多用户环境下的即时更新恢复技术。即时更新的优缺点各是什么?

3.14　使用即时更新的UNDO/REDO和UNDO/NO-REDO恢复算法有什么不同？简略叙述UNDO/NO-REDO算法。

3.15　描述基于镜像分页的恢复技术。什么情况下使用该技术不需要日志？

3.16　描述ARIES恢复方法的3个阶段。

3.17　什么是ARIES中的日志序列号（LSN）？如何使用？在脏页表和事务表中含有什么信息？描述在ARIES中如何使用模糊检查点。

3.18　在事务处理的缓冲区管理中，潜入/非潜入和强制/非强制分别指的是什么？

3.19　描述多数据库事务的两阶段提交协议。

3.20　讨论如何从灾难性故障中进行恢复。

练习题

3.21　在图3-1b中，假设在[read_item, T_3, A]项写入日志之前系统发生崩溃。这对恢复过程会有什么不同影响？

3.22　在图3-1b中，假设在[write_item, T_2, D, 25,26] 项写入日志之前系统发生崩溃。这对恢复过程会有什么不同影响？

3.23　图3-7所示的是系统崩溃点处，与4个事务T_1、T_2、T_3和T_4的一个特定调度相对应的日志。假设使用的是带有检查点的即时更新协议。描述系统崩溃后的恢复过程。指出哪些事务需要回滚？日志中的哪些操作需要重做和取消（如果有的话）？恢复过程中是否发生了级联回滚？

[start_transaction, T_1]
[read_item, T_1, A]
[read_item, T_1, D]
[write_item, T_1, D, 20, 25]
[commit, T_1]
[checkpoint]
[start_transaction, T_2]
[read_item, T_2, B]
[write_item, T_2, B, 12, 18]
[start_transaction, T_4]
[read_item, T_4, D]
[write_item, T_4, D, 25, 15]
[start_transaction, T_3]
[write_item, T_3, C, 30, 40]
[read_item, T_4, A]
[write_item, T_4, A, 30, 20]
[commit, T_4]
[read_item, T_2, D]
[write_item, T_2, D, 15, 25] ← 系统崩溃

图3-7　一个调度示例及其相应的日志

3.24　假设图3-7的示例使用的是延迟更新协议。说明通过删除不必要的日志项，对于延迟更新的系统日志有何不同？然后使用修改后的日志描述恢复过程。假设只运用REDO操作，指出日志中的哪些操作需要重做，哪些操作被忽略。

3.25　ARIES中的检查点和3.1.4节中的检查点有何不同？

3.26　ARIES如何使用日志序列号减少恢复时REDO的工作量？使用图3-6所示的信息举例说明。至于何时把页写回磁盘，读者可以自己做出假设。

3.27　非潜入/强制缓冲区管理策略对检查点和恢复有何含义？

对下列多选问题选择正确的答案:

3.28　带有延迟更新的增量日志，隐含了恢复系统必须要

a．在日志中存储更新数据项的旧值。

b．在日志中存储更新数据项的新值。

c．在日志中存储更新数据项的旧值和新值。

d. 只在日志中存储开始事务和提交事务记录。

3.29　先写日志（WAL）协议所表示的是

a. 应该在任何日志操作之前写数据项。

b. 应该在写入实际数据之前先写对于操作的日志记录。

c. 所有日志记录应该在新的事务开始执行之前写入。

d. 从不需要把日志写到磁盘。

3.30　在使用基于延迟更新的增量日志策略中发生事务故障时，需要下述哪些操作？

a. 撤销操作。

b. 重做操作。

c. 撤销和重做操作。

d. 以上都不是。

3.31　对于带有即时更新的增量日志，事务的日志记录应当包含

a. 事务名称、数据项名称、数据项旧值和新值。

b. 事务名称、数据项名称、数据项旧值。

c. 事务名称、数据项名称、数据项新值。

d. 事务名称和数据项名称。

3.32　对于恢复期间的正确行为，撤销和重做操作必须是

a. 可交换的。

b. 关联的。

c. 幂等的。

d. 分布的。

3.33　当发生故障时，检查日志并且每个操作或被取消或被重做。这是一个问题，因为

a. 搜索全部日志时间开销太大。

b. 许多重做是不必要的。

c. (a)和(b)。

d. 以上都不是。

3.34　当使用基于恢复模式的日志时，下述哪个方法可以在提高性能的同时还可以提供恢复机制

a. 当每个事务提交时把日志记录写入磁盘。

b. 在事务执行过程中把相应的日志记录写入磁盘。

c. 直到多个事务提交之后，再把写日志作为批处理执行。

d. 从不需要把日志记录写入磁盘。

3.35　下述哪种情况可能发生级联回滚

a. 事务写那些只被某个已提交事务写过的数据项。

b. 事务写先前已被某个未提交事务写过的数据项。

c. 事务读先前已被某个未提交事务写过的数据项。

d. (b)和(c)。

3.36　为处理介质（磁盘）故障，必须

a. 对于DBMS，只在单用户环境下执行事务。

b. 保存数据库的冗余副本。

c. 从不撤销一个事务。

d. 以上都必须。

3.37　如果使用镜像方法把数据项刷新到磁盘，那么

a. 只在事务提交后把数据项写入磁盘。

b. 数据项被写到磁盘上的不同位置。

c. 在事务提交前把数据项写到磁盘。

d. 把数据项写到它被读出时相同的磁盘位置。

参考文献

（Bernstein et al.，1987）和（Papadimitriou，1986）介绍了并发控制和恢复的理论和原理。（Gray & Reuter，1993）全面介绍了并发控制、恢复及其他事务处理问题。

（Verhofstad，1978）是一本关于数据库系统恢复技术的教程和综述。（Haerder & Reuter，1983）与（Bernstein et al.，1983）讨论了基于UNDO/REDO特性的分类算法。（Gray，1978）讨论了恢复以及为数据库实现操作系统的其他系统方面的问题。（Lorie，1977）、（Verhofstad，1978）和（Reuter，1980）讨论了镜像分页技术。（Gray et al.，1981）讨论了SYSTEM R的恢复机制。（Lockeman & Knutsen，1968）、（Davies，1972）和（Bjork，1973）是最早关于恢复技术的论文。在（Chandy et al.，1975）中讨论了事务的回滚。（Lilien & Bhargava，1985）讨论了完整性块的概念，并介绍了其在提高恢复效率中的作用。

（Jhingran & Khedkar，1992）分析了基于先写日志的恢复技术，并应用于ARIES系统中（Mohan et al.，1992a）。更新一些的关于恢复的著作包括补偿事务（Korth et al.，1990）和主存数据库恢复（Kumar，1991）。ARIES恢复算法（Mohan et al.，1992）在实际应用中非常成功。（Franklin et al.，1992）讨论了EXODUS系统的恢复。最近有两本书（Kumar & Hsu，1998）与（Kumar & Son，1998）详细讨论了恢复技术，并介绍了目前许多关系数据库产品中的恢复方法。

第二部分

Part 2

对象和对象-关系数据库

本部分内容

- 第 4 章　面向对象数据库的概念
- 第 5 章　对象数据库标准、语言和设计
- 第 6 章　对象-关系和扩展-关系系统

第4章

面向对象数据库的概念

本章和下一章将讨论面向对象的数据模型和面向对象的数据库系统[①]。传统的数据模型和数据库系统（如关系的、网状的和层次的），在许多传统的商业数据库应用所要求的数据库技术的发展中，取得了很大的成功。然而，在设计和实现更为复杂的数据库应用时，传统的数据模型和系统就暴露出了一些缺陷。例如，在设计和实现工程设计和制造（CAD/CAM和CIM[②]）数据库、科学实验数据库、电信数据库、地理信息系统数据库，以及多媒体数据库[③]的时候，传统的数据模型和系统往往不能满足这些复杂数据库应用的要求。这些新的应用具有与传统商业应用不同的要求和特征。例如对象的结构更加复杂，事务持续时间更长，需要存储图像或大文本项等新的数据类型，以及需要定义非标准的专门应用操作。为了满足这些复杂的应用需求，有人提出了面向对象的数据库。面向对象的方法为处理这些应用需求提供了很大的灵活性，使人们摆脱了传统数据库系统对数据类型和查询语言的限制。面向对象数据库的一个关键特征是：它不但能让设计者定义复杂对象的结构，还能定义可以应用于这些对象的操作。

创建面向对象数据库的另一个原因是：人们在开发应用软件时，越来越多地使用面向对象的程序设计语言。现在，数据库已经成为许多软件系统的基本组成部分，但如果使用C++、SMALLTALK或JAVA这样的面向对象编程语言开发软件，很难把传统数据库嵌入到这种面向对象的应用软件中。所以人们设计出面向对象的数据库，这样就可以把数据库和使用面向对象编程语言开发的软件直接或无缝地集成在一起。

目前，关系DBMS的生产厂商也认识到了增加数据建模特征的必要性，所以在关系数据库系统的新版本里，增添了许多面向对象数据库的特征，这就导致了这些系统具有对象-关系DBMS或扩展-关系DBMS的特征（见第6章）。关系DBMS SQL标准的最新版本将会包含一些面向对象数据库的特征。

虽然已经创建了许多实验原型和商业面向对象数据库系统，但由于关系数据库和对象-关系系统还未流行，因而它们并没有被广泛应用。实验原型包括：MCC[④]开发的ORION系统、Texas Instruments开发的OpenOODB、Hewlett-Packard开发的IRIS系统、AT&T 贝尔实验室[⑤]开发的ODE系统、布朗大学开发的ENCORE/ObServer。可用的商业面向对象数据库系统包括：GemStone系统的GEMSTONE/OPAL，Ontos开发的ONTOS，Objectivity有限公司的Objectivity，Versant对象科技的Versant，Object Design的ObjectStore，ARDENT软件公司的ARDENT[⑥]，以及POET软件公司的POET。以上列出的只是现已创建的面向对象数据库系统的实验原型和商业系统的一小部分。

---

① 人们经常把这种数据库系统称为**对象数据库**（Object Database）或**对象数据库管理系统**（ODBMS）。但是因为本章会讨论许多面向对象的概念，所以本章将会用术语面向对象代替对象。

② 计算机辅助设计/计算机辅助制造和计算机集成制造。

③ 多媒体数据库必须存储许多种多媒体对象，比如视频、音频、图像、图片和文档（见第8章）。

④ 得克萨斯州，奥斯汀市，微电子和计算机科技公司。

⑤ 现在称为朗讯科技。

⑥ 原来的O2科技的O2系统。

当面向对象的商业DBMS已经逐渐可用时，人们认识到建立标准面向对象模型和语言的必要性。由于采取正式程序通过标准认可通常需要好几年的时间，所以一个由面向对象DBMS的销售商和用户组成的协会，称为ODMG[①]，提出了ODMG-93标准（ODMG-93标准经过修订以后，最新的版本是ODMG 2.0）本书将在第5章中介绍ODMG标准的特点。

面向对象的数据库吸收了许多原本从面向对象编程语言中发展起来的概念[②]。在4.1节中，将探讨面向对象方法的起源，并讨论如何把面向对象的方法应用于数据库系统。随后，在4.2节到4.6节中，都将阐述面向对象数据库系统中使用的关键概念。4.2节讨论对象标识、对象结构和类型构造器。4.3节提出关于操作封装的概念和方法的定义，作为类声明的一部分，还将讨论为使对象持久地存储在数据库中的机制。4.4节阐述面向对象数据库的类型和类的层次和继承。4.5节简要概括在表示和存储复杂对象时可能发生的问题。4.6节将讨论其他一些概念，包括多态性、操作符重载、动态绑定、多重继承和选择性继承、对象的版本和配置。

本章主要介绍面向对象数据库的一般概念，第5章将用具体的示例（ODMG标准）说明如何实现这些概念。如果读者不想详细了解4.5节和4.6节介绍的内容，可以跳过这两节。

4.1　面向对象的概念

本节将简述面向对象数据库（object-oriented database，OODB）的历史和主要概念。4.2节到4.6节将会更详细地解释OODB的概念。术语面向对象，简写为*OO*或*O-O*，来源于面向对象的程序设计语言（OO programming language，OOPL）。目前OO概念已经广泛应用于数据库、软件工程、知识库、人工智能和计算机系统等领域。OOPL 可以溯源至 20 世纪 60 年代末提出的 SIMULA 语言。在SIMULA语言中，类的概念把类声明里对象的内部数据结构组织在一起。后来，研究人员提出了抽象数据类型的概念。抽象数据类型隐藏了内部数据结构，并指定了所有可能应用于这个对象的外部操作，这就产生了封装的概念。1970年Xerox PARC[③]开发出了Smalltalk编程语言。Smalltalk是一种最早显式结合了其他OO概念的语言之一，如加入了消息传递和继承的概念。Smalltalk是一种纯OO编程语言，这意味着它是明确地被设计成面向对象的编程语言。与纯OO编程语言相对应的是混合OO编程语言，它是把OO概念融入某个现有编程语言的产物。C++就是OO概念与流行的C语言结合的产物，属于混合OO编程语言。

一个典型的**对象**（object）有两个组成部分：状态（值）和行为（操作）。因此，在某种程度上它很类似于程序设计语言中的程序变量，只不过对象通常具有复杂的数据结构和程序员定义的特定操作[④]。OOPL中的对象只在程序执行过程中存在，所以称为临时对象。而OO数据库可以延长对象的存在，把对象持久地存储起来，因此对象在程序结束之后仍然持续存在，并且能够在以后由其他程序获取并共享。换句话说，OO数据库永远在辅助存储器中存储持久的对象，并允许多个程序和应用共享这些对象。这需要结合数据库管理系统的其他已知特性，例如索引机制、并发控制和恢复机制等。一个OO数据库系统必须具有与一种或多种OO编程语言交互的接口，以便提供使对象持久化和共享对象的功能。

OO数据库的一个目标就是维持现实世界和数据库对象之间的直接对应关系，从而使对象不会丢失其完整性和标识，可以很容易地识别并对其进行操作。因此，OO数据库为每一个对象提供一个唯一的、由系统生成的对象标识符。可以把对象标识符与关系模型的主码作比较，在关系模型中，每个关系都必须有一个主码属性，主码属性的值唯一标识每一个元组。但是在关系模型中，如果主码

① Object Database Management Group，对象数据库管理组。
② 在语义数据模型和知识表达领域也发展了类似的概念。
③ 加利福尼亚州，帕洛阿尔托市，帕洛阿尔托研究中心。
④ 对象还有许多其他的特点，本章将讨论这些特点。

的值被改变，尽管这个元组仍然代表现实世界中的原有对象，但这个元组将会有一个新的标识。另外，在不同的关系中，现实世界对象对于码属性可以有不同的名，这就很难确定这些码是否表示同一个对象（例如，在某一个关系中可能用Emp_id代表对象的标识符，但是在另一个关系中可能会用Ssn代表对象的标识符）。

面向对象数据库的另一个特性是：对象可以具有任意复杂程度的对象结构，这使对象能够包含所有描述该对象的必要信息。与此不同，在传统的数据库系统中，复杂对象的信息常常分散在多个关系或记录中，从而丧失了真实世界对象与其数据库表示之间的直接对应关系。

在OOPL中，对象的内部结构包括**实例变量**的说明，实例变量中保存的值定义了对象的内部状态。因此，实例变量类似于属性的概念，但是实例变量可以被封装在对象里，也就是说，实例变量对外部用户来说是不可见的。实例变量也可以是任意复杂程度的数据类型。面向对象系统允许定义可以应用于特定类型对象的操作或函数（行为）。实际上，一些OO模型要求预先定义用户可以应用于对象的操作，也就是强制对对象进行封装。但是由于某些原因，大部分OO数据模型都放松了这一项严格的要求。首先，数据库的用户通常必须知道属性名，这样他们才能指定在属性上的选择条件，以检索特定的记录。其次，完全封装意味着必须为所有的简单检索提前定义好相应的操作，这就使表达特殊的查询变得很困难。

为了有利于封装，操作被定义为两个部分。第一部分称为操作的*签名或接口*，用来指明操作的名称和变元（或参数）。第二部分称为*方法或体*，用来指明操作的实现。可以通过给对象传递一条消息来调用操作，消息中包括操作名和参数。然后对象将执行该操作的方法。这样的封装允许对对象的内部结构进行修改，也允许对对象操作的实现进行修改，而且这些修改不会影响调用这些操作的外部程序。因此，封装提供了一种保证数据和操作之间独立性（见《数据库系统基础：初级篇》的第2章）的形式。

OO系统中的另一个关键概念是类型和类的层次与继承。这将允许新类型或类可以继承以前所定义类型或类的大部分结构与/或操作。因此，在OO系统中可以系统地指定对象类型。这样就更加便于增加系统中的数据类型，而且在创建新的对象类型时可以简便地重用已有的类型定义。

困扰早期OO数据库的一个问题是如何表示对象间的联系。有些人坚持早期完全封装的OO数据模型，这就导致了以下争议，即不应该显式地把联系表示出来，而是应该通过定义适当的方法来定位相关对象，从而来表示联系。然而，在具有许多联系的复杂数据库中，上述完全封装的方法并不适用，因为标识这样的联系并使它们对用户可见是很有用的。ODMG 标准已经认识到了把联系明确表示出来的必要性，所以它提出用一对反向引用来表示二元联系。也就是说，把与某个对象相关的对象的OID放在那个对象内，并维护参照完整性。本书将在第5章详细介绍这种方法。

一些OO系统还具有管理同一个对象的多个版本的能力，这个特性对于设计和工程应用是至关重要的。例如，如果一个对象的旧版本代表一个已经通过测试和鉴定的设计方案，那么应保存这个版本直到新版本通过测试和鉴定。一个复杂对象的新版本可能只包含了一部分新版本的组件，而其他的组件都没有改变过。除了允许版本变化外，OO数据库也应该允许模式演变。当类型声明发生变化时，或创建了新的类型或联系时，称为发生了模式演变。多版本和模式演变并非OODB特有的特点，所有理想的DBMS类型都应具备这两个特征[①]。

另一个OO概念是*操作符重载*（operator overloading），它指的是一个操作符可以被应用于不同对象类型的能力。在这种情况下，同一个操作符名可能对应好几个不同的操作实现，这取决于它所应用的对象类型。这个特性也称为*多态性*（polymorphism）。例如，一个计算几何对象面积的操作可能有不同的方法（实现），到底采用哪种实现依赖于几何对象的类型，也就是说依赖于这个对象是三角形、圆形还是矩形。这就要求在运行时，当知道了应用操作的对象类型后，用*滞后绑定*（late binding）

① 关系SQL 标准已经定义了几种模式演变操作，比如ALTER TABLE（见《数据库系统基础：初级篇》的8.3节）。

把操作绑定到适当的方法上。

本节简要概述了OO数据库的主要概念。4.2节到4.6节则更详细地讨论这些概念。

4.2　对象标识、对象结构和类型构造器

本节将首先讨论对象标识（Object Identity）的概念，然后介绍定义对象状态结构的典型结构操作。这些结构操作通常称为**类型构造器**（type constructor），它们定义了基本数据结构操作，可以结合这些操作来构成复杂对象结构。

4.2.1　对象标识

OO数据库系统为每个存储在数据库中的独立对象提供一个**唯一标识**。典型的唯一标识是一个唯一的、由系统生成的**对象标识符**（OID）。OID的值对于外部用户来说是不可见的，但是系统会在内部用这个值唯一地标识各个对象，并用这个值创建和管理内部对象引用。需要时，可以把OID指定为适当类型的程序变量。

OID必须具备的主要特性是**永远不变性**（immutable），也就是说，特定对象的OID值应该是固定不变的。这就保持了这个对象OID值所表示的现实世界对象的身份。因此，OO数据库系统必须具有生成OID，并维护OID永远不变性的机制，而且要求每个OID只被使用一次。也就是说，即使从数据库中删除了某个对象，也不应该把这个对象的OID再赋给另外一个对象。这两个性质意味着OID不应该依赖于对象的任何属性值，因为属性值可能会被修改或纠正。如果用对象在存储器中的物理地址作为OID，通常认为这种做法也是不适当的，因为对数据库进行物理重组之后，物理地址会有所改变。然而有些系统为了提高检索对象的效率，正是用物理地址作为OID的。如果对象的物理地址发生了变化，就在原有地址中放置一个间接指针，这个指针指向对象新的物理位置。通常使用长整型作为OID值，然后用某种形式的散列表把OID值映射到对象的物理地址上。

一些早期的OO数据模型要求把所有的一切都表示为对象，无论是一个简单的值还是一个复杂的对象。所以每一个基本值，比如整型、字符串或布尔值，都有一个OID。这就允许两个基本值有不同的OID。在某些情况下，这是很有意义的。例如，整数50可以表示重量（单位为千克），有时则可以表示一个人的年龄。可以创建两个具有不同OID的对象，但是这两个对象都代表整数值50。尽管从理论上讲这是一种很有用的模型，但是由于这种模型导致生成了太多的OID，所以不实用。因此，大多数OO数据库系统允许有对象和**值**（value）两种表示方法。每个对象必须有一个永远不变的OID，但是值没有OID，值只代表它自己。所以，通常把值存储在对象中，而且其他对象不能引用这个值。在有些系统中，如果需要的话，还可以创建没有相应OID的结构复杂的值。

4.2.2　对象结构

在OO数据库中，可以使用一些**类型构造器**（type constructor），从其他对象（或其他值）构造一个复杂对象的状态（当前值）。表示这些对象的一种形式化方法是把每个对象看作一个三元组（i，c，v），其中i是唯一的对象标识符（OID），c是一个类型构造器[①]（也就是关于如何构造对象状态的说明），v是对象的状态（当前值）。典型的数据模型通常包括若干种类型构造器。3种最基本的构造器是**原子**（atom）、**元组**（tuple）和**集合**（set）。其他常用的构造器还包括**列表**（list）、**包**（bag）和**数组**（array）。原子构造器表示所有的基本原子值，如整数、实数、字符串、布尔值和其他系统支持的基本数据类型。

一个对象（i，c，v）中对象状态v可根据构造器c解释。如果c = atom，状态（值）v是该系统支持的基本值域中的一个原子值。如果c = set，状态（值）v是一个对象标识符的集合$\{i_1,i_2,\cdots,i_n\}$，它是相同类型对象的OID。如果c = tuple，状态（值）v是一个形式为$<a_1:i_1,a_2:i_2,\cdots,a_n:i_n>$的元组，其中$a_j$

① 这里的类型构造器不同于C++和其他OOPL中的构造器操作。

是一个属性名[①]，i_j是一个OID。如果c = list，状态（值）v是相同类型对象OID的有序列表$[i_1,i_2,\cdots,i_n]$。列表类似于有序列表中只有OID的集合，所以有“列表中的第一个对象，第二个对象或第j个对象”的说法。如果c = array，状态（值）v就是对象标识符的一维数组。数组和列表的主要区别是：list中可以有任意个元素，但是数组有最大长度限制。集合和包[②]的区别在于：一个集合中的所有元素必须是不同的，但是一个包中可以有重复的元素。

这种对象模型允许集合、列表、元组以及其他构造器的任意嵌套。非原子类型对象的状态可以通过其对象标识符引用其他对象。因此，只有当对象的类型是atom时，状态中出现的才是实际的值[③]。

类型构造器set、list、array和bag称为**汇集**（collection）类型或**批量**（bulk）类型，以便把它们与基本类型和tuple类型相区别。汇集类型的主要特点是：这种对象的状态是对象的汇集，而且这些对象可以是无序的（如set和bag），也可以是有序的（如list和array）。由于**元组**类型构造器对应于C和C++编程语言中的struct结构，所以通常被称为**结构化类型**。

示例1：一个复杂对象。这里采用前述的模型来表示《数据库系统基础：初级篇》中图5-6所示的关系数据库中的一些对象。一个对象被定义为一个三元组（OID，类型构造器，状态），并且可用的类型构造器是atom、set和tuple。用i_1，i_2，i_3，…表示系统生成的唯一标识符。考虑下列对象：

$o_1 = (i_1,$ atom, ‘Houston’)
$o_2 = (i_2,$ atom, ‘Bellarie’)
$o_3 = (i_3,$ atom, ‘Sugarland’)
$o_4 = (i_4,$ atom, 5)
$o_5 = (i_5,$ atom, ‘Research’)
$o_6 = (i_6,$ atom, ‘1988-05-22’)
$o_7 = (i_7,$ set, $\{i_1, i_2, i_3\})$
$o_8 = (i_8,$ tuple, <Dname:i_5, Dnumber:i_4, Mrg:i_9, Location:i_7, Employees:i_{10}, Project:i_{11}>)
$o_9 = (i_9,$ tuple, <Mgr:i_{12}, Manager_start_date:i_6>)
$o_{10} = (i_{10},$ set, $\{i_{12}, i_{13}, i_{14}\})$
$o_{11} = (i_{11},$ set, $\{i_{15}, i_{16}, i_{17}\})$
$o_{12} = (i_{12},$ tuple, <Fname:i_{18}, Minit:i_{19}, Lname:i_{20}, Ssn:i_{21}, …, Salary:i_{26}, Supervisor:i_{27}, Dept:i_8>)
…

前六个对象（o_1~o_6）代表原子值。数据库中每一个不同的常量原子值都有许多类似的对象[④]。对象o_7是一个值的集合对象，代表第5部门位置的集合；集合$\{i_1,i_2,i_3\}$是对原子对象的引用，这些原子对象是{‘Houston’,‘Bellarie’,‘Sugarland’}。对象o_8是一个元组对象，代表第5部门本身，具有Dname、Dnumber、Mgr、Locations等属性。前两个属性Dname和Dnumber用原子对象o_5和o_4作它们的值。属性Mgr用元组对象o_9作它的值，o_9有两个属性。属性Manager的值是OID等于i_{12}的对象，i_{12}表示管理第5部门的雇员“John B. Smith”，而Manager_start_date的值是另一个原子对象，这个原子对象的值是一个日期。o_8的属性Employees的值是一个集合对象，这个集合对象的OID=i_{10}，i_{10}的值是为DEPARTMENT工作的雇员的对象标识符的集合（对象i_{12}，加i_{13}和i_{14}，这里没有表示）。类似地，o_8的属性Projects的值是一个OID=i_{11}的集合对象，i_{11}的值是第5部门管理的项目的对象标识符的集合（对象i_{15}、i_{16}和i_{17}，这里没有表示）。OID=i_{12}的对象代表雇员“John B. Smith”的所有原子属性和引用OID=i_{27}的雇员对象的属性Supervisor，以及引用OID=i_8的部门对象的属性Dept。“John B. Smith”的所有原子

---

① 在OO术语里也称为实例变量名。
② 也称为多集合。
③ 就像本章前面指出的一样，为每个值都生成唯一的系统标识符是不现实的。所以实际系统既允许OID，也允许结构化的值。用构造对象的类型构造器就可以构造结构化的值，只是值没有OID。
④ 如果这个模型是直接（directly）实现的，由于使用了太多的对象标识符，这些原子对象可能会引起一个问题。

属性Fname，Minit，Lname，Ssn，…，Salary分别引用原子对象i_{18}，i_{19}，i_{20}，i_{21}，…，i_{26}。OID = i_{27}的雇员对象代表管理“John B. Smith”的“James E. Berg”。OID = i_8的部门对象代表“John B. Smith”所在的部门号5。

在这种三元模型中，对象可以表示为一个图结构，可以通过递归地应用类型构造器来构造。首先为对象o_i本身创建一个结点来构造表示对象o_i的图。o_i的结点上标以OID和对象构造器c。然后在图中为每一个基本原子值创建一个结点。如果对象o_i有一个原子值，就直接画一条从o_i结点到代表这一原子值的结点的弧。如果对象的值是被构造的，就画一条从对象结点到代表被构造值的结点的弧。图4-1是前面给出的示例中DEPARTMENT对象o_8的构造器图。

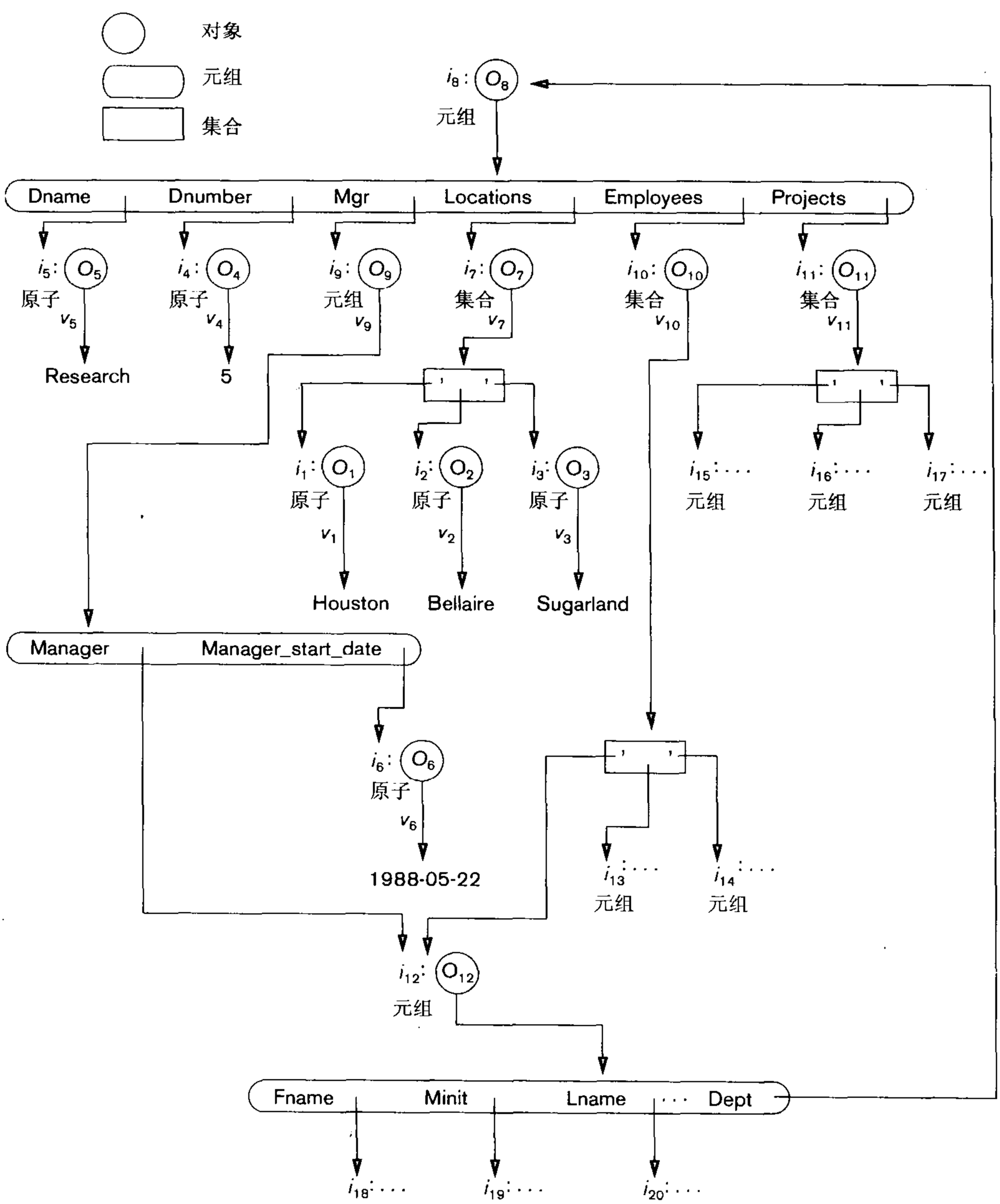


图4-1　复杂对象DEPARTMENT的图形表示

前面给出的模型对于比较两个对象的状态是否相等有两种定义方法。如果表示两个对象状态的图在各个方面都相同（这包括每一层次的OID），则称这两个对象具有**同一状态**（或深相等，deep equality）。另一种较松散的相等定义称为**相等状态**（或浅相等，shallow equality）。在这种情况下，两个对象的图形结构必须相同，而且图中所有相应的原子值也必须相同。然而，两幅图中一些相应的内部结点可能会有不同*OID*的对象。

示例2：同一对象与相等对象。下面举例说明比较对象状态是否相等的不同方法。考虑下列对象o_1、o_2、o_3、o_4、o_5和o_6：

$o_1 = (i_1,\ \text{tuple},\ <a_1:i_4,\ a_2:i_6>)$
$o_2 = (i_2,\ \text{tuple},\ <a_1:i_5,\ a_2:i_6>)$
$o_3 = (i_3,\ \text{tuple},\ <a_1:i_4,\ a_2:i_6>)$
$o_4 = (i_4,\ \text{atom},\ 10)$
$o_5 = (i_5,\ \text{atom},\ 10)$
$o_6 = (i_6,\ \text{atom},\ 20)$

因为对象o_1和o_2在原子层上状态相同，但值是通过不同的对象o_4和o_5达到，所以对象o_1和o_2具有相等状态。尽管对象o_1和o_3有不同的OID，但是o_1和o_3的状态是同一的。与此类似，尽管o_4和o_5的状态是相同的，但是它们的OID是不同的，所以对象o_4和o_5是相等的而不是同一的。

4.2.3　类型构造器

将**对象定义语言**（object definition language，ODL）[①]与前面阐述的类型构造器结合在一起，可以为特定的数据库应用定义对象类型。在第5章将会讨论ODMG的标准ODL，但是本小节只使用更为简单的符号逐步阐述这些概念。人们可以用类型构造器为OO数据库模式定义数据模型。在4.3节将会见到把操作（或方法）的定义融入到OO模式的方法。图4-2展示了如何声明对应于图4-1中的对象实例的EMPLOYEE和DEPARTMENT类型。图4-1把DATE类型定义为原子值，而图4-2却把DATE类型定义为元组。此处用关键字tuple、set和list表示类型构造器，用标准数据类型（整型、字符串、符点型等）作为原子类型。

```
define type EMPLOYEE
   tuple ( Fname:        string;
           Minit:        char;
           Lname:        string;
           Ssn:          string;
           Birth_date:   DATE;
           Address:      string;
           Sex:          char;
           Salary:       float;
           Supervisor:   EMPLOYEE;
           Dept:         DEPARTMENT;

define type DATE
   tuple ( Year:         integer;
           Month:        integer;
           Day:          integer; );

define type DEPARTMENT
   tuple ( Dname:        string;
           Dnumber:      integer;
```

图4-2　用类型构造器定义对象类型EMPLOYEE、DATE和DEPARTMENT

① 对应于数据库系统的数据定义语言（Data Definition Language，DDL）（见《数据库系统基础：初级篇》的第2章）。

```
Mgr:          tuple (  Manager:    EMPLOYEE;
                       Start_date: DATE;     );
Locations:    set(string);
Employees:    set(EMPLOYEE);
Projects      set(PROJECT);    );
```

图4-2（续）

引用其他对象的属性是对其他对象的**基本引用**，因此可以用来表示对象类型间的联系。EMPLOYEE中的Dept和DEPARTMENT中的Projects就是这样的属性。例如，EMPLOYEE中的属性Dept的类型是DEPARTMENT，用来引用一个特定的DEPARTMENRT对象（也就是EMPLOYEE所工作的部门）。这个属性的值应该是某个Department对象的OID。可以单向地表示一个二元联系，也可以使用反向引用表示二元联系。例如，DEPARTMENRT的属性Employees的值是EMPLOYEE类型对象的引用集合（也就是OID集合），这些EMPLOYEE对象是为该部门工作的雇员。这个联系的反向引用是EMPLOYEE的属性Dept。我们将在第5章说明ODMG标准为了确保应用的一致性，允许把反向引用明确声明为联系的属性。

4.3　操作、方法和持久性的封装

封装是OO语言和OO系统的主要特征之一。封装还与编程语言中抽象数据类型和信息隐藏的概念有关。在传统的数据库模型和数据库系统中，因为数据库对象的结构对用户和外部程序来说通常是可见的，所以并没有使用封装这个概念。在传统数据库模型中，一些标准的数据库操作可以应用于所有类型的对象。以关系模型为例，选择、插入、删除和修改元组这样的操作具有普遍性，可以应用于数据库中的任一关系。对于用户以及通过这些操作访问关系的外部程序来说，关系和它的属性都是可见的。

4.3.1　通过类操作指定对象行为

信息隐藏和封装的概念可以应用于数据库对象。主要的思想是：基于**操作**定义对象类型的**行为**，这些操作可以从外部应用于这种类型的对象。对象的内部结构被隐藏起来，只有通过一些预先定义过的操作才能访问对象。有些操作可以用来创建（插入）对象或销毁（删除）对象；有些操作可以更新对象的状态；还有一些操作可以用来检索对象的内部状态或执行某些运算。剩下的其他操作可能会混合执行检索、计算和更新等操作。一般情况下，通用的编程语言提供了定义操作的灵活性和能力，可以用这种语言指明操作的**实现**。

对象的外部用户只能看到对象类型的**接口**，接口定义了每个操作的名称和变元（参数），操作的实现对外部用户来说是隐藏的，操作的实现包括对象内部数据结构的定义和访问这些结构的操作的实现方法。在OO术语中，每个操作的接口部分被称为**签名**，而操作的实现部分被称为**方法**。通常的情况是通过给对象发送**消息**来调用方法。需要注意：作为执行方法的一部分，该对象可以向另一个对象发送后续消息，并且可以用消息机制向外部环境或其他对象传送返回值。

对于数据库应用来说，必须对所有对象进行完全封装的要求过于严格。放宽这个要求的一种方法是把对象的结构划分为**可见**属性和**隐藏**属性（实例变量）。可见属性可以直接访问，以供外部操作符或高级查询语言读取。而对象的隐藏属性则被完全封装起来，只有通过预先定义的操作才能访问对象的隐藏属性。大多数OODBMS允许高级查询语言访问可见属性。在第5章将讨论OQL查询语言，这是OODB提出的一种标准查询语言。

在大多数情况下，更新对象状态的操作会被封装起来。由于许多OO数据模型很少预先定义模式的完整性约束，所以封装是一种定义对象更新语义的方法。每种类型的对象都将其完整性约束编码写入创建、删除和更新对象的方法中，即显式地编写代码来检查违反约束的情况并处理异常。在这

种情况下，所有更新操作都会被封装起来。最近ODMG标准的ODL允许指定例如码和反向联系（参照完整性）这样的约束，以便系统自动实施这些约束（见第5章）。

术语**类**通常涉及对象类型定义和对这种对象类型进行操作的定义[①]。图4-3是扩展了图4-2中的类型定义的示意图，在类定义中加入了操作，从而定义了完整的类。每个类都声明了一些操作，而且类定义中包括了每个操作的签名（接口）。但是必须用编程语言在其他地方定义每个操作的方法（实现）。典型的操作包括**对象构造器**（object constructor）操作，这是创建对象的操作；而**析构器**（destructor）操作，这是销毁对象的操作。还可以声明一些**对象修改器**（object modifier）操作来修改对象的各种属性的状态（值）。还有一些操作可以**检索**关于对象的信息。

通常地，用**点符号**把一个操作应用于一个对象。例如，如果d引用一个DEPARTMENT对象，可以用d.no_of_emps的形式调用no_of_emps操作。类似地，如果写成d.destroy_dept，则d引用的对象会被销毁（删除）。唯一例外的是构造器操作，构造器操作返回的是一个新DEPARTMENT对象的引用。因此，构造器操作通常把类的名字作为默认的操作名，虽然图4-3中并没有使用构造器操作[②]。还可以用点符号引用对象的属性，例如，写为d.Dnumber和d.Mgr.Start_date。

```
define class EMPLOYEE
   type tuple (   Fname:        string;
                  Minit:        char;
                  Lname:        string;
                  Ssn:          string;
                  Birth_date:   DATE;
                  Address:      string;
                  Sex:          char;
                  Salary:       float;
                  Supervisor:   EMPLOYEE;
                  Dept:         DEPARTMENT;   );
   operations     age:          integer;
                  create_emp:   EMPLOYEE;
                  destroy_emp:  boolean;
end EMPLOYEE;

define class DEPARTMENT
   type tuple (   Dname:        string;
                  Dnumber:      integer;
                  Mgr:          tuple (  Manager:      EMPLOYEE;
                                         Start_date:   DATE;     );
                  Locations:    set(string);
                  Employees:    set(EMPLOYEE);
                  Projects      set(PROJECT);    );
   operations     no_of_emps:   integer;
                  create_dept:  DEPARTMENT;
                  destroy_dept: boolean;
                  assign_emp(e: EMPLOYEE): boolean;
                  (* 把一个雇员分配到该部门*)
                  remove_emp(e: EMPLOYEE): boolean;
                  (* 从该部门中开除一个雇员*)
end DEPARTMENT;
```

图4-3　在EMPLOYEE和DEPARTMENT的定义中加了操作

① 这里类的定义类似于C++编程语言里的类。ODMG标准在类上还增加了接口（见第5章）。EER模型的术语类指的是对象类型和所有属于这个类型的对象的集合（见《数据库系统基础：初级篇》的第4章）。

② C++编程语言里，构造器操作和析构器操作都有默认的操作名。以类EMPLOYEE为例，默认的构造器操作名是EMPLOYEE，默认的析构器操作名是~EMPLOYEE。通常使用new操作创建新的对象。

4.3.2　通过命名和可达性指定对象的持久性

OODBMS通常和OOPL紧密配合。OOPL可用于指定方法实现及其他应用代码。创建对象的典型方法是执行某个应用程序，以及通过调用对象的构造器操作。数据库并不会永久存储所有的对象。**临时对象**（transient object）在程序执行期间存在，一旦程序终止就会消失。而**持久对象**（persistent object）被保存在数据库中，在程序终止后仍然存在。使一个对象具有持久性的典型机制是命名和可达性。

命名机制就是赋给对象一个唯一的、永久的名字，创建这个对象的程序和其他程序可以通过这个名字检索这个对象。可以通过程序中专用的语句或操作给出持久的对象名，如图4-4所示。在一个特定的数据库中，所有赋给对象的持久名字必须是唯一的、互不重复的。因此，命名持久对象可以作为数据库的**入口点**（entry point），用户和应用可以通过这个入口点开始访问数据库。显而易见，大型数据库中的对象可能成千上万，对一个大型数据库中的所有对象进行命名是不现实的。所以通常采用称为**可达性**的第二种机制，以使大多数对象具有持久性。可达性机制的工作是：使得从一些持久对象出发可以到达某个对象。如果在对象图中有一系列从对象A到对象B的引用，就说对象B是从对象A可以达到的。例如，在图4-1中的所有对象是从对象o_8可以达到的，因此，如果使o_8具备持久性，那么图4-1中的所有其他对象也都具有持久性。

如果首先创建一个已命名的持久对象N，而其状态是某个类C的对象的集合或列表，只要把类C的对象加入到这个集合或列表中，就能使它们从N可达到，从而使C的对象也是持久的。因此，N定义了类C对象的**持久汇集**（persistent collection）。例如，可以定义一个类DEPARTMENT_SET（见图4-4），这个类的对象的类型是set(DEPARTMENT)[①]。

```
define class DEPARTMENT_SET:
    type set (DEPARTMENT);
    operations add_dept(d: DEPARTMENT):  boolean;
          (* 把一个部门加到DEPARTMENT_SET对象 *)
               remove_dept(d: DEPARTMENT): boolean;
          (* 从DEPARTMENT_SET对象中删除一个部门  *)
               create_dept_set:      DEPARTMENT_SET;
               destroy_dept_set:     boolean;
end DepartmentSet;
. . .

persistent name ALL_DEPARTMENTS: DEPARTMENT_SET;
(* ALL_DEPARTMENTS是一个被命名过的、DEPARTMENT_SET对象  *)
. . .

d:= create_dept;
(* 在变量d中创建一个新的DEPARTMENT对象              *)
. . .

b:= ALL_DEPARTMENTS.add_dept(d);
(* 把d加到持久集合ALL_DEPARTMENTS，使d成为持久对象    *)
```

图4-4　通过命名和可达性创建持久对象

假设创建了一个类DEPARTMENT_SET的对象，而且假设这个对象被命名为ALL_DEPARTMENTS，因而这个对象成为持久对象，如图4-4所示。只要用操作add_dept把任一DEPARTMENT对象加入到ALL_DEPARTMENTS的集合中，从ALL_DEPARTMENTS出发就可达到该对象，那么该DEPARTMENT对象就具有了持久性。由于这个ALL_DEPARTMENTS对象保存了所有DEPARTMENT类

① 在第5章可以看到，ODMG ODL语法用set<DEPARTMENT>代替了set(DEPARTMENT)。

型的持久对象，所以通常称ALL_DEPARTMENTS对象为类DEPARTMENT的**外延**（extent）。我们将会在第5章见到，ODMG DDL标准为模式设计者提供了一种选择，即可以把命名一个类外延作为类定义的一部分。

请注意传统数据库模型和OO数据库在这方面的区别。传统数据库模型，比如关系模型或EER模型，所有的对象都被假设是持久的。因此，如果在EER模型中定义了一个实体类型或类，例如EMPLOYEE，那么它表示EMPLOYEE的类型声明和一个包括所有EMPLOYEE对象的持久集合。在OO方法中，EMPLOYEE的类声明只指定了这个对象类的类型和操作。如果要使EMPLOYEE对象持久化，用户必须分别定义一个类型是set(EMPLOYEE)或list(EMPLOYEE)的持久对象，令这个对象的值是所有引用持久性EMPLOYEE对象的汇集，如图4-4所示[①]。实际上，只要用户愿意，可以在同一个类定义中定义好几个持久汇集。这样就允许临时对象和持久对象遵循相同的ODL或OOPL类型声明和类声明了。如果有必要，同一个类定义通常也可以定义多个持久汇集。

4.4　类型、类层次和继承

OO数据库系统的另一个主要特征是允许类型层次和继承。数据库中的类型层次结构通常隐含了相应层次中类型的外延上存在的某种约束。首先，在4.4.1节中将讨论类型层次，然后在4.4.2节中讨论对类外延的约束。在本节中将使用一个与前面所述不同的OO模型。由于属性和操作都可以被继承，所以这个模型把属性和操作同等看待。在第5章将讨论ODMG标准的继承模型，它不同于在此所讨论的模型。

4.4.1　类型层次和继承

在大多数数据库应用中，有许多属于同一个类型或类的对象。因此，如同其他数据库系统那样，OO数据库必须提供基于对象类型对其进行分类的功能。不过在OO 数据库中还有一个要求，即系统允许基于其他预定义类型来定义新的类型，这就引出了**类型层次**（type hierarchy）或**类层次**（class hierarchy）的概念。某些系统支持用户分别定义类型层次。

定义一个类型的典型方法是：首先赋予该类型一个类型名，然后为该类型定义一些属性（实例变量）和操作（方法）[②]。

在某些情况下，属性和操作一起被称为函数，因为属性类似于带有零个变元的函数，所以可以使用函数名来引用属性的值或引用操作（方法）的结果值。在本节中，我们使用术语函数来引用一个对象类型的属性和操作，因为在对继承的基本介绍中，属性和操作的处理是类似的[③]。

定义一个类型的最简单的形式就是给出**类型名**，然后列出这个类型的可见（公共）**函数**的名字。为了简化讨论，本节用下面这种格式指定类型（在此没有指明函数的变元）：

TYPE_NAME: function, function, · · ·, function

例如，描述一个PERSON特征的类型可以用下面这种方式定义：

PERSON[人]:Name[姓名],Address[地址],Birth_date [出生日期],Age[年龄],Ssn [社会保险号]

在PERSON类型中，Name、Address、Ssn和Birth_date函数可以作为存储属性实现，而Age函数可以作为方法实现，这个方法的功能就是根据属性Birth_date的值和当前日期计算出Age的值。

当设计者或用户必须创建一个新的类型，而且既要求这个类型类似于已经定义过的类型，又要求与其不完全相同时，那么引入**子类型**（subtype）这个概念是很有用的。子类型将继承预先定义过

① 有些系统，比如POET，自动为每一个类创建类外延。

② 本章将用术语类型和类表示同样的事物，所以对象的类型也具有属性和操作。

③ 在第5章里读者将会看到带函数的类型类似于ODMG ODL采用的接口。

的类型的所有函数，而那个预先定义过的类型称为**超类型**（supertype）。例如，假设用下面这种方式定义两个新的类型EMPLOYEE和STUDENT：

EMPLOYEE[雇员]：Name,Address,Birth_date,Age,Ssn,Salary[薪水],Hire_date[雇用日期],Seniority[资历]

STUDENT[学生]：Name,Address,Birth_date,Age,Ssn,Major[专业],Gpa[平均学分]

由于STUDENT和EMPLOYEE包含了为PERSON定义的所有函数，以及一些为它们各自定义的其他函数，因此可以将它们声明为PERSON的子类型。STUDENT和EMPLOYEE都继承了原先为PERSON定义的函数Name、Address、Birth_date、Age、Ssn。对STUDENT来说，它只需定义新的（局部）函数Major和GPA，这两个函数不是继承而来的。Major可以被定义为一个存储属性；而GPA可能被作为一个方法实现，这个方法的功能是访问存储（隐藏）在每一个STUDENT对象中的专有属性Grade的值，来计算学生的平均学分。对EMPLOYEE来说，Salary和Hire_date函数可能是存储属性；而Seniority可能是方法，其功能是根据Hire_date的值计算Seniority。

在定义类型的时候必须把所有的函数定义为属性或方法，并且实现这些函数。但是在定义子类型的时候，子类型已经继承了预先定义过的类型的所有函数及函数实现。所以只需要定义和实现那些子类型专用或**局部**（local），而在超类型中没有实现的函数。所以，可以用下面这种方式声明EMPLOYEE和STUDENT：

EMPLOYEE **subtype-of** PERSON: Salary, Hire_date, Seniority

STUDENT **subtype-of** PERSON: Major, Gpa

通常，子类型不但包括所有在超类型中定义的函数，还包括一些只有在子类型中定义的其他函数。所以，可以生成**类型层次**（type hierarchy），用这个类型层次表示系统声明的所有类型间的超类型/子类型联系。

再来考虑一个描述平面几何对象的类型，可以定义如下：

GEOMETRY_OBJECT[集合对象]:Shape[形状],Area[面积],Referece_point[基准点]

GEOMETRY_OBJECT类型把Shape作为属性来实现（Shape定义域可以是一个枚举类型，值为“triangle”、“rectangle”、“circle”等）；而把Area作为一个方法，用这个方法计算面积。Reference_point指定了确定对象位置的点的坐标。现在假设要定义如下一些GEOMETRY_OBJECT类型的子类型：

RECTANGLE[矩形] subtype-of GEOMETRY_OBJECT:Width[宽],Height[高]
TRIANGLE[三角形] subtype-of GEOMETRY_OBJECT:Side1[边1],Side2[边2],Angle[夹角角度]
CIRCLE[圆形] subtype-of GEOMETRY_OBJECT:Radius[半径]

注意，由于计算矩形、三角形和圆形的面积的过程各不相同，所以每个子类型的操作Area可能会有不同的实现方法。同理，每一种子类型的属性Referece_point[基准点]可能有不同的含义：RECTANGLE和CIRCLE对象的基准点是中心，而TRIANGLE的基准点是两条给定边之间的顶点。在有些OO数据库系统中，允许不同的子类型对继承的函数**重新命名**（renaming），以便使函数的名字更贴近函数的含义。

另外一种声明这3个子类型的方法是：指定属性Shape的值，把指定的值作为条件，每个子类型的对象都必须满足这个条件。如下所示：

RECTANGLE **subtype-of**
　GEOMETRY_OBJECT (Shape='rectangle'): Width, Height

TRIANGLE **subtype-of**
　GEOMETRY_OBJECT (Shape='triangle'): Side1, Side2, Angle

CIRCLE **subtype-of** GEOMETRY_OBJECT (Shape='circle'): Radius

在这里，只有那些Shape值等于“rectangle”的GEOMETRY_OBJECT对象才属于子类型

RECTANGLE，其他两个子类型也类似。这3个子类型都继承了超类型GEOMETRY_OBJECT的所有函数，但是每个子类型在Shape之上都各有特定的值。

注意，类型定义只是描述对象，其本身并不生成对象。类型定义仅仅是声明某些类型，而每一个类型的函数的实现作为这个声明的一部分被指定。在数据库应用中，每种类型都有许多对象。一旦创建了某个对象，这个对象通常属于一种或多种预先定义过的类型。比如，一个CIRCLE对象既属于类型CIRCLE，也属于类型GEOMETRY_OBJECT（通过继承）。每个对象还会成为一个或多个对象的持久汇集（或外延）的成员，一般会将这些对象汇集与对数据库应用有意义的对象组一起使用。

4.4.2　对应于类型层次的外延约束[①]

在大多数OO数据库中，在外延中对象的汇集都有相同的类型或类。然而，具有相同类型或类并不是必要条件。以Smalltalk为例，它被称为无类型（typeless）OO语言，允许一个对象的汇集包含不同类型的对象。当其他非面向对象的无类型语言，例如LISP，被扩展成具有OO概念的语言时，也可以使一个对象的汇集中包含不同类型的对象。然而，由于大部分OO数据库支持类型，因此假定在本节下面的内容中，**外延**是相同类型对象的汇集。

在数据库应用中，通常每一种类型或超类型都有一个与之相关联的外延，这个外延包含了所有属于这个类型或超类型的持久对象的汇集。在这种情况下，约束是：如果一个对象属于某个子类型的外延，这个对象就必须属于该子类型的超类型的外延。有些OO数据库系统有一种预定义系统类型（称为ROOT类或OBJECT类），这个类型的外延包含了系统中的所有对象[②]。

因此，分类的过程就是把对象指派到另外的子类型中，这些子类型对应用来说是有意义的。分类的结果是创建了该系统的**类型层次**或**类层次**。所有的系统定义的外延和用户定义的外延都直接或间接是类OBJECT外延的子集。在ODMG模型（见第5章）中，用户有时可以定义每种类（类型）的外延，有时却不可以，这依赖于应用。

在大多数OO系统中，持久与临时对象和汇集之间有明显的区别。**持久汇集**里是永久性存储在数据库中的对象，因此持久汇集可以被多个程序访问和共享。而**临时汇集**只在程序执行期间短暂存在，当程序结束时就消失了。例如，在程序中可以创建一个临时汇集，然后把从某个持久汇集中查询选择出来的对象复制到这个临时汇集中。这个临时汇集中的对象和持久汇集中的对象具有相同的类型。程序可以操纵临时汇集中的对象，一旦程序终止，临时汇集就不复存在。通常，对绝大多数汇集，无论是临时的还是持久的，都包含着相同类型的对象。

需要注意，4.2节中讨论的类型构造器，允许一个对象的状态是对象的汇集。因此，如果汇集对象的类型是基于集合构造器，那么它就可以定义多个汇集，每个汇集对应于一种对象。集合值对象本身是另一个汇集的成员。这就允许多级分类模式，在多级分类模式中，一个汇集中的对象的状态可以是不同类的对象汇集。

在第5章中将会见到类型继承和外延继承约束在ODMG模型中的区别。ODMG把类型继承称为接口继承，并用符号“:”来表示；用关键字EXTEND表示外延继承约束。

4.5　复杂对象

开发OO系统的一个主要原因是为了能够表示复杂对象。复杂对象主要分为两类：结构化的和非结构化的。结构化复杂对象由组件组成，通过以各种方式递归地使用类型构造器就可以定义结构化的复杂对象。非结构化的复杂对象通常是需要大量存储空间的数据类型，例如表示图像或大文本对

① 本书的第二版用类层次来描述类外延约束。因为类有太多的含义，所以这一版采用了类外延这一说法。这也与ODMG的术语一致（见第5章）。

② ODMG模型称这个类为OBJECT（见第5章）。

象的数据类型。

4.5.1　非结构化复杂对象和类型可扩展性

DBMS提供了**非结构化复杂对象**，从而允许存储和检索数据库应用所需的大对象。典型的大对象就是位图图像和长文本串（如文档），这些大对象被称为**二进制大对象**（binary large object，BLOB）。字符串也称为**字符型大对象**（character large object，CLOB）。由于DBMS不知道这些BLOB的结构，所以说它们是非结构化的。只有使用这些大对象的应用才能解释它们的含义。例如，应用程序可能有显示图像的函数，或者在长字符串中搜索某个关键字语句、字段的函数。这些对象需要很大的存储区域，而且不是传统 DBMS 所提供的标准数据类型，因而认为是复杂的对象。由于BLOB通常很大，所以DBMS可以先检索BOLB对象的一部分，并把这部分先提供给应用程序，直至完成对整个对象的检索。DBMS还可以使用缓冲和高速缓存技术，在应用程序访问大对象之前预先获取大对象的部分内容。

DBMS 软件不能直接处理基于这些对象的值的选择条件和其他操作，除非应用提供了必要的代码来完成选择所需的比较操作。在OODBMS中，可以通过为无法解释的对象定义一个新的抽象数据类型，并提供选择、比较和显示这种对象的方法，来实现上述情形。例如，考虑一个二维位图图像对象。假设应用要从这类对象的汇集中，选择出包括某种特定模式的对象。在这种情况下，用户必须提供模式识别程序作为位图类型对象的一个方法。然后OODBMS就可以检索数据库中的对象，在对象上应用模式识别方法，判断这个对象是否包括了定义的模式。

因为OODBMS允许用户创建新类型，也因为每个类型都包括结构和操作，所以可以认为OODBMS是一个**可扩展类型系统**（extensible type system）。通过定义新类型的结构和操作，可以创建新类型的库，包括复杂类型。然后应用程序就可以使用或修改这些类型，要创建复杂类型的话，可以通过创建库里提供的类型的子类型来实现。然而，DBMS必须提供基本的存储和检索这些大对象的能力，这样才能对大对象进行高效的操作。许多OODBMS能够存储和检索非结构化大对象，例如字符串或位串，. 这使DBMS有点类似于解释这些非结构化大对象的应用程序。近来，关系DBMS和扩展的关系DBMS都具备了存储和检索非结构化大对象的能力。

4.5.2　结构化复杂对象

结构化复杂对象（structured complex object）与非结构化复杂对象的区别在于，结构化复杂对象的结构是通过重复应用OODBMS提供的类型构造器定义的。因此，对于 OODBMS 来说，对象结构已定义且为已知。以图4-1中所示的DEPARTMENT对象为例。在第一层里，对象的元组结构包括6个属性：Dname、Dnumber、Mgr、Locations、Employees和Projects。然而，其中只有2个属性Dname和Dnumber有基本值，其他4个属性都有复杂结构，由此建立了该复杂对象结构的第二层。这4个二层属性中的Mgr具有一个元组结构，其他的3个属性（Locations、Employees、Projects）都是集合结构。在第三层里，一个Mgr的元组值有2个属性：其中属性Manager_start_data是一个基本值，而属性Manager引用了一个有元组结构的employee对象。Locations集合是一个基本值的集合，但是集合Employees和Projects都是元组结构对象的集合。

每一层的复杂对象和它的组件之间都存在着两种类型的引用语义。第一种语义类型，称为**属主语义**（ownership semantics），指的是复杂对象的子对象被封装在复杂对象中，并被看作是复杂对象的组成部分。第二种语义类型，称为**引用语义**（reference semantics），指的是复杂对象的组件本身可能既是独立的对象，又是对复杂对象的引用。例如，可以认为属性Dname、Dnumber、Mgr和Locations属于DEPARTMENT，不过同时Employees和Projects都引用独立的对象。第一种语义类型也称为*is-part-of*或*is-component-of*联系；第二种语义类型描述的是两个独立对象之间的对等联系，所以也称为*is-associated-with*联系。构成复杂对象的is-part-of（属主语义）有一个特性：组件对象

被封装在复杂对象中，被看作内部对象状态的一部分。组件对象不需要有对象标识符，而且只能通过这一种对象的方法被访问。如果复杂对象被删除，它的组件对象也会被删除。另一方面，被引用的复杂对象却包含独立的对象，这些独立对象都有自己的标识和方法。当复杂对象要访问它的引用组件时，由于组件的方法并没有被封装在复杂对象中，所以复杂对象必须调用组件的适当方法。所以，引用语义代表独立对象间的联系。另外，一个组件对象可能会被多个复杂对象引用，所以当某个复杂对象被删除时，被引用对象并不会被自动删除。

OODBMS应该提供把一个复杂对象的组件对象**聚簇**（clustering）存储在二级存储器上的选项，这样可以提高访问复杂对象的效率。在许多情况下，都把对象结构以未经解释的形式存储在磁盘页上。然后，在把包括某个对象的磁盘页检索到内存中时，OODBMS将根据磁盘页上的信息建立结构化复杂对象。当然这个磁盘页上的信息指出了其他必须被检索的磁盘页。这就是**复杂对象组装**（complex object assembly）。

4.6　其他面向对象的概念

在本节中将简要给出一些其他的OO概念，包括多态性（操作符重载）、多重继承、选择性继承、版本和配置。

4.6.1　多态性（操作符重载）

OO系统的另一个特征是操作的**多态性**（polymorphism），有时也称为**操作符重载**（operator overloading）。这个概念允许把多个不同的操作符的实现绑定到同一个操作符名称或符号上，到底绑定到哪个操作符上取决于使用操作符的对象的类型。可以用简单的编程语言为例说明这个概念。在有些语言里，把操作符“+”应用于不同类型的对象意味着不同的操作。如果把“+”应用于整型对象，就会调用整数加法。如果把“+”应用于浮点型对象，就会调用浮点数加法。如果把“+”应用于集合类型的对象，就会调用集合并运算。编译器可以根据“+”的运算对象的类型决定执行哪种操作。

在OO数据库中，也会有类似的情况发生。可以使用在4.4节讨论的GEOMETRY_OBJECT为例说明OO数据库的多态性[①]。假设用下面这种形式定义GEOMETRY_OBJECT和它的子类型。

```
GEOMETRY_OBJECT: Shape, Area, ReferencePoint
RECTANGLE subtype-of
   GEOMETRY_OBJECT (Shape='rectangle'): Width, Height
TRIANGLE subtype-of
   GEOMETRY_OBJECT (Shape='triangle'): Side1, Side2, Angle
CIRCLE subtype-of GEOMETRY_OBJECT (Shape='circle'): Radius
```

在这里，所有GEOMETRY_OBJECT类型的对象都声明了函数Area，但是每个GEOMETRY_OBJECT子类型的Area方法的实现都不同。可以为Area提供一种计算各种GEOMETRY_OBJECT对象面积的通用的实现（例如，写一个普通算法以计算平面多边形的面积），然后写一个更有效的算法以计算几何对象的特殊类型的面积，例如计算圆形、矩形和三角形等规则图形面积的算法。这就用不同的实现重载了Area函数。

现在OODBMS必须根据应用Area函数的几何对象的类型，选择适当的Area方法。在强类型系统中，对象的类型是必须知道的，所以可以在编译的时候选择方法。上述这种方式被称为**提前绑定**（early binding）或**静态绑定**（static binding）。但是，在弱类型或无类型的系统里，比如Smalltalk和LISP，

① 在编程语言里有好几种多态性。有兴趣的读者可以参阅参考文献，那里有更详细的讨论。

直到运行时刻才能知道应用函数的对象的类型。那么函数必须在运行时刻检查对象的类型，然后调用适当的方法。这通常被称为**滞后绑定**（late binding）或**动态绑定**（dynamic binding）。

4.6.2　多重继承与选择性继承

如果某个子类型T是两个（或多个）类型的子类型，并且继承了两个超类型的函数（属性和方法），这就是类型层次中的**多重继承**。例如，可以创建MANAGER和ENGINEER的子类型ENGINEERING_MANAGER。这就创建了一个**类型格**（type lattice），而不是类型层次结构。多重继承的一个问题是，如果子类型继承的超类型有同名的不同函数，就会产生二义性。例如，MANAGER和ENGINEER可能都有叫Salary的函数。如果MANAGER和ENGINEER这两个超类型使用不同的方法实现Salary函数，那么就不知道它们的子类型ENGINEERING_MANAGER应该继承哪一个实现了。然而，也有可能MANAGER和ENGINEER是从同一个更高的超类型（比如EMPLOYEE）那里继承Salary的。多重继承的一般规则是：如果从同一个超类型那里继承了函数，那就只继承一次，这种情况下没有二义性；只有当两个超类型中的同名函数不相同时才会发生二义性。

在多重继承里有多种处理二义性的方案。一种方案是在创建子类型时检查是否有二义性，如果有，就让用户明确选择继承哪一种实现。另一种方案是由系统默认选择一种函数实现。第三种方案是如果出现了函数名的二义性，就不允许多重继承。实际上，一些OO系统根本不允许多重继承。

如果子类型只继承了超类型中的一部分函数，其他函数并没有被继承，就称为**选择性继承**。只要在列出超类型的函数时使用EXCEPT子句，子类型就不会继承这样的函数了。一般，在OO数据库系统中并不提供选择性继承，但是选择性继承在人工智能应用中是很有用的[①]。

4.6.3　版本和配置

许多OO系统的数据库应用，要求为同样的对象保留多个版本[②]。以软件工程的数据库应用为例，软件工程环境中保存着各种软件构件，比如设计模块、源代码模块、配置信息和测试用例，配置信息用来描述哪些模块应该连接在一起构成一个复杂的程序，测试用例则用来测试系统。需求有变化时，通常要对软件系统应用维护活动。维护通常包括修改某些设计和实现模块。如果系统正在运行，而且必须修改一个或多个模块时，设计者应该为每个被修改的模块创建一个**新版本**。类似地，也要生成新版本的测试用例来测试新版本的模块。但是，只有当新版本已经被详细测试过并被批准之后，才能用新版本代替旧版本，然后丢弃旧的版本。

需要注意的是：一个对象可能有两个以上的版本。例如，两个程序员在并发地工作来更新同一个软件模块。在这种情况下，除了原始模块外还要有两个版本。程序员可以并发地更新同一个软件模块的不同版本（其各自的版本）。通常称这种情况为**并发工程**。然而，最终必须合并这两个被修改过的版本，使新的（混合）版本包括两个程序员所作的修改。在合并过程中，应该确保所作的修改是兼容的。这就要求把两个被独立更新的版本合并起来，创建该对象的另一个版本。

从上述讨论可以看出，OODBMS必须能够存储和管理同一个概念对象的多个版本。有些系统确实具有这种能力，允许应用维护一个对象的多个版本，并根据需要可显式地引用某个特定的版本。但是，要对两个不同版本进行合并以及协调修改，这个问题往往要留给程序开发人员，他们了解应用的语义。有些DBMS可以协助版本合并，把两个新版本与原始的对象进行比较，判断所作的修改是否兼容。另外一些系统则维护一张表示版本间联系的**版本图**（version graph）。如果版本v_1是另一个版本v的副本，就画一条从v到v_1的弧线；如果合并版本v_2和v_3生成了版本v_4，就画一条从v_2到v_4的弧线和一条从v_3到v_4的弧线。版本图能够帮助用户理解众多版本间的联系，系统还可以在内部利用版本图

① ODMG 模型中的类型继承指的是继承操作，不包括继承属性（见第5章）。

② 版本不仅是OODB的问题，也是关系DBMS或其他类型DBMS的问题。

管理创建版本和删除版本。

如果把版本管理应用于复杂对象，就会出现很多需要解决的问题。一个复杂对象，例如一个软件系统，可能包括众多的模块。如果系统支持版本管理，每个模块都可能会有许多版本和一个版本图。一个复杂对象的**配置**是每个模块的每个版本的汇集，并且用适当的方法安排这些版本，使配置里的模块版本互相兼容，共同构成这个复杂对象的一个有效版本。复杂对象的新版本或新配置不必包括每个模块的新版本。因此，某些未修改过的模块版本可能属于复杂对象的多个配置。需要注意，一个配置是不同对象的版本的汇集，这些版本组成了一个复杂对象，但是版本图表示的是同一个对象的版本。一个配置应该遵循复杂对象的类型结构；同一个复杂对象的多个配置类似于一个组件对象的多个版本。

小结

在本章中讨论了与数据库系统相关的面向对象的概念，这些概念的提出是为了满足复杂数据库应用的需要，也是为了把数据库功能加入到面向对象编程语言中，例如C++编程语言。本章首先讨论了在OO数据库中使用的以下主要概念。

- **对象标识**：每个对象拥有唯一的标识，这个标识独立于该对象的属性值。
- **类型构造器**：递归使用基本类型构造器的集合就可以构造出复杂对象结构。基本类型构造器包括tuple、set、list、bag等。
- **封装操作**：对象类的定义中既包括了对象的结构，也包括了可以应用于该对象的操作。
- **编程语言兼容性**：临时对象和持久对象都被统一处理。把对象加到持久汇集或者通过显式命名就可以使对象具有持久性。
- **类型层次和类型继承**：可以用类型层次指定对象类型，允许新的对象类型继承预先定义过的类型的属性和方法。一些模型还允许多重继承。
- **外延**：可以把某一类型的所有持久对象存储在一个外延里。对应于某个类型层次的外延上有一些强制的集合/子集约束。
- **支持复杂对象**：能够存储和操作结构化和非结构化的复杂对象。
- **多态性和操作符重载**：重载操作名和方法名，可以把不同的实现应用于不同的对象类型。
- **版本**：有些OO系统可以为维护同一个对象的多个版本提供支持。

下一章将介绍这些概念在 ODMG 标准中如何实现。

复习题

4.1　面向对象方法的起源是什么？

4.2　OID的主要特征是什么？

4.3　讨论各种类型构造器，如何使用它们创建复杂对象结构？

4.4　讨论封装的概念，说明如何用封装创建抽象数据类型。

4.5　说明下列词语在面向对象数据库术语中的含义：方法、签名、消息、汇集、外延。

4.6　在类型层次中，类型和它的子类型之间有什么联系？对应于类型层次中的类型的外延上有什么强制约束？

4.7　持久对象和临时对象之间有什么区别？在典型的OO数据库里如何处理持久对象？

4.8　正规继承、多重继承和选择性继承之间有什么区别？

4.9　讨论多态性/操作符重载的概念。

4.10　结构化复杂对象和非结构化复杂对象之间有什么区别？

4.11　在结构化复杂对象里，属主语义和引用语义之间有什么区别？

4.12　什么是版本?它有什么重要性？版本和配置之间有什么区别?

练习题

4.13　用图4-3所示的符号改写4.4.1节中给出的例子GEOMETRY_OBJECT，区分属性和操作。用关键字INHERIT表示一个类来自继承另一个类。

4.14　比较EER模型中的继承（见《数据库系统基础：初级篇》的第4章）和4.4节描述的OO模型中的继承。

4.15　考虑《数据库系统基础：初级篇》中图4-10所示的UNIVERSITY EER模式。思考使用这个模式时需要为实体类型/类定义哪些操作？但是不必考虑构造器操作和析构器操作。

4.16　考虑《数据库系统基础：初级篇》中图3-2所示的COMPANY ER模式。思考使用这个模式需要为实体类型/类定义哪些操作？不必考虑构造器操作和析构器操作。

参考文献

面向对象数据库概念综合了OO编程语言、数据库系统和概念数据模型的概念。有些教科书描述了OO编程语言，例如，（Stroustrup，1986）和（Pohl，1991）是关于C++的，（Goldberg，1989）是关于Smalltalk的。最近（Cattell，1994）和（Lausen & Vossen，1997）两本书描述了OO数据库的概念。还有其他一些面向对象模型的书，包括详细介绍了Microelectronic Computer Corporation开发，被称为ORION的实验性面向对象的关系数据库设计系统，以及（Kim & Lochovsky，1989）关于面向对象主题的书。（Bancilhon et al.，1992）讲述了一个关于开发O2面向对象数据库管理系统的设计决定和语言实现的事例。（Dogac et al.，1994）详尽描述了北约工作组专家关于面向对象数据库为主题的讨论。

有很多关于OO数据库的文献，这里只列出了有代表性的文献。*CACM* 1991年10月刊和IEEE *Computer* 1990年12月刊描述了面向对象数据库的概念。（Dittrich，1986）和（Zaniolo et al.，1986）概述了面向对象数据模型的基本概念。（Baroody & DeWitt，1981）是一篇关于面向对象数据库的早期论文。（Su et al.，1988）介绍了CAD/CAM应用中采用的面向对象数据模型。（Gupta et al.，1992）讨论了面向对象在CAD、网络管理和其他一些领域的应用。（Mitschang，1989）扩展了关系代数，使之能够适用于复杂对象。（Gyssens et al.，1990）、（Kim，1989）、（Alashqur et al.，1989）、（Bertino et al.，1992）、（Agraw al et al.，1990）和（Cruz，1992）描述了OO查询语言和图形用户接口。

（Atkinson et al.，1989）中一篇有趣的文章The Object-Oriented Manifesto，报告了关于专家小组强制性位置和选择对象数据库管理的特点。（Osborn，1989）、（Atkinson & Buneman，1987）、（Danforth & Tomlinson，1988）讨论了数据库和面向对象编程语言的多态性。（Abiteboul & Kanellakis，1989）讨论了对象标识。（Kent，1991）讨论了数据库的OO编程语言。（Delcambre et al.，1991）和（Elmasri et al.，1993）讨论了对象约束。（Rabitti et al.，1991）和（Bertino，1992）讨论了OO数据库的授权和安全性。

第5章末尾将提供更多的参考资料。

第5章

对象数据库标准、语言和设计

正如《数据库系统基础：初级篇》中第8章开始所讨论的，对于特定类型的数据库系统，制定一套标准是非常重要的，因为标准可以支持数据库应用的可移植性。通常，**可移植性**（portability）被定义为：对特定的应用程序做最少的修改，就可以使得这个应用程序能够运行在不同系统上。在对象数据库领域中[①]，可移植性使得那些专门为访问某种对象数据库管理系统（ODBMS）包而编写的程序，也可以访问另外一种ODBMS包，只要这两种ODBMS都忠实地支持统一的标准。这一点对数据库用户十分重要，因为他们在投资一项新技术时，通常很注意不同的软件商是否都遵守同一个标准。为了说明可移植性的重要性，这里举一个例子：假设有一个用户投资数千美元，开发了一个运行在某个特定软件商的产品上的应用程序，后来由于某些原因，比如说该产品的性能不能满足用户应用的需要，引起他们对这个产品的不满。这时，如果这个应用程序是使用标准语言结构来编写的，并且另外一个软件商的产品也遵守同样的语言标准，那么该用户就可以把这个应用移植到新的软件商的产品上去，这样的移植不必做很大的改动，又可以提高应用程序的性能，从而节省了大量时间与金钱的投资。

拥有和遵守标准的第二个潜在优点是，有助于获得**互操作性**（interoperability）。一般意义上说，互操作性是指一个应用程序可以访问多个不同系统的能力。在数据库术语中，它是指同一个应用程序既可以访问存储在一个ODBMS包中的某些数据，又可以访问存储在另一个包中的其他数据。互操作性分为不同的等级。例如，数据库管理系统可以是同一种类型的两种不同的DBMS包，比如两种都是对象数据库系统；也可以是两种不同类型的DBMS包，比如一种是关系型DBMS，而另一种是对象DBMS。支持统一标准的第三个优点是，使得消费者更容易对商业产品进行比较，他们只要区分每种产品分别支持标准的哪些部分就可以了。

《数据库系统基础：初级篇》中第8章的引言中已经讨论过，商用的关系型数据库管理系统（RDBMS）成功的原因之一在于SQL标准。但是，直到最近都没有一个关于对象数据库管理系统的标准，这就导致许多原本有可能转向这种新技术的潜在用户望而却步。对象数据管理组织（Object Data Management Group，ODMG）这个ODBMS软件商的国际联盟曾经提出一种标准，即ODMG-93或者ODMG 1.0标准。该标准已经修订为ODMG 2.0，本章将在稍后作介绍。该标准由以下几部分组成：**对象模型**、**对象定义语言**（ODL）、**对象查询语言**（OQL），以及面向对象编程语言的**绑定**。面向对象编程语言的绑定包括3种面向对象的编程语言：C++、Smalltalk和Java。一些软件商只提供特定的语言绑定，不提供ODL和OQL的全部能力。本章将在5.1节中描述ODMG对象模型，5.2节中阐述ODL，5.3节中阐述OQL，5.4节中介绍C++语言绑定。并将使用《数据库系统基础：初级篇》的第4章中介绍的UNIVERSITY数据库为例，来说明如何使用ODL、OQL和C++语言绑定。本章的阐述将遵循（Cattell et al.，1997）阐述的ODMG 2.0对象模型[②]。值得注意的是，体现在ODMG对象模型中的

---

① 本章使用“对象数据库”来代替以前章节中提到的“面向对象的数据库”（见第4章），因为目前“对象数据库”是更加普遍接受的标准术语。

② 对象模型的前一个版本于1993年发表。

许多思想，都是基于大量研究人员20多年来对概念建模（conceptual modeling）以及面向对象数据库的深入研究而取得的成果。

阐述了ODMG模型之后，我们将在5.5节中描述一种关于对象数据库概念设计的技术，讨论面向对象的数据库与关系型数据库的区别，以及如何把用EER模型设计的概念数据库，映射到ODMG模型的ODL语句。

关于这个主题，如果读者不希望了解太多的细节，可以跳过5.3节到5.7节的内容。

5.1　ODMG 对象模型概述

ODMG对象模型是对象定义语言（ODL）和对象查询语言（OQL）的基础。实际上，这种对象模型提供了数据类型、类型构造器及其他一些可以用于ODL来说明对象数据库模式的概念。因此，它必须为面向对象的数据库提供一种标准的数据模型，就像SQL为关系型数据库所描述的标准数据模型一样。ODMG对象模型也为该领域提供了一套标准术语，但是在过去，该领域的同一个术语有时会用来表述不同的概念。本章尽量遵照ODMG术语。ODMG模型中的许多概念已经在第4章中进行过讨论，这里假定读者已经阅读了4.1节到4.5节中的内容。如果ODMG术语和第4章中使用的术语有所不同，我们会及时指出。

5.1.1　对象和文字

对象和文字是对象模型的基本构建块。二者的主要区别在于：对象既包括一个对象标识，又包括一个**状态**（或当前值）；而文字只有一个值，没有对象标识符[①]。二者的值都可以有一个复杂的结构。对象状态通过修改对象值而随时改变。文字基本上是一个常量值，尽管它也可能有一个不变的复杂结构。

对象可以用4个特征来描述：标识符、名称、生存期和结构。**对象标识符**是该对象在系统范围内的唯一标识（或者叫作Object_id）[②]。每个对象必须有一个对象标识符。除了Object_id外，一些对象在特定的数据库中也可以有选择地给出一个唯一的**名称**（name），这个名称可以用来在一个程序中引用该对象，并且系统应该可以通过给定的名称定位到该对象[③]。显然，不是所有独立的对象都会有唯一的名称。通常地，主要是那些拥有某种特定类型对象的汇集的少数对象（比如类外延）才会有名称。这些名称被用来作为数据库的**入口点**，即通过它们唯一的名称来定位这些对象，由此用户可以定位被这些对象引用的其他对象。应用中的其他一些重要对象也可能有唯一的名称。在一个特定数据库中，所有这些名称必须唯一。对象的**生存期**确定了该对象是一个持久对象（即数据库对象），还是一个临时对象（即该对象只存在于一个正在执行的程序中，随着程序的中止而消失）。最后，对象的**结构**指定该对象是如何使用类型构造器构造出来的。它确定了该对象是原子对象还是汇集对象[④]。原子对象这个术语和在4.2.2节中定义的原子构造器不同，也和原子文字（稍后会讲到）有很大的区别。在ODMG模型中，原子对象是指任何非汇集对象的对象，所以也包括使用结构构造器[⑤] 建立的结构化对象。在5.1.2节中将讨论汇集对象，5.1.3节讨论原子对象。首先，我们定义文字的概念。

在对象模型中，**文字**是指没有对象标识符的一个值。但是，这个值可以有一个简单的结构，也可以有一个复杂的结构。有3种类型的文字：原子文字、汇集文字和结构化文字。**原子文字**[⑥]对应于

① 这里的两个术语：值和状态是可以交换的。
② 对应于第4章的OID。
③ 对应于4.3节中描述的持久命名机制。
④ 在ODMG模型中，原子对象并不是对应于那些值是基本数据类型的对象。所有基本的值（整数、实数等等）都被认为是文字。
⑤ 结构构造器对应于第4章中的元组构造器。
⑥ 使用原子文字中的“原子（atomic）”一词对应于4.2.2节中用于原子构造器的方法。

基本数据类型的值，并且是预先定义的。对象模型的基本数据类型包括**长整型**(long)、**短整型**(short)、**无符号整型**（unsigned long与unsigned short）、**单精度浮点型**（**float**）、**双精度浮点型**（**double**）、**布尔型**（**boolean**）、**字符型**（**char**）、**字符串型**（**string**）以及**枚举类型**（**enum**）。**结构化文字**（structured literal）大致对应于在4.2.2节中描述的使用元组构造器构造的值。它们包括日期（date）、间隔（interval）、时间（time）和时间戳（timestamp）作为内置结构（见图5-1b），还包括每个应用所需要的任何一个用户定义的类型结构[①]。在ODL中，用户定义的结构使用关键字Struct来创建，这和在C与C++编程语言中一样。**汇集文字**是指这样的值，该值是对象或者值的汇集，但是汇集本身没有object_id。对象模型中的汇集有Set<*T*>、Bag<*T*>、List<*T*>和Array<*T*>，其中*T*是汇集中的对象或值的类型[②]。另一种汇集类型是Dictionary<*K*,*V*>，这是关联<*K*,*V*>的汇集，其中每个*K*是一个与值*V*相关联的码（唯一搜索值），它可以用来在值的汇集上创建索引。

```
interface Object {
    . . .
    boolean             same_as(in object other_object);
    object              copy();
    void                delete();
};
```

(a) 被所有对象继承的基本Object接口

```
interface Date : Object {
    enum                Weekday
                            {Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday};
    enum                Month
                            {January, February, March, April, May, June, July, August, September,
                            October, November, December};
    unsigned short      year();
    unsigned short      month();
    unsigned short      day();
    . . .
    boolean             is_equal(in date other_date);
    boolean             is_greater(in date other_date);
    . . .    };

interface Time : Object {
    . . .
    unsigned short      hour();
    unsigned short      minute();
    unsigned short      second();
    unsigned short      millisecond();
    . . .
    boolean             is_equal(in time other_time);
    boolean             is_greater(in time other_time);
    . . .
    time                add_interval(in interval some_interval);
    time                subtract_interval(in interval some_interval);
    interval            subtract_time(in time other_time);   };

interface Timestamp : Object {
    . . .
    unsigned short      year();
    unsigned short      month();
```

图5-1　ODMG对象模型中的部分接口定义概览

① 日期、间隔、时间以及时间戳的结构既可以用来创建文字值，也可以用来创建带有标识符的对象。

② 这些类似于4.2.2节中描述过的类型构造器。

```
    unsigned short    day();
    unsigned short    hour();
    unsigned short    minute();
    unsigned short    second();
    unsigned short    millisecond();
    ...
    timestamp         plus(in interval some_interval);
    timestamp         minus(in interval some_interval);
    boolean           is_equal(in timestamp other_timestamp);
    boolean           is_greater(in timestamp other_timestamp);
    ...   };

interface Interval : Object {
    unsigned short    day();
    unsigned short    hour();
    unsigned short    minute();
    unsigned short    second();
    unsigned short    millisecond();
    ...
    interval          plus(in interval some_Interval);
    interval          minus(in Interval some_Interval);
    interval          product(in long some_value);
    interval          quotient(in long some_value);
    boolean           is_equal(in interval other_Interval);
    boolean           is_greater(in interval other_interval);
    ... };
```

(b) 结构化文字的一些标准接口

```
interface Collection : Object {
    ...
    exception         ElementNotFound{any element; };
    unsigned long     cardinality();
    boolean           is_empty();
    ...
    boolean           contains_element(in any element);
    void              insert_element(in any element);
    void              remove_element(in any element)
                          raises(ElementNotFound);
    iterator          create_iterator(in boolean stable);
    ... };

interface Iterator {
    exception         NoMoreElements();
    ...
    boolean           is_stable();
    boolean           at_end();
    void              reset();
    any               get_element() raises(NoMoreElements);
    void              next_position() raises(NoMoreElements);
    ... };

interface set : Collection {
    set               create_union(in set other_set);
    ...
    boolean           is_subset_of(in set other_set);
    ... };

interface bag : Collection {
    unsigned long     occurrences_of(in any element);
    bag               create_union(in bag other_bag);
    ... };
```

图5-1　（续）

```
interface list : Collection {
    exception       Invalid_Index{unsigned_long index; );
    any             remove_element_at(in unsigned long position)
                        raises(InvalidIndex);
    any             retrieve_element_at(in unsigned long position)
                        raises(InvalidIndex);
    void            replace_element_at(in any element, in unsigned long position)
                        raises(InvalidIndex);
    void            insert_element_after(in any element, in unsigned long position)
                        raises(InvalidIndex);
    . . .
    void            insert_element_first(in any element);
    . . .
    any             remove-first-element() raises(InvalidIndex);
    . . .
    any             retrieve_first_element() raises(InvalidIndex);
    . . .
    list            concat(in list other_list);
    void            append(in list other_list);
};

interface array  : Collection {
    exception       Invalid_Index{unsigned_long index; };
    any             remove_element_at(in unsigned long index)
                        raises(InvalidIndex);
    any             retrieve_element_at(in unsigned long index)
                        raises(InvalidIndex);
    void            replace_element_at(in unsigned long index, in any element)
                        raises(InvalidIndex);
    void            resize(in unsigned long new_size);
};

struct association {any key; any value; };

interface dictionary : Collection {
    exception       KeyNotFound{any key; };
    void            bind(in any key, in any value);
    void            unbind(in any key) raises(KeyNotFound);
    any             lookup(in any key) raises(KeyNotFound);
    boolean         contains_key(in any key);
};
```

(c) 汇集对象的接口定义

图5-1　（续）

图5-1给出了一个对象模型基本组件的简化视图。ODMG的符号表示法中采用关键字interface来代替第4章中采用的关键字*type*和*class*。实际上，接口是一个更合适的术语，因为它描述了对象类型的接口，即它们可见的属性、联系和操作[①]。通常地，这些接口是不可实例化的（也就是说，对于接口不能直接创建对象），但是它们用于定义操作，而这些操作可以被特定应用中用户定义的对象所继承。对象模型中保留关键字*class*，表示用户说明的形成数据库模式的类声明，用于创建应用对象。图5-1所示是对象模型的简化版本，完整的描述可以参见（Cattell et al.，1997）。下面通过图5-1所示的例子来描述对象模型。

在对象模型中，所有的对象都继承了Object基本接口，如图5-1a所示。因此，被所有对象（从Object接口）继承的基本操作是copy（创建对象的一个新副本）、delete（删除对象）及same_as（对

① interface也是JAVA编程语言中使用的关键字。

比两个对象是否相同）[①]。一般说来，使用**圆点符号**（.）把操作应用于对象。例如，给定对象*O*，将它和另一个对象*P*进行比较，写作：

O.same_as (*P*)

该表达式的返回值是布尔型，如果*P*和*O*的标识符相同，则返回值为真；否则返回值为假。类似地，为对象*O*创建一个副本*P*，写作：

P = *O*.copy ()

圆点符号可以用**箭头符号**（→）来代替，即：*O*→same_as (*P*) 或者*O*→copy ()。

类型继承用于定义类型/子类型的关系。与在C++编程语言中一样，在对象模型中也使用冒号（:）来说明类型继承。在图5-1中，所有接口，比如Collection（汇集）、Date（日期）及Time（时间）都继承Object接口。在对象模型中，有两种主要的对象类型：5.1.2节中描述的汇集对象，5.1.3节中描述的原子（以及结构化）对象。

5.1.2　汇集对象的内置接口

任何一个**汇集对象**都继承图5-1c中给出的基本的Collection接口，这里展示了用于所有汇集对象的操作。给定一个汇集对象*O*，操作*O*.cardinality()返回该汇集中的元素个数。操作*O*.is_empty()返回一个布尔值，如果汇集对象*O*为空，则返回真，否则返回假。操作*O*.insert_element(*E*)向汇集对象*O*中插入元素*E*；操作*O*.remove_element(*E*)从汇集*O*中删除元素*E*。最后，操作*O*.contains_element(*E*)返回一个布尔值，如果汇集*O*中包括元素*E*，则返回值为真，否则返回值为假。操作*I*=*O*.create_iterator()为汇集对象*O*创建一个**迭代对象**（iterator object）*I*，可以用来迭代汇集中的每个元素，迭代对象的接口也如图5-1c所示。操作*I*.reset()将迭代器置于汇集中的第一个元素（对于无序汇集，可能是某个任意的元素）；而*I*.next_position()将迭代器置于下一个元素；*I*.get_element()检索**当前元素**，也就是迭代器当前所在的元素位置。

ODMG对象模型使用**异常**来报告错误或者特殊的情况。例如，如果元素*E*不是汇集*O*中的元素，那么操作*O*.remove_element(*E*)就会引发Collection接口中的ElementNotFound异常；再假如迭代器当前位于汇集中的最后一个元素，操作*I*.next_position()将会引发迭代器接口中的NoMoreElement异常，因为这时已经不存在任何元素可以让迭代器指向了。

Collection对象进一步特化为set、list、bag、array、dictionary，它们都继承Collection接口中的操作。set<*T*>对象类型可以用于创建对象，该对象*O*的值是一个类型为*T*的元素组成的集合。Set接口中还包括一些另外的操作*P* = *O*.create_union(*S*)（见图5-1c），返回一个set<*T*>类型的新对象*P*，它是两个集合*O*与*S*的并集。与create_union(*S*)类似的两个操作（图5-1c中没有列出）是create_intersection(*S*)和create_difference(*S*)。集合比较的操作有*O*.is_subset_of(*S*)，如果set对象*O*是另一个set对象*S*的子集，那么该操作返回真值，否则返回假值。类似的操作（图5-1c中没有列出）有is_proper_subset_of(*S*)、is_superset_of(*S*)以及is_proper_superset_of(*S*)。包bag<*T*>对象类型也继承Collection接口，与集合不同，它允许汇集中有重复的元素。包的3个操作create_union(b)、create_intersection(b)以及create_difference(b)都返回bag<*T*>类型的一个新对象。例如，*P*= *O*.create_union(b)返回的bag对象*P*是*O*与*B*的并包（保留重复元素）。另外，*O*.occurrences_of(*E*)操作返回包*O*中元素*E*重复出现的次数。

列表list<*T*>对象类型继承Collection操作，并且用于创建汇集。在该汇集中，元素的次序是重要的。这种对象*O*的每个值是类型为*T*的元素组成的有序列表。因此，在列表中指出第一个、最后一个以及第*i*个元素是有意义的。同样，向列表中增加一个元素时，必须指定该元素插入到列表中的位置。某些list操作已经在图5-1c中给出。如果*O*是列表类型的对象，那么操作*O*.insert_element_first(*E*)（见

① 定义在对象上用作封锁目的的附加操作，在图5-1中没有给出。第2章讨论了数据库的封锁概念。

图5-1c）向列表中第一个元素前插入元素*E*，从而*E*就变成了该列表的第一个元素。一个类似的操作（图中没有列出）是*O*.insert_element_last(*E*)。图5-1c中列出的操作*O*.insert_element_after(*E*,*I*)向列表*O*中第*i*个元素之后插入元素*E*，如果列表*O*中不存在第*i*个元素，该操作就会引起异常InvalidIndex。类似的操作*O*.insert_element_before(*E*,*I*)没有列在图中。从列表中移出元素的操作有*E* = *O*.remove_first_element()、*E* = *O*.remove_last_element()及*E* = *O*.remove_element_at(*I*)。这些操作从列表中移出指定的元素，并且将该元素作为结果返回。有一些操作从列表中取回一个元素，但是并不删除该元素。这些操作包括*E* = *O*.retrieve_first_element()、*E* = *O*.retrieve_last_ element()及*E*=*O*.retrieve_element_at(*I*)。最后，还定义了两个操纵列表的操作。其中一个是*P*=*O*.concat(*I*)，该操作创建一个新的列表*P*，它是列表*O*与列表*I*合并后得到的（列表*O*中的元素依次排列在列表*I*之后）；另一个是*O*.append(*I*)，该操作将列表*I*的元素追加到列表*O*之后（并不创建新的列表）。

数组array<*T*>对象类型也继承Collection的操作。它类似于列表，不同的是一个数组中的元素数目固定。下面介绍数组对象的具体操作。操作*O*.replace_element_at(*I*, *E*)将数组*O*中的第*I*个元素替换为元素*E*；操作*E* =*O*.remove_element_at(*I*)取回数组*O*中的第*i*个元素，并将该位置放上空值；操作*E* =*O*.retrieve_element_at(*I*)只是单纯地取回数组*O*中的第*i*个元素。当*I*大于数组的尺寸时，以上这些操作都会引起异常InvalidIndex。另外，操作*O*.resize(*N*)将数组*O*的元素数目重置为*N*。

最后一种汇集对象是字典dictionary<*K*,*V*>类型。它允许创建一个由关联对<*K*,*V*>组成的汇集，其中所有的*K*（码）值是唯一的。该类型也允许进行关联检索，即给定码值，取回对应的一个特定对（与索引类似）。如果*O*是一个字典类型的汇集对象，那么操作*O*.bind(*K*,*V*)将值*v*和码*K*绑定，以作为汇集中的一个关联< *K*, *V* >；而操作*O*.unbind(*K*)从汇集*O*中移去码*K*的关联；操作*V* =*O*.lookup(*K*)返回*O*中与码*K*相关联的*V*值。后两个操作可能会引起异常KeyNotFound。最后，操作*O*.contains_key(*K*)返回一个布尔值，如果*O*中的码*K*存在，则返回值为真，否则为假。

图5-2反映了对象模型的内置构造的继承层次关系。子类型从超类型中继承操作。上面讲过的汇集对象接口是不可直接实例化的，也就是说，基于这些接口，不可以直接创建对象。更准确地说，对于特定的数据库应用，接口可以用来确定用户定义的汇集对象，可以是集合类型、包类型、列表类型、数组类型、或者字典类型。当用户设计一个数据库模式时，他们将声明和数据库应用相关的、自己的对象接口以及类。如果一个接口或者类是一个汇集对象，比如集合，那么该接口会继承set接口中的操作。例如，在一个UNIVERSITY数据库应用中，用户可以指定一个类set<STUDENT>，它的对象是STUDENT对象的集合。这样，编程人员就可以使用集合的操作来处理set<STUDENT>类型的对象。典型的是使用对象定义语言ODL（见5.2节）来创建应用程序的类。

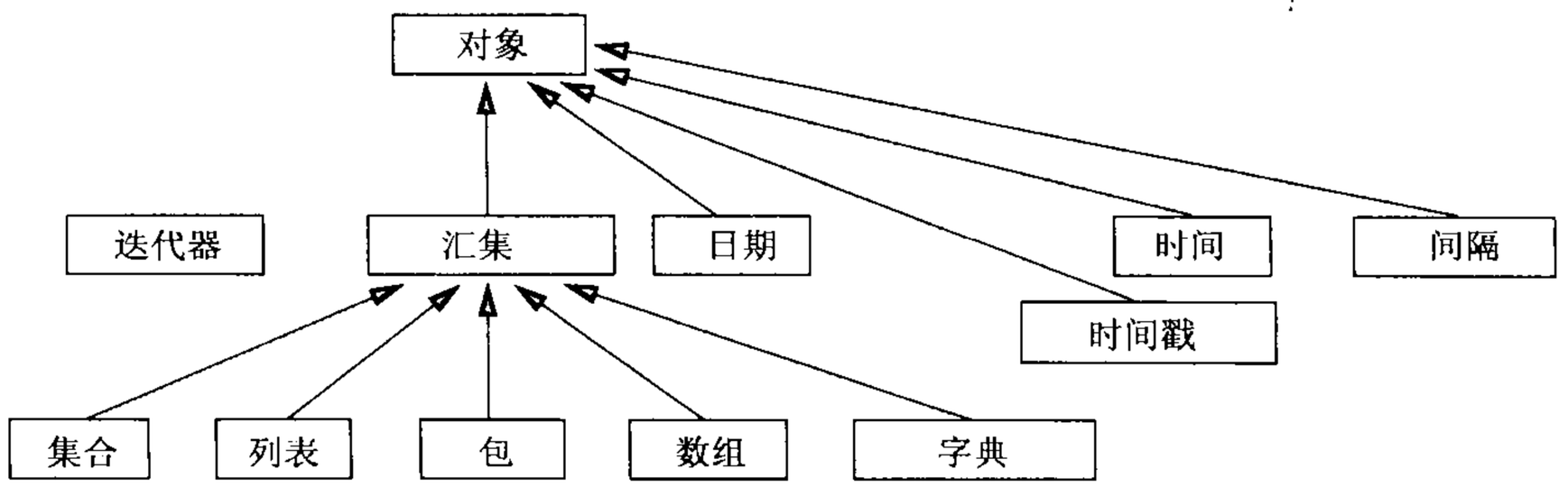


图5-2　对象模型的内置接口之间相互继承的层次关系

需要强调的是，一个特定汇集中的所有对象必须是同一种类型的。因此，尽管在图5-1c所列的汇集接口的详细说明中出现过关键字any，它并不表示任何类型的对象可以混合在同一个汇集中。更精确地说，any表明：当为一个特定的汇集指定元素类型时，可以使用任何一种类型（包括其他的汇集类型）。

5.1.3　原子（用户定义）对象

在上一节中描述了对象模型内置的汇集类型。这一节讨论对于原子对象，对象类型如何构建。现在，我们讨论如何在ODL中使用关键字class来构造对象类型（类）。在对象模型中，任何一种不是汇集对象的用户定义的对象都叫作**原子对象**[①]（atomic object）。

例如，在一个UNIVERSITY数据库的应用中，用户可以为STUDENT对象指定一个对象类型（类）。大部分原子对象都是**结构化对象**（structured object），例如STUDENT对象有一个带有许多属性、联系以及操作的复杂结构，但是由于它不是一个汇集，所以仍然被看作是原子对象。这样，一个用户定义的原子对象类型被定义成为一个类，需要指定它的**特性**和**操作**。特性定义对象的状态，并且进一步分为**属性**和**联系**。在这一小节中，将详细介绍对于原子（结构化的）对象，一个用户定义的对象类型可以包括的3种组件，即属性、联系和操作。下面通过图5-3中列出的两个类EMPLOYEE（雇员）和DEPARTMENT（院系部门）来进行讨论。

```
class EMPLOYEE
(   extent          ALL_EMPLOYEES
    key             Ssn    )
{
    attribute       string                  Name;
    attribute       string                  Ssn;
    attribute       date                    Birth_date;
    attribute       enum Gender{M, F}       Sex;
    attribute       short                   Age;
    relationship    DEPARTMENT              Works_for
                        inverse DEPARTMENT::Has_emps;
    void            reassign_emp(in string New_dname)
                        raises(dname_not_valid);
};

class DEPARTMENT
(   extent          ALL_DEPARTMENTS
    key             Dname, Dnumber    )
{
    attribute       string                      Dname;
    attribute       short                       Dnumber;
    attribute       struct Dept_mgr {EMPLOYEE Manager, date Start_date}
                                                Mgr;
    attribute       set<string>                 Locations;
    attribute       struct Projs {string Proj_name, time Weekly_hours)
                                                Projs;
    relationship    set<EMPLOYEE>   Has_emps inverse EMPLOYEE::Works_for;
    void            add_emp(in string New_ename) raises(ename_not_valid);
    void            change_manager(in string New_mgr_name; in date Start_date);
};
```

图5-3　类定义中的属性、联系及操作

属性是描述对象某个方面的一个特性。属性有存储在对象中的值。典型的属性值有简单结构或复杂结构的文字；但是，也可以是其他对象的Object_id，甚至可以是由方法来指定的，利用这些方法计算属性值。在图5-3中[②]，EMPLOYEE（雇员）的属性包括姓名（Name）、社会保险号（Ssn）、出生日期（Birth_date）、性别（Sex）和年龄（Age）；DEPARTMENT（部门）的属性包括部门名称（Dname）、部门编号（Dnumber）、管理者（Mgr）、部门地址（Locations）及项目（Projs）。DEPARTMENT的属性Mgr与Projs有复杂的结构，并且通过struct来定义。struct对应于第4章中的元组构造器。这样，

① 在前面提到过，ODMG对象模型中对原子对象的定义不同于第4章中给出的原子构造器的定义，后者是在面向对象的数据库文献中经常使用的定义。

② 图5-3中使用对象定义语言（ODL）的符号表示法。关于ODL，将在5.2节中更详细地讨论。

每个DEPARTMENT对象中的Mgr的值有两个组件：一个是Manager（管理者），它的值是参照EMPLOYEE对象的一个Object_id，表示管理该部门的人员；另一个是Start_date（开始日期），它的值是一个日期。DEPARTMENT的属性Locations通过集合构造器来定义，因为每个DEPARTMENT对象可以有一系列的地址。

联系说明了把数据库中的两个对象关联在一起的一个特性。在ODMG的对象模型中，只有二元联系（见《数据库系统基础：初级篇》第3章）被明确地提出。每个二元联系通过关键字relationship来指定，并且被表示为一对逆向的引用。在图5-3中，EMPLOYEE中存在一个联系Works_for，它把每个部门和工作在该部门中的所有雇员相联系。关键字inverse表示，这两个联系逆向地确定了一个单个的概念的联系[①]。

通过指定inverse，数据库系统可以自动地维护联系的参照完整性。也就是说，如果一个特定的雇员E，Works_for的值指向某个部门D，那么对于部门D，Has_emps的值（雇员的集合）中必须包括雇员E。如果数据库设计者希望拥有一个只在一个方向上体现的联系，那么系统必须将其设计成一个属性（或者操作）。DEPARTMENT中Mgr属性的成分Manager就是这样一个例子。

除了属性和联系，在对象类型（类）的说明中，设计者还可以定义**操作**。每个对象类型可以有多个**操作签名**，用来指定操作名、参数类型及返回值（如果是可应用的话）。在每个操作对象类型中，操作名是唯一的，但是它们可以通过出现在不同对象类型中的相同的操作名来重载。操作签名也可以指定可能出现在操作执行过程中的**异常**的名称，操作的实现将包含引起这些异常的代码。在图5-3中，类EMPLOYEE有一个操作reassign_emp；类DEPARTMENT中有两个操作：add_emp和change_manager。

5.1.4　接口、类和继承

在ODMG对象模型中，指定对象类型存在两个概念：接口和类。另外，存在两种类型的继承联系。在本节中将讨论这些概念的区别和共同点。遵循ODMG的术语，这里使用**行为**意指操作，**状态**意指特性（属性和联系）。

接口是对一个对象类型的抽象行为的规范说明，它指定了操作签名。尽管接口是作为说明的一部分，它可以有状态特性（属性和联系），但是应该知道，接口中的这些状态是不能被继承的。接口也是**不可实例化的**（noninstantiable），也就是说，对应于一个接口定义，不能创建对象[②]。**类**是对一个对象类型的抽象行为和抽象状态的规范说明，并且类是**可实例化的**（instantiable）。也就是说，对应于一个类定义，可以创建单独的对象实例。由于接口是不可实例化的，所以它主要用于声明可以被类或者其他接口所继承的抽象操作。这被称为**行为继承**，通过符号“:”指定[③]。这样，在ODMG对象模型中，行为继承要求超类型是一个接口，子类型则既可以是一个类，也可以是另一个接口。

另一种继承联系称为EXTENDS，用关键字extends来指定。它用来严格地继承类中的状态和行为。在一个extends继承中，超类型和子类型都必须是类，并且不允许通过extends进行多重继承。然而，对于通过“:”的行为继承允许进行多重继承。这样，一个接口可以从多个不同的接口中继承行为。一个类通过“:”可以从多个接口中继承行为，而通过extends，一个类最多可以从另外一个类中继承行为和状态。在5.2节中将举例说明“:”和extends这两种继承联系的用法。

5.1.5　类外延、码和工厂对象

在ODMG对象模型中，数据库设计者可以对任何通过**类**声明来定义的对象类型，声明一个类**外**

---

① 《数据库系统基础：初级篇》的第3章中讨论了一个联系在反向的两个方向中怎样由两个属性来表示。

② 这在一定程度上类似于C++编程语言中抽象类的概念。

③ ODMG报告也把接口继承叫作类型/子类型，以及泛化/特化关系；尽管在文献中，这些术语也用来描述状态和操作的继承（见《数据库系统基础：初级篇》的第4章与本书第4章）。

延（extent，使用关键字extent），并且给该类外延一个名称。这个类外延将包括那个类中所有的持久对象。这样，它的行为好像是包含那个类中所有持久对象的集合对象。图5-3中，EMPLOYEE类和DEPARTMENT类有各自的类外延，分别称为ALL_EMPLOYEES和ALL_DEPARTMENTS。这就相当于创建了两个集合对象，一个是set<EMPLOYEE>类型，另一个是set<DEPARTMENT>类型，并且给它们两个分别命名为ALL_EMPLOYEES和ALL_DEPARTMENTS，使得它们持久。类外延也用于自动地在超类型的类外延和子类型的类外延之间强制建立一种集合/子集合的关系。如果两个类A和B都有类外延，分别是ALL_A和ALL_B，并且假设类B是类A的子类型（即，类B EXTENDS类A），那么在任何时间点上，ALL_B中对象的汇集必然是ALL_A中对象的汇集的一个子集。

一个带有类外延的类可以拥有一个或多个码。**码**（key）由一个或多个特性（属性和联系）组成，在该类外延中，对于每个对象来说，这些特性的值是唯一的。例如，在图5-3中，EMPLOYEE类有Ssn属性作为码（在这个类外延中，每个EMPLOYEE对象必须有一个唯一的Ssn值）；DEPARTMENT类中有两个不同的码：Dname和Dnumber（每个DEPARTMENT必须有唯一的Dname和唯一的Dnumber）。对于一个由多个特性组成的组合码[①]，那些组成该码的特性包括在一对圆括号中。例如，若一个带有类外延ALL_VEHICLES的类VEHICLE有一个组合码，由State和License_number两个属性组成，则在码声明中，这两个属性应该写在圆括号里，即（State,License_number）。

下面介绍**工厂对象**（factory object）的概念。工厂对象可以通过它的操作生成或者创建单独的对象。图5-4中列出了ODMG对象模型中部分工厂对象的一些接口。ObjectFactory接口只有一个操作new()，它返回一个带有Object_id的新对象。通过继承这个接口，用户可以为每个用户定义的（原子的）对象创建他们自己的工厂接口，并且编程人员可以针对每种对象类型，实现不同的*new*操作。图5-4也给出一个DateFactory接口，它有其他一些操作，calendar_date用来创建一个新的日期，current用来创建一个值为current_date的对象。这些其他的操作在图5-4中没有列出。正如我们所看到的，工厂对象基本上是为新对象提供**构造器操作**的。

```
interface ObjectFactory {
    Object          new();
};

interface DateFactory : ObjectFactory {
    exception       InvalidDate{};
    . . .
    date            calendar_date( in unsigned short year,
                                   in unsigned short month,
                                   in unsigned short day)
                    raises(InvalidDate);
    . . .
    date            current();
};

interface DatabaseFactory {
    Database        new();
};

interface Database {
    void            open(in string database_name);
    void            close();
    void            bind(in any some_object, in string object_name);
    object          unbind(in string name);
    object          lookup(in string object_name)
                        raises(ElementNotFound);
. . . };
```

图5-4　说明工厂对象和数据库对象的接口

① 在ODMG的报告中，组合码也被称为混合码。

最后，讨论一下**数据库**的概念。一个ODBMS可以创建许多不同的数据库，每个数据库都有它自己的模式。正如图5-4所示，ODMG对象模型对DatabaseFactory和Database对象都定义了接口。每个数据库都有自己的数据库名，并且bind操作可以用来在一个特定数据库中为持久对象指派单独且唯一的名称；lookup操作从数据库中返回一个由object_name指定的对象；unbind操作从数据库中移去一个持久的且已命名对象的名称。

5.2　对象定义语言（ODL）

上一节概述了ODMG 对象模型，这一节将阐示通过使用对象定义语言（ODL）[①]，如何应用上节中的各个概念创建一个对象数据库模式。

ODL被设计成支持ODMG对象模型的语义结构，并且独立于任何特定的编程语言。它的主要用途是创建对象说明，也就是类和接口。因此，ODL不是一个完全的编程语言。用户可以独立于任何编程语言在ODL中指定一种数据库模式，然后使用特定的语言绑定来指明如何将ODL结构映射到特定编程语言中的结构，比如C++、Smalltalk以及Java。我们将在5.4节中简要给出C++绑定。

图5-5b展示了一种可能的模式，它是《数据库系统基础：初级篇》中第4章给出的UNIVERSITY数据库的一部分。我们将通过这个示例以及图5-7中的示例来描述ODL的概念。图5-5a是图5-5b的图形表示法的展示，后者可以看成是EER图（见《数据库系统基础：初级篇》的第4章）的一种变种，它增加了接口继承的概念，但是却没有某些其他的EER概念，比如范畴（union类型）以及联系的属性。

图5-6展示了关于UNIVERSITY数据库ODL类定义的一种可能的集合。一般说来，从一个对象模型图（或EER模式图）向ODL类的映射，存在多种不同的可能。在5.5节中，我们将进一步讨论这些选择。

图5-6展示了映射部分UNIVERSITY数据库（见《数据库系统基础：初级篇》的第4章）的一种直接方法。实体类型映射成ODL类，并且继承是通过使用EXTENDS来完成。但是，不存在直接的方式来映射范畴（union类型）或者多重继承。在图5-6中，类PERSON（人员）、FACULTY（教职员工）、STUDENT（学生）及GRAD_STUDENT（研究生）都有类外延，依次分别是PERSONS、FACULTY、STUDENTS及GRAD_STUDENTS。FACULTY和STUDENT都是EXTENDS PERSON，GRAD_STUDENT EXTENDS STUDENT。因此，STUDENTS的汇集（以及FACULTY汇集）在任何时间点上都必须是PERSON汇集的一个子集。类似地，GRAD_STUDENT汇集必须是STUDENT的一个子集。同时，单独的STUDENT以及FACULTY对象，将继承PERSON的特性（属性和联系）和操作，单独的GRAD_STUDENT对象则继承STUDENT的特性和操作。

图5-6中的类DEPARTMENT（院系部门）、COURSE（课程）、SECTION（单元）及CURR_SECTION（当前学期）都是图5-5b中相应实体类型的直接映射。但是，类GRADE需要一些说明。在图5-5b中，GRADE类相应于STUDENT和SECTION之间的M:N联系。它被转换成一个单独的类（而不是一对逆向联系）的原因在于它包括联系属性Grade[②]。

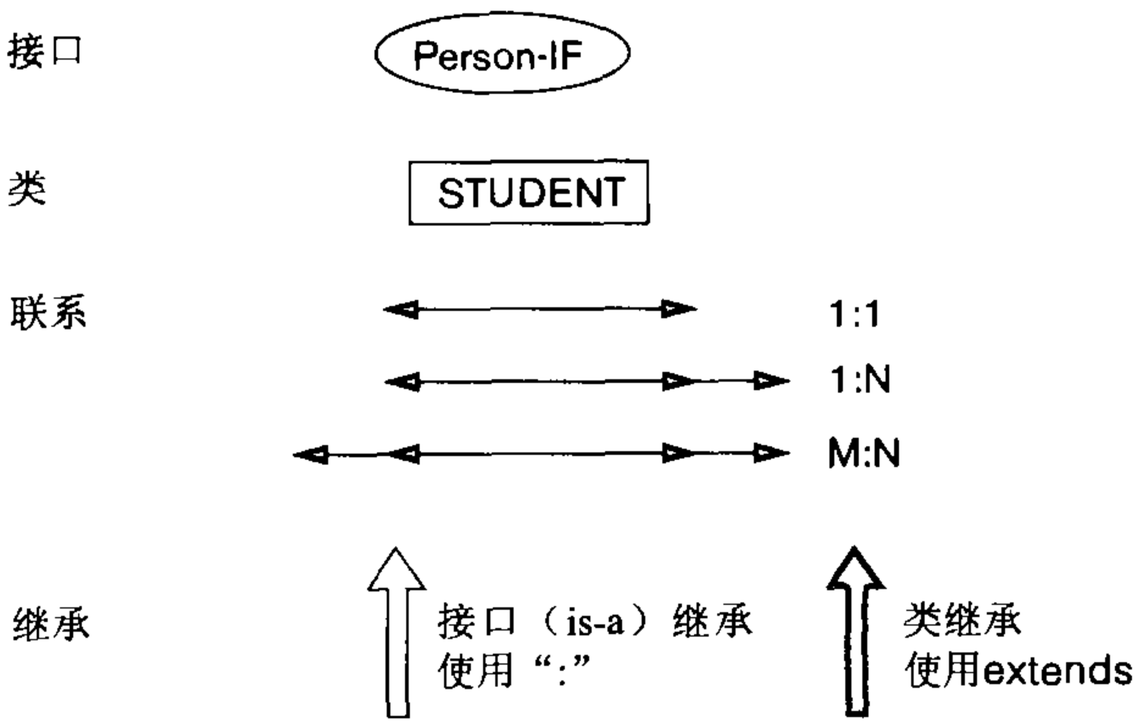
因此，M:N联系被映射成类GRADE和一对1:N联系，其中一个是STUDENT与GRADE之间的联系，另一个是SECTION与GRADE之间的联系[③]。这两个联系由以下联系的特性表示：STUDENT的Completed_sections、GRADE的Section和Student以及SECTION的Students（见图5-6）。最后，类DEGREE用来表示GRAD_STUDENT的复合的多值属性degrees（见《数据库系统基础：初级篇》图4-10）。

---

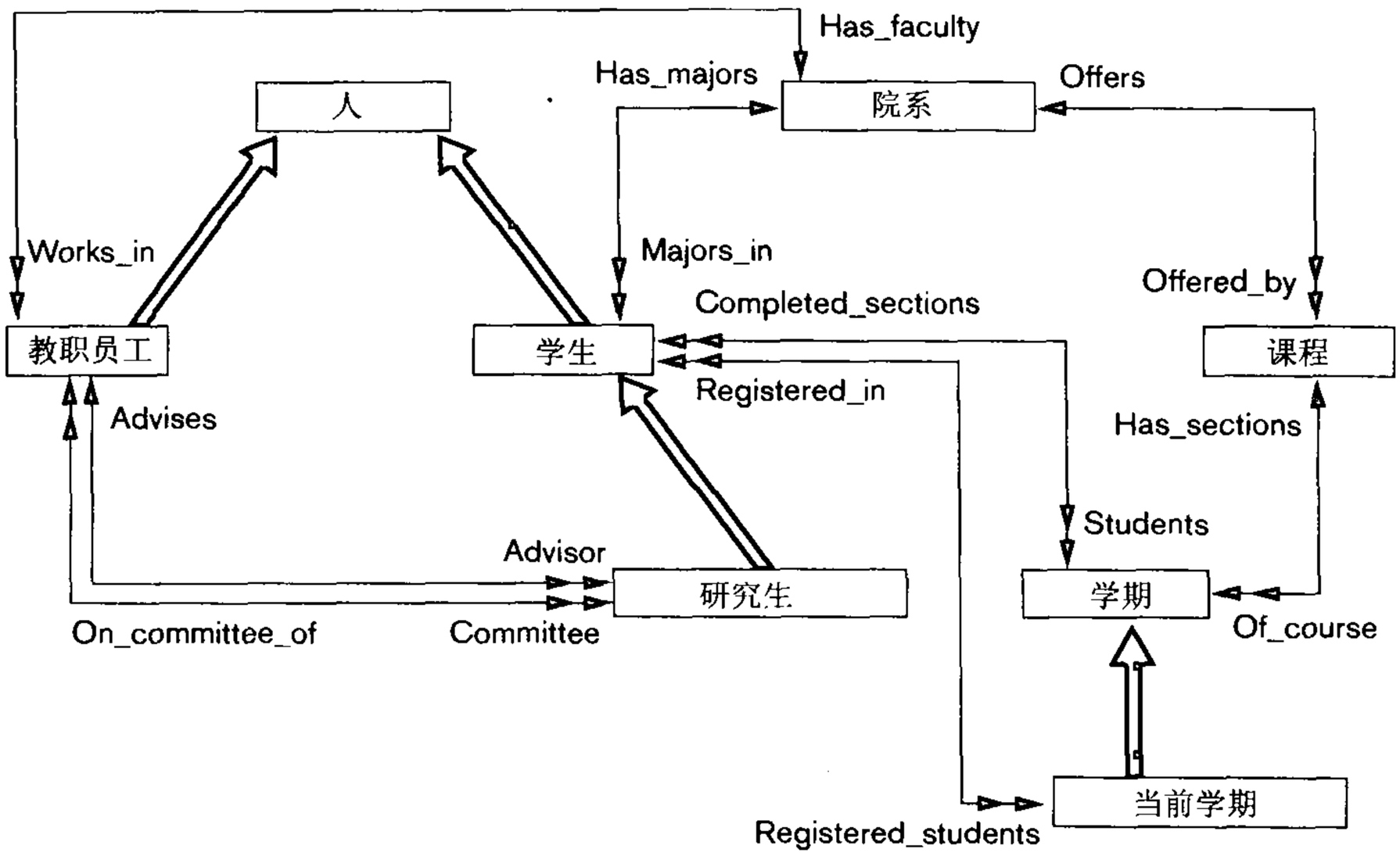
① 使用联系和其他一些数据库概念的扩展，ODL语法及数据类型与公共对象请求代理体系结构（Common Object Request Broker Architecture，CORBA）的接口定义语言（IDL）是相容的。

② 5.5节中将讨论对于联系属性的不同映射。

③ 这类似于在联系模型（见《数据库系统基础：初级篇》的第7章）以及传统网状模型（见附录D）中，M:N联系如何映射。



(a) 表示ODL模式的图例说明



(b) 对于部分UNIVERSITY数据库的一个图形对象数据库模式（类GRADE和类DEGREE没有在图中表示出）

图5-5　数据库模式的一个例子

```
class PERSON
(   extent      PERSONS
    key         Ssn  )
{   attribute   struct Pname {   string   Fname,
                                 string   Mname,
                                 string   Lname  }     Name;
    attribute   string                                 Ssn;
    attribute   date                                   Birth_date;
    attribute   enum Gender{M, F}                      Sex;
    attribute   struct Address {  short    No,
                                  string   Street,
                                  short    Apt_no,
                                  string   City,
                                  string   State,
                                  short    Zip  }     Address;
    short       Age();    };
```

图5-6　图5-5b中的UNIVERSITY数据库可能的ODL模式

```
class FACULTY extends PERSON
(   extent          FACULTY )
{   attribute       string              Rank;
    attribute       float               Salary;
    attribute       string              Office;
    attribute       string              Phone;
    relationship DEPARTMENT             Works_in inverse DEPARTMENT::Has faculty;
    relationship set<GRAD_STUDENT> Advises inverse GRAD_STUDENT::Advisor;
    relationship set<GRAD_STUDENT> On_committee_of inverse GRAD_STUDENT::Committee;
    void            give_raise(in float raise);
    void            promote(in string new rank);    };


class GRADE
(   extent          GRADES )
{
    attribute       enum GradeValues{A,B,C,D,F,I, P} Grade;
    relationship SECTION Section inverse SECTION::Students;
    relationship STUDENT Student inverse STUDENT::Completed_sections;    };


class STUDENT extends PERSON
(   extent          STUDENTS )
{   attribute       string              Class;
    attribute       Department          Minors_in;
    relationship  Department Majors_in inverse DEPARTMENT::Has_majors;
    relationship set<GRADE> Completed_sections inverse GRADE::Student;
    relationship set<CURR_SECTION> Registered_in inverse CURR_SECTION::Registered_students;
    void            change_major(in string dname) raises(dname_not_valid);
    float           gpa();
    void            register(in short secno) raises(section_not_valid);
    void            assign_grade(in short secno; in GradeValue grade)
                        raises(section_not_valid,grade_not_valid);    };

class DEGREE
{   attribute       string              College;
    attribute       string              Degree;
    attribute       string              Year;     };

class GRAD_STUDENT extends STUDENT
(   extent          GRAD_STUDENTS )
{   attribute  set<Degree>          Degrees;
    relationship Faculty_advisor inverse FACULTY::Advises;
    relationship  set<FACULTY> Committee inverse FACULTY::On_committee_of;
    void            assign_advisor(in string Lname; in string Fname)
                        raises(faculty_not_valid);
    void            assign_committee_member(in string Lname; in string Fname)
                        raises(faculty_not_valid);  };

class DEPARTMENT
(   extent          DEPARTMENTS
    key             Dname )
{   attribute       string              Dname;
    attribute       string              Dphone;
    attribute       string              Doffice;
    attribute       string              College;
    attribute       FACULTY             Chair;
    relationship    set<FACULTY> Has_faculty inverse FACULTY::Works_in;
    relationship    set<STUDENT> Has_majors inverse STUDENT::Majors_in;
    relationship    set<COURSE> Offers inverse COURSE::Offered_by;  };
```

图5-6　（续）

```
class COURSE
(   extent          COURSES
    key             Cno )
{   attribute       string          Cname;
    attribute       string          Cno;
    attribute       string          Description;
    relationship  set<SECTION> Has_sections inverse SECTION::Of_course;
    relationship <DEPARTMENT> Offered_by inverse DEPARTMENT::Offers; };

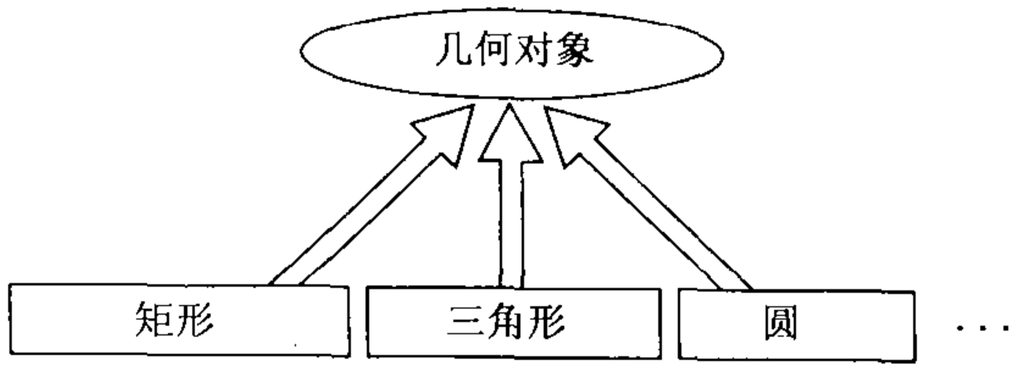
class SECTION
(   extent          SECTIONS )
{   attribute       short           Sec_no;
    attribute       string          Year;
    attribute       enum Quarter{Fall, Winter, Spring, Summer}
                                    Qtr;
    relationship set<Grade> Students inverse Grade::Section;
    relationship COURSE Of_course inverse COURSE::Has_sections; };

class CURR_SECTION extends SECTION
(   extent          CURRENT_SECTIONS )
{   relationship set<STUDENT> Registered_students
                        inverse STUDENT::Registered_in
    void            register_student(in string Ssn)
                        raises(student_not_valid, section_full); };
```

图5-6　（续）

因为上例中只有类，没有包括任何的接口，所以这里使用另一个例子来说明接口和接口（行为）继承。图5-7a所示是用于存储几何对象的一个数据库模式的一部分。其中指定了一个接口GeometryObject，它包括用于计算一个几何对象的周长（perimeter）和面积（area）的操作，还包括用于移动（translate）和旋转（rotate）一个对象的操作。有一些类（RECTANGLE（矩形）、TRIANGLE（三角形）、CIRCLE（圆）、……）继承GeometryObject接口。由于GeometryObject是一个接口，因此它是不可实例化的，也就是说，基于该接口不能直接创建任何对象。但是，可以创建RECTANGLE、TRIANGLE、CIRCLE、……类型的对象，并且这些对象继承GeometryObject接口的所有操作。需要注意，使用接口继承，只有操作可以被继承，特性（属性和联系）则不能被继承。因此，如果在继承类中需要一个特性，它必须在类定义中重复声明，就像图5-7b中的Reference_point属性。注意，在每个类中被继承的操作可以有不同方式的实现。例如，对于RECTANGLE、TRIANGLE和CIRCLE来说，area和perimeter操作的实现可以是不同的。

一个类对不同接口的多重继承是允许的，就像一个接口对其他不同接口的多重继承一样。但是，使用EXTENDS（类）继承，多重继承是不允许的。因此，通过EXTENDS，一个类最多可以从另一个类中继承（除了从零或多个接口中继承以外）。



(a) 图形模式表示

图5-7　说明通过“:”的接口继承

```
interface GeometryObject
{  attribute    enum      Shape{RECTANGLE, TRIANGLE, CIRCLE, . . . } Shape;
   attribute    struct    Point {short x, short y}   Reference_point;
   float        perimeter();
   float        area();
   void         translate(in short x_translation; in short y_translation);
   void         rotate(in float angle_of_rotation); };

class RECTANGLE : GeometryObject
(  extent       RECTANGLES)
{  attribute    struct    Point {short x, short y}   Reference_point;
   attribute    short     Length;
   attribute    short     Height;
   attribute    float     Orientation_angle; };

class TRIANGLE : GeometryObject
(  extent       TRIANGLES )
{  attribute    struct    Point {short x, short y}   Reference_point;
   attribute    short     Side_1;
   attribute    short     Side_2;
   attribute    float     Side1_side2_angle;
   attribute    float     Side1_orientation_angle; };

class CIRCLE : GeometryObject
(  extent       CIRCLES )
{  attribute    struct    Point {short x, short y}   Reference_point;
   attribute    short     Radius; };
. . .
```

(b) ODL中相应的接口和类的定义

图5-7 （续）

5.3 对象查询语言 OQL

对象查询语言（OQL）是专门为ODMG对象模型制定的查询语言。OQL在设计时就要求与编程语言紧密配合使用，这些编程语言定义了一个ODMG绑定，比如C++、Smalltalk以及Java。因此，嵌入到某种编程语言的一个OQL查询，可以返回与那种语言的类型系统相匹配的对象。另外，一个ODMG模式中的类操作的实现，可以通过这些编程语言来编写它们的代码。对于查询，OQL语法和关系型标准查询语言SQL的语法相似，只是增加了有关ODMG概念的特征，比如对象标识、复杂对象、操作、继承、多态性以及联系。

在5.3.1节中将讨论简单OQL查询的语法，以及使用命名对象或者类外延作为数据库入口点的概念。然后，在5.3.2节中讨论查询结果的结构，以及使用路径表达式来遍历对象中的联系。在5.3.3节讨论处理对象标识、继承、多态性等其他的OQL特性，以及其他面向对象的概念。这部分用来说明OQL查询的示例是基于图5-6中给出的UNIVERSITY数据库模式。

5.3.1 简单 OQL 查询、数据库入口点和迭代变量

与SQL一样，基本的OQL语法也是select…from…where…结构。例如，对于查询“检索‘Engineering’学院中所有系的名称”，可以写为：

```
Q0:   select    D.Dname
      from      D in DEPARTMENTS
      where     D.College = 'Engineering';
```

通常，每个查询都需要一个到数据库的**入口点**（entry point），它可以是任何命名的持久对象。对许多查询来说，入口点是一个类外延的名称。类外延的名称可以作为一个持久对象的名称，该持久对象的类型是该类对象的一个汇集（大部分情况下是一个集合）。见图5-6中的类外延名称，命名对象DEPARTMENTS的类型为set<DEPARTMENT>；PERSONS的类型为set<PERSON>；FACULTY的类型为set<FACULTY>等等。

Q0中的DEPARTMENTS就是一个类外延名称，它被用来作为一个入口点，表示对象的一个持久汇集。在一个OQL查询中，无论什么时候参照一个汇集，都应该定义一个**迭代变量**[①]，即在Q0中的*D*，来遍及该汇集中的每一个对象。在许多情况下，正如在Q0中一样，查询将从汇集中选择出特定的对象。这个选择基于where子句中指定的条件。在Q0中，对于汇集DEPARTMENTS中的持久对象*D*来说，只有满足条件*D*.College = 'Engineering'的对象才会被选择出来作为查询的结果。在该查询结果中，对于每一个被选择出来的对象*D*，*D*.Dname的值就被检索出来。因此，Q0的结果类型是bag<string>，因为每一个Dname的类型都是string（尽管实际结构是一个集合，但是Dname是码属性）。一般说来，“select…from…”的查询结果是包类型，而“select distinct…from…”的查询结果则是集合类型。这一点如同SQL（加上关键字distinct用于消除重复）。

在示例Q0中，有3种可选择的语法方式用来指定迭代变量：

D **IN** DEPARTMENTS
DEPARTMENTS *D*
DEPARTMENTS **AS** *D*

在本章的例子中，将使用第一种结构的语法[②]。

对于OQL查询，被用作数据库入口点并不局限于类外延的名称。任何命名的持久对象，无论它是表示一个原子（单个）对象，还是表示一个汇集对象，都可以用作数据库的入口点。

5.3.2　查询结果和路径表达式

一般说来，查询结果可以是在ODMG对象模型中能够被表达出来的任何类型。查询不必一定要遵循“select…from…where…”结构。在最简单的情形中，任何持久名称本身就是一个查询，其结果就是那个持久对象。例如，查询：

Q1:　DEPARTMENTS;

返回对所有持久DEPARTMENT对象汇集的一个引用，它的类型是set<DEPARTMENT>。类似地，假设对一个单独的DEPARTMENT对象（计算机科学系），给定（通过数据库绑定操作，见图5-4）一个持久对象的名称CS_DEPARTMENT，那么，查询：

Q1A:　CS_DEPARTMENT;

返回对类型为DEPARTMENT的单独对象的一个引用。一旦指定入口点，**路径表达式**（path expression）的概念就可以用来指定通向相关属性和对象的一条路径。典型地，一个路径表达式开始于一个持久对象名称，或者开始于遍及汇集中每个单独对象的迭代变量。这个名称之后会跟着零个或多个联系名称或属性名称，用圆点符号来连接。例如，对于图5-6中的UNIVERSITY数据库，下面是路径表达式的示例，它们也是OQL中的有效查询：

Q2:　CS_DEPARTMENT.Chair;
Q2A:　CS_DEPARTMENT.Chair.Rank;
Q2B:　CS_DEPARTMENT.Has_faculty;

第一个表达式Q2返回一个FACULTY类型的对象，因为DEPARTMENT类中的属性Chair的类型就

① 迭代变量类似于在SQL查询中遍及元组的元组变量。
② 注意：后面两种方式类似于SQL查询中指定元组变量的语法。

是FACULTY。这将是对FACULTY对象的一个引用，它通过属性Chair和持久名称为CS_DEPARTMENT的DEPARTMENT对象相关联；也就是说，是一个FACULTY对象的引用，该FACULTY是计算机科学系的系主任。第二个表达式Q2A类似，只不过它返回的是该FACULTY对象（计算机科学系主任）的级别（Rank），而不是对该对象的引用。因此，Q2A返回的类型是字符串，即FACULTY类中Rank属性的数据类型。

路径表达式Q2和Q2A都返回单独的值，因为属性Chair（DEPARTMENT类的Chair属性）和Rank（FACULTY类的Rank属性）都是单值的，并且它们都应用于一个单独的对象。第三个表达式Q2B则不同，即使应用于一个单独的对象，它也返回set<FACULTY>类型的一个对象，因为DEPARTMENT类中的联系Has_faculty的类型就是set<FACULTY>。这个返回的汇集包括所有FACULTY对象的引用，这些对象是通过联系Has_faculty和持久名称为CS_DEPARTMENT的DEPARTMENT对象相关联的；换句话说，它包括所有这样的FACULTY对象的引用，他们都是计算机科学系的教职员工。这时，如果要返回计算机科学系所有教员的级别，就不能写成下面的形式：

```
Q3′:   CS_DEPARTMENT.Has_faculty.Rank;
```

因为这样并不清楚返回的对象类型是集合set<string>，还是包bag<string>（后一种的可能性更大，因为多个教员可以有相同的级别）。因为这个类型的二义性问题，所以OQL不允许像Q3′这样的表达式。更准确地说，必须像下面Q3A或者Q3B那样，使用迭代变量来遍及这些汇集：

```
Q3A:   select    F.Rank
       from      F in CS_DEPARTMENT.Has_faculty;

Q3B:   select    distinct F.Rank
       from      F in CS_DEPARTMENT.Has_faculty;
```

这里，Q3A返回包bag<string>（结果中出现重复的级别rank值），Q3B则返回集合set<string>（通过关键字distinct消除了重复）。Q3A与Q3B都说明了如何在from子句中定义一个迭代变量，来遍及查询中特定的受约束的汇集。Q3A和Q3B中的变量*F*都是遍及汇集CS_DEPARTMENT. Has_faculty中的元素，该汇集的类型是set<FACULTY>，并且只包括计算机科学系的教职员工。

一般说来，使用struct关键字，OQL查询可以返回带有在查询自身中指定的复杂结构的结果。考虑下面两个示例：

```
Q4:    CS_DEPARTMENT.Chair.Advises;

Q4A:   select struct (  name:struct(last_name: S.name.Lname,
                        first_name: S.name.Fname),
                        degrees:( select struct (  deg: D.Degree,
                                                   yr: D.Year,
                                                   college: D.College)
                                  from D in S.Degrees)
       from S in CS_DEPARTMENT.Chair.Advises;
```

这里，Q4直接返回一个set<GRAD_STUDENT>类型的对象作为结果，这是那些由计算机科学系主任指导的研究生的汇集。现在，假设需要一个查询来检索这些研究生的姓名，以及每个人过去获得学位的列表，则可以写成Q4A，其中变量*S*遍及由系主任指导的研究生的汇集，变量*D*遍及每个学生*S*的学位。Q4A的结果类型是一个struct的汇集（第一级），每个struct有两个组件：name和degrees[①]。

name组件更深层的一个struct，是由last_name和first_name这两个单独的字符串组成的；degrees组件由一个嵌入的查询来定义，并且其本身是一个更深层的（第二级）struct的汇集，每个struct有3个字符串组件：deg、yr和college。

① 就像前面提到的，struct对应于第4章中讨论的元组构造器。

需要注意，关于指明的路径表达式，OQL是正交的（orthogonal）。也就是说，只要OQL的类型系统不受牵连，那么属性、联系和操作名称（方法）就能够在路径表达式中交替使用。例如，要检索所有主修计算机科学系的高年级学生的平均成绩，且其结果按平均成绩降序排列，相同成绩的学生再依次按照姓、名的升序排列，则可以编写如下的查询：

```
Q5A:  select struct ( last_name: S.name.Lname, first_name:
                      S.name.Fname, gpa: S.gpa )
      from            S in CS_DEPARTMENT.Has_majors
      where           S.Class = 'senior'
      order by        gpa desc, last_name asc, first_name asc;

Q5B:  select struct ( last_name: S.name.Lname, first_name: S.name.Fname,
                      gpa: S.gpa )
      from            S in STUDENTS
      where           S.Majors_in.Dname = 'Computer Science' and
                      S.Class = 'senior'
      order by        gpa desc, last_name asc, first_name asc;
```

Q5A使用命名的入口点CS_DEPARTMENT直接定位到计算机科学系，然后再通过联系Has_majors定位到学生；而Q5B查找STUDENTS类外延来定位到主修计算机科学系的所有学生。需要注意，在一个路径表达式中，属性名称、联系名称以及操作（方法）名称可交替地使用（正交性）。gpa是一个操作；Majors_in和Has_majors是联系；Class、Name、Dname、Lname以及Fname是属性。gpa操作的实现对于每个被选出的STUDENT计算其平均成绩，并且返回一个浮点类型的值。

order by子句类似于SQL中相应的结构，它指定查询结果按什么样的顺序来显示。因此，带有order by子句的查询返回的汇集是列表类型的。

5.3.3　OQL 的其他特性

1．指定视图作为命名查询

OQL中的视图机制使用了**命名查询**这个概念。关键字define用来指定该命名查询的标识，该命名查询的标识在该模式中是唯一的，所有命名对象、类名、方法名、或者函数名也必须都是唯一的。如果该标识和一个已存在的命名查询有相同的名称，则新的定义会替换以前的定义。一旦被定义，查询定义就是持久的，直到它被重新定义或者删除。视图在它的定义中也可以有参数（变元）。

例如，下面的视图V1定义了一个命名查询Has_minors，它用来检索辅修某个给定院系部门的学生的对象集合：

```
V1:   define    Has_minors(Dept_name) as
      select    S
      from      S in STUDENTS
      where     S.Minors_in.Dname = Dept_name;
```

因为图5-6中的ODL模式给每个STUDENT只提供了一个单向的Minors_in属性，所以可以使用上面的视图来表示它的逆，而不必明确地定义一个联系。这种类型的视图可以用来表示那些预期不经常使用的逆向联系。用户可以使用上面的视图来编写如下的查询：

```
Has_minors('Computer Science');
```

它返回在计算机科学系辅修的学生的一个包。注意，在图5-6中，已经将Has_majors定义为一个明确的联系，也许是因为预期它的使用会更加频繁。

2．从单一汇集中检索单个元素

通常，OQL查询将返回一个汇集作为其结果，比如包、集合（如果指明了distinct）或者列表（如果使用了order by子句）。如果用户需要一个只返回单个元素的查询，在OQL中有一个element操作符

用来保证从单一汇集*C*中返回单个元素*E*，其中*C*只包含一个元素。如果*C*包含多于一个的元素或者*C*为空，那么element操作符会引起一个异常。比如，Q6返回一个引用计算机科学系的单个对象：

```
Q6:   element  ( select  D
                 from    D in DEPARTMENTS
                 where   D.Dname = 'Computer Science');
```

因为在所有的系中，系名称是唯一的，所以结果应该是单个元素。结果的类型是*D*: DEPARTMENT。

3. 汇集操作符（聚集函数，量词）

因为许多查询表达式指定汇集作为它们的结果，所以定义了许多应用于这些汇集的操作符，包括聚集操作符（aggregate operator）、成员（membership）以及量词[quantification，包括全称（universal）量词和存在（existential）量词]。

聚集操作符（min、max、count、sum、avg）在汇集上的操作[①]：count操作返回一个整数类型。其他聚集操作符（min、max、sum、avg）返回的类型和应用该操作的汇集的类型相同。下面有两个示例。查询Q7返回辅修计算机科学的学生人数，Q8返回所有主修计算机科学的高年级学生的GPA。

```
Q7:   count (S in Has_minors('Computer Science'));

Q8:   avg ( select   S.Gpa
            from     S in STUDENTS
            where    S.Majors_in.Dname = 'Computer Science' and
                     S.Class = 'Senior');
```

需要注意，聚集操作可以应用于任何合适类型的汇集，并且可以用在查询的任何一部分中。比如，要检索主修人数超过100的所有系名称，这样的查询可以编写成如下形式：

```
Q9:   select   D.Dname
      from     D in DEPARTMENTS
      where    count (D.Has_majors) > 100;
```

成员和量词表达式返回一个布尔类型，即，真或假。令*V*是一个变量，*C*是一个汇集表达式，*B*是一个布尔类型的表达式（即一个布尔条件），*E*是一个汇集*C*中元素类型的元素。则有：

（*E* in *C*）当元素*E*是汇集*C*的成员时，返回真。

（for all *V* in *C* : *B*）当汇集*C*中的所有元素满足*B*时，返回真。

（exists *V* in *C* : *B*）当汇集*C*中至少有一个元素满足*B*时，返回真。

为了说明这些成员条件，假定我们要检索所有完成了课程“Database Systems Ⅰ”的学生姓名。这个查询可以编写成Q10，其中的子查询返回每个学生*S*已经完成的所有课程名称的汇集；对于某个特定的学生*S*，当“Database Systems Ⅰ”在该子查询返回的汇集中时，成员条件返回真：

```
Q10:  select   S.name.Lname, S.name.Fname
      from     S in STUDENTS
      where    'Database Systems I' in
               ( select   C.Cname
                 from     C in S.Completed_sections.Section.Of_course);
```

Q10也说明了一种较为简单的方法，用来指定返回一个struct汇集的查询select子句。Q10返回的类型是包bag<struct(string，string)>。

也可以编写返回真/假结果的查询。例如，假定存在一个STUDENT类型的名为JEREMY的命名对象。于是，查询Q11回答了下面的问题：“JEREMY辅修了计算机科学吗？”。类似地，Q12回答了：

---

① 这些操作对应于SQL中的聚集函数。

“所有的计算机科学研究生都由计算机科学的教师指导吗？”。Q11和Q12都返回真或假，可以解释成上述问题的答案：是或否。

```
Q11:  JEREMY in Has_minors('Computer Science');

Q12:  for all G in
      ( select    S
        from      S in GRAD_STUDENTS
        where     S.Majors_in.Dname = 'Computer Science' )
      : G.Advisor in CS_DEPARTMENT.Has_faculty;
```

需要注意，查询Q12也说明了属性、联系以及操作继承是如何应用于查询的。*S*是一个遍及类外延GRAD_STUDENT的迭代变量，也可以写作*S*.Majors_in，因为Majors_in联系被GRAD_STUDENT从STUDENT中通过扩展继承了（见图5-6）。最后，为了说明存在量词exists，查询Q13回答了“是否有主修计算机科学的研究生的*gpa*为4.0？”的问题。这里，操作gpa同样也被GRAD_STUDENT通过扩展从STUDENT中继承下来。

```
Q13:  exists G in
      ( select    S
        from      S in GRAD_STUDENTS
        where     S.Majors_in.Dname = 'Computer Science' )
      : G.Gpa = 4;
```

4. 有序（索引）汇集表达式

正如在5.1.2节中所讨论的那样，汇集中的列表和数组有一些附加的操作，比如，检索第*i*个、第一个或者最后一个元素。另外，还有抽取一个子汇集和合并两个列表的操作。因此，包含列表和数组的查询表达式可以引用这些操作。下面将通过示例来说明其中的某一些操作。Q14检索薪水最高的教员的姓氏：

```
Q14:  first ( select    struct(fname: F.name.Lname, salary: F.Salary)
              from      F in FACULTY
              order by  F.Salary desc);
```

Q14说明了在一个列表汇集上使用first操作的方法，该列表包含了教员的薪水并且按照薪水的降序排列。这样该有序列表的第一个元素就包含了薪水最高的教员。这个查询假定只有一个教员享有最高的薪水。下一个查询Q15检索出主修计算机科学的平均成绩GPA排在前三名的学生：

```
Q15:  ( select    struct( last_name: S.name.Lname, first_name: S.name.Fname,
                          gpa: S.Gpa )
        from      S in CS_DEPARTMENT.Has_majors
        order by  gpa desc) [0:2];
```

这个select-from-order-by查询返回了一个按gpa降序排列的计算机科学系学生的列表。一个有序汇集中的第一个元素的索引位置是0 ，因此表达式[0：2]返回的列表就包含了select…from…orderby结果中的前3个元素。

5. 分组操作符

OQL中的group by子句和SQL中相应的子句类似，在每一个分组或者分区中提供了对象汇集的显式引用。首先给出一个示例，然后描述这些查询的一般形式。

Q16检索每个系的主修人数。在这个查询中，主修专业相同的学生被划分到相同的分区（组），也就是说，这些学生的*S*.Majors_in.Dname有相同的值。

```
Q16:  ( select    struct( dept_name, number_of_majors: count (partition) )
        from      S in STUDENTS
        group by  dept_name: S.Majors_in.Dname;
```

该分组说明结果的类型是集合set<struct (dept_name: string, partition: bag<struct(*S*: STUDENT)>)>，它对每一个分组（分区）都包含一个struct，这个struct由两个组件组成：分组属性值（dept_name）和该分组（分区）中STUDENT对象组成的包。select子句返回了分组属性（系名称）以及每个分区中元素数目的统计值（即，每个系中的学生人数），其中关键字partition用来指代每个分区。select子句返回的类型是set<struct (dept_name:string, number_of_majors: integer)>。一般说来，group by子句的语法如下：

group by F_1: E_1, F_2: E_2, … , F_k: E_k

其中F_1: E_1, F_2: E_2, …,F_k: E_k是分区（分组）属性和每个分区属性的描述的列表，F_i: E_i定义了一个属性（字段）名F_i和一个表达式E_i。应用分组（在group by子句中指定）的结果是一个结构的集合：

set<struct(F_1: T_1, F_2: T_2, … , F_k: T_k, partition: bag<B>)>

其中T_i是表达式E_i返回的类型，partition是一个识别字段名（或关键字）；B是一个结构，B的字段是在from子句中声明的合适类型的迭代变量（Q16中的S）。

正如在SQL中一样，having子句可以用来过滤分区的集合，也就是说，只是基于分组条件选择一些分组。在Q17中，修改了前面的查询来说明having子句的用法（也显示了select语句的简化语法）。Q17检索每个超过100名主修学生的系，和这些主修学生的平均GPA。Q17中的having子句只选择了那些超过100个元素（即，超过100个学生的系）的分区（分组）。

```
Q17:  select      dept_name, avg_gpa: avg ( select P.S.gpa from P
                                            in partition)
      from        S in STUDENTS
      group by    dept_name: S.Majors_in.Dname
      having      count (partition) > 100;
```

需要注意，Q17的select子句返回了分区中学生的平均GPA。表达式：

select $P.S$.Gpa **from** P **in partition**

返回分区中学生GPA的包。from子句声明了一个遍及分区汇集的迭代变量P，它的类型是包bag<struct (S: STUDENT)>。路径表达式$P.S$.gpa用来在分区中访问每个学生的GPA。

5.4　C++语言绑定概述

C++语言绑定指明了从ODL构造如何映射到C++构造。这是通过C++的一个类库完成的，该类库提供了实现ODL构造的类和操作。需要有一个对象操纵语言（Object Manipulation Language，OML）来指明在C++程序中如何检索和操纵一个数据库对象，而这又是基于C++编程语言的语法和语义的。另外，对于ODL/OML绑定，定义了一个称为物理附注（physical pragmas）的构造集合，以便允许程序员控制某些物理存储问题，比如对象的聚簇、索引的使用以及存储管理。

为ODMG标准增加的C++的类库中，使用前缀d_[①]来处理数据库概念的类声明。其目的是让程序员觉得自己只使用了一种语言，而不是两种相对独立的语言。为了让程序员在一个程序中引用数据库对象，对模式中的每个数据库类T都定义了一个类D_Ref<T>。这样，D_Ref<T>类型的程序变量既可以用来表示类T的持久对象，也可以用来表示类T的临时对象。

为了在ODMG对象模型中使用各种不同的内置类型，例如汇集类型，在类库中指定了各种模板类。例如，抽象类D_Object<T>指定了所有对象都可以继承的操作。类似地，抽象类D_Collection<T>指定了汇集的操作。这些类不是可实例化的，它们只是指定了可以被所有对象和汇集对象各自继承的操作。一个模板类是为一种汇集模型指定的，它们包括D_Set<T>、D_List<T>、D_Bag<T>、

① 假定d_表示数据库类。

D_Varray<*T*>以及D_Dictionary<*T*>，这些与对象模型（见5.1节）中的汇集类型相对应。因此，程序员可以创建例如D_Set<D_Ref<STUDENT>>类型的类，它的实例则是字符串的集合。另外，类d_Iterator对应于对象模型的迭代器类。

C++ ODL不仅允许用户使用对象数据库的类库所提供的构造来指定一个数据库模式的类，也允许用户使用C++的构造来指定数据库模式的类。为了指明属性的数据类型[①]，可以使用一些基本类型，比如短整型（d_Short）、无符号短整型（d_UShort）、长整型（d_Long）以及浮点型（d_Float）。除了这些基本的数据类型，还存在一些结构化的文字类型，它们和ODMG对象模型中结构化的文字类型相对应，包括字符串（d_String）、间隔（d_Interval）、日期（d_Date）、时间（d_Time）以及时间戳（d_Timestamp）。（见图5-1b）。

关键字rel_用来作为类型名称的前缀，来指定联系。例如，在STUDENT类中编写：

```
d_Rel_Ref<DEPARTMENT, _has_majors> Majors_in;
```

以及在DEPARTMENT类中编写：

```
d_Rel_Set<STUDENT, _majors_in> Has_majors;
```

这里声明的Majors_in和Has_majors是一对逆向的联系特性，它们体现了DEPARTMENT和STUDENT之间1:N的二元联系。

对于OML，绑定重载操作*new*，能够用于创建持久对象或者临时对象。要创建持久对象，就必须提供数据库名称和该对象的持久名称。例如，编写：

```
D_Ref<STUDENT> S = new(DB1, 'John_Smith') STUDENT;
```

程序员可以在数据库DB1中创建一个命名的持久类型对象STUDENT，其持久名为John_Smith。另一个操作delete_object（）能够用来删除对象。对象的修改是由程序员定义在每个类中的操作（方法）来完成的。

C++绑定同样允许使用库类d_Extent来创建extent。例如，编写：

```
D_Extent<PERSON> ALL_PERSONS(DB1);
```

程序员可以在数据库DB1中创建一个命名的汇集对象All_PERSONS，它的类型是D_Set<PERSON>，其中包含PERSON类型的持久对象。但是，在C++绑定中不支持码约束，并且任何码检查必须在类的方法中编程实现[②]。同时，C++绑定也不支持通过可达性实现的持久性，对象必须在它被创建的时刻静态地声明为持久对象。

5.5　对象数据库概念设计

5.5.1节讨论对象数据库（ODB）设计与关系数据库（RDB）设计的区别。5.5.2节给出了一个映射算法，可以用来从概念EER模式中创建由ODMG ODL类定义组成的ODB模式。

5.5.1　ODB 与 RDB 概念设计的区别

在对象数据库设计与关系数据库设计之间，一个最主要的区别是如何处理联系。在对象数据库中，联系典型地是通过使用联系特性或者包括相关对象的OID的参照属性来处理，这可以认为是相关对象的*OID*参照。单个参照或者参照汇集都是允许的。二元联系的参照可以单向声明，也可以双向声明，这取决于预期的存取类型。如果双向声明，它们可以被指定为一对互逆的联系，这样就实施了关系参照完整性约束的ODB等价性。

① 也就是面向对象编程术语中的成员变量。

② 这里只给出了C++绑定的一个简要概述，详细内容请参照（Cattell et al.，1997）中的第5章。

在关系数据库中，元组（记录）中的联系是通过匹配值的属性来指定的。这可以认为是通过外码来指定的值参照，它是重复出现在参照关系元组中的主码属性的值。因为在基本的关系模型中多值属性是不允许的，所以它们在每个记录中被限制为单值。这样M:N联系不可以直接地表示，而必须表示成单独的关系（表）（见《数据库系统基础：初级篇》的7.1节中相关的讨论）。

在对象数据库中，包含属性的二元联系的映射不是直接的，因为设计者必须选择属性应该被包含在哪个方向上。如果两个方向中都包含属性，那么就存在存储冗余，并且还会导致不一致的数据。因此，有时更希望通过创建一个单独的类来创建单独的表，使用这种关系型的方法来表示联系。这种方法也可以用在$n>2$的n元联系。

对象数据库设计与关系数据库设计的另一个重要的区别在于如何处理继承。在对象数据库中，这些结构是内置在模型中的，因此可以通过使用继承构造来完成映射，例如导出（:）和*EXTENDS*。在关系型的设计中，就像在《数据库系统基础：初级篇》的7.2节中所讨论的，由于在基本的关系模型中不存在内置的构造用来继承，所以有几种可以选择的方案。有一点很重要，对象-关系系统和扩展-关系系统都增加了一些特性，可以直接建立这些构造，也可以包括抽象数据类型中的操作说明（见第6章）。

二者的第三个主要区别是：在对象数据库设计中，有必要在设计时尽早指定操作，因为它们是类描述中的一部分。尽管对于所有类型的数据库，在设计阶段确定操作都是重要的，但是在关系数据库设计中却可以延迟，因为在RDB中直到实现阶段它才被严格要求。

在关系模型和对象模型的行为说明中有一个概念上的不同。关系模型不会提前定义一组有效的行为或操作，在对象模型中则是必须的。关系模型中的一个公认的优点是支持特定查询和事务处理，尽管这似乎违背封装原理。

实际上，数据库设计者在概念设计阶段就使用基于面向对象的方法，已经是很普遍的策略，所以结构与使用或操作数据，以及一个完整的规范说明文档在概念设计阶段都会被考虑到。这些规范说明将会被映射到关系模式、约束和行为动作中，例如触发器和存储过程（见《数据库系统基础：初级篇》的8.7节和9.4节）。

5.5.2　EER 模式到 ODB 模式的映射

从一个既不包含类别也不包含$n>2$的n元联系的EER模式中，为ODBMS设计对象类的类型声明相对比较简单。然而，类的操作并不在EER图中指明，而必须在结构映射完成后加入类声明中。从EER到ODL的映射应包括如下步骤：

第一步：为每个EER实体类型或者子类创建一个ODL类。ODL类的类型应该包括这个EER类的所有属性[①]。多值属性通过使用集合、包或者列表构造器来声明[②]。如果一个对象的多值属性的值是有序的，则要选择列表构造器；如果允许多值属性的值重复，则应该选择包构造器；否则就选择集合构造器。组合属性映射到元组构造器（通过使用ODL中的结构声明）。

为每个类声明一个外延，并且指定一个码属性作为该类外延的码（只有当类外延功能和码约束声明在ODBMS中是有效时，这才是可能的）。

第二步：把联系特性或者每个二元联系的参照属性，加入到参与该联系的ODL类中。这既可以单向创建，也可以双向创建。如果一个二元联系是通过双向参照来表示，则将参照声明为一对互逆的联系特性（如果支持这个功能）[③]。如果二元联系只通过单向参照来表示，则把该参照声明成参照类中的一个属性，它的类型是被参照的类的名称。

---

① 这在类型声明的顶层隐式地使用元组构造器，但是一般说来，元组构造器并不显式地列在ODL的类声明中。

② 要决定使用哪种构造器，则需要对应用领域的进一步分析，因为从EER模式中不能得到这个信息。

③ ODL标准提供了逆向联系的显式定义。一些ODBMS产品可能不提供这种支持，在这种情况下，程序员必须通过编写适当的更新对象的方法来显式地维护每一个联系。

根据二元联系的基数比，联系特性或者参照属性可以是单值的类型，也可以是汇集类型。对于1:1或N:1的二元联系，它们是单值的；对于1:N或M:N的二元联系，它们则是汇集类型（集合值的或者列表值的[①]）。映射二元M:N联系的另一种方法将在下面的第七步中讨论。

如果联系属性存在，则可以使用元组构造器创建形如<reference, relationship attributes>的结构，来代替参照属性。但是，这不允许使用逆向的约束。另外，如果这种选择是双向表示的，那么属性值会被表示两次，这就产生了冗余。

第三步：为每个类包含合适的操作。这是不能从EER模式中获得的，必须参照原始需求，加入到数据库设计中去。构造方法应该包含检查约束的程序代码，创建新的对象时必须满足这些约束。析构器方法则应该检查那些删除对象时可能破坏的约束。其他的方法应该包含一些相关的进一步的约束检查。

第四步：对应EER模式中一个子类的ODL类，继承（通过EXTENDS）在ODL模式中的超类的类型和方法。它专有的（非继承的）属性、参照联系以及操作的指定在前三步中已经讨论。

第五步：弱实体类型可以采用与常规实体类型相同的方法来映射。对于除了其标识联系外不参与任何联系的弱实体类型来说，另一种映射也是可行的：即，使用set<struct<…>>或者list<struct<…>>构造器，它们可以被当作其属主实体类型的复合多值属性那样映射。该弱实体的属性包含在struct<…>构造器中，它对应于一个元组构造器。属性的映射同第一步和第二步中所讨论的那样。

第六步：EER模式中的类别（union类型）映射到ODL比较困难。通过声明一个表示该类别的类，并且在类别和它的每个超类之间定义1:1联系，来创建类似于EER到联系的映射（见《数据库系统基础：初级篇》的7.2节）的一个映射是可行的。另一种选择是，如果union类型可用，则使用union类型。

第七步：度n>2的n元联系可以映射到一个单独的类，并对每个参与类有合适的参照。这些参照基于一个1:N联系的映射，该映射是从表示参与实体类型的每一个类到表示该n元联系的类。一个M:N二元联系，特别是如果它包含联系属性时，也可以使用这种映射方法。

该映射按照ODMG对象数据库标准应用于《数据库系统基础：初级篇》的图4.10中UNIVERSITY数据库模式的一个子集；映射后的对象模式所使用的ODL符号在图5-6中列出。

小结

本章讨论了为面向对象数据库提出的ODMG 2.0标准。从描述ODMG对象模型的各种构造开始，各种内置类型，比如对象、汇集、迭代器、集合、列表等等都是通过它们的接口来描述的，接口还指定了每种类型的内置操作。这些内置类型是对象定义语言（ODL）和对象查询语言（OQL）的基础。本章还描述了对象与文字之间的区别，前者有对象标识符OID，而后者则是值，没有OID。用户可以为他们的应用声明类，并从适当的内置接口中继承操作。在一个用户定义的类中，可以指定两种类型的特性，即属性和联系，还可以指定能够应用于类对象的操作。ODL允许用户指定接口和类，并且允许两种不同的继承类型，分别是通过“:”的接口继承和通过EXTENDS的类继承。一个类可以有一个外延和多个码。

接着介绍了ODL，并且通过一个关于UNIVERSITY数据库的数据库模式的示例说明了ODL构造。然后，概述了对象查询语言（OQL）。OQL在查询的构造上遵循正交性概念，也就是说，只要一个操作的结果符合另一个操作的正确输入类型，就可以将该结果应用于那个操作。OQL语法遵循SQL中的许多构造，但是还包括一些补充的概念，比如路径表达式、继承、方法、联系以及汇集。通过示例给出了如何对UNIVERSITY数据库使用OQL的说明。

---

① 从EER模式中不能决定使用集合还是列表，这一点必须进一步考虑需求。

接下来给出了C++语言绑定，它使用ODL类型构造器扩展了C++的类声明，允许C++与对象数据库管理系统（ODBMS）的无缝集成。

在描述了ODMG模型之后，又描述了设计面向对象数据库模式的一般技术，讨论了面向对象数据库与关系型数据库在3个主要领域中的区别：表示联系参照、包含操作和继承。最后，概要地介绍了支持分布式对象系统中的互操作性的CORBA标准，以及它和对象数据库标准的关系。

在1997年，Sun认可了ODMG API（应用程序接口）。O2是第一家开发支持ODMG的数据库系统的公司。还有很多OODBM的参与者，包括Object Design（现在的eXcelon公司），Gemstone systems，Poet Software，还有Versant Object Technology，都批准了ODMG标准。

复习题

5.1　在ODMG对象模型中，对象和文字有哪些相同和不同之处？

5.2　列出以下ODMG对象模型中的内置接口的基本操作：对象、汇集、迭代器、集合、列表、包、数组、字典。

5.3　描述ODMG对象模型中的内置结构化文字以及它们的操作。

5.4　在一个用户定义的（原子的）类中，属性和联系的性质有哪些相同和不同之处？

5.5　EXTENDS继承和接口“:”继承有哪些相同和不同之处？

5.6　讨论在ODMG对象模型中，如何使用C++绑定指定持久性？

5.7　在数据库应用中，外延和码的概念为什么非常重要？

5.8　描述下列OQL概念：数据库入口点、路径表达式、迭代变量、命名查询（视图）、聚集函数、分组、谓词。

5.9　OQL的类型正交性的含义是什么？

5.10　讨论ODMG标准中的C++绑定的一般的原则。

5.11　设计关系数据库与设计对象数据库最主要的区别是什么？

5.12　描述对象数据库设计的从EER到OO映射的算法步骤。

练习题

5.13　针对你自己感兴趣的数据库应用，设计一个OO模式。首先为这个应用构造一个EER模式，然后在ODL中创建对应的类。为每个类指定一些方法，并且为数据库应用指定一些使用OQL的查询。

5.14　考虑练习题4.21中的AIRPORT数据库。为这个应用定义一些你认为合适的操作/方法，并且为这个数据库指定ODL类和方法。

5.15　将《数据库系统基础：初级篇》的图3-2中COMPANY的ER模式映射到ODL类。对于每个类应该包括适当的方法。

5.16　对于《数据库系统基础：初级篇》的第7章和第8章中应用到COMPANY数据库的练习题，用OQL编写其中的查询。

5.17　使用搜索引擎或其他资源，列举有哪些商业OODBMS产品使用了ODMG3.0标准。

参考文献

（Cattell et al.，1997）一书中描述了ODMG 2.0标准，而且（Cattell et al.，1993）书中描述了该标准的早期版本。还有一些描述CORBA体系结构的书，例如，（Baker，1996）。其他关于面向对象的数据库的资料已经在第4章的参考文献中列出。

（Deux et al.，1991）一书中描述了O2系统，而（Bancilhon et al.，1992）一书中则列出了其他一些描

述O2各个方面内容的出版物。O2模型正式定义是在（Velez et al.，1989）一书中。（Lamb et al.，1991）一书描述了ObjectStore系统。（Fishman et al.，1987）和（Wilkinson et al.，1990）论述了IRIS，并且在Hewlett-Packard实验室中开发了一个面向对象的数据库管理系统（DBMS）。（Maier et al.，1986）和（Butterworth et al.，1991）描述了GEMSTONE的设计。在（Thompson et al.，1993）一书中描述了在Texas Instruments开发的一个支持开放体系结构的OO系统。（Agrawal & Gehani，1989）则介绍了在AT&T Bell实验室开发的ODE系统。（Kim et al.，1990）一书描述了MCC开发的ORION系统。（Morsi et al，1992）一书介绍了一个面向对象的测试平台。

（Cattell，1991）研究了关系数据库和对象数据库的概念，还讨论了一些基于对象的原型系统，扩展了关系数据库系统。（Alagic，1997）指出ODMG数据模型和它的语言绑定和意图的一些定义。（Bertino & Guerrini，1998）为组合对象扩展了ODMG模型。（Alagic，1999）定义了属于ODMG模型的一些数据模型。

第6章

对象-关系和扩展-关系系统

我们已经主要讨论了3种数据模型：《数据库系统基础：初级篇》第3章和第4章中的实体-联系（ER）模型及它的增强版，即EER模型，第5章至第9章中的关系数据模型及其语言与系统和本书第4章和第5章中的面向对象数据模型及对象数据库语言和标准。我们按照以下特性讨论了已被完全开发的这些数据模型：

- 开发数据库应用模式的建模构造。
- 表示特定类型联系的约束工具以及应用语义所确定的数据约束。
- 操纵数据库的操作和语言工具。

这3种模型中，ER模型及其变种主要用于数据库和软件设计的CASE工具，另外两种模型则是商业DBMS的基础。本章主要讨论一种新兴的商业DBMS，称为*对象-关系系统*（object-relational system）或*扩展关系系统*（extended relational system），以及这些系统的一些基本概念。这些系统通常被称为对象-关系DBMS（ORDBMS），它作为一种新兴的增强关系DBMS（RDBMS）能力的方法，具备了对象DBMS（ODBMS）中出现的一些特性。

6.1节首先给出SQL:99标准的一个概述，它为RDBMS的SQL标准提供了扩展的对象功能。6.2节我们回顾数据库技术演变的历史，并介绍了当前的发展趋势，以便读者理解这些系统出现的原因。6.3节作为商业扩展ORDBMS的一个例子，给出Informix数据库服务器的简要介绍。6.4节讨论Oracle的对象-关系和扩展特性。6.5节讨论有关实现扩展-关系系统的一些问题。6.6节给出嵌套关系模型的简要介绍，该模型提供带有复杂对象的扩展-关系模型的一些理论基础。

对ORDBMS的典型特性感兴趣的读者只需阅读6.1节到6.4节，在介绍性的课程中可以略过本章的其他部分。

6.1 SQL及其对象-关系特性概述

在《数据库系统基础：初级篇》的第8章中已经介绍了作为RDBMS标准语言的SQL。如前所述，SQL首先由（Chamberlin & Boyce，1974）提出，并于1989年和1992年有所改进。SQL语言还在继续向着被称为SQL3的新标准演变，这个标准增加了面向对象以及其他的一些特性。SQL3标准的一个子集，称为SQL:99已经得到了认可。这一节主要介绍SQL3和SQL:99中的一些特性，重点是对象-关系概念。

6.1.1 SQL标准及其组件

本节将简单介绍SQL标准各个部分的用途，然后描述与SQL对象扩展相关的一些SQL特性。SQL标准目前包括以下部分[①]：

- SQL/框架、SQL/基础、SQL/绑定、SQL/对象。
- 寻求SQL时态、事务及其他方面的新部分。
- SQL/CLI（调用层接口，Call Level Interface）。

① 有关标准的讨论主要基于（Melton & Mattos，1996）。

- SQL/PSM（持久存储模块，Persistent Stored Modules）。

SQL/基础处理新数据类型、新谓词、关系操作、游标、规则和触发器、用户定义类型、事务功能及存储例程（stored routine）。SQL/CLI（调用层接口）（见《数据库系统基础：初级篇》的9.3节）提供了一些规则，这些规则允许执行应用代码而无需提供源代码，这就避免了预处理。它大约包含了50个例程用于完成各种任务，例如连接至SQL服务器、分配与释放资源、获得诊断和执行信息、控制事务终止等。SQL/PSM（永久存储模块）（见《数据库系统基础：初级篇》的9.4节）指定了在客户机与服务器之间划分应用的工具。其目的是通过最小化网络流量来改进性能。SQL/绑定包括嵌入式SQL和直接调用。嵌入式SQL已经得到改进，包含了附加的异常声明。SQL/时序由TSQL2委员会提出[①]，负责处理历史数据、时间序列数据以及其他时态外延。SQL/事务规范说明形式化XA接口，以便可被SQL实现所使用。

6.1.2　SQL-99 中的对象-关系支持

SQL/对象规范说明扩展了SQL-92，使之包含面向对象功能。在此将引用在第4章讨论的相应的面向对象概念来讨论这些特性。下面是SQL-99中包含的部分特性。

- 增加了一些**类型构造器**（type constructor）来指定复杂对象。包括行类型（row type），它对应于第4章提到的元组（或结构）构造器；还给出了用于定义汇集的数组类型。尽管一些系统已经包括了其他汇集类型构造器，例如集合、列表和包构造器，并且这些类型构造器最终将会加入到将来的版本中，但SQL-99规范说明中尚未包含这些类型构造器。
- 包括了一种使用引用类型来指定**对象标识**的机制。
- 提供了**操作封装**，这是通过用户自定义类型机制将操作作为声明的一部分。
- 提供了**继承**机制。

下面将更详细地讨论以上各个概念。

1．类型构造器

行和数组类型构造器用于指定复杂类型。这些复杂类型也称为**用户定义类型**（UDT），因为这是用户为特定应用而定义的。**行类型**可以使用如下语法定义。

```
CREATE TYPE ROW_TYPE_NAME AS [ ROW ] (<component declarations>);
```

关键字ROW是可选的。指定地址和员工的行类型示例可以写作：

```
CREATE TYPE ADDR_TYPE AS (
    Street   VARCHAR (45),
    City     VARCHAR (25),
    Zip      CHAR (5)
);

CREATE TYPE EMP_TYPE AS (
    Name   VARCHAR (35),
    Addr   ADDR_TYPE,
    Age    INTEGER
);
```

需要注意，我们可以使用一个已定义类型作为属性的类型，如以上的Addr属性。可以指定一个**数组类型**，它的值可以是一个汇集。例如，假设一个公司有多达10个位置，那么该公司的行类型可定义如下：

```
CREATE TYPE COMP_TYPE AS (
    Comp_name      VARCHAR (20),
    Location       VARCHAR (20) ARRAY [10]  );
```

---

① 完整的提议见（Snodgrass & Jensen，1996）。我们将在第8章讨论时态建模并介绍TSQL2。

使用常用的方括号表示法来引用定长数组类型中的元素。例如，Location[1]表示Location属性中的第一个位置的值。对于行类型，可用常用的点表示法引用组件。例如：Addr.City表示Addr属性的City组件。目前，数组元素本身不能是数组，这就限制了可创建的对象结构的复杂性。

2．使用引用的对象标识符

用户定义类型可以用做属性类型，如：EMP_TYPE的Addr属性，也可用来指定表的行类型。例如，可以基于前面给出的行类型声明创建两个表，如下所示：

```
CREATE TABLE EMPLOYEE OF EMP_TYPE REF IS Emp_id
   SYSTEM GENERATED;

CREATE TABLE COMPANY OF COMP_TYPE (
   REF IS Comp_id SYSTEM GENERATED,
   PRIMARY KEY (Comp_name) );
```

上述示例说明了对于要创建的表中的单个行，用户如何指定系统生成的对象标识符。通过使用如下语法：

```
REF IS <oid_attribute> <value_generation_method> ;
```

用户声明了名为<oid_attribute>的属性，用于标识表中的单个元组。<value_generation_method>可以是SYSTEM GENERATED（系统生成）或DERIVED（派生）。对于前者，系统将为每个元组自动生成一个唯一标识符。对于后者，则将应用传统的方法，使用用户提供的主码值来标识元组。

元组的组件属性可能是另一个（也可以是同一个）表的元组的引用（使用关键字REF指定）。例如，可以定义如下附加行类型和相应表，从而使员工与公司相关联：

```
CREATE TYPE EMPLOYMENT_TYPE AS (
   Employee REF (EMP_TYPE) SCOPE (EMPLOYEE),
   Company REF (COMP_TYPE) SCOPE (COMPANY) );

CREATE TABLE EMPLOYMENT OF EMPLOYMENT_TYPE;
```

关键字SCOPE指定了表名，该表的元组可以由引用属性引用。需要注意，这与外码很类似，只不过使用的是系统生成的值而不是主码值。

SQL使用**点表示法**建立引用元组以及行类型的组件属性的**路径表达式**。但是，对于REF类型的属性，要使用解除引用符号->。例如，下面的查询通过查询EMPLOYMENT表来检索在‘ABCXYZ’公司工作的员工：

```
SELECT  E.Employee->Name
FROM    EMPLOYMENT AS E
WHERE   E.Company->Comp_name = 'ABCXYZ';
```

在SQL中，->用于**解除引用**（dereferencing），这与C程序设计语言中->的含义相同。因此，如果 r 是元组的引用，a 是该元组中的一个组件属性，那么 $r->a$ 就是该元组中的属性 a 的值。

对象标识符（object identifier）也可以在类型定义中而不是在表声明中显式地声明。例如，EMP_TYPE的定义可修改如下。

```
CREATE TYPE EMP_TYPE AS (
   Name      CHAR (35),
   Addr      ADDR_TYPE,
   Age       INTEGER,
   Emp_id    REF (EMP_TYPE)
);
```

在上面的示例中，使用下面的命令可以指定Emp_id值为系统生成值：

```
CREATE TABLE EMPLOYEE OF EMP_TYPE
VALUES FOR Emp_id ARE SYSTEM GENERATED;
```

如果存在相同行类型的若干关系，SQL提供了关键字SCOPE，通过SCOPE可以引用属性指向该类型的一个特定表，其语法如下：

SCOPE FOR <attribute> **IS** <relation>

3．SQL中的操作封装

SQL提供了类似于类定义的构造，用户可以通过该构造创建命名的**用户定义类型**，除了指定属性外，通过指定方法（或操作），用户定义类型可以有自己的行为规范说明。带方法的UDT（用户定义类型）的一般形式是：

```
CREATE TYPE <type-name>(
    带有单独类型的组件属性列表
    EQUAL和LESS THAN函数声明
    其他函数（方法）声明
);
```

例如，对于前面声明的ADDR_TYPE行类型，假设希望从构造其Street属性组件的一个串中抽取房间号（如果提供），可以为ADDR_TYPE指定一个方法：

```
CREATE TYPE ADDR_TYPE AS (
   Street  VARCHAR (45),
   City    VARCHAR (25),
   Zip     CHAR (5) )
METHOD apt_no() RETURNS CHAR (8);
```

还要编写实现这个方法的代码。通过指定包含该方法代码的文件，可以引用该方法实现，如下所示：

```
METHOD
CREATE FUNCTION apt_no() RETURNS CHAR (8) FOR ADDR_TYPE AS
EXTERNAL NAME '/x/y/aptno.class' LANGUAGE 'java';
```

在这个示例中，方法的实现用Java语言编写，代码存储在指定的文件路径中。

SQL为用户定义类型提供了某些内置函数。对于名为TYPE_T的UDT，**构造函数**TYPE_T()返回该类型的一个新对象。在这个新的UDT对象中，每个属性都初始化为其默认值，会为每个属性*A*隐式地创建一个**观测函数***A*来读取其值。因此，如果*X*的类型是TYPE_T，则*A*(*X*)或*X*.*A*会返回TYPE_T的属性*A*的值。更新属性的**变异函数**将属性值设置为一个新值。SQL允许锁定这些函数，不公开使用，访问这些函数需要有EXECUTE权限。

UDT通常有大量与之相关的用户定义函数。其语法是：

METHOD <name> (<argument_list>) **RETURNS** <type>;

可以定义两种类型的函数：内部SQL函数和外部SQL函数。内部函数用SQL的扩展PSM语言编写（见《数据库系统基础：初级篇》的第9章）。外部函数用宿主语言编写，仅签名（接口）出现在UDT定义中。外部函数定义可以声明如下：

```
DECLARE EXTERNAL <function_name> <signature>
LANGUAGE <language_name>;
```

许多ORDBMS中都采用了以下方法，即定义一个抽象数据类型（Abstract Data Type, ADT）包，并为特定应用领域定义相关的函数，后者可以在基本系统之外单独购买。例如，Informix Universal Server中的数据刀片和Oracle中的数据配件以及DB2中的扩展包等，都可以看做是特定应用领域的ADT之类的包或库。

ADT可以作为SQL3中的属性类型，也可以作为函数或过程中的参数类型，或者作为区分类型中的源类型。SQL中定义的**类型等价**分为两个层次：当且仅当两个类型同名时，这两个类型是**名字等价**；当且仅当两个类型的组件数量相同，并且其中的成员类型两两等价的时候，两个类型是**结构等价**的。

UDT中的属性和函数分成3类：

- PUBLIC（在UDT接口可视）。
- PRIVATE（在UDT接口不可视）。
- PROTECTED（仅对子类可视）。

也可以定义虚属性作为UDT的一部分，这要通过函数来进行计算和更新。

4．SQL中的继承与函数重载

第4章中已经讨论过许多继承原则。SQL提供了一些规则来处理继承（通过关键字UNDER来指定）。与继承关联的是函数实现的重载规则，以及函数名的解析规则。这些规则可以总结如下：

- 所有属性都可以被继承。
- UNDER子句中超类型的顺序确定了继承的层次。
- 在使用超类型实例的任何环境中都可以使用子类型的实例。
- 只要签名相同，子类型可以重定义它的超类型中已定义的任何函数。
- 当调用被函数时，要基于所有变元类型选择最佳匹配。
- 对于动态链接，要考虑参数的运行时类型。

考虑如下示例来说明类型继承。假设需要创建一个子类型MANAGER_TYPE，它要继承EMP_TYPE的所有属性（与方法），而且还有一个附加属性Dept_managed。则可以编写如下的语句：

```
CREATE TYPE MANAGER_TYPE UNDER EMP_TYPE AS (
    Dept_managed    CHAR (20) );
```

它继承了超类型EMP_TYPE的所有属性和方法，而且还有一个附加的特定属性Dept_managed。也可以为这个子类型指定附加的特定方法。

SQL中的另一个工具是超表/子表工具，这类似于第4章中讨论的类或外部继承。在此，子表继承超表的所有列；子表的每一行对应超表中的一行且仅一行；超表中的每一行至多对应子表中的一行。INSERT、DELETE和UPDATE操作将被适当地传播。例如，考虑如下定义的表REAL_ESTATE_INFO：

```
CREATE TABLE REAL_ESTATE_INFO (
    Property        REAL_ESTATE,
    Owner           CHAR(25),
    Price           MONEY  );
```

可以定义下列子表：

```
CREATE TABLE AMERICAN_REAL_ESTATE UNDER REAL_ESTATE_INFO;
CREATE TABLE GEORGIA_REAL_ESTATE UNDER AMERICAN_REAL_ESTATE;
CREATE TABLE ATLANTA_REAL_ESTATE UNDER GEORGIA_REAL_ESTATE;
```

在这个示例中，子表AMERICAN_REAL_ESTATE中的每个元组都必须在其超表REAL_ESTATE_INFO中出现，子表GEORGIA_REAL_ESTATE中的每个元组必须存在于其超表AMERICAN_REAL_ESTATE中，依此类推。不过，超表中的元组不一定要出现在子表中。

5．SQL中的非结构化复杂对象

SQL中包括了大对象（LOB）的新数据类型和大对象定位函数，并且包括了面向二进制大对象（BLOB）和字符大对象（CLOB）的两个变种。SQL在DBMS内操纵LOB而不必使用外部文件。某些操作符不能应用于值为LOB的属性，例如算术比较、分组和排序。另一方面，检索部分值的操作可以应用于LOB，如LIKE比较、连接、求子串、定位和求长度。随后我们将会在6.4节见到Oracle 8中如何使用大对象。

我们已经对SQL中提出的面向对象工具进行了简要介绍。现在，SQL/基础和SQL/对象规范都已经得到标准化。显然，随着商业ORDBMS中相关技术的实现，SQL更具面向对象性的工具会紧随其后问世。SQL/MM（多媒体）已经作为多媒体数据库管理的单独标准提出，它由多个部分组成：框

架、全文本、空间、通用工具和静止图像。我们将讨论Informix Universal Server中二维数据类型、图像和文本数据刀片的使用。

6.1.3　SQL 中一些新操作和特性

一个主要的新操作是对于指定递归查询的**线性递归**（liner recursion）。为了说明这一点，假设有一个表PART_TABLE(Part1, Part2)，只要部件p_1包含部件p_2作为其一个组件，表中就包含一个元组<p_1, p_2>。要生产部件p_1的**材料清单**（bill of material），即生产p_1所需的所有组件，可以编写以下递归查询：

```
WITH RECURSIVE
BILL_MATERIAL (Part1, Part2) AS
   ( SELECT Part1, Part2
     FROM PART_TABLE
     WHERE Part1 = 'p1'
           UNION ALL
     SELECT BILL_MATERIAL.Part1, PART_TABLE.Part2
     FROM BILL_MATERIAL, PART_TABLE
     WHERE PART_TABLE.Part1 = BILL_MATERIAL.Part2 )
SELECT * FROM BILL_MATERIAL
ORDER BY Part1, Part2;
```

最终结果被包含在BILL_MATERIAL(Part1,Part2)中。计算UNION ALL操作是通过取内部块生成的所有元组的并集，直到不再生成新元组为止。因为SQL2中没有递归，所以要由程序员自己通过适当的迭代来完成递归操作。

对于SQL3中的安全，引入了**角色**（role）的概念，它类似于任务描述，并受所授予的权限支配。被赋予某个角色的现实的人（用户账号）可能会发生改变，但是不必改变角色授权本身。SQL3还包括了**触发器**（trigger）用作主动规则和规范说明（见第8章）的语法。触发事件包括对表的INSERT、DELETE和UPDATE操作。可以指定触发器是在触发事件之前还是在事件之后触发。SQL3还包括了**触发器粒度**（trigger granularity）的概念，从而可以指定行级触发器（对于每个受影响的行都要考虑此触发器）或语句级触发器（此触发器只对各触发事件考虑一次）[①]。对于分布式（客户-服务器）数据库（见第9章），SQL3包含了**客户机模块**（client module）的概念。客户机模块可以包含外部调用过程、游标和临时表，它们都可以使用SQL3语法来指定。

SQL3还可以利用程序设计语言工具来扩展。用SQL/CLI编写并使用数据类型和集成环境完全匹配的例程称为**SQL例程**。为了使语言具有计算完备性，SQL3语法包含了以下程序设计控制结构：CALL/RETURN、BEGIN/END、FOR/END_FOR、IF/THEN/ELSE/END_IF、CASE/END_CASE、LOOP/END_LOOP、WHILE/END_WHILE、REPEAT/UNTIL/ END_REPEAT和LEAVE。变量用DECLARE来声明，SET用来指定赋值。**外部例程**指用宿主语言（ADA、C、COBOL和PASCAL等）编写的程序，其中可能包含嵌入式SQL，并有可能存在类型失配。外部例程的优点是，现在已经有大量被广泛使用的例程库，这可以大大减少实现应用所投入的精力。另一方面，SQL例程更纯，但没有得到广泛使用。SQL例程可以用作服务器例程（模式层例程或模块）或客户机模块，它们可以是过程或带有返回值的函数。SQL/CLI已经在《数据库系统基础：初级篇》的第9章中进行了描述。

6.2　数据模型的演变和数据库技术的当前发展趋势

在如今的商业领域中，有多个可用的DBMS产品家族。其中很重要的两个是RDBMS和ODBMS，分别支持关系数据模型和对象数据模型。DBMS产品的另外两种主要类型是：层次和网状DBMS，现

① 这些概念将在第8章中进行更详细的讨论。

在称为**传统DBMS**；层次和网状DBMS以层次数据模型和网状数据模型为基础，两者都是在20世纪60年代中期提出的。层次产品家族中有一个主导产品，即IBM的IMS，而网状家族包括大量的DBMS，例如IDS II（Honeywell）、IDMS（Computer Associates）、IMAGE（Hewlett Packard）、VAX-DBMS（Digital）和TOTAL/SUPRA（Cincom），等等。附录D和E对层次数据模型和网状数据模型[①]做了总结。

如同数据库技术的演变，传统DBMS将会逐渐地被新型的DBMS所取代。在过渡时期，必须面对一个主要问题——**互操作性**问题，即分别属于不同DBMS家族的数据库的互操作性，以及与传统文件管理系统之间的互操作性。为处理这类问题，一整套新系统和工具正在出现。最近，XML跃然成为Web上数据交换的新标准（见第11章）。

扩展ODBMS发展的主要动力来自传统DBMS和基本关系数据模型，以及早期RDBMS遇到了新应用的挑战。这主要涉及包含各种不同类型数据的领域。例如，计算机辅助桌面排版系统中的文本、人造卫星成像或天气预报中的图像、工程设计中复杂的非常规数据、生物基因组信息、农业绘图、股票市场交易历史或销售历史中的时间序列数据、地图中的空间与地理数据、空气/水污染数据和交通数据。因此，显然需要设计新的数据库来开发、操纵和维护这些从应用中产生的复杂对象。而且，为处理表示音频和视频数据流（划分成单独的帧）的数字化信息，还需要在DBMS中存储BLOB（二进制大对象）。

商业DBMS牢固的基础结构促进了关系模型的普及，但是，基本关系模型及SQL语言的早期版本不能满足上述挑战的需求。类似于网状数据模型的传统数据模型可以显式地对关系建模，但是实现时需要使用大量的指针，而且没有诸如对象标识、继承、封装等概念，也不支持多种数据类型和复杂对象。层次模型可以很好的表示实际生活中以及组织中自然产生的层次体系，但是数据的内置层次路径上过于局限和固定。所以，出现了一种发展趋势，即把对象数据模型和语言的最佳特性结合到关系数据模型中，从而扩展-关系数据模型，使之能够处理当前复杂的应用。

本章后面将重点介绍两个具有代表性的DBMS的特性，即Informix Universal Server和Oracle 8，由此可以展示ORDBMS方法。最后将简要讨论嵌套关系模型，它起源于一系列的研究提议和原型实现，这为在关系框架中层次地嵌入结构化复杂对象提供了一种理论框架。

6.3　Informix Universal Server[②]

Informix Universal Server是一个ORDBMS，它利用已有的Informix和Illustra这两个产品，并结合了关系数据库和对象数据库技术。Illustra系统起源于POSTGRES DBMS，它是California大学（Berkeley）中的一个研究项目，后来被商业化为Montage DBMS，在被命名为Illustra之前还曾经被命名为Miro。然后Informix收购了Illustra，并集成到RDBMS中，然后命名为Informix Universal Server。这是一个ORDBMS。Informix被IBM收购后，现在的产品叫做IBM Informix Dynamic Server。

为了解ORDBMS出现的原因，我们先重点讨论一种按照两维或两个轴划分DBMS应用的方法：数据复杂性是 X 维；查询复杂性是 Y 维。我们可以将这两个坐标轴排列成一个简单的0-1空间，它具有4个象限。

象限1（$X=0, Y=0$）：简单数据，简单查询。

象限2（$X=0, Y=1$）：简单数据，复杂查询。

象限3（$X=1, Y=0$）：复杂数据，简单查询。

象限4（$X=1, Y=1$）：复杂数据，复杂查询。

传统RDBMS属于第2象限。虽然它们支持复杂的特定查询和更新（以及事务处理），但是只能处

① 可以在本书的网站上获得专门介绍网状数据模型和层次数据模型的章节。

② 本节中的讨论主要基于由Michael Stonebraker和Dorothy Moore 1996年所著的*Object-Relational DBMS*一书，以及Informix公司的Magdi Morsi提供的输入数据。我们的讨论可能引用的是Informix的较早版本可能不是最新版本。

理能够模型化为表中行集的简单数据。许多对象数据库（ODBMS）属于第3象限，因为它们擅长复杂数据的管理，但是基于导航的查询能力却有所局限[①]。

为了转移到第4象限，同时支持复杂数据和复杂查询，RDBMS结合了更为复杂的数据对象，而ODBMS则结合了更复杂的查询（例如，第5章中讨论的OQL高级查询语言）。Informix Universal Server属于第4象限，它对其基本关系模型作了扩展，结合了大量特性，使之成为对象-关系模型。

当前从RDBMS演变来的其他的ORDBMS还包括Oracle公司的Oracle、IBM的Universal DB（UDB），Hewlett Packard（HP）的Odapter（它扩展了Oracle的DBMS）以及HP的Open ODB（它扩展了HP自己的 Allbase/SQL产品）。在引入额外功能的同时，还能够维持RDBMS工作特性的产品看来更为成功。在此不打算对这些产品进行比较分析，而只是简要介绍这两个具有代表性的系统。

Informix Universal Server如何扩展-关系数据模型。关系数据模型的扩展是由Illustra提供的，并被合并到Informix Universal Server中，有如下几类：

- 支持附加的或可扩展的数据类型。
- 支持用户定义例程（过程或函数）。
- 隐含的继承表示法。
- 支持索引扩展。
- 数据刀片应用编程接口[②]（API）。

以下各节将对这些特性分别做简要介绍。前面我们已经大体上介绍了以下概念：数据类型、类型构造器、复杂对象以及在面向对象模型环境中的继承（见第4章）。

6.3.1　可扩展数据类型

Informix Universal Server的体系结构包括基本DBMS以及大量的**数据刀片模块**（Data Blade module）。其思想是把DBMS看成一个刀片架，可以插入特定的刀片来支持特定的数据类型。它已经提供了大量的数据类型，包括二维几何对象（例如点、线、圆和椭圆）、图像、时间序列、文本和Web网页。当Informix发布Universal Server时，可得到的数据刀片就有29个[③]。应用也可以创建自己的类型，这使得数据类型概念是完全可扩展的。除了内置类型外，Informix Universal Server还为用户提供了以下4种构造来声明附加的类型：

(1) Opaque类型。

(2) Distinct类型。

(3) Row类型。

(4) Collection类型。

当基于前3种选项之一创建一个类型时，用户必须提供操纵和转换的函数和例程，其中包括内置、聚集函数和操作符函数，以及附加的任何用户定义的函数和例程。下面将详细介绍这4种类型。

1．opaque类型

opaque类型隐藏了内部表示，所以用于封装类型。用户必须提供强制类型转换函数，从而将opaque对象在服务器（数据库）中的隐藏表示与客户机（调用程序）的可见表示之间进行转换。需要用户函数*send/receive*将服务器内部表示转换为客户机表示，或者将客户机表示转换为服务器内部表示。类似地，要用*import/export*函数把大容量副本的外部表示转换为内部表示，或者把内部表示转换为外部表示。可能还要定义其他一些函数来处理opaque类型，包括*assign*()、*destroy*()和*compare*()。

---

① 象限1包含了涉及数据处理但没有复杂数据检索和操纵特性的所有软件包。这包括诸如Microsoft Excel的电子表格，Microsoft Word的文字处理软件，或者任何文件管理软件。

② 数据刀片提供了对基本系统的扩展，稍后将在6.3.6节中讨论。

③ 要得到Informix Universal Server数据刀片的更多信息，可以访问网站：http://www-306.ibm.com/software/data/informix/blades/。

opaque类型的规范说明包括：类型名、内部长度（如果固定）、最大内部长度（如果长度可变）、对齐方式（字节界限）以及是否是散列的（用于创建散列存取结构）。如果编写如下语句：

```
CREATE OPAQUE TYPE FIXED_OPAQUE_UDT (INTERNALLENGTH = 8,
  ALIGNMENT = 4, CANNOTHASH);

CREATE OPAQUE TYPE VAR_OPAQUE_UDT (INTERNALLENGTH = variable,
  MAXLEN=1024, ALIGNMENT = 8);
```

第一个语句创建了一个定长的用户定义opaque类型，命名为FIXED_OPAQUE_UDT；第二个语句创建了一个变长的opaque类型，命名为VAR_OPAQUE_UDT。这两个opaque类型都是用带有内部参数的实现来描述的，这些内部参数对客户不可见。

2．Distinct类型

distinct数据类型通过继承来扩展已有的类型。新定义的类型将继承它的基类型的函数/例程，除非这些函数/例程被覆盖。例如语句：

```
CREATE DISTINCT TYPE HIRING_DATE AS DATE;
```

创建了一个新的用户定义类型HIRING_DATE，可以像其他任何内置类型一样使用。

3．row类型

row类型表示一个复合属性，它类似于C程序设计语言中的*struct*类型[①]。它是由一个或多个字段组成的一个复合类型。row类型可以通过使用关键字UNDER来支持继承，但是类型系统只支持单重继承。通过创建元组为特定row类型的表，可以创建一个关系，该关系可作为面向对象模式的一部分，并建立关系之间的继承联系。在下面的row类型声明中，EMPLOYEE_T和STUDENT_T就继承了PERSON_T(或在PERSON_T下声明)：

```
CREATE ROW TYPE PERSON_T(Name VARCHAR(60),
  Social_security NUMERIC(9), Birth_date DATE);

CREATE ROW TYPE EMPLOYEE_T(Salary NUMERIC(10,2), Hired_on
  HIRING_DATE) UNDER PERSON_T;

CREATE ROW TYPE STUDENT_T(Gpa NUMERIC(4,2), Address
  VARCHAR(200)) UNDER PERSON_T;
```

4．collection类型

Informix Universal Server的汇集包括内置类型和用户定义类型的列表、集合和多集（包）[②]。Collection类型可以是row类型中的字段，也可以是表中的列。**集合**（set）汇集中的元素不能包含重复的值，也没有特定的顺序。**列表**（list）可以包含重复的元素，并且元素的顺序很重要。**多集**（multiset）可以包括重复的元素，也没有特定的顺序。考虑如下示例：

```
CREATE TABLE EMPLOYEE (Name VARCHAR(50) NOT NULL,
  Commission MULTISET(MONEY) );
```

其中，EMPLOYEE表包含commission列，其类型为多集。

6.3.2　支持用户定义例程

Informix Universal Server支持用户定义函数和例程来操纵用户定义类型。这些函数可以用存储过程语言（Stored Procedure Language，SPL）或C或Java程序设计语言实现。用户定义函数使得用户可以定义操作符函数，例如*plus*()、*minus*()、*times*()、*divide*()、*positive*()、*negate*()；还可以定义内置函数，例如*cos*()和*sin*()；还可以定义聚集函数，例如*sum*()和*avg*()；以及用户定义例程。只要定义了所

① 这类似于第4章讨论的元组构造器。

② 这类似于第4章和第5章讨论的汇集类型。

需函数，Informix Universal Server可以将用户定义类型作为内置类型来处理。下面的示例指定了一个等价函数来比较前面声明的FIXED_OPAQUE_UDT类型的两个对象：

```
CREATE FUNCTION equal (Arg1 FIXED_OPAQUE_UDT, Arg2
   FIXED_OPAQUE_UDT) RETURNING BOOLEAN;
EXTERNAL NAME "/usr/lib/informix/libopaque.so
   (fixed_opaque_udt_equal)" LANGUAGE C;
END FUNCTION;
```

Informix Universal Server还支持**cast**函数，即将对象从源类型转换为目的类型的函数。有两种用户定义的强制类型转换：隐式转换和显式转换。隐式cast函数会自动调用，而显式cast函数只有在使用“::”或CAST AS显式指定了cast操作符时才会被调用。如果源类型和目标类型内部结构相同（例如，当使用*distinct*类型说明时），则无需用户定义函数。

考虑下面的示例来说明显式强制类型转换，这里employee表有Col1列，它的类型为VAR_OPAQUE_UDT，以及Col2列，它的类型为FIXED_OPAQUE_UDT。

```
SELECT Col1 FROM EMPLOYEE WHERE FIXED_OPAQUE_UDT::Col1 = Col2;
```

为了比较Col1和Col2，要对Col1应用强制类型转换操作符，强行令其从VAR_OPAQUE_UDT转换为FIXED_OPAQUE_UDT。

6.3.3　支持继承

在Informix Universal Server中，可以在两个层次上处理继承：数据（属性）继承和函数（操作）继承。

1．数据继承

在已有的row类型下创建子类型，可以使用前面讨论过的UNDER关键字。考虑下面的示例：

```
CREATE ROW TYPE EMPLOYEE_TYPE (
   Ename VARCHAR(25),
   Ssn CHAR(9),
   Salary INT) ;
CREATE ROW TYPE ENGINEER_TYPE (
   Degree VARCHAR(10),
   License VARCHAR(20) )
   UNDER EMPLOYEE_TYPE;
CREATE ROW TYPE ENGR_MGR_TYPE (
   Manager_start_date VARCHAR(10),
   Dept_managed VARCHAR(20) )
   UNDER ENGINEER_TYPE;
```

上述语句创建了一个EMPLOYEE_TYPE类型和一个ENGINEER_TYPE子类型，后者表示工程师雇员，继承了雇员的所有属性，并且具有Degree和License这两个附加特性。另一个ENGR_MGR_TYPE类型是ENGINEER_TYPE的一个子类，继承了ENGINEER_TYPE，并且隐式继承了EMPLOYEE_TYPE。Informix Universal Server不支持多重继承。现在可以基于这些row类型来创建EMPLOYEE、ENGINEER和ENGR_MGR表。

需要注意，在表中存储类型层次结构的存储方式会有所不同。Informix Universal Server提供了以不同组合存储实例的选项。例如，每个层次上的实例（记录），或者合并了所有层次的实例，这些实例对应于《数据库系统基础：初级篇》的7.2节中的映射选项。继承属性可以在较低层次的表中重复表示，也可以用指向超类型对象的引用来表示。基于类型层次结构可以对SQL命令的处理做适当的修改。例如，以下查询：

```
SELECT  *
FROM    EMPLOYEE
WHERE   Salary > 100000;
```

从所有的表中返回特定的雇员信息，表示每一个选中的雇员。因此EMPLOYEE表的范围扩展到了EMPLOYEE下的所有元组。默认的，对超表（supertable）的查询不仅会返回超表中的列，还会返回继承自超表的子表的列。与之相反，下面的查询只返回EMPLOYEE表中的实例，因为在此使用了关键字ONLY：

```
SELECT  *
FROM ONLY (EMPLOYEE)
WHERE   Salary > 100000;
```

可以使用相关变量对超表进行查询，这样，结果不仅包含子表的超表类型的列，还包括子表特定的列。这种查询返回尺寸不同的行，其结果称作**锯齿行结果**(jagged row result)。使用如下语句会以锯齿形式检索所有层次上雇员的所有信息：

```
SELECT  E
FROM    EMPLOYEE E;
```

对于每个雇员，基于他或她是工程师还是某个其他子类型，该查询将从适当的子类型中返回附加的属性集。

在超表上定义的视图不能更新，因为被插入行的位置不明确。

2．函数继承

在表中可以沿着类型层次继承在相同路径中的数据。与此类似，在ORDBMS中函数也可以继承。例如，可以在EMPLOYEE_TYPE上定义一个overpaid函数，用来选择工资高于Bill Brown的雇员，如下所示：

```
CREATE FUNCTION overpaid (EMPLOYEE_TYPE)
RETURNS BOOLEAN AS
RETURN $1.Salary > ( SELECT   Salary
                     FROM     EMPLOYEE
                     WHERE    Ename = 'Bill Brown' );
```

在EMPLOYEE表下的表会自动继承这个函数。然而，对于ENGR_MGR_TYPE，该函数可以重新定义，选择那些工资高于Jack Jones的雇员，如下所示。

```
CREATE FUNCTION overpaid (ENGR_MGR_TYPE)
RETURNS BOOLEAN AS
RETURN $1.Salary > ( SELECT   Salary
                     FROM     EMPLOYEE
                     WHERE    Ename = 'Jack Jones');
```

例如，考虑查询。

```
SELECT      E.Ename
FROM ONLY   (EMPLOYEE) E
WHERE       overpaid (E);
```

它用overpaid的第一个定义来计算。下面查询：

```
SELECT      G.Ename
FROM        ENGINEER G
WHERE       overpaid (G);
```

也使用overpaid的第一个定义（因为对engineer来说overpaid并没有重新定义），而

```
SELECT      GM.Ename
FROM        ENGR_MGR GM
WHERE       overpaid (GM);
```

则使用overpaid的第二个定义，它覆盖了第一个定义。如同在4.6节讨论多态性时所述的那样，这被称为**操作（或函数）重载**。注意overpaid 以及其他函数也可以作为虚属性处理。因此在查询中，overpaid可以使用EMPLOYEE.overpaid或ENGR_MGR.overpaid来引用。

6.3.4　支持索引扩展

Informix Universal Server支持基于用户定义例程的单个表或表层次索引。例如：

```
CREATE INDEX EMPL_CITY ON EMPLOYEE (City (Address));
```

使用city函数值在表EMPLOYEE上创建了一个索引。

为了支持用户定义的索引，Informix Universal Server支持操作符类。操作符类用于支持通用B树及其他辅助存取方法（例如R树）中的用户定义数据类型。

6.3.5　支持外部数据源

Informix Universal Server支持外部数据源（例如文件系统中存储的数据），外部数据可映射到数据库的一个表中，称为**虚拟表接口**（virtual table interface）。通过此接口，用户可以定义作为其他操作代理的操作，此操作是访问和操纵与底层数据源相关联的行时所需要的。这些操作包括open、close、fetch、insert和delete。Informix Universal Server还支持一组函数，使用这些函数可以在用户定义例程内调用SQL语句，节省了通过客户机接口的开销。

6.3.6　支持数据刀片应用编程接口

Informix Universal Server的数据刀片应用编程接口为特定类型的应用，提供了新的数据类型和函数。我们将介绍面向二维操作（在GIS或CAD应用中需要）[①]的可扩展数据类型、与图像存储和管理相关的数据类型、时间序列数据类型、以及文本数据类型的一些特性。ORDBMS处理这些新的非常规应用的强大能力，主要归功于这些特殊的数据类型以及它们提供的裁制功能。

1．二维（空间）数据类型

对于二维应用，相关的数据类型包括：

- **点**（point），由坐标（*X*,*Y*）定义。
- **线段**（line），由其两个端点定义。
- **多边形**（polygon），由作为其顶点的*n*个点的有序列表定义。
- **路径**（path），由点序列（有序列表）定义。
- **圆**（circle），由其圆心和半径定义。

给定上述数据类型，通过用某种程序设计语言，实现适当的距离算术表达式，就可以定义两点之间、点线之间、线圆之间的一个*distance*函数。类似地，可以在线与多边形之间、路径与多边形之间、线和圆之间定义一个布尔相交函数，根据两个几何对象是否相交返回true或false。对于GIS应用，还有其他一些相关的布尔型函数，包括*overlap* (*polygon*, *polygon*)、*contains* (*polygon*,*polygon*)、*contains* (*point*,*polygon*)，等等。需要注意，函数名相同但参数类型不同时，就要应用重载（操作多态性）的概念。

2．图像数据类型

图像可以以多种标准格式存储，如*tiff*、*gif*、*jpeg*、*photoCD*、*Group 4*和*FAX*，因此可以为每种格式定义一个数据类型，并使用适当的库函数从其他媒体中输入图像，或输出图像以供显示。还有一种做法，图像（IMAGE）可以看作是有诸多数据存储选项的单个数据类型。这种做法允许表有图像类型的列，并且可以接受不同格式的图像。以下是可以应用于图像的函数（操作）：

---

① GIS表示地理信息系统（Geographic Information System），CAD表示计算机辅助设计（Computer Aided Design）GIS将在14.3节中详细讨论。

```
rotate (image, angle) returns image.
crop (image, polygon) returns image.
enhance (image) returns image.
```

*crop*函数抽取图像与多边形相交的部分。*enhance*函数通过执行增强对比度来提高图像质量。可以把多幅图像作为参数提供给下列函数：

```
common (image1, image2) returns image.
union (image1, image2) returns image.
similarity (image1, image2) returns number.
```

*similarity*函数考虑的是两个带有属性<*color*, *shape*, *texture*, *edge*>的向量之间的距离，向量描述的是两个图像的内容。基于上述相似性度量，Informix Universal Server中的VIR数据刀片，可用于完成基于图像内容的搜索。

3．时间序列数据类型

Informix Universal Server支持时间序列数据类型，处理时间序列数据要比将其存储在多表中简单得多。例如，考虑纽约证券交易所每个工作日开盘时所存储的超过3 000只股票的收盘价。这个表可以定义如下：

```
CREATE TABLE STOCK_PRICES (
    Company_name      VARCHAR(30),
    Symbol            VARCHAR(5),
    Prices            TIME_SERIES OF FLOAT
);
```

基于价格属性的时间序列数据类型，这3 000个公司在大段时间，如几年内的股票价格只需一个关系就足够了。如果没有这种数据类型，每个公司都需要一个表。例如，描述Coca-Cola公司的表可以声明如下：

```
CREATE TABLE COCA_COLA (
    Recording_date    DATE,
    Price             FLOAT  );
```

在这个表中，每年大约有260个元组，每个交易日一个。时间序列数据类型考虑了日历、起始时间、记录时间间隔（例如，每日、每周、每月）等等。时间序列数据类型适用于以下函数：抽取时间序列的子集（例如，1999年1月期间的收盘价）、在一个较粗的粒度上进行汇总（例如，从每日收盘价中得出平均每周收盘价）以及构造移动平均数等。

要查询股票价格表，给出Coca-Cola股票从1999年6月1日开始为期30天的移动平均数。可以使用moving-avg函数，查询如下：

```
SELECT   moving-avg(Prices, 30, '1999-06-01')
FROM     STOCK_PRICES
WHERE    Symbol = "KO";
```

使用SQL语言对coca_cola表做相同的查询则会复杂得多，它要访问多个元组，而上述对股票价格表的查询只需对表中相应公司的行进行处理。据称，使用时间序列数据类型处理这类查询时，其性能的提升是呈数量级的。

4．文本数据类型

文本数据刀片支持文本对象的存储、查找和检索。它定义了一个称为DOC的单个数据类型，实例采用属于内置数据类型LARGE-TEXT的大对象的形式存储。下面将简要介绍这种数据类型的一些重要特性。

large-text的底层存储与large-object数据类型的底层存储相同。对单个大对象的引用是记录在REFCOUNT系统表中的，该表存储了一些信息，例如引用该对象的行数、该对象的OID、该对象的存储管理器、最后修改时间，以及其归档存储管理器等。在LARGE-TEXT和text之间的数据类型自动转换，使得任何带有文本变元的函数都可以应用于LARGE-TEXT对象。这样就可以将LARGE-TEXT对

象作为串进行连接，并且可以从LARGE-TEXT中抽取子串。

文本数据刀片参数包括格式，其默认格式为ASCII，其他还包括Postscript、Dvipostscript、Nroff、Troff和Text。文本转换数据刀片（Text Conversion DataBlade）用于转换各种格式的文档，它是独立于文本数据刀片的。外部文件参数将doc的内部表示展现为存储指向外部文件的指针，而不是将外部文件复制到一个大对象中。

为了操纵doc对象，可以使用如下函数：

```
import_doc (doc, text) returns doc.
export_doc (doc, text) returns text.
assign (doc) returns doc.
destroy (doc) returns void.
```

对于内置LARGE-OBJECT和LARGE-TEXT数据类型，assign和destroy函数已经存在，但是对于doc类型的对象，用户必须重新定义这两个函数。下面的语句创建了一个表LEGAL_DOCUMENTS，每行都有一列是文档标题，而在其他列中是文档本身：

```
CREATE TABLE LEGAL_DOCUMENTS (
    Title           TEXT,
    Document        DOC );
```

为了在这个文档表中插入一个新行“lease.contract”，可以使用如下语句：

```
INSERT INTO LEGALDOCUMENTS (Title, Document)
VALUES ('lease.contract','format {troff}:/user/local/
    documents/lease');
```

values子句中的第二个值是路径名，指定该文档的文件位置。此格式规范指出这是一个TROFF文档。为了搜索文本，必须创建一个索引，如以下语句所示：

```
CREATE INDEX LEGAL_INDEX
ON          LEGAL_DOCUMENTS
USING       DTREE (Document TEXT_OPS);
```

在上述语句中，TEXT_OPS是一个op-class（操作符类），可应用于称为dtree索引的存取结构中，该索引是针对文档的特殊索引结构。当doc数据类型的文档被插入到一个表时，文本将解析为单独的词。文本数据刀片不区分大小写；因此，Housenumber、HouseNumber或housenumber被认为是相同的词。数据刀片根据WORDNET辞典抽取单词的词根。例如，*houses*或*housing*抽取为*house*，*quickly*抽取为*quick*、*talked*抽取为*talk*。系统保留了一个**stopword**文件，它包含无关紧要的词，例如冠词或介词，它们在查找中要被忽略。stopword包含的词如：*is*、*not*、*a*、*the*、*but*、*for*、*and*、*if*等等。

Informix Universal Server提供了两个例程集合：contains**例程**和text-string**函数**，使得应用程序能够确定哪些文档包含了特定的词，以及哪些文档是相似的。将这些函数应用于查找条件中时，返回的数据按文档与条件匹配度降序排序，最前面的为最佳匹配。函数WeightContains（index to use，tuple_id of the documert，input string）和一个类似的函数WeightContainsWords可返回一个介于0到1之间的精确数字，用来表示输入字符串或者输入单词与元组id所对应特定文档的匹配程度。为了说明这些函数的用法，考虑下面的查询：查找包含了文档“lease contract”中前十项的法律文件的标题。这个查询可以指定如下：

```
SELECT   D.Title
FROM     LEGAL_DOCUMENTS D, LEGAL_DOCUMENTS L
WHERE    CONTAINS (D.Document, AndTerms (TopNTerms( L.Document,10)))
         AND L.Title = 'lease.contract' AND D.Title <> 'lease.contract';
```

这个查询说明了如何通过这些数据类型专用函数增强SQL，从而得到非常强大的处理与文本相关的函数的能力。在这个查询中，变量*D*是指所有法律文件，而*L*表示标题为“lease.contract”的特定文档。TopNTerms从“lease.contract”文档（*L*）中抽取出前十项；AndTerms把这些项合并成一个列

表。Contains把该列表中的项与表LEGAL_DOCUMENTS中其他各个文档（*D*）中的词根相比较。

5. 数据刀片小结

可以看到，通过为抽象数据类型（ADT）提供各种不同的构造器，数据刀片可以增进RDBMS，从而允许用户对数据进行操作，就如同使用ADT将这些数据作为类存储在ODBMS中那样。这使得关系系统表现得类似于ODBMS，与只使用嵌入到程序设计语言中的SQL相比，这种方法大大减少了编程所需的工作。

6.4　Oracle 8 的对象-关系特性

在这一节中我们将介绍与Oracle DBMS产品Release 8.X版本相关的一些特性，此产品通过结合对象-关系特性而有所增进。在Oracle的后续版本中可能还会增加其他的特性。增加了大量与称为**配件**[①]的操纵工具有关的附加数据类型。例如，空间配件允许处理基于地图的信息和地理信息。这些新的数据类型有利于多媒体数据的管理。这里我们重点介绍Oracle 的release 8.X版本(在写本书时已经可以得到)与先前的Oracle版本在新的面向对象特性与数据类型以及一些存储选项方面的区别。在6.1节中已经介绍过的部分SQL-99语言，它适用于Oracle，在此不再讨论这些特性。

6.4.1　Oracle 对象-关系特性的一些示例

作为ORDBMS，Oracle 8依然提供了RDBMS的功能，并且额外地支持面向对象概念。这提供了更高层次的抽象，从而使应用开发人员可以操纵应用对象，而不是从关系数据构造对象。一个对象的复杂信息可以隐藏，但是对象性质（属性、联系）和方法（操作）可以在数据模型中识别。此外，可以通过继承重用对象类型声明，这就减少了应用开发的时间和工作量。为了便于对象建模，Oracle引入了以下的特性（以及6.1节所述SQL-99的一些特性）。

1. 用VARRAY表示多值属性

对象/实体的一些属性可以是多值的。在关系模型中，多值属性必须通过建立新表来处理（见《数据库系统基础：初级篇》的7.1节和10.3.2节关于第一范式的介绍）。如果一个大表的10个属性都是多值的，经规范化后将生成11个表。为了获得原来的数据，开发人员必须对这些表做10个连接操作。对象模型中不会发生这种情况，因为对象的所有属性，包括多值属性，都封装在对象内。Oracle 8使用变长度数组(VARRAY)数据类型实现了这一点，它具有以下性质：

(1) COUNT：当前元素个数

(2) LIMIT：VARRAY所包含的最大元素个数，这是用户定义的

考虑一个CUSTOMER VARRAY实体的示例，它具有属性Name和Phone_numbers，其中Phone_numbers是多值的。首先，需要定义表示Phone_number的对象类型：

```
CREATE TYPE PHONE_NUM_TYPE AS OBJECT (Phone_number CHAR(10));
```

然后定义一个VARRAY，其元素是PHONE_NUM_TYPE类型的对象：

```
CREATE TYPE PHONE_LIST_TYPE AS VARRAY (5) OF PHONE_NUM_TYPE;
```

现在可以创建CUSTOMER_TYPE数据类型作为一个对象，它具有属性Customer_name和Phone_numbers：

```
CREATE TYPE CUSTOMER_TYPE AS
OBJECT ( Customer_name VARCHAR(20),
         Phone_numbers PHONE_LIST_TYPE );
```

现在可以创建customer表：

```
CREATE TABLE CUSTOMER OF CUSTOMER_TYPE;
```

---

① Oracle的配件有些类似于Informix的数据刀片。

为了获取包含客户及其电话号码的列表，可以发出一个没有任何连接的简单查询：

```
SELECT  Customer_name, Phone_numbers
FROM    CUSTOMERS;
```

2．使用嵌套表来表示复杂对象

在对象模型中，对象的某些属性本身也可以是对象。Oracle 8通过**嵌套表**（nested table）来实现这一点（见4.6节）。在此，列（等价于对象属性）可以声明为表。在上面的示例中，假定每个电话号码都有一个附带描述（例如，住宅电话、办公室电话、移动电话）。可以使用一个嵌套表来建模，首先重新定义PHONE_NUM_TYPE如下：

```
CREATE TYPE PHONE_NUM_TYPE AS
OBJECT ( Phone_number CHAR(10), Description CHAR(30) );
```

下一步将PHONE_LIST_TYPE重新定义为PHONE_NUMBER_TYPE表：

```
CREATE TYPE PHONE_LIST_TYPE AS TABLE OF PHONE_NUMBER_TYPE;
```

然后可以像前面一样创建customer_type类型和customer表。唯一不同的是PHONE_LIST_TYPE现在是一个嵌套表而不是一个VARRAY。两个结构具有类似的功能，只有少许不同。嵌套表对于数据项的数量没有上限，而VARRAY有限制。从嵌套表可以检索出单个数据项，但VARRAY不可以。在嵌套表中还可以建立附加索引来提高数据访问速度。

3．对象视图

对象视图可以用于从关系型数据构建虚对象，允许程序员改进现有的模式以支持对象。这使得关系和对象应用可以共存于同一个数据库中。在书中的示例中，假定已经使用关系模型为客户数据库建模，但是管理部门决定将来的所有应用要用对象模型建模。将现有关系数据移至对象视图，这将有助于数据模型的转换。

6.4.2　管理大对象以及其他存储特性

Oracle目前可以存储极大的对象，例如视频、音频和文本文档。为此引入了新的数据类型。包括：

- BLOB（二进制大对象）。
- CLOB（字符大对象）。
- BFILE（存储在数据库之外的二进制文件）。
- NCLOB（固定宽度的多字节CLOB）。

上述除存储在数据库之外的BFILE类型，其他的类型都与其他数据一同存储在数据库内。对于BFILE，数据库中只存储它的目录名。

1．唯一索引表

标准Oracle 7.X要维护诸如B^+树等索引，其中包含指向数据块的指针（见《数据库系统基础：初级篇》的第14章）。B^+树在大部分情况下都能提供很好的性能。但是，要读取数据就必须访问索引和数据块，而且码值要存储两次：表中一次和索引中一次，这就增加了存储的开销。Oracle 8支持标准索引机制，也支持**唯一索引表**，在唯一索引表中，数据记录和索引一起以B树结构保存（见《数据库系统基础：初级篇》的第14章）。对于小型或中型文件，即对于记录不是太大的文件，唯一索引表可以加快数据的检索，而且需要的存储空间较少。

2．分区表和索引

大型的表和索引可以分解为较小的分区（partition）。这时，表就成为一个逻辑结构，分区则成为保存数据的实际物理结构。这具有如下优点：

- 如果某些分区处于部分故障事件时，数据继续可用。
- 可缩放性能允许大幅增长的数据列。
- 在查询和事务处理中提高了总体性能。

6.5　扩展类型系统的实现和相关问题

支持带有相关函数（操作）的扩展类型系统，存在多种实现问题。简单总结如下[①]：

- 只在需要时，ORDBMS才需要动态地将用户定义函数链接到它的地址空间中。正如在以上两个ORDBMS中所见到的那样，许多函数操作需要在二维或三维空间数据、图像、文本等等上进行。如果所有的函数库都静态链接，那么DBMS的地址空间可能会呈数量级增加。在我们所研究的两个ORDBMS中，可以采用动态链接。
- 客户-服务器问题涉及函数的放置和激活问题。如果服务器需要执行某个函数，最好在DBMS地址空间中完成而不是远程执行，因为远程执行会导致大量开销。如果函数需要进行过于耗时的计算，或者如果服务器要与客户机进行多次连接，服务器可能会把这个函数转移到一个单独的客户机。出于安全的原因，最好在使用该用户的ID在客户机上运行函数。将来，很可能使用解释语言，比如Java，来编写函数。
- 在函数内进行查询将是可能的。一个函数必须以相同的方式操作，无论它被用在使用应用编程接口（API）的应用中，还是将函数嵌入到SQL语句中作为执行SQL的一部分由DBMS调用。系统应该支持这些回调（callback）的嵌套。
- 由于ORDBMS中存在各种数据类型和相关的操作符，数据的高效存储和访问就十分重要。对于空间数据或多维数据，可以使用新的存储结构，比如R树、四叉树或者网格文件。ORDBMS必须允许用新的存取结构定义新类型。处理大文本串或二进制文件还导致了许多存储和搜索选项。通过在ORDBMS中定义新数据类型，应当能够采用这些新的选项。

对象-关系系统的其他相关问题

在以上对Informix Universal Server和Oracle 8的讨论中，重点是ORDBMS如何扩展-关系模型。我们讨论了所提供的有关特性和工具，使得对存储为表的关系数据进行操作，就好像是在对象数据库上操作一样。在ORDBMS的语境中还有一些很明显的问题需要考虑：

- **对象-关系数据库设计**。5.5节介绍了设计对象模式的过程。对象-关系数据库设计则更为复杂，因为不仅要考虑应用语义的底层设计因素和关系数据模型中的依赖（见《数据库系统基础：初级篇》的第10章和第11章中的讨论），而且还要考虑前面所讨论的面向对象的扩展特性。
- **查询处理和优化**。通过使用函数和规则对SQL进行扩展，这个问题将会更加复杂，它超出了在《数据库系统基础：初级篇》的第15章中所讨论的关系模型查询优化的范围。
- **规则和事务的交互**。SQL中隐含的规则处理包含的不只是更新-更新规则（见8.1节），这在RDBMS中由触发器实现。而且，RDBMS目前只实现了触发器的立即执行，触发器的延迟执行还涉及额外的处理。

6.6　嵌套关系数据模型

为了使讨论趋于完整，本节中将总结一种假设使用嵌套表的方法，这也被称为非范式关系（nonnormal form relation）。所有商业DBMS都并非以其原始范式来实现这个概念。**嵌套关系模型**（nested relational model）消除了基本关系模型中的第一范式（1NF，见《数据库系统基础：初级篇》的第11章）的限制，因此也被称为**非1NF或非第一范式**（NFNF）或NF^2关系模型。在基本关系模型，也称为**平面关系模型**（flat relational model）中，要求属性是单值属性并且要有原子域。嵌套关系模型允许复合和多值属性，从而产生了有层次结构的复杂元组。这对表示本身就是层次结构的对象很有用。在图6-1a中，展示了基于部分COMPANY数据库的一个嵌套关系模式DEPT，而图6-1b给出了

① 这个讨论大部分取自于（Stonebraker & Moore，1996）。

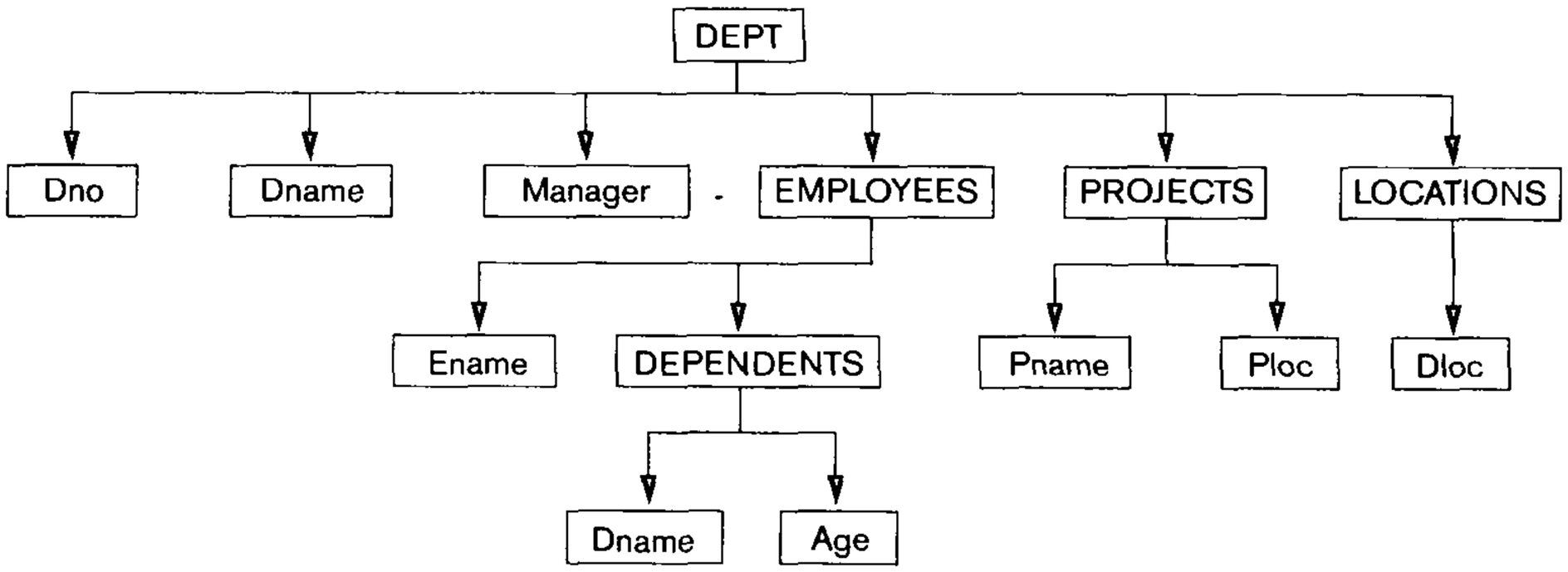
DEPT中的一个非1NF元组的示例。

Dno	Dname	Manager	EMPLOYEES			PROJECTS		LOCATIONS
			Ename	DEPENDENTS		Pname	Ploc	Dloc
				Dname	Age			

(a) DEPT模式

4	Administration	Wallace	Zelaya	Thomas	8	New benefits	Stafford	Stafford
				Jennifer	6	Computerization	Stafford	Greenway
			Wallace	Jack	18	Phone system	Greenway	
				Robert	15			
				Mary	10			
			Jabbar					

(b) DEPT的非1NF元组的示例



(c) DEPT模式的树表示

图6-1　举例说明一个嵌套关系

要将DEPT模式定义为嵌套结构，可以编写如下代码：

DEPT = (Dno, Dname, Manager, EMPLOYEES, PROJECTS, LOCATIONS)
EMPLOYEES = (Ename, DEPENDENTS)
PROJECTS = (Pname, Ploc)
LOCATIONS = (Dloc)
DEPENDENTS = (Dname, Age)

首先，定义DEPT关系的所有属性。接着定义DEPT的嵌套属性本身，即EMPLOYEES、PROJECTS和LOCATIONS。接下来，再定义所有第二层嵌套属性，例如EMPLOYEES的DEPENDENTS，等等。在嵌套关系定义中，所有的属性名必须是不同的。需要注意，嵌套属性通常是**多值复合属性**（multivalued composite attribute），这样在每个元组中就会产生“嵌套关系”。例如，各DEPT元组的PROJECTS属性值都是带有两个属性(Pname,Ploc)的关系。在图6-1b的DEPT元组中，PROJECTS属性包含了3个元组作为其值。其他的嵌套属性可能是**多值简单属性**（multivalued simple attribute），例如DEPT的LOCATIONS。嵌套属性也可能是**单值的**（single-valued）和**复合的**（composite）的，不过大部分嵌套关系模型都将嵌套属性作为多值属性来处理。

当定义嵌套关系数据库模式时，它包含了大量外部关系模式，它们定义了顶层的单个嵌套关系。另外，嵌套属性也被称为**内部关系模式**（internal relation schema），因为它们定义了嵌套在另一个关系内部的关系结构。在上述示例中，DEPT是唯一的外部关系。其他的关系：EMPLOYEES、PROJECTS、LOCATIONS和DEPENDENTS，都是内部关系。最后，**简单属性**出现在叶子层，没有嵌套。可以用树

结构表示各个关系模式，如图6-1c所示，其中根结点是外部关系模式，叶结点是简单属性，内部结点是内部关系模式。注意这种表示法与层次模式（见附录D）以及XML（见第11章）之间的相似性。

重要的一点是，DEPT中第一层的3个嵌套关系表示独立的信息。因此，EMPLOYEES表示在该部门工作的雇员，PROJECTS表示该部门负责的项目，LOCATIONS表示各个部门的位置。在这个模式中并没有表示EMPLOYEES与PROJECTS之间的联系。这是一个M:N的联系，在层次结构中难以表示。

扩展-关系代数和关系估算，以及扩展SQL，都已经提出了嵌套关系。感兴趣的读者可以参考本章最后的参考文献以获取有关的详细内容。在此需要说明两个操作，NEST和UNNEST，它们可以用于扩充标准的关系代数操作，从而实现嵌套关系和平面关系之间的转换。考虑《数据库系统基础：初级篇》中图10-3b的平面关系EMP_PROJ，假定我们将其投影到属性Ssn、Pnumber、Hours、Ename属性上：

$$EMP_PROJ_FLAT \leftarrow \pi_{Ssn,\ Ename,\ Pnumber,\ Hours}(EMP_PROJ)$$

要创建这个关系的嵌套版本，即对应每个雇员都有一个元组，并嵌套有(Pnumber, Hours)，可以使用如下NEST操作：

$$EMP_PROJ_NESTED \leftarrow NEST_{PROJS\ =\ (Pnumber,\ Hours)}(EMP_PROJ_FLAT)$$

这个操作的结果是在外部关系EMP_PROJ_NESTED内创建一个内部嵌套关系PROJS=(Pnumber, Hours)。因此，对于未在NEST操作中指定的属性，NEST会将这些具有相同属性值的元组分成一组，上例中即为属性Ssn和Ename。在书中的示例中每个这样的组表示一个雇员，都创建了一个带有内部嵌套关系PROJS=(Pnumber, Hours)的嵌套元组。因此，EMP_PROJ_NESTED关系就类似于《数据库系统基础：初级篇》的图11-9a和图11-9b中的EMP_PROJ关系。

需要注意，聚集函数的嵌套与分组之间存在着相似性。在嵌套中，元组的每个组成为一个嵌套元组；在分组中，对每个组应用聚集函数后，该组变成一个汇总元组。

UNNEST操作是NEST的逆操作。可以把EMP_PROJ_NESTED转换成EMP_PROJ_FLAT：

$$EMP_PROJ_FLAT \leftarrow UNNEST_{PROJS\ =\ (Pnumber,\ Hours)}(EMP_PROJ_NESTED)$$

这里，PROJS嵌套属性被拉平为其成员Pnumber、Hours。

小结

本章首先给出了在SQL-99中面向对象特性的一个概述，这些特性也适用于对象-关系系统。随后讨论了在数据库管理系统的历史及其当前趋势，由此引出对象-关系DBMS（ODBMS）。我们着重讨论了Informix Universal Server和Oracle 8的一些特性，以此说明如何使用对象特性扩展商业RDBMS。虽然Informix Universal Server和Oracle的现行版本具有额外的特性，但本章的意图是描述关系数据以及基于对象的模式和查询语言展示给用户的基本功能。

其他的商业DBMS也提供了类似的扩展功能。我们了解到，这些系统还提供了数据刀片或配件，它们为新兴的应用领域提供了特殊类型的扩展，例如空间、时间序列或文本/文档数据库。由于ORDBMS的可扩展性，只要用户需要实现它们支持的应用类型，就可以把这些包作为抽象数据类型（ADT）库包含进来。需要时，通过使用这些系统的ADT工具，用户还可以根据需要实现自己的扩展。我们简单讨论了ADT的一些实现问题。最后，简要给出了嵌套关系模型，通过层次结构的复杂对象，嵌套关系模型扩展了平面关系模型。

参考文献

在第4章和第5章中提供的面向对象数据库方法的有关参考文献，这些也与对象-关系系统相关。

（Stonebraker & Moore，1996）提供了对象-关系DBMS的详尽介绍。该书中对Illustra相关概念的讨论也适用于Informix Universal Server。（Kim，1995）讨论了包含面向对象的现代数据库系统一些相关的问题。关于Informix和Oracle的最新信息，可以访问其网站http://www.informix.com和http://www.oracle.com。

SQL3标准在所出版的各个ISO WG3（第3工作组）报告中有所介绍，可以参见（Kulkarni et al，1995）和（Melton et al，1991）两份报告。在超大规模数据库会议（Very Large Data Bases Conference）上，（Melton & Mattos，1996）是一本出色的SQL3教程。（Ullman & Widom，1997）一书通过例子很好地讨论了SQL3。

关于规则和触发器的相关问题，（Widom & Ceri，1995）一书汇集了一些关于主动数据库的文章。一些比较研究，例如（Ketabchi et al.，1990）一书，对关系DBMS和对象DBMS做了比较，书中结论显示了面向对象方法在非传统应用上的优越性。（Schek & Scholl，1985）、（Jaeshke & Schek，1982）、（Chen & Kambayashi，1991）、（Makinouchi，1977）等书讨论了嵌套关系模型。（Paredaens & VanGucht，1992）、（Pistor & Andersen，1986）、（Roth et al.，1988）、（Ozoyoglu et al.，1987）等书介绍了嵌套关系的代数和查询语言。（Dadam et al.，1986）、（Deshpande & VanGucht，1988）以及（Schek & Scholl，1989）等书描述了嵌套关系系统的原型实现。

第三部分

Part 3

高级主题：安全性、高级建模和分布式

本部分内容

- 第 7 章　数据库的安全性
- 第 8 章　面向高级应用的增强数据模型
- 第 9 章　分布式数据库和客户-服务器体系结构

第7章

数据库的安全性

本章将讨论用于保护数据库免受各种威胁的各种技术。同时也论述了向已授权用户授予数据库访问权限的模式。7.1节对数据库安全问题、数据库所受的各种威胁进行了简要的介绍，并概述了本章其他部分涉及到的控制措施；7.2节讨论关系数据库系统以及SQL中用于授予权限和撤销权限的机制，这些机制通常称为**自主访问控制**（discretionary access control）；7.3节概述一种实施多级安全的机制——近年来数据库系统安全领域里的一个热点问题，称作**强制访问控制**（mandatory access control）。本节还论述了近年来发展的**基于角色访问控制**（role-based access control）策略，并简要介绍了XML访问控制；7.4节简要讨论了统计数据库中的安全性问题；7.5节介绍了流控制以及与隐蔽信道相关的问题；7.6节是对加密以及公钥构架方案的简要总结；7.7节讨论了保密性保护技术；7.8节介绍了当前数据库安全性所面临的挑战；7.9节总结了本章的内容。如果读者只对基本的数据库安全机制感兴趣，则只需阅读7.1节和7.2节的内容。

7.1 数据库安全性问题简介

7.1.1 安全性的类型

数据库安全性是一个涵盖许多问题的广阔领域，它主要包括如下几个方面。

- 某些信息的访问关系到法律和伦理的问题。有些信息可能会认为是属于私人信息，未授权人员不能合法地对其进行访问。在美国，有大量的法律条款用于控制信息的私有性。
- 有关政府、机构或公司层次上的政策问题，这些政策确定哪些信息不应该向公众公开。例如：信用等级和个人医疗记录。
- 与系统相关的问题。例如，系统级上应该加强哪几类安全功能，比如：一个安全功能是应该在物理硬件级上、或者是在操作系统级上、还是在DBMS级上进行实现？
- 一些组织需要把安全性问题划分为多个安全级别，并基于这些级别对数据和用户进行划分。例如：绝密、机密、秘密和无分类等。这些组织中还必须具有各自的安全性政策，以确定是否允许访问被强制分类的各种数据。

对数据库的威胁

数据库的安全性威胁会丧失或削弱如下部分或所有人们认可的安全目标：完整性、可用性、机密性。

- **丧失完整性**。数据库完整性是指防止对信息进行不恰当的修改。对数据的修改包括创建、插入、更新、改变数据状态，以及删除数据。如果有意或无意的行为对数据进行了未授权的修改，数据就会丧失完整性。如果系统或数据丧失的完整性没有得到纠正，继续使用受污染的系统或者被破坏的数据，就会导致不正确、欺骗性或者错误的决定。
- **丧失可用性**。数据库可用性是指有合法权限的用户或程序可以使用这些数据库对象。
- **丧失机密性**。数据库机密性是指防止未授权的数据泄漏。未授权的机密信息的泄漏，其影响小则违反数据私有行为，大则危害国家安全。未授权的、未预料到的、或者未意识到的泄漏

都将会导致公众信任的丧失、令人困窘，或者违反组织的法律行为。

为了保证数据库免受这些威胁，有四种控制措施：访问控制、推理控制、流控制和加密。本章将一一进行讨论。

在一个多用户数据库系统中，DBMS必须提供相应的技术以保证特定的用户或用户组，只能访问数据库的指定部分，而不能访问数据库的其他部分。这一点对于同一个组织中许多不同用户共同使用一个大型集成数据库的情况来说尤其重要。例如，雇员工资或业绩报告等敏感信息，就应该对大多数数据库系统的用户保密。典型的DBMS包含一个**数据库安全和授权子系统**（database security and authorization subsystem），由它来负责实现一个数据库的安全性功能以避免发生未授权的访问。目前一般涉及到两种类型的数据库安全性机制。

- **自主安全性机制**：用于向用户授予特权，包括以指定的方式（比如：读、插入、删除或更新）访问指定的数据文件、记录或字段的能力。
- **强制安全性机制**：用于对多级安全性进行控制。先将用户和数据分为多个安全类别（或安全级别），随后执行组织中适当的安全性策略。例如：典型的安全性策略能够使特定分类级别上的用户只能看到他所在级别（或更低级别）上的分类数据项。对这一机制的一个扩展是基于角色的安全性机制，它基于角色的概念来强制执行安全性策略和权限。

我们将在7.2节中进一步讨论自主安全性机制，而强制安全性机制以及基于角色的安全性机制将在7.3节中进行讨论。

7.1.2　控制措施

在数据库中有4种主要的用于提供数据库中数据安全性的控制措施。它们是

- 访问控制。
- 推理控制。
- 流控制。
- 数据加密。

计算机系统中常见的一种安全性问题是防止非授权人员访问系统本身，或获取信息，或在部分数据库中制造恶意的修改。DBMS的安全性机制必须包括限制对数据库系统整体访问的规定。这个功能称为**访问控制**（access control），它的实现是由DBMS通过创建用户账号和口令的方式来控制登录过程。将在7.1.3节讨论访问控制技术。

统计数据库是用于提供基于各种标准的统计信息和值汇总数据。例如：人口统计数据库可能会基于各年龄段、收入水平、住房面积、教育程度和其他标准提供相应的统计数据。统计数据库的用户，比如政府统计部门或市场研究公司等，他们被允许访问统计数据库，以检索与人口有关的统计信息，但同时还需要禁止他们访问有关特定个人的详细机密性信息。统计数据库的安全性必须确保有关个人的信息不能被访问。有时可能仅仅从一些与用户组有关的统计信息查询中，就可以推导出与个人有关的数据。所以，这种查询也必须被禁止。这个被称为**统计数据库安全性**（statistical database security）问题将在7.4节中进行简要的讨论。相应的控制措施被称为**推理控制**（inference control）措施。

另一个安全性问题是**流控制**（flow control），它防止信息向未授权用户流动。这将在7.5节中讨论。信息流中以违反某个组织安全性策略的形式流动的路径信道被称为**隐蔽信道**（covert channels）。将在7.5.1节中简要讨论一些与隐蔽信道有关的问题。

最后一个控制措施是**数据加密**（data encryption），它用于保护通过某类通信网络传输的敏感数据（比如信用卡号）。加密同样也可以用于为数据库的敏感部分提供额外的保护措施。使用一些编码算法可以对数据进行**编码**。未授权用户要存取已经编码的数据时他们将很难破译出真正的数据。但是，对于授权用户则给予了解码和解密的算法（或密钥）来破译数据。如果不知道密钥加密技术就很难解码。加密技术最早是从军事应用程序中发展而来的。7.6节将简要讨论加密技术，包括流行的用于

支持基于Web的数据库事务的公钥加密和用于个人通信的数字签名技术。

对计算机系统和数据库安全的全面讨论超出了本书的范围。在这里我们只给出数据库安全性的简单概述。感兴趣的读者可以参考本章最后的参考文献来进行进一步讨论。

7.1.3　数据库安全性和 DBA

数据库管理员（DBA）是管理数据库系统的核心人物。DBA的职责包括：授权给需要使用系统的用户、按照组织的政策对用户和数据进行分类。DBA在DBMS中有一个**DBA账户**，有时也称作**系统账户**或**超级用户账户**，这个账户提供了一般账户和用户[①]都不能使用的强大的功能。DBA权限命令包括向单个账户、用户或用户组授予特权和收回特权的命令，还包括执行如下类型的动作。

(1) **账户创建**：这类动作为一个用户或一组用户创建一个新的账号和口令，以使得他们能够访问DBMS。

(2) **权限授予**：这类动作允许DBA为某个账户授予某种权限。

(3) **权限收回**：这类动作允许DBA收回（取消）以前授予给某个账户的某种权限。

(4) **安全性级别指定**：这类动作把某个适当的安全性分类级别指派给某个用户账户。

DBA负责保证数据库系统的总体安全。上面列表中的动作1用来控制访问整个DBMS，动作2和动作3用于控制数据库的自主性授权，而动作4用于控制数据库的强制性授权。

7.1.4　访问保护、用户账户和数据库审计

当一个人或一组人需要访问数据库系统时，这个人或这组人必须首先申请一个用户账户。如果对数据库的访问请求是合法的，那么DBA将会为该用户创建一个新的**账号**和**口令**。以后每当需要对数据库进行访问的时候，该用户都必须通过输入账号和口令**登录**（log in）到DBMS。DBMS检查账号和口令是否有效，如果有效，就允许该用户使用DBMS并访问数据库。应用程序也可以被看成是一个用户，并且也被要求提供相应的口令才能对数据库进行访问。

跟踪数据库用户以及他们的账号和口令是很简单的，只需要建立一个含有两个字段（账号和口令）的加密表或加密文件即可。DBMS可以很容易地维护这个表。每当创建一个新的账户，就在这个表中插入一条新记录；每当删除一个账户，就从表中删除相应的一条记录。

数据库系统还必须跟踪在数据库上实施的所有操作，这些操作是用户在每次**登录会话**（log in session）期间对数据库实施的。登录会话包括用户从登录到数据库的时刻开始到注销为止，这一期间内用户所执行的对数据库的全部交互操作序列。当一个用户登录后，DBMS可以记录下该用户的账号并与这个账号有关的用户登录终端相关联。从这个终端发出的所有操作都属于这个用户的账号，直到该用户注销。记录对数据库的更新操作尤其重要，这样一旦数据库被篡改，那么DBA就可以发现这个篡改是由哪个用户发出的。

为了记录应用到数据库的所有更新操作和实施每个更新操作的特定用户，我们可以对系统日志（system log）进行修改。回顾第1章和第3章，**系统日志**包括对数据库实施的每个操作的条目（entry），当事务故障和系统崩溃发生时，就需要利用这些记录对数据库进行恢复。我们可以扩充记录每个操作的日志条目，以便它们也能包括用户账号和日志中记录的操作的在线终端ID。如果怀疑数据库被篡改，就执行**数据库审计**（database audit），审计将扫描日志，以检查某一特定时间段内所有应用于数据库的访问和操作。当发现一个非法的或未授权的操作时，DBA就可以确定执行这个操作的账号。数据库审计对于被多个事务和用户更新的敏感型数据库是非常重要的，例如：被多个银行出纳员更新的银行数据库。主要用于安全性目的的数据库日志有时也称为**审计跟踪**（audit trial）。

---

① 这个账户与计算机系统中给予系统管理员的*root*账户或超级用户账户相似，它们都允许存取一些需要严格限制的操作系统命令。

7.2　基于授予和收回权限的自主访问控制

数据库系统中实施**自主访问控制**的典型方法是基于授予和收回**权限**（privi lege）的机制。考虑到关系型DBMS环境中的权限，本书所要讨论的权限系统与最初开发的SQL语言中的权限系统有某些相似之处（见《数据库系统基础：初级篇》的第8章）。许多当前的关系型DBMS都在使用这种技术的变种。这种技术的主要思想是在查询语言中增加额外的语句，这些语句允许DBA和一些选定的用户向其他用户授予或者收回权限。

7.2.1　自主性权限的类型

在SQL2中，**权限标识符**的概念通常表示的是一个用户的账户（或者是一组用户的账户）。为了简化，我们使用用户或账户来替换权限标识符这个词。DBMS必须能够基于指定的账户，提供对数据库中每个关系的选择性访问。能够控制账户执行的操作，因此，没有必要让一个账户拥有DBMS提供的所有功能。一般情况下，使用数据库系统可以在两个级别上分配权限。

- **账户级**：在这个级别上，DBA可以为每个账户指定其独立持有的在数据库关系上的特定权限。
- **关系（表）级**：在这一级别上，DBA可以控制数据库中每个单独的关系或视图的访问权限。

在**账户级**上的权限可以为该账户自身提供一些能力，包括：用于创建模式或基本关系的CREATE SCHEMA或CREATE TABLE权限；用于创建视图的CREATE VIEW权限；用于支持在关系中增加或删除属性等模式变化的ALTER权限；用于删除关系或视图的DROP权限；用于插入、删除或更新元组的MODIFY权限；用于通过SELECT查询操作从数据库中获取信息的SELECT权限等。需要注意，这些账户权限通常适用于普通账户。如果某个账户没有CREATE TABLE权限，那么这个账户就不能创建任何关系。账户级权限并没有作为SQL2的内容定义在SQL2标准之中，这些定义留给了DBMS的实现者。在SQL标准的早期版本中，有一个CREATETAB的权限可授予账户，用来创建表（关系）。

第二个级别上的权限应用于**关系级**上，它们可以是基本关系或虚（视图）关系。这些权限的定义已经包含在SQL2的标准之中。在下面的讨论中，关系这个术语既可以指基本关系也可以指视图，除非我们明确说明它们是基本关系还是视图。关系级别上的权限用于为每个用户指定在每个单独的关系上适用哪些类型的命令。有些权限还可以作用于关系中单独的列（属性）。SQL2命令只能提供关系和属性级别上的权限，尽管这种权限很具有一般性，但是使用它很难创建一个持有有限权限的账户。权限的授予和收回一般都要遵从一个自主型权限的授权模型，这种模型称为**访问矩阵模型**（access matrix model）。这个矩阵M的行表示主体（用户、账号和程序），而列表示对象（关系、记录、列、视图和操作）。矩阵中的每个位置$M(i, j)$表示主体i持有对象j的权限类型（读、写和更新）。

为了控制关系权限的授予和收回，数据库中的每个关系R都被分配了一个**属主账户**（owner account），它一般是第一次创建该关系的那个用户使用的账户。关系的属主拥有这个关系上的所有权限。SQL2标准中，DBA可以使用CREATE SCHEMA命令（见《数据库系统基础：初级篇》的8.1.1节）创建一个模式并把一个适当的权限标识符与这个模式关联起来，为整个模式指定一个属主。这个属主账户的持有者可以把所拥有的任何关系上的权限，通过**授予**权限命令传递给其他用户账户。在SQL中，每个单独的关系R上可以授予如下类型的权限：

- **关系R上的选择（检索或读取）权限**：授予账户检索的权限。SQL标准中，它授予账户使用SELECT语句从关系R中检索元组的权限。
- **关系R上的修改权限**：它授予账户更改R中元组的权限。SQL标准中，这个权限被进一步划分为UPDATE、DELETE和INSERT权限，以便与R上的SQL命令相对应。另外，INSERT和UPDATE权限都具有指定账户只能更新R上的某些属性的能力。
- **关系R上的参照权限**：它授予账户在指定完整性约束时参照关系R的能力。这个权限也可以限定在R的某些特定属性上。

需要注意，如果某个账户要创建一个视图，那么它必须拥有定义这个视图所涉及的所有关系上的SELECT权限。

7.2.2　使用视图指定权限

视图机制也是一种重要的自主性授权机制。例如：如果关系R的拥有者A想让另一个账户B只能检索R上的某些字段，那么A就可以为R创建一个视图V，视图中只包括那些允许B检索的属性，随后把视图V上的SELECT权限授予B。用同样方式也可以限定B只能检索R上的某些元组：可以创建视图V'，通过查询的方式只从R中选出那些A允许B访问的元组，这样定义的视图V'就可以限制B访问的元组了。7.2.5节中还将用示例来进一步说明上述讨论。

7.2.3　收回权限

某些情况下，可能需要暂时为一个用户授予某些权限。例如，一个关系的拥有者可能只想给负责某个特定任务的用户授予一个SELECT特权，一旦这个任务完成就收回这一权限。因此，这时就需要有一个权限的**收回**机制。SQL标准中的REVOKE命令就是为了达到收回权限这一目的的。我们将在7.2.5节的示例中观察到如何使用REVOKE命令。

7.2.4　使用 GRANT OPTION 选项传播权限

每当关系R的拥有者A把R上的一个权限授予另一个账户B的时候，可以给B授予带有GRANT OPTION选项和不带GRANT OPTION选项的特权。如果带有GRANT OPTION选项，表示B也可以把R上的权限授予其他账户。假定A授予B的权限带有GRANT OPTION选项，那么B就可以把R上的权限也同样带有GRANT OPTION选项授予第三个账户C。使用这种方法，R上的权限就有可能在R的所有者不知道的情况下**传播**（propagate）给其他账户。如果R的拥有者账户A现在收回了授予B的权限，那么，系统应当自动收回B基于这些特权传播出去的所有权限。

一个用户有可能从两个或多个来源获得某个权限。例如：A4可能会从A2和A3那里获得UPDATE R的权限。在这种情况下，如果A2从A4那里收回了那个权限，那么A4将仍能拥有从A3那里获得的这个权限。但是，如果A3随后也从A4那里收回了那个权限，那么A4就会完全丧失这个权限。因此，允许权限传播的DBMS还必须记录所有的权限是如何被授予的，以便能够正确而彻底地收回所有相关的权限。

7.2.5　一个示例

假定DBA创建了4个账户：A1、A2、A3和A4。其中只想让A1能够创建基本关系，那么，DBA必须用SQL发出如下的GRANT命令：

```
GRANT CREATETAB TO A1;
```

CREATETAB（创建表）权限授予了账户A1具有创建新的数据库表（基本关系）的能力，因此，这是一个账户权限。这个权限包含在SQL早期版本中，但是现在它的定义已经留给单独的DBMS去实现了。

在SQL2标准中，DBA可以通过发出一个如下所示的CREATE SCHEMA命令达到同样的效果：

```
CREATE SCHEMA EXAMPLE AUTHORIZATION A1;
```

现在，用户账户A1就可以在EXAMPLE模式下创建表了。为了继续前面的例子，假定A1创建了两个基本关系EMPLOYEE和DEPARTMENT，如图7-1所示。那么A1就是这两个关系的**属主**，因此A1也就拥有了对关系EMPLOYEE和关系DEPARTMENT的所有权限。

EMPLOYEE

Name	Ssn	Bdate	Address	Sex	Salary	Dno

DEPARTMENT

Dnumber	Dname	Mgr_ssn

图7-1　两个关系EMPLOYEE和DEPARTMENT的模式

接着，假定账户A1想授予账户A2在这两个关系上执行插入和删除元组操作的权限。但是，A1不想让A2把它的权限传播给其他账户，那么，A1就可以发出如下的命令：

```
GRANT INSERT, DELETE ON EMPLOYEE, DEPARTMENT TO A2;
```

需要注意，关系的属主账户A1自动地拥有使用GRANT OPTION选项的能力，它可以把自己在关系上的权限授予其他账户。但是，账户A2并不能把关系EMPLOYEE和DEPARTMENT上的INSERT和DELETE权限授予其他账户，因为上面的命令中并没有为A2给出GRANT OPTION选项。

接下来，假定A1想允许账户A3从两个表中的任何一个检索信息，并且也让A3有能力把SELECT权限传播给其他账户。那么，A1可以发出如下的命令：

```
GRANT SELECT ON EMPLOYEE, DEPARTMENT TO A3 WITH GRANT OPTION;
```

WITH GRANT OPTION子句表示A3现在可以通过使用GRANT命令把权限传播给其他账户。例如，A3可以把EMPLOYEE关系上的SELECT权限授予给A4，命令如下：

```
GRANT SELECT ON EMPLOYEE TO A4;
```

注意，A4不能再把SELECT权限传播给其他账户了，因为A4并没有得到GRANT OPTION选项。

现在，假定A1决定从A3那里收回EMPLOYEE关系上的SELECT权限，那么A1可以发出如下的命令：

```
REVOKE SELECT ON EMPLOYEE FROM A3;
```

因为A3向A4授予了EMLOYEE上的SELECT权限，但是A3已经不再拥有这个权限了，那么DBMS还必须自动地从A4那里收回这一权限。

再接着，假定A1想再次赋予A3一个EMPLOYEE关系上有限的SELECT能力，并且还想允许A3传播这个权限。但是，只能限制它对属性Name、Bdate和Address进行检索，并且只能检索那些Dno=5的元组。那么A1可以创建如下的视图：

```
CREATE VIEW A3EMPLOYEE AS
SELECT Name, Bdate, Address
FROM EMPLOYEE
WHERE Dno = 5;
```

视图创建之后，A1可以在视图A3EMPLOYEE上给A3授予SELECT权限，即，

```
GRANT SELECT ON A3EMPLOYEE TO A3 WITH GRANT OPTION;
```

最后，假定A1想允许A4只能更新EMPLOYEE关系的Salary属性，那么A1可以发出如下的命令：

```
GRANT UPDATE ON EMPLOYEE (Salary) TO A4;
```

UPDATE和INSERT权限可以指定在一个关系中更新或插入特定的属性。其他的权限（SELECT、DELETE）不能指定到特定的属性，因为这种指定可以很容易地通过创建只包含所需属性的视图并在视图上授予相应的权限来控制。然而，因为更新视图并不是总能实现的（见《数据库系统基础：初级篇》的第9章），所以就要给UPDATE和INSERT权限加入选项来指定基本关系中可能需要更新的特定属性。

7.2.6　指定权限传播的限定

尽管大多数DBMS并没有实现限定权限传播的技术，而且限定权限传播也不是SQL标准的一部

分，但是这种技术已经得到了很大的发展。可以用一个正整数*i*来限定**水平传播**（horizontal propagation），它表示一个设定GRANT OPTION选项的账户 *B*最多只能把权限授予*i*个其他的账户。**垂直传播**（vertical propagation）更复杂一些，它限定了授权的深度。0深度垂直传播授权相当于以不含有GRANT OPTION选项的方式进行授权。如果账户*A*对账户*B*以垂直传播方式进行授权，并且设置一个整数*j*>0，那么就表示账户*B*在授权中拥有了GRANT OPTION选项，但是*B*只能以小于*j*的垂直传播方式把自己的权限授予其他账户。这样，垂直传播方式限定了基于单一的初始授权命令从一个账户到另一个账户的GRANT OPTION序列。

我们简要说明一下水平传播和垂直传播的限定，它们当前在SQL或其他关系系统中仍然是不可用的。假定A1以水平传播等于1和垂直传播等于2的方式把EMPLOYEE关系上的SELECT权限授予了A2。那么，A2最多只能把SELECT权限授予一个账户，因为它的水平传播限定被设置为1。另外，A2只能以垂直传播等于0（没有GRANT OPTION选项）或等于1的方式为其他账户授权，这是因为当A2把权限传播给其他账户的时候必须把垂直传播深度至少减1。由示例可以说明，水平传播和垂直传播技术的设计可以限定权限的传播范围。

7.3　多级安全性的强制访问控制和基于角色访问控制[①]

在关系上授予和收回权限的自主访问控制技术，已经发展成为关系数据库系统的主要安全机制。这种机制是一种“all-or-nothing”方法，即一个用户要么拥有该特权，要么没有该特权。但是在很多应用中，还需要另外一种安全性策略，这种策略需要在安全性级别（security class）的基础上对数据或用户进行分类。这种方法也称为**强制访问控制**，通常它会与7.2节描述的自主访问控制机制混合使用。需要注意：当前大多数商业DBMS都只提供自主访问控制机制。但是，在政府、军事和智能型应用中，都存在对多级安全性管理的需求，而且在很多工业和企业的应用中也同样存在这种需求。

典型的**安全性级别**有：绝密（top secret，TS）、机密（secret，S）、秘密（confidential，C）和无分类（unclassified，U）。其中TS是最高级别而U是最低级别。另外，还存在其他更复杂的安全性分类模式，在这种安全模式中，安全级性别以格（lattice）的形式来组织。为简单起见，本书使用前面提到的四级安全性分类的系统。在这个系统中，级别从高到低的次序为：TS≥S≥C≥U。通常多级安全性所使用的模型被称为Bell-LaPadula模型，这个模型把每个**主体**（用户、账户和程序）和**客体**（关系、元组、列、视图和操作）指派到一个安全性级别TS、S、C或U中。我们把每个主体S的**许可证**（clearance）或**级别**（classification）记作**class**（S），每个客体O的安全性**级别**记做**class**（O）。这样就可以在数据上实施基于主体/客体安全性级别的两个限定条件了。

(1) 除非class（*S*）≥ class（*O*），主体*S*不能读取客体*O*。这个约束称作**简单安全性性质**。

(2) 除非class（*S*）≤ class（*O*），主体*S*不能写客体*O*。这也被称为***性质**（star property）。

第一个限定条件很直观，它执行的是显而易见的规则：安全性级别低的主体不能读取安全性级别比它高的客体。而第二个限定条件就不那么直观了：它禁止主体写安全性级别比它的安全性级别低的客体。如果违反了这个规则，那么就是允许信息从较高的安全性级别流向较低的安全性级别，这就违反了多级安全性的基本原则。例如，一个拥有TS许可证级别的用户（主体）可能会制作一个拥有TS安全性分类级别的客体拷贝，然后把它作为一个新的客体以安全性级别U写回，这就使得这个客体在整个系统中都变成可见的了。

为了把多级安全性的思想融入到关系数据库的模型中，很容易就会想到要把属性值和元组也作为数据对象。因此，每个属性*A*都与模式中的一个**分类属性***C*相关联，并且元组中的每个属性值也都与相应的安全性级别相关联。另外，在一些模式中，**元组分类**属性*TC*也被加入了关系属性中，以便

① 感谢Fariborz Farahmand对此节以及后面章节所做的贡献。

把每个元组作为一个整体进行分类。因此，一个拥有n个属性的**多级关系**模式R就可以被表示为：

$$R\ (A_1,\ C_1,\ A_2,\ C_2,\ \cdots,\ A_n,\ C_n,\ TC)$$

其中，每个C_i表示与每个属性A_i相关联的分类属性。

每个元组t的TC属性的值是t中所有分类属性值中的最高值，它提供了对元组本身的总的分类，而每个C_i则对元组中的每个属性值提供一个更精确的安全性分类。多级关系中的**显码**（apparent key）是一个形成常规（单级）关系中主码的属性的集合。这样一个多级关系看起来就好像拥有不同安全性级别的主体（用户）所包含的数据也是不同的。在某些情况下，有可能在关系中存储一个分类级别较高的元组，并通过一个称为**过滤**（filtering）的处理，产生相应的分类级别较低的元组。在另外一些情况下，有必要在不同的分类级别上存储两个或多个有相同显码值的元组。这就引出了**多重实例**（polyinstantiation）[①] 的概念，即几个元组可以拥有相同的显码值，但是处于不同分类级别上的用户却拥有不同的属性值。

我们用图7-2a所示的简单示例来说明一下多级关系的概念，图中每个属性值旁边的数字是该属性的分类属性值。假定属性Name是显码，同时考虑查询SELECT * FROM EMPLOYEE。因为所有的元组分类级别都低于或等于S，所以带有许可证级别S的用户能够看到的关系与图7-2a所示的关系相同。但是，带有许可证级别C的用户就不允许看到“Brown”的Salary值和“Smith”的Job-performance值，因为它们拥有更高的分类级别。这些元组将被过滤掉，如图7-2b所示，其中Salary和Job-performance的位置显示为*null*。对于一个拥有许可证级别U的用户，过滤处理只能允许显示“Smith”的Name属性，而所有其他的属性都显示为*null*，如图7-2c所示。因此，过滤处理会把分类级别高于用户的许可证级别的属性值都设置为*null*值。

EMPLOYEE

Name	Salary	JobPerformance	TC
Smith U	40000 C	Fair S	S
Brown C	80000 S	Good C	S

（a）初始EMPLOYEE元组

EMPLOYEE

Name	Salary	JobPerformance	TC
Smith U	40000 C	NULL C	C
Brown C	NULL C	Good C	C

（b）为许可证级别为C的用户过滤后得到的EMPLOYEE关系

EMPLOYEE

Name	Salary	JobPerformance	TC
Smith U	NULL U	NULL U	U

（c）为许可证级别为U的用户过滤后得到的EMPLOYEE关系

EMPLOYEE

Name	Salary	JobPerformance	TC
Smith U	40000 C	Fair S	S
Smith U	40000 C	Excellent C	C
Brown C	80000 S	Good C	S

（d）Smith元组的多重实例

图7-2　用于说明多级安全性的多级关系

① 这个概念与数据库中代表现实世界中同一个对象的多个版本的概念很相似。

一般情况下，多级关系的**实体完整性**规则规定所有作为显码成员的属性都不能为空值，而且在每个单独的元组中都必须具有相同的安全性级别。另外，一个元组中所有其他的属性值都必须具有高于或等于显码的安全性级别。这个约束保证了如果允许用户看到元组的一部分，那么这个用户就能看到这个元组的码。其他的完整性规则称为**空值完整性**（null integrity）和**实例间完整性**（interinstance integrity），它们保证了如果可以从一个较高安全级别的元组中过滤出（取得）处于某个安全性级别的元组值，那么在多级关系中只需存储较高安全性级别的元组就足够了。

为了进一步说明多重实例的概念，假定一个带有许可证级别C的用户试图把图7-2所示的“Smith”的Job-performance属性的值更新为“Excellent”，那么应该发出如下相应的SQL更新命令：

```
UPDATE  EMPLOYEE
SET     Job_performance = 'Excellent'
WHERE   Name = 'Smith';
```

由于持有许可证级别C的用户得到的视图（如图7-2b所示）允许这样的一个更新，那么系统就不应该拒绝它；否则，用户就可以推断出“Smith”的Job-performance属性存在其他的非空值，而不是所看到的null值。这就是一个可以通过**隐蔽信道**推断出信息的例子，这种情况在高安全性的系统（见7.5.1节）中是不允许出现的。但是，不应该允许用户覆盖“Smith”元组已经存在并拥有更高分类级别的Job-performance值。解决这个问题的办法是：为“Smith”元组创建一个带有更低分类级别C的**多重实例**，如图7-2d所示。当不能从已经存在并且分类级别是S的元组过滤得到新的元组时，就有必要这样做。

关系模型中的基本更新操作（INSERT、DELETE和UPDATE）必须进行相应的修改，以处理这种情况或与此相似的情况，但是这个问题所涉及的内容已经超出了本书所讨论的范围。有兴趣的读者可以阅读本章末尾提供的参考文献，以便获得与此有关的详细信息。

7.3.1　自主访问控制与强制访问控制的比较

自主访问控制（DAC）策略的特征是有高度的灵活性，这使得它们适用于多个应用领域。DAC模型的主要缺点是防范恶意攻击的脆弱性，例如，嵌入在应用程序中的特洛伊木马。原因是一旦被授权用户访问之后，自主授权模型就不能对如何传播和如何使用信息进行任何的控制了。相反，强制访问控制策略可以保证更高程度的保护，在这种方法中，它们防止了信息的非法流动。因此，它们适用于需要高度保护的军事应用。然而，强制访问控制策略也有如下的缺点：它过于严格，要求将主体和客体严格地划分到安全级别中，因此仅适用于少数环境。在很多实际情况下，首选的是自主访问控制策略，因为它在安全性和应用性之间进行了较好的折中。

7.3.2　基于角色的访问控制

作为管理和增强大规模企业级系统中安全的一种成熟技术，基于角色的访问控制（RBAC）在20世纪90年代迅速兴起。其基本思想是把许可与角色关联起来，把适当的角色指派给用户。角色可以使用CREATE ROLE和DESTROY ROLE命令创建。可以使用讨论DAC时介绍的GRANT和REVOKE命令为角色授予和收回权限。

RBAC对于传统的自主访问控制和强制访问控制来说，都是一个可行的选择，它确保只有授权用户才能访问特定的数据或者资源。在用户创建的会话期间，可以激活该用户所属角色的一个子集。每个会话可以指派多个角色，但是这些角色只能映射到一个用户或者一个单--主体。许多DBMS都有角色的概念，并可以为角色指派权限。

RBAC系统中的角色层次是一种组织角色，它反映了该组织中的授权和职责的自然方式。通常，随着角色层次的提高，较底层次的角色会与较高层次的角色连接起来。层次图是偏序的，所以它们是自反的、传递的和非对称的。

RBAC系统中另一个需要考虑的重要方面是角色上可能存在时态约束，如角色激活的时间和持续时间、一个角色的激活引起的另外一个角色的触发。对于Web应用的密钥安全需求，使用RBAC模型是一个非常令人满意的目标。可以指派给角色工作流任务，这样就可以为拥有与任务相关的任何角色的用户授权来执行这一任务，并且只在特定时间段内扮演特定的角色。

RBAC模型有一些令人满意的特性，例如：灵活性、策略中立性、安全管理的良好支持性以及其他一些特性，这些特性使得RBAC成为开发安全Web应用最具吸引力的选择。相对而言，DAC和MAC模型缺乏支持新兴企业和Web应用的安全需求能力。此外，RBAC模型则既能表示传统的DAC和MAC策略，又可以表示用户自定义或者组织特定的策略。因此，RBAC是一个超集模型，可以依次模拟DAC和MAC系统的行为。而且，RBAC模型提供了一种可以解决与任务和工作流执行相关的安全性问题的自然机制。更易于在Internet上部署，这是RBAC模型成功的另一个原因。

7.3.3　XML 访问控制

随着XML在世界范围内的商业和科学应用领域中的广泛应用，其安全性标准的制订工作也取得了一定的成果。在这些成果中，包括了XML的数字签名和加密标准。XML签名语法和处理的规范说明，描述了为了表示密码签名和XML文档或其他电子资源之间的关联的XML语法。规范说明也包括计算和验证XML签名的过程。一个XML数字签名与其他消息签名协议（例如PGP—Pretty Good Privacy，一个用在电子邮件和文件存储应用程序中的机密性和认证服务）的不同之处在于它支持的是对XML树（见第10章）中特定部分而不是整个文档进行签名。另外，XML签名规范说明定义了联署和转换机制，即所谓的权威性（canonicalization），它保证了同一文本的两个实例可生成相同的用于签名的摘要，即使它们的描述有少许的差别。例如，印刷中的空白不同。

XML加密语法和处理的规范说明为保护整个或部分XML文档或非XML数据的机密性定义了XML词汇和处理规则。加密内容以及接收者的附加处理信息都以良好格式的XML文档来描述，以便可以用XML工具进一步处理其结果。与其他常用的加密技术，例如SSL（Secure Sockets Layer，一种最重要的因特网安全协议），以及虚拟私有网络不同，XML加密还可以应用于部分文档和持久存储的文档中。

7.3.4　电子商务和 Web 的访问控制策略

电子商务（**E-commerce**）环境的特征是任何事务都是以电子方式执行的。它们需要详尽的访问控制策略，这已经超出了传统DBMS的能力范围。在常规的数据库环境中，访问控制通常是使用一组由安全管理员或者用户根据安全策略指定的授权来实现的。这样一个简单的模式并不能很好地适应诸如电子商务这样的动态环境。而且，在电子商务环境中，要保护的资源不仅是传统的数据，还有知识和经验。这些特性要求能够更加灵活地指定访问控制策略。访问控制机制必须足够灵活，以支持大范围的异构保护对象。

第二个相关的需求是支持基于内容的访问控制。基于内容的访问控制允许用户来表示涉及保护对象内容的访问控制策略。为了支持基于内容的访问控制，访问控制策略必须允许包括基于对象内容的条件。

第三个需求与主体的多样性有关，它要求基于用户的特征和资格，而不是基于特定的或单个的特征（例如用户ID）的访问控制策略。为了在访问控制策略的表述中更好地考虑账户用户基本情况，一个可能的解决方法是支持凭证表示法。**凭证**（credential）是与安全性目的相关的一组用户特性的集合（例如年龄或者在组织中的岗位）。举例来说，使用凭证，可以简单地建立以下策略：“只有拥有5年或5年以上资历的正式员工才可以访问系统内部文档。”

可以相信，XML语言在电子商务应用的访问控制中将会扮演重要角色[①]。其原因是XML正在成为Web上文档交换的公共描述语言，并且也正在成为电子商务语言。因此，一方面有必要使得XML可以通过提供适用于XML文档保护的访问控制机制来描述安全性。另一方面，访问控制信息（即访问控制策略和用户凭证）就可以用XML本身来表示。目录服务标记语言（Directory Service Markup Language）为此提供了基础：目录服务通信标准负责提供并验证用户凭证。保护对象和访问控制策略的统一表示也可以应用于策略和凭证本身。举例来说，一些凭证特性（例如用户名）有可能被每个人访问，但是其他特性则只对一些有严格限制的用户可见。此外，对于指定凭证和访问控制策略，使用基于XML的语言更便于安全凭证的提交和访问控制策略的输出。

7.4　统计数据库的安全性简介

统计数据库主要用于产生与各类群体有关的统计数据。数据库可能包含个体的秘密数据，这类数据应该在用户的访问过程中得到保护。但是，应该允许用户检索关于群体的统计信息，如平均值、总和、计数值、最大值、最小值以及标准差等。保护统计数据库中私人信息的技术已经超出了本书所要讨论的范围。我们这里只用一个非常简单的示例来说明这个问题，该示例中所用到的关系如图7-3所示。这个PERSON关系具有以下属性：Name、Ssn、Income、Address、City、State、Zip、Sex和Last_degreee。

PERSON

Name	Ssn	Income	Address	City	State	Zip	Sex	Last_degree

图7-3　用于说明统计数据库安全性的PERSON关系模式

群体（population）是一个关系（表）中满足若干选择条件的元组集合。因此，每个PERSON关系上的选择条件都能确定PERSON关系上一个特殊的群体。例如：条件 Sex = 'M'就可以指定所有的男性群体；条件（（ Sex = 'F' ）AND（Last_degree = 'M.S.' OR Last_degree = 'Ph.D.'））就可以指定最高学位为M.S.或Ph.D.学位的女性群体；而条件City = 'Houston'则可以指定所有居住在Houston的群体。

统计查询是指对群体所包含的元组进行统计。例如，检索群体中所含的人数，或者某个群体的平均收入水平。但是，不允许统计数据库的用户检索到关于个体的信息，如某个人的收入等。**统计数据库安全性**技术必须能够禁止对个体数据的检索，其方法可以是禁止检索属性值的查询，只允许使用统计性聚集函数（如COUNT、SUM、MIN、MAX、AVERAGE和标准DEVIATION等）进行查询这样的查询有时也称为**统计查询**（statistical query）。

当为用户提供有用的关于个体数据的统计求和功能时，确保个体信息的秘密性是数据库管理系统的职责。在统计数据库中提供用户**隐私的保护**是极为重要的，如下的示例说明了违反隐私保护的情况。

某些情况下，可能从统计查询的结果中就可以**推断**（infer）出单个元组的值。尤其是一个群体中只包含很少数目的元组时，这种情况就极有可能发生。下面的两个统计查询可以说明这种情况：

```
Q1:   SELECT COUNT (*) FROM PERSON
      WHERE <condition>;

Q2:   SELECT AVG (Income) FROM PERSON
      WHERE <condition>;
```

现在，假设我们对找出Jane Smith的Salary比较感兴趣，我们已经知道她拥有Ph.D.学位，并且也

① 见(Thuraisingham et al. 2001)。

知道她住在Texas州的城市Bellaire。那么我们就可以发出如下所示的统计查询Q1的查询条件：

(Last_degree='Ph.D.' AND Sex='F' AND City='Bellaire' AND
State='Texas')

如果得到的查询结果中只有一条记录，那么只要在统计查询Q2中限定同样的查询条件就可以得到Jane Smith的Salary情况。即使基于上述查询条件的查询Q1得到的查询结果不是1条，但是是一个很小的数字，如2条或3条，那么仍可以使用MAX、MIN和AVERAGE函数得到Jane Smith的Salary的一个可能的范围。

可以规定当某个选择条件所指定的群体中元组的数目低于某一阈值的时候，就不允许执行这个统计查询。这样的规定可以降低从统计查询中推断出个体信息的可能性。还有一种可以禁止检索个体信息的技术是：禁止对相同群体的元组重复地执行查询序列。还可以有意地在统计查询的结果中引入一些误差或“噪音”，这样就可以使从统计查询的结果中推断出个体信息变得很困难。另一种技术是数据库划分。划分意味着记录是以最小规模的组的形式存储。查询可以查到任何一个完整的组或组的集合，但是不能查到组中的记录子集。有兴趣的读者可以阅读本章末尾提供的参考文献，以便获得与此技术有关的讨论。

7.5　流控制简介

流控制可以调节可访问对象间的信息分布或流向。当程序从X读取值，并将其写入Y时，对象X与对象Y之间的数据流就产生了。**流控制**检测包含在某些对象中的信息不会显式地或隐式地流向缺少保护的对象。这样，用户就不可能从Y间接得到其不能从X直接得到的信息。主动流控制研究开始于20世纪70年代早期。大多数流控制都使用了安全级别的概念。只有当接收者的安全级别至少等于发送者的安全级别时，才能把信息从发送者传递给接收者。以下是两个流控制的示例：防止服务程序泄露客户的秘密数据，以及防止机密军事数据传输给未知安全级别的用户。

流策略（flow policy）指定了允许信息移动所通过的信道。最简单的流策略只指定了信息的两个等级：秘密（*C*）和非秘密（*N*），除了从等级*C*到等级*N*的数据流之外，所有数据流都是允许的。这个策略可以解决当一个服务程序处理一些例如顾客信息的数据时所引起的限制问题，而在这些顾客信息中有一些信息可能是秘密的。例如，所得税计算服务程序可能允许获得客户的地址和所接受服务的账单，但是不能获得客户的收入或者所得税减免信息。

访问控制机制负责检查用户访问资源的授权情况：只能执行已授权的操作。流控制可以通过扩展的访问控制机制来实现，其中包括为每个运行的程序指派安全等级（通常称为许可证）。仅当程序的安全等级与特定内存段的安全等级一样高时，才允许程序读取该内存段；仅当程序的安全等级与特定内存段的安全等级一样低时，才允许程序写该内存段。这样就自动保证了一个人发出的信息不会从高安全级传输到低安全级。例如，有机密许可证的军事程序只能从无分类和秘密的对象中读取信息，另外只能向机密或者绝密的对象写信息。

流可分为两种类型：在赋值指令中产生的是显式流（explicit flow），例如$Y:=f(X_1, X_m)$；由条件指令产生的是隐式流（implicit flow），例如if $f(X_{m+1}, \cdots, X_n)$ then Y: $=f(X_1, X_m)$。

流控制机制必须验证只能执行已授权的信息流动（包括显式的和隐式的）。为了保证信息流的安全，必须满足一组规则。这些规则使用不同安全级别之间的流关系来表示，并被赋值给信息，以表示系统内已被授权的流。（当与*A*相关的信息影响了与*B*相关的信息的值时，就产生了从*A*到*B*的信息流。信息流产生于操作，这些操作使信息从一个对象传输到另一个对象）。对于一个安全级别，这些关系可以定义（在那个安全级别中的分类）信息能够流动的安全级别集合，或者可以指出允许信息从一个级别流向另一个级别时需要检验两个级别之间的哪些特定关系。通常，流控制机制实现控制是通过为每个对象指派一个标签，或者指定一个安全级别。标签则用于检验模型中定义的流关系。

隐蔽信道

隐蔽信道允许传输违反安全或策略的信息。特别是，**隐蔽信道**允许信息通过不适当的方法从较高安全级别层次流向较低的安全级别层次。隐蔽信道可以分为两大类：计时信道和存储信道。二者之间的区别是：在**计时信道**中，信息根据事件或者进程的时间来传送，而**存储信道**不需要任何时间上的同步，信息是通过访问系统信息或者用户不可访问的信息来传送的。

在隐蔽信道的一个简单示例中，考虑一个分布式数据库系统，在这个分布式数据库系统中两个结点的用户安全层次分别为机密（S）和未分类（U）。为了提交一个事务，两个结点必须都同意提交。它们之间只能进行满足“*性质”的操作，这说明：在任一事务中，S站点不能写或者传递信息给U站点。然而，如果两个站点共谋在它们之间建立一条隐蔽信道，那么包含机密数据的事务就可以由U站点无条件地提交，而S站点以某种预定义的协定方法来实现它，从而使得特定的信息可以从S站点传送到U站点，这违反了“*性质”。在事务重复运行的地方，可能会出现这样的情况，从而使S站点采取的动作隐式地把信息传送给了U站点。有一些措施，如加锁方法，我们在第1章和第2章讨论过，它可以防止不同安全级别的用户同时向相同的对象写信息，进而可防止存储类型的隐蔽信道。操作系统和分布式数据库提供了对操作的多程序控制，它允许共享资源，但是一个程序或者进程不能侵占系统中其他程序或进程的内存或其他资源，这样就防止了面向计时的隐蔽信道。在一般情况下，隐蔽信道在良好执行的鲁棒数据库实现中不是主要问题。然而，一些聪明的用户可能会发明一些特定的方案来隐式地传输信息。

一些安全专家认为，避免隐蔽信道的一个方法是：不允许程序员真正访问程序投入运行之后要处理的敏感数据。例如，银行的程序员不需要访问储户的名字或余额。经纪人公司的程序员不需要知道客户的买卖顺序。在程序测试过程中，可能有必要对真实数据或者某些样本测试数据进行访问，但是在程序验收并正式投入使用之后，程序员就不能再访问真实数据了。

7.6　加密和公钥构架

前面讨论的访问控制和流控制方法，尽管是功能强大的安全控制措施，但是并不能保护数据库免受某些威胁。假设我们正在进行数据传输，但是数据却落到了某个非法用户的手中。在这种情况下，可以使用加密技术伪装消息，这样即使数据被传递到非法用户手中，消息也无法显示出来。加密技术是在不安全的环境中维持数据安全的一种方法。加密技术首先使用某个预先指定的**加密密钥**（encryption key）对数据应用**加密算法**（encryption algorithm），然后结果数据必须要使用**解密密钥**（decryption key）进行**解密**（decrypt），以恢复原来的数据。

7.6.1　数据加密标准和高级加密标准

数据加密标准（Data Encryption Standand，DES）是一个美国政府为公众开发的系统。它被美国和其他国家广泛接受为密码标准。DES能够在发送者A和接收者B之间的信道上提供端到端的加密。DES算法是替换和置换这两个基本加密模块精细和复杂的组合。DES算法重复应用这两个模块共计16次来达到其强度。明文（消息的原始形式）被加密成64位的块，虽然密钥是64位长的，但实际上密钥可以是任意的56位数字。在发现对DES是否能满足加密需要存在疑问之后，美国国家标准委员会（National Institute of Standards，NIST）提出了高级加密标准（Advanced Encryption Standards，AES）。相对于DES的56位块，这个算法使用了128位的块；相对于DES的56位密钥，AES可以使用128、192或者256位的密钥。与DES相比，AES引入了更多的可能密钥，因此需要更长时间来破解。

7.6.2　公钥加密

1976年，Diffie和Hellman提出了一种新的密码系统，称为**公钥加密**（public key encryption）。公

钥算法是基于数学函数而不是基于位模式的操作。相对于只使用一个密钥的传统加密算法，它使用了两个独立的密钥。在机密性、密钥分发和鉴别方面，使用两个密钥有重要意义。公钥加密中使用的两个密钥分别是**公钥**（public key）和**私钥**（private key）。私钥总是保密的，它更倾向于称为私钥而不是秘钥（传统加密中使用的密钥），这样可以避免与传统加密相混淆。

公钥加密模式，或者公钥加密基础设施有6个组成部分：

(1) **明文**：明文（plaintext）是指作为算法的输入数据或者可读消息。

(2) **加密算法**：在明文上执行各种转换的算法。

(3)和(4) **公钥和私钥**：这是一对密钥，如果一个用于加密，另一个就用于解密。加密算法实现的精确转换依赖于作为输入提供的公钥和私钥。

(5) **密文**：密文（ciphertext）是指作为算法的输出所产生的零乱消息，它依赖于明文和密钥。对于一条给定的消息，两个不同的密钥会产生不同的密文。

(6) **解密算法**：解密算法接收密文以及相匹配的密钥，以便产生原来的明文。

公钥是供他人公开使用的密钥，而私钥只有其拥有者才知道。通用的公钥加密算法依赖于加密密钥以及与之相关但不同的解密密钥。主要步骤如下所示：

(1) 每个用户产生一对密钥，用于对消息加密和解密。

(2) 每个用户把其中一个密钥放到公共寄存器或者其他可访问的文件中。这个密钥就是公钥，另一个密钥就是私有的。

(3) 如果发送者打算发送私有消息给接收者，那么发送者可使用接收者的公钥加密消息。

(4) 接收者接收到该消息时，他使用自己的私钥进行解密。其他的接收者无法解密该消息，因为只有接收者知道其私钥。

RSA公钥加密算法

第一个公钥算法是1978年由MIT的Ron Rivest、Adi Shamir和Len Adleman提出的，并根据他们的名字命名为RSA模式。从此以后，RSA模式作为公钥加密方法得到最广泛的接受和实现，并在此领域中占据主导地位。RSA加密算法使用了数论方面的研究成果，并考虑了确定目标的素数因子的难度。RSA算法还使用了取模运算符进行运算，即mod n。

两个密钥d和e分别用来解密和加密。一个重要的特性是它们是可以互换的。选择n作为一个大整数并生成两个不同的大素数a和b。加密密钥e是在1~n之间随机选择的一个相对于$(a-1)\times(b-1)$的数。明文块P被加密为$P^e \bmod n$。由于对幂执行了mod n运算，因此想要解开因子P^e的加密明文是很难的。为此，选择解密密钥d要很小心，以使得$(P^e)^d \bmod n = P$。解密密钥d可以通过条件$d\times e = 1 \bmod ((a-1)\times(b-1))$来计算。这样，知道密钥$d$的合法接收者只需简单计算$(P^e)^d \bmod n = P$，而不用分解$P^e$即可恢复$P$。

7.6.3　数字签名

数字签名是一个使用加密技术在电子商务应用中提供验证服务的例子。如同手写的签名那样，**数字签名**（digital signature）是一种使用文本块把一个标记与某个个体（人）关联起来的方法。标记应当是不可忘记的，这意味着其他人应该能够验证签名是否来自最初的签名者。

数字签名是由一个符号串组成。如果一个人的数字签名对每个消息都是相同的，那么其他人就可以通过简单复制这个符号串来假冒这个签名。因此，签名对每个应用来说必须是不同的。这可以通过使每个数字签名成为其所需签名消息的函数来实现，并且带有一个时间戳。为了使数字签名对签名者和防伪证据（counterfeitproof）是唯一的，每个数字签名还必须依赖于对于签名者唯一的某个秘密数。因此，在一般情况下，防伪证据的数字签名必须依赖于该消息和该签名者的一个唯一秘密数。然而，签名的验证者并不需要知道任何的秘密数。公钥技术是创建此类特性的数字签名的最好办法。

7.7　保密性问题和保护

保护数据的保密性对数据库安全和保密性专家来说是一个不断增强的挑战。从某些方面来看，为了保护数据保密性我们甚至应该限制进行大规模的数据挖掘和分析。解决这个问题的最普遍的技术是避免建造过大的中心数据仓库作为单一的关键信息资料库。另一种可行的方法是有意修改或扰乱数据。

如果在单个数据仓库中所有数据都是可用的，仅仅违反单个资料库的安全性就会暴露所有数据。避免中心数据仓库和使用分布式数据挖掘算法，可以在开发全局有效模型中需要的数据交换最小化。通过修改、扰乱以及匿名化数据，我们还可以减轻与数据挖掘有关的保密性威胁。这可以通过从发布的数据中移去等同信息，以及向数据中插入干扰项来实现。但是，通过使用这些技术，我们还应该关注数据库中结果数据的质量，这些结果数据可能经历过太多的修改。我们必须能够估计到这些修改可能引入的错误。

保密性是数据库管理方面正在进行的研究中的一个重要的领域。由于保密性的多学科性质，并且对保密性、信任等的解释中的主观性问题，使得它非常复杂。举例来说，考虑医学和法律方面的记录和事务：当它们被定义和执行时，需要维护特定的保密性需求。同样，对于移动设备提供访问控制和保密性，也越来越多地被人们所关注。DBMS需要强健的技术来支持小型设备上与安全性相关的信息的有效存储，受信任的协商技术也一样。在哪里保存信息，这关系到用户的身份、基本信息、信用、和许可，以及如何使用它作为可信用户的认证，仍然是一个重要的问题。由于大规模数据流在这样的环境中生成，所以必须设计访问控制的有效技术，并与支持连续查询的处理技术相结合。最后，必须保证从探测器和通信网络中获得的用户位置数据的保密性。

7.8　数据库安全性所面临的挑战

考虑数据库和信息资产受到的威胁在数量和速度上的快速增长，研究工作需要致力于如下问题：数据质量、知识产权和数据库生存能力。

7.8.1　数据质量

数据库团体需要技术和有组织的解决方案来评估和验证数据的质量。这些技术可以包括一些简单的机制，例如在Web站点上张贴的质量戳。我们还需要提供基于某些技术的，例如记录连接技术，更有效的完整语义校验技术和数据质量评定工具。同样，也需要面向应用层的恢复技术来自动地修复不正确的数据。目前，数据仓库（见13.4节）中用来加载数据的抽取、转换、装载（extract, transform, load，ETL）的工具正是针对这些问题而提出的。

7.8.2　知识产权

随着Internet和Intranet的广泛使用，数据的合法性和信息化正在逐步成为组织中主要考虑的问题。对于这些问题的关注，最近提出了关系数据的水印技术。数字水印的主要目的是通过授予内容可证明的所有权来防止非授权的复制和分发。在传统意义上，它依赖于一个大噪声域的可用性，在这个噪声域中，对象能够在保持其本质特性的同时被改变。但是，仍然需要在评估这些技术的健壮性方面和为防止侵犯知识产权而以不同方法调查方面进行研究。

7.8.3　数据库生存能力

尽管存在着分裂性事件，例如信息战争攻击，但数据库系统仍然需要运转并维持其功能，即使只有简化的功能。除了许多预防攻击工作，以及检测一个突发事件工作之外，DBMS还应具有以下功能：

- **禁闭**。采取即时动作消除攻击者对系统的访问，并隔离或包容这一问题以防止进一步扩散。
- **损害评估**。确定问题的范围，包括失效的功能和损坏的数据。
- **重新配置**。当恢复运行时，重新配置以允许操作在降低级别的模式下继续进行。
- **修复**。恢复被破坏的或被丢失的数据，并修复或重新安装失效的系统功能，以重建一个正常级别的操作。
- **默认处理**。在可能范围中，识别在攻击中被利用的系统漏洞，并采取措施预防再次发生。

信息战争攻击者的目的是通过破坏信息系统来损害组织的运转，以履行攻击者自身的使命。攻击的特定目标可能是系统本身或是系统的数据。把系统彻底击垮的攻击是严重的并且也是戏剧性的，必须对它们计时以获得攻击者的目标，因为攻击将被即时接收并被集中关注，以便将系统恢复到可运行状态，检查攻击是如何发生的并且设置预防措施。

到目前为止，与数据库生存能力相关的问题还没有被充分地调查研究。对于保障数据库系统生存能力的技术和方法学，需要投入更多的研究。

小结

本章讨论了几种增强数据库系统安全性的技术。我们提出了与丢失数据库的完整性、可用性和秘密性有关的不同威胁。随后我们讨论了处理这些威胁的各种类型的控制方法：访问控制、推理控制、流控制和加密。

增强安全性时，把数据库系统作为一个整体进行访问控制，并控制数据库特定部分的访问权限。前者通常通过为用户分配一个带有口令的账户来实现。后者可以通过为每个账户授予或收回访问数据库特定部分的权限的系统来实现。这种方法一般指的是自主访问控制技术。本章列举了一些授予和收回权限的SQL命令，并用示例说明了它们的用法。随后，本章简单介绍了可以实施多级安全控制的强制访问控制机制。这种机制需要把用户和数据值分为几个安全级别，并且禁止信息从高安全性级别流向低安全性级别。本章还给出了多级关系模型所基于的几个主要概念，如：过滤和多重实例。同时，我们还介绍了基于角色的访问控制，它指定基于用户所扮演的角色的权限。本章还简要地讨论了对统计数据库的访问控制问题：提供统计数据库群体记录访问的同时，还要保护个体信息的机密性。随后，又讨论了涉及到流控制的问题以及与隐蔽信道相关的问题。我们还介绍了数据加密，包括公钥构架和数字签名。我们强调了一些保密性问题的重要性，并介绍了一些保密性保护技术。最后我们讨论了安全性面临的一些挑战，包括数据质量、知识产权和数据库生存能力等方面。

复习题

7.1　讨论下列术语的含义：数据库认证、访问控制、数据加密、授权（系统）账户、数据库审计和审计跟踪。

（a）讨论账户级上的权限类型，以及在关系级上的权限类型。

7.2　哪个账户被指定为关系的属主（拥有者）？关系的属主都有哪些特权？

7.3　如何把视图机制作为一种授权机制来使用？

7.4　授予权限的含义是什么？

7.5　收回权限的含义是什么？

7.6　讨论权限的传播系统，对水平传播和垂直传播限定所加的限制是什么？

7.7　列出SQL中可用的权限类型。

7.8　自主访问控制和强制访问控制之间有哪些区别？

7.9　典型的安全级别有哪些？讨论简单安全性质和*性质，并解释执行这些多级安全规则的原因。

7.10　描述多级关系数据模型。定义下列术语：显码、多重实例和过滤。

7.11　使用DAC或MAC的相对优点是什么？

7.12　什么是基于角色的访问控制？在什么情况下它优于DAC和MAC？

7.13　什么是统计数据库？讨论统计数据库的安全性问题。

7.14　保密性与统计数据库安全性有什么关系？可以采取什么措施确保统计数据库的某种程度上的保密性？

7.15　什么是作为一种安全性措施的流控制？现有的流控制有哪些类型？

7.16　什么是隐蔽信道？给出一个隐蔽信道的示例。

7.17　加密的目标是什么？加密数据然后在另一端恢复该数据包含了哪些处理？

7.18　给出一个加密算法的示例，并解释它是如何工作的。

7.19　针对流行的RSA算法，重新回答上面的问题。

7.20　什么是公钥构架机制？它如何提供数据库安全性的？

7.21　什么是数字签名？它是如何工作的？

练习题

7.22　如何保护数据库中数据的保密性？

7.23　数据库安全性当前面临的主要挑战是什么？

7.24　考虑《数据库系统基础：初级篇》中图5-5所示的关系数据库模式。假定所有的关系都由用户*X*创建（因此他也拥有这些关系），他要给用户账户*A*、*B*、*C*、*D*和*E*授予如下权限：

(a) 账户*A*可以检索或修改除DEPENDENT之外的任何关系，并且可以给任何其他用户授予这些权限。

(b) 账户*B*可以检索EMPLOYEE和DEPARTMENT中除Salary、Mgr_ssn和agr_start_date之外的所有属性。

(c) 账户*C*可以检索或修改WORKS_ON，但是只能检索EMPLOYEE关系的Fname、Minit、Lname和Ssn属性以及PROJECT的Pname和Pnumber属性。

(d) 账户*D*可以检索EMPLOYEE或DEPENDENT的任意属性，而且可以修改DEPENDENT的任意属性。

(e) 账户*E*只能检索EMPLOYEE关系中DNO=3的元组的任意属性。

(f) 编写授予以上权限的SQL语句，并在需要的时候使用视图。

7.25　假设复习题7.1中的权限a带有GRANT OPTION选项，账户*A*最多只能把它的权限授予5个其他的账户，但是这些账户中的每个都可以不带GRANT OPTION选项把权限传给其他账户。那么在这种情况下，水平传播和垂直传播的限定应该是什么样的？

7.26　考虑图7-2d所示的关系。对一个安全级别为U的用户，将是什么样的结果？假定一个安全级别为U的用户试图将“Smith”的工资更新为$50 000，那么这个动作的结果将是什么？

参考文献

建立在授予和收回权限基础上的授权机制是为实验性DBMS，SYSTEM R而提出的，（Griffiths & Wade，1976）对它进行了描述。许多书籍中都讨论了数据库和计算机系统中的安全性，其中包括（Leiss，1982a）和（Fernandez et al.，1981）等等。

很多论文都讨论了统计数据库的设计和保护方面的技术问题，如（McLeish，1989）、（Chin & Ozsoyoglu，1981）、（Leiss，1982）、（Wong，1984）和（Denning，1980）等。（Ghosh，1984）还讨论了用于质量控制的统计数据库的应用。也有很多论文对加密方法和数据加密的问题进行了讨论，如（Diffie & Hellman，1979）、（Rivest et al.，1978）、（Akl，1983）、（Pfleeger，1997）、（Omura et al.，1990）、（Stalling，2000）和（Iyer et al.，2004）等。

（Jajodia & Sandhu，1991）、（Denning et al.，1987）、（Smith & Winslett et al.，1992）、（Stachour &

Thuraisingham，1990）以及（Lunt et al.，1990）、（Bertino et al.，2001）都对多级安全问题进行了讨论。（Lunt & Fernandez，1990）以及（Jajodia & Sandhu，1991）、（Bertino et al.，1998）、（Castano et al.，1995）以及（Thuraisingham et al.，2001）对数据库安全研究所涉及的问题进行了全面的介绍。（Atluri et al.，1997）讨论了多级安全性对并发控制的影响。（Rabbiti et al.，1991）、（Jajodia & Kogan，1990）和（Smith，1990）讨论了语义和面向对象数据库的下一代安全性问题。（Oh，1999）描述了一个适用于自主安全和强制安全的模型。（Joshi et al.，2001）讨论了Web应用的安全模型和基于角色访问控制的安全模型。（Farahmand et al.，2005）讨论了电子商务应用环境中管理的安全问题，以及选择适当的安全控制措施需要的风险评估模型。

（Bertino & Sandhu，2005）讨论了数据库的安全性和保密性方面的最新发展以及面临的挑战。（Naedele，2003）讨论了XML和访问控制。（Vaidya & Clifton，2005）讨论了保密性保护技术的更多细节，（Sion et al.，2004）讨论了知识产权，（Jajodia et al.，1999）讨论了数据库生存能力。

第8章

面向高级应用的增强数据模型

随着人们对数据库系统应用需求的增长，用户要求DBMS软件包提供更多的功能，以帮助他们实现更高级、更复杂的应用。面向对象数据库（object-oriented database）与对象-关系数据库（object-relational database）允许用户为自己的应用指定附加的抽象数据类型，以扩展他们的系统。定义抽象数据类型这一功能是非常有用的，它可以帮助用户识别某些高级应用程序的公共特性，并创建表示这些特性的模型。此外，为了提高这些公共特性的性能，还实现了特殊的存储结构和索引方法。这些特性可以作为抽象数据类型或类库来实现，并可以与基本的DBMS软件包分开购买。例如，Informix中的**data blade**和Oracle中的**cartridge**（见第6章）就是可包含在DBMS软件包中的可选子模块。如果这些新特性适合于用户的应用，用户就可以直接选用这一模块，而无须重新设计、实现和编写这些公共特性。

本章将介绍数据库的某些公共特性的概念，这些特性是数据库高级应用所需要的，并已开始广泛使用。我们将介绍主动数据库应用中采用的*主动规则*、时态数据库应用中采用的时态概念，还将简要介绍一下*多媒体数据库*中的一些问题。我们还将讨论*演绎数据库*。必须说明的是，所有这些主题都涉及了很广泛的领域，而本章只对这些主题进行简要介绍。实际上，这些主题中的任何一个都可以单独作为一本完整的书。

8.1节将介绍主动数据库，主动数据库提供了用以指定**主动规则**的一些附加功能。这些规则可以因发生某些事件而被自动触发，例如，更新数据库时或到达某一时间点时。这些规则还可以在规则声明中指定当某些条件满足时所要采取的动作。很多商业数据库都已通过**触发器**（trigger）的形式提供了类似的功能。触发器现在是SQL-99标准的一部分。

8.2节将介绍**时态数据库**（temporal database）的概念。时态数据库允许数据库系统存储其记录变化的历史，同时允许用户查询数据库当前和过去的状态。一些时态数据库模型还允许用户存储未来期望的信息，例如已定制好的日程表。需要注意，很多数据库应用虽然已经具备了时态数据库的功能，但它们对于“时态概念”的支持却不是从DBMS软件包中获得的，也就是说，它们所谓的“时态概念”其实是由访问数据库的应用程序来实现的。

8.3节将对空间数据库和多媒体数据库进行简要的介绍。**空间数据库**（spatial database）提供了可以在多维空间中跟踪对象的数据库模型。例如，地图数据库存储的是二维空间中对象的一些坐标信息，包括国家、州、河流、城市、公路、海洋等。其他数据库，例如面向天气信息的气象数据库是三维的，温度和其他与气象相关的参数一起构成了三维空间中的点。**多媒体数据库**则提供了允许用户存储和查询不同类型多媒体信息的特性，这些多媒体信息包括：**图像**（如图片或绘图）、**视频片断**（如电影、新闻磁带或家庭录像）、**音频片断**（如歌曲、电话留言或演说录音）以及**文档**（如书籍或文章）。

8.4节将讨论演绎数据库[①]，演绎数据库涉及数据库、逻辑推理、人工智能或知识库等技术。**演绎数据库系统**（deductive database system）包含了定义（**演绎**）**规则**的能力，可以从存储在数据库中

① 8.4节是本书第三版中第25章的总结，完整的一章见本书的网站。

的事实演绎或者推断出其他信息。由于数理逻辑是一些演绎数据库系统理论基础的一部分，所以这些规则也常常被**逻辑数据库**（logic database）所引用。其他类型的系统，包括**专家数据库系统**（expert database system）或**基于知识的系统**（knowledge-based system），其中也结合了推理或者推断的能力。这些系统使用了先进的人工智能领域中的技术，包括语义网络、框架、生产系统或获取领域知识的规则等。

本章各节中的内容彼此都是相对独立的，读者可以选择自己感兴趣的主题阅读。

8.1　主动数据库概念和触发器

指定通过某些特定事件而能够被自动地触发的动作的规则，在很长时间内被认为是数据库技术的一个重大进展。事实上，**触发器**（一种指定主动规则的特定类型的技术）的概念在关系数据库语言SQL的早期版本定义中就已经存在，如今触发器是SQL-99标准的组成部分。商业关系DBMS，如Oracle、DB2和SYBASE也都提供了各种版本的触发器。然而，对主动数据库一般模型应该如何实现的许多研究，从提出初级的触发器模型的概念之后才开始。8.1.1节将阐述用于为主动数据库定义规则的一般概念。考虑到Oracle商业关系数据库系统中触发器的概念与SQL标准中指定的规则非常接近，我们将在示例中使用Oracle语法来说明这些概念。8.1.2节将讨论主动数据库的一般设计和实现的细节问题。由于STARBURST数据库系统在其框架中已经提供了一般性主动数据库的许多概念，因此，本章在8.1.3节中将给出如何在实验性STARBURST DBMS中实现主动数据库的示例。8.1.4节将讨论主动数据库的可能应用。最后，8.1.5节将描述在SQL-99标准中如何声明触发器。

8.1.1　主动数据库和 Oracle 触发器的一般模型

指定主动数据库规则的模型也称为**事件**（Event）-**条件**（Condition）-**动作**（Action）模型，即**ECA**模型。ECA模型的规则包括3个成分：

(1) 触发规则的**事件**：这些事件通常指显式地应用于数据库的更新操作。但在一般模型中，这些事件也可能指时态事件[①] 或其他外部事件。

(2) 决定是否执行规则中动作的**条件**：一旦定义的触发事件发生，可能要计算一个可以选择的条件。如果规则中没有指定条件，那么只要事件发生，就执行规则中的动作。如果指定了条件，那么先计算条件，若计算结果为*true*，就执行规则中的动作。

(3) 采取的**动作**：动作通常是一个SQL语句序列，也可能是一个将被自动执行的数据库事务或外部程序。

为了说明上述概念，我们来看一些示例，这些示例基于《数据库系统基础：初级篇》的图5-7中COMPANY数据库应用的一个简化版。如图8-1所示，雇员关系EMPLOYEE中的属性有：雇员姓名（Name）、社会保险号（Ssn）、工资（Salary）、当前所属部门（Dno，也是对DEPARTMENT关系的外码）和该雇员的直接管理者（Supervisor_ssn，也是EMPLOYEE关系本身的（递归）外码）。在这个示例中，假定Dno可以为NULL，表示一名雇员可能暂时还没有指定所属部门。部门关系DEPARTMENT中的属性有：部门名称（Dname）、部门编号（Dno）、本部门全部雇员的工资总额（Total_sal）和部门经理（Manager_ssn，也是对EMPLOYEE关系的外码）。

请注意：Total_sal属性是一个导出属性，它是按部门分别计算各部门所有雇员工资的总和。维护这样一个导出属性值的正确性，可以通过使用主动规则来实现。首先，必须确定可能引起Total_sal值改变的**事件**，它们是：

(1) 插入（一个或多个）新雇员元组。

(2) 改变（一个或多个）现有雇员的工资。

---

① 例如，可以把一个在每天早晨5:30周期性触发规则的事件作为一个时态事件。

(3) 改变现有雇员所属部门，从一个部门调到另一个部门。

(4) 删除（一个或多个）雇员元组。

EMPLOYEE

Name	Ssn	Salary	Dno	Supervisor_ssn

DEPARTMENT

Dname	Dno	Total_sal	Manager_ssn

图8-1　用于说明主动规则的简化COMPANY数据库

对于事件1，如果一个新雇员被分配到一个部门，对于要被插入的新雇员元组中，其Dno值不为NULL（前面已经假定Dno可以为NULL），那么我们只需要重新计算Total_sal的值。因此，这就成为要被检查的**条件**。相似的条件也将在事件2（和事件4）中被检查，以确定在当前指定的部门中，是否有雇员的工资发生改变(或是否有雇员要被删除)。对于事件3，总是要执行一个动作以维护Total_sal值的正确性，因此不需要条件（即动作总是被执行）。

事件1、2、4对应的**动作**是自动更新雇员所在的部门关系中的Total_sal值，以反映最新插入、更新或删除雇员工资的操作。对于事件3，需要执行一个双重的动作：一个是更新雇员原来所属部门的Total_sal值，另一个是更新雇员新的所属部门的Total_sal值。

对于上述情况，可以在Oracle DBMS中定义4个主动规则（或触发器）：R1、R2、R3和R4，如图8-2a所示。首先考虑规则R1，用它来说明Oracle中定义主动规则的语法。CREATE TRIGGER语句为规则R1指定了一个触发器（或主动规则），称为Total_sal1。AFTER子句指定要在事件发生之后规则才被触发。触发事件（在本例中是指插入新雇员元组）在AFTER关键字后指定[①]。

ON子句指定被该规则作用的关系，在本例中规则R1作用的关系是EMPLOYEE。可选关键字FOR EACH ROW 指明一旦触发事件影响到关系中的任意一行，规则都将被触发[②]。

可选子句WHEN用于指定在规则已被触发但动作尚未执行前需要计算的条件。最后，要采取的动作将被作为一个PL/SQL代码块指定，一般包含一个或多个SQL语句或过程调用以执行外部程序。

R1、R2、R3和R4这4个触发器（主动规则）表现了主动规则的许多特性。首先，可用于触发器规则的基本**事件**都是一些标准的SQL更新命令：插入、删除和更新，它们在Oracle中分别用INSERT、DELETE和UPDATE关键字指定。在UPDATE情况中，用户还可以指定要更新的属性，如：UPDATE OF Salary, Dno。其次，规则的设计者还需要给出引用被触发事件——插入、删除或更新元组的一种方法。这一方法在Oracle表示法中使用OLD和NEW两个关键字来表示。关键字NEW用来指定新插入或新更新的元组，而关键字OLD则用来指定被删除的元组或更新前的元组。

因此，规则R1将在对关系EMPLOYEE应用了一个INSERT操作后被触发。R1中条件语句（NEW.Dno IS NOT NULL）将被检查，如果结果为真，则说明新插入employee的元组与某个部门相关，相应的动作就被执行。通过将新插入雇员的工资值（NEW.Salary）加到相应部门的Total_sal属性值之中，这一动作更新了对应于新插入雇员元组的DEPARTMENT元组。

规则R2与R1相似，但它是由修改关系EMPLOYEE中SALARY属性的一个UPDATE操作所触发的，而不是像R1中那样被一个INSERT操作触发。规则R3由一个对关系EMPLOYEE的Dno属性进行UPDATE操作而触发，这个UPDATE操作将改变一个雇员的所属部门。由于在规则R3中没有指定条件，

---

① 正如我们将会看到的那样，BEFORE关键字也可能会代替AFTER关键字，表明在触发事件执行之前该规则被触发。

② 同样，我们也会看到，可以指定规则仅被触发一次来替代这一关键字，即使触发事件可能会影响到多行（元组）数据。

因此，每当触发事件发生都将导致规则动作的执行。这一动作更新了雇员的旧部门和新分配的部门中的元组，把他们的工资额从原部门的Total_sal属性中减去并加到新部门的Total_sal属性中。注意，由于在这种情况下，对于这一规则没有部门的元组将被选中[①]，所以即使Dno为NULL时规则也仍将有效。

```
R1:     CREATE TRIGGER Total_sal1
        AFTER INSERT ON EMPLOYEE
        FOR EACH ROW
        WHEN ( NEW.Dno IS NOT NULL )
            UPDATE DEPARTMENT
            SET Total_sal = Total_sal + NEW.Salary
            WHERE Dno = NEW.Dno;

R2:   CREATE TRIGGER Total_sal2
      AFTER UPDATE OF Salary ON EMPLOYEE
      FOR EACH ROW
      WHEN ( NEW.Dno IS NOT NULL )
          UPDATE DEPARTMENT
          SET Total_sal = Total_sal + NEW.Salary - OLD.Salary
          WHERE Dno = NEW.Dno;

R3:   CREATE TRIGGER Total_sal3
      AFTER UPDATE OF Dno ON EMPLOYEE
      FOR EACH ROW
          BEGIN
          UPDATE DEPARTMENT
          SET Total_sal = Total_sal + NEW.Salary
          WHERE Dno = NEW.Dno;
          UPDATE DEPARTMENT
          SET Total_sal = Total_sal - OLD.Salary
          WHERE Dno = OLD.Dno;
          END;

R4:   CREATE TRIGGER Total_sal4
      AFTER DELETE ON EMPLOYEE
      FOR EACH ROW
      WHEN ( OLD.Dno IS NOT NULL )
          UPDATE DEPARTMENT
          SET Total_sal = Total_sal - OLD.Salary
          WHERE Dno = OLD.Dno;
```

(a) 自动维护DEPARTMENT关系中Total_sal属性一致性的触发器

```
R5:   CREATE TRIGGER Inform_supervisor1
      BEFORE INSERT OR UPDATE OF Salary, Supervisor_ssn
            ON EMPLOYEE
      FOR EACH ROW
      WHEN ( NEW.Salary > ( SELECT Salary FROM EMPLOYEE
                            WHERE Ssn = NEW.Supervisor_ssn ) )
          inform_supervisor(NEW.Supervisor_ssn, NEW.Ssn );
```

(b) 用于比较一个雇员和他（她）的管理者的工资的触发器

图8-2　在Oracle的表示法中指定作为触发器的主动规则

① R1、R2和R4规则也可以被写成没有条件约束的形式。但是，加入条件可能会使规则的执行更加有效，这样可以只在需要时才执行相应的动作。

选择项FOR EACH ROW子句的作用是很重要的，它表明对于每个元组，规则将分别被触发。所以这种触发器被称为**行级触发器**（row-level trigger）。如果省略了这个子句，那么该触发器就被称为**语句级触发器**（statement-level trigger），对每个触发语句仅被触发一次。为了对这两种不同类型的触发器进行比较，考虑下面的更新操作：为Dno=5的部门中的所有雇员增加10%的工资，这一操作将是触发规则R2的一个事件：

```
UPDATE  EMPLOYEE
SET     Salary = 1.1 * Salary
WHERE   Dno = 5;
```

因为上述语句可能会更新多个记录，所以如果使用图8-2中R2定义的行级语义规则，那么每一行的更新都会触发该规则。若使用语句级语义规则，那么该规则将只触发一次。在Oracle系统中，允许用户在每条规则中使用上述两种选项中的任何一种，包括使用可选择的FOR EACH ROW子句创建一个行级触发器，而省略这一子句就将创建一个语句级触发器。请注意：NEW和OLD关键字只能在行级触发器中使用。

再看一个示例，假定要检查“雇员的工资何时大于他们的直接管理者的工资”这一条件，那么有多个事件可能触发这一规则：插入一个新雇员；改变一个雇员的工资；改变一个雇员的直接管理者。假定规则中执行的动作将是调用一个外部过程inform_supervisor来通知其管理者[①]，则这条规则的编写类似于规则R5（见图8-2b）。

图8-3展示了指定Oracle触发器中可用的主要选项的语法。我们将在8.1.5节中描述SQL-99标准中的触发器语法。

```
<trigger>          ::=  CREATE TRIGGER <trigger name>
                        ( AFTER | BEFORE ) <triggering events> ON <table name>
                        [ FOR EACH ROW ]
                        [ WHEN <condition> ]
                        <trigger actions> ;
<triggering events>  ::= <trigger event> {OR <trigger event> }
<trigger event>      ::= INSERT | DELETE | UPDATE [ OF <column name> { , <column name> } ]
<trigger action>     ::= <PL/SQL block>
```

图8-3　Oracle系统中指定触发器的主要语法规则（仅含主要选项）

8.1.2　主动数据库的设计和实现

上一节概述了指定主动规则所需要的一些主要概念。本节将讨论如何设计和实现主动规则的若干问题。第一个问题涉及规则的激活、不激活和分组。除了创建规则之外，主动数据库系统还应该允许用户通过引用规则的名字对规则进行激活、不激活或删除。**不激活规则**（deactivated rule）指的是不由触发事件触发的规则。这个特性允许用户有选择地在某些不需要规则的特殊时间周期内将其设置为不激活状态。**激活命令**（activate command）将使规则再次生效。**删除命令**（drop command）则将规则从系统中删除。另一个选项是把规则分组成命名的**规则集**（rule set），这样就可以对整组规则统一地进行激活、不激活和删除。用户也可以发出一个显式的PROCESS RULES命令来触发一条规则或一个规则集，这也是很有用的。

第二个要讨论的问题是：触发动作应在触发事件发生之前、之后、还是同时开始执行？与此相关的另一个问题是：被执行的动作是作为一个单独的事务，还是作为触发规则的事务的一部分？我们首先把各种选项进行分类。需要注意的是，并不是所有选项都可以在一个特定的主动数据库系统中使用。事实上，目前大多数商用系统都仅限于使用我们讨论的选项中的一项或两项。

① 假定一个合适的外部过程已经被声明。这一特性在SQL3中已经可用。

假定触发事件的发生是作为事务执行的一部分，那么首先应该考虑如何把对规则条件的估算与触发事件关联起来的各种选项。规则条件的估算也被称为**规则考虑**，因为必须在估算之后，根据其结果是true还是false，才能决定是否执行规则中的动作。对于规则考虑有3种主要的可能情况：

(1) **立即考虑**（immediate consideration）：条件估算作为包含该触发事件的同一事务的一部分，并且这一条件将被立即估算。对于这种情况还可以进一步分为3个选项：

- 在触发事件执行之前估算条件。
- 在触发事件执行之后估算条件。
- 估算条件替代触发事件的执行。

(2) **延迟考虑**（deferred consideration）：在包含触发事件的事务结束时才进行条件估算。在这种情况下，可能会有许多被触发的规则在等待条件的估算结果。

(3) **分离考虑**（detached consideration）：条件估算作为一个独立的事务，即与触发事务相分离。

接下来考虑的一组选项是估算规则条件与执行规则动作之间的联系。同样的，在这种情况下也有3个可能的选项：**立即执行**（immediate execution）、**延迟执行**（deferred execution）和**分离执行**（detached execution）。大多数主动系统使用第一个选项。也就是说，一旦规则条件被估算并返回true，那么就会立即执行规则动作。

Oracle系统（见8.1.1节）使用了立即考虑模型，但是它允许用户为每条规则指定立即条件估算是否采用了*before*或*after*选项。它还使用了立即执行模型。STARBURST系统（见8.1.3节）采用的是延迟考虑选项，这意味着所有被这个事务触发的规则必须等待，直到该触发事务执行完毕，在规则条件估算完成之前发出COMMIT WORK命令[①]。

主动数据库规则所要考虑的另一个问题是行级规则（row-level rule）和语句级规则（statement-level rule）的区别。因为SQL的更新语句（作为触发事件）可能指定一个元组的集合，所以必须有能力区别这条规则应该被考虑作为语句级规则仅执行一次，还是应该被考虑作为行级规则分别对每一行（即每一元组）都执行。SQL-99标准（见8.1.5节）和Oracle系统（见8.1.1节）允许用户对每条规则选择上述两种不同执行方式中的一种，而STARBURST系统仅使用语句级语义。我们将在8.1.3节中举例说明如何在STARBURST系统中指定一个语句级触发器。

尽管主动规则可以潜在地简化数据库和软件的开发过程，但要使主动规则能真正地广泛应用还存在一些困难。其中的一个困难是还没有可以在规则的设计、编写和检验中容易使用的技术。例如，目前很难检验一组规则它们之间是**一致的**，这就是说在一组规则中任意两个或多个规则之间不会互相矛盾。另外，目前也很难保证在所有环境中都能**终止**一组规则。为简要说明“终止规则”这个问题，考虑图8-4所示的例子。规则R1被一个TABLE1上的INSERT事件触发，它将采取的动作之一是对TABLE2中Attribute1属性的一个UPDATE事件。但是触发规则R2的触发事件正是一个对TABLE2中Attribute1属性的UPDATE事件，并且它执行的动作之一就是在TABLE1上的一个INSERT事件。在这个例子中，很容易看出，这两条规则会彼此不断地触发对方而导致无法终止规则。然而，如果需要编写很多规则，那么将很难保证是否会出现“无法终止规则”的情况。

```
R1:  CREATE TRIGGER T1
     AFTER INSERT ON TABLE1
     FOR EACH ROW
        UPDATE TABLE2
        SET Attribute1 = . . . ;

R2:  CREATE TRIGGER T2
     AFTER UPDATE OF Attribute1 ON TABLE2
     FOR EACH ROW
        INSERT INTO TABLE1 VALUES ( . . . ) ;
```

图8-4　解释主动规则中终止问题的示例

如果要继续发掘主动规则的潜力，那么就有必要开发一些用于设计、调试和监控主动规则的工具，以帮助用户设计和调试他们的规则。

---

① STARBURST系统也允许用户通过PROCESS RULES命令显式地启动规则考虑模型。

8.1.3　STARBURST 系统中使用语句级主动规则的示例

本节将给出一些如何在STARBURST实验型DBMS中指定规则的示例。语句级规则是STARBURST系统中唯一允许使用的规则形式，这个示例将有助于我们了解如何编写一条语句级规则。

在图8-5中定义了3条主动规则R1S、R2S和R3S，它们分别对应于图8-2中的前3条规则，所不同的是这里使用的是STARBURST语法和语句级语义。我们可以使用R1S规则来解释一下规则的结构。CREATE RULE语句指定了R1S规则的名字为Total_sal1；ON子句指定了该规则所在的关系，本例中规则R1S所在的关系是EMPLOYEE；WHEN子句被用来指定触发器规则的**事件**[①]；可选的IF子句被用来指定任何需要检查的**条件**。最后，THEN子句被用来指定所要采取的某个**动作**（或多个动作），它们一般由一条或多条SQL语句组成。

```
R1S:   CREATE RULE Total_sal1 ON EMPLOYEE
       WHEN   INSERTED
       IF     EXISTS    ( SELECT * FROM INSERTED WHERE Dno IS NOT NULL )
       THEN   UPDATE      DEPARTMENT AS D
              SET         D.Total_sal = D.Total_sal +
                          ( SELECT SUM (I.Salary) FROM INSERTED AS I WHERE D.Dno = I.Dno )
              WHERE       D.Dno IN ( SELECT Dno FROM INSERTED );

R2S:   CREATE RULE Total_sal2 ON EMPLOYEE
       WHEN   UPDATED   ( Salary )
       IF     EXISTS    ( SELECT * FROM NEW-UPDATED WHERE Dno IS NOT NULL )
              OR EXISTS ( SELECT * FROM OLD-UPDATED WHERE Dno IS NOT NULL )
       THEN   UPDATE      DEPARTMENT AS D
              SET         D.Total_sal = D.Total_sal +
                          ( SELECT SUM (N.Salary) FROM NEW-UPDATED AS N
                            WHERE D.Dno = N.Dno ) -
                          ( SELECT SUM (O.Salary) FROM OLD-UPDATED AS O
                            WHERE D.Dno = O.Dno )
              WHERE       D.Dno IN ( SELECT Dno FROM NEW-UPDATED ) OR
                          D.Dno IN ( SELECT Dno FROM OLD-UPDATED );

R3S:   CREATE RULE Total_sal3 ON EMPLOYEE
       WHEN   UPDATED   ( Dno )
       THEN   UPDATE      DEPARTMENT AS D
              SET         D.Total_sal = D.Total_sal +
                          ( SELECT SUM (N.Salary) FROM NEW-UPDATED AS N
                            WHERE D.Dno = N.Dno )
              WHERE       D.Dno IN ( SELECT Dno FROM NEW-UPDATED );

              UPDATE      DEPARTMENT AS D
              SET         D.Total_sal = Total_sal -
                          ( SELECT SUM (O.Salary) FROM OLD-UPDATED AS O
                            WHERE D.Dno = O.Dno )
              WHERE       D.Dno IN ( SELECT Dno FROM OLD-UPDATED );
```

图8-5　STARBURST表示法中使用语句级语义的主动规则

在STARBURST系统中，用于指定触发规则的基本事件一般是标准SQL的更新命令：INSERT、DELETE和UPDATE。它们在STARBURST表示法中分别对应于INSERTED、DELETED和UPDATED关

① 请注意WHEN关键字在STARBURST系统中用于指定事件，但在SQL和Oracle系统的触发器定义中则用于指定规则的条件。

键字。其次，规则的设计者还需要一种能够引用已被更新过的元组的方法。在STARBURST表示法中，这一需求可以通过INSERTED、DELETED、NEW-UPDATED和OLD-UPDATED4个关键字所指定的4个**转换表**（transition table）（关系）来实现。转换表中分别包含新插入的元组、已删除的元组、被更新元组被更新之前的“旧”元组、和被更新元组被更新之后的“新”元组。显然，对于不同的触发事件，使用的转换表及其个数也不同。规则的制订者可以在定义规则条件和规则动作的时候引用这些表。转换表中保存了与ON子句指定的关系表具有相同类型的元组，对R1S、R2S和R3S规则而言，这个关系表就是EMPLOYEE关系。

在语句级语义中，规则的设计者可以将转换表看作一个整体，并且规则也只被触发一次，因此规则的写法也必定区别于行级语义的写法。因为在一个简单的插入语句中，可能会有多个雇员元组被插入，所以必须检查是否至少有一个新插入的雇员元组会与某一部门相关。在规则R1S中，要检查的条件是：

```
EXISTS (SELECT * FROM INSERTED WHERE Dno IS NOT NULL )
```

如果其估算结果为true，该动作就会被执行，被执行的动作会更新与新插入雇员元组相关的DEPARTMENT关系中的元组，并将新插入雇员的工资加到相关部门的Total_sal属性中。因为有不止一个被插入的雇员属于同一个部门，所以使用SUM聚合函数以确保他们所有的工资都会被加上。

规则R2S与规则R1S类似，不同的是它由一个或多个雇员的工资的UPDATE操作所触发，而不像R1S那样是由一个INSERT操作触发的。规则R3S则是由一个对EMPLOYEE关系的Dno属性进行更新操作触发，这种情况发生在一个或多个雇员从原所属部门调到另一个部门的时候。R3S规则没有条件，因此无论何时只要该事件发生动作都将被执行[①]。执行该动作的结果将更新被重新分配部门的那个雇员的新旧两个部门的Total_sal属性，即将该雇员的工资加到新部门中对应元组的Total_sal属性值中，并从原来所属部门中对应元组的Total_sal属性值中减去其工资。

在这个示例中，通过对照图8-2和图8-5，编写一个语句级规则要比编写一个行级规则复杂一些。但这也不是一个普遍的规律，其他类型的主动规则使用的语句级语法可能比使用行级语法还容易。

STARBURST系统中主动规则的执行模型使用的是**延迟考虑**，即在一个事务中被触发的所有规则都将被放到一个称为**冲突集**（conflict set）的集合中。只有当事务结束时（即事务发出COMMIT WORK命令之后），才考虑规则条件的估算和动作执行中要用到的冲突集。STARBURST也允许用户通过发出PROCESS RULES命令，在一个事务执行过程中进行规则条件的估算。因为有很多条规则等待进行条件的估算，因此有必要为它们指定一个顺序。在STARBURST中规则声明的语法允许指定一组规则的执行顺序，以指导系统按事先制订好的顺序执行多条规则[②]。另外，转换表INSERTED、DELETED、NEW-UPDATED和OLD-UPDATED中还包含了影响这些表的某个事务中所有操作带来的净效应（net effect），因为在该事务中每个表都可能被执行了多个操作。

8.1.4　主动数据库的潜在应用

我们简要讨论主动规则的一些潜在应用。显然，一个重要的应用是可以**获知**发生的某些情况。例如，一个主动数据库可用于监控工作，如监控一个工业锅炉的炉温。应用程序可以周期性地向数据库中插入一个从温度传感器读取的炉温记录，而主动规则就可以设计成当插入一条炉温记录时被触发，当炉温超过警戒值的情况发生时，就执行规则定义的动作：发出警报。

通过指定引起违反约束的事件类型，主动规则也可以用来**增强完整性约束**，接下来计算合适的条件以检查是否有引起违反约束的事件发生。因此，复杂的应用约束，通常指一些**业务规则**，常常会采用这种方式来增强完整性。例如，在UNIVERSITY数据库应用中，无论什么时候输入一个新的成

---

① 正如前面Oracle的例子那样，规则R1S和R2S也可以写成无条件的形式。但加入条件可能就会使规则的执行更加有效，这样可以只在需要规则的时候才执行相应的动作。

② 如果在一组规则中不指定执行顺序，那么系统的默认顺序将根据规则声明时的先后顺序确定。

绩，都会有一条规则来监控学生的GPA。如果某位学生的GPA低于某个阈值，便会向他的指导教师发出警示信息。另外，还可以有一条规则在允许一位学生选某门课程之前，检查他是否已经完成了在选修这门课之前必须学习的预备课程，等等。

其他应用还包括自动**维护导出数据**（maintain derived data），如前面示例中的R1到R4规则，当一个雇员的元组发生变化的时候，就要维护与它相关的导出属性Total_sal。其他与此类似的应用是使用主动规则在每个基本表发生变化时维护**物化视图**（materialized view）（见《数据库系统基础：初级篇》的8.8节）的一致性。这类应用也与新的数据仓库技术有关（见第13章）。相关的应用通过在主表变化时指定修改副本的规则来维护**复制表**的一致性。

8.1.5　SQL-99 中的触发器

SQL-99标准中的触发器与8.1.1节中讨论的示例十分类似，只是语法上有些不同。可以为触发规则指定的基本**事件**是标准SQL更新命令：INSERT、DELETE和UPDATE。在UPDATE的情况下，可以指定要更新的属性。行级触发器和语句级触发器都可以使用，在触发器中分别通过FOR EACH ROW子句和FOR EACH STATEMENT子句表示。语法上的一点不同是触发器可以为新元组和原元组指定特定的元组变量名，而不是使用图8-1所示的关键字NEW和OLD。图8-6中的触发器T1说明了如何按照SQL-99语法指定图8-1a中的行级触发器R2。在REFERENCING子句中，我们定义元组变量（别名）O和N米分别表示OLD元组（修改之前）和NEW元组（修改之后）。图8-6中的触发器T2说明了如何用SQL-99语法指定图8-5中的语句级触发器R2S。对于语句级触发器，REFERENCING子句把包含所有新元组的表（新插入的或者新更新的）表示为N，而把包含所有原元组的表（删除的元组或者被更新之前的元组）表示为O。

```
T1:  CREATE TRIGGER Total_sal1
     AFTER UPDATE OF Salary ON EMPLOYEE
     REFERENCING OLD ROW AS O, NEW ROW AS N
     FOR EACH ROW
     WHEN  ( N.Dno IS NOT NULL )
     UPDATE DEPARTMENT
     SET Total_sal = Total_sal + N.salary – O.salary
     WHERE Dno = N.Dno;

T2:  CREATE TRIGGER Total_sal2
     AFTER UPDATE OF Salary ON EMPLOYEE
     REFERENCING OLD TABLE AS O, NEW TABLE AS N
     FOR EACH STATEMENT
     WHEN  EXISTS  ( SELECT * FROM N WHERE N.Dno IS NOT NULL ) OR
           EXISTS  ( SELECT * FROM O WHERE O.Dno IS NOT NULL )
     UPDATE DEPARTMENT AS D
     SET D.Total_sal = D.Total_sal
     + ( SELECT SUM (N.Salary) FROM N WHERE D.Dno=N.Dno )
     – ( SELECT SUM (O.Salary) FROM O WHERE D.Dno=O.Dno )
     WHERE Dno IN ( ( SELECT Dno FROM N ) UNION ( SELECT Dno FROM O ) );
```

图8-6　用SQL-99语法定义触发器T1

8.2　时态数据库概念

广义上讲，时态数据库包含所有在组织信息时需要时间方面信息的数据库应用。因此，这些应用提供了一个很好的示例，以说明有必要开发提供给应用开发者使用的一套统一概念。虽然在早期的数据库应用中，就已经有人开始开发时态数据库应用了。然而，在创建这些应用过程中，

应用程序的设计者和开发者的主要任务仍然是去发现、设计、编程和实现他们所需要的时态概念。有许多应用的例子可以说明在维护一个数据库中的信息时，是需要时间方面的信息的。这些例子包括：健康卡，用于维护病人的历史记录；保险，用于索赔和记录事故历史，同时记录保险契约有效期间的时间信息；日常生活中的预订系统（如预订酒店、航班、出租车、火车等），在预约有效期内有关时间和日期的信息也都是必需的；科学数据库：从实验获得的数据包括每个数据的测量时间；等等。即使是本书中所使用的两个示例也能够很容易地被扩展成时态应用。在COMPANY数据库中，可能会希望保存每位雇员的工资（SALARY）、工作类型（JOB）和从事过的项目（PROJECT）等方面的历史记录。在UNIVERSITY数据库中，时间信息已被包含在每门课所在的学期（SEMESTER）和学年（YEAR）之中了，它还包括每位学生的成绩记录，以及参与研究的信息。事实上，由此可以得出一个结论：大多数数据库应用都会包含一些时态方面的信息。但因为与时间有关的信息可能会增加应用的复杂性，所以用户常常会简化甚至忽略时间方面的信息。

本节中，我们将介绍一些用于处理时态数据库应用复杂性的概念。8.2.1节将给出在一个数据库中如何表示时间的概述，因为在实际应用中，不仅需要表示不同类型的时态信息，还需要表示不同时间维上的信息。8.2.2节将讨论在关系数据库中如何加入时间信息的内容。8.2.3节给出了一些在允许复杂结构对象的数据库模型（例如对象数据库）中表示时态的方法。8.2.4节将介绍一些查询时态数据库的操作，并概述了TSQL2语言，它是用时态概念扩展了SQL语言。8.2.5节将讨论时间序列数据，这是一种实际应用中非常重要的时态数据类型。

8.2.1　时间表示法、日历和时间维

对时态数据库来说，时间被认为是多个**点**（point）的*有序序列*，这些点的**粒度**（granularity）由应用本身确定。例如，假定某个时态应用可能永远不会用到少于1秒的时间单元，那么每个时间点在这种粒度下将代表一秒钟。但现实世界中一秒钟是一个（短暂的）时间段，而不是一个点，因为它还可以进一步细分为毫秒、微秒等更小的时间单元。因此，时态数据库的研究者们使用了**时间元子**（chronon）这一术语，它取代了点的概念，以描述某一特殊应用的最小粒度。选定了时间的最小粒度（如1秒钟），我们就认为同一秒钟内发生的多个事件是同时发生的，即使实际情况可能并非如此。

因为我们并不知道时间确切的起点和终点，因此需要一个参照时间点，根据它就可以度量特定的时间点。由于文化的不同，按照不同的参照时间点表示的各种日历也是不同的（例如罗马日历、中国日历、伊斯兰日历、印度日历、犹太日历、哥普特日历等等）。为了方便，一个**日历**通常将时间分为不同的单位。大多数日历都将1分钟分为60秒、1小时分为60分、1天分为24个小时（这是基于地球自转一周的物理时间），1星期分为7天等。进而，将1个月分为多少天和1年包括多少天是由太阳和月亮运行等自然现象决定的，并且有时候并不是规律性的。在西方国家普遍采用的罗马日历中，1个月被分为28、29、30或31天，并将1年分为12个月。有一些复杂的公式可用于不同的时间单位之间的转换。

在SQL2中，时态数据类型（见《数据库系统基础：初级篇》的第8章）包括：DATE（指定年、月、日为YYYY-MM-DD形式）、TIME（指定小时、分、秒为HH:MM:SS形式）、TIMESTAMP（指定Date和Time两种数据类型的组合形式，如有必要还可以选择加入比秒更小的时间单位）、INTERVAL（一段相对时间的持续时间，如10天或250分钟）、PERIOD（从一个固定的时间点开始的一段固定的持续时间，例如，从2007年1月1日到2007年1月10日的一段长度为10天的时间区间）[①]。

1．事件信息与持续事件（或状态）信息

时态数据库要存储关于某些事件的发生时间或某一事实判定为真时的时间等信息。目前有几种

① 很遗憾，在使用这一术语时其含义并不总是一致的，例如：*interval*这一术语也经常被用于表示一段固定的持续时间。为了达到一致性的目的，本书将统一使用SQL术语集。

不同类型的时态信息。**点事件**或**事实**一般与数据库中按某一粒度划分的**单个时间点**有关。例如，银行存款事件可能与存款结束时的时间戳有关；再如，某产品（事实）的月销售总额可能与某个特定月份有关（如2007年2月）。注意，即使这样的事件（或事实）有不同的时间粒度，但它们都只与数据库中的某一单个时间点的值有关。这种类型的信息经常表示为**时间序列数据**（time series data），在8.2.5节中将进一步讨论这类信息。另一方面，**持续事件**或**事实**与数据库中的一个特定**时间区间**（time period）有关[①]。例如，一位雇员可能从2001年8月15日到2006年11月20日这段时间里一直在某个公司中工作。

时间区间由它的**开始时间点**和**结束时间点**来表示，即，可表示为[START-TIME，END-TIME]的形式。例如，上面的时间区间就可表示为[2001-08-15，2006-11-20]。因此，时间区间通常解释为一个包括开始时间和结束时间在内的所有时间点的集合。当然，开始时间和结束时间也都是按照一定粒度划分的。因此，若假定时间粒度为天，那么时间区间[2001-08-15，2006-11-20]将代表包括2001年8月15日和2006年11月20日在内的所有天的集合[②]。

2．有效时间和事务时间维

在数据库中，当一个特定的事件或事实与某一特定时间点或时间区间相关联时，对这种相关性可能存在不同的解释。其中最自然的一种解释就是：相关时间（associated time）是现实世界中事件发生的时间或一个事实被认为是真的一段时间。如果使用这种解释，相关时间通常是指**有效时间**（valid time），而使用这种解释的时态数据库则被称为**有效时间数据库**（valid time database）。

然而，相关时间也可以采用另一种解释。相关时间指的是信息实际存储在数据库中的时间，它的值实际上是信息在系统中处于可用状态期间系统时钟的值[③]。在这种情况下，相关时间则被称为**事务时间**（transaction time）。而使用这种解释的时态数据库就被称为**事务时间数据库**（transaction time database）。

当然，还有其他解释，但上述两种解释一般认为是最常见的，它们也被称为**时间维**（time dimension）。在一些应用中，可能仅需要时间维中的一种，但在另一些情况中，时间维的这两种解释可能都需要考虑。在后一种情况下，时态数据库就被称为**双时态数据库**（bitemporal database）。对于其他与时间相关的解释，用户也可以定义适合于自己的语法并编写相应的应用，这种相关时间则被称为**用户自定义时间**（user-defined time）。

下一节将举例说明如何将上面讨论的这些概念加入到关系数据库中。8.2.3节中将介绍一种将时态概念加入到对象数据库中的方法。

8.2.2　使用元组版本化在关系数据库中加入时间

1．有效时间关系

现在来考虑如何在关系模型中表示不同类型的时态数据库。首先，假定要将现实世界中发生变化的历史放入数据库中。重新考虑一下图8-1中的数据库，并假定这一应用的粒度是天。这样就可以通过增加数据类型为DATE、时间粒度为天的两个属性Vst（Valid Start Time，有效开始时间）和Vet（Valid End Time，有效结束时间），将关系EMPLOYEE和DEPARTMENT转换为两个**有效时间关系**。修改后的关系如图8-7a所示，关系名已分别改为EMP_VT和DEPT_VT。

---

① 与一段固定的持续时间一样，时间区间这一术语也经常被称为**时间间隔**（time interval），但是为了避免混淆，本书将统一使用SQL术语集中的区间period这一术语。

② [2001-08-15 ，2006-11-20]表示法被称为闭区间表示法。当然，也可以使用开区间表示法，表示为[2001-08-15 ，2006-11-21)，这时时间结束点将不包含在时间点集合中。尽管有时开区间用起来更方便一些，但是为了避免混淆，本书中将统一使用闭区间表示法。

③ 关于相关时间的更多说明请参阅8.2.3节。

EMP_VT

Name	Ssn	Salary	Dno	Supervisor_ssn	Vst	Vet

DEPT_VT

Dname	Dno	Total_sal	Manager_ssn	Vst	Vet

(a) 有效时间数据库模式

EMP_TT

Name	Ssn	Salary	Dno	Supervisor_ssn	Tst	Tet

DEPT_TT

Dname	Dno	Total_sal	Manager_ssn	Tst	Tet

(b) 事务时间数据库模式

EMP_BT

Name	Ssn	Salary	Dno	Supervisor_ssn	Vst	Vet	Tst	Tet

DEPT_BT

Dname	Dno	Total_sal	Manager_ssn	Vst	Vet	Tst	Tet

(c) 双时态数据库模式

图8-7　不同类型的时态关系数据库

考虑时态关系EMP_VT与无时态关系EMPLOYEE（如图8-1所示）有什么不同[①]。在EMP_VT关系中，每个元组V表示的是一个仅在时间段 [V.Vst,V.Vet]中有效的（在现实世界中）雇员信息**版本**（version），而在EMPLOYEE关系中每个元组仅代表每位雇员的当前状态和当前版本。在EMP_VT关系中，每位雇员的**当前版本**（current version）都有一个特殊值*now*作为它的有效结束时间。这一特殊值*now*是一个在时间进程中代表当前时间的**时态变量**（temporal variable）。而无时态的关系EMPLOYEE仅包括那些EMP_VT关系中Vet值为*now*的元组。

图8-8列出了有效时间关系EMP_VT和DEPT_VT中的一些元组版本。雇员Smith有两个版本，Wong有3个，Brown有一个，Narayan有一个。现在，应该可以了解到当信息发生改变的时候，有效时间关系应该采取什么样的动作了。每当一个或多个雇员的属性被**更新**时，它不像无时态关系那样用新的元组覆盖原有元组，而是创建一个新的元组版本，同时通过将VET的值改为结束时间来**关闭**当前版本。因此，当用户发出操作命令："在2003年6月1日把Smith的工资更新为$30 000"，Smith的第二个版本便会被创建（见图8-8）。在进行更新时，Smith的第一个版本是当前版本，其VET值为*now*，但更新过后，*now*被改为2003年5月31日（因为在粒度为天的情况下，旧版本的结束时间应该是新版本有效开始时间2003年6月1日的前一个时间点，即2003年5月31日），以表明该版本已经处于**关闭**状态或者可以称为**历史版本**（history version），而新的版本（即第二个版本）成为当前版本。

值得注意的是，在有效时间关系中，用户通常必须提供更新的有效时间。例如，现实世界中工资变更要到2003年6月1日才会生效，但对Smith工资的更新可能会早在2003年5月15日上午8点52分12秒就已经输入了数据库。这种情况称为**预期更新**（proactive update），因为更新在现实世界中生效之前就已经被应用于数据库了。如果更新是在现实世界已经生效之后才被应用于数据库的，那么就称为**滞后更新**（retroactive update）。如果更新是在现实世界中生效的同时被应用于数据库的，就称为**同步更新**（simultaneous update）。

① 无时态关系也被称为**快照关系**（snapshot relation），因为它仅代表数据库的一个当前状态或当前快照。

EMP_VT

Name	Ssn	Salary	Dno	Supervisor_ssn	Vst	Vet
Smith	123456789	25000	5	333445555	2002-06-15	2003-05-31
Smith	123456789	30000	5	333445555	2003-06-01	Now
Wong	333445555	25000	4	999887777	1999-08-20	2001-01-31
Wong	333445555	30000	5	999887777	2001-02-01	2002-03-31
Wong	333445555	40000	5	888665555	2002-04-01	Now
Brown	222447777	28000	4	999887777	2001-05-01	2002-08-10
Narayan	666884444	38000	5	333445555	2003-08-01	Now

...

DEPT_VT

Dname	Dno	Manager_ssn	Vst	Vet
Research	5	888665555	2001-09-20	2002-03-31
Research	5	333445555	2002-04-01	Now

...

图8-8　有效时间关系EMP_VT和DEPT_VT中的一些元组版本

从无时态数据库中**删除**一个雇员的相对应动作，可以通过关闭被删除的雇员的当前版本，应用到有效时间数据库中。例如，如果Smith将于2004年1月19日真正离开公司，这种情况在有效时间数据库中，将通过把Smith当前版本的Vet值*now*改置为2004-01-19来实现。在图8-8中，Brown没有当前版本，这是因为他可能已经于2002年8月10日离开了公司，并被逻辑删除了。但因为使用的是时态数据库，所以Brown的原有信息仍将保留在数据库中。

在一个无时态数据库中**插入**新雇员的操作对应的动作是：为新雇员创建第一个元组版本，并通过将Vst的值置为该雇员正式开始工作的有效（现实世界）时间，使其成为当前版本。在图8-7中，雇员Narayan的元组说明了这种情况，因为它的第一个版本还从来没有被更新过。

需要注意，无时态关系中的码（如EMPLOYEE关系中的Ssn）对于有效时间关系中的每个元组（版本）不再保持唯一性。新关系EMP_VT的码将是一个无时态关系的码和有效开始时间属性Vst的组合[①]，即，使用(Ssn ,Vst)作为主码。这是因为在时间的任何点上，每个实体最多只能有一个有效版本。因此，在有效时间关系中就存在这样的约束：代表同一个实体的任意两个元组版本在有效时间区间中没有交集。要说明的是，如果无时态关系的主码值也随时间而变化，为了将现实世界中同一个实体的所有版本联系在一起，那么我们必须有一个唯一的**代理码属性**（surrogate key attribute），对于现实世界中的实体它的值应该永远保持不变。

有效时间关系可以跟踪所有在现实世界中实际发生过的变化。因此，如果所有现实世界的变化都被记录下来，那么数据库中就可以保留一个表示现实世界状态的历史记录。然而，因为更新、插入和删除操作可能会采取预先执行和滞后执行的方式，因此在任意时间点上，都不会有一个实际的数据库状态的记录。如果实际的数据库状态对一个数据库应用来说非常重要，那么就应该采用事务时间关系的形式。

2．事务时间关系

在一个事务时间数据库中，对数据库的修改无论何时发生，用于修改（插入、删除或更新）的实际事务的**时间戳**都会被记录下来。对于大多数修改总是同时发生的情形，这种数据库是最有用的，例如，在实时股票交易或银行事务中。如果要把图8-1所示的无时态数据库转换为一个事务时间数据库的话，那么就可以通过在EMPLOYEE和DEPARTMENT两个关系中分别增加数据类型为TIMESTAMP的两个属性Tst（Transaction Start Time，事务开始时间）和Tet（Transaction End Time，事务结束时间）

① 也可以使用一个无时态关系的码和有效结束时间Vet的组合作为新关系EMP_VT的码。

来将它们转换为**事务时间关系**。如图8-7b所示，两个关系分别重新命名为EMP_TT和DEPT_TT。

在EMP_TT关系中，每个元组V代表在实际时间V.Tst创建和在实际时间V.Tet（逻辑）删除雇员信息版本（之所以删除，是因为信息不再正确）。在EMP_TT中，每个雇员的当前版本一般都会有一个专门值*uc*（Until Changed，直至被改变）[①]，作为事务的结束时间。这说明，在元组被某些其他事务改变之前，它所表示的信息都是正确的。事务时间数据库也称为**回滚数据库**（rollback database）[②]，因为用户可以通过检索事务时间区间为 [V.Tst ,V.Tet]，并且包含时间点T的所有元组版本V，使数据库在逻辑上回滚到过去的任何时间点T的数据库状态。

3．双时态关系（bitemporal relation）

一些数据库应用可能既需要有效时间，又需要事务时间，这就产生了**双时态关系**。例如，图8-7c展示了如何使两个无时态关系（见图8-1）EMPLOYEE和DEPARTMENT看起来像两个双时态关系EMP_BT和DEPT_BT。图8-9列出了这两个关系中的一些元组。在这些表中，事务结束时间Tet为*uc*的元组，是表示当前有效信息的元组，而那些Tet属性为一个绝对时间戳的元组，是这个时间戳之前都是有效的元组。图8-9中具有*uc*值的元组分别对应于图8-7中的有效时间元组。在每个元组中事务的开始时间属性Tst，即是创建这个元组的事务的时间戳。

EMP_BT

Name	Ssn	Salary	Dno	Supervisor_ssn	Vst	Vet	Tst	Tet
Smith	123456789	25000	5	333445555	2002-06-15	Now	2002-06-08, 13:05:58	2003-06-04,08:56:12
Smith	123456789	25000	5	333445555	2002-06-15	2003-05-31	2003-06-04, 08:56:12	uc
Smith	123456789	30000	5	333445555	2003-06-01	Now	2003-06-04, 08:56:12	uc
Wong	333445555	25000	4	999887777	1999-08-20	Now	1999-08-20, 11:18:23	2001-01-07,14:33:02
Wong	333445555	25000	4	999887777	1999-08-20	2001-01-31	2001-01-07, 14:33:02	uc
Wong	333445555	30000	5	999887777	2001-02-01	Now	2001-01-07, 14:33:02	2002-03-28,09:23:57
Wong	333445555	30000	5	999887777	2001-02-01	2002-03-31	2002-03-28, 09:23:57	uc
Wong	333445555	40000	5	888667777	2002-04-01	Now	2002-03-28, 09:23:57	uc
Brown	222447777	28000	4	999887777	2001-05-01	Now	2001-04-27, 16:22:05	2002-08-12,10:11:07
Brown	222447777	28000	4	999887777	2001-05-01	2002-08-10	2002-08-12, 10:11:07	uc
Narayan	666884444	38000	5	333445555	2003-08-01	Now	2003-07-28, 09:25:37	uc

...

DEPT_VT

Dname	Dno	Manager_ssn	Vst	Vet	Tst	Tet
Research	5	888665555	2001-09-20	Now	2001-09-15,14:52:12	2001-03-28,09:23:57
Research	5	888665555	2001-09-20	1997-03-31	2002-03-28,09:23:57	uc
Research	5	333445555	2002-04-01	Now	2002-03-28,09:23:57	uc

图8-9　双时态关系EMP_BT和DEPT_BT中的一些元组版本

现在考虑如何在一个双时态关系中完成一个**更新操作**。在这个双时态数据库模型中[③]，除了具有*uc*值的事务结束时间属性Tet外，元组中没有任何属性会发生物理上的变化[④]。为了说明如何创建元组，考虑EMP_BT关系。一个雇员元组的当前版本V，其Tet属性值为*uc*，而Vet属性值为*now*。如果要更新某个属性（如Salary），那么执行更新操作的事务T应具有两个参数：新工资开始生效（在现实世界中）时的新Salary值和有效时间VT。假定VT-是给定有效时间粒度下的VT前的一个时间点，并且事务T有一个时间戳TS(T)，那么将在EMP_BT表中发生如下所示的物理变化：

(1) 制作当前版本V的一个副本V_2，且令V_2.Vet=VT-，V_2.Tst=TS(T)，V_2.Tet=*uc*，并将V_2插入到

---

① 事务时间关系中的*uc*变量对应于有效时间关系中的*now*变量，它们只在语法上存在细微差别。

② 这里，回滚这一概念与数据库恢复时的事务回滚概念（见第3章）不同，数据库恢复时，更新操作物理上是undo操作，但是这里更新只是逻辑上的undo操作，因为它允许用户检查数据库在前一个时间点上的状态。

③ 现在已经提出的时态数据库模型有多种，为了说明概念，这里仅描述指定的模型。

④ 一些双时态模型也允许更改Vet属性，但是在那些模型中，元组的解释就不同了。

EMP_BT中。V_2是前一个当前版本V在有效时间VT一关闭之后的一个副本。

(2) 制作当前版本V的一个副本V_3，且令V_3.Vst=VT，V_3.Vet=*now*，V_3.Salary为新的工资值，V_3.Tst=TS(T)，V_3.Tet=*uc*，将V_3插入到EMP_BT关系中，V_3成为新的当前版本。

(3) 既然当前版本V已不再代表正确信息，则将V.Tet置为TS(T)。

作为一个说明，考虑在图8-9中EMP_BT关系的前三个元组V_1、V_2和V_3。在将Smith的工资从25 000更新为30 000之前，只有V_1在EMP_BT关系中，并且是当前版本，它的Tet属性值为*uc*。随后，一个时间戳TS(T)为“2003-06-04 , 08:56:12”的事务T，随着有效时间“2003-06-01”的生效将工资更新为30000。除了Vet属性被设置为比新的有效时间早一天的“2003-05-31”，并且Tst属性是更新事务的时间戳之外，元组V_2作为V_1的副本被创建。接着，元组V_3也被创建，它拥有新的工资值，并且它的Vst属性被设置为“2003-06-01”，其Tst属性也是更新事务的时间戳。最后，V_1的属性Tet被设置为更新事务的时间戳“2003-06-04 , 08:56:12”。需要注意的是，这里执行的是*滞后更新*，因为它在2003年6月4日才被执行，但工资的实际变更却在2003年6月1日就已经生效。

类似地，当Wong的工资和部门（同时）被更新为30 000和5时，更新事务的时间戳是“2001-01-07 , 14:33:02”，并且更新的有效时间为“2001-02-01”。因此，这里执行的是一个*预期更新*操作，因为事务在2001年1月7日运行，却在2001年2月1日生效。这种情况下，元组V_4将逻辑地被V_5和V_6代替。

接下来，考虑图8-9所示的EMP_BT关系中的元组V_9和V_{10}，用它们来说明如何在一个双时态关系中完成一个**删除操作**。这里，雇员Brown离开公司的生效日期为2002年8月10日，这一逻辑删除是通过一个TS(T)= 2002-08-12, 10:11:07的事务T 来执行。在执行之前，V_9是Brown的当前版本且其Tet值为*uc*。这个逻辑删除的实现是通过设置V_9.Tet为“2002-08-12,10:11:07”使其失效，并为Brown创建*最终版本*（final version）V_{10}，且令其Vet=2002-08-10（见图8-9）来完成。最后，通过创建在EMP_BT表中的V_{11}元组所示的*最初版本*（first version）来完成一个**插入操作**。

4．实现考虑

存储时态关系元组的方式有许多种。其中一种就是在同一个表中存储所有元组，如图8-8和图8-9所示。还有一种方法是创建两个表：一个用于存储当前的有效信息，另一个用于存储其他的元组。例如，在双时态关系EMP_BT中，把那些Tet属性值为*uc*和Vet属性值为*now*的元组存储在一个关系中，因为它们代表当前的有效元组（也可以说，代表当前的快照），所以都被存入*当前表*中。而所有其他元组保存在另一个关系中。这样就允许数据库管理员可以为不同的关系选择不同的存取路径，如为每一个关系建立索引。另外，还可以保持当前表大小的合理性。另一种可能的方式是创建第三个表，用来保存那些Tet属性不是*uc*的元组。

存储时态关系元组的另一种方法是：将时态关系的属性*垂直分片*（vertically partition）为多个分离的关系。这样做是因为如果一个关系有很多属性，那么每当这些属性中的任何一个被更新，就需要新建一个包含全部属性的、新的元组版本。如果属性的更新是异步的，则每个新版本元组就可能仅在一个属性上与其他元组不同，因而没有必要重复保存元组的其他相同属性。因此，如果创建一个单独的关系，使它仅包含每个关系中的那些经常发生同步变化的属性和被重复的主码，那么这个数据库将被认为是**时态范式**（temporal normal form）的。但是，为了合并信息，需要一种被称为**时态相交连接**（temporal intersection join）的特殊连接，它的实现代价一般会比较昂贵。

需要重点注意的是，双时态数据库允许保存变化的完整记录，甚至对一条记录的错误更正也是可能实现的。例如，数据库中同一位雇员可能有两个元组版本，只要它们的事务时间没有交集，那么它们就有可能拥有相同的有效时间，却拥有不同的属性值。在这种情况下，拥有较晚事务时间的元组版本将是对其他元组版本的一个**更正**。这样即使用户输入了不正确的有效时间，也可以通过这一方法进行更正。因此，为了便于日后查询以往的数据库状态而保存的不正确的数据库状态也是可用的，这种保存了记录的所有变化和更正的数据库有时候被称为**只追加数据库**（append only database）。

8.2.3　使用属性版本化在面向对象数据库中加入时间

前面讨论了实现时态数据库的**元组版本化方法**（tuple versioning approach）。在这种方法中，无论何时一个属性值发生变化，就需要创建一个包含所有属性的新的元组版本，即使这个新元组版本的所有其他属性与前一个元组版本完全相同。在支持**复杂结构化对象**的数据库系统，例如面向对象数据库（见第4章和第5章）或对象-关系系统（见第6章）中，可以选择另一种方法，这种方法称为**属性版本化**（attribute versioning）方法[①]。

在属性版本化方法中，使用单个的复杂对象来存储该对象的所有时态变化。每一个随时间变化的属性称为**时间变更属性**（time-varying attribute）。通过在某个属性上增加一个时态区间来为该属性赋予一个相应的版本值。时态区间根据应用需求的不同可以代表有效时间、事务时间或双时态。不随时间而变化的属性被称为**非时间变更属性**（nontime-varying attribute），它们与时态区间无关。为了说明这一点，考虑图8-10中的示例。在这个示例中，使用了在对象数据库中使用的ODL表示法（见第5章）来表示一个雇员对象，它的属性中已加入了有效时间的版本。这里，假设姓名与社会保险号都是非时间变更属性（即它们不随时间而变化），而工资、部门和直接管理者都是时间变更属性（即它们可能随时间而变化）。每个时间变更属性都用一个形如 <Valid_start_time, Valid_end_time, Value> 元组的列表来表示，并按有效开始时间排序。

```
class TEMPORAL_SALARY
{   attribute     Date                  Valid_start_time;
    attribute     Date                  Valid_end_time;
    attribute     float                 Salary;
};

class TEMPORAL_DEPT
{   attribute     Date                  Valid_start_time;
    attribute     Date                  Valid_end_time;
    attribute     DEPARTMENT_VT         Dept;
};

class TEMPORAL_SUPERVISOR
{   attribute     Date                  Valid_start_time;
    attribute     Date                  Valid_end_time;
    attribute     EMPLOYEE_VT           Supervisor;
};

class TEMPORAL_LIFESPAN
{   attribute     Date                  Valid_ start time;
    attribute     Date                  Valid end time;
};

class EMPLOYEE_VT
(   extent EMPLOYEES   )
{   attribute     list<TEMPORAL_LIFESPAN>          lifespan;
    attribute     string                           Name;
    attribute     string                           Ssn;
    attribute     list<TEMPORAL_SALARY>            Sal_history;
    attribute     list<TEMPORAL_DEPT>              Dept_history;
    attribute     list <TEMPORAL_SUPERVISOR>       Supervisor_history;
};
```

图8-10　使用属性版本化方法的有效时间对象类EMPLOYEE_VT可能的ODL模式

每当这个模型中的一个属性发生变化时，该属性的当前版本就被关闭，并且这个属性的一个新

① 属性版本化也可以在嵌套关系模型中使用（见第6章）。

属性版本就出现在该列表中。这种方式允许一些属性异步地变化。每个属性的当前值中Valid_end_time为*now*。当使用属性版本化方法时，有些对象会有一个或多个有效时间区间来表明其整个对象存在的有效时间，那么对于这种对象，包括一个与那个对象相关联的**寿命时态属性**（lifespan temporal attribute）是很有用的。因此，一个对象的逻辑删除就可以通过关闭其寿命时态属性而实现。为此，数据库中就应该相应的存在这样一个约束条件，即一个对象中任何一个属性的任何时间区间都应该是这个对象的寿命时态属性的子集。

对于双时态数据库，每个属性版本应该是一个包含5个成分的元组：

<Valid_start_time, Valid_end_time, Trans_start_time, Trans_end_time, Value>

对象寿命还将包括有效时间和事务时间两个维。因此，使用属性版本化的方法可以保证双时态数据库所有功能的可用性。与前面讨论的更新元组版本方法类似的机制也可以应用到更新属性版本中。

8.2.4　时态查询构造与 TSQL2 语言

到目前为止，我们已经讨论了如何使用时态构造扩展数据模型。本节将简要概述如何为时态查询扩展查询操作。我们将简要地讨论TSQL2语言。TSQL2语言扩展了SQL，使它可用于查询有效时间数据库、事务时间数据库以及双时态关系数据库。

在非时态关系数据库中，典型的选择条件包括属性条件，满足这些条件的元组从当前元组的集合中被选择出来。接下来，要被查询的属性由一个投影操作（见《数据库系统基础：初级篇》的第5章）指定。例如，要检索所有在部门编号为5，且工资大于30 000的雇员的名字，这一选择条件如下：

((Salary > 30000) AND (Dno = 5))

投影属性是Name。在时态数据库中，除了属性，在检索条件中可能也包括时间。一个**纯时间条件**（pure time condition）只包含时间，例如，选择在某一时间点T有效，或在某一时间区间[T_1，T_2]内有效的所有雇员元组版本。在这种情况下，指定的时间区间将与每个元组版本[T.Vst，T.Vet]的有效时间区间作比较，只有满足条件的元组才会被选择出来。在这些操作中，时间区间（即[T_1，T_2]）被认为与一个包含从时间点T_1到时间点T_2的时间点集合是等价的，因此也就可以使用标准的集合比较操作。也需要其他一些操作，如判定一个时间区间是否在另一个时间区间开始前结束的操作[①]。

以下是一些查询中较常用的操作：

[T.Vst, T.Vet] INCLUDES [T_1 , T_2]	等价于：$T_1 \geqslant T$.Vst AND $T_2 \leqslant T$.Vet
[T.Vst, T.Vet] INCLUDED_IN [T_1 , T_2]	等价于：$T_1 \leqslant T$.Vst AND $T_2 \geqslant T$.Vet
[T.Vst, T.Vet] OVERLAPS [T_1 , T_2]	等价于：（$T_1 \leqslant T$.Vet AND $T_2 \geqslant T$.Vst）[②]
[T.Vst, T.Vet] BEFORE [T_1 , T_2]	等价于：$T_1 \geqslant T$.Vet
[T.Vst, T.Vet] AFTER [T_1 , T_2]	等价于：$T_2 \leqslant T$.Vst
[T.Vst, T.Vet] MEETS_BEFORE [T_1 , T_2]	等价于：$T_1 = T$.Vet+ 1 [③]
[T.Vst, T.Vet] MEETS_AFTER [T_1 , T_2]	等价于：T_2 +1 = T.Vst

此外，也需要一些操作来操纵时间区间，例如计算两个时间区间的并或交。这些操作的结果形式可能并不是一个区间，而是一些**时态元素**（temporal element），即一个或多个不相交的时间区间的集合。那就是说，在同一个时态元素中，没有两个时间区间是直接相邻的。也就是说，对在同一个时态元素内的任何两个时间区间[T_1，T_2]和[T_3，T_4]来说，它们必须满足如下3个条件：

- [T_1，T_2] 与[T_3，T_4] 的交集为空。

---

① 一个用于比较时间区间的完整的操作集合被称为**Allen代数**。

② 如果两个时间区间的交集非空，则该操作返回值为真，这一操作也被称为INTERSECTS_WITH操作。

③ 这里，1指的是在一个指定时间粒度下的时间点，MEETS操作一般用于表示一个时间区间结束后另一个时间立刻开始的情况。

- 在一个已经给定的时间粒度下，T_3不是紧跟T_2的下一个时间点。
- 在一个已经给定的时间粒度下，T_1不是紧跟T_4的下一个时间点。

最后的两个条件是保证时态元素表示法的唯一性所必需的。如果这两个时间区间[T_1, T_2] 与[T_3, T_4]是相邻的，那么就可以把它们合并成一个单一的时间区间[T_1, T_4]，这称为时间区间的**联合**(coalescing)。联合操作也可以合并具有交集的时间区间。

为了说明如何使用纯时间条件，假定一个用户想选出2002年中任一时间点的所有有效雇员版本。那么，应用于如图8-8中关系的选择条件将是：

[*T*.Vst, *T*.Vet] OVERLAPS [2002-01-01 , 2002-12-31]

一般地，大多数时态选择应用于有效时间维中。对于双时态数据库而言，通常会将条件应用于那些事务结束时间为*uc*的当前正确的元组上。然而，如果需要对原有数据库状态进行查询，那么就需要在查询中追加一个AS_OF *T*子句，这表明查询将应用于数据库中那些仅在时间*T*上处于正确状态的有效时间元组。

除了纯时间条件外，其他一些选择还涉及**属性**和**时间条件**。例如，我们想在EMP_VT关系中检索所有2002年期间在5号部门工作过的雇员元组版本*T*。这种情况下，选择条件是：

[*T*.Vst, *T*.Vet] OVERLAPS [2002-01-01, 2002-12-31] AND(*T*.Dno = 5)

最后，概述一下TSQL2查询语言，它针对时态数据库构造扩展了SQL。TSQL2的主要思想是允许用户指定一个关系是非时态的(即标准SQL关系)或者是时态的。它使用可选子句AS扩展了CREATE TABLE语句，允许用户声明不同的时态选项，下面是一些可用的选项：

- AS VALID STATE　<GRANULARITY>
  （带有有效时间区间的有效时间关系）
- AS VALID EVENT　< GRANULARITY>
  （带有有效时间点的有效时间关系）
- AS TRANSACTION
  （带有事务时间区间的事务时间关系）
- AS VALID STATE　<GRANULARITY> AND TRANSACTION
  （双时态关系，有效时间区间）
- AS VALID EVENT　<GRANULARITY> AND TRANSACTION
  （双时态关系，有效时间点）

关键字STATE和EVENT用于指定时间区间或时间点是否与有效时间维有关。在TSQL2中，不是让用户实际地看到如何去实现一个时态表（如同我们在前面讨论过的那样），而是在TSQL2语言中加入了新的查询语言构造，以指定不同类型的时态选择、时态投影、时态聚集、粒度间的转换以及许多其他概念。(Snodgrass et al., 1995）一书描述了这种语言。

8.2.5　时间序列数据

时间序列数据在金融、销售以及与经济相关的应用中经常会用到。它们包含根据特定的预定义时间点序列而记录的数据值。因此，它们是一种特殊类型的**有效事件数据**，其事件的时间点可根据一个固定的日历而预先确定。考虑纽约证券交易所里一个公司每日股票收盘价的示例。这里的粒度为天，但是证券市场哪天开放大家都是知道的（非假日的工作日）。因此，可以共同指定一个计算过程，估算一个与特定时间序列相关的特定的**日历**。在时间序列上的一般查询包括在更高粒度间隔上的**时态聚集**（temporal aggregation），例如，从每日的信息中发现每周股票收盘价的平均值和最大值，或每月收盘价的最大值和最小值。

作为另一个示例，考虑某个拥有许多连锁店的公司的下属每个连锁店的日营业额。此时，典型

的时态聚集又将用于从每日销售信息中获取每周、每月或每年的销售额（使用sum聚集函数），或进行同一连锁店本月的销售额与上月的销售额的对比，等等。

由于时间序列数据的专用性，以及以前的DBMS对其缺乏支持等原因，目前通常采用专用的**时间序列管理系统**（time series management system）而不采用通用的DBMS来管理这类信息。在这种系统中，通常在一个文件中顺序地存储时间序列的值，并且用特殊的时间序列过程分析这些信息。使用这种方法的问题是，在这类系统中，不能使用诸如SQL之类的高层查询语言的全部功能。

最近，一些商业DBMS软件包正在提供它们在时间序列上的扩展功能，如Informix Universal Server 中的时间序列数据刀片（见第6章）。另外，TSQL2语言也以事件表的形式提供了对时间序列的某些支持。

8.3　空间和多媒体数据库

因为本节要讨论的两个主题涉及的面很广，所以只能对每个主题作简要的介绍。8.3.1节介绍空间数据库的一些概念，8.3.2节将简要地讨论多媒体数据库。

8.3.1　空间数据库概念介绍

空间数据库（spatial database）提供了描述多维空间对象的数据库概念。例如，存储二维空间对象描述的地图数据库，这些对象从国家、州到河流、城市、公路、海洋等。这些应用也被称为地理信息系统（GIS）[①]。地理信息系统还应用于环境管理、紧急事件处理和战争管理等。其他类型的空间数据库，比如用于管理气象信息的气象数据库，它是三维的，因为温度和其他气象信息可以与三维空间中的点发生联系。一般来说，空间数据库中存储的是可以描述具有空间特征的对象。对象之间的空间联系非常重要，在查询空间数据库的时候经常会用到它们。尽管在通常情况下，一个空间数据库对任意的n，都可以参照一个n维空间，但在本节讨论的示例中，将仅限于以二维空间为例加以说明。

空间数据库所需要的主要扩展是能够解释空间特征的数据模型。另外，为了提高性能还经常需要一些专门的索引和存储结构。这里首先讨论对于二维空间数据库所做的一些模型扩展。为了能够指定对象的空间特征，所需的基本扩展应包括一些二维空间的几何概念，例如点、线、线段、圆、多边形、弧等。另外，为了操作对象的空间特征，还需要一些空间操作，例如，计算两个对象间的距离，判别空间布尔条件，例如判定两个对象在空间中是否存在重叠。考虑一个在紧急事件管理应用中所使用的数据库，它需要对各种对象的空间位置进行描述。这些对象中的某一些总是具有静态的空间特征，例如街道、高速公路、消防水泵、警察局、消防中心和医院，还有一些对象则具有随时间变化的空间特征，例如警车、救护车、救火车。

下面的分类说明了空间查询的3种典型类型：

- **范围查询**（range query）：在给定的空间区域或在一个给定位置的特定距离范围内，查找某一特定类型的对象（例如，找出在Dallas城区的所有医院，或找出事故发生地点5公里范围内所有的救护车）。
- **最邻近查询**（nearest neighbor query）：查找一个与给定地点距离最近的特定类型的对象（例如，找出与一个特定地点最近的警车）。
- **空间连接或重叠**（spatial joins and overlays）：基于某些空间条件的两类对象的连接，例如进行对象间空间的交叉重叠或将其距离限制在一定的范围内（例如，找出一条主要高速公路沿线的所有城市或距一个湖两公里以内的所有住所）。

为了有效地回答上述类型和其他类型的空间查询请求，需要专门的空间索引技术，**R树**及其变

① 14.3节将详细讨论GIS的数据管理问题。

种便是比较著名的技术之一。R树将物理上相邻的空间对象分为一组，放在树结构索引的同一个叶结点上。由于一个叶结点仅能指向一定数量的对象，因而需要一种算法，用于将空间分为一些包含这些对象的矩形子空间（rectangular subspace）。划分空间的典型方法是最小化矩形区域，因为这样能使搜索的空间范围迅速缩小。诸如存在重叠空间的对象，这一问题可能会被许多不同的R树变种以不同的方法处理。R树中的内部结点和矩形相关，矩形区域覆盖了所有子树中的矩形。因此，R树可以很容易地回答下面一类查询：查找给定区域中的所有对象，可以通过对树的搜索限定在其矩形子空间与给定查询区域相交的子树上。

其他的空间存储结构包括**四分树**（quadtrees）及其变种，四分树将空间或其子空间分为大小相同的区域，并且再对每个子空间进行细分以识别各种对象的位置。最近，还出现了许多新的空间访问结构，这个领域仍然是一个活跃的研究领域。

8.3.2　多媒体数据库概念介绍[①]

多媒体数据库提供了允许用户存储和查询不同类型的多媒体信息的特性。多媒体信息包括图像（例如照片和绘图）、视频片断（例如电影、新闻报道及家庭录像）、音频片断（例如歌曲、电话录音及演讲）和文档（例如书籍及文章）等。需要的数据库查询类型包括定位包含某一感兴趣对象的多媒体数据源等。例如，某人想在一个视频数据库中，寻找有某一人物（比如Bill Clinton）出现的所有视频片断。他还可能想检索其中包含某些特定活动的视频片断，比如一场足球比赛中某一队员或球队射门得分的视频片断。

以上类型的检索都被称为**基于内容的检索**，因为正在检索的多媒体数据源都是基于其所含的特定对象和活动而进行检索的。因此，一个多媒体数据库必须采用一些模型使其可以基于内容来组织多媒体数据源，并为它们建立相应的索引。但识别多媒体数据源的内容是一项困难而耗时的工作。目前主要有两种方法，一种是基于**自动分析**以识别多媒体数据源内容中的某些数学特征。这一方法因数据源类型（包括图像、文本、音频或视频等）的不同所采用的技术也不同。第二种方法是在每个多媒体数据源中**手动识别**感兴趣的对象及其活动，并识别出为多媒体数据源建立索引所需的信息。这一方法可用于所有的多媒体数据源，但它需要一个手动的预处理阶段，在这个阶段中必须人工浏览每个多媒体数据源，手动地识别和分类它们所包含的对象和活动，并使它们可用来为这些多媒体数据源建立索引。

在本节后面的内容中，将按图像、视频、音频和文本数据源的顺序非常简要地讨论一下每种多媒体数据源的一些特征。

图像（image）一般以像素或单元值集合的原始形式存储，或者以压缩形式存储以节省空间。图像的形状描述信息描述了原始图像的几何形状，一般是具有一定长度和宽度的矩形**单元**。因此，每个图像可以表示为$m \times n$个网格单元，每个网格单元包含了一个描述本单元内容的像素值。在黑白图像中，像素可能仅仅是一个bit（位），在灰度图像或彩色图像中，一个像素是多个bit。因为存储一个图像可能需要大量空间，因此它们经常以压缩形式存储。如GIF、JPEG、MPEG等压缩标准使用各种数学变换，以减少存储单元的数量，同时仍保持图像的主要特征。可应用的数学变换形式包括：离散傅里叶变换（Discrete Fourier Transform，DFT）、离散余弦变换（Discrete Cosine Transform，DCT）和小波变换（wavelet transforms）。

为了识别一个图像中感兴趣的对象，一般使用同质谓词（homogeneity predicate）将图像分为若干同类段（homogeneous segments）。例如，在彩色图像中，彼此相邻并且像素值接近的单元分组成一个段。同类性谓词为这些元素自动分组定义条件。因此分段和压缩可以识别一个图像的主要特征。

通常，图像数据库的查询是寻找数据库中与一个给定图像相似的图像。给定的图像可能是一个

---

① 14.2节详细讨论了多媒体数据库和多媒体信息管理。

独立的包含一个感兴趣图形模式的段，则该查询就是要找到包含同样图形模式的另一个或另一些图像。对于这类搜索，目前有两种主要的技术。第一种方法是使用**距离函数**，将给定的图像与存储在数据库中的图像以及它们的段进行比较。如果返回的距离值很小，就认为两个图像匹配的可能性就高。为了限制搜索的区域，可以创建索引将距离尺度接近到限定搜索范围以内的图像放在一个组中。第二种方法称为**变换法**（transformation approach），它通过将一个图像作少量变换后与另一个图像进行匹配。通过这一方式来测定图像间的相似度。变换包括旋转、平移和缩放。尽管第二种方法更具普遍性，但它的实现也更耗时、更困难。

视频数据源（video source）一般地表示为一个帧序列，每一帧都是一个静止的图像。但独立的帧并不能识别出视频对象和活动。一个视频流被分为多个**视频段**，每个视频段由包含相同对象或活动的一系列连续有序的帧组成，每个段由它的开始帧和结束帧来识别。可以使用在每个视频段中标识的对象和活动来为段创建索引。现在已经有一种称为帧段树的索引技术用来创建视频索引。视频索引包括对象：例如人、房屋、汽车等，以及活动：例如一个人在演说或两个人在交谈等。视频也常常用一些标准如MPEG来压缩。

文本/文档源（text/document source）基本上是指一些文章、书籍或杂志的全部文本。这些数据源一般通过标识文本中出现的关键字及其出现的频率等信息来创建索引。然而，文本中的空白在索引建立过程中被消除。因为当试图为一个文档集合建立索引时可能会出现很多的关键字，所以要求开发一种技术以减少关键字的数量，只保留那些与文档集合最相关的关键字。目前，一种称为奇异值分解（singular value decompositions，SVD）的技术是基于矩阵转换（matrix transformation）技术实现的，它可以被用于筛选关键字这一目的。还有一种被称为嵌入向量树（telescoping vector trees）（或TV树）的索引技术，可用于分组相似文档。

音频源（audio source）包括存储的录音信息，如讲话、演讲甚至在法律执行过程中获得的电话记录或谈话记录。在这里，可以使用离散变换来标识某个人声音的主要特征，以便于进行基于相似性的索引和检索。音频特征一般包括：音量（loudness）、强度（intensity）、音调（pitch）、清晰度（clarity）等。

8.4　演绎数据库介绍

8.4.1　演绎数据库概述

在演绎数据库系统中，通常通过一种**说明性语言**（declarative language）来指定规则，使用说明性语言可以指定要完成什么，而不是如何完成它。系统中的**推理引擎**（inference engine）（或**演绎机制**（deduction mechanism））可以通过解释这些规则从数据库中演绎出新的事实。演绎数据库的数据模型与关系数据模型紧密相关，特别是域关系演算形式化方法（见《数据库系统基础：初级篇》的6.6节）。这个模型还与**逻辑编程**领域及**Prolog**语言有关。演绎数据库工作基于逻辑并使用Prolog作为起点。Prolog语言的一个变种称为**Datalog**，这种语言用来声明与一个已存在的关系集合相联结的规则，并且这些规则在Datalog语言中被作为它们自己的文字对待。尽管Datalog的语言结构与Prolog相似，但是其操作语法，即一个Datalog程序是如何被执行的，仍然是不同的。

演绎数据库使用两种主要的规范说明类型：即事实与规则。**事实**用类似于指定关系的方式来指定，但事实不必包括属性名。关系中的元组描述了现实世界的一些事实，由属性名来确定这些事实的含义。在演绎数据库中，元组中属性值的含义仅仅由属性在元组中的位置确定。**规则**在某种程度上类似于关系视图，它们指定了实际上没有物理存储的虚拟关系，但是它们可以通过应用基于规则规范说明的推理机制从事实中形成。规则和视图之间的主要区别是：规则可以包含递归，因此可以生成虚拟关系，而这些关系不能以基本关系视图的形式定义。

Prolog程序的估算是基于称为反向链接（backward chaining）的技术，它自顶向下地进行目标估

算。在使用Datalog的演绎数据库系统中，重点是处理关系数据库中存储的大量数据。因此，估算技术通常被设计为类似于自底向上的估算形式。Prolog的局限是，规则和事实的说明顺序在估算中有比较大的影响；并且，规则中的文字顺序（在8.4.3节中定义）对估算的结果也有重要影响。Datalog程序的执行技术试图解决这些问题。

8.4.2　Prolog/Datalog 表示法

Porlog/Datalog使用的表示法提供带有唯一名称的谓词。**谓词**（predicate）的名字隐含了该谓词的含义，而且每个谓词都有一组固定数量的**变元**（argument）。如果这些变元都是常量值，那么这个谓词就简单地指出某一事实为真。另一方面，如果谓词的变元中有变量，那么这个谓词被认为是一个查询，或者是规则或约束的一部分。在本章中，我们采用Prolog中的约定：谓词中的所有**常量值**或者是数值型数据或者是字符串型数据；这些常量值用小写字母开始的标识符（或名字）表示，而**变量名**常用大写字母开始的标识符表示。

考虑图8-11所示的例子，它基于《数据库系统基础：初级篇》图5-6所示的关系数据库，但是它的形式更加简单。这个数据库中有3个谓词：*supervise*、*superior*和*subordinate*。谓词supervise通过一个事实集合来定义，每个事实有两个变元：第一个是管理者的姓名，第二个是该管理者的直接下属的姓名。这些事实对应于存储在数据库中的实际数据，可以认为它们构成了有两个属性的SUPERVISE关系中的元组集合，它的关系模式是：

SUPERVISE(Supervisor，Supervisee)

因此，supervise(*X*,*Y*)表明了*X*管理*Y*这样一个事实。注意，Prolog表示法中省略了属性名。属性名由谓词中每个变元的位置来表示：第一个变元表示管理者，第二个变元表示直接下属。

Facts

SUPERVISE(franklin, john).
SUPERVISE(franklin, ramesh).
SUPERVISE(franklin, joyce).
SUPERVISE(jennifer, alicia).
SUPERVISE(jennifer, ahmad).
SUPERVISE(james, franklin).
SUPERVISE(james, jennifer).
. . .

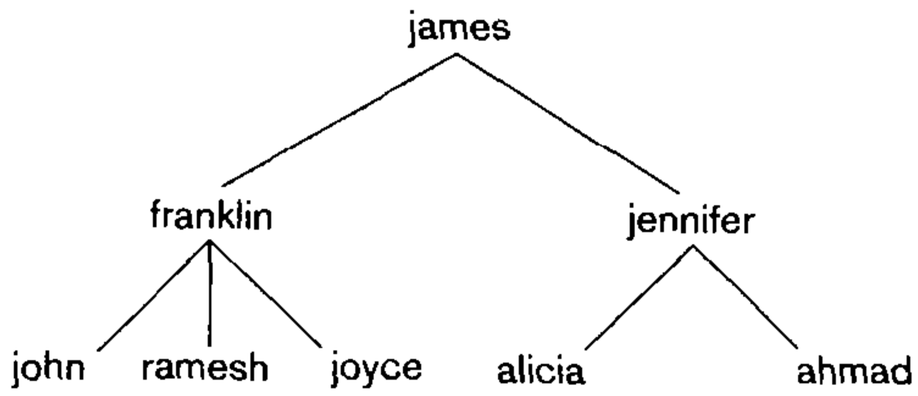
Rules

SUPERIOR(*X*, *Y*) :– SUPERVISE(*X*, *Y*).
SUPERIOR(*X*, *Y*) :– SUPERVISE(*X*, *Z*), SUPERIOR(*Z*, *Y*).
SUBORDINATE(*X*, *Y*) :– SUPERIOR(*Y*, *X*).

Queries

SUPERIOR(james, *Y*)?
SUPERIOR(james, joyce)?

(a) Prolog表示法



(b) 监管（supervisory）树

图　8-11

另外两个谓词名由规则来定义。演绎数据库的主要贡献是能够指定递归规则，并提供了一个基于指定规则推理信息的框架。每条规则的形式为head:-body，其中，:-读作当且仅当。在:-符号的左边是规则的**单一谓词**，称为规则的**头**（head）或者**左手边**（left-hand side，LHS），或者称为规则的**结论**（conclusion）；在:-右边有**一个或多个谓词**，称为规则的**体**（body）或者**右手边**（right-hand side，RHS），或者称为规则的**前提**（premise）。用常量作为变元的谓词被称为**基**（ground），也称为**实例化谓词**（instantiated predicate）。尽管谓词也可以用常量作为变元，但规则中的谓词变元通常包含若干个变量符号。规则表明，如果一个特定的常量值赋给或者**绑定**到规则体（RHS谓词）中的变量，可以使得所有RHS谓词都为**真**，那么把同样的常量值赋给变量，也能使得规则头（LHS谓词）为真。

因此，每条规则提供了一个生成新事实的方法，新事实是规则体的实例。这些新事实基于已经存在的事实，它们对应于规则体中谓词的实例（或者绑定）。需要注意，通过列出规则体中的多个谓词，我们就隐式地向这些谓词应用了**逻辑与**（logical and）操作符。因此，RHS谓词之间的逗号可以读作与。

考虑图8-11中谓词superior的定义，它的第一个变元是一个雇员的姓名，第二个变元是第一个雇员直接或间接管理的一个下属雇员的姓名。间接下属是指比直接下属更下层的下属，因此superior(*X*,*Y*)代表这样的事实：*X*是*Y*的主管，*X*可能是*Y*的直接主管也可能是间接主管。可以使用两条规则一起指定新谓词的含义。图8-11中的第一条规则表明，对于每个*X*和*Y*的值，如果规则体：supervise(*X*,*Y*)为真，那么规则头：superior(*X*,*Y*)也为真，因为*Y*是*X*的直接下属（比X低一级）。用这条规则可以从定义谓词supervise的事实中生成所有直接主管/下属联系。第二条递归规则表明，如果supervise(*X*,*Z*)与superior(*Z*,*Y*)都为真，那么superior(*X*,*Y*)也为真。这就是**递归规则**的一个示例，出现在RHS规则body中的一个谓词与LHS规则head中的谓词相同。通常，规则体定义了多个前提，如果这些前提都为真，就可以推断出规则head中的结论也为真。需要注意，如果有两条或多条规则具有相同的规则head（LHS谓词），则说明如果这些规则body中有一个为真，则规则head的谓词就为真（也就是说，它们能够被实例化）；因此，它等价于**逻辑或**操作。例如，如果有两条规则：X:-Y和X:-Z，它们等价于一条规则X:-Y or Z。然而后一种形式在演绎数据库中并没有使用，因为它不是Horn子句规则的标准形式，我们将在8.4.4节中讨论这个问题。

Prolog系统包含很多系统可以直接解释的**内置**（built-in）谓词。这些谓词一般包括相等比较操作符=（*X*,*Y*），如果*X*和*Y*是相等的，它就返回true；也可以使用标准的中缀表示法写为*X*=*Y*[①]。其他算术比较操作符，例如<、<=、>、>=等等，也都可以作为二元谓词对待。而算术功能操作符，例如＋、－、＊、/等，也可以作为Prolog谓词的变元。与Prolog相比，Datalog（以它的基本形式）不允许使用算术操作符作为变元的功能；实际上这也是Prolog与Datalog之间的一个主要区别。但是，在后来的Datalog扩展中已经建议要包括这些功能。

查询一般会包含一些变量变元的谓词符号，其含义（或结果）是演绎出所有不同的常量组合，从而当把这些值**绑定**（赋给）给变量时，使得谓词为真。例如，图8-11中的第一个查询要求查找由*james*管理的所有各级下属的名字。另一种查询，它只把常量符号作为变元，这种查询会返回true或false结果，返回的结果是true还是false取决于所提供的变元是否能够使用事实和规则演绎出来。例如，由于可以被演绎出superior(james, joyce)，所以图8-11中的第二个查询返回true。

8.4.3　Datalog 表示法

在Datalog中，像其他基于逻辑的语言一样，程序由称为**原子公式**的基本对象构建。通常是通过描述原子公式的语法，以及说明如何组合这些原子公式而形成程序来定义基于逻辑语言的语法。在Datalog中，原子公式是形如$p(a_1,a_2,\cdots,a_n)$的**文字**，其中p是谓词名，n是谓词p的变元个数。不同的谓词符号可以有不同数量的变元，谓词p的变元个数n有时也被称为p的**元数**（arity）或**度**（degree）。变元可以是常量值也可以是变量名。如前所述，我们仍然使用如下约定：常量值或者是数值，或者以小写字母开始的字符串，而变量名总是以大写字母开始的字符串。

Datalog中有大量的**内置谓词**，这些内置谓词也可以用来构造原子公式。内置谓词主要有两种类型：一种是有序域上的二元比较谓词：＜（小于）、＜＝（小于等于）、＞（大于）、＞＝（大于等于）；另一种是有序或无序域上的比较谓词：＝（相等）和/＝（不等）。这些谓词可以作为与其他谓词有相同功能语法的二元谓词，例如，写作less(*X*,3)，或者使用惯用的中缀表达式X＜3来指定。需要注意，因为这些谓词的域可能是无限的，所以在规则定义中应当小心使用。例如，如果单独使用谓词greater(*X*,3)，就会生成满足此谓词的X值的一个无限集合（所有大于3的整数）。

---

① Prolog系统一般有多个不同的相等谓词，它们都有不同的解释。

文字（literal）可以是上面定义的原子公式，称为**肯定文字**（positive literal），也可以是加前缀**not**的原子公式。后者是否定原子公式，称为**否定文字**（negative literal）。Datalog程序可以看作是**谓词演算**（predicate calculu）公式的子集，它在某种程度上类似于域关系演算公式（见《数据库系统基础：初级篇》的6.7节）。然而，在Datalog中，在表达这些公式之前，需要先把这些公司转换成**子句形式**，并且只有按严格的子句形式给出的公式才能在Datalog中使用，这种严格的子句称为Horn子句[①]。

8.4.4　子句形式与 Horn 子句

根据《数据库系统基础：初级篇》的6.6节，关系演算中的公式是--个包括称为原子的谓词（基于关系名）的条件。另外，公式中可以有量词，即全称量词（对所有的）和存在量词（对存在的）。在子句形式中，一个公式必须被转换为有如下特征的另一个公式：

- 公式中的所有变量必须是全称量词。因此，没有必要显式地标出全称量词（对所有）；删除量词时，隐式地由全称量词来限定公式中的所有变量。
- 在子句形式中，公式由一些子句组成，其中每个**子句**由只使用OR逻辑连接符连接的一些文字组成。因此，每个子句都是文字的析取（disjunction）。
- 子句本身只通过AND逻辑符连接起来，以形成一个公式。因此，公式的子句形式是子句的合取（conjunction）。

由此得知，任何公式都能转换为子句形式。为此，我们主要对单个子句的形式感兴趣，其中每个子句都是文字的析取。前面讲过，文字可以是肯定文字也可以是否定文字。考虑以下形式的子句：

$$\text{NOT}(P_1)\ \text{OR}\ \text{NOT}(P_2)\ \text{OR}\cdots\text{OR}\ \text{NOT}(P_n)\ \text{OR}\ Q_1\ \text{OR}\ Q_2\ \text{OR}\cdots\text{OR}\ Q_m \tag{1}$$

这个子句中有n个肯定文字和m个否定文字。这样的一个子句可以转换为如下的等价逻辑公式：

$$P_1\ \text{AND}\ P_2\ \text{AND}\cdots\text{AND}\ P_n \Rightarrow Q_1\ \text{OR}\ Q_2\ \text{OR}\cdots\text{OR}\ Q_m \tag{2}$$

其中=>是**蕴含**（imply）符号。公式(1)和(2)是等价的，说明它们的真值总是相同的。这是因为，如果所有文字P_i(i=1,2,…，n)为真，当且仅当至少有一个Q_i为真的时候，公式(2)才为真。对于公式(1)，如果所有文字P_i(i=1,2,…，n)为真，则其否定（negation）则全为假。在这种情况下，当且仅当至少有一个Q_i为真的时候，公式(1)才为真。在Datalog中，规则被表示成一种严格形式的子句，称为**Horn子句**，在Horn子句中，一个子句最多只能包含一个肯定文字。因此，Horn子句的形式或者是：

$$\text{NOT}\ (P_1)\ \text{OR}\ \text{NOT}(P_2)\ \text{OR}\cdots\text{OR}\ \text{NOT}(P_n)\ \text{OR}\ Q \tag{3}$$

或者是：

$$\text{NOT}\ (P_1)\ \text{OR}\ \text{NOT}(P_2)\ \text{OR}\cdots\text{OR}\ \text{NOT}(P_n) \tag{4}$$

公式(3)中的Horn子句可以转换为子句：

$$P_1\ \text{AND}\ P_2\ \text{AND}\cdots\text{AND}\ P_n \Rightarrow \text{Q} \tag{5}$$

在Datalog中则将(5)写为如下规则：

$$Q :\!- P_1, P_2, \cdots, P_n. \tag{6}$$

公式(4)中的Horn子句可转换为：

$$P_1\ \text{AND}\ P_2\ \text{AND}\cdots\text{AND}\ P_n \Rightarrow \tag{7}$$

在Datalog中(7)可以写为：

$$P_1, P_2, \cdots, P_n. \tag{8}$$

因此，像公式(6)那样，Datalog规则是一个Horn子句，并且它的含义基于公式(5)，也就表示：如

① 根据数学家Alfred Horn的名字命名。

果对变量变元实施一个特殊的绑定，使谓词P_1 AND P_2 AND$\cdots P_n$都为真，那么就能推断出Q也为真。Datalog表达式(8)可以看作是一个完整性约束，当且仅当所有谓词都为真时才能满足这个查询要求。

通常，Datalog中的查询包括两个部分：

- Datalog程序，它是规则的有限集合。
- 文字$P(X_1, X_2, \cdots, X_n)$，其中每个X_i是一个变量或者常量。

Prolog或者Datalog系统中有一个内部**推理引擎**，用来处理和计算这些查询结果。对于一个查询，Prolog推理引擎一般一次返回一个结果（也就是说，查询中变量取值的一个集合），而且必须提示返回其他结果。与此相反，Datalog一次可以返回一个结果集合。

8.4.5　规则解释

解释规则的理论含义主要有两种方法：证明-理论（proof-theoretic）和模型-理论（model-theoretic）。在实际的系统中，系统内的推理机制可以定义精确的解释，可能与这两种理论解释并不一致。推理机制是一个计算的过程，因此提供了规则含义的计算解释。在这节中，我们首先讨论这两种理论解释，然后简要讨论作为定义规则含义方法之一的推理机制。

在规则的**证明-理论**解释中，我们把事实和规则看作是真语句，或者是**公理**（axiom）。**基本公理**（ground axiom）不包含变量。事实是取值为真的基础公理。规则被称为**演绎公理**（deductive axiom），因为它们可以用来演绎新的事实。演绎公理可以用来构造从现存事实推出新事实的证明。例如，图8-12展示了如何从图8-11给出的规则和事实来证明事实SUPERIOR(james, ahmand)。证明-理论解释给出了计算Datalog查询结果的过程或计算方法。证明某一事实（理论）是否为真的过程称为理论证明。

1. SUPERIOR(X, Y) :- SUPERVISE(X, Y).　（规则1）
2. SUPERIOR(X, Y) :- SUPERVISE(X, Z), SUPERIOR(Z, Y).　（规则2）

3. SUPERVISE(jennifer, ahmad).　（基础公理，给定）
4. SUPERVISE(james, jennifer).　（基础公理，给定）
5. SUPERIOR(jennifer, ahmad).　（应用规则1于3上）
6. SUPERIOR(james, ahmad).　（应用规则2于4和5上）

图8-12　证明一个新事实

解释的第二种类型被称为**模型-理论**解释。在这里，给定一个有限或无限的常量值域[①]，把每一种值的可能组合作为变元赋给谓词，然后必须确定该谓词是真还是假。在一般情况下，只需指定使谓词为真的变元的组合，并陈述其他所有组合都使谓词为假就可以了。如果对每个谓词都这样做，则称为谓词集合**解释**。例如，考虑图8-13中对谓词SUPERVISE和SUPERIOR的解释。这个解释把一个真值（真或假）赋给这两个谓词的每个可能的变元值（来自一个有限域）的组合。

如果一个特定规则集合中的所有规则在某种解释下总为真，则称这一解释是这个规则集合的一个**模型**；也就是说，对于赋给规则中变量的任何值，当使用这个解释代替赋给规则体中谓词的真值时，规则头就为真。因此，只要把一组特定值代入（绑定）到规则的变量中，如果规则体中所有谓词在这个解释下都为真，则规则头中的谓词必然也为真。在图8-13中展示的解释是所示两条规则的一个模型，因为它永远不会违反这两条规则。需要注意，如果一个特定的常量绑定到变量使得规则体中的所有谓词都为真，但是却使得规则头中的谓词为假，那么就违反了该规则。例如，如果在某种解释下，SUPERVISE(a,b)和SUPERIOR(b,c)都为真，但是SUPERIOR(a,c)不为真，那么这个解释就不可能是下面递归规则的一个模型：

SUPERIOR(X, Y) :- SUPERVISE(X, Z), SUPERIOR(Z, Y).

① 最常选择的域是称为郝勃郎全域（Herbrand Universe）的有限域。

规则
SUPERIOR (X, Y) :- SUPERVISE (X,Y) .
SUPERIOR (X,Y) :- SUPERVISE (X,Z) , SUPERIOR (Z,Y) .

解释
已知事实:
SUPERVISE (franklin, john) 为**true**.
SUPERVISE (franklin, ramesh) 为**true**.
SUPERVISE (franklin, joyce) 为**true**.
SUPERVISE (jennifer, alicia) 为**true**.
SUPERVISE (jennifer, ahmad) 为**true**.
SUPERVISE (james, franklin) 为**true**.
SUPERVISE (james, jennifer) 为**true**.
SUPERVISE (X, Y) 为**false**，对于所有其他可能的 (*X*,*Y*) 组合.

导出事实:
SUPERIOR (franklin, john) 为**true**.
SUPERIOR (franklin, ramesh) 为**true**.
SUPERIOR (franklin, joyce) 为**true**.
SUPERIOR (jennifer, alicia) 为**true**.
SUPERIOR (jennifer, ahmad) 为**true**.
SUPERIOR (james, franklin) 为**true**.
SUPERIOR (james, jennifer) 为**true**.
SUPERIOR (james, john) 为**true**.
SUPERIOR (james, ramesh) 为**true**.
SUPERIOR (james, joyce) 为**true**.
SUPERIOR (james, alicia) 为**true**.
SUPERIOR (james, ahmad) 为**true**.
SUPERVISE (*X*, *Y*) 为**false**，对于所有其他可能的 (*X*,*Y*) 组合.

图8-13　一个最小模型的解释

在模型-理论方法中，规则的含义是通过为这些规则提供一个模型而建立的。如果我们无法将任何事实从真改为假而仍然能够得到这些规则的模型，则这个模型称为该规则集的一个**最小模型**。例如，考虑图8-13中的解释，假设SUPERVISE谓词通过一个已知事实集合来定义，而SUPERIOR谓词定义为规则的一个解释（模型）。假设我们在真谓词中增加谓词SUPERIOR(james, bob)。这个模型仍然是规则的一个模型，但不是最小模型，因为把SUPERIOR(james, bob)的真值从真改为假，则这个解析仍然提供给我们规则的一个模型。图8-13所示的模型是由SUPERVISE谓词所定义的事实集合的最小模型。

一般来说，与模型理论解释中给定事实集合相对应的最小模型，应该与同样的一组初始基础公理和演绎公理中由证明-理论解释生成的事实相同。然而，这只是对只有简单结构的规则才是真的。一旦允许在规则中出现否定形式的谓词，那么这两种解释之间的对应关系就不存在了。实际上，如果允许有否定形式的谓词，则给定的事实集合可能存在多个最小模型。

解释规则含义的第三种方法，是定义被系统用来从规则演绎事实的推理机制。这种推理机制针对规则的含义定义了**计算解释**。Prolog逻辑程序设计语言就是使用其推理机制来定义Prolog程序中规则与事实的含义。不是所有的Prolog程序都对应于证明-理论解释或者模型-理论解释；是否对应这取决于程序中的规则类型。然而，对于许多简单Prolog程序来说，Prolog推理机制推断的事实或者对应于证明-理论解释，或者对应于模型-理论解释下的最小模型。

8.4.6　Datalog 程序及其安全性

在实际的Datalog程序中，有两种定义谓词真值的主要方法。**事实定义谓词**（fact-defined predicate）或**关系**，可以通过列出所有使谓词为真的值（或元组）的组合来定义。这些谓词对应于内容存储在

数据库系统中的基关系。图8-14给出了事实定义谓词EMPLOYEE、MALE、FEMALE、DEPARTMENT、SUPERVISE、PROJECT和WORKS_ON，这些谓词对应于图5-6所示关系数据库的一部分。**规则定义谓词**（rule-defined predicate）或**视图**，是由一个或多个Datalog规则头（LHS）定义的，它对应于其内容可以由推理引擎推出的虚关系。图8-15给出了一些规则定义谓词。

```
EMPIOYEE(john).
EMPLOYEE(franklin).
EMPLOYEE(alicia).
EMPLOYEE(jennifer).
EMPLOYEE(ramesh).
EMPLOYEE(joyce).
EMPLOYEE(ahmad).
EMPLOYEE(james).

SALARY(john, 30000).
SALARY(franklin, 40000).
SALARY(alicia, 25000).
SALARY(jennifer, 43000).
SALARY(ramesh, 38000).
SALARY(joyce, 25000).
SALARY(ahmad, 25000).
SALARY(james, 55000).

DEPARTMENT(john, research).
DEPARTMENT(franklin, research).
DEPARTMENT(alicia, administration).
DEPARTMENT(jennifer, administration).
DEPARTMENT(ramesh, research).
DEPARTMENT(joyce, research).
DEPARTMENT(ahmad, administration).
DEPARTMENT(james, headquarters).

SUPERVISE(franklln, john).
SUPERVISE(franklln, ramesh)
SUPERVISE(frankin , joyce).
SUPERVISE(jennifer, alicia).
SUPERVISE(jennifer, ahmad).
SUPERVISE(james, franklin).
SUPERVISE(james, jennifer).

MALE(john).
MALE(franklin).
MALE(ramesh).
MALE(ahmad).
MALE(james).

FEMALE(alicia).
FEMALE(jennifer).
FEMALE(joyce).

PROJECT(productx).
PROJECT(producty).
PROJECT(productz).
PROJECT(computerization).
PROJECT(reorganization).
PROJECT(newbenefits).

WORKS_ON(john, productx, 32).
WORKS_ON(john, producty, 8).
WORKS_ON(ramesh, productz, 40).
WORKS_ON(joyce, productx, 20).
WORKS_ON(joyce, producty, 20).
WORKS_ON(franklin, producty, 10).
WORKS_ON(franklin, productz, 10).
WORKS_ON(franklin, computerization, 10).
WORKS_ON(franklin, reorganization, 10).
WORKS_ON(alicia, newbenefits, 30).
WORKS_ON(alicia, computerization, 10).
WORKS_ON(ahmad, computerization, 35).
WORKS_ON(ahmad, newbenefits, 5).
WORKS_ON(jennifer, newbenefits, 20).
WORKS_ON(jennifer, reorganization, 15).
WORKS_ON(james, reorganization, 10).
```

图8-14　对应于《数据库系统基础：初级篇》的图5-6中部分数据库的事实谓词

```
SUPERIOR(X, Y) :- SUPERVISE(X, Y).
SUPERIOR(X, Y) :- SUPERVISE(X, Z), SUPERIOR(Z, Y).

SUBORDINATE(X, Y) :- SUPERIOR(Y, X).

SUPERVISOR(X) :- EMPLOYEE(X), SUPERVISE(X, Y).

OVER_40K_EMP(X) :- EMPLOYEE(X), SALARY(X, Y), Y >= 40000.
UNDER_40K_SUPERVISOR(X) :- SUPERVISOR(X), NOT(OVER_40_K_EMP(X)).
MAIN_PRODUCTX_EMP(X) :- EMPLOYEE(X), WORKS_ON(X, productx, Y), Y >= 20.
PRESIDENT(X) :- EMPLOYEE(X), NOT(SUPERVISE(Y, X) ).
```

图8-15　规则定义谓词

如果一个程序或者一条规则生成一个有限的事实集合，则称它为**安全的**。判断规则集合是否安全的一般理论问题尚未解决。然而，现在可以判断严格形式规则的安全性。例如，图8-16所示的规则就是安全的。通常，如果规则中的一个变量可以在一个无限域上取值，并且所取的值并不是限制在某个有限关系中，就会出现能够生成无限多个事实的不安全规则。例如，考虑以下规则：

BIG_SALARY(Y) :– Y>60 000

这里，如果Y的取值范围为所有整数，我们会得到无限个结果。但是假如把该规则改为：

BIG_SALARY(Y) :– EMPLOYEE(X), Salary(X, Y), Y>60 000

可以看出，在第二条规则中，结果是有限的，因为Y的取值现在限定在数据库中一些员工的工资范围内，而它是一个有限值集合。也可以重写这条规则，如下：

BIG_SALARY(Y) :– Y>60 000, EMPLOYEE(X), Salary(X, Y)

在这种情况下，规则理论上仍然是安全的。然而，在Prolog或者其他任何使用自顶向下、深度优先推理机制的系统中，由于我们首先要为Y搜索一个值，然后检查它是否是一个雇员的工资，因此这条规则会导致一个无限循环。结果产生无限多个Y值，尽管从某个时间点起再也找不到使得RHS谓词为真的Y值。因为Datalog定义并不依赖于特定的推理机制，所以它认为这两种规则都是安全的。尽管如此，一般建议还是要使用最安全的形式编写这一规则，在编写时把限制变量可能绑定的谓词放在前面。考虑下面一个不安全规则的示例：

HAS_SOMETHING(X, Y) :– EMPLOYEE(X)

这里，由于变量Y只出现在规则头中，因此不受有限值集合的限制，所以可能再次产生无限个Y值。为了更正式地定义安全规则，可以使用受限变量（limited variable）的概念。如果满足以下条件之一，我们就称变量X在规则中是**受限的**：(1)变量出现在规则体的常规（非内置）谓词中；(2)变量出现在规则体中形如$X=c$或$c=X$或（$c_1<=X$ and $X<=c_2$）的谓词中，其中c、c_1和c_2都是常量值；(3)变量出现在规则体中形如$X=Y$或者$Y=X$的谓词中，其中Y也是一个受限变量。如果一条规则中的所有变量都是受限的，就称该规则是**安全的**。

8.4.7　关系操作的使用

以Datalog规则的形式指定关系代数的许多操作是很直观的，因为Datalog通过在数据库关系（事实谓词）上应用这些操作来定义结果。这意味着在Datalog中，可以很容易地指定关系查询和视图。Datalog提供的附加功能包括规范递归查询和基于递归查询的视图。在本节中，将展示如何将一些标准的关系操作指定为Datalog规则。在我们的示例中，使用基本关系（事实定义谓词）REL_ONE、REL_TWO和REL_THREE，它们的模式如图8-16所示。在Datalog中，不需要像图8-16那样指定属性名；重要的是要指定每个谓词的元数（度）。在实际系统中，每个属性的域（数据类型）对于各种操作也是很重要，如UNION、INTERSECTION和JOIN等操作。在此假设属性类型对于各种不同的操作是相容的，如《数据库系统基础：初级篇》的第5章所述。

图8-16说明了一些基本关系操作。需要注意，如果Datalog模型基于关系模型，并因此假设谓词（事实关系和查询结果）指定了元组集合，那么同一谓词中的重复元组会被自动消除。对此可能为真也可能为假，取决于Datalog推理引擎。但是，在Prolog中这一点是不成立的，因此图8-16中任何包含消除重复元组的规则对于Prolog都是不成立的。例如，如果我们需要指定含有消除重复元组的UNION操作的Prolog规则，必须把这些规则重写为：

UNION_ONE_TWO(X, Y, Z) :– REL_ONE(X, Y, Z).
UNION_ONE_TWO(X, Y, Z) :– REL_TWO(X, Y, Z), NOT(REL_ONE(X, Y, Z)).

然而，如果自动消除重复元组，则图8-16所示的规则应该适用于Datalog。类似地，在这种情况

下，图8-16所示的PROJECT操作的规则也应该适用于Datalog，但是对于Prolog则是不正确的，因为后者会出现重复元组。

```
REL_ONE(A, B, C).
REL_TWO(D, E, F).
REL_THREE(G, H, I, J).

SELECT_ONE_A_EQ_C(X, Y, Z) :- REL_ONE(C, Y, Z).
SELECT_ONE_B_LESS_5(X, Y, Z) :- REL_ONE(X, Y, Z), Y< 5.
SELECT_ONE_A_EQ_C_AND_B_LESS_5(X, Y, Z) :- REL_ONE(C, Y, Z), Y<5

SELECT_ONE_A_EQ_C_OR_B_LESS_5(X, Y, Z) :- REL_ONE(C, Y, Z).
SELECT_ONE_A_EQ_C_OR_B_LESS_5(X, Y, Z) :- REL_ONE(X, Y, Z), Y<5.

PROJECT_THREE_ON_G_H(W, X) :- REL_THREE(W, X, Y, Z).

UNION_ONE_TWO(X, Y, Z) :- REL_ONE(X, Y, Z).
UNION_ONE_TWO(X, Y, Z) :- REL_TWO(X, Y, Z).

INTERSECT_ONE_TWO(X, Y, Z) :- REL_ONE(X, Y, Z), REL_TWO(X, Y, Z).

DIFFERENCE_TWO_ONE(X, Y, Z) :- REL_TWO(X, Y, Z) NOT(REL_ONE(X, Y, Z).

CART PROD _ONE_THREE(T, U, V, W, X, Y, Z) :-
   REL_ONE(T, U, V), REL_THREE(W, X, Y, Z).

NATURAL_JOIN_ONE_THREE_C_EQ_G(U, V, W, X, Y, Z) :-
   REL_ONE(U, V, W), REL_THREE(W, X, Y, Z).
```

图8-16　用来说明关系操作的谓词

8.4.8　非递归 Datalog 查询的估算

为了把Datalog作为演绎数据库系统使用，基于关系数据库查询处理概念来定义推理机制是比较合适的。固有的策略是从基本关系出发自底向上地进行估算；操作的顺序比较灵活，但受限于查询优化。本节我们将讨论一种基于关系操作（可应用到**非递归**Datalog查询中）的推理机制。这里使用图8-14和图8-15所示的事实和规则基进行讨论。

如果一个查询只包含事实定义谓词，则推理就变成在事实间搜索一个查询结果。例如，如下所示的查询：

DEPARTMENT(*X*, Research)?

它是要找出在Research部门工作的所有雇员的姓名。在关系代数中这个查询可表示为：

$$\pi_{\$1}\left(\sigma_{\$2 = \text{“Research”}}(\text{DEPARTMENT})\right)$$

可以通过搜索事实定义谓词department(*X*, *Y*)来回答这个查询。这个查询包括在基本关系上的关系操作SELECT和PROJECT，这可以用《数据库系统基础：初级篇》中第15章讨论的数据库查询处理与优化技术来处理。

当一个查询包括规则定义谓词时，推理机制必须基于规则定义计算结果。如果一个查询是非递归的，并且包含谓词p，这里p是规则p :- p_1, p_2,…, p_n的头，那么策略首先计算对应于p_1, p_2,…, p_n的关系，然后计算对应于p的关系。在**谓词依赖图**（predicate dependency graph）中，跟踪演绎数据库谓词之间的依赖关系是很有用的。图8-17给出了图8-14和图8-15所示事实和规则谓词的依赖图。依赖图中，每个谓词表示为一个**结点**。无论何时在一个规则的规则体（RHS）中指定谓词A，并且该规

则的头（LHS）是谓词*B*，则称*B*依赖于*A*，从*A*到*B*画一条有向边。这表明，为了计算谓词*B*（规则头）的事实，首先必须计算规则体中的所有谓词*A*的事实。如果依赖图中没有环，则称规则集合是**非递归的**（nonrecursive）。如果至少有一个环，则称规则集合是**递归的**（recursive）。在图8-17中，有一个递归定义的谓词——即SUPERIOR，它有一条指向自己的递归边。另外，因为谓词subordinate依赖于SUPERIOR，所以计算它的结果时也需要递归。

只包含非递归谓词的查询称为**非递归查询**（nonrecursive query）。本节我们只讨论非递归查询的推理机制。图8-17中，任何不包含谓词subordinate或superior的查询都是非递归的。在谓词依赖图中，对应事实定义谓词的结点都没有任何进入的边，因为事实定义谓词的所有事实都存储在数据库关系中。事实定义谓词的内容可以通过直接检索相应数据库关系中的元组来计算。

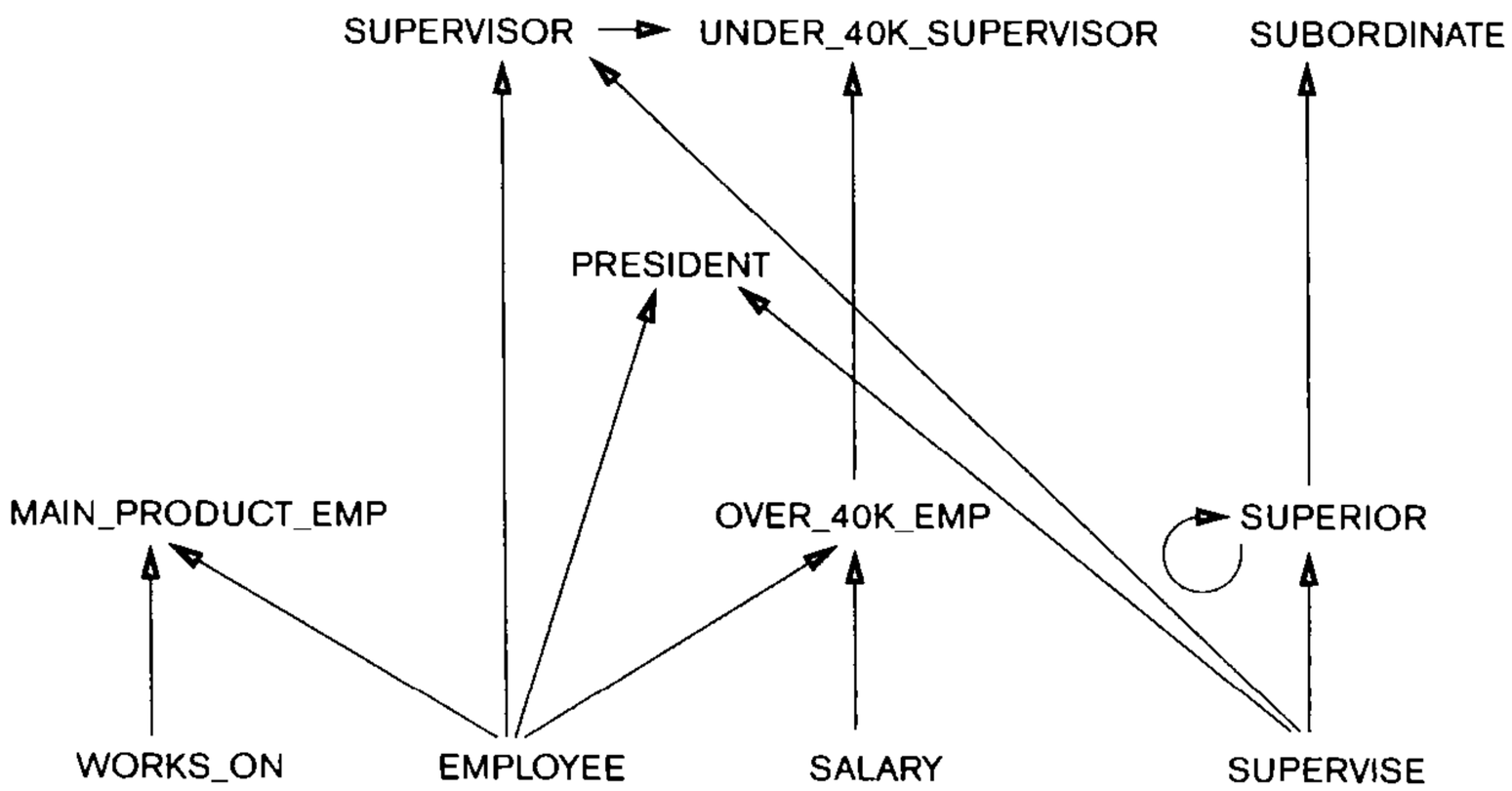


图8-17　图8-14和图8-15的谓词依赖图

推理机制的主要功能是计算对应于查询谓词的事实。这可以通过生成一个包括SELECT、PROJECT、JOIN、UNION和SET DIFFERENCE等关系操作符的**关系表达式**来完成（与处理安全问题的适当操作一起使用），执行后提供查询结果。接着可以利用关系数据库管理系统的内部查询处理和优化操作来执行这个查询。无论何时推理机制要计算对应于一个非递归规则定义谓词*p*的事实集合，它首先要找到以*p*作为头部的所有规则。这主要思想是计算每一个规则的事实集合，然后在结果上应用UNION操作，因为UNION操作对应于逻辑OR操作。依赖图表明了每个*p*所依赖的所有谓词*q*，而且由于已经假定谓词是非递归的，所以可以确定这些谓词*q*之间的偏序。在计算*p*的事实集合之前，我们首先基于*p*所依赖的所有谓词*q*的偏序关系来计算这些谓词*q*的事实集合。例如，如果一个查询包含谓词UNDER_40K_SUPERIOR，我们必须首先计算supervisor和OVER_40K_EMP。因为后两个谓词仅依赖于事实定义谓词EMPLOYEE、SALARY和SUPERVISE，它们可以直接从存储的数据库关系中计算得出。

本节概述了演绎数据库。其他一些资料可以从本书的Web网站上找到，其中包括了第3版中的完整的第25章，它包括对递归查询处理算法的讨论。

小结

在这一章中，我们介绍了高级应用所需要的数据库概念的公共特性：包括主动数据库、时态数据库、空间数据库和多媒体数据库。再次提醒读者注意的是，上述每个主题都是很广泛的，都可以

单独作为一本完整的教科书。

首先，我们介绍了主动数据库这一主题，它提供了指定主动规则的附加功能。还介绍了主动数据的事件-条件-动作模型（ECA模型）。规则可以被发生的事件自动触发（例如数据库更新事件），如果某个条件为真时，它们会触发在规则声明中定义的指定动作。很多商业软件包都以触发器的形式提供了主动数据库的功能。本章还讨论了指定规则的多种不同的选项，如行级和语句级、前触发和后触发、立即执行与延迟执行等。给出了Oracle商业系统中行级触发器的示例和STARBURST实验系统中语句级规则的示例，讨论了SQL-99标准中的触发器语法。最后，还简要地讨论了一些设计问题和主动数据库的一些可能应用。

接下来，介绍了时态数据库的一些概念，它允许数据库系统存储它的历史变化，允许用户查询数据库当前和过去两种状态。讨论了如何在有效时间和事务时间维上表示和区别时间的问题。还讨论了如何使用元组版本化方法在关系模型中实现有效时间、事务时间以及双时态关系，并用示例说明了如何实现更新、插入和删除操作。还说明了如何利用属性版本化方法使用复杂对象来实现时态数据库。最后，列举了时态关系数据库的查询操作，并给出了TSQL2语言的简要介绍。

随后，我们转向空间和多媒体数据库。空间数据库提供了记录具有空间特征的对象的数据库概念。这类数据库需要特殊的数据模型来表示对象的空间特征，并进行空间比较操作和操纵。多媒体数据库提供了允许用户存储和查询不同类型多媒体信息的特性，多媒体信息包括图像（例如图片或绘图）、视频片断（例如影片、新闻报道或家庭录像）、音频片断（例如歌曲、电话录音或演讲）以及文档（例如书籍或文章）。我们还简述了各种类型的多媒体数据源和如何为这些多媒体数据源建立索引等内容。更多有关多媒体数据库和GIS空间数据管理的细节，读者可参考第14章“新兴数据库技术和应用”。

最后，我们介绍了演绎数据库和Datalog。

复习题

8.1　行级主动规则和语句级主动规则的区别是什么？

8.2　主动规则条件中，立即考虑、延迟考虑和分离考虑之间的区别是什么？

8.3　主动规则条件中，立即执行、延迟执行和独立执行之间的区别是什么？

8.4　简要说明设计一套主动规则时可能遇到的一致性问题和终止问题。

8.5　给出一些主动数据库的应用。

8.6　讨论如何在时态数据库中表示时间，并对不同的时间维进行比较。

8.7　有效时间关系、事务时间关系和双时态关系之间有什么区别？

8.8　描述如何在有效时间关系上实现插入、删除和更新命令。

8.9　描述如何在双时态关系上实现插入、删除和更新命令。

8.10　描述如何在事务时间关系上实现插入、删除和更新命令。

8.11　元组版本化和属性版本化之间的主要不同是什么？

8.12　如何区别空间数据库与常规数据库？

8.13　多媒体数据源都有哪些不同类型？

8.14　如何为多媒体数据源建立索引以实现基于内容的检索？

练习题

8.15　考虑《数据库系统基础：初级篇》中图5-6所示的COMPANY数据库。使用Oracle触发器语法，为如下要求编写主动规则：

a. 每当一个雇员的项目分配发生变化时，检查该雇员每周在此项目上所花费的总时间是否小于30小时或大于40小时；如果是的话，则通知该雇员的直接管理者。

b. 每当删除一个雇员时，也删除与该雇员有关的DEPENDENT中的元组和PROJECT中的元组。但是如果该雇员正在管理一个部门或其他雇员，那么把该部门的Mgr_ssn属性设置为NULL，并将被该雇员管理的雇员的直接管理者的Super_ssn属性设置为NULL。

8.16　使用STARBURST主动规则的语法再次完成练习题8.15中的要求。

8.17　考虑如图8-18所示的关系模式。编写满足以下要求的主动规则：对于每个销售人员来说，其Sales_person关系中的Sum_commissions属性值等于SALES关系中Commission属性值的总和。规则还应该能够检查Sum_commissions是否超过了100 000。如果是，则调用一个名为Notify_manager（S_id）的过程。分别编写STARBURST语句级规则和Oracle行级规则。

SALES

S_id	V_id	Commission

SALES_PERSON

Salesperson_id	Name	Title	Phone	Sum_commissions

图8-18　sales和salesperson数据库模式

8.18　考虑《数据库系统基础：初级篇》中图4-10的UNIVERSITY EER模式。编写你认为与这个应用有关的一些主动规则，以便实现一些常用的完整性约束。

8.19　讨论图8-9中所示的每个元组创建的更新操作，哪个是用于滞后更新的？哪个是用于预期更新的？

8.20　说明如果按顺序实施下面的更新操作，图8-9所示的双时态关系EMP_BT中的内容会发生怎样的变化？对每个更新陈述它们是预期更新还是滞后更新？

a. 2004-03-10，17:30:00，雇员Narayan的工资被更新为40 000，并在2004-03-01生效。

b. 2003-07-30，08:31:00，雇员Smith的工资被更正为31 000（不是如图8-9所示的30 000），并在2003-06-01生效。

c. 2004-03-18，08:31:00，更改数据库以表明Narayan将要离开公司（即逻辑删除），并在2004-03-31生效。

d. 2004-04-20，14:07:33，更改数据库以表明雇用一位名叫Johnson的新雇员，其元组为<'Johnson', '334455667', 1, NULL>，并在2004-04-20生效。

e. 2004-04-28，12:54:02，更改数据库以表明Wong将要离开公司（即逻辑删除），并在2004-06-01生效。

f. 2004-05-05，13:07:33，更改数据库以表明重新雇用Brown，部门和管理者不变，但工资变更为35000，并在2004-05-01生效。

8.21　说明如果顺序应用习题8.20中的更新操作，图8-8所示的有效时间关系EMP_VT的内容将做如何变化？

8.22　在图8-3所示的数据库中加入如下事实：

SUPERVISE(ahmad, bob), SUPERVISE(franklin, gwen).

首先修改图8-1b中的监管树以反映这个变化。然后，修改图8-4中的图，以展示对查询SUPERIOR(james, *Y*)自顶向下的估算。

8.23　考虑对于关系PARENT(*X*, *Y*)的如下事实集合，这里*Y*是*X*的父亲。

PARENT(a, aa), PARENT(a, ab), PARENT(aa, aaa), PARENT(aa, aab),
PARENT(aaa, aaaa), PARENT(aaa, aaab).

考虑如下定义*Y*是*X*的祖先的规则：

r_1: ANCESTOR(*X*, *Y*) :– PARENT(*X*, *Y*)
r_2: ANCESTOR(*X*, *Y*) :– PARENT(*X*, *Z*), ANCESTOR(*Z*, *Y*)

a. 展示如何执行Datalog查询ANCESTOR(aa, *X*)?使用固有策略，说明每一步中你的工作。

b. 通过每次只计算祖先关系中的变更和使用规则2来展示相同的查询。

（这个问题来自于（Bancilhon & Ramakrishnan，1986））

8.24　考虑含有如下规则的演绎数据库：

ANCESTOR(*X*, *Y*) :– FATHER(*X*, *Y*)
ANCESTOR(*X*, *Y*) :– FATHER(*X*, *Z*), ANCESTOR(*Z*, *Y*)

需要注意：FATHER(*X*, *Y*) 表示*Y*是*X*的父亲；ANCESTOR(*X*, *Y*) 表示*Y*是*X*的祖先。

考虑如下的基本事实：

FATHER(Harry, Issac), FATHER(Issac, John), FATHER(John, Kurt).

a. 使用给定事实构造上述规则的一个模型–理论解释。

b. 考虑一个包含上述关系FATHER(*X*, *Y*)、另一个关系BROTHER(*X*, *Y*)，以及第三个关系BIRTH(*X*, *B*)的数据库，其中*B*是*X*的生日。阐述计算第一个堂兄弟（cousin）的规则，使得他们的父亲必须是兄弟。

c. 给出一个完整的基于事实和基于规则文字的Datalog程序，用以计算如下关系：堂兄弟对的列表，其中第一个人出生在1960年之后，第二个人出生在1970年之后。可以使用*greater than*作为一个内置谓词。（注意：必须也表示出brother、birth和person的样本事实）。

8.25　考虑如下规则：

REACHABLE(*X*, *Y*) :– FLIGHT(*X*, *Y*)
REACHABLE(*X*, *Y*) :– FLIGHT(*X*, *Z*), REACHABLE(*Z*, *Y*)

其中，REACHABLE(*X*, *Y*)表示城市*Y*可以从城市*X*到达，FLIGHT(*X*, *Y*)表示从城市*X*到城市*Y*有航班通行。

a. 构造描述下述内容的事实谓词：

i. Los Angles、New York、Chicago、Atlanta、Frankfurt、Paris、Singapore、Sydney 都是城市。

ii. 存在下列航班：LA至NY、NY至Atlanta、Atlanta至Frankfurt、Frankfurt至Atlanta、Frankfurt至Singapore以及Singapore至Sydney。（注意：可以自动地假设不存在反向的航班）

b. 以上给出的数据是否是循环的？如果是，是在什么意义上的循环？

c. 对上述事实和规则建立一个模型–理论解释（即类似于图9-3中的解释）。

d. 考虑查询：

REACHABLE(Atlanta, Sydney)?

使用自然和半自然（naive and semenaive）方法，这个查询将如何执行？列出它执行的步骤序列。

e. 考虑如下规则定义谓词：

ROUND-TRIP-REACHABLE(*X*, *Y*) :–
　　REACHABLE(*X*, *Y*), REACHABLE(*Y*, *X*)
DURATION(*X*, *Y*, *Z*)

为上述谓词画出谓词依赖图（注意：DURATION(*X*, *Y*, *Z*)表示可以乘飞机经过*Z*小时从*X*到达*Y*）

f. 考虑如下查询：哪些城市可以从Atlanta出发，经12小时抵达？给出其datalog表示。假定内置谓词，如greater-than(*X*, *Y*)。它可以直接转化为关系代数表达式吗？为什么可以或者为什么不可以？

g. 考虑谓词population(*X*, *Y*),其中*Y*是城市*X*的人口数。考虑如下查询：列出所有可能的谓词对(*X*, *Y*)的绑定，其中*Y*是一个从人口超过100万的城市*X*有两架航班可以抵达的城市。在Datalog中展示这个查询。用关系代数术语画出相应的查询树。

参考文献

（Zaniolo et al.，1997）一书由若干部分组成，每个部分都描述了一种高级数据库的概念，例如主动数据库、时态数据库和空间/文本/多媒体数据库。（Widom & Ceri,1996）以及（Ceri & Fraternali，1997）着重介绍了主动数据库的概念和系统。（Snodgrass et al.，1995）描述了TSQL2语言和相应的数据模型。（Khoshafian & Baker，1996）、（Faloustsos，1996）和（Subrahmanian，1998）描述了多媒体数据库的概念。（Tansel et al.，1992）汇集了与时态数据库相关的内容。

（Widom & Finkelstein，1990）描述了STARBURST规则。有关主动数据库的早期工作包括HiPAC项目，在（Chakravarthy et al.，1989）和（Chakravarthy，1990）两本书中有所讨论。（Jensen et al.，1994）给出了时态数据库的专业词典。（Snodgrass，1987）着重介绍了TQuel，一种早期的时态查询语言。

在（Navathe & Ahmed，1989）一书中定义了时态规范化（temporal normalization）的概念。（Paton，1999）、（Paton & Diaz，1999）对主动数据库进行了全面的调查。（Chakravarthy et al.，1994）描述了SENTINEL和基于对象的主动系统。（Lee et al.，1998）讨论了时间序列的管理问题。

（Gallaire et al.，1984）概述了逻辑和数据库方法的早期发展。（Reiter，1984）提供了关系数据库理论的重构，而（Levesque，1984）则从逻辑学角度论述了不完整知识。（Gallaire & Minker，1978）是关于这个主题早期的书籍。更为细致的对逻辑和数据库的论述出现在（Ullman，1989,第二卷）中，（Ullman，1988，第一卷）中也有相关章节。（Ceri, Gottlob，& Tanca，1990）一书给出了有关逻辑和数据库全面而简明的处理。（Das，1992）是一本全面的关于演绎数据库和逻辑程序设计的书。（Maier & Warren，1988）描述了Datalog的早期历史。（Clocksin & Mellish，1994）是一本很好的Prolog语言参考书。

（Aho & Ullman，1979）一书中提出了一个早期的处理递归查询的算法，它使用了最小固定点运算符。（Bancilhon & Ramakrishnan，1986）一书给出了一个关于递归查询程序处理方法的很好且详细的描述，并给出了自然（naïve）和半自然（seminaive）方法的示例。关于演绎数据库和递归查询的优秀文章包括（Warren，1992）、（Ramarkrishnant & Ullman，1993）。（Bancilhon，1985）一书给出了基于关系代数的半自然（seminaive）方法的完整论述。

其他的递归查询处理方法包括（Vieille，1986）一书中的递归查询/子查询理论，它是一个自底向下的解释策略，而（Henschen-Naqvi，1984）一书中给出的是自顶向下的编译迭代策略。（Balbin & Rao，1987）一书讨论了对于多谓词的一种有别于半自然方法的扩展。

关于魔集（magic set）最早的论文是（Bancilhon et al.，1986）。（Beeri & Ramakrishnan，1987）对其进行了扩展。（Mumick et al.，1990）展示了魔集在非递归嵌入式SQL查询中的应用。在（Vieille，1986, 1987）一书中描述了可以不重写规则就能优化它们的方法。（Kifer & Lozinskii，1986）一书提出了一种不同的技术。（Bry，1990）一书讨论了如何使得自顶向下和自底向上方法相容。（Whang & Navathe，1992）一书描述了一种扩展的分离的范式技术来处理关系代数表达式中的递归，以便在一个关系DBMS上提供一个专家系统的接口。

（Chang，1981）一书描述了一个把演绎规则和关系数据库组合起来的早期系统。（Chimenti et al.，1990）一书描述了LDL系统原型。（Krishnamurthy & Naqvi，1989）在LDL中引入了选择表示法。（Zaniolo，1988）一书讨论了LDL系统中的语言问题。（Ramakrishnan et al.，1992）提供了CORAL语言的概述，它的实施在（Ramakrishnan et al.，1993）一书中有所描述。CORAL语言支持面向对象特性的扩展，被称为CORAL++，在（Srivastava et al.，1993）一书中有所描述。（Ullman，1985）一书提供了（Morris et al.，1987）一书中描述的NAIL！系统的基础。（Phipps et al.，1991）阐述了GLUE-NAIL！演绎数据库系统。

（Zaniolo，1990）一书回顾了演绎数据库的理论背景及其应用的重要性。（Nicolas，1997）一书给出了很好的演绎数据库的发展史，这些发展最终导致了演绎面向对象数据库系统（DOODs）的产生。（Falcone et al.，1997）对DOOD的前景进行了调查。VALIDITY系统的参考文献包括（Friesen et al.，1995）、（Vieille，1997）以及（Dietrich et al.，1999）。

第9章

分布式数据库和客户-服务器体系结构

在这一章中我们将转而讨论分布式数据库（distributed database，DDB）和分布式数据库管理系统（distributed database management system，DDBMS），以及如何将客户-服务器体系结构作为一个平台，用来开发数据库应用。DDB技术是作为两种技术的混合体出现的：数据库技术以及网络与数据通讯技术。后者在有线和无线技术领域已经取得了巨大的进展——从卫星、手机通信、城域网（Metropolitan Area Network，MAN）到诸如以太网、TCP/IP和异步传输模式（Asynchronous Transfer Mode，ATM）等协议的标准化，以及Internet的爆发性发展。然而早期的数据库趋于集中化，导致在20世纪70年代和80年代早期产生了单片式大型数据库，在80年代后期，数据库的发展趋势更多地转向了分散化和处理的自治性。随着操作系统领域的分布式处理和分布式计算的发展，数据库研究团体在有关数据分布、分布式查询和事务处理、分布式数据库元数据管理，以及其他一些主题的研究方面做了大量的工作，并且开发出许多研究原型。然而，还没有哪种DDBMS商业产品能够完整地完全实现DDB研究中提出的功能和技术。大部分主要的厂商把他们的精力从开发一个纯的DDBMS产品重新定位为开发基于客户-服务器体系结构的系统，或者开发访问分布式异构数据源的技术。

但是，当组织机构对他们的在地理上分布系统中的数据库、应用以及用户的信息资源进行集成（在逻辑层面上）的同时，对于处理（在系统层面上）的分散性也非常感兴趣。伴随着通讯技术的进展，人们现在已经普遍认可面向应用开发的客户-服务器体系结构，而这可能会引发许多DDB问题。

本章将讨论分布式数据库和客户-服务器体系结构[①]，数据库技术的发展与通讯和网络技术的发展有着密切的联系。后者的细节问题不属于本书讨论的范畴，读者可以查阅有关数据通讯和网络方面的系列教材（见本章末尾列出的参考文献）。

9.1节将介绍分布式数据库管理和相关的概念。分布式数据库设计的细节问题，如设计数据的分片和将数据在可能复制的多个站点上进行部署的问题，将在9.2节中讨论。9.3节将介绍不同类型的分布式数据库系统，包括联邦式数据库系统和多数据库系统，并且重点将介绍异构性问题和联邦式数据库系统对自治性的要求问题，这在未来的几年中将占主导地位。9.4节和9.5节将分别介绍分布式数据库查询和事务处理技术。9.6节将讨论客户-服务器体系结构概念如何与分布式数据库关联起来。9.7节将详细阐述有关客户-服务器体系结构的未来问题。9.8节将讨论Oracle RDBMS分布式数据库的特性。

如果只想简要了解一下分布式数据库这个主题，可以只阅读9.1节、9.3节以及9.6节的内容。

9.1 分布式数据库概念

分布式数据库为数据库管理领域带来了分布式计算的许多好处。一个**分布式计算系统**（distributed computing system）由大量的与计算机网络互连的处理单元组成，这些处理单元不必都是同构的，它们通过计算机网络相互连接并协作完成某些指派的任务。分布式计算系统通常的目标是：将大型的、难以处理的问题分解成一些较小的块，然后以一种协调的方式有效地加以解决。这种方法在经济方面的可行性基于以下两个原因：(1)可以将更多的计算机能力投入到解决一项复杂的任务

① 读者可复习一下《数据库系统基础：初级篇》的2.5节中对客户-服务器体系结构的介绍。

中；(2)每一个自治的处理单元可以被独立地管理，也可以开发它们自身的应用。

我们可以把**分布式数据库（DDB）**定义为一个分布于计算机网络上的、逻辑上相关的若干数据库的集合，将**分布式数据库管理系统（DDBMS）**定义为一个管理分布式数据库的软件系统，它使得“分布”对用户来说是透明的[①]。

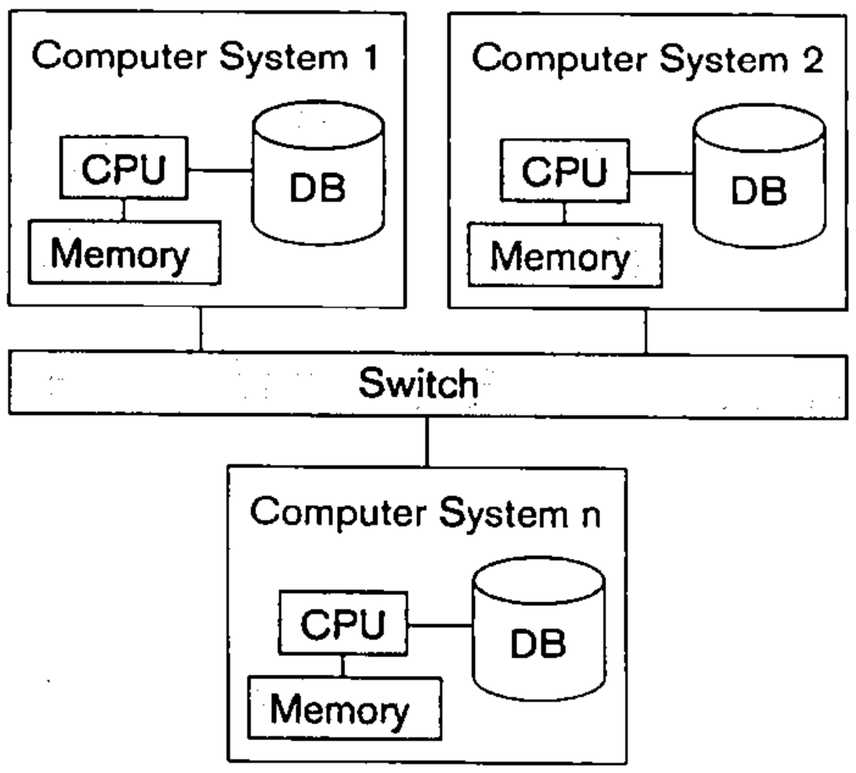
存储在网络上的多个不同结点中的文件的汇集，以及通过超链接来维护它们之间的内部联系，这种方式已经成为互联网上使用Web页面文件的一种常见的组织形式。数据库管理的一般功能，包括统一查询处理和事务处理，还不适用于这种情况。不过，技术正在朝着这样的方向发展：分布式World Wide Web（WWW）数据库在不久的将来将成为现实。我们将在第10章讨论基于Web页面的数据库存取问题。在先前给出的定义中，没有一个数据库可以称作DDB。

9.1.1　并行与分布式技术

下面我们转向并行系统的体系结构，常见的多处理器系统体系结构有两种主要的类型：

- **共享内存（shared memory）（紧密耦合）体系结构**：多个处理器共享辅存（磁盘）存储器并且也共享主存。
- **共享磁盘（shared disk）（松散耦合）体系结构**：多个处理器共享辅存（磁盘）存储器，但是每个处理器都有自己的主存。

这两种类型的体系结构使处理器间的通讯不需要在网络上花费交换信息的开销[②]。采用以上结构类型来开发的数据库管理系统被称作**并行数据库管理系统**（parallel database management system），而不是DDBMS，因为它们使用了并行处理器技术。另一种多处理器结构类型称为**无共享结构**（shared nothing architecture）。在这种结构中，每一个处理器都有自己的主存和辅存（磁盘）存储器，不存在公共的存储器，并且处理器通过高速互连网络（总线或交换机）实现通讯。虽然无共享结构类似于分布式数据库计算环境，但在操作模式上仍存在着很大的不同。在无共享的多处理器系统中，存在结点的对称性和同构性；在分布式数据库环境中则不是这样，因为在分布式数据库环境中每个结点的硬件和操作系统的异构是非常常见的。无共享的体系结构也被看作是并行数据库的一种环境。图9-1比较了这些不同的体系结构。

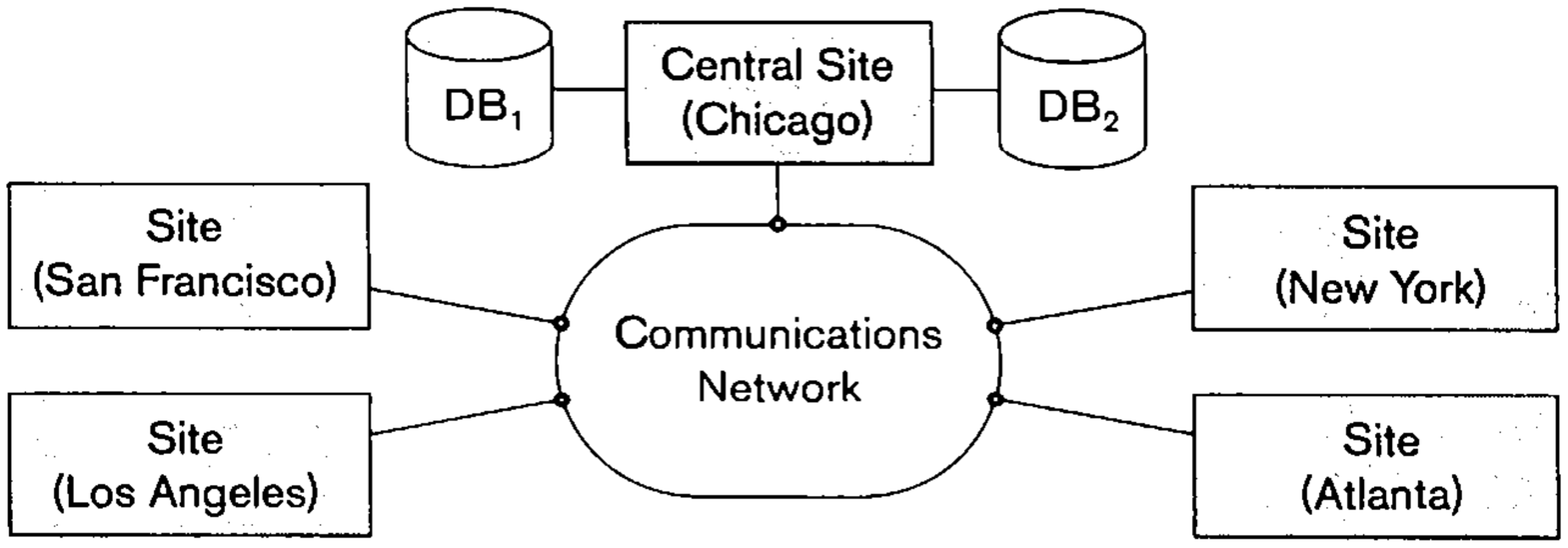


(a) 无共享的体系结构

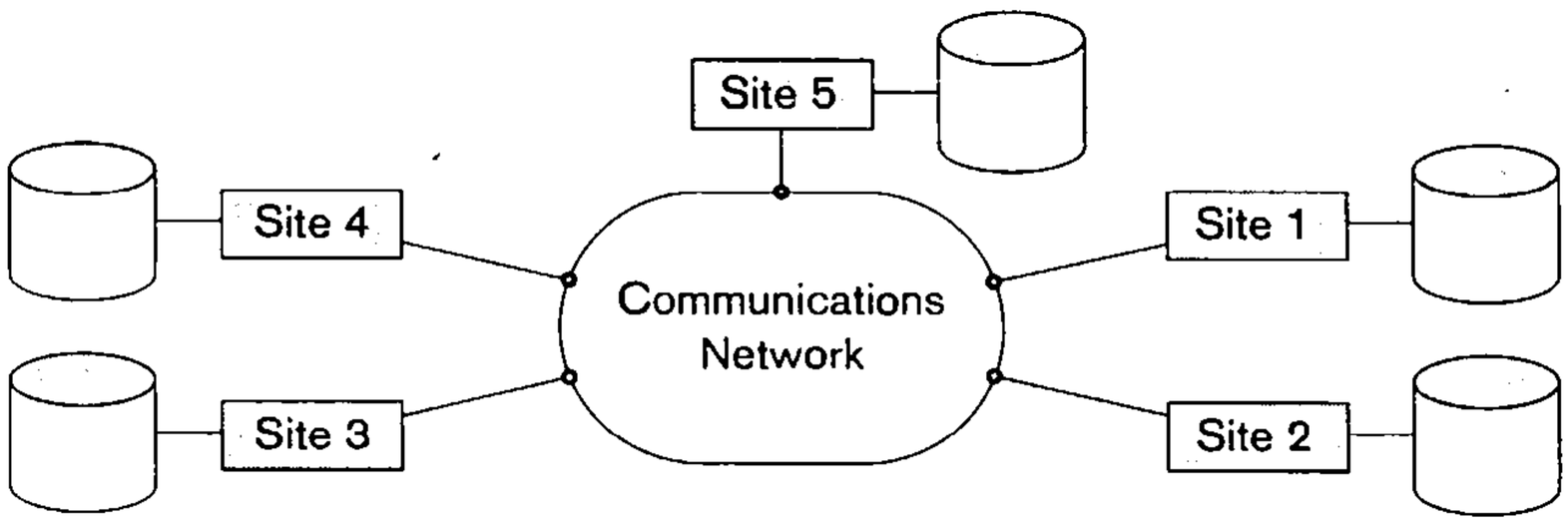
图9-1　一些不同的数据库系统体系结构

① 本节中的这个定义和一些讨论是基于（Ozsu & Valduriez，1999）一书的研究成果。

② 如果主存和二级存储器都共享的话，该结构也称作**全共享结构**（shared everything architecture）。



(b) 在某个站点上具有集中式数据库的网络结构



(c) 真正的分布式数据库体系结构

图9-1　（续）

9.1.2　分布式数据库的优势

提出分布式数据库管理有多种原因，从组织上的分散性和处理的经济性，到更高的自治性等，这里重点描述其中的一些优点：

(1) 不同透明度层次的分布式数据管理

一个理想化的DBMS在隐藏诸如每个文件（表或关系）在系统中的物理存储位置之类的细节问题上应当是具有**分布透明性**（distribution transparent）的。考虑《数据库系统基础：初级篇》中图5-5所示的company数据库。表EMPLOYEE、PROJECT和WORKS_ON可以被水平分片（即分割成行的集合，这将在9.2节中讨论），并且与图9-2所示的可能的复制一起存储。以下是几种可能的透明性类型：

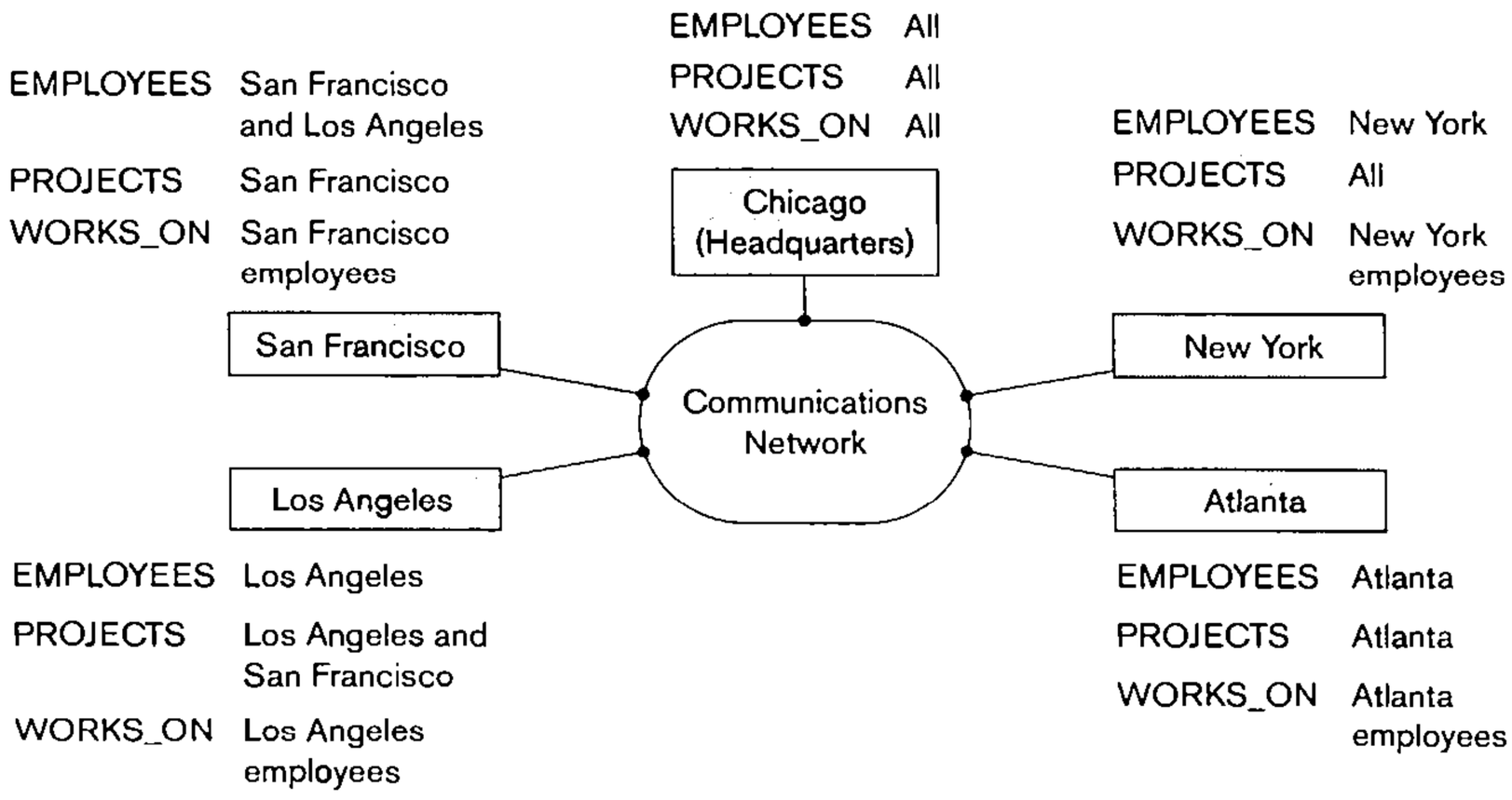


图9-2　分布式数据库中的数据分布和复制

- **分布透明性或网络透明性**：这是指用户能够从网络的操作细节中解脱出来，它可以分为位置透明性和命名透明性。**位置透明性**（location transparency）是指用于执行任务的命令对于数据的位置和发出命令的系统的位置来说是独立的。**命名透明性**（naming transparency）意味着对象一旦被命名了，就可以在没有附加说明的情况下无二异性地存取该命名对象。
- **复制透明性**（replication transparency）：如图9-2中所示，为了获得更好的可用性、更高的性能和可靠性，数据的副本会存储在多个网络站点上。复制透明性使用户不知道有副本的存在。
- **分片透明性**：有两种可能的分片类型。**水平分片**（horizontal fragmentation）将关系分割成元组（行）的集合，**垂直分片**（vertical fragmentation）将关系分割成子关系，每个子关系被定义成原来关系的列的子集。用户的一个全局查询必须转换成几个片段查询。分片透明性使得用户不必知道分片和片段的存在。

(2) 增加了可靠性和可用性

这是对分布式数据库而言最为普遍的两种潜在优势。**可靠性**（reliability）被广泛定义为系统在某个时间点正常运行（没有故障）的可能性，而**可用性**（availability）是指一个时间段内系统连续可用的可能性。当数据和DBMS软件分布在几个站点时，一个站点可能发生故障而其他的站点仍然可以继续操作，只是存储在故障站点的数据和软件不能访问而已，这就提高了整个系统的可靠性和可用性。还可以通过在多于一个的站点上灵活地复制数据和软件来实现进一步的改进。在一个集中式的系统中，单个站点的故障会使得整个系统对所有用户都变为不可用的。在分布式数据库中，一部分数据可能不能获得，但是用户仍然可以存取数据库中的其他部分。

(3) 提高了性能

分布式DBMS通过将数据存储在最靠近它且频繁使用的地方的方法来对数据库进行分片。**数据本地化**（data localization）降低了对CPU和I/O服务的争用，同时减少了在广域网中的存取延迟。大型数据库分布在多个站点上，而使每个站点上都存在较小型的数据库，这样做的结果是，本地查询和仅在单个站点上存取数据的事务，会因为本地较小型的数据库而执行得更好。此外，在每个站点上执行的事务数量要比把所有事务都提交到单个集中式数据库的事务数量要小。而且，可以通过将多个查询放在不同站点上分别执行，或将一个查询分解成一组可以并行执行的子查询，来实现网间查询和网内查询的并行性。这对性能的提高有很大的作用。

(4) 更容易扩充

在分布式环境中，涉及诸如增加更多的数据、扩充数据库规模或增加更多的处理器之类的系统扩充会变得很容易。

上面1中所讨论的透明性使得在易用性和因提供透明性而带来的开销之间达到了折衷。完全的透明性为全局用户提供了一个整体DDBS的视图，用户可将其视为一个单一的集中式系统。透明性是作为**自治性**（autonomy）的补充而提供的，自治性使得用户可以更严格地控制他们自己的本地数据库。透明性特性可以作为用户语言的一部分来实现，用户语言可以将请求服务转换为适合的操作。此外，透明性还受那些必须由操作系统和DBMS提供的特性的影响。

9.1.3　分布式数据库的附加功能

分布导致了系统设计和实现中复杂性的增加。为了获得前面提到的分布式数据库潜在的优势，DDBMS软件必须能够提供以下集中式DBMS功能之外的附加功能：

- **数据跟踪**。通过扩展DDBMS目录来跟踪数据分布、分片和复制的能力。
- **分布式查询处理**。通过通讯网络存取远程站点中的数据，以及在不同站点间传输请求和数据的能力。
- **分布式事务管理**。为从多个站点存取数据的查询和事务设计执行策略的能力，以及将存取分布式数据和维护整个数据库完整性保持同步的能力。

- **复制数据的管理**。能够确定存取复制数据项的哪个副本，以及维护复制数据项副本一致性的能力。
- **分布式数据库恢复**。能够把数据库从单个站点的故障和例如通讯链路故障那样的新类型故障中恢复的能力。
- **安全性**。分布式事务的执行必须有适当的数据安全性管理，以及用户授权 / 存取权限的安全管理作为保证。
- **分布式目录管理**。目录包含了数据库中有关数据的信息（元数据），它对于整个DDB来说是全局的，对于每个站点来说又是局部的。目录的布局和分布是设计和策略的问题。

相对于集中式DBMS而言，上述这些功能增加了DDBMS的复杂性。在我们意识到所有潜在的分布优势之前，必须找到这些设计问题的满意解决方案。由此可见，要完全实现所有这些附加功能是很难的，而要找到最优的解决方案更增加了实现的难度。

在物理**硬件**层，区别DDBMS和集中式系统的主要因素如下：

- 有多个被称为**站点**或**结点**的计算机。
- 这些站点必须通过某些类型的**通信网络**（communication network）连接起来，以便在站点间传输数据和命令，如图9-1c所示。

所有物理位置邻近的站点，比如同一幢楼或邻近楼群，可以通过**局域网**（local area network）连接，或者如果它们分布在更大的地理区域，就要通过**远程**（long-haul）或**广域网**（wide area network）连接。局域网通常使用电缆，而远程网络使用电话线或卫星。也有可能使用多种网络的组合。

网络可以采用不同的**拓扑结构**（topology），该结构定义了站点间的直接通讯路径。所用网络的类型和拓扑结构都会影响系统的性能，进而对分布式查询处理和分布式数据库设计的策略也会产生重要的影响。然而，对高层的体系结构而言，并不关心使用哪种类型的网络，而只关心每个站点是否能够直接地或间接地与其他站点进行通讯。在本章的其他部分，我们假设站点间存在某些类型的通讯网络，而不管特定的拓扑结构。我们不打算寻求任何的网络专有问题，尽管这对理解DDBS的有效操作来说是很重要的，因为网络设计和性能问题才是关键的。

9.2　分布式数据库设计的数据分片、复制和分配技术

本节将讨论用于将数据库分割成被称为**片段**（fragment）的逻辑单位的技术，片段可以被分配到不同站点上进行存储。我们还将讨论使用**数据复制**技术，把某些数据存储在多个站点中，以及将片段或片段的副本**分配**在不同站点上存储的过程。这些技术将在**分布式数据库设计**的过程中使用。有关数据分片、分配和副本的信息存储在**全局目录**中，该目录在需要时可以被DDBS应用访问。

9.2.1　数据分片

在DDB中，必须确定哪个站点应该用于存储数据库的哪部分信息。我们假设现在数据没有任何副本，也就是说，每个关系或关系的一部分只存储在一个站点上。在本节后面的内容中将讨论副本及其作用。对于其他的数据模型我们也使用与关系数据库术语相似的术语。这里假设我们的讨论以关系数据库模式开始，并且必须确定如何将关系分布到各个站点上。使用《数据库系统基础：初级篇》中图5-5所示的关系数据库模式来说明这样的讨论。

在确定如何分布数据之前，必须确定要被分布的数据库的逻辑单元。最简单的逻辑单元是关系本身，也就是说，整个关系被存储在一个特定的站点中。在本例中，必须选出一个站点去存储《数据库系统基础：初级篇》中图5-5中EMPLOYEE、DEPARTMENT、PROJECT、WORKS_ON和DEPENDENT中的每一个关系。然而，在很多情况下，为了分布，一个关系被分割成了更小的逻辑单位。例如，考虑《数据库系统基础：初级篇》中图5-6所示的公司数据库，并且假设有3个计算机站点，每个站

点代表公司的一个部门[①]。

我们想在每个部门的计算机站点上存储与该部门有关的数据库信息，可以使用一种被称为水平分片的技术按部门划分每个关系。

1．水平分片（Horizontal Fragmentation）

一个关系中的**水平片段**（horizontal fragment）是该关系中元组的一个子集。关系中的元组按一个或多个属性的特定条件划分成水平片段。通常只涉及单个的属性。例如，可以为《数据库系统基础：初级篇》图5-6中的EMPLOYEE关系定义3个水平片段，赋予以下条件：（Dno=5）、（Dno=4）和（Dno=1），每个片段包含了为特定部门工作的EMPLOYEE元组。类似的，为PROJECT关系定义3个水平片段，赋予以下条件：（Dnum=5）、（Dnum=4）和（Dnum=1），每个片段包含了被某个特定部门所控制的PROJECT元组。**水平分片**将一个关系水平地进行分组，以创建元组的子集，每个子集都有特定的逻辑含义。然后这些片段被分配到分布式数据库系统中的不同站点上。**导出水平分片**（derived horizontal fragmentation）将对基本关系（例如示例中的DEPARTMENT）的分片借助于其他辅助关系（例如示例中的EMPLOYEE和PROJECT）中的属性，这些辅助关系与基本关系通过外码进行联系。这样，在基本关系和辅助关系之间的相关数据以相同的方式被分片。

2．垂直分片（Vertical Fragmentation）

每个站点也许不需要一个关系的所有属性，这就表明需要不同的分片类型。**垂直分片**将一个关系以列为单位“垂直地”分割。关系的**垂直片段**（vertical fragment）只保留关系的某些属性。例如，把关系EMPLOYEE分成两个垂直的片段，第一个片段包含个人信息：Name、Bdate、Address和Sex，第二个片段包含与工作有关的信息：Ssn、Salary、Super_ssn、Dno。这样的垂直分片不是完全合适的，因为，如果两个片段被分开存储，由于在两个片段之间没有共同的属性，不能将原来的雇员元组重新放到一起。因此，有必要在每一个垂直片段中包含主码或一些候选码属性，以便可以从这些片段中重构完整的关系。因此，必须把Ssn属性加到个人信息片段中。

需要注意，关系R上的每一个水平分片可以通过关系代数中$\sigma_{Ci}(R)$操作符确定。一组条件为C_1，C_2，…，C_n的水平分片的集合包含了R中的所有元组，也就是说，R中满足（C_1ORC_2OR…ORC_n）的每个元组，被称为R的**完备水平分片**（complete horizontal fragmentation）。在许多情况下，一组完备水平分片是**不相交的**（disjoint），因为对于任何$i\neq j$来说R中元组没有满足（C_i AND C_j）的。前面对EMPLOYEE关系和PROJECT关系进行水平分片的两个示例都是完备的且是不相交的。为了从这些完备水平分片重构关系R，需要在这些片段上使用UNION操作。

关系R上的垂直片段可以通过关系代数中的$\pi_{Li}(R)$操作来指定。如果垂直片段集合的投影列表L_1，L_2，…，L_n包括了R中的所有属性，但只有R的主码属性是共享的，那么这个垂直片段集合被称为**完备垂直分片**（complete vertical fragmentation）。在这种情况下投影列表满足以下两个条件：

- $L_1 \cup L_2 \cup \cdots \cup L_n$=ATTRS($R$)。
- 对于任何$i\neq j$，$L_i \cap L_j$=PK(R)，其中ATTRS(R)是R的属性的集合，而pk(R)是R的主码。

为了从一个完备垂直分片中重构关系R，需要在垂直片段（假设没有采用水平分片）上应用OUTER UNION操作。需要注意：对于完备垂直分片，甚至在已经应用了一些水平分片的时候再应用FULL OUTER JOIN也可获得相同的结果。具有投影列表L_1={Ssn, Name, Bdate, Address, Sex}和L_2={Ssn, Salary, Super_ssn, Dno }的EMPLOYEE关系的两个垂直分片，构成了EMPLOYEE的一个完备垂直分片。

《数据库系统基础：初级篇》中图5-5所示的EMPLOYEE关系中既不完备又不是不相交的两个垂直片段是由条件（Salary>50 000）和（Dno=4）确定的。它们可以不包含所有的EMPLOYEE元组。不完备的两个垂直片段是被属性列表L_1={Name，Address}和L_2={Ssn，Name，Salary}确定的，这些列表违反了完备垂直分片的两个条件。

---

① 当然，在实际情况下，关系中会有比《数据库系统基础：初级篇》中图5-6所示更多的元组。

3. 混合分片（Mixed Fragmentation）

还可以将两种类型的分片混合，生成**混合分片**。例如，可连接先前给出的EMPLOYMEE关系的水平和垂直分片，将它们组合成包含六个片段的混合分片。在这种情况下，可以以适当的顺序使用UNION和OUTER UNION（或OUTER JOIN）操作来重构原来的关系。总之，关系R的**片段**可以被操作$\pi_L(\sigma_C(R))$的SELECT-PROJECT组合来指定。如果C=TRUE（即，所有元组都被选中）且$L \neq$ATTRS(R)，则可以得到一个垂直片段，而如果$C \neq$TURE且L=ATTRS(R)，则可以得到一个水平片段。最后，如果$C \neq$TRUE且$L \neq$ATTRS(R)，可以得到一个混合片段。注意到关系自身可以被看作是一个C=TRUE且L=ATTRS(R)的片段。在随后的讨论中，术语片段用来表示一个关系或上述类型的任意一个关系片段。

数据库的**分片模式**（fragmentation schema）是对片段集合的定义，这些片段包括数据库中的所有属性和元组，并且满足这样的条件：整个数据库可以通过应用OUTER UNION（或OUTER JOIN）和UNION操作的某个序列来重构整个数据库。这种分片模式尽管不是必要的，但有时也是很有用的。除非在垂直（或混合）片段中重复存储主码，否则，它使得所有片段都互不相交。对于后面一种情况，所有片段的复制和分布显然将在后续阶段指定，并与片段相分离。

分配模式（allocation schema）描述了在DDBS站点中片段的分配情况。因此，它是每个片段在其指定存储站点上的映射。如果一个片段存储在多个站点上，就称它是**被复制的**。接下来我们讨论数据的复制和分配。

9.2.2　数据复制和分配

复制对于增强数据的可用性是很有用的。最极端的情况是在分布式数据库系统中的每个站点上都复制了整个数据库，因此就创建了一个**全复制分布式数据库**（fully replicated distributed database）。这样可以显著地提高系统的可用性，因为只要至少有一个站点在运行，系统就可以继续运行。这也改善了对全局查询的检索性能，因为这类查询的结果可以从任何一个站点本地获得；因此，如果某个站点包含服务器模块的话，检索查询可以在它提交的本地站点进行处理。全复制的不利之处在于它会急剧地降低更新操作效率，因为为了保持副本的一致性，对单个副本的逻辑更新都必须在数据库的每个副本上执行同样的更新。如果存在很多数据库副本的话，系统效率降低尤其明显。全复制使并发控制和恢复技术比9.5节将看到的无复制情形需要更大的开销。

与全复制对应的另一个极端是**无复制**，即，每个片段恰好只存储在一个站点上。在这种情况下，除了垂直（或混合）片段中有主码重复外，所有的片段都必须是不相交的。这也称为**无冗余分配**（nonredundant allocation）。

介于这两种极端情况之间有一种广泛使用的情况，即数据**部分复制**（partial replication），即复制数据库中的一些片段而其余的片段则不复制。在分布式数据库系统中每个片段副本的数量可以从一个到系统中所有站点的数目。在诸如销售人员、金融计划制定者、索赔调停者等移动工作的应用中，经常使用部分复制。那些移动工作的雇员需用便携式电脑和个人数字助理（PDA）随身携带着部分复制的数据库，并且与服务器的数据库保持阶段性的同步[①]。片段复制的描述有时称为**复制模式**（replication schema）。

每个片段或片段的每个副本，必须分配到分布式数据库系统的特定站点，这个过程称为**数据分布**（data distribution）或**数据分配**（data allocation）。站点选择和复制的程度取决于系统的性能和可用性目标，以及每个站点上提交事务的类型和频率。例如，如果要求高可用性且事务可在任何一个站点上提交，并且大部分事务只是执行检索，那么全复制数据库是一个合适的选择。然而，如果那些访问数据库特定部分的特定事务主要是在特定站点上提交，那么相应的片段集合应分配在那个特定的站点上。对于要被多个站点访问的数据，应该在那些站点上都被复制。如果要执行许多更新，

① 有许多方法使部分复制数据库保持同步，见（Mahajan et al，1998）。

则有限的复制是很有用的。对于分布式数据的分配，寻找一个最佳的、有效的方案是一个复杂的优化问题。

9.2.3　分片、分配和复制示例

现在考虑《数据库系统基础：初级篇》中图5-5和图5-6所示的公司数据库的分片和分布的示例。假设该公司有3个计算机站点，即每个当前部门各为一个站点。站点2和3分别属于部门5和4。我们预期在每个站点上都要频繁地访问EMPLOYEE和PROJECT表中有关工作在该部门的雇员和该部门控制的项目信息。进一步说，假设这些站点主要存取EMPLOYEE的Name、Ssn、Salary和Super_ssn属性。站点1供公司总部使用，并且除了为了保险目的而记录的DEPENDENT信息外，还定期地存取所有雇员和项目的信息。

根据这些要求，图5-6中的整个数据库可以存储在站点1。为了确定将要复制到站点2和3上的片段，首先根据DEPARTMENT表的主码Dnumber的值进行水平分片，然后基于它的外码部门号，在图5-5中分别称为Dno、Dnum和Dnumber，将导出的片段应用到关系EMPLOYEE、PROJECT和DEPT_LOCATIONS上，然后在刚才得到的EMPLOYEE片段上进行垂直分片，得到只含属性{Name, Ssn, Salary, Super_ssn, Dno}的片段。图9-3给出了EMPD_5和EMPD_4的混合分片，它包含了分别满足条件Dno=5和Dno=4的EMPLOYEE元组。类似地，PROJECT、DEPARTMENT和DEPT_LOCATIONS都按部门编号进行水平分片。所有这些存储在站点2和3的片段都被复制，因为它们也被要求存储在总部站点1上。

EMPD_5

Fname	Minit	Lname	Ssn	Salary	Super_ssn	Dno
John	B	Smith	123456789	30000	333445555	5
Franklin	T	Wong	333445555	40000	888665555	5
Ramesh	K	Narayan	666884444	38000	333445555	5
Joyce	A	English	453453453	25000	333445555	5

DEP_5

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22

DEP_5_LOCS

Dnumber	Location
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON_5

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0

PROJS_5

Pname	Pnumber	Plocation	Dnum
Product X	1	Bellaire	5
Product Y	2	Sugarland	5
Product Z	3	Houston	5

Data at site 2

(a) 站点2上对应于部门5的关系片段

图9-3　站点的片段分配

EMPD_4

Fname	Minit	Lname	Ssn	Salary	Super_ssn	Dno
Alicia	J	Zelaya	999887777	25000	987654321	4
Jennifer	S	Wallace	987654321	43000	888665555	4
Ahmad	V	Jabbar	987987987	25000	987654321	4

DEP_4

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Administration	4	987654321	1995-01-01

DEP_4_LOCS

Dnumber	Location
4	Stafford

WORKS_ON_4

Essn	Pno	Hours
333445555	10	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0

PROJS_4

Pname	Pnumber	Plocation	Dnum
Computerization	10	Stafford	4
New_benefits	30	Stafford	4

Data at site 3

(b) 站点3上对应于部门4的关系片段

图9-3　（续）

现在需要对WORKS_ON关系进行分片，并且确定哪些WORKS_ON片段要存储在站点2和3上。这时面临这样一个问题：WORKS_ON中的属性没有一个可以直接指明每个元组所属的部门。实际上，WORKS_ON中的每个元组都与雇员e和项目*P*相联系。我们既可以以e工作的部门*D*也可以以管理项目*P*的部门*D*′来对表WORKS_ON进行分片。如果对表WORKS_ON中所有元组增加一个*D*= *D*′的约束，分片就会变得更加容易，即雇员只能为那些他所在部门管理的项目而工作。然而，在图5-6中的数据库没有这样的约束。例如，WORKS_ON元组<333445555, 10, 10.0>关联到一个为部门5工作的雇员和一个被部门4管理的项目。在这种情况下，可以基于雇员工作的部门（用条件*C*表达）来分片WORKS_ON，然后进一步基于雇员参与项目的管理部门来分片，如本书中图9-4所示。

G1

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0

C1 = C and (Pno in (SELECT
Pnumber FROM PROJECT
WHERE Dnum = 5))

G2

Essn	Pno	Hours
333445555	10	10.0

C2 = C and (Pno in (SELECT
Pnumber FROM PROJECT
WHERE Dnum = 4))

G3

Essn	Pno	Hours
333445555	20	10.0

C3 = C and (Pno in (SELECT
Pnumber FROM PROJECT
WHERE Dnum = 1))

（a）关系WORKS_ON中在部门5工作的雇员的片段

（C=[Essn in (SELECT Ssn FROM EMPLOYEE WHERE Dno=5)]）

图9-4　关系WORKS_ON的完备且不相交的片段

G4

Essn	Pno	Hours

C4 = C and (Pno in (SELECT Pnumber FROM PROJECT WHERE Dnum = 5))

G5

Essn	Pno	Hours
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0

C5 = C and (Pno in (SELECT Pnumber FROM PROJECT WHERE Dnum = 4))

G6

Essn	Pno	Hours
987654321	20	15.0

C6 = C and (Pno in (SELECT Pnumber FROM PROJECT WHERE Dnum = 1))

（b）关系WORKS_ON中在部门4工作的雇员的片段
（C=[Essn in (SELECT Ssn FROM EMPLOYEE WHERE Dno=5)]）

G7

Essn	Pno	Hours

C7 = C and (Pno in (SELECT Pnumber FROM PROJECT WHERE Dnum = 5))

G8

Essn	Pno	Hours

C8 = C and (Pno in (SELECT Pnumber FROM PROJECT WHERE Dnum = 4))

G9

Essn	Pno	Hours
888665555	20	Null

C9 = C and (Pno in (SELECT Pnumber FROM PROJECT WHERE Dnum = 1))

（c）关系WORKS_ON中在部门1工作的雇员的片段
（C=[Essn in (SELECT Ssn FROM EMPLOYEE WHERE Dno=1)]）

图9-4　（续）

在图9-4中，片段G_1、G_2和G_3的并集给出了为部门5工作的雇员的所有WORKS_ON元组。同样地，片段G_4、G_5和G_6的并集给出了为部门4工作的雇员的所有WORKS_ON元组。另一方面，片段G_1、G_4和G_7的并集给出了被部门5管理的项目的所有WORKS_ON元组。G_1到G_9每个片段的条件都在图9-4中展示出来了。对于表示M:N联系（多对多联系）的关系经常会有多种可能的逻辑分片，如WORKS_ON。在图9-3所示的分布中，选择包含可以与站点2和3上的EMPLOYEE元组或PROJECT元组进行连接的所有片段。因此，可以把片段G_1、G_2、G_3、G_4和G_7的并集分配到站点2上，而把片段G_4、G_5、G_6、G_2和G_8的并集分配到站点3上。注意片段G_2和G_4在两个站点上均被复制。该分配策略允许在站点2和站点3上的本地EMPLOYEE片段或PROJECT片段与本地WORKS_ON片段之间进行完全的本地执行。这就清楚地论证了对于大型数据库而言，数据的分片和分配是一个多么复杂的问题了。本章末的参考文献中提供了在这个领域内的一些相关工作。

9.3　分布式数据库系统的类型

术语“分布式数据库管理系统”可以描述各种各样的在许多方面互不相同的系统。这些系统的主要特征是数据和软件被分布到由通讯网络连接起来的多个站点上。本节将讨论各种类型的DDBMS，以及区分这些系统的标准和因素。

首先需要考虑的因素是DDBMS软件的**同构性程度**（degree of homogeneity）。如果所有的服务器（或单个的本地DBMS）使用同样的软件并且所有的用户（客户端）也使用同样的软件，那么该DDBMS就称为**同构的**（homogeneous）；否则就是**异构的**（heterogeneous）。另外一个与同构度有关的因素是**本地自治性程度**（degree of local autonomy）。如果本地站点没有权利运行独立的DBMS，这类系统就是**无本地自治性**（no local autonomy）的。另一方面，如果允许本地事务对服务器的直接存取，那么系统就具备一定程度的本地自治性。

自治性程度中的一个极端情形是，对用户来说DDBMS看起来像是集中式DBMS。只存在单一的概念模式，所有对系统的存取都通过作为DDBMS一部分的站点来实现，这就是说不存在本地自治性。

另一个极端情形是，使用一个称为联邦式的*DDBMS*（或多数据库系统）类型。在这种类型的系统中，每个服务器都是一个独立的、自治的集中式DBMS，它们有自己的本地用户、本地事务和DBA，因此具有最高程度的本地自治性。术语**联邦式数据库系统**（federated database system，FDBS）应用于以下情况，即用全局的视图或模式来描述被各个应用共享的数据库联盟。另一方面，多数据库系统（multidatabase system）没有全局模式，当应用需要时就互动地构造一个。两个系统都是介于分布式和集中式系统之间的混合体，并且它们之间的区别也不是严格的。通常意义上，把它们都称为FDBS。

在异构的FDBS中，一个服务器可以是关系DBMS，另一个是网状DBMS，而第三个可以是对象DBMS或层次DBMS。在这种情况下，必须要有一种规范的系统语言及语言转换器，使用该语言转换器将子查询从规范语言转换成每个服务器自己的语言。在这里简要地讨论一下影响FDBS底层设计的相关问题。

1. 联邦式数据库管理系统问题

存在于FDBS中的异构性可能源于很多原因。我们先讨论这些原因，然后再指出不同的自治性类型如何形成语义的异构性，这种语义的异构性必须在异构的FDBS中得到解决。

- **数据模型的不同**：一个组织机构中的数据库来源于多种不同类型的数据模型，包括所谓的遗留模型（网状和层次，见附录D和E）、关系数据模型、对象数据模型，甚至是文件。各种模型的建模能力是相异的。因此，通过单一的全局模式来统一地处理它们，或以单一语言来处理它们，这些都将是很复杂的问题。即使两个数据库都来自RDBMS环境，相同的信息在不同的数据库中会有不同的表示形式，可能被表示为属性名、关系名或者值。这就要求有一个智能的查询处理机制，它具有基于元数据来关联信息的能力。
- **约束条件的不同**。约束机制的描述和实现是因系统而异的。这在全局模式结构中必须有一致的可比性。例如，从ER模式得到的联系在关系模型中被表述为参照完整性约束，关系模型中使用触发器来实现这类约束。全局模式也要处理约束之间的潜在冲突。
- **查询语言的不同**。即使采用了相同的数据模型，查询语言以及语言的版本都有可能不同。例如，SQL就有各种版本，如SQL-89、SQL-92和SQL-99，每个系统都有它自己的一套数据类型集合、比较操作符、串操作特性，等等。

2. 语义异构性

当相同或相关数据具有不同的含义、解释和用法时，语义异构性就会产生。各成员数据库系统（DBS）间的语义异构性，在设计异构数据库的全局模式时会导致最大的障碍。各成员DBS的**设计自治性**（design autonomy）是指它们具有选择以下设计参数的自由，而这种选择反过来又会影响FDBS最终的复杂性：

- **抽取数据的论域**。例如，在联邦中的两个客户账户数据库可能来自美国和日本，因为有不同的会计实务需要，使得有关客户账户的属性集也可能完全不同。当前汇率的波动也将是一个问题。因此，有着相同名称的两个数据库中的关系CUSTOMER和ACCOUNT，可能会有一些共同的信息，也会有一些全然不同的信息。
- **表示和命名**。对于每个本地数据库，数据元素的表示和命名以及数据模型的结构，可能已被预先指定。
- **数据的理解、含义和主观解释**。这是产生语义异构性的主要原因。
- **事务和策略约束**。这些约束涉及到可串行性标准、补偿事务以及其他的一些事务策略。
- **导出摘要**。系统支持聚集、摘要以及其他数据处理特性和操作。

各成员DBS的**通讯自治性**（communication autonomy）是指自主决定是否和其他成员DBS通讯的能力。**执行自治性**（execution autonomy）是指成员DBS执行局部操作而不受其他成员DBS的外部操作干预的能力。成员DBS的**关联自治性**（association autonomy）隐含了它有能力确定是否和怎样与其他成员DBS共享功能（它支持的操作）和资源（它管理的数据）。设计FDBS时的主要挑战在于为各

成员DBS提供上述类型的自治性的同时，使它们能够互操作。

图9-5[①]所示的是一个典型的在FDBS环境中支持全局应用的五层模式的体系结构。在这个结构中，**局部模式**是成员数据库的概念模式（完全的数据库定义），通过将局部模式转换成FDBS的规范数据模式或通用数据模式（CDM）来导出**成员模式**。把局部模式转换为成员模式的模式转换，是通过将成员模式上的命令转换为相应的局部模式命令来完成的。**输出模式**代表着可用于FDBS的成员模式的子集。**联邦模式**是全局的模式或视图，它是集成所有可共享的输出模式的结果。**外部模式**定义了一个面向用户组或应用的模式，就像三层模式体系结构中的模式一样[②]。

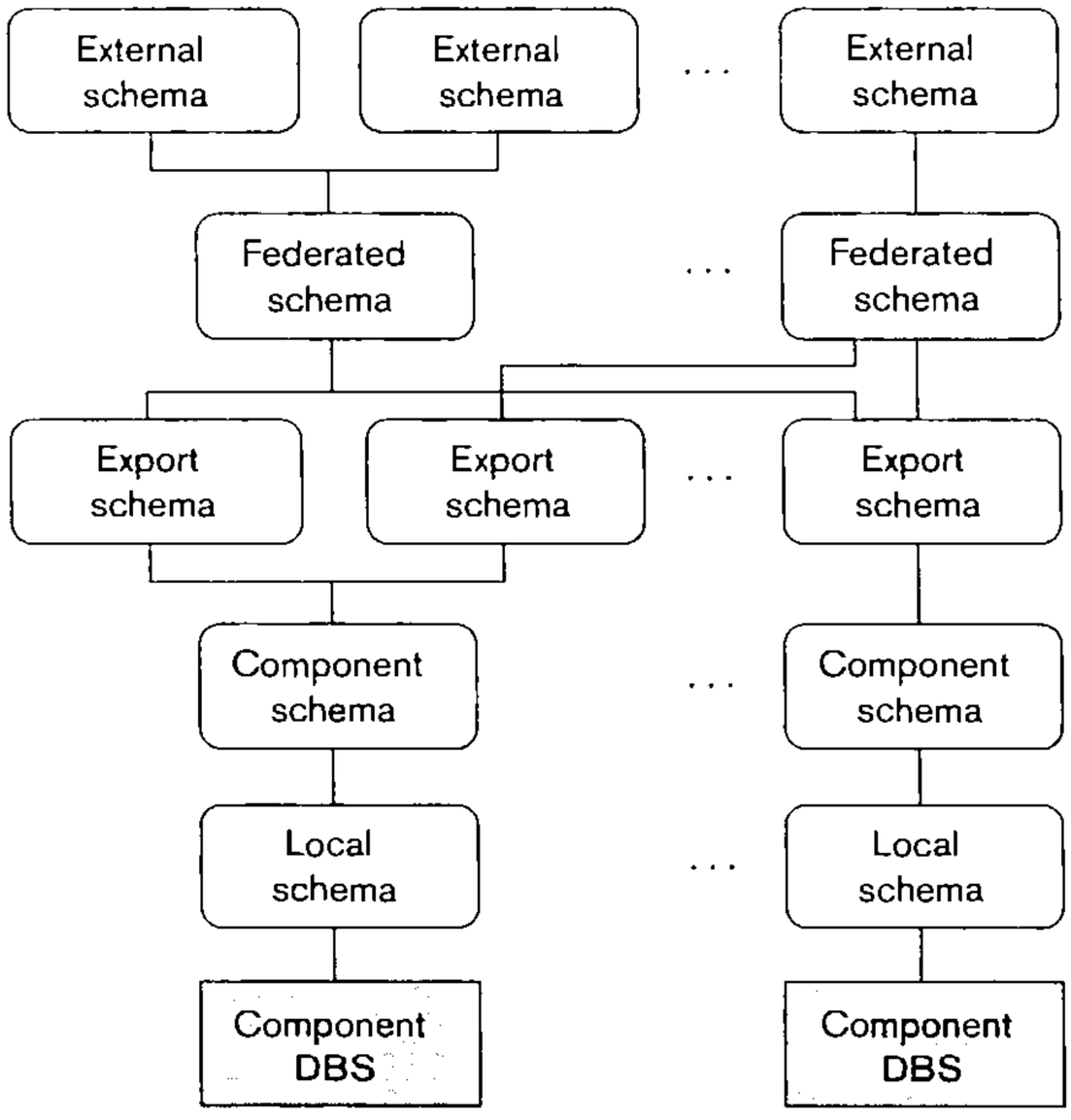


图9-5　一个联邦式数据库系统（FDBS）的五层模式体系结构

所有与查询处理、事务处理、目录和元数据的管理和恢复有关的问题，在应用到FDBS时需要做额外的考虑。对于它们的详细讨论超出了我们的范围。

9.4　分布式数据库中的查询处理

现在概述DDBMS如何处理和优化一个查询。首先讨论处理一个分布式查询的通讯代价，然后讨论一个称为半连接的特殊操作，它用于优化DDBMS中的某些类型的查询。

9.4.1　分布式查询处理的数据传输代价

在《数据库系统基础：初级篇》的第15章中我们曾经讨论过集中式DBMS的查询处理和优化问题。在分布式数据库系统中，一些附加因素使查询处理进一步复杂化。首先是在网络上传输数据的代价。这些数据包含中间文件，这些中间文件是为了进一步处理而需要传输到其他站点上的，也包含最终的结果文件，它们必须要传输到需要查询结果的站点上。尽管这些站点通过高性能的局域网连接时的代价可能不会太高，但它们在其他类型的网络中就会变得非常可观了。因此，DDBMS查询

① 来源：节选自Sheth & Larson，“Federated Database Systems for Managing Distributed Heterogeneous Autonomous Databases”，*ACM Computing Surveys*（Vol.22 No.3，September 1990）。

② FDBMS的自治性和五层体系结构的详细论述请见（Sheth & Larson，1990）一书。

优化算法在选择分布式查询执行策略时，将以减少传输的数据量作为优化目标。

在此，我们用两个简单的查询示例来说明。假设《数据库系统基础：初级篇》中图5-5的关系EMPLOYEE和DEPARTMENT按照如图9-6所示的形式进行分布，在这个示例中，假设没有一个关系被分片。根据图9-6中所给出的关系EMPLOYEE的大小是100*10 000=10^6字节，关系DEPARTMENT的大小是35*100=3 500字节。考虑查询Q：对于每一个雇员，检索他的姓名和他所在部门的名称。这个查询在关系代数中可以表示为如下形式：

Q: $\pi_{Fname,Lname,Dname}(EMPLOYEE \bowtie_{Dno=Dnumber} DEPARTMENT)$

Site 1:

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno

10 000 records
each record is 100 bytes long
Ssn field is 9 bytes long　　Fname field is 15 bytes long
Dno field is 4 bytes long　　Lname field is 15 bytes long

Site 2:

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date

100 records
each record is 35 bytes long
Dnumber field is 4 bytes long　　Dname field is 10 bytes long
Mgr_ssn field is 9 bytes long

图9-6　说明数据传输量的示例

假设每个雇员都与某个部门有关系，这个查询的结果将包含10 000条记录。假设查询结果中的每条记录长度是40字节，且查询在站点3提交，该站点被称作**结果站点**（result site），因为那里需要查询的结果。关系EMPLOYEE和DEPARTMENT都没有存储在站点3上。

执行此分布式查询的3种简单的策略是：

(1) 将关系EMPLOYEE和DEPARTMENT传输到结果站点，并在站点3上完成连接。在这种情况下，总共必须传输1 000 000＋3 500＝1 003 500个字节。

(2) 将关系EMPLOYEE传输到站点2，并在站点2执行连接，再将结果传输到站点3上。该查询结果的大小是40*10 000＝400 000个字节，因此必须传输400 000＋1 000 000＝1 400 000个字节。

(3) 将关系DEPARTMENT传输到站点1，并在站点1上执行连接，再将结果送到站点3上。这种情况下必须传输400 000＋3 500＝403 500个字节。

如果优化准则中要求考虑使数据传输量最小化，就应当选择策略3。现在考虑另外一个查询Q′：对于每一个部门，检索部门名称和部门经理的姓名。这个查询在关系代数中可以表示为如下形式：

Q′: $\pi_{Fname,Lname,Dname}(DEPARTMENT \bowtie_{Mgr_ssn=Ssn} EMPLOYEE)$

我们仍然假设查询在站点3上提交，假设每个部门都有一个经理，除了Q′的结果只含100条记录以外，执行查询Q的3种策略同样可以运用于Q′：

(1) 将关系EMPLOYEE和DEPARTMENT传输到结果站点，并在站点3上完成连接。在这种情况下，总共必须传输1 000 000＋3 500＝1 003 500个字节。

(2) 将关系EMPLOYEE传输到站点2，并在站点2执行连接，再将结果送到站点3上。查询结果的大小为40*100＝4 000字节，因此必须传输4 000＋1 000 000＝1 004 000字节。

(3) 将关系DEPARTMENT传输到站点1，并在站点1上执行连接，再将结果传输到站点3上。这种情况要传输4 000＋3 500＝7 500字节。

在这种情况下，由于策略3对于策略1和2具有绝对的优势，我们再次选择策略3。前面的3种策略其结果站点（站点3）与所有包含涉及查询文件的站点（站点1和2）显然是不同的。然而，假设结果站点是站点2，那么有两种简单的策略：

(1) 将关系EMPLOYEE传输到站点2，执行查询，并且在站点2上将结果呈现给用户。在这里，对于Q和Q′而言，需要传输的字节数量相同，都是1 000 000。

(2) 将关系DEPARTMENT传输到站点1，在站点1上执行查询，并将结果返回给站点2。这种情况下对于Q必须传输400 000＋3 500＝403 500字节，而对于Q′必须传输4 000＋3 500＝7 500字节。

还有一种更为复杂的策略，它有时比这3种简单策略更有效率，它使用一个称为**半连接**（semijoin）的操作。接下来介绍这种操作并讨论使用半连接的分布式执行。

9.4.2　使用半连接的分布式查询处理

使用半连接操作的分布式查询处理的思想，就是将关系从一个站点传输到另一个站点之前减少该关系中元组的数量。直观地看，其思想是将一个关系R的连接列（joining column）传输到另一个关系S所在的站点；然后将这些列与S连接。由此，再将连接属性和结果中要求的属性投影出来，并且传回到初始站点与R做连接。因此，只有R的连接列在一个方向上传输，而无额外元组或属性的S的子集在另一个方向上传输。要是只有S中的一小部分元组参与连接的话，这是一个使数据传输量最小化的非常有效的方案。

为了说明这个问题，考虑以下执行Q和Q′的策略：

(1) 在站点2投影DEPARTMENT的连接属性，并把它们传输到站点1。对于Q，传输$F=\pi_{Dnumber}$(DEPARTMENT)，它的大小是4*100＝400字节。而对于Q′，传输$F'=\pi_{Mgr_ssn}$(DEPARTMENT)，它的大小是9*100＝900字节。

(2) 在站点1上将传输的文件与关系EMPLOYEE进行连接，然后把要求的属性从结果文件传输到站点2上。对于Q，传输$R=\pi_{Dno,\ Fname,\ Lname}(F \bowtie_{Dnumber=Dno}$ EMPLOYEE)，它的大小是34*10 000＝340 000字节。而对于Q′，传输$R'=\pi_{Mgr_ssn,\ Fname,\ Lname}(F' \bowtie_{Mgr_ssn=Ssn}$ EMPLOYEE)，它的大小是39*100＝3 900字节。

(3) 通过对被传输文件R或R'与DEPARTMENT进行连接来执行查询，然后在站点2上将结果呈现给用户。

采用这种策略，对于Q需要传输340 400字节，对于Q′需要传输4 800字节。在第2步中限制了传输到站点2的关系EMPLOYEE的属性和元组，只传输那些在第3步中实际要与DEPARTMENT元组进行连接的属性和元组。对于查询Q，它包含了所有的EMPLOYEE元组，所以只实现了很少的改进。而对于Q′，10 000个EMPLOYEE元组中只有100个是需要的。

半连接操作被设计为形式化这一策略。**半连接操作**（semijoin operation）$R \ltimes_{A=B} S$，其中A和B分别是R和S中域相容的属性，产生出与关系代数表达式$\pi_R(R \bowtie_{A=B} S)$相同的结果。在分布式环境下，R和S分布在不同的站点，半连接的典型实现是首先将$F=\pi_B(S)$传输到R所在的站点，然后在那里连接F和R，从而产生了在这里论述的策略。

注意半连接操作是不可交换的，即，$R \ltimes S \neq S \ltimes R$。

9.4.3　查询和更新分解

在一个不具备分布透明性的DDBMS中，用户根据指定的片段直接表达一个查询。例如，考虑另一个查询Q：检索参与由部门5管理的项目的每一个雇员的姓名和每周的工作时间。在前面给出的分布式数据库示例中，指定在站点2和站3上存储的关系如图9-3所示，在站点1上存储的关系如《数据库系统基础：初级篇》中图5-6所示，这与前面的示例一样。提交这类查询的用户必须指定该查询所涉及的是站点2上的关系PROJS_5和WORKS_ON_5（图9-3），还是站点1上的关系PROJECT和WORKS_ON（《数据库系统基础：初级篇》的图5-6）。用户在更新一个没有复制透明性的DDBMS时，

也必须要维护复制数据项的一致性。

另一方面，一个支持完全分布、分片和复制透明性的DDBMS，它允许用户如同在集中式DBMS那样，在《数据库系统基础：初级篇》中图5-5的模式上指定查询或更新请求。对于更新，DDBMS负责维护复制项之间的一致性，这是通过使用一种分布式并发控制算法来实现的，这种算法将在9.5节中讨论。对于查询，**查询分解**（query　decomposition）模块必须将一个查询分裂或**分解**（decompose）成可以在单个站点上执行的若干个**子查询**（subquery）。此外，还必须产生一个连接各个子查询结果以组成该查询结果的策略。每当DDBMS确定该查询涉及的项中存在被复制的项时，它在查询执行过程中必须选择或**物化**（materialize）特定的副本。

为了确定哪个副本包含查询中引用的数据项，就要使用DDBMS目录，该目录中存储着涉及DDBMS分片、复制和分布的信息。对于垂直分片，每个片段的属性列表都存储在目录中。对于水平分片，为每一个片段保存了被称作**防护**（guard）的条件。防护条件基本上是一个选择条件，它确定了哪些元组在片段中；它之所以被称作防护是因为只有满足该条件的元组才被允许存储在该片段中。对于混合分片，属性列表和防护条件都被存储在目录中。

在前面的示例中，站点1（《数据库系统基础：初级篇》的图5-6）中片段的防护条件为TRUE（所有元组），并且属性列表为*（所有属性）。对于图9-3中所示的片段，在图9-7中给出了防护条件和属性列表。当DDBMS分解一个更新请求时，它可以通过检查防护条件来确定哪些片段需要更新。例如，一个用户请求插入一个新的EMPLOYEE元组<‘Alex’，‘B’，‘Coleman’，‘345671239’，‘22-APR-64，‘3306 Sandstone，Houston，TX’，M，33000，‘987654321’，4>，DDBMS会将它分解成为两个插入请求：第一个请求将这个元组插入到站点1的EMPLOYEE片段中，第二个请求把投影元组<‘Alex’，‘B’，‘Coleman’，‘345671239’，33000，‘987654321’，4>插入到站点3的EMPD4片段中。

对于查询分解，DDBMS通过比较查询条件和防护条件，就可以确定哪些片段可能包含所要求的元组。例如，考虑查询Q：检索参与由部门5管理的项目的雇员的姓名和每周的工作时间；这个查询是在《数据库系统基础：初级篇》中图5-5所示的模式上，用以下的SQL语句指定：

```
Q:  SELECT   Fname, Lname, Hours
    FROM     EMPLOYEE, PROJECT, WORKS_ON
    WHERE    Dnum=5 AND Pnumber=Pno AND Essn=Ssn;
```

EMPD5
　attribute list: Fname, Minit, Lname, Ssn, Salary, Super_ssn, Dno
guard condition: Dno=5
DEP5
　attribute list: * (all attributes Dname, Dnumber, Mgr_ssn, Mgr_start_date)
guard condition: Dnumber=5
DEP5_LOCS
　attribute list: * (all attributes Dnumber, Location)
guard condition: Dnumber=5
PROJS5
　attribute list: * (all attributes Pname, Pnumber, Plocation, Dnum)
guard condition: Dnum=5
WORKS_ON5
　attribute list: * (all attributes Essn, Pno,Hours)
guard condition: Essn IN (π_{Ssn} (EMPD5)) OR Pno IN ($\pi_{Pnumber}$ (PROJS5))

(a) 站点2上的片段

图9-7　片段的防护条件和属性列表

EMPD4
　attribute list: Fname, Minit, Lname, Ssn, Salary, Super_ssn, Dno
guard condition: Dno=4
DEP4
　attribute list: * (all attributes Dname, Dnumber, Mgr_ssn, Mgr_start_date)
guard condition: Dnumber=4
DEP4_LOCS
　attribute list: * (all attributes Dnumber, Location)
guard condition: Dnumber=4
PROJS4
　attribute list: * (all attributes Pname, Pnumber, Plocation, Dnum)
guard condition: Dnum=4
WORKS_ON4
　attribute list: * (all attributes Essn, Pno, Hours)
guard condition: Essn IN (π_{Ssn} (EMPD4))
　OR Pno IN ($\pi_{Pnumber}$ (PROJS4))

(b) 站点3上的片段

图9-7　（续）

假设这个查询是在站点2上被提交，同时它又是需要查询结果的站点。根据PROJS5和WORKS_ON5的防护条件，DDBMS得出站点2上的所有元组均满足条件（Dnum=5 AND Pnumber=Pno）。因此，DDBMS将该查询分解为如下关系代数子查询：

$$T_1 \leftarrow \pi_{Essn}(\text{PROJS5} \bowtie_{Pnumber=Pno} \text{WORKS_ON5})$$

$$T_2 \leftarrow \pi_{Essn,\ Fname,\ Lname}(T_1 \bowtie_{Essn=Ssn} \text{EMPLOYEE})$$

$$\text{RESULT} \leftarrow \pi_{Fname,\ Lname,\ Hours}(T_2 * \text{WORKS_ON5})$$

这一分解可以用于执行采用半连接策略的查询。DDBMS根据防护条件知道，PROJS5 恰好包含了那些满足（Dnum=5）的元组，而WORKS_ON5则包含了所有要与PROJS5进行连接的元组。因此，子查询T_1可以在站点2上执行，投影列Essn会被传输到站点1。然后子查询T_2可以在站点1上执行，其结果会被送回到站点2上，在站点2计算最终的查询结果并呈现给用户。另一种策略是将查询Q本身送到站点1，站点1上包含了所有的数据库元组并且可以局部地执行，然后将结果送回到站点2，查询优化器将估计两种策略的代价，并且选择代价估计较小的那一个。

9.5　分布式数据库中并发控制和恢复概述

对于并发控制和恢复，分布式DBMS环境中会出现大量的在集中式DBMS环境中碰不到的问题。这些问题包括：

- **处理数据项的多个副本**：并发控制方法负责维护数据项的多个副本之间的一致性。恢复方法负责在存储该副本的站点发生故障时生成一个与其他副本一致的副本，以便恢复时使用。
- **单个站点的故障**：当一个或多个单个站点发生故障时，如果可能的话，DDBMS应当与其他正在运行的站点一起继续工作。当那个故障站点被恢复时，它的本地数据库必须在它与系统重新连接之前保持和其余站点同步。
- **通讯链路故障**：系统必须有能力处理一个或多个连接这些站点的通讯链路的故障。这个问题的一个极端情况是可能发生**网络分割**（network partitioning），即，将这些站点分裂成为两个或多个分区，这样一个分区中的站点只能与该分区中的另一些站点通讯，而不能与其他分区中的站点通讯。
- **分布式提交**：当提交一个访问存储在多个站点上数据库的事务时，如果某些站点在提交过程

中发生故障的话，就会产生问题。**两阶段提交协议**（two-phase commit protocol）（见第3章）经常用于处理这类问题。

- **分布式死锁**：死锁可能会在多个站点间发生，所以必须要扩展处理死锁的技术以便处理这种情况。

分布式并发控制和恢复技术必须能够处理这些问题以及其他问题。在以下的各小节中我们将回顾在DDBMS中用于处理恢复和并发控制而提出的一些技术。

9.5.1　基于识别数据项副本的分布式并发控制

为了处理分布式数据库中的复制数据项，提出了大量的并发控制方法，这些方法扩展了集中式数据库的并发控制技术。我们将在扩展集中式加锁的上下文中讨论这些技术。类似的扩展也适用于其他的并发控制技术。其思想是为每个数据项指定一个特定的副本作为该数据项的**识别副本**（distinguished copy）。对数据项的加锁与该数据项的识别副本相关，并且所有的加锁和解锁请求都被传输到包含那个副本的站点上。

许多不同的方法都是基于这一思想的，但是这些方法的区别在于对识别副本的选择。在**主站点技术**（primary site technique）中，所有的识别副本都被保留在相同的站点上。对这种方法的一种修改是带有**备份站点**（backup site）的主站点。另一个方法是**主副本**（primary copy）方法，对于这种方法，各种数据项的识别副本可以被存储在不同的站点上。在并发控制中，存储数据项识别副本的站点，基本上都起着该数据项的**协调者站点**（coordinator site）的作用。接下来我们将讨论这些技术。

1. 主站点技术

在这种方法中，单个**主站点**（primary site）被指派为所有数据库项的**协调者站点**。因此，所有的锁都保留在那个站点上，并且所有的加锁和解锁请求都被传输到那里。因此，这种方式是集中式加锁方案的扩展。例如，如果所有的事务都遵守两阶段加锁协议，那么就可以保证可串行化。这种方法的好处就是它是集中式方案的简单扩展，因此不太复杂。然而，它也有某些固有的缺点：其一就是所有的加锁请求都被送到单个站点上，可能会使那个站点超负荷并且导致系统瓶颈；其二是主站点的故障会使整个系统瘫痪，因为所有的加锁信息都保存在那个站点上。这样会制约系统的可靠性和可用性。

尽管所有的锁都在主站点上存取，但数据项本身可以在它们所在的任何站点上存取。例如，一旦事务从主站点获得了一个数据项的读锁（Read_lock），它便可以访问该数据项的任何一个副本。然而，一旦事务获得了一个数据项的写锁（Write_lock），并且更新了数据项，DDBMS要负责在释放锁之前更新该数据项的所有副本。

2. 带有备份站点的主站点

这种方法通过指派第二个站点为**备份站点**（Backup Site）来解决主站点方法的第二个缺点。所有的加锁信息都在主站点和备份站点上保留。万一主站点发生故障，备份站点将接管它而成为主站点，同时新的备份站点被选出。这样就简化了主站点的故障恢复过程，因为备份站点接管了它，并且处理可以在选择了一个新的备份站点之后重新开始，加锁的状态信息将被拷贝到新的备份站点上。但是，这会减慢请求加锁的过程，因为在请求事务得到响应之前，所有的加锁请求和锁的授予信息都必须记录在主站点和备份站点上。主站点和备份站点可能会因为大量请求而超负荷，并且使得系统运行速度变慢，这类问题现在仍不能得到很好的解决。

3. 主副本技术

这种方法试图把各个站点之间锁协调的负载进行分布，它通过将不同数据项的识别副本存储在不同站点上来实现。一个站点的故障只会影响需要存取主副本存储在该故障站点上的数据项锁的事务，而其他的事务不会受到影响。这种方法也可以采用备份站点来提高可靠性和可用性。

4. 发生故障时选择一个新的协调者站点

在前面的任何一种技术中，每当协调者站点故障时，其他站点若要继续运行，则必须选择一个新的协调者站点。在主站点方法没有备份站点的情况下，所有正在执行的事务都必须被终止，并且要在一个冗长的恢复过程后重启。恢复过程部分包含选择新的主站点，并创建一个锁管理器进程和在那个站点上所有加锁信息的记录。对于采用备份站点的方法，指派备份站点为新的主站点，并且选出新的备份站点，同时将所有加锁信息的副本从新的主站点传输到新的备份站点时，事务处理是要被挂起的。

如果备份站点*X*将要成为新的主站点，*X*可以从系统正在运行的其他站点中选择新的备份站点。然而，如果不存在备份站点，或主站点和备份站点都有故障，一个称为**选举**（election）的过程会被用来选择新的协调者站点。在这个过程中，任何试图不断与协调者站点联系而总是失败的站点*Y*，它可以假设协调者站点已失效，并且向所有运行站点发出消息，提议*Y*成为新的协调者，以重新启动选举过程。一旦*Y*收到大多数赞成票，*Y*就可以宣布自己成为新的协调者站点。选举算法本身是非常复杂的，这是选举方法的主要思想。该算法解决了两个或多个站点同时试图成为协调者的问题。本章最后的选读文献中的一些文献从细节上详细讨论了这个过程。

9.5.2　基于投票方法的分布式并发控制

前面讨论的基于复制项的并发控制方法都采用了维护数据项锁的识别副本思想。在**投票方法**（voting method）中，没有识别副本；相反地，加锁请求被发送到所有包含该数据项副本的站点上。每个副本维护它自己的锁并且可以授予或拒绝对它的加锁请求。如果一个请求加锁的事务被大多数的副本授予锁的话，它将持有该锁，并告知所有的副本它已经被授予了锁。如果一个事务在特定的超时周期中没有收到大多数授予它锁的投票，它将取消它的请求并把取消决定告知所有的站点。

投票方法被认为是真正的分布式并发控制方法，因为决策的职责在于所有涉及到的站点。模拟研究表明投票方法在站点间产生的信息通信量比识别副本方法产生的要高。如果算法考虑投票过程中可能的站点故障的话，它就会变得极端复杂了。

9.5.3　分布式恢复

分布式数据库中的恢复过程相当复杂。在此只给出一些简要概念。在某些情况下，不通过与其他站点交换大量信息就确定该站点是否会失效是非常困难的。例如，站点*X*发送一条消息给站点*Y*，并且期待从*Y*来的响应，但是没有收到。有以下几种可能的解释：

- 由于通讯故障，消息没有传输到*Y*。
- 站点*Y*失效而不能响应。
- 站点*Y*在运行并且发出了响应，但响应没有传输到站点*X*。

如果没有发送另外的信息或附加消息，很难判断实际上发生了什么。

另一个与分布式恢复有关的问题是分布式提交。当一个事务更新在多个站点上的数据时，直到它确认每个站点上的事务都有结果且没有丢失时才能够提交。这就意味着每个站点首先将事务执行的局部结果永久地记录在本地站点磁盘的日志上。在3.6节已讨论过的两阶段提交协议通常用于确保分布式提交的正确性。

9.6　三层客户-服务器体系结构概述

正如在本章引言中所指出的，能够支持讨论过的所有功能类型的DDBMS至今还没有完全开发出来。然而，分布式数据库应用正在客户-服务器体系结构环境中开发。我们在《数据库系统基础：初级篇》的2.5节中讨论过两层客户-服务器体系结构。现在更为普遍的则是三层体系结构，尤其在Web

应用中。这种体系结构可用图9-8表示。

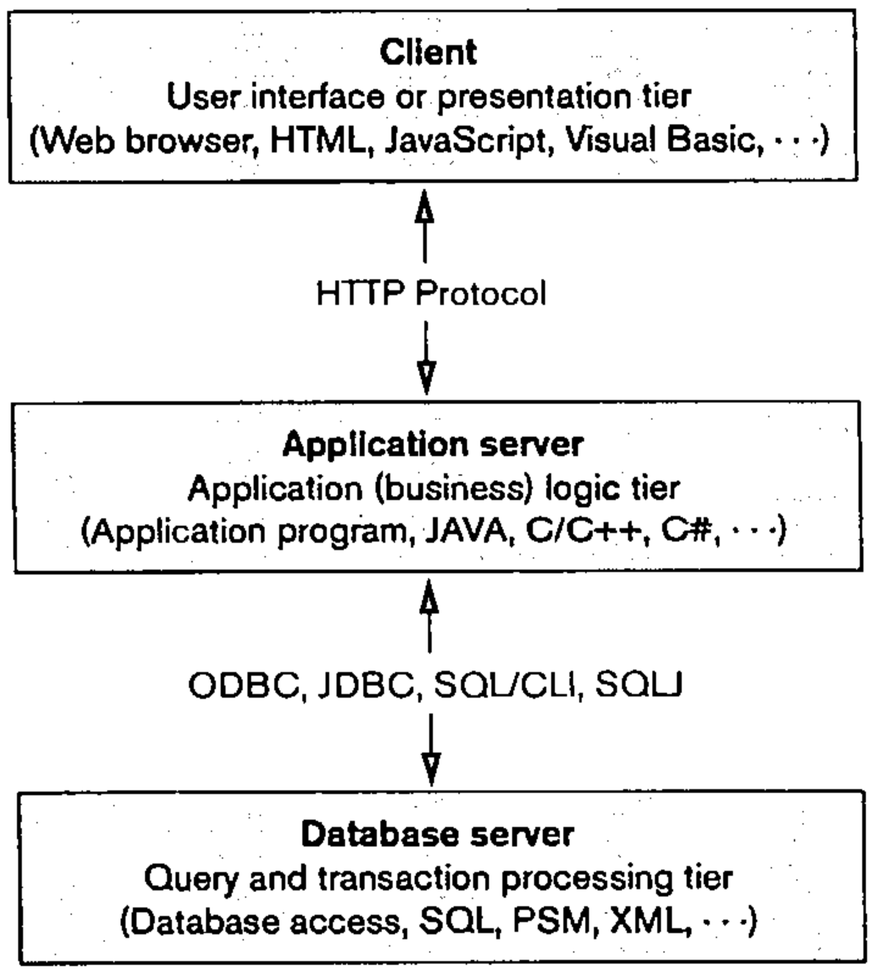


图9-8　三层客户-服务器体系结构

在三层客户-服务器体系结构中，有以下3个层次：

(1) **表示层（客户层）**

这一层提供了用户界面并同用户进行交互。这一层的程序为用户提供了Web界面或表单，以便客户与应用交互。常用的是Web浏览器，使用的语言包括HTML、JAVA、JavaScript、PERL、Visul Basic等等。表示层通过接受用户命令并向用户显示其需要的信息，以此处理用户输入、输出以及导航，向用户显示的信息通常以静态或动态Web页面的形式。当用户交互包括数据库访问时，通常采用后者。当使用Web界面时，表示层一般通过HTTP协议与应用层进行通信。

(2) **应用层（业务逻辑层）**

该层对应用逻辑进行编程。例如，基于来自客户端的用户输入可以将查询形式化，或者查询结果可以被格式化表示并发送到客户端。在这一层还可以处理附加的应用功能，例如安全检查、身份验证以及其他功能。需要时，应用层可以与一个或多个数据库或数据源进行交互，这通过ODBC、JDBC、SQL/CLI或其他的数据库访问技术连接数据库来实现。

(3) **数据库服务器层**

这一层处理来自应用层的查询与更新请求，处理请求并发送结果。如果是关系或者对象-关系数据库，通常使用SQL来访问数据库，还可能调用数据库存储过程。查询结果（以及查询）在应用服务器和数据库服务器之间传输时可以格式化为XML形式（见第11章）。

在客户机、应用服务器以及数据服务器之间如何确切地划分DBMS的功能可能会变化不定。通常的方法是将集中式DBMS的功能包含在数据库服务器层上。许多关系DBMS产品都已采用了这种方案，它们为客户机提供了一个SQL服务器。这样，应用服务器必须形式化为合适的SQL查询，并在需要时连接数据库服务器。客户端提供用户界面交互的处理。由于SQL是关系数据库语言的标准，所以不同厂商提供的各种SQL服务器都可以通过ODBC、JDBC、SQL/CLI（见《数据库系统基础：初级篇》的第9章）等标准接受SQL命令。

在这种体系结构中，应用服务器也可以引用数据字典，数据字典包含了在各种SQL服务器中的数据分布信息，还可以引用一些功能模块，这些功能模块把全局查询分解为可以在不同的站点上执行的若干个局部查询。在处理SQL查询的过程中，应用服务器和数据服务器之间的交互可以按如下

步骤进行：

(1) 应用服务器基于客户层的输入形式化用户查询，并将其分解成为若干个独立站点的子查询。然后把每个站点的查询发送到适当的数据库服务器站点。

(2) 每个数据库服务器处理局部查询并将结果关系发送到应用服务器站点。目前，把XML作为数据交换的标准（见第11章）的呼声正在日益高涨，所以在把查询结果发送到应用服务器之前，数据库服务器可能会把查询结果格式化为XML形式。

(3) 应用服务器把子查询结果合并为初始请求的查询结果，并将其格式化为HTML或其他客户端能够接受的形式，然后发送到客户端站点显示。

应用服务器负责为多站点查询或事务生成一个分布式执行计划，并通过向服务器发送命令监督分布式执行。这些命令包括将要执行的局部查询和事务，也包括向其他客户机或服务器传输数据的命令。应用服务器（或协调者）控制的另一个功能是通过使用分布式（或全局）并发控制技术，来确保数据项复制副本的一致性。当某个站点失效时，应用服务器必须通过执行全局恢复来确保全局事务的原子性。我们已经在9.5节讨论过分布式恢复和并发控制技术。

如果DDBMS能够隐藏应用服务器的数据分布细节，那么它就使得应用服务器执行全局查询和事务时如同一个集中式数据库，而不必指定查询或者事务所引用数据的所在站点。这种性质称作**分布透明性**（distributed transparency）。有些DDBMS不提供分布透明性，而是要求应用必须了解数据分布的细节。

9.7　Oracle 分布式数据库[①]

在客户-服务器体系结构中，Oracle数据库系统分为两部分：作为客户机部分的前端和作为服务器部分的后端。客户机部分是与用户交互的前端数据库应用。客户机没有数据库存取职责，而只处理被服务器管理的数据的请求、处理和表示。服务器部分运行Oracle并处理与并发共享存取相关的操作，它接受源自客户机应用的SQL和PL / SQL语句，处理它们并将结果返回给客户机。Oracle客户-服务器应用提供位置透明性，这是通过使数据的位置对用户来说是透明的方法来实现的；一些诸如视图、同义词和过程之类的特性都有助于实现位置透明性。通过采用<TABLE_NAME@DATABASE_NAME>来实现表的全局命名，从而实现引用唯一性。

Oracle采用两阶段提交协议来处理并发的分布式事务。COMMIT语句触发两阶段提交机制。RECO（recoverer）后台程序自动地处理那些提交被中断的分布式事务的输出。每个本地Oracle服务器的RECO自动地提交或回滚任何一个在所有涉及的站点上一致性有疑惑的分布式事务。对于长期故障，Oracle允许每个局部的DBA人工地提交或回滚任何有疑惑的事务并且释放资源。全局一致性可以通过在每个站点上将数据库恢复到以往某个预先确定的固定点的方法来维护。

Oracle的分布式数据库体系结构如图9-9所示。分布式数据库系统中的一个结点可以作为一个客户机、一个服务器或二者兼有之，这依情形而定。图中展示出两个站点，称为总部（headquarters，HQ）数据库和销售（Sales）数据库。例如，在图中所示的在总部运行的应用中，对于一个针对局部数据的SQL语句（例如，DELETE FROM DEPT…），HQ计算机充当一台服务器，而对于一个针对远程数据的语句（例如，INSERT INTO EMP@SALES），HQ计算机作为一台客户机。

分布式数据库系统（DDBS）中的所有Oracle数据库，使用Oracle的网络软件Net8进行数据库间通讯。Net8允许数据库通过网络与支持远程的事务和分布式事务进行通讯。它把SQL语句打包成众多通讯协议的一种，来简化客户机到服务器的通讯，并且同样地将结果打包返回给客户机。每个数据库有唯一的全局名称，它以层次排列的网络域名作为数据库名称的前缀构成，以保持其唯一性。

---

① 讨论是基于版本8.0的。Oracle 10g已被设计可以支持优化的分布式操作。

Oracle支持数据库链接，数据链接被定义为从一个Oracle数据库到其他数据库的单向通讯路径。例如：

```
CREATE DATABASE LINK sales.us.americas;
```

这个语句建立了一条从如图9-9[①]所示的域americas中的网络域us到sales数据库的连接。

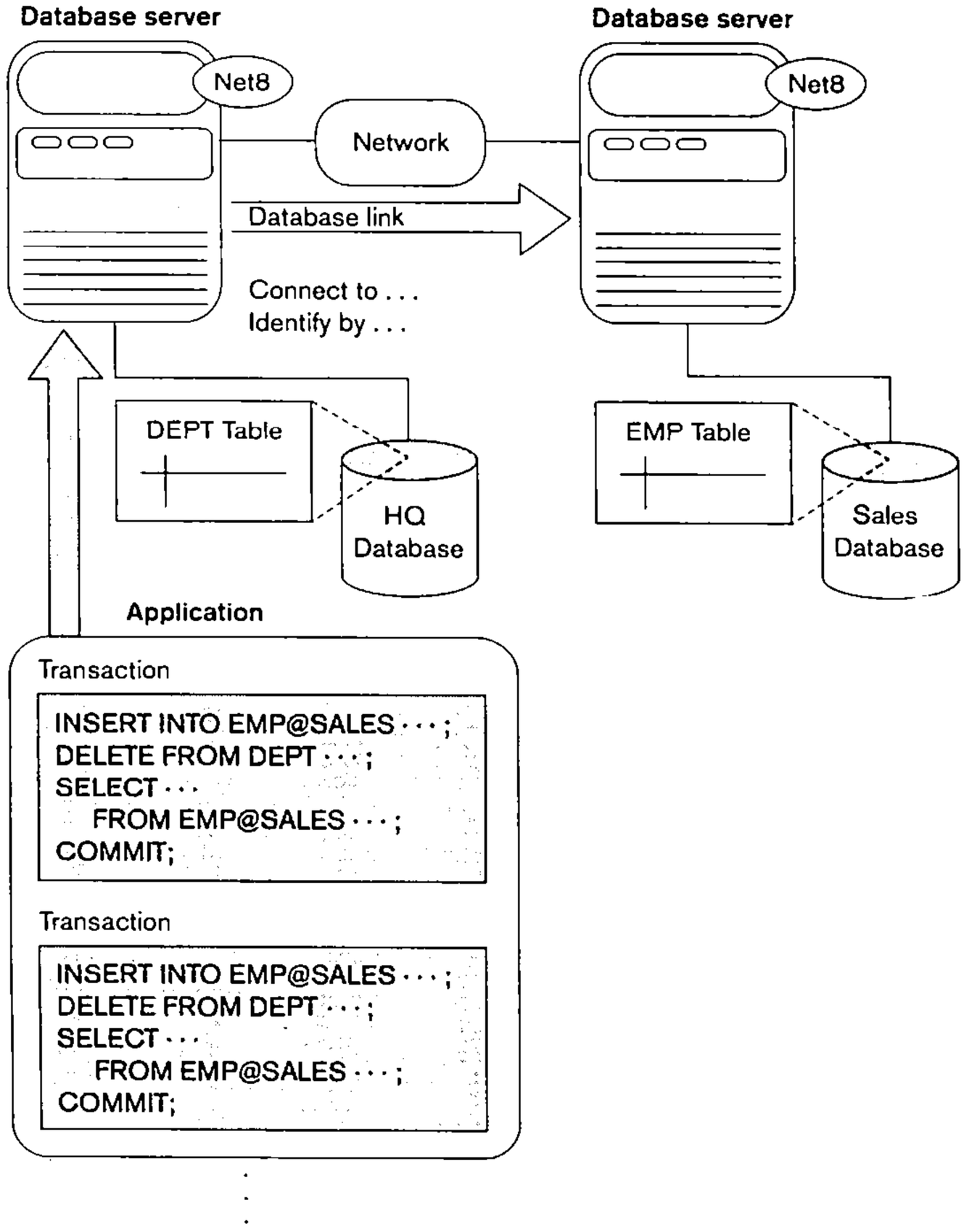


图9-9　Oracle分布式数据库系统

Oracle DDBS中的数据复制可以通过采用快照或复制主表来实现。在以下层次上提供复制：

- **基本复制**。表的副本只对只读访问管理。对于更新，数据必须在单个主站点上存取。
- **高级（对称）复制**。这种复制扩展了基本复制，它允许应用通过复制一个DDBS来更新表复制。数据可在任意站点上被读或被更新，这需要一个称为Oracle高级复制选项的附加软件。**快照**（snapshot）通过一个称为快照定义查询的查询，生成表的一部分的副本。一个简单的快照定义如下：

```
CREATE SNAPSHOT SALES_ORDERS AS
SELECT * FROM SALES_ORDERS@hq.us.americas;
```

Oracle将快照归为刷新操作组中。通过指定一个刷新间隔，最多有10个**快照刷新进程**（SNP）以此间隔自动周期性地刷新快照。如果一个快照的定义查询包含一个distinct或aggregate函数、一个

① 来源：来自于Oracle（1997a）。版权属于Oracle Corporation 1997。所有权受到保护。

GROUP BY或CONNECT BY子句、一个连接或集合操作，该快照就被称为一个**复杂快照**，并要求额外的处理。Oracle（直到版本7.3）都支持ROWID快照，它基于主表中的行的物理行标识符。

Oracle 中的异构数据库

在一个异构DDBS中，至少有一个数据库是非Oracle系统的。**Oracle开放式网关**（Oracle Open Gateway）提供了从一个Oracle服务器到一个非Oracle数据库的访问，它使用一个数据库链接来访问非Oracle数据库中的数据，或执行在非Oracle系统中远程过程。开放式网关的特性包括如下：

- **分布式事务**。在两阶段提交机制下，事务可以跨越Oracle和非Oracle系统。
- **透明的SQL存取**。应用发出的SQL语句被透明地转换成非Oracle 系统可以理解的SQL语句。
- **传递SQL和存储过程**。一个应用可以使用非Oracle 系统的各种SQL版本直接访问该非Oracle系统。基于SQL的非Oracle系统中的存储过程被作为PL/SQL的远程过程来处理。
- **全局查询优化**。非Oracle系统上的基数信息，索引等被Oracle服务器查询优化器解释，从而执行全局查询优化。
- **过程化访问**。类似于消息或排队系统的过程化系统，被Oracle服务器使用PL/SQL远程过程调用访问。

除了上述几点，非Oracle数据字典引用也被翻译，使得它看起来像Oracle服务器数据字典的一部分。为了连接多语言的数据库，还需要在国家语言字符集中做字符集的转换。

小结

在本章中我们介绍了分布式数据库，这是一个非常广泛的主题，我们只讨论了分布式数据库的一些基本技术。首先，我们讨论了使用分布式的原因和分布式数据库相对于集中式数据库的潜在优势。我们也定义了分布透明性的概念和与之相关的分片透明性和复制透明性的概念。之后我们讨论了与数据分片、复制和分布有关的设计问题，并且对关系的水平分片和垂直分片进行了区分。还讨论了使用数据复制方法来提高系统的可靠性和可用性。并通过使用诸如软件模块同构性程度和局部自治性程度的标准来对DDBMS进行分类。我们还相当详细地讨论了联邦式数据库管理的问题，着重讨论了对不同自治性类型的支持和语义异构性的处理。

我们列举了一些用于分布式查询处理的技术，并讨论了站点间的通讯代价，这在分布式查询优化中被认为是主要因素。我们比较了执行连接的不同技术，并提出了半连接技术，它被用于连接存储在不同站点上的关系。此外，还简要地讨论了DDBMS中使用的并发控制和恢复技术，回顾了一些在集中式环境中不会出现，而在分布式环境中又必须解决的其他问题。

最后，我们讨论了客户-服务器体系结构的概念，并把这些概念与分布式数据库相关联，同时描述了Oracle中支持分布式数据库的一些功能。

复习题

9.1　分布式数据库潜在的优点以及采用它的主要原因是什么？

9.2　DDBMS具有哪些集中式DBMS所不具有的功能？

9.3　DDBMS的主要软件模块是什么？在客户-服务器体系结构的环境中讨论这些模块的主要功能。

9.4　什么是关系的片段？片段有哪些主要类型？为什么分片在分布式数据库设计中是一个有用的概念？

9.5　为什么数据复制在DDBMS中很有用？什么是数据复制的典型单位？

9.6　在分布式数据库设计中数据分配的含义是什么？在站点上分布数据的典型单位是什么？

9.7　怎样指定一个关系的水平分片？关系如何从完备水平分片中还原成原关系？

9.8　怎样确定关系的垂直分片？怎样从完备垂直分片中还原关系？

9.9　讨论以下术语的含义：*DDBMS*的同构性程度、*DDBMS*的本地自治性程度、联邦*DBMS*、分布透明性、分片透明性、复制透明性、多数据库系统。

9.10　讨论分布式数据库中的命名问题。

9.11　讨论执行分布在不同站点上的两个文件的等值连接（equijoin）技术。什么是影响数据传输代价的主要因素？

9.12　讨论执行分布在不同站点上的两个文件等值连接的半连接方法。在什么条件下等值连接策略最有效？

9.13　讨论影响查询分解的因素。在查询分解过程中怎样使用防护条件和片段属性列表？

9.14　更新请求分解与查询分解有什么不同？在更新请求的分解过程中怎样使用防护条件和片段属性列表？

9.15　讨论在集中式数据库中不会出现而在分布式数据库中会影响并发控制和恢复的因素。

9.16　比较分布式并发控制中的主站点方法和主副本方法。对这两种方法分别使用备份站点各有什么影响？

9.17　在分布式数据库中什么时候使用投票和选举方法？

9.18　什么是客户-服务器DDBMS中的软件成分？比较两层的和三层的客户-服务器体系结构。

练习题

9.19　考虑COMPANY数据库的数据分布，站点2和站点3上的片段如图9-3所示，站点1上的片段如《数据库系统基础：初级篇》的图5-6所示。对于下面两个查询，至少给出两种查询分解和执行的策略。在什么条件下哪个策略会更好？

a．对于部门5的每个雇员，检索雇员的姓名和该雇员的受赡养人的姓名。

b．打印所有属于部门5，但其工作的项目不属于部门5管理的雇员的姓名。

9.20　考虑如下关系：

BOOKS(Book#, Primary_author, Topic, Total_stock, $price)
BOOKSTORE(Store#, City, State, Zip, Inventory_value)
STOCK(Store#, Book#, Qty)

Total_stock是库存书籍的总量，Inventory_value是用美元表示的库存的总存货价值。

a．给出对于BOOKSTORE关系的水平分片有意义的两个简单谓词的示例。

b．如何定义基于BOOKSTORE分片的STOCK的一个导出的水平分片？

c．展示可以按Topic对关系BOOKS进行水平分片的谓词。

d．通过添加（c）中的谓词，展示如何对（b）中的STOCK片段进一步分片。

9.21　考虑一个称为National Books的连锁书店的分布式数据库，它有3个站点，分别是EAST、MIDDLE和WEST。关系模式如练习题8.20中所示。考虑按$price对BOOKS进行分片得到：

B_1: BOOK1: $price up to $20
B_2: BOOK2: $price from $20.01 to $50
B_3: BOOK3: $price from $50.01 to $100
B_4: BOOK4: $price $100.01 and above

同样地，按ZIP代码对BOOK_STORES进行分片得到：

S_1: EAST: Zip up to 35000
S_2: MIDDLE: Zip 35001 to 70000
S_3: WEST: Zip 70001 to 99999

假设STOCK是一个仅基于BOOKSTORE导出的片段。

a．考虑如下查询：

```
SELECT Book#, Total_stock
FROM Books
WHERE $price > 15 AND $price < 55;
```

假设BOOKSTORE的片段是无复制的，并且是基于地区分配的。进一步假设BOOKS被分配如下：

EAST: B_1, B_4
MIDDLE: B_1, B_2
WEST: B_1, B_2, B_3, B_4

假设查询在EAST提交，它将生成什么样的远程子查询（用SQL编写）？

b. 如果在站点MIDDLE上，Book#=1234的书价从$45更新到$55，将产生怎样的更新？先用中文描述，然后再用SQL编写。
c. 给出一个在WEST发出的查询的示例，它将为MIDDLE生成一个子查询。
d. 编写一个包含对上述关系的选择和投影的查询，然后展示两种表示不同执行方法的可能的查询树。

9.22 假设你被要求为一个大型组织（例如通用汽车公司）设计一个数据库体系结构，要求合并包含遗留数据库（附录D和E中描述的层次和网状模型，不需要这些模型的特定知识）和关系数据库中的所有数据，这些数据在地理上是分布的，以便于支持全局应用。假设方案一是使所有的数据库保持不变，方案二首先把它们转换成为关系模型，然后支持在一个分布式集成数据库上的应用。

a. 画出上述两种方案的模式图，展示合适模式之间的连接。对于方案一，选出为每个数据库提供输出模式和为每个应用构建统一模式的方法。
b. 对于每种方案，列出从当前状态到全局应用的可行步骤。
c. 比较这两个问题：(i) 设计时间的考虑；(ii) 运行时间的考虑。

参考文献

（Ceri & Pelagatti，1984a）和（Ozsu & Valduriez，1999）致力于分布式数据库的研究。（Halsaal，1996）、（Tannenbaum，1996）和（Stallings，1997）致力于数据通信和计算机网络的研究。（Comer，1997）论述了网络和Internet。（Dewire，1993）介绍了客户-服务器计算。（Ozsu et al.，1995）是有关分布式对象管理的论文集。

分布式数据库设计对水平和垂直分片、分配和复制进行了论述。（Ceri et al.，1982）定义了最小水平分片的概念。（Ceri et al.，1983）对于水平分片和分配开发了一个基于整数编程的优化模型。（Navathe et al.，1984）开发了基于属性相似性的垂直分片算法，并给出了为垂直片段分配的各种环境。（Wilson & Navathe，1986）提出了一个片段最优分配的分析模型。（Elmasri et al.，1987）论述了ECR模型的分片。（Karlapalem et al.，1994）论述了对象数据库的分布式设计问题。（Navathe et al.，1996）论述了组合水平分片和垂直分片的混合分片；（Karlapalem et al.，1996）提出了分布式数据库的重设计模型。

分布式查询处理、优化和分解在（Hevner & Yao，1979）、（Kerschberg et al.，1982）、（Apers et al.，1983）、（Ceri & Pelagatti，1985）、（Bodorick et al.，1992）中有详细的论述。（Bernstein & Goodman，1981）论述了半联结处理所基于的原理。（Wong，1983）讨论了关系分片中联系的用法。并发控制和恢复模式在（Bernstein & Goodman，1981a）中有所论述。（Kumar & Hsu，1998）描述了一些关于分布式数据库恢复的内容。（Garcia-Molina，1982）讨论了分布式系统中的选举。（Lamport，1978）论述了分布式系统中与生成唯一时间戳有关的问题。

有关基于投票方法的复制数据的并发控制技术由（Thomas，1979）提出。（Gifford，1979）提出了加权投票方法的使用，（Paris，1986）描述了一种称为有证人投票的方法。（Jajodia & Mutchler，1990）论述了动态投票方法。一种称为有效副本（available copy）的技术由（Bernstein & Goodman，1984）提出，而一种采用分组思想的方法由（ElAbbadi & Toueg，1988）阐述。其他的一些讨论复制数据的文献包括

（Gladney，1989）、（Agrawal & ElAbbadi，1990）、（ElAbbadi & Toueg，1990）、（Kumar & Segev，1993）、（Mukkamala，1989）和（Wolfson & Milo，1991）。（Bassiouni，1988）论述了DDB并发控制的乐观协议。（Garcia-Molina，1983）以及（Kumar & Stonebraker，1987）论述了采用事务语义学的技术。基于加锁和识别副本的分布式并发控制技术在（Menasce et al.，1980）以及（Minoura & Wiederhold，1982）中有所阐述。（Obermark，1982）提出了分布式死锁检测的算法。

分布式系统中恢复技术的综述在（Kohler，1981）中给出。（Reed，1983）论述了关于分布式数据的原子动作。（Bhargava，1987）描述了关于分布式系统中的并发和可靠性的多种方法和技术。

联邦式数据库系统最早是由（McLeod & Heimbigner，1985）定义的。联邦式数据库中的模式集成技术由（Elmasri et al.，1986）、（Batini et al.，1986）、（Hayne & Ram，1990）、（Motro，1987）阐述。（Elmagarmid & Helal，1988）以及（Gamal-Eldin et al.，1988）讨论了异构DDBS的更新问题。异构分布数据库问题在（Hsiao & Kamel，1989）中论述。（Sheth & Larson，1990）对联邦式数据库管理问题进行了详细的综述。

近期，多数据库系统和互操作性已成为重要的主题。在多数据库中处理语义不兼容性的技术在（DeMichiel，1989）、（Siegel & Madnick，1991）、（Krishnamurthy et al.，1991）和（Wang & Madnick，1989）中进行了分析。（Castano et al.，1998）对模式分析技术做了一个很好的综述。（Pitoura et al.，1995）论述了多数据库系统中的对象定位。

多数据库中的事务处理在（Mehrotra et al.，1992）、（Georgakopoulos et al.，1991）、（Elmagarmid et al.，1990）和（Brietbart et al.，1990）中有所讨论。（Elmagarmid et al.，1992）论述了高级应用的事务处理，其中包括（Heiler et al.，1992）论述的工程应用。

在复杂的组织结构中普遍用于管理信息的工作流系统，在与分布式数据库连接时采用多层次和嵌套的事务。（Weikum，1991）论述了多层次事务管理。（Alonso et al.，1997）论述了当前工作流系统的局限性。

许多实验型的分布式DBMS已经实现。这些系统包括（Epstein et al.，1978）的分布式INGRES，（Devor & Weeldreyer，1980）的DDTS，（Rothnie et al.，1980）的SDD-1，（Lindsay et al.，1984）的System R*，（Ferrier & Stangret，1982）的SIRIUS-DELTA和（Smith et al.，1981）的MULTIBASE。联邦式DDBMS的例子有：（Rusinkiewicz et al.，1988）论述的OMNIBASE系统，和采用Candide数据模型开发的联邦信息库（Federated Information Base）（Pitoura et al.，1995）给出了一个有关联邦式数据库系统原型的比较综述。大多数商业DBMS厂商都已发布采用客户-服务器方式的产品，并提供了这些系统的分布式版本。一些涉及客户-服务器DBMS体系结构的系统问题在（Carey et al.，1991）、（DeWitt et al.，1990）和（Wang & Rowe，1991）中有所论述。（Khoshafian et al.，1992）论述了客户-服务器环境下关系型DBMS的设计问题。客户-服务器管理的问题在很多书中都有论述，例如（Zantinge & Adriaans，1996）。

Part 4

第四部分

新兴技术

本部分内容

- 第 10 章　使用 PHP 进行 Web 数据库程序设计
- 第 11 章　XML：可扩展标记语言
- 第 12 章　数据挖掘概念
- 第 13 章　数据仓库和 OLAP 概述
- 第 14 章　新兴的数据库技术及应用

第10章

使用PHP进行Web数据库程序设计

我们现在把目光转向在Internet上如何使用和访问数据库。许多电子商务和其他Internet应用提供了Web接口，以便访问存储在一个或多个数据库中的信息。这些数据库一般被称为**数据源**。通常在Internet应用中会使用二层或三层的客户-服务器体系结构（见《数据库系统基础：初级篇》的2.5节）。在某些情况下，也使用客户-服务器模型的其他变种。电子商务和其他Internet数据库应用通过显示网页的Web界面与用户交互。指定内容和格式化Web页面的一般方法是使用**超文本文档**。编写这种文档有多种语言，最常见的是超文本标识语言（HTML）。虽然HTML被广泛用于格式化和结构化Web文档，可是它并不适用于指定从数据库中抽取的结构化数据。于是一种新的语言，称为可扩展标识语言（XML）应运而生，并成为Web上构建和交换数据的标准。XML可以被用来提供关于网页中的数据的结构和含义的信息，而不仅仅是指定如何格式化网页以便在屏幕上显示。格式化是可以被分别指定的，例如，通过使用一种格式化语言，例如可扩展样式语言（XSL）。

基本HTML可以用来生成带有固定文本和其他对象的静态网页。但是大多数电子商务应用需要网页向用户提供交互特性。例如，考虑以下这种情况：一个顾客想要查看某特定航班的到达时间和出发信息，顾客首先在网页的某些表单项中填写诸如日期和航班号等信息，然后，为了获取这些信息，Web程序必须向航班数据库提交一个查询，并随后显示这些信息。在这样的网页中，其部分信息是从数据库或其他数据源中获取的，这种网页称为动态网页，因为每次被抽取和被显示的数据将是不同的航班和日期。

在网页中进行动态特性程序设计有各种各样的技术。这里我们只关注一项基于使用PHP开源脚本语言的技术。PHP近期得到广泛的应用。PHP相关的解释器是免费提供的，并且是用C语言编写的，这样在大多数计算机平台上都是可用的。一个PHP解释器提供了超文本预处理器，这个超文本预处理器可以执行文本文件中的PHP命令，并创建所需的HTML文件。为了访问数据库，需要在PHP解释器中包含PHP函数库，我们在10.4节中将讨论这个问题。PHP程序是在Web服务器计算机中执行的。这一点与一些脚本语言不同，例如JavaScript，它是在客户端计算机上执行的。

本章的结构如下。10.1节讨论如何区分在网页中显示的信息与储存在结构化数据库中的信息，并讨论结构化、半结构化和非结构化信息的不同之处。10.2节给出了一个简单的示例来说明PHP语言是如何使用的。10.3节给出了PHP语言的概述，以及它在动态网页基本功能程序设计中是如何使用的。10.4节重点描述使用PHP通过称为PEAR DB的函数库与SQL数据库交互。最后，是本章的小结。在第11章中，我们将讨论用于Web数据表达和交换的XML语言，以及讨论它的一些使用方法。

10.1 结构化、半结构化以及非结构化数据

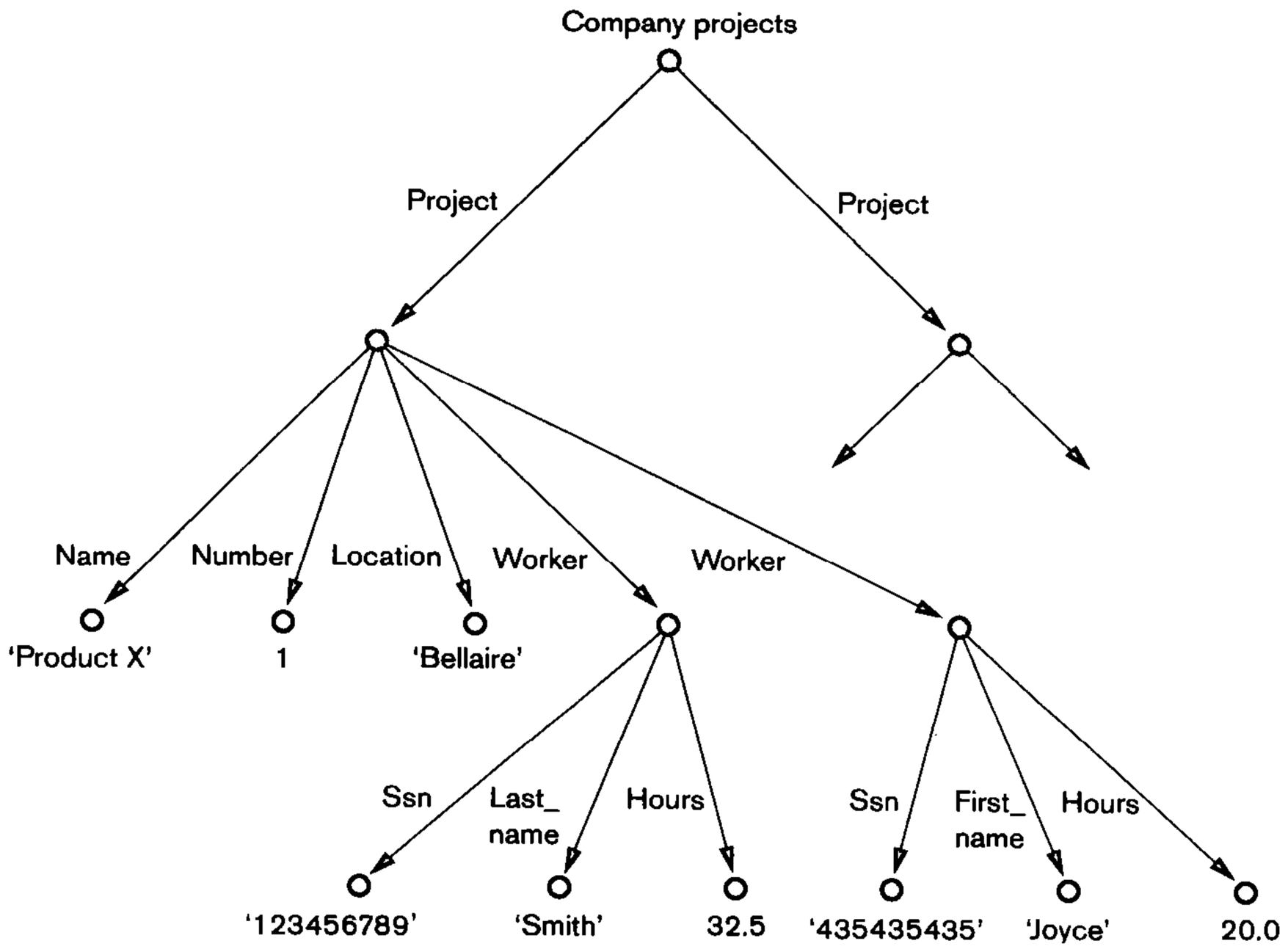
存储在数据库中的数据是**结构化数据**，因为它是以严格的格式表示的。例如，在关系数据库表中的每条记录，例如《数据库系统基础：初级篇》中图5.6的EMPLOYEE表，该表中的所有记录都遵循同一格式。对于结构化数据，为了创建数据库模式，一般都应用如《数据库系统基础：初级篇》中第3、第4、第7、第10和第11章中所描述的那些技术来详细设计数据库。然后DBMS将进行检查以

确保所有数据遵循模式中所指定的结构和约束。

但是，并不是所有的数据都能被汇集并都能插入到设计好的结构化数据库中。在某些应用中，在知道数据将被如何存储和管理之前，就已经以特定的模式汇集。这些数据可能有特定的结构，但不是所有汇集来的信息都有同样的结构。可能某些属性会被多个实体所共享，但另一些属性可能仅存在于少量实体中。此外，在一些较新的数据项中随时可能会被引入附加属性，而且没有预先设计好的模式。这种类型的数据称为**半结构化数据**。为了表示半结构化数据，已经引入了大量的数据模型，这些模型一般都是基于树或图数据结构，而不是基于平面关系模型结构。

结构化数据和半结构化数据的关键区别在于模式结构（例如属性、联系和实体类型的名称）是如何被处理的。在半结构化数据中，模式信息与数据值混合在一起，因为每个数据对象可能有不同的、预先无法知道的多个属性。于是，这种类型的数据有时被称为**自描述数据**。考虑下面这个示例。我们想要汇集一份关于某研究项目的参考文献的清单。这些文献有些可能是书或技术报告，有些可能是期刊或会议论文集中的研究文章，有些甚至可能是整份期刊或会议论文集。显而易见，它们中的每一种都可能有不同的属性和不同的信息类型。就算是对于同类型的参考文献，例如会议论文，我们也有可能得到不同的信息。例如，一篇文章的引证或许非常完整，有着关于作者姓名、题目、论文集、页码等的完整信息，而其他文章的引证或许没有这样完整的信息。未来或许会出现新类型的参考文献来源，比如说网页或会议手册，而这些新类型的参考文献来源可能会有新的属性来描述它们。

半结构化数据可以用有向图来表示，如图10-1所示。图10-1中显示的信息与《数据库系统基础：初级篇》的图5-6中显示的结构化数据是对应的。我们可以看到，这种模型在表示复杂对象和嵌套结构的能力方面与对象模型（见图4-1）是类似的。图10-1中，用带箭头直线上的**标号**或**标签**表示模式名：属性名、对象类型（或实体类型或类）以及联系。内部结点表示单个对象或组合属性。叶结点表示简单（原子）属性的实际数值。



10-1　半结构化数据的图示

正如我们在第4章中所讨论的，半结构化模型和对象模型之间有两个主要区别：

(1) 模式信息：半结构化模型中的属性、联系和类（对象类型）的名称与对象及其数据值混合在同一数据结构中。

(2) 在半结构化模型中，不需要让数据对象遵循预先定义好的模式。

除了结构化和半结构化数据之外，还有第三类数据存在，这就是**非结构化数据**，因为这种类型的数据表示有非常多的限制。一个代表性的示例是包含了嵌套信息的文本文档。包含一些数据的HTML网页被认为是非结构化数据。考虑HTML文件的片段，如图10-2所示。在尖括号<…>之间的文本是**HTML标签**。包含反斜线符号的标签</…>表示**结束标签**，它表示与之匹配的**开始标签**的结束。为了指示HTML处理器如何显示开始标签和对应结束标签之间的文本，这种标签在文档上做了**标记**[①]。因此，这些标签指定了文档格式而不是文档中各种数据元素的含义。HTML标签指定了信息，例如文档中的字体大小和形式（黑体、斜体等）、颜色、标题级别等等。一些标记提供了文档的文本结构，例如指定有编号的或无编号的清单或表单。即使这些结构化的标签指定了嵌入式文本数据可以以某种方式显示，也不等于指明了表中所表示的数据的类型。

```
<HTML>
   <HEAD>
   . . .
   </HEAD>
   <BODY>
      <H1>List of company projects and the employees in each project</H1>
      <H2>The ProductX project:</H2>
      <TABLE width="100%" border=0 cellpadding=0 cellspacing=0>
         <TR>
            <TD width="50%"><FONT size="2" face="Arial">John Smith:</FONT></TD>
            <TD>32.5 hours per week</TD>
         </TR>
         <TR>
            <TD width="50%"><FONT size="2" face="Arial">Joyce English:</FONT></TD>
            <TD>20.0 hours per week</TD>
         </TR>
      </TABLE>
      <H2>The ProductY project:</H2>
      <TABLE width="100%" border=0 cellpadding=0 cellspacing=0>
         <TR>
            <TD width="50%"><FONT size="2" face="Arial">John Smith:</FONT></TD>
            <TD>7.5 hours per week</TD>
         </TR>
         <TR>
            <TD width="50%"><FONT size="2" face="Arial">Joyce English:</FONT></TD>
            <TD>20.0 hours per week</TD>
         </TR>
         <TR>
            <TD width= "50%"><FONT size="2" face="Arial">Franklin Wong:</FONT></TD>
            <TD>10.0 hours per week</TD>
         </TR>
      </TABLE>
   . . .
   </BODY>
</HTML>
```

图10-2　表示非结构化数据的一个HTML文件的片断

① 这就是为什么它被称作超文本标识语言。

HTML使用了大量预定义标签，可以用来定义多种格式化Web文档的命令。开始和结束标签指定了每条命令所格式化的文本的范围。在图10-2中展示的一些标签的示例其含义如下：

- <HTML>…</HTML>标签指定了文档的边界。
- **文档头**信息是在<HEAD>…</HEAD>间的信息，它指定了在文档中的其他地方可能使用的各种命令。例如，它可以定义文档中可能用到的多种**脚本函数**或者**格式化形式**（包括字体、段落、标题样式等）。它同样可以指定一个题目来表明该HTML文件的目的，以及其他不会在文档中显示的类似的信息。
- 文档的**体**在<BODY>…</BODY>标签间指定，它包括了文档文本以及指定文本是如何被格式化和显示的标记标签。它还包括了对其他对象的引用，例如图像、视频、声音消息以及其他文档。
- <H1>…</H1>标签指定文本被显示为1级标题。有许多种标题等级（<H2>、<H3>等等），每一种都将以逐级变小的标题格式显示文本。
- <TABLE>…<TABLE>标签指定紧跟其后的文本将以表单形式显示。表中的每一行被包括在<TR>…</TR>标签之中，而一行中的实际文本数据被显示在<TD>…</TD>标签中[①]。
- 一些标签可以有**属性**，在开始标签中出现，描述了标签的其他特性[②]。

在图10-2中，开始标签<TABLE>有4个用于描述表的各种特性的属性。之后的开始标签<TD>和<FONT>分别也有1个和2个属性。

HTML有着大量的预定义标签，以至于需要著书来描述这些标签是如何使用的。如果设计得好，HTML文档可以被格式化，这样人们可以轻易地理解文档的内容，且能够浏览，并由之产生WEB文档。但是，源HTML文本文档是非常难以由计算机程序自动解释的，因为它们没有归纳关于文档中数据类型的模式信息。随着电子商务和其他网络应用日渐自动化，在各个网站间交换Web文档并自动解释其内容变得越来越重要。这种需求就是导致XML发展的原因之一，在第11章中我们会讲到XML。

图10-2中的示例展示了一个**静态**HTML网页，因为显示的所有信息已被HTML文件中的固定文本清楚地说明。在许多情况下，一些显示的信息或许是从数据库中抽取出来的。例如，项目名称和工作于每个项目的雇员是从《数据库系统基础：初级篇》中图5-6所显示的数据库中，通过适当的SQL查询抽取出来的。我们可能想要使用相同的HTML格式化标签来显示每个项目和在其中工作的相关雇员，但我们也可能希望改变显示的某些项目（及雇员）。例如，我们可能希望看到显示项目*ProjectX*信息的网页，稍后再显示关于项目*ProjectY*信息的网页。尽管两个网页都通过相同的HTML格式化标签显示出来，实质上显示的数据项是不同的。这样的网页称为**动态的**，因为页面的数据部分每次显示时都是不同的，即使表示形式是相同的。

有多种技术可以定义这些动态网页。本章的剩余部分将给出一种关于使用PHP脚本语言技术定义动态网页的概述。

10.2　一个简单的 PHP 示例

PHP是一种开源的普通目的的脚本语言。PHP的解释器引擎是使用C语言编写的，这样它就可以在几乎所有类型的计算机和操作系统上使用。PHP一般在UNIX操作系统安装时就一起安装了。对于其他计算机平台，如Windows、Linux或Mac OS，PHP解释器可以从以下网址下载：http://www.php.net。与其他的脚本语言一样，PHP特别适合操作文本页面，尤其适合在Web服务器上操纵动态HTML网页。这一点与JavaScript相反，JavaScript是将网页下载至客户端后再执行的。

① <TR>表示表行而<TD>表示表中数据。

② 这就是属性在文档标识语言中是如何应用的，与其在数据库模型中的用法不同。

PHP中有函数库，可以用来访问所有支持ODBC标准（见《数据库系统基础：初级篇》的第9章）的、存储于Oracle、MySQL、SQLServer等各种类型的关系数据库系统中的数据库。在三层体系结构（见《数据库系统基础：初级篇》的第2章）下，DBMS处于最底层的数据库服务器。PHP会运行在中间层的Web服务器上，在这里PHP命令可以操纵HTML文件来创建客户定制的动态网页，这些网页会随即传送至客户层来显示并与用户交互。

考虑图10-3a展示的示例，该示例提示用户输入姓名，然后向用户输出一条欢迎信息。行号并不是代码的一部分，使用它们只是为了便于解释。

(1) 假设包含程序段P1中的PHP脚本的文件存储于以下网址：http://www.myserver.com/example/greeting.php。那么如果某用户在浏览器中输入该网址，PHP解释器将开始解释代码并产生如图10-3b所示的表单。当我们阅读代码段P1时我们就会解释这一切是如何发生的。

(2) 第0行表示的是PHP的开始标签“<?php”，用来指示PHP解释器引擎，它处理后续的文本行，直到遇到第16行所示的PHP结束标签“?>”。在这些标签之外的文本会按其样式打印出来。这使得PHP程序段被包含在一个较大的HTML文件中。只有在“<?php”和“?>”之间的文件部分会由PHP预处理器处理。

(3) 第1行显示了在PHP程序中添加注释的一种方法，即在一行的开始用“//”表示。单行注释也可以用“#”开始，在其输入的行末端结束。多行注释以“/*”开始，以“*/”结束。

(4) 自动-全局预定义PHP变量$_POST（第2行）是一个包含了所有通过形式参数输入值的数组。PHP中的数组是动态的，而且没有固定元素号。它们可以是以数字（0，1，2，…）等作为索引（位置）的索引数组，也可以是索引是任何字符串值的关联数组。例如，基于颜色的索引关联数组可以有这样的索引{“red”、“blue”、“green”}。在这个示例里，$_POST是关联索引，为显示值user_name的名称，这在第10行的输入标签的名称属性处进行了定义。这样，$_POST['user_name']将会包含用户输入的值。在10.3.2节中我们会进一步讨论PHP数组。

(5) 当http://www.myserver.com/example/greeting.php处的网页被初次打开时，第2行的if-condition将估算为false，因为在$_post['user_name']中还没有值。所以，PHP解释器将处理第6至15行，这一段是为一个HTML文件创建的一段文本，用于显示如图10-3b所示的形式。

(6) 第8行显示了一种在HTML文件中创建长文本串的方法。我们稍后会在本节中讨论其他定义串的方法。所有在开始标签“<<<_HTML_”和结束标签“_HTML_;”之间的文本会依原样写入HTML文件。结束标签“_HTML_;”必须单独占一行。因此，编写发送到客户端HTML文件的文本会是第10行和第13行之间的部分。这包括了用于创建如图10-3b所示形式的HTML标签。

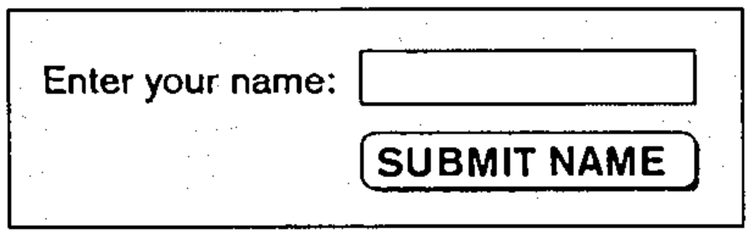
(7) PHP变量名称以“$”标记开始，它包括字符、数字以及下划线字符“_”。PHP自动-全局（预定义）变量$_SERVER（第9行）是一个包含本地服务器信息的数组。该数组中的元素$_SERVER['PHP_SELF']是正在服务器上执行的PHP文件的路径名。所以，一旦形式参数被用户输入，形式标签（第9行）的动作属性指示PHP解释器再次处理同一个文件。

(8) 当用户在文本框中输入姓名*John Smith*，并且点击*SUBMIT NAME*按钮（图10-3c）时，程序段*P1*会被再次执行。这一次，$_POST['user_name']会包括字符串 "John Smith"，所以第3和第4行会被放入发送到客户端的HTML文件中，这在图10-3d的消息中显示。

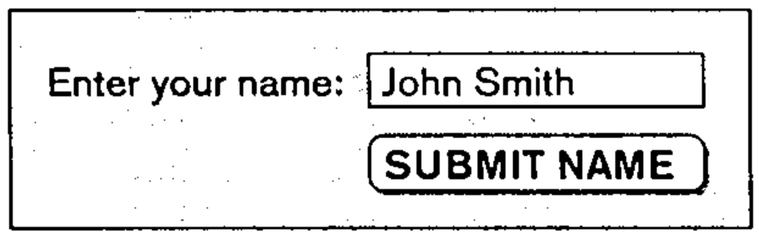
从这个示例我们可以看到，PHP程序可以按照用户是否刚开始或者用户是否已经通过表单提交了他们姓名这两种情况，产生两种不同的HTML命令。一般来说，根据程序中特定条件路径的不同，PHP程序可以在服务器上的HTML文件中生成大量HTML文本的变种。所以，发往客户的HTML文件会根据与用户交互情况的不同而不同。这是PHP用来创建动态网页的一种方法。

```
//Program Segment P1:
 0) <?php
 1) // Printing a welcome message if the user submitted their name
    // through the HTML form
 2) if ($_POST['user_name']) {
 3)   print("Welcome,  ") ;
 4)   print($_POST['user_name']);
 5) }
 6) else {
 7)   // Printing the form to enter the user name since no name has
      // been entered yet
 8)   print <<<_HTML_
 9)   <FORM method="post" action="$_SERVER['PHP_SELF']">
10)   Enter your name: <input type="text" name="user_name">
11)   <BR/>
12)   <INPUT type="submit" value="SUBMIT NAME">
13)   </FORM>
14)   _HTML_;
15) }
16) ?>
```

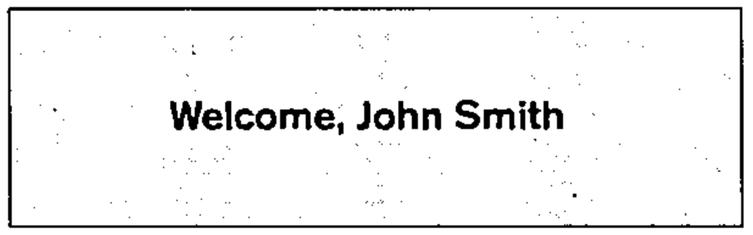
(a) 输入一个问候的PHP程序段



(b) PHP程序段显示的初始窗体



(c) 用户输入名字*john smith*



(d) 窗体打印欢迎*john smith*的信息

图　10-3

10.3　PHP 基本特性概述

在本节中我们将给出一些PHP特性的概述，这些特性有利于创建交互式HTML页面。下一节将集中介绍PHP程序是如何访问数据库来进行查询和更新的。我们将不对PHP作综合性的讨论，因为在这一方面已有一些专门的书籍。取而代之的，我们将重点介绍PHP特别适合创建包含数据库访问命令的动态网页的一些特性。尽管本节没有涵盖PHP的数据库访问功能，但包含了PHP的其他一些概念和特性，在10.4节讨论数据库访问时会用到这些概念和特性。

10.3.1　PHP 变量、数据类型和程序设计结构

PHP变量名以$符号开始，并且包含字符、字母、下划线符号（_）。其他特殊字符是不允许的。变量名是区分大小写的，且首字符不能是数字。变量是没有类型的，赋予变量的值决定了它们的类型。事实上当一个变量被赋予新值时其类型也就改变。赋值是通过“＝”操作符进行的。

由于PHP是针对文本处理的，串值有若干种不同的类型，还有许多用于处理串的函数。在这里我们只讨论串值和变量的几种基本性质。图10-4说明了一些串值，表示串和文本有如下3种主要的方法：

(1) 单引号串

用单引号将串括起来，如第0、第1和第2行所示。如果该串中要求有单引号符，使用转义字符反斜杠（\）（见第2行）。

(2) 双引号串

用双引号将串括起来，如第7行所示。在这种情况下，串中的变量名被当前储存于该变量中的值所代替。解释器根据起始字符$来识别以双引号括起来的串中的变量名，并使用该变量中的值来替换它们。这就是所谓的插值变量（interpolating variable）。在单引号串中不发生插值。

(3) 此处文档

包括在以<<<DOCNAME开头，并以只含文档名DOCNAME的一个单行结束的一段文档。只要被用来作为此处文档的开始和结束，这个DOCNAME可以是任何的串。如图10-4中的第8～11行所示。当变量出现在此处文档中时，也会因以它们的串值替代变量而被插值。

(4) 单引号和双引号

PHP中的用于括住串的单引号和双引号将由一个文本编辑器创建，该文本编辑器在串的周围不产生斜体的开和闭引号（""），在该串的两头它们将是直体的引号（""）。

还有一种串连接操作符，用句号（.）指定，如图10-4中第6行所示。有许多串函数，这里我们仅介绍其中一对。函数strtolower将串中的字母字符改为全小写，而函数ucwords将串中所有字母改为全大写。如图10-4中第4和第5行所示。

```
0) print 'Welcome to my Web site.';
1) print 'I said to him, "Welcome Home"';
2) print 'We\'ll now visit the next Web site';
3) printf('The cost is $%.2f and the tax is $%.2f', $cost, $tax) ;
4) print strtolower('AbCdE');
5) print ucwords(strtolower('JOHN smith'));
6) print 'abc' . 'efg'
7) print "send your email reply to: $email_address"
8) print <<<FORM_HTML
9) <FORM method="post" action="$_SERVER['PHP_SELF']">
10) Enter your name: <input type="text" name="user_name">
11) FORM_HTML
```

图10-4　基本PHP串和文本值示例

一般的规则是对于不含任何PHP程序变量的文本串使用单引号串，而当需要将变量值更改并放入串中时使用其他两种类型（双引号串和此处文档）。对大块的多行文本，程序应当对串使用此处文档类型。

PHP也含有大量的数值数据类型来表示整数和浮点数，并且一般来说遵循C语言处理这些类型数据的规则。数字在写入串之前将被格式化，按十进制指定其数字位数。打印函数的一个变种printf（格式化打印）可对串内数字进行格式化，如图10-4中第3行所示。

PHP中还有由for循环、while循环和条件if语句组成的标准程序设计语言结构。它们一般与C语言中的相对应语句的含义类似。我们在此就不多赘述了。类似地，如果使用了布尔表达式，除了数值0和空串为假外，任何值都为真。也可以赋予文字的真值和假值。比较运算符一般也遵循C语言的规则。它们是==（相等）、!=（不等）、>（大于）、>=（大于等于）、<（小于）、<=（小于等于）。

10.3.2　PHP 数组

PHP中的数组是非常重要的，因为它们允许元素列表得以存在。它们经常使用在下拉式菜单这样的表单中。一维数组用于保存下拉菜单中的选项列表。对于数据库查询，二维数组的第一维表示表的行，而另一维表示行中的属性。主要的数组有两种类型：数字型和关联型。我们在下面的单维数组的上下文中将对它们进行讨论。

数值数组将数组中的所有元素与一个数值索引（或位置或序列号）相关联起来。索引是从0开始并依次递增的整数。每个元素通过其索引被访问。**关联数组**提供元素对（key => value）。每个元素值通过其码被访问，而在一个特定数组中的所有码值必须是唯一的。元素值可以是串或整数，也可以是数组本身，这样会产生更高维的数组。

图10-5给出了数组变量的两个示例：$teaching和$courses。第一个数组$teaching是关联型的（见图10-5第0行），每个元素将课程名（作为码）和课程教师（作为值）相关联。在这个数组中有3个元素。第1行展示了数组如何更新。第1行的第1个命令通过更新课程“Graphics”的值将它的教师替换。因为码值已经存在，没有创建新的元素，但是现有的值被更新了。第2个命令创建了一个新的元素，因为码值“Data Mining”在之前的数组中并不存在，新元素被添加在数组的末尾。

```
0)  $teaching = array('Database' => 'Smith', 'OS' => 'Carrick',
                      'Graphics' => 'Kam');
1)  $teaching['Graphics'] = 'Benson'; $teaching['Data Mining'] = 'Kam';
2)  sort($teaching);
3)  foreach ($teaching as $key => $value) {
4)    print " $key : $value\n";}
5)  $courses = array('Database', 'OS', 'Graphics', 'Data Mining');
6)  $alt_row_color = array('blue', 'yellow');
7)  for ($i = 0, $num = count($courses); i < $num; $i++) {
8)    print '<TR bgcolor="' . $alt_row_color[$i % 2] . '">';
9)    print "<TD>Course $i is</TD><TD>$course[$i]</TD></TR>\n";
10) }
```

图10-5　基本PHP数组处理示例

如果我们只提供值（即无码）作为数组元素，码会自动取数值，并被赋以0，1，2，…，如图10-5中的第5行$courses数组所示。关联数组和数值数组都没有大小限制。如果其他数据类型的一些值，比如整型，被赋予持有一个数组的PHP变量，该变量将持有一个整型值，而且该数组内容将被丢失。基本上说，几乎所有变量在任何时候都可以被赋予任何数据类型的值。

有几种不同的遍历PHP数组的技术。我们在图10-5中展示了其中两种方法。第3行和第4行展示了一种使用*for each*结构，遍历数组中所有元素的方法，并打印一行的码和每个元素的值。第7～10行展示了传统的*for*循环结构是如何使用的。一个内置的函数count（第7行）返回了数组中元素的当前值，该值被赋给变量$num并用来控制结束循环。

这个示例的第7～10行也展示了HTML表是如何使用交替行颜色来显示的，通过在数组$alt_row_color（第8行）中设定两种颜色来实现。每次循环，求余（remainder）函数$i%2从一行（index 0）转至下一行（index 1）（见第8行）。颜色被赋予<TR>（表行）标签的HTML bgcolor属性。

count函数（第7行）返回数组元素的当前个数。sort函数（第2行）根据数组中的元素值（非码）将数组排序。对于关联数组，每个码仍保持与在排序后的同一元素值的关联，这在对数值数组排序时不会发生。还有许多可应用于PHP数组的其他函数，但是对它们完整的讨论超出了我们介绍的范围。

10.3.3　PHP 函数

如同其他程序设计语言那样，可以在PHP中自定义函数以便更好地设计一个复杂的程序，并在多个应用中共享可重用的公用代码。PHP的较新版本PHP5也有面向对象的特征，不过我们在此不作讨论，因为我们只关注PHP的基本特征。基本PHP函数可以有通过值传递的变元，可以在函数中访问全局变量。标准范围（standard scope）规则适用于在函数中出现的变量，以及在调用该函数的代码中出现的变量。

我们在此给出两个简单的示例来介绍基本PHP函数。图10-6展示了如何使用函数来重写图10-3a中的代码段P1。图10-6中的代码段P1′有两个函数：display_welcome()（第0～3行）以及display_empty_form()（第5～13行）。这两个函数都没有变元，也都没有返回值。第14～19行展示了可以调用这些函数来产生与图10-3a中的代码段P1相同的效果。在这个示例中我们可以看到，函数可以使得PHP代码结构更加优化，并更加易读。

```
//Program Segment P1':
 0) function display_welcome() {
 1)      print("Welcome,  ") ;
 2)      print($_POST['user_name']);
 3) }
 4)
 5) function display_empty_form(); {
 6) print <<<_HTML_
 7) <FORM method="post" action="$_SERVER['PHP_SELF']">
 8) Enter your name: <INPUT type="text" name="user_name">
 9) <BR/>
10) <INPUT type="submit" value="Submit name">
11) </FORM>
12) _HTML_;
13) }
14) if ($_POST['user_name']) {
15)    display_welcome();
16) }
17) else {
18)    display_empty_form();
19) }
```

图10-6　使用函数重写程序段P1为P1′

第二个示例如图10-7所示。这里我们使用了图10-5中引入的$teaching数组。图10-7的第0～8行中的函数course_instructor()有两个变元：$course（含有课程名称的字符串）以及$teaching_assignments（含有课程作业的关联数组，与图10-5中的$teaching数组类似）。函数查找讲授某一特定课程的教师的姓名。图10-7中的第9～14行展示了这一函数是如何被使用的。

```
0) function course_instructor ($course, $teaching_assignments) {
1)    if (array_key_exists($course, $teaching_assignments)) {
2)       $instructor = $teaching_assignments[$course];
3)       RETURN "$instructor is teaching $course";
4)    }
5)    else {
6)       RETURN "there is no $course course";
7)    }
8) }
9) $teaching = array('Database' => 'Smith', 'OS' => 'Carrick',
                     'Graphics' => 'Kam');
10) $teaching['Graphics'] = 'Benson'; $teaching['Data Mining'] = 'Kam';
11) $x = course_instructor('Database', $teaching);
12) print($x);
13) $x = course_instructor('Computer Architecture', $teaching);
14) print($x);
```

图10-7　一个带有变元和返回值的函数示例

第11行的函数调用将会返回这样的一个串："*Smith is teaching Database*",因为码值为"Database"的数组项具有值为"Smith"的教师。另一方面，第13行的函数调用返回串："*there is no Computer Architecture Course*"，因为在该数组中没有码为"Computer Architecture"的项。以下是有关这个示例和PHP函数的一些说明：

(1) 如果变量$k的值是关联数组中变量$a的码，则内置PHP数组函数array_key_exists($k,$a)返回真。在我们的示例中，它检查提供的$course值是否为数组$teaching_assignments（图10-7中的第1行）的码。

(2) 函数变元按值传递。因此，在本例中，第11行和第13行中的调用不能改变作为调用变元的数组$teaching。当函数被调用时，在这个变元中提供的值被传递（复制）到该函数变元中。

(3) 函数的返回值被放置在RETURN关键字之后。函数可以返回任意类型。在本例中，它返回串类型。在我们的示例中，两个不同的串将会被返回，它依赖于$course被提供的码值是否存在于数组中。

(4) 变量名应用的范围规则如同在其他程序设计语言中那样，函数外部的全局变量不可以被使用，除非它们使用的内置PHP数组$GLOBALS被引用。基本上说，$GLOBALS['abc']会访问在函数外部定义的全局变量$abc的值。否则，函数内部的变量是局部的，即使有同名的全局变量。

以上讨论给出了PHP函数的简介。没有讨论更详细的PHP，因为这不是我们的目标。

10.3.4　PHP 服务器变量和表单

在被称为$_SERVER的PHP自动-全局内置数组变量中有不少内置项，$_SERVER向程序员提供关于PHP解释器正在其上运行的服务器的有用信息以及其他信息。这些信息在构建HTML文档（例如，见图10-6中第7行）中的文本时是需要的。下面是一些这样的项。

(1) $_SERVER['SERVER_NAME']

它提供了PHP解释器运行的服务器计算机上的网站名。例如，如果PHP解释器运行在网站http://www.uta.edu上，那么该串将会是$_SERVER['SERVER_NAME']中的值。

(2) $_SERVER['REMOTE_ADDRESS']

它是客户端用户计算机访问服务器的IP（Internet协议）地址，如129.107.61.8。

(3) $_SERVER['REMOTE_HOST']

它是客户端用户计算机的网站名，例如abc.uta.edu。在这种情况下，服务器需要将该名称转换为IP地址来访问客户端。

(4) $_SERVER['PATH_INFO']

它是URL地址的一部分，紧接着URL尾部的斜杠（/）之后。

(5) $_SERVER['QUERY_STRING']

它提供紧挨着URL尾部问号（?）后的URL参数的串。例如它可以包含搜索参数。

(6) $_SERVER['DOCUMENT_ROOT']

这是包含所有客户端用户可访问的Web服务器上文件的根目录。

当创建一个为了显示而发送的HTML文档时，常常会需要这些项和$_SERVER数组中的其他项。

另一个重要的PHP自动-全局内置数组变量称作$_POST。它向程序员提供了当用户通过HTML表单中的HTML<INPUT>标签和其他类似标签所提交的值。例如，在图10-6中的第14行，变量$_POST['user_name']向程序员提供了HTML表单中用户通过第8行的<INPUT>标签指定的值。这个数组的码是通过表单提供的各种输入参数名，例如使用第8行中HTML<INPUT>标签的name属性。当用户通过表单输入数据时，数据的值将会被存入这个数组中。

10.4　PHP 数据库程序设计概述

有多种用于通过程序设计语言访问数据库的技术。我们在《数据库系统基础：初级篇》的第9章中已经讨论过一些这样的技术，当时简要介绍了如何使用C和Java程序设计语言访问SQL数据库。特别地，我们讨论了嵌入式SQL、JDBC、SQL/CLI（类似ODBC）以及SQLJ。本节简要介绍如何使用脚本语言PHP来访问数据库，PHP适合用来创建用于搜索和更新数据库的Web界面以及动态网页。

有一个PHP数据库访问函数库，它是PHP扩展和应用资源库（PHP Extension and Application Repository，PEAR）的一部分，PEAR是一个用于增强PHP功能的函数库的集合。PEAR DB库提供了用于数据库访问的函数。通过这个库许多数据库系统都可以被访问，如Oracle、MySQL、SQLite以及Microsoft SQL server等。

我们将会结合本文中的一些示例来讨论PEAR DB中的一些函数。10.4.1节展示了如何使用PHP连接数据库。10.4.2节讨论从HTML表单中汇集的数据是如何被用于在表中插入一条新记录的。10.4.3节展示了如何执行检索查询，并使查询结果显示在动态网页中。

10.4.1　连接数据库

要在PHP程序中使用数据库函数，必须装载称为DB.php的PEAR DB库模块。这在图10-8中示例的第0行完成。现在通过使用DB::<function_name>来访问DB库函数。用于连接数据库的函数称为DB::connect('string')，在这里串变元指定了该数据库的信息。'string'的格式为：

<DBMS software>://<user account>:<password>@<database server>

在图10-8中，第1行连接到使用Oracle存储的数据库（经由串oci8指定）。通过PEAR DB可以访问的一些DBMS软件包如下所示：

(1) MySQL：早期版本指定为mysql，从4.1.2版开始的版本指定为mysqli。

(2) Oracle：在第7、第8、第9版指定为oc8i。在图10-8的第1行中被使用。

(3) SQLite：指定为sqlite。

(4) Microsoft SQL Server：指定为mssql。

(5) Mini SQL：指定为msql。

(6) Informix：指定为ifx。

(7) Sybase：指定为sybase。

(8) **任何支持ODBC的系统**：指定为odbc。

以上并不是一个完整的列表。

紧跟着传送到DB::connect的串变元中的<DB software>的是分隔符://，之后是用户的账户名<user accout>，之后是分隔符：以及账户密码<password>。这些之后是分隔符@以及服务器名和存储着数据库的目录<database server>。

在图10-8中的第1行中，用户使用存储在Oracle DBMS oci8中的账户名acct1、密码pass12来连接位于www.host.com/db1的服务器，使用DB::connect来传送整个串。连接信息保存在数据库连接变量$d中，该变量在对这个数据库进行任何操作时都会用到。

图10-8的第2行展示了如何检查数据库连接的建立是否成功。PEAR DB有一个函数DB::isError，可以用来判定任何一个对数据库的访问操作是否成功。这一函数的变元是数据库连接变量（本例中的$d）。通常，PHP程序员可以在每次数据库调用之后判定上次数据库操作是否成功，如果操作不成功就终止程序（使用die函数）。一条错误信息也会通过操作$d->get_message()从数据库返回。图10-8中的第2行展示了这种情况。

一般来说，一旦连接已经建立起来，大多数SQL命令可以被发送到数据库。函数$d->query将SQL命令作为它的串变元，并将其发送到数据库服务器来执行。在图10-8中，第3～7行发送一个CREATE

TABLE命令以创建一个称为EMPLOYEE的有4个属性的表。一个查询无论什么时候被执行，查询结果都会被赋给一个查询变量，该查询变量在本例中被称为$q。第8行检查查询是否成功执行。

PHP PEAR DB库为检查每条数据库命令之后的错误提供了一种备选方法。如果当通过连接$d（见图10-8中的第9行）访问数据库时发生了某些错误，就可以用函数$d->setErrorHandling（PEAR_ERROR_DIE）来终止程序，并打印默认的错误信息。

```
0)  require 'DB.php';
1)  $d = DB::connect('oci8://acct1:pass12@www.host.com/db1');
2)  if (DB::isError($d)) { die("cannot connect – " .  $d->getMessage());}
    ...
 3) $q = $d->query("CREATE TABLE EMPLOYEE
 4)    (Emp_id INT,
 5)    Name VARCHAR(15),
 6)    Job VARCHAR(10),
 7)    Dno INT)" );
 8) if (DB::isError($q)) { die("table creation not successful – " .
                           $q->getMessage()); }
    ...
 9) $d->setErrorHandling(PEAR_ERROR_DIE);
    ...
10) $eid = $d->nextID('EMPLOYEE');
11) $q = $d->query("INSERT INTO EMPLOYEE VALUES
12)    ($eid, $_POST['emp_name'], $_POST['emp_job'], $_POST['emp_dno'])" );
    ...
13) $eid = $d->nextID('EMPLOYEE');
14) $q = $d->query('INSERT INTO EMPLOYEE VALUES (?, ?, ?, ?)',
15) array($eid, $_POST['emp_name'], $_POST['emp_job'], $_POST['emp_dno']) );
```

图10-8　连接数据库、创建表和插入记录

10.4.2　从表单中汇集数据并插入记录

在数据库应用中，通过HTML或其他Web表单汇集信息是很常见的。例如，当订购机票或申请信用卡时，用户需要输入个人信息，如姓名、地址和电话号码。这些信息被汇集起来并存储在数据库服务器的数据库记录中。

图10-8的第10～12行介绍了汇集信息是如何实现的。在这个示例中，我们省略了创建表单和汇集数据的代码，这些代码是图10-3中示例的一个变种。我们假设用户将有效值输入了称为emp_name，emp_job和emp_dno的输入参数中。这些参数通过PHP自动-全局数组$_POST将成为可访问的，如10.3.4节结尾处讨论的那样。

在图10-8中的第11行和第12行所示的SQL INSERT命令中，数组项$POST['emp_name']，$POST['emp_job']和$POST['emp_dno']将持有通过HTML的输入表单从用户那里汇集来的值。这些值将随即作为新的雇员记录插入到雇员表中。

在这个示例中还说明了PEAR DB的另一个特性。在某些应用中一般对每个新插入数据库的记录会创建一个唯一记录标识符①。

PHP中的函数$d->next ID可以为一个特定表创建一个由唯一值构成的序列。在本例中，employee表（见图10-8第4行）中的域Emp_id就是为该目的而创建的。第10行展示了如何为EMPLOYEE表获取序列中的下一个唯一值，并将其作为在第11行和第12行中的新记录的一部分插入。

图10-8中的第10～12行的插入代码可能会有恶意串进入，这些恶意串会更改INSERT命令。一个

① 这与讲述对象和对象关系数据库系统的第4章和第6章中讨论的由系统生成的OID相似。

比较安全的进行插入和查询的方式是使用占位符(placeholder)(使用？符号指定)。图10-8中的第13～15行的示例说明了在这里插入了另一条记录。在$d->query()函数中，有两个变元。第一个变元是带有一个或多个？符号（占位符）的SQL语句。第二个变元是一个数组，其元素值被用来按占位符所指定的顺序替换这些占位符。

10.4.3　数据库表的检索查询

我们在此将给出三个通过PHP检索查询的示例，如图10-9所示。第0～3行建立了一个数据库连接$d，并将错误处理设置为默认值，如同我们在上一节中讨论的那样。第一个查询（第4～7行）检索所有employee记录的姓名和部门号。查询变量$q用来表示查询结果。第5～7行展示了一个在查询结果中遍历每一行的while循环。第5行的函数$q->fetchRow()用于检索查询结果中的下一条记录，并控制循环。循环从第一个记录开始。

第二个查询示例在第8～13行中展示，它展示了一个动态查询，其中行的选择条件是基于用户的输入值的。这里我们想检索为某一特定部门工作的、有着专门职务的雇员们的姓名。专门职务和部门号会通过数组变量$POST['emp_job']和$POST['emp_dno']输入。如果用户在专门职务处输入“Engineer”而在部门号处输入5，查询会选取所有在部门5工作的工程师的姓名。正如我们所看到的那样，这是一个结果随着用户输入选择的不同而不同的动态查询。本例中我们使用了两个?占位符，如同在10.4.2节结尾处讨论的那样。

最后一个查询（第14～17行）展示了另一种指定一个查询并循环遍历所有行的方法。在这个示例中，函数$d=>getAll持有针对一个单一变量的查询结果的所有记录，称为$allresult。为了遍历每一个记录，使用了foreach循环，并使用了行变量$r迭代$allresult中的每一行[①]。

```
 0) require 'DB.php';
 1) $d = DB::connect('oci8://acct1:pass12@www.host.com/dbname');
 2) if (DB::isError($d)) { die("cannot connect — " .  $d->getMessage()); }
 3) $d->setErrorHandling(PEAR_ERROR_DIE);
    ...
 4) $q = $d->query('SELECT Name, Dno FROM EMPLOYEE');
 5) while ($r = $q->fetchRow()) {
 6)   print "employee $r[0] works for department $r[1] \n" ;
 7) }
    ...
 8) $q = $d->query('SELECT Name FROM EMPLOYEE WHERE Job = ? AND Dno = ?',
 9)   array($_POST['emp_job'], $_POST['emp_dno']) );
10) print "employees in dept $_POST['emp_dno'] whose job is
      $_POST['emp_job']: \n"
11) while ($r = $q->fetchRow()) {
12)   print "employee $r[0] \n" ;
13) }
    ...
14) $allresult = $d->getAll('SELECT Name, Job, Dno FROM EMPLOYEE');
15) foreach ($allresult as $r) {
16)   print "employee $r[0] has job $r[1] and works for department $r[2] \n" ;
17) }
    ...
```

图10-9　数据库检索查询示例

① $r变量类似于《数据库系统基础：初级篇》的第9章、本书第4章和第5章中讨论的游标和迭代变量。

小结

在本章中，我们从10.1节开始，讨论了各种数据类型间的区别，可分为3大类型：结构化、半结构化和非结构化数据。结构化数据存储于传统的数据库中。非结构化数据指的是在Web上显示的信息，这些信息通过HTML指定，有关数据项类型的信息是缺失的。半结构化数据将数据类型名称和数据值混合在一起，但是数据并不是都一定要遵循一个固定的预定义结构。我们将在第11章中讨论XML，它就是一个半结构化数据模型的示例。

接下来我们将注意力转向了如何将一些数据库中的结构化数据转化为显示在网页上的元素。我们重点关注了PHP脚本语言，这是一种正在变得非常流行的网络数据库程序设计语言。10.2节通过一个简单的示例展示了用于网络编程的PHP基础。10.3节介绍了PHP语言的一些基本概念，包括使用得非常广泛的数组和串数据类型。10.4节简单介绍了如何使用PHP来指定各种数据库命令的类型，包括创建表、插入新记录以及检索数据库记录。PHP在服务器计算机上运行，这与其他的在客户端计算机上运行的脚本语言不同。

我们对PHP只给出了一个非常简单的介绍。有许多书籍和网站专门介绍入门PHP程序设计和高级PHP程序设计。PHP还有许多函数库，因为它是一个开源产品。

复习题

10.1　结构化、半结构化和非结构化数据之间有什么不同？对每种数据类型给出数据模型的示例。

10.2　PHP是哪种类型的程序设计语言？

10.3　讨论PHP中指定串的不同方法。

10.4　讨论PHP中数组的不同类型。

10.5　什么是PHP自动-全局变量？试给出一些PHP全局数组的示例，并讨论它们是如何使用的。

10.6　什么是PEAR？什么是PEAR DB？

10.7　讨论用于访问数据库的PEAR DB中的主要函数，以及它们是如何使用的。

10.8　讨论在PHP中循环遍历查询结果的不同方法。

10.9　什么是占位符？在PHP数据库程序设计中是如何使用它们的?

练习题

10.10　考虑《数据库系统基础：初级篇》中图6-14的LIBRARY数据库模式。编写PHP代码来创建该模式的多个表。

10.11　编写PHP程序来创建用于输入一个新BORROWER实体信息的Web表单。对于一个新的BOOK实体重复以上过程。

10.12　为《数据库系统基础：初级篇》中练习题6.18中指定的查询编写PHP Web接口。

实验练习题

10.13　考虑《数据库系统基础：初级篇》中实验练习题3.34中的CONFERENCE_REVIEW数据库，该关系数据库在《数据库系统基础：初级篇》中实验练习题8.28中用MySQL创建并置入了数据。有3种类型的系统用户：*contact author*、*reviewer*和*administrator*。联系作者应当能够注册并将论文提交给大会；评审者应当能够提交分配给他（她）的论文的评审意见（review）；而管理员应当能够为每篇论文分配评审者（每篇论文至少3名评审者），并打印一份有关所有论文的报告，在报告中按照评审者赋给的平均等级进行排序。评审者和管理员具有预赋给的用户id和口令；但是联系作者在他们可以使用系统前必须进行注册。编写基于PHP的应用程序来完

成如下功能：

a. 一个用户注册页面，允许联系作者去注册电子邮件地址以及其他信息。作者在注册页面上选择一个口令。

b. 一个用户登录界面，带有GUI文本框元素用来接受用户id及口令，以及单选按钮用来选择联系作者、评审者和管理员。每个用户的分离菜单应当在身份验证后显示出来。

c. 联系作者应当可以填写关于论文提交的细节并上传文件。

d. 评审者应当可以选择分配给他的论文并提交评审意见。

e. 管理员应当可以给每篇论文分配至少3名评审者。管理员还应当可以显示按照平均等级（从高到低）进行排序的有关所有论文的报告。

10.14　考虑《数据库系统基础：初级篇》的实验练习题4.28中的GRADE_BOOK数据库，这个关系数据库在该书的实验练习题8.29中用Oracle和MySQL创建并置入了数据。

a. 编写并测试一个Java程序，用于与Oracle数据库交互，并为每门课程打印一份等级记分报告（grade count report）。该报告应当按照课程号排序列出每门课程等级分别为A、B、C、D、F的人数。报告格式如下所示：

```
Grade Count Report
CNO        Course Title   #A's   #B's   #C's   #D's   #F's
_____________________________________________________________
CSc 2010   Intro to CS    12     8      15     5      2
CSc 2310   Java           15     2      12     8      1
...
...
_____________________________________________________________
```

b. 编写并测试用来访问MySQL数据库的PHP脚本，针对给定的一个课程单元实现如下的功能。课程编号、学期以及单元信息将作为URL中GET参数的形式提供。

(i) 允许教师为特定等级成分填写分数/成绩。教师应当能够通过下拉列表来选择等级成分。

(ii) 允许教师浏览所有学生所有等级成分的成绩，这些成绩放在一个表单中，每个学生一行，每个等级成分一列。

10.15　考虑《数据库系统基础：初级篇》的实验练习题4.29中的ONLINE_AUCTION数据库，这个关系数据库在该书的实验练习题8.30中用MySQL创建并置入了数据。编写一个PHP脚本来实现一个在线竞拍网站。成员可以注册并选择一个口令，然后登录到系统。应当实现以下功能：

a. 买家应当可以通过输入关键字搜索物品。搜索结果应为一个物品清单，每个物品都带有超链接。点击该超链接，会显示出一个详细描述该物品的页面，用户在该页面可以竞拍物品。

b. 卖家应当可以放置一个物品以供竞拍，需要提供该物品详细信息、竞拍截止期限等。

c. 买家和卖家应当可以浏览所有竞拍已经结束的物品列表。

d. 买家和卖家应当可以对于与他们有关的交易放置和浏览评价（rating）。

参考文献

PHP程序设计有许多资源，无论是印制成书的还是在Web上的。我们给出两本书作为示例。（Sklar，2004）一书给出了对PHP非常好的介绍。对于高级网站开发，（Schlossnagle，2004）一书中提供了许多详细的示例。

第11章

XML：可扩展标记语言

XML语言已经成为Web上构建和交换数据的标准。XML可以用来提供（显示在网页上的）数据特定组成部分的结构和含义的信息，而不同于HTML那样仅仅关注如何将信息显示在网上。有关显示的格式可被单独指定，比如说可以通过可扩展样式语言（Extensible Stylesheet Language，XSL）指定。近期，XML还被提议作为数据存储与检索的可能模型，尽管到现在为止只有一些实验数据库系统是基于XML开发的。

本章中，我们将集中描述XML数据模型，以及如何把从关系数据库中抽取的数据格式化为XML文档以进行网上数据交换。11.1节给出了一个XML数据模型，这个模型是基于树（层次）结构的，而不是基于平面关系数据模型结构的。11.2节讨论XML文档的结构，以及用于指定这些文档结构的语言，例如文档类型定义（Document Type Definition，DTD）和XML模式。11.3节展示了XML和关系数据库之间的联系，并讨论如何把从关系数据库中抽取的数据格式化为XML文档。11.4节给出了XML查询语言XPath和XQuery的简介。

11.1 XML层次（树）数据模型

现在来介绍XML中使用的数据模型。基本对象是XML文档中的XML。为了构建XML文档，我们使用了两个主要的结构化概念：**元素**和**属性**。有一点要特别注意，在XML中术语“属性”一词的使用不同于数据库中所使用的“属性”，而是类似于它在诸如HTML和SGML等文档描述语言中的用法[①]。我们将会看到XML中的属性提供了所描述的元素的附加信息。XML中还有一些附加的概念，例如实体、标识符和引用，但我们首先集中介绍元素和属性来展示XML模型的本质。

图11-1给出了一个名为<Projects>的XML元素的示例。与HTML类似，XML文档中的元素也是由开始标记和结束标记来标识的。标记名包含在尖括号<…>之间，结束标记也由反斜线</…>来标识[②]。

复杂元素是由其他元素按层次来进行构造的，而**简单元素**包含了数据值。XML和HTML的主要区别是：定义XML标记名是用来描述文档中的数据元素的含义，而不是描述如何显示文本。这使得计算机程序有可能自动处理XML文档中的数据元素。

可以很直观地看出图11-1所示的XML文本描述与图10-1所示的树结构之间的对应关系。在树结构中，内部结点表示复杂元素，而叶结点表示简单元素。这就是XML模型称为树模型或层次模型的原因。在图11-1中，带有标记名为<Name>、<Number>、<Location>、<Dept_no>、<Ssn>、<Last_name>、<First_name>和<Hours>的元素是简单元素，而带有标记名为<Projects>、<Project>和<Worker>的元素是复杂元素。通常，在元素的嵌套层次上没有限制。

---

① 标准通用置标语言（Standard Generalized Markup Language, SGML）是一种描述文档的更常用的语言，提供了指定新标记的能力。不过，它比HTML和XML更为复杂。

② 左右尖括号字符（<和>）是保留字符，类似于符号（&）、撇号（' ）和单引号（'）。为了在文档的文本中包含这些字符，必须将其分别编码为<、>、&、'、&quot。

```
<?xml version= "1.0" standalone="yes"?>
<Projects>
   <Project>
      <Name>ProductX</Name>
      <Number>1</Number>
      <Location>Bellaire</Location>
      <Dept_no>5</Dept_no>
      <Worker>
         <Ssn>123456789</Ssn>
         <Last_name>Smith</Last_name>
         <Hours>32.5</Hours>
      </Worker>
      <Worker>
         <Ssn>453453453</Ssn>
         <First_name>Joyce</First_name>
         <Hours>20.0</Hours>
      </Worker>
   </Project>
   <Project>
      <Name>ProductY</Name>
      <Number>2</Number>
      <Location>Sugarland</Location>
      <Dept_no>5</Dept_no>
      <Worker>
         <Ssn>123456789</Ssn>
         <Hours>7.5</Hours>
      </Worker>
      <Worker>
         <Ssn>453453453</Ssn>
         <Hours>20.0</Hours>
      </Worker>
      <Worker>
         <Ssn>333445555</Ssn>
         <Hours>10.0</Hours>
      </Worker>
   </Project>
...
</Projects>
```

图11-1　一个称为<Projects>的复杂XML元素

XML文档可以特征化为3种主要类型：

- **数据中心XML文档**（Data-centric XML document）。这类文档有多个小的数据项，这些数据项遵循一种特定的结构，因此可以从结构化的数据库中抽取。它们被格式化为XML文档，从而在Web上交换或显示它们。
- **文档中心XML文档**（Document-centric XML document）。这类文档带有大量文本，例如新闻文章或书籍。在这些文档中很少有、甚至没有结构化数据元素。
- **混合XML文档**（Hybrid XML document）。这类文档可能有一部分是结构化数据，其他部分主要是文本或非结构化的数据。

有一点要特别注意，数据中心XML文档可以被认为是半结构化数据或者结构化数据，如同在10-1节中定义的那样。如果XML文档符合预定义的XML模式或者DTD（见11.2节），可以把文档看作是结构化数据。另一方面，XML允许文档不遵循任何模式，这样的文档可以看作是半结构化数据。后者也称为*无模式XML文档*。当XML文档中的standalone属性的值为“yes”时，如图11-1的第1行所示，

文档就是独立的和无模式的。

一般来说，在XML中属性的使用方法类似于在HTML中属性的使用方法（见图10-2），也就是描述所出现的元素（标签）的性质和特征。也有可能用XML属性来保存简单数据元素的值，但是建议不要这样做。我们将在11.2节中讨论XML模式和DTD时进一步讨论XML属性。

11.2　XML 文档、DTD 和 XML 模式

11.2.1　合式、有效 XML 文档及 XML DTD

在图11-1中，我们看到了一个简单XML文档的形式。如果一个XML文档遵循某些条件，则称其为**合式的**（well-formed）。具体来说，它必须以XML declaration开始，以便说明正在使用的XML的版本以及其他的相关属性，如图11-1中第1行所示。它还必须遵循树模型的语法规则。这就意味着应该只有一个根元素，其他每个元素必须包括在置于父元素的开始标记和结束标记之间的一对相匹配的开始标记和结束标记之中。这就保证了嵌套元素指定了一个合式的树结构。

一个合式的XML文档在语法上是正确的，这样才能由通用处理器进行处理，该处理器遍历文档并创建一个内部树表示。一个应用程序接口（API）函数的标准集合，称为**文档对象模型**（Document Object Model，DOM），使得程序可以操纵与一个合式XML文档对应的结果树表示。然而，在使用DOM时，必须提前解析整个文档。另外一个称为SAX的API，在使用SAX时，每当遇到一个开始标记或结束标记就要通知处理程序，这样就可以及时地处理XML文档了。使用这种方法使得对大文档的处理更为容易，并且可以处理所谓的**流式XML文档**（streaming XML document），这种情况下当处理程序遇到标记时就会进行处理。

合式XML文档内的元素可以有任意的标记名。对于处理文档的程序，不要求知道有预定义元素（标记名）集合。这样就使文档创建者可以自由地指定新元素，但是这也限制了对文档中元素进行自动解释的可能性。

还有一个更严格的标准，即要求XML文档是**有效**（valid）的。在这种情况下，文档必须是合式的，并且在开始和结束标记对中使用的元素名必须遵循在一个独立的XML DTD文件或者一个XML模式文件中指定的结构。首先来讨论XML DTD，然后在11.2.2节简要介绍XML模式。图11-2给出了一个简单的XML DTD文件，其中指定了一些元素（标记名）及其嵌套结构。符合该DTD的任何有效文档都应该遵循指定的结构。指定DTD文件有一个特定的语法，如图11-2所示。首先，为文档的**根标记**赋予一个标记名，如图11-2中的第1行所示，称作Projects。然后指定元素及其嵌套结构。

```
<!DOCTYPE Projects [
   <!ELEMENT Projects (Project+)>
   <!ELEMENT Project (Name, Number, Location, Dept_no?, Workers)>
   <!ELEMENT Name (#PCDATA)>
   <!ELEMENT Number (#PCDATA)>
   <!ELEMENT Location (#PCDATA)>
   <!ELEMENT Dept_no (#PCDATA)>
   <!ELEMENT Workers (Worker*)>
   <!ELEMENT Worker (Ssn, Last_name?, First_name?, Hours)>
   <!ELEMENT Ssn (#PCDATA)>
   <!ELEMENT Last_name (#PCDATA)>
   <!ELEMENT First_name (#PCDATA)>
   <!ELEMENT Hours (#PCDATA)>
]>
```

图11-2　一个称为*Projects*的XML DTD文件

当指定元素时，使用如下符号表示法：

- *。紧跟元素名，表示元素在文档中可以重复0或多次。这类元素称为可选的多值（重复）元素。
- +。紧跟元素名，表示元素在文档中可以重复1或多次。这类元素称为需求的多值（重复）元素。
- ?。紧跟元素名，表示元素在文档中可以重复0或1次。这类元素称为可选的单值（无重复）元素。
- 没有与上述3种元素一起出现的元素在文档中必须恰好出现一次。这种元素称为需求的单值（无重复）元素。
- 元素**类型**由紧跟元素的圆括号指定。如果圆括号包括其他元素的名字，紧跟着的那些元素就是树结构中该元素的**子元素**。如果圆括号包括关键字#PCDATA或者XML DTD中可用的其他某个数据类型，此元素就是叶结点。PCDATA可理解为解析字符数据，它有些类似于字符串数据类型。
- 指定元素时，圆括号可以嵌套。
- 竖线符号（$e_1|e_2$）表示e_1或者e_2 可以出现在文档中。

我们观察到，图10-1中的树结构和图11-1中的XML文档符合图11-2中的XML DTD。若要求检查XML文档是否符合DTD，必须在文档声明中指定这一点。例如，可以把图11-1中的第1行修改为：

```
<?xml version="1.0" standalone="no"?>
<!DOCTYPE Projects SYSTEM "proj.dtd">
```

当XML文档中的standalone属性值为“no”时，需要检查文档以防一个分离的DTD文档。图11-2所示的DTD文件应该与XML文档存储于同一个文件系统中，并应该给出文件名为“proj.dtd”。另一种方法是，可以在XML文档本身的开始处包含DTD文档文本以支持检查。

虽然XML DTD对于指定含有必需的、可选的和重复的元素的树结构来说已经足够了，但它还是有一些局限性。首先，DTD中的数据类型并不通用。其次，DTD有自己专门的语法，因此需要特定的处理器。使用XML本身的语法规则来指定XML模式文档很有优势，这样用于XML文档的处理器也能够处理XML模式描述。第三，所有DTD元素必须遵循指定的文档顺序，所以其中不允许无序的元素。这些缺点推动了XML模式的开发，使得它成为指定XML文档结构和元素的更通用的一种语言。

11.2.2　XML 模式

XML模式语言（XML schema language）是一种指定XML文档结构的标准。它使用的语法规则与常规XML文档相同，所以同一个处理器对两者都是适用的。为了区分这两种类型的文档，我们使用术语XML实例文档（XML instance document）或XML文档（XML document）来表示常规的XML文档，XML模式文档指的是指定了XML模式的文档。图11-3给出了对应于图3-2和图5-5（以上两图见《数据库系统基础：初级篇》）中COMPANY数据库的XML模式文档。尽管不大可能把整个数据库作为一个文档来显示，但是有人建议可以以本机XML（native XML）的格式存储数据，作为在关系数据库中存储数据的一种候选方法。如果以本机XML系统存储，图11-3中的模式就能够指定COMPANY数据库结构。我们将在11.3节中进一步讨论这个话题。

```
<?xml version="1.0" encoding="UTF-8" ?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
   <xsd:annotation>
      <xsd:documentation xml:lang="en">Company Schema (Element Approach) - Prepared by Babak
         Hojabri</xsd:documentation>
   </xsd:annotation>
<xsd:element name="company">
   <xsd:complexType>
      <xsd:sequence>
         <xsd:element name="department" type="Department" minOccurs="0" maxOccurs= "unbounded" />
         <xsd:element name="employee" type="Employee" minOccurs="0" maxOccurs= "unbounded">
            <xsd:unique name="dependentNameUnique">
```

图11-3　一个称为*company*的XML模式文件

```
            <xsd:selector xpath="employeeDependent" />
            <xsd:field xpath="dependentName" />
          </xsd:unique>
        </xsd:element>
        <xsd:element name="project" type="Project" minOccurs="0" maxOccurs="unbounded" />
    </xsd:sequence>
</xsd:complexType>
<xsd:unique name="departmentNameUnique">
    <xsd:selector xpath="department" />
    <xsd:field xpath="departmentName" />
</xsd:unique>
<xsd:unique name="projectNameUnique">
    <xsd:selector xpath="project" />
    <xsd:field xpath="projectName" />
</xsd:unique>
<xsd:key name="projectNumberKey">
    <xsd:selector xpath="project" />
    <xsd:field xpath="projectNumber" />
</xsd:key>
<xsd:key name="departmentNumberKey">
    <xsd:selector xpath="department" />
    <xsd:field xpath="departmentNumber" />
</xsd:key>
<xsd:key name="employeeSSNKey">
    <xsd:selector xpath="employee" />
    <xsd:field xpath="employeeSSN" />
</xsd:key>
<xsd:keyref name="departmentManagerSSNKeyRef" refer="employeeSSNKey">
    <xsd:selector xpath="department" />
    <xsd:field xpath="departmentManagerSSN" />
 </xsd:keyref>
 <xsd:keyref name="employeeDepartmentNumberKeyRef"
    refer="departmentNumberKey">
    <xsd:selector xpath="employee" />
   <xsd:field xpath="employeeDepartmentNumber" />
</xsd:keyref>
<xsd:keyref name="employeeSupervisorSSNKeyRef" refer="employeeSSNKey">
   <xsd:selector xpath="employee" />
   <xsd:field xpath="employeeSupervisorSSN" />
</xsd:keyref>
<xsd:keyref name="projectDepartmentNumberKeyRef" refer="departmentNumberKey">
   <xsd:selector xpath="project" />
   <xsd:field xpath="projectDepartmentNumber" />
</xsd:keyref>
<xsd:keyref name="projectWorkerSSNKeyRef" refer="employeeSSNKey">
   <xsd:selector xpath="project/projectWorker" />
   <xsd:field xpath="SSN" />
</xsd:keyref>
<xsd:keyref name="employeeWorksOnProjectNumberKeyRef"
   refer="projectNumberKey">
      <xsd:selector xpath="employee/employeeWorksOn" />
      <xsd:field xpath="projectNumber" />
   </xsd:keyref>
</xsd:element>
<xsd:complexType name="Department">
   <xsd:sequence>
      <xsd:element name="departmentName" type="xsd:string" />
      <xsd:element name="departmentNumber" type="xsd:string" />
      <xsd:element name="departmentManagerSSN" type="xsd:string" />
```

图11-3 （续）

```
        <xsd:element name="departmentManagerStartDate" type="xsd:date" />
        <xsd:element name="departmentLocation" type="xsd:string" minOccurs="0"
          maxOccurs="unbounded" />
    </xsd:sequence>
</xsd:complexType>
<xsd:complexType name="Employee">
    <xsd:sequence>
        <xsd:element name="employeeName" type="Name" />
        <xsd:element name="employeeSSN" type="xsd:string" />
        <xsd:element name="employeeSex" type="xsd:string" />
        <xsd:element name="employeeSalary" type="xsd:unsignedInt" />
        <xsd:element name="employeeBirthDate" type="xsd:date" />
        <xsd:element name="employeeDepartmentNumber" type="xsd:string" />
        <xsd:element name="employeeSupervisorSSN" type="xsd:string" />
        <xsd:element name="employeeAddress" type="Address" />
        <xsd:element name="employeeWorksOn" type="WorksOn" minOccurs="1"
          maxOccurs="unbounded" />
        <xsd:element name="employeeDependent" type="Dependent" minOccurs="0"
          maxOccurs="unbounded" />
    </xsd:sequence>
</xsd:complexType>
<xsd:complexType name="Project">
    <xsd:sequence>
        <xsd:element name="projectName" type="xsd:string" />
        <xsd:element name="projectNumber" type="xsd:string" />
        <xsd:element name="projectLocation" type="xsd:string" />
        <xsd:element name="projectDepartmentNumber" type="xsd:string" />
        <xsd:element name="projectWorker" type="Worker" minOccurs="1"
          maxOccurs="unbounded" />
    </xsd:sequence>
</xsd:complexType>
<xsd:complexType name="Dependent">
    <xsd:sequence>
        <xsd:element name="dependentName" type="xsd:string" />
        <xsd:element name="dependentSex" type="xsd:string" />
        <xsd:element name="dependentBirthDate" type="xsd:date" />
        <xsd:element name="dependentRelationship" type="xsd:string" />
    </xsd:sequence>
</xsd:complexType>
<xsd:complexType name="Address">
    <xsd:sequence>
        <xsd:element name="number" type="xsd:string" />
        <xsd:element name="street" type="xsd:string" />
        <xsd:element name="city" type="xsd:string" />
        <xsd:element name="state" type="xsd:string" />
    </xsd:sequence>
</xsd:complexType>
<xsd:complexType name="Name">
    <xsd:sequence>
        <xsd:element name="firstName" type="xsd:string" />
        <xsd:element name="middleName" type="xsd:string" />
        <xsd:element name="lastName" type="xsd:string" />
    </xsd:sequence>
</xsd:complexType>
<xsd:complexType name="Worker">
    <xsd:sequence>
        <xsd:element name="SSN" type="xsd:string" />
        <xsd:element name="hours" type="xsd:float" />
    </xsd:sequence>
```

图11-3 （续）

```
</xsd:complexType>
<xsd:complexType name="WorksOn">
   <xsd:sequence>
      <xsd:element name="projectNumber" type="xsd:string" />
      <xsd:element name="hours" type="xsd:float" />
   </xsd:sequence>
</xsd:complexType>
</xsd:schema>
```

图11-3 （续）

与XML DTD类似，XML模式也是基于树数据模型的，并且把元素和属性也作为主要的结构化概念。但是，它还借用了数据库和对象模型中的其他概念，如码、引用和标识符。下面以图11-3中的XML模式文档为例，介绍XML模式的特性。我们将按照这些模式概念在图11-3中出现的顺序来介绍和描述它们。

(1) 模式描述与XML命名空间

有必要通过指定存储在Web站点位置上的文件，来标识所用到的XML模式语言元素（标记）的特定集合。图11-3中的第2行指定了这个示例中使用的文件，即http://www.w3.org/2001/XMLSchema。这是XML模式命令中最常用的标准。每个这样的定义都被称为一个**XML命名空间**，因为它定义了能够使用的命令（名称）集合。使用属性xmlns（XML命名空间），把文件名指派给变量xsd（XML模式描述），并把该变量作为所有XML模式命令（标记名）的前缀。例如，在图11-3中，当编写xsd:element或者xsd:sequence时，指的是文件http://www.w3.org/2001/XMLSchema中element和sequence标记的定义。

(2) 注释、文档和所用语言

图11-3中接下来的两行说明了XML模式元素（标记）xsd:annotation和xsd:documentation，它们可用于在XML文档中提供注释和其他描述。元素xsd:documentation的属性xml:lang指定了所用的语言，这里的“en”表示英语。

(3) 元素和类型

接下来，指定XML模式的根元素。在XML模式中，标记xsd:element的name属性指定了元素名，在我们的示例中，根元素称作company（见图11-3）。随后就可以指定company根元素的结构了，在该示例中为xsd:complexType。可以进一步使用XML模式的xsd:sequence结构指定一个部门、雇员、项目的序列。有一点需要特别注意：这并不是为COMPANY数据库指定XML模式的唯一途径。其他方法将在11.3节中讨论。

(4) COMPANY数据库中的第一层元素

接下来，指定图11-3中根元素company下面的3个第一层元素。这些元素分别命名为employee、department和project，每个元素都在xsd:element标记中指定。注意，如果标记仅有一个属性并且没有更深一层的子元素或者数据，就可以用反斜线符号（/>）来替代分离的匹配结束标记，从而直接结束该标记。这些元素称为**空元素**，如图11-3中名为department和project的xsd:element元素。

(5) 指定元素类型及最小值和最大值

在XML模式中，xsd:element标记中的属性type、minOccurs和maxOccurs指定了符合模式规范的文档中每个元素的类型和多重性。如果在xsd:element中指定一个type属性，则必须单独描述元素的结构，通常使用XML模式的xsd:complexType元素。如图11-3中的元素employee、department和project所示。另一方面，如果没有指定type属性，可以在标记之后直接指定元素结构，如图11-3中的company根元素所示。标记minOccurs和maxOccurs用于指定元素在符合模式规范的文档中出现次数的最小和最大值。如果未指定，默认值为1。这些属性的作用类似于XML DTD中的符号*、+和?，以及ER模型中的（min,max）约束（见《数据库系统基础：初级篇》的3.7.4节）。

(6) 指定码

在XML模式中，可以指定对应于关系数据库中的唯一性约束和主码约束（见《数据库系统基础：

初级篇》的5.2.2节），外码（或参照完整性）约束（见《数据库系统基础：初级篇》的5.2.4节）也可以指定。标记xsd:unique指定的元素对应于关系数据库中非主码的唯一属性。我们可以为每个这样的唯一性约束指定一个名字，并且必须指定为xsd:selector和xsd:field标记，以此来标识包含此唯一元素的元素类型及元素名，元素名唯一性通过xpath属性保证。如图11-3中的元素departmentNameUnique和projectNameUnique所示。对于指定**主码**，使用标记xsd:key代替xsd:unique，如图11-3中的元素projectNumberKey、departmentNumberKey和employeeSsnKey所示。对于指定**外码**，使用标记xsd:keyref，如图11-3中的6个xsd:keyref元素所示。当指定一个外码时，标记xsd:keyref的属性refer指定了引用的主码，而标记xsd:selector和xsd:field指定了引用的元素类型和外码（见图11-3）。

(7) 通过复杂类型指定复杂元素的结构

在我们的示例中，接下来的部分使用标记xsd:complexType指定了复杂元素Department、Employee、Project和Dependent的结构（见图11-5）。使用XML模式的标记xsd:sequence和xsd:element，可以把每个元素都指定为子元素序列，这对应于数据库中每个实体类型的属性（见《数据库系统基础：初级篇》的图3-2和图5-7）。通过xsd:element的属性name和type为各元素指定一个名字和类型。如果需要修改minOccurs和maxOccurs属性的默认值（默认为1），还可以指定minOccurs和maxOccurs属性。对于允许为空的数据库属性（可选的），需要指定minOccurs=0；对于允许多值的数据库属性，需要在相应元素上指定maxOccurs = “unbounded”。需要注意，如果不指定任何码约束，则可以在父元素定义中直接嵌入子元素，而不必指定复杂类型。然而，需要指定唯一约束、主码约束和外码约束时，必须定义复杂类型来指定元素结构。

(8) 复合（组合）属性

图3-2（见《数据库系统基础：初级篇》）中的复合属性在图11-5中也被指定为复杂类型，如复杂类型Address、Name、Worker和WorksOn所示。这些复杂类型可以直接嵌入到它们的父元素中。

这个示例说明了XML模式的一些主要特性。除了这些特性之外还有其他特性，不过这超出了我们的讨论范围。下一节，讨论从关系数据库创建XML文档和存储XML文档的不同方法。

11.3　XML 文档和数据库

现在我们来讨论如何存储和检索各种类型的XML文档。11.3.1节概述了存储XML文档的各种方法。11.3.2节详细地讨论了其中的一种方法，它可以从已有的数据库中抽取数据中心XML文档。特别的，我们展示了如何从图结构化（graph-structured）的数据库中创建树结构化（tree-structured）文档。11.3.3节讨论了环问题及其处理方法。

11.3.1　存储 XML 文档的方法

为了便于随后的查询和检索，目前已经提出多种方法来组织XML文档的内容。最常用的方法有：

(1) 使用DBMS把文档存储为文本

用关系或者对象DBMS可以把整个XML文档存储为DMBS记录或对象中的文本字段。如果DBMS中含有专门的文档处理模块，并且能够存储无模式文档和文档中心XML文档，就可以采用这种方法。可以使用文档处理模块的关键字索引函数（见第6章）建立索引，并加快文档的搜寻和检索。

(2) 使用DBMS把文档内容存储为数据元素

这种方法可以存储遵循特定XML DTD或XML模式的文档集合。因为所有文档的结构相同，所以可以设计一个关系（或对象）数据库存储XML文档中叶子层的数据元素。这种方法需要使用映射算法来设计一个数据库模式，该模式要与XML模式或DTD指定的XML文档结构一致，并且需要使用映射算法从存储数据中重新创建XML文档。这些算法可以作为内部DBMS模块实现，或者作为并不属于DBMS组成部分的独立中间件实现。

(3) 设计专门的系统存储本机XML数据

可以设计和实现基于层次（树）模型的新类型数据库系统。这种系统称作**本机XML DBMS**。这种系统含有专门的索引与查询技术，并且适用于所有类型的XML文档。其中还包含数据压缩技术，以减少存储的文档长度。Software AG的Tamino和eXcelon的Dynamic Application Platform是两种提供**本机XML DBMS**功能的流行产品。

(4) 从已有的关系数据库创建或发布定制的XML文档

因为关系数据库中已经存储了大量的数据，可能需要把其中的一部分格式化为文档并在Web上交换或显示。这种方法将会使用独立的中间件软件层处理XML文档与关系数据库之间的转换。

在过去的几年中，以上这些方法都受到了极大的关注。在下一节，我们集中讨论第4种方法，因为它帮助我们很好地理解了XML树数据模型与基于平面文件（关系模型）和图描述（ER模型）的传统数据库模型之间的区别。

11.3.2　从关系数据库中抽取 XML 文档

这一节讨论从数据库系统到XML文档进行数据转换时引发的表示问题。前面已经讨论过，XML采用层次（树）模型来表示文档，而使用最广泛的数据库系统采用了平面关系数据模型。在增加参照完整性约束时，关系模式被认为是图结构（例如，见《数据库系统基础：初级篇》的图5-7）。类似的，ER模型也采用类似结构的图来表示数据（例如，见《数据库系统基础：初级篇》的图3-2）。我们在第7章中观察到了ER模型与关系模型之间的直接映射，所以，在概念上可以采用相应的ER模型来表示关系数据库模式。尽管在讨论和示例中使用ER模型来说明树模型与图模型之间的概念区别，然而这些问题也同样适用于关系数据到XML的转换。

我们以图11-4中给出的简化UNIVERSITY ER模式为例进行讨论。假设应用需要从UNIVERSITY数据库中抽取包含student、course和grade信息的XML文档。这些文档需要的数据被包含在图11-4所示的实体类型COURSE、SECTION和STUDENT以及它们之间的联系S-S和C-S的数据库属性中。通常，从数据库中抽取的大部分文档将仅仅使用数据库中属性、实体类型和联系的一个子集。在这个示例中，需要的数据库子集如图11-5所示。

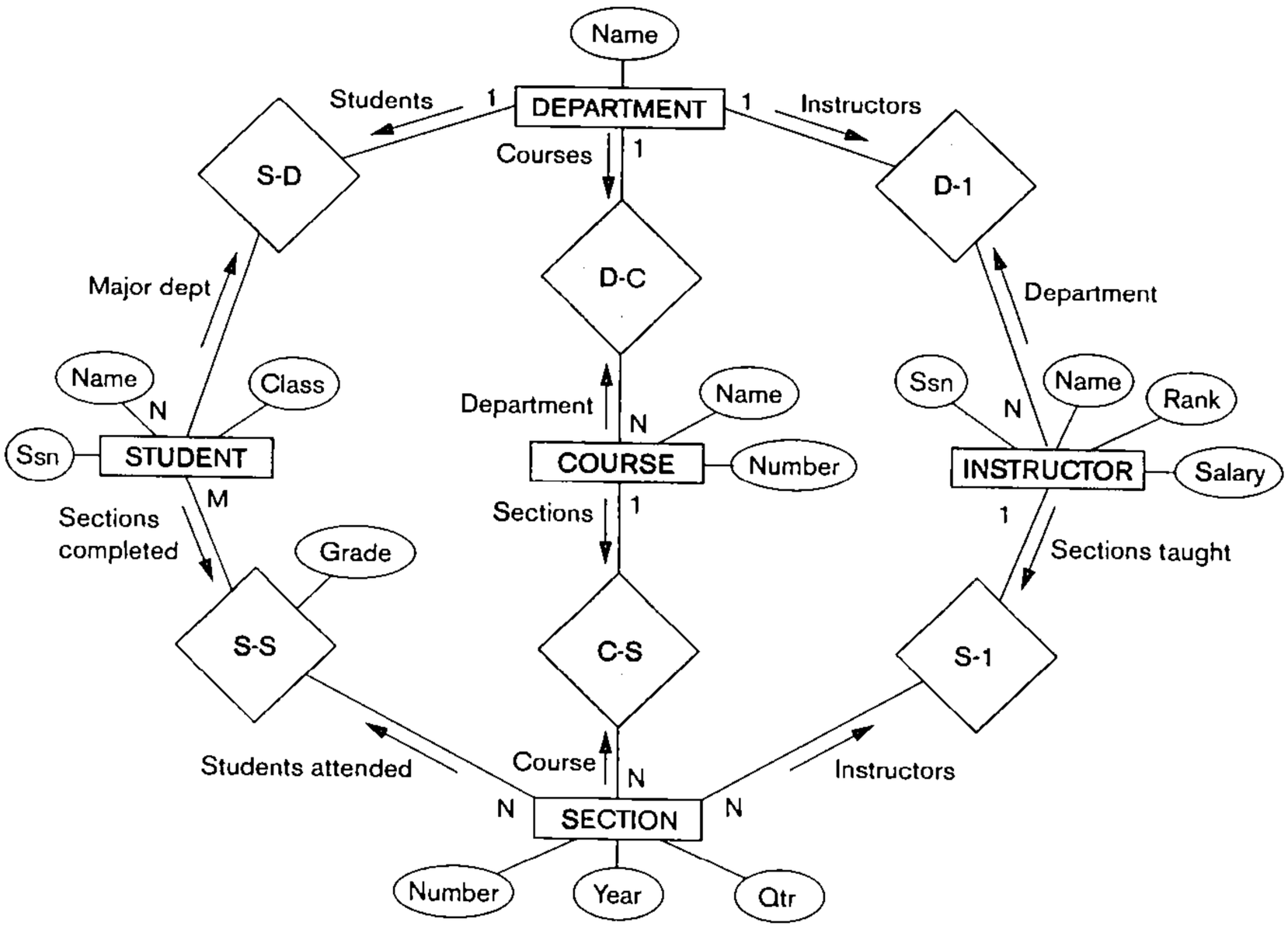


图11-4　简化的UNIVERSITY数据库的ER模式图

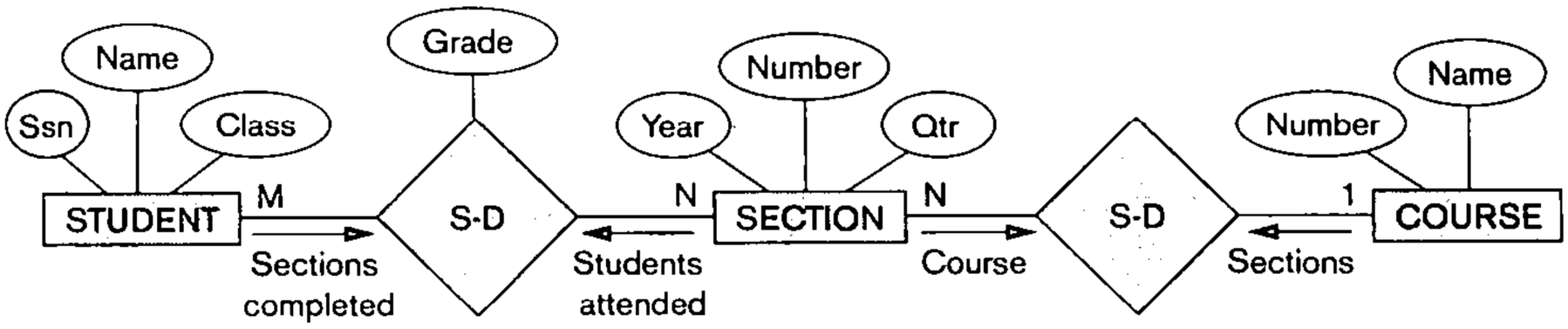


图11-5　XML文档抽取需要的UNIVERSITY数据库模式的子集

从图11-5所示的数据库子集中至少可以抽取3个可能的文档层次。首先，可以选择以COURSE为根，如图11-6所示。在这里，每一个课程实体有作为其子元素的单元集合，而每个单元又有作为其子元素的学生。我们可以观察到以层次树结构对其进行建模的结果。如果一个学生选择了多个课程单元，该学生的信息将会在文档中出现多次，因为在每个课程单元出现一次。该视图的一个可能的简化XML模式如图11-7所示。S-S联系中的成绩数据库属性被迁移到STUDENT元素中。这是因为在这个层次结构中，STUDENT变成了SECTION的子结点，所以一个特定的SECTION下的每个STUDENT元素，对于该课程单元都有一个特定的成绩。在这个文档层次结构中，对于选择多于一个课程单元的学生将会有多个副本，每个副本对应一个课程单元，并且在该特定的课程单元中的每一个副本都将有一个特定的成绩。

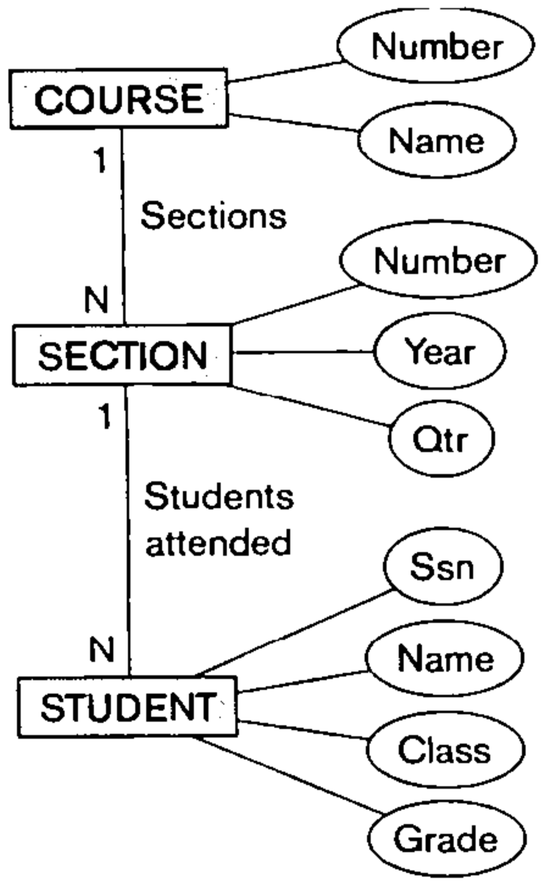


图11-6　COURSE作为根的层次（树）视图

```
<xsd:element name="root">
<xsd:sequence>
<xsd:element name="course" minOccurs="0" maxOccurs="unbounded">
   <xsd:sequence>
      <xsd:element name="cname" type="xsd:string" />
      <xsd:element name="cnumber" type="xsd:unsignedInt" />
      <xsd:element name="section" minOccurs="0" maxOccurs="unbounded">
         <xsd:sequence>
            <xsd:element name="secnumber" type="xsd:unsignedInt" />
            <xsd:element name="year" type="xsd:string" />
            <xsd:element name="quarter" type="xsd:string" />
            <xsd:element name="student" minOccurs="0" maxOccurs="unbounded">
               <xsd:sequence>
                  <xsd:element name="ssn" type="xsd:string" />
                  <xsd:element name="sname" type="xsd:string" />
                  <xsd:element name="class" type="xsd:string" />
                  <xsd:element name="grade" type="xsd:string" />
               </xsd:sequence>
            </xsd:element>
         </xsd:sequence>
      </xsd:element>
   </xsd:sequence>
</xsd:element>
</xsd:sequence>
</xsd:element>
```

图11-7　COURSE作为根的XML模式文档

在第2个层次文档视图中，可以把STUDENT作为根（见图11-8）。在这个层次视图中，每个学生都有一个课程单元集合作为其子元素，而每个课程单元与一门课程相关联并作为课程的孩子，因为SECTION与COURSE之间的联系为N:1。因此，在这个视图中可以合并元素COURSE和SECTION，如图11-8所示。另外，GRADE数据库属性可以移到SECTION元素中。在这个层次结构中，组合的COURSE/SECTION信息被复制在完成该课程单元的每个学生下面。对于这个视图的一个可能的简化的XML模式如图11-9所示。

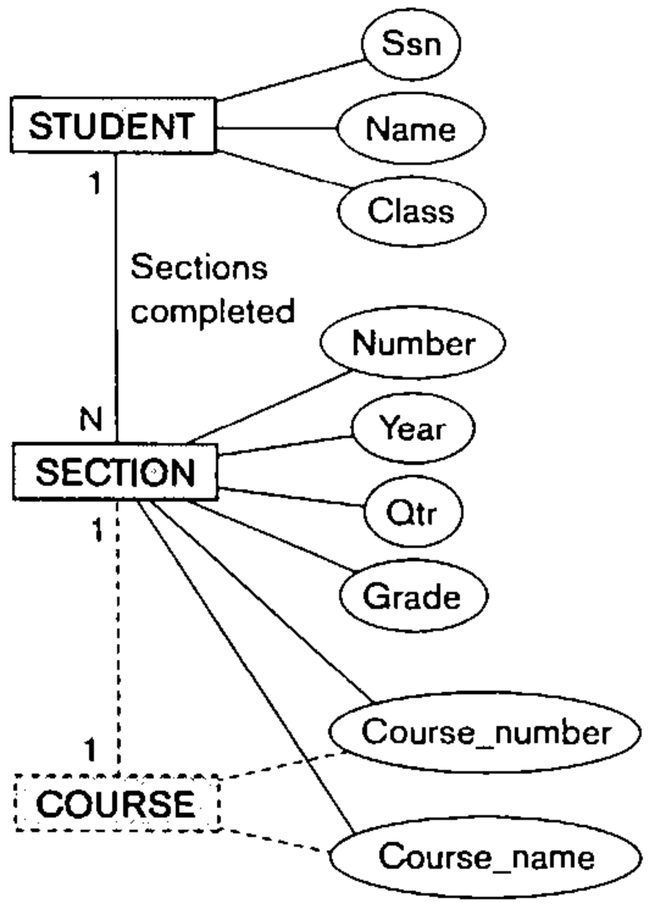


图11-8　STUDENT作为根的层次（树）视图

```
<xsd:element name="root">
<xsd:sequence>
<xsd:element name="student" minOccurs="0" maxOccurs="unbounded">
   <xsd:sequence>
      <xsd:element name="ssn" type="xsd:string" />
      <xsd:element name="sname" type="xsd:string" />
      <xsd:element name="class" type="xsd:string" />
      <xsd:element name="section" minOccurs="0" maxOccurs="unbounded">
         <xsd:sequence>
            <xsd:element name="secnumber" type="xsd:unsignedInt" />
            <xsd:element name="year" type="xsd:string" />
            <xsd:element name="quarter" type="xsd:string" />
            <xsd:element name="cnumber" type="xsd:unsignedInt" />
            <xsd:element name="cname" type="xsd:string" />
            <xsd:element name="grade" type="xsd:string" />
         </xsd:sequence>
      </xsd:element>
   </xsd:sequence>
</xsd:element>
</xsd:sequence>
</xsd:element>
```

图11-9　STUDENT作为根的XML模式文档

第3个可能的方法是把SECTION作为根，如图11-10所示。与第2个层次视图类似，COURSE信息可以合并到SECTION元素中。GRADE数据库属性可以迁移到STUDENT元素中。如同我们所观察到的那样，即使在这个简单的示例中，也有多个层次文档视图，每个视图对应于不同的根和不同的XML文档结构。

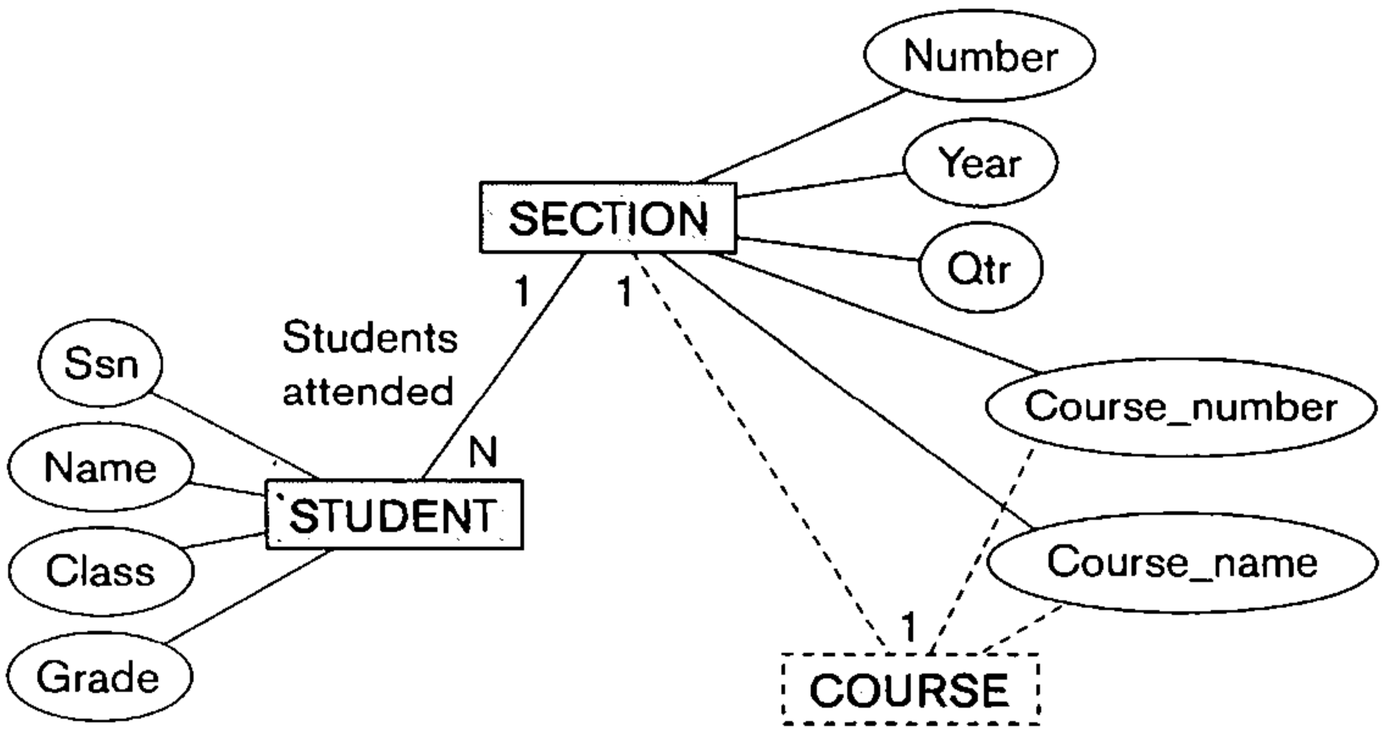


图11-10　SECTION作为根的层次（树）视图

11.3.3　断开环以实现从图到树的转换

在前面的示例中，我们所关注的数据库子集中没有环。更为复杂的子集可能会包括一个或多个环，以表示实体之间的多重联系。这种情况下，要确定如何创建文档层次结构将会更加复杂。可能需要附加的实体副本来表示多重联系。这里将以图11-4中的ER模式为例进行说明。

假设一个特定的XML文档需要图11-4中所有实体类型和联系的信息，其中STUDENT作为根元素。图11-11说明了如何为这个文档创建可能的层次树结构。首先，我们得到一个以STUDENT为根的格，如图11-11a所示。这不是一个树结构，因为其中存在环。断开这个环的一个方法是复制该环中的实体类型。首先，我们复制INSTRUCTOR，如图11-11b所示，把该副本称为右INSTRUCTOR1。左边的INSTRUCTOR副本表示教师与其讲授课程单元之间的联系，而右边的INSTRUCTOR1副本表示教师与其所在院系之间的联系。这样操作之后，仍存在包含COURSE的环，所以可以用类似的方式复制COURSE，这会产生如图11-11c所示的层次结构。左边的COURSE1副本表示课程及其课程单元之间的联系，右边的COURSE副本表示课程与开设每门课程的院系之间的联系。

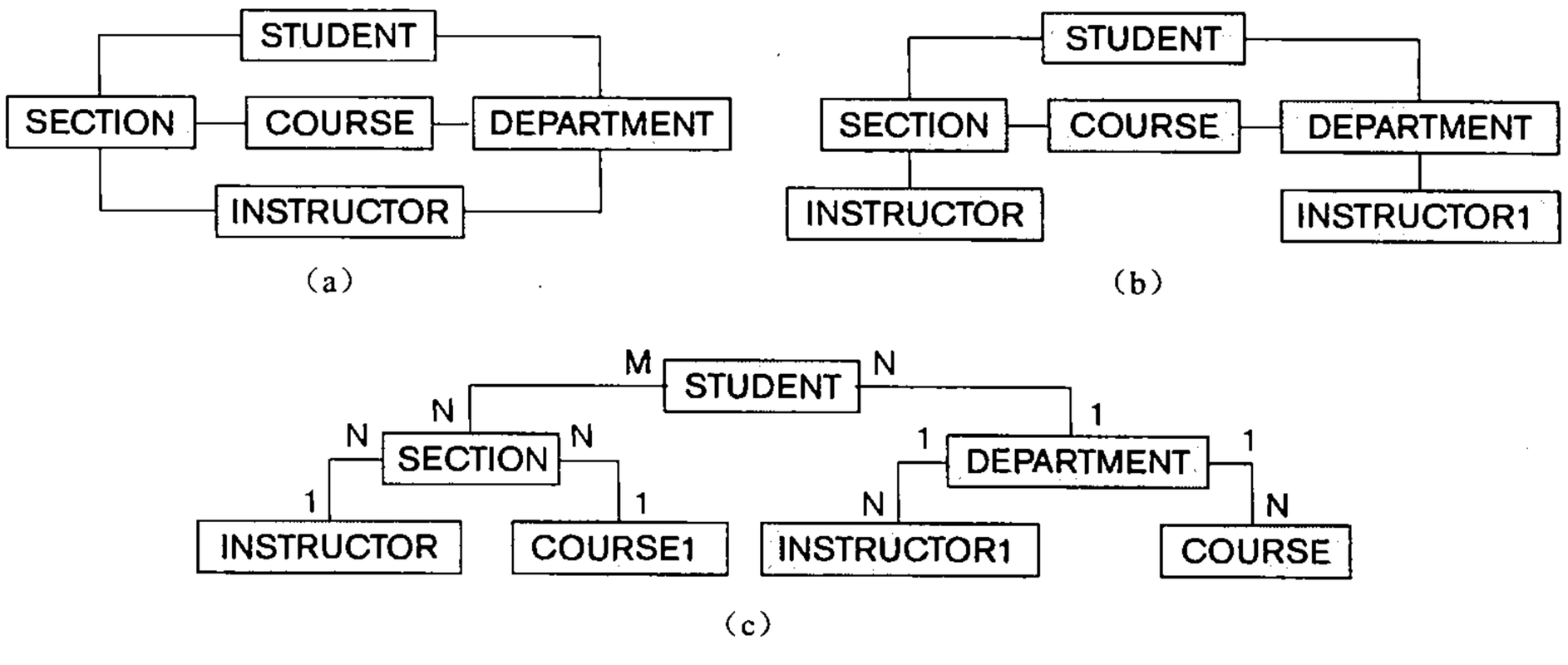


图11-11　把带有环的图转化为层次（树）结构

在图11-11c中，我们把原始图转换为层次图。如果需要（如前例），可以在创建最终的层次图和相应XML模式结构之前进行进一步的合并。

11.3.4　从数据库抽取 XML 文档的其他步骤

除了创建适当的XML层次结构以及相应的XML模式文档之外，从数据库中抽取特定的XML文档

还需要其他一些步骤：

(1) 必须创建正确的SQL查询以便为XML文档抽取所需的信息。

(2) 一旦查询被执行，其结果必须从平面关系形式被结构化为XML树结构。

(3) 可以定制查询，以便选择单个对象或多个对象到文档中。例如，在图11-9的视图中，查询可以选择单个的学生实体，并创建与这个学生对应的文档，或者选择多个甚至所有学生，并创建含有多个学生的文档。

11.4　XML 查询

对于XML查询语言的提议有很多，但只有两个标准被采用。第一个是XPath，它提供了指定路径表达式的语言构造，以识别符合特定模式的XML文档中的某些结点（元素）。第二个是XQuery，它是一种更为通用的查询语言。XQuery采用XPath表达式，但还有其他的构造。本节将简要介绍这两种语言。

11.4.1　XPath：在 XML 中指路径表达式

XPath表达式返回一个元素结点的集合，这些结点满足表达式中指定的特定模式。XPath表达式中的名字是XML文档树中的结点名，可以是标记（元素）名或属性名，并可能会有附加的**限定条件**，来进一步限定符合该模式的结点。指定路径主要使用两种**分隔符**（separator）：单斜杠（/）和双斜杠（//）。标记前的单斜杠指定该标记必须作为前一个（父）标记的直接孩子出现；而双斜杠指定该标记可以作为前一个标记的任意层次的后代而出现。下面我们看一下如图11-12所示的XPath的一些示例。

```
1. /company
2. /company/department
3. //employee [employeeSalary gt 70000]/employeeName
4. /company/employee [employeeSalary gt 70000]/employeeName
5. /company/project/projectWorker [hours ge 20.0]
```

图11-12　遵循图11-3中XML模式文件*company*的XML文档上的XPath表达式的一些示例

图11-12中的第1个XPath表达式返回company根结点以及它的所有后代结点，即返回整个XML文档。应该注意，在XPath查询中习惯包括文件名，这样可以指定任意的本机文件名，甚至可以指定Web上某特定文件的任意路径名。例如，如果COMPANY XML文档存储在如下位置中：

www.company.com/info.XML

那么图11-12中的第1个XPath表达式可以写作：

doc(www.company.com/info.XML)/company

这个前缀也被包括在其他示例中。

图11-12中的第2个示例返回所有的部门结点（元素）以及它们的后代子树。注意，XML文档中的结点（元素）是有序的，所以返回多个结点的XPath结果会按结点在文档树中的顺序进行排序。

图11-12中的第3个XPath表达式说明了//的用法，如果不知道要搜索的完整路径名，而只是知道XML文档中感兴趣的一些标记名，那么使用//会相当方便。这对于无模式XML文档或者结点[①]多重嵌套的文档而言特别有用。

这个表达式返回employee结点的所有直接孩子结点employeeName，这样employee结点还有另一个值超过了70 000的孩子元素employeeSalary。这说明了限定条件的用法，它将XPath表达式选择

① 在此交替使用术语“结点”、“标记”和“元素”。

的结点限定为满足条件的那些结点。可以在限定条件中使用多个XPath比较操作，其中包括标准算术、字符串和集合比较操作。

第4个XPath表达式应当返回同上一个表达式相同的结果，除了在这个示例中指定了完整的路径名之外。图11-12中的第5个表达式返回所有projectWorker结点及其后代结点，这些后代结点是路径/company/project下的孩子，并有一个值超过20.0小时的孩子结点hours。

11.4.2　XQuery：在 XML 中指定查询

使用XPath可以编写从树结构XML文档中选择结点的表达式。XQuery则允许在一个或多个XML文档上指定更一般的查询。Xquery查询的典型形式称作**FLWR表达式**，它表示Xquery的4个主要子句，它们有如下形式：

```
FOR<单个结点（元素）的变量绑定>
LET<结点（元素）集合的变量绑定>
WHERE<限定条件>
RETURN<查询结果规范说明>
```

图11-13包括了多个XQuery查询的示例，它们可以在遵循图11-3中XML模式文档的XML实例文档上指定。第1个查询检索工资超过$70 000的雇员的姓名。变量$x绑定到employee元素的各个孩子元素employeeName上，但是要求employee元素必须满足一定的限定条件：其employeeSalary值大于$70 000。检索结果得到选定的employeeName的元素的孩子元素firstName和lastName。第2个查询采用了另一种方法来检索与第一个查询相同的元素。

```
1. FOR $x IN
       doc(www.company.com/info.xml)
       //employee [employeeSalary gt 70000]/employeeName
       RETURN <res> $x/firstName, $x/lastName </res>

2. FOR $x IN
       doc(www.company.com/info.xml)/company/employee
       WHERE $x/employeeSalary gt 70000
       RETURN <res> $x/employeeName/firstName, $x/employeeName/lastName </res>

3. FOR $x IN
       doc(www.company.com/info.xml)/company/project[projectNumber = 5]/projectWorker,
          $y IN doc(www.company.com/info.xml)/company/employee
       WHERE $x/hours gt 20.0 AND $y.ssn = $x.ssn
       RETURN <res> $y/employeeName/firstName, $y/employeeName/lastName, $x/hours </res>
```

图11-13　遵循图11-3中XML模式文件*company*的XML文档上的XQuery查询的一些示例

第3个查询展示了如何通过多个变量实现连接操作。在此，变量$x绑定至每个projectWorker的元素，那是项目号为5的项目的孩子。而变量$y绑定至每个employee元素。为了检索雇员的姓名，连接条件要与ssn值匹配。

就此结束对XQuery的简要介绍。感兴趣的读者可以访问www.w3.org ，其中包含了描述XML相关的最新标准的文档。

小结

本章概要介绍了Internet上表示和交换数据的标准。首先描述了XML标准与它的树结构（层次）

数据模型，并讨论了XML文档与指定这些文档结构的语言，即XML DTD（文档类型定义）及XML模式。接着对存储XML文档的各种合适的方法进行了概述，可以是本机（文本）格式或者压缩形式存储，或者采用关系和其他类型数据库存储。并讨论了需要把传统数据库中存储的数据转换为XML文档时出现的映射问题。最后对XPath与XQuery语言做了简要介绍，这是对查询XML数据而提出的两种查询语言。

复习题

11.1　结构化数据、半结构化数据和非结构化数据之间有什么不同？

11.2　XML文档可以划归为上题中的哪一类？自描述数据又应当划归为哪一类？

11.3　XML和HTML中标记的用法有什么不同？

11.4　数据中心XML文档与文档中心XML文档之间有什么不同？

11.5　在XML中，属性和元素有什么不同？列出XML模式中用于指定元素的一些重要属性。

11.6　XML模式和XML DTD之间有什么不同？

练习题

11.7　创建对应于存储在《数据库系统基础：初级篇》中图5-6所示关系数据库中的数据的部分XML实例文档，使得XML文档符合图11-3中的XML模式文档。

11.8　创建对应于图11-10和图11-11c所示层次结构的XML模式文档和XML DTD。

11.9　考虑《数据库系统基础：初级篇》中图6-14的LIBRARY关系数据库模式。创建对应于此数据库模式的XML模式文档。

11.10　对于图11-3所示的*company* XML模式，将以下视图指定为XQuery查询。

(a) 对于每个部门，视图中包含部门名称、经理姓名和经理工资。

(b) 对于在Research部门工作的每个雇员，视图中包含雇员姓名、主管姓名和雇员工资。

(c) 对于每个项目，视图中包含项目名称、项目管理部门名称、雇员数量以及每周在该项目上工作的总小时数。

(d) 对于多个雇员参加的项目，视图中包含项目名称、项目管理部门名称、雇员数量以及其每周在该项目上工作的总小时数。

参考文献

现在有很多关于XML各个方面的文章和书籍，以至于要给出一个比较合适的列表是不可能的。我们将推荐一本书：（Chaudhri，Rashid & Zicari，eds，2003）。该书讨论了XML的许多方面，并且包含了一个有关XML研究和实践最新参考文献的列表。

第12章

数据挖掘概念

在过去的30年中，许多组织已经产生了大量的机器可读的数据，它们以文件和数据库的形式存在。为了处理这些数据，我们有了像SQL这样有效的支持查询语言的数据库技术。SQL有一个问题：它是一种假设用户清楚数据库模式的结构化语言。SQL支持关系代数运算，允许用户从表中选择行或列数据，或者从基于公共字段的表中连接相关的信息。在下一章中，可以看到数据仓库技术提供的多种功能，如数据的合并、聚集、汇总。它允许我们从多个维度观察同一信息。在这一章中，我们把注意力集中到另一个非常流行的重要领域——数据挖掘。顾名思义，**数据挖掘**（data mining）是指从大量的数据中根据模式或者规则来挖掘、发现新的信息。在实际应用中，数据挖掘一定是在大型文件和数据库上才能有效地实现。迄今为止，数据挖掘还没有与数据库管理系统很好地集成在一起。

首先我们将简要地回顾一下数据挖掘领域的发展情况，由于它涉及到很多的技术领域，包括机器学习、统计学、神经网络、遗传算法等，所以我们将重点阐述被发现的信息的本质，在数据库中进行挖掘时遇到的问题类型，以及数据挖掘应用的类型。还将介绍多个有用的数据挖掘商业工具（见12.7节），并描述一些有关数据挖掘领域的研究进展。

12.1 数据挖掘技术概述

在一些报告中，比如非常流行的Gartner Report[①]，数据挖掘作为未来最热门的技术之一而备受关注。在本节中，我们把数据挖掘与知识发现这个更广泛的领域联系在一起，并且通过一个示例对二者进行比较。

12.1.1 数据挖掘和数据仓库

数据仓库（见第13章）的目标是使用数据来支持决策。数据挖掘可以与数据仓库结合起来，有助于实现特定类型的决策。数据挖掘可以应用于具有单独事务的操作型数据库。为了使数据挖掘更加有效，数据仓库应该具有一个聚合的或汇总的数据集合。数据挖掘可以有助于抽取有意义的新模式，这些新模式是不可能通过单纯的查询或者处理数据仓库中的数据和元数据而发现的。因此，数据挖掘应用应该尽早考虑，最好是在数据仓库设计阶段就应充分考虑。同样，数据挖掘工具应该被设计成能方便地与数据仓库连接的软件。实际上，对于拥有万亿字节（terabyte）级数据的特大型数据库，成功地使用数据挖掘应用将首先依赖于数据仓库的构建。

12.1.2 数据挖掘是知识发现过程的一部分

数据库中的知识发现经常缩写为**KDD**（Knowledge Discovery in Database），通常包括比数据挖掘更大的范围。知识发现过程有6个阶段[②]：数据选择、数据清洗、数据增强、数据转换或编码、数据

① Gartner Report是许多技术调查刊物中的一种，它们是公司经理做出技术选择决策的依据。

② 这一讨论主要基于（Adriaans & Zantinge，1996）。

挖掘、被发现信息的报告与展示。

作为一个示例，考虑由专门经营消费品的零售商所维护的一个事务数据库。假设客户数据包括消费者姓名、邮政编码、电话号码、购买日期、商品代码、商品价格、购买数量和总计。在这个客户数据库中可以通过KDD过程发现多种新知识。在数据选择过程中，被选择的数据可以是有关特定的商品或商品种类的数据，或者是来自某个特定区域或地区的商店数据等等。数据清洗过程可以改正无效的邮政编码或者删除具有错误电话号码前缀的记录。数据增强过程通常利用其他的信息源来增强数据。例如，给定客户姓名和电话，商店可以购买关于这些客户的其他信息，包括年龄、收入、信用级别等数据，并把它们添加到相应的每条记录中。数据转换和编码可用来减少数据量。例如，商品代码可以根据产品种类分组，如：音频、视频、生活用品、小电器、照相机、配件等等。邮政编码可以根据地区聚类，收入可以划分为若干等级等等。图12-1展示了一个称作清洗的过程，它是数据仓库构建的先行任务。如果数据挖掘以一个已经存在的零售商店数据仓库为基础，那么我们希望已经进行过数据清洗。数据挖掘技术只有在进行了这些预处理之后才能够被用来挖掘不同的规则和模式。

挖掘的结果可能会发现如下所示的新信息类型：

- **关联规则**。例如，每当一个消费者购买视频设备时，他或她也会购买另外一个小电器。
- **序列模式**。例如，假设一个消费者购买了一个照相机，并且在3个月之内他或她又购买了一些摄影用品，那么他或她在6个月之内可能会购买一个相关配件。这就定义了事务的一个序列模式。又如，在经济不景气时期购物超过两次的消费者，这个消费者很可能在圣诞节期间至少购物一次。
- **分类树**。例如，可以根据消费者的光顾频率、付款方式、消费总额或者根据所购买商品类型之间的关系对他们进行分类，就有可能会产生一些与分类有关的揭示性的统计数据。

由此可见，在购买模式中发现新知识存在着多种可能性，其相关因素包括年龄、收入、住址、消费者所购物品及其数量等。这些信息可以用来根据人口统计信息来计划新增分店的地址，用来执行商店促销活动，用来在广告中组合不同商品，或者用来制定季节性的市场营销策略等。正如这个零售商店示例所展示的那样，数据挖掘必须在进行过大量的数据预处理之后，才可以产生直接影响商务决策的有用信息。

数据挖掘的结果可以产生各种形式的报告，如列表、图形、汇总表或其他可视化形式。

12.1.3　数据挖掘和知识发现的目标

数据挖掘一般都会有一些最终目标或应用。广义地讲，数据挖掘的目标分为以下几类：预测、识别、分类和优化。

- **预测**（Prediction）。数据挖掘可以展示数据的某些属性在未来将如何表现。预测性数据挖掘的示例有：通过对购买事务的分析来预测顾客在某些折扣下会购买什么？在给定时期内一个商店可以获得多大的销售额？撤销一条生产线是否可以提高利润？在这样的应用中，业务逻辑和数据挖掘结合使用。如同在一个科学环境中，确定的震波模式能在很大程度上预测地震一样。
- **识别**（Identification）。数据模式可以用来识别一件商品、一个事件或一项活动的存在。例如，可以通过程序的执行、文件的访问以及每个会话期的CPU时间，来识别是否有入侵者试图破坏系统。在生物学应用中，基因的存在可以通过DNA序列中核苷酸符号的特定序列来识别。认证（authentication）是识别的一种形式，它查明用户是否真的是某个特定的用户，或者是否来自一个被授权的类，与数据库中的认证相比，它还包括一些参数、图像或者信号的比较。
- **分类**（Classification）。数据挖掘可以划分数据，这样的分类可以根据参数的组合来识别不同的类或类别。例如，超市的顾客可以划分为追求折扣的购买者、冲动购买者、忠实购买者、

追求名牌的购买者和偶然购买者。这种分类可以作为一种挖掘之后的活动，用于对顾客的购买事务进行不同的分析。有时，可将基于共同领域知识的分类用于作为分解挖掘问题的输入，并使问题更为简单。例如，保健食品、聚会食品、或者学校午餐食品，在超市业务中是不同的类别。这就使得把分析类别内和类别间的联系作为独立的问题对待，这是很有意义的。在作进一步的数据挖掘之前，这种分类可以用于对数据进行适当的编码。

- **优化**（Optimization）。数据挖掘的一个最终目标是优化有限资源的使用，例如时间、空间、金钱或者物质，并且在给定的约束条件下使输出变量最大化，例如销售额或利润。这样，数据挖掘的这个目标就像在运筹学中使用目标函数在约束条件下取得最优值一样。

数据挖掘这个术语目前的含义非常广。在某些情况下，它包括统计分析、约束优化和机器学习。没有严格的界线能把数据挖掘从这些学科中分离开来。因此，对于这项巨大工程中的所有应用进行详细讨论超出了本书的范围。要详细了解这个领域，可以参考数据挖掘领域的专著。

12.1.4　数据挖掘期间知识发现的类型

知识这个术语被广泛地解释为含有一定程度的智能。经过我们的附加处理，存在一个从原始数据到信息再到知识的发展过程。知识通常被分为归纳知识和演绎知识。**演绎知识**（deductive knowledge）基于对给定数据应用预先指定的逻辑演绎规则来演绎出新信息。数据挖掘致力于**归纳知识**（inductive knowledge）的处理，即从所提供的数据中发现新规则与新模式。知识可以表示为多种形式：在非结构化意义上，它可以通过规则或者命题逻辑来表示；而使用结构化形式，它可以表示为决策树、语义网络、神经网络、类层次或者框架层次。数据挖掘期间的知识发现通常有以下各种描述方式。

- **关联规则**（association rules）。这些规则把一个项目集合的出现与另一个变量集合的值的范围相关联。例如：(1)一个女性消费者在购买手提包时，有可能还买鞋子；(2)一个包含特性a与b的X光图像，可能也显露出特性c。
- **分类层次**（classification hierarchy）。这个目标是从现存事件或事务的集合中建立一个类层次。例如：(1)可以根据以前赊购事务的历史将人们分为5个信用级别；(2)开发一个模型以反映商店位置是否有利，其测定因素可以分为1～10个等级；(3)公共基金可以基于诸如增长、收入、稳定性等特征数据来进行分类。
- **序列模式**（sequential pattern）。要找到动作或事件的序列。例如：如果一个病人因为动脉阻塞和动脉瘤动过心脏旁路手术，手术后一年以内又显示出高血尿症，他或她有可能会在以后的18个月内患肾衰竭。检测序列模式等价于在具有确定的时态关系的事件中，检测它们之间的关联。
- **时间序列中的模式**（patterns within time series）。相似性可以在数据的**时间序列**位置中被发现，时间序列数据是指在相同时间间隔下获得的一个数据序列，比如日销售额或者日股票收盘价。例如：(1)一个公用事业公司ABC和一个名叫XYZ证券的金融公司的股票，2002年在收盘价上表现出相同的模式；(2)两种产品在夏季表现出相同的销售模式，但是在冬季却不同；(3)太阳风中的一种模式可以用来预测地球大气环境的变化等。
- **聚类**（Clustering）。给定的事件或项的集合可以被划分（分割）成“相似”的元素集合。例如：(1)某种病的全部治疗数据可以根据所产生的相似的副作用分组；(2)对于某种新产品，从最可能购买该种新产品到最不可能购买该种新产品，可以将美国成人划分为5种；(3)一群用户对Web中一个文档集合（比如在数字图书馆中）的访问，可以根据文档关键字来分析，从而揭示用户的聚簇或类别。

对于大多数应用，期望得到的知识是上述类型的组合。下面各小节将对上述每种知识类型进行详细的阐述。

12.2　关联规则

12.2.1　市场购物篮模型、支持度与可信度

关联规则的发现是数据挖掘中的主要技术之一。数据库被认为是事务的汇集，每个事务包括了一个项的集合。一个常见的示例是**市场购物篮数据**（market-basket data）。这里购物篮对应于一个顾客在一次去超市时购买的商品。考虑图12-1所示的4个随机取样的事务：

事务id	时间	购买的商品
101	6:35	牛奶、面包、饼干、果汁
792	7:38	牛奶、果汁
1130	8:05	牛奶、鸡蛋
1735	8:40	面包、饼干、咖啡

图12-1　市场购物篮模型中的事务示例

一个**关联规则**的形式如：$X => Y$，其中$X=\{x_1, x_2, \cdots, x_n\}$，$Y=\{y_1, y_2, \cdots, y_m\}$是商品项的集合，对于所有的$i$和所有的$j$，$x_i$与$y_j$是不同的商品。这个关联表明了如果一个顾客购买了$X$，那么他或她也有可能会购买$Y$。一般来说，任何关联规则都有形如LHS（左手边）=> RHS（右手边），这里LHS和RHS是商品项的集合。集合LHS∪RHS称为**项集**（itemset），即顾客购买的商品项的集合。数据挖掘人员关心的关联规则应满足某个度量。两个常见的度量是支持度和可信度。

规则LHS => RHS的**支持度**（support）与项集相关，它表示数据库中特定项集的出现频率。也就是说，支持度是包含项集LHS∪RHS中所有项的事务的百分比。如果支持度低，说明没有足够的证据证明LHS∪RHS中的项目一起出现，因为这个组合只是所有事务的一小部分。支持度的另一个术语是规则的*流行性*（prevalence）。

可信度（confidence）与规则中表现的含义相关。规则LHS=> RHS的可信度计算方法是support（LHS∪RHS）/support（LHS），即（LHS∪RHS）的支持度/（LHS）的支持度。可以把它看作是，一名顾客购买LHS的商品后又购买RHS的商品的可能性。可信度的另一个术语是规则的强度（strength）。

以下面两条规则为例来说明支持度和可信度：“牛奶 => 果汁”和“面包 => 果汁”。考虑图12-1种的4个事务样本，可以看出｛牛奶，果汁｝的支持度为50%，而｛面包，果汁｝的支持度仅为25%。“牛奶 => 果汁”的可信度是66.7%（意味着在3个牛奶出现的事务中，有两个包含果汁）。“面包 => 果汁”的可信度是50%（意味着两个包含面包的事务中有一个包含果汁）。

正如我们所看到的那样，支持度和可信度并不是必然一致的。挖掘关联规则的目标是产生所有可能的规则，这些规则超过用户指定的可信度与支持度的最小阈值。这个问题可以分解为下面两个子问题：

(1) 生成支持度超过阈值的所有的项集。这些项目的集合称为大项集（large itemset或frequent itemset）。注意这里的大是指支持度大。

(2) 对于每个大项集，所有达到最小可信度的规则可以按照如下方法产生：对每一个大项集X和Y，$Y \subset X$，令$Z=X-Y$，如果support（X）/ support（Z）> 最小可信度，则规则$Z => Y$（也就是$X-Y => Y$）是一个有效规则。

通过使用所有大项集和它们的支持度来生成规则是相对简单的。然而，如果项目集合的基数很大，发现所有的大项集和它们的支持度的值是一个主要问题。一个典型的超市有数千种商品，不同项集的数目是2^m，这里m是商品的数量，这样对所有可能的项集支持度的计算就变成计算密集型了。为了减少组合的搜索空间，寻找关联规则的算法应具有以下性质：

- 一个大项集的子集也必须是大项集（也就是说，大项集的每个子集也都超过所要求的最小支持度）。

- 相反地，一个小项集的超集也是小项集（也就是说，没有足够的支持度）。

第一个性质指**向下闭合**（downward closure）。第二个性质称为**反单调性**（antimonotonicity）性质，它有助于减少可能的解决方法的搜索空间。也就是说，一旦找到一个小项集（不是大项集），那么通过对集合增加一个或多个项，对该项集的任意扩展也都产生小项集。

12.2.2　Apriori 算法

使用向下闭合与反单调性性质的第一个算法是**Apriori算法**，如算法12.1所示。

算法12.1　用于发现频繁（大）项集的Apriori算法

输入：m个事务的数据库D，最小支持度$mins$，$mins$表示m的一个分数。

输出：频繁项集，$L_1, L_2, \cdots, L_k$。

Begin　/*对步骤或者语句进行编号是为了增加可读性*/

(1) 扫描一次数据库，计算事务中出现项i_j的事务数量（即count（i_j）），由此计算每个单独项$i_1, i_2, \cdots, i_n$的支持度support（i_j）=count（i_j）/m；

(2) 候选的频繁1项集C_1，C_1为项$i_1, i_2, \cdots, i_n$的集合；

(3) 包含在C_1中的i_j且支持度support（i_j）>=mins的项子集则为频繁1项集L_1；

(4) k=1；

termination = false；

repeat

(1) L_{k+1}=；

(2) 通过组合具有k–1个公共项的L_k的成员，创建候选频集（k+1）项集C_{k+1}，（即有选择地对频繁k项集扩展一个项而形成候选频繁（k+1）项集）；

(3) 此外，只考虑作为C_{k+1}成员的k+1项，使得其每个大小为k的子集都出现在L_k中；

(4) 扫描一次数据库，计算C_{k+1}中每个成员的支持度。如果C_{k+1}中某个成员的支持度大于等于mins，则把该成员加入到L_{k+1}中；

(5) 如果L_{k+1}为空，则termination=ture

否则k=k+1；

until termination;

End;

我们使用图12-1中的事务数据来说明算法12.1，并设最小支持度为0.5。侯选频繁1项集为{牛奶，面包，果汁，饼干，鸡蛋，咖啡}，它们的支持度分别为0.75、0.5、0.5、0.5、0.25和0.25。前4项满足L_1，因为它们的支持度都大于或等于0.5。在repeat循环的第一次迭代中，扩展频繁1项集来创建候选的频繁2项集C_2。C_2包含{牛奶，面包}、{牛奶，果汁}、{面包，果汁}、{牛奶，饼干}、{面包，饼干}和{果汁，饼干}。注意，{牛奶，鸡蛋}没有出现在C_2中，是因为{鸡蛋}是小项集（根据反单调性质），它并没有在L_1中出现。C_2中6个项的支持度分别是0.25、0.5、0.25、0.25、0.5和0.25，这是通过扫描事务集合计算出来的。只有第2个2项集{牛奶，果汁}和第5个2项集{面包，饼干}的支持度大于等于0.5。这两个项集组成了频繁2项集L_2。

在repeat循环的下一次迭代中，构建候选的频繁3项集可以通过为L_2中的集合增加附加的项来实现。但是，对于L_2中的项集，无论做任何扩展都不能保证其所有子集都是包含在L_2中的2项集。例如，考虑{牛奶，果汁，面包}，L_2中没有2项集{牛奶，面包}，根据向下闭合性质，{牛奶，果汁，面包}不是频繁3项集。至此，该算法以L_1和L_2结束，L_1等于{{牛奶}，{面包}，{果汁}，{饼干}}，L_2等于{{牛奶，果汁}，{面包，饼干}}。

目前已经提出了几种其他算法来挖掘关联规则，主要考虑了如何生成候选项集以及如何计算候选项集的支持度。一些算法采用了诸如位图和散列树的数据结构来保存项集信息。还有一些算法建议对数据库进行多次扫描，因为在单次扫描中，项集的潜在数量2^m可能会很大，以至于不能在一次扫描过程中设置计数器。我们将讨论关联规则挖掘的3个改进算法（相对于Apriori算法）：采样算法、

频繁模式树算法和分区算法。

12.2.3　采样算法

采样算法（Sampling algorithm）的主要思想是，从事务数据库选择一个能放入主存的小样本，从这个样本中确定频繁项集。如果这些频繁项集构成了整个数据库的频繁项集的超集，则可以通过扫描数据库的其余部分来确定真正的频繁项集，以计算超集项集的准确支持度值。频繁项集的超集一般可以通过降低最小支持度从样本中找到，例如，Apriori算法。

在一些不常见的情况下，可能会丢失一些频繁项集，所以需要对数据库进行第二次扫描。为了确定是否丢失了频繁项集，可以使用负边界（negative border）的概念。对频繁项集*S*和项集*I*而言，负边界是包含在PowerSet（*I*）中而不包含在*S*中的最小项集。基本思想是，频繁项集集合的负边界包含了可能是频繁项集的最近项集。考虑集合*X*不包含在频繁项集中的情况，如果*X*的所有子集都包含在频繁项集中，那么*X*就在负边界中。

我们将用以下的示例来说明。考虑项集*I*={A, B, C, D, E}，令组合的频繁项集为*S*={{A}, {B}, {C}, {D}, {AB}, {AC}, {BC}, {AD}, {CD}, {ABC}}，其大小为1~3。负边界为{{E}, {BD}, {ACD}}。集合{E}是未包含在S中的唯一1项集，{BD}是未包含在S中的唯一2项集，但是其1项集子集包含在S中，{ACD}是未包含在S中的唯一3项集，但是其2项集子集都包含在S中。为了确保分析样本数据时没有丢失大项集，必须确定负边界中项集的支持度，所以负边界是很重要的。

在扫描数据库其余部分时要确定负边界的支持度。如果找到了满足以下条件的一个项集*X*：它在负边界中，并属于所有频繁项集集合，则可能*X*的一个超集也是频繁的。如果是这样，就需要第二次扫描数据库来确定是否已经找到了所有的频繁项集。

12.2.4　频繁模式树算法

频繁模式树算法（frequent-pattern tree algorithm）是根据以下事实提出的：基于Apriori的算法会生成并测试数据量非常大的候选项集。例如，若有1000个频繁1项集，那么Apriori算法要生成

$$\binom{1000}{2}$$

即499 500个候选2项集。FP增长算法是防止生成大量候选项集的一种方法。

这个算法首先按照FP树（频繁模式树）生成数据库的一个压缩版本。FP树存储了有关的项集信息，并允许高效地发现频繁项集。实际的挖掘过程采用了分而治之的策略，将此挖掘过程分解为较小的任务集合，每个集合处理一个条件FP树，条件FP树是原始树的子集（投影）。我们首先来试验如何构造FP树。先扫描数据库并计算频繁1项集及其支持度。在这个算法中，支持度是包含该项的事务的数量，而不是包含该项的事务的分数。然后把频繁1项集按支持度非增的顺序排序。接着，使用NULL标号创建FP树的根。再次扫描数据库，对于该数据库中的每个事务*T*，把*T*中的频繁1项集按照上述对频繁1项集的做法排列。我们可以将*T*的有序列表指定为包括第一项（即表头）和其余项（即表尾）的表。从根结点开始，将项集信息（表头、表尾）递归插入到FP树中，如下：

(1) 如果FP树的当前结点*N*有一个孩子的项名等于表头（head），则把与该结点*N*相关的计数器值加1，否则创建一个新结点*N*，计数器值为1，把*N*链接到其父结点，并把*N*与项头表链接（用于高效的树遍历）。

(2) 如果表尾非空，则仅以表尾作为有序表重复步骤1，也就是说，删除旧的表头，而新表头是表尾中的第一项，其余的项成为新的表尾。

项头表是在建立FP树的过程中创建的，对于每个频繁项，每一个条目包含3个字段：项标识符、支持度计数和结点链。项标识符和支持度计数不需要多做说明。结点链是一个指向FP树中该项出现

地方的指针。由于FP树中一个项可能有多次出现，所以把这些项链接到一起构成一个列表，项头表中的结点链指向列表的开始结点。我们使用图12-1中的事务数据来说明如何构建FP树：设最小支持度为2，对4个事务的一次扫描会生成如下具有相关支持度的频繁1项集：{(牛奶，3)}，{(面包，2)}，{(饼干，2)}，{(果汁，2)}。再次扫描数据库并重新处理每个事务。

对于第1个事务，创建一个有序列表*T*，*T*={牛奶，面包，饼干，果汁}。*T*中的项来自第一个事务的频繁1项集。这些项按照第1次扫描中发现的1项集的降序排列（也就是先是“牛奶”，然后是“面包”，等等）。我们为FP树创建一个NULL根结点，插入一个“牛奶”作为根结点的孩子，“面包”作为“牛奶”的孩子，“饼干”作为“面包”的孩子，“果汁”作为“饼干”的孩子。并调整项头表中对应频繁项的条目。

对于第2个事务，有序列表为{牛奶，果汁}。从根结点开始可以看到，存在一个带有标号为“牛奶”的孩子结点，所以移去该结点并更新它的计数（该计数中包含了“牛奶”的第二个事务）。此时可以看到，当前结点没有带有标号为“果汁”的孩子，所以创建一个带有标号为“果汁”的新结点。对项头表进行调整。

第3个事务只有1频繁项{牛奶}。再从根结点开始，可以看到，存在带有标号为“牛奶”的结点，所以移去该结点，增加它的计数，并调整项头表。最终的事务包含频繁项{面包，饼干}。在根结点可以看到，没有带有标号为“面包”的孩子。因此，为根创建一个新的孩子，并初始化它的计数器，然后插入“饼干”作为该结点的孩子，并初始化它的计数器。更新项头表之后即可结束，我们得到FP树和项头表，如图12-2所示。如果检查FP树就可以看到，它的确以压缩形式表示了原始的事务（即，只展示了来自每个大1项集事务中的项）。

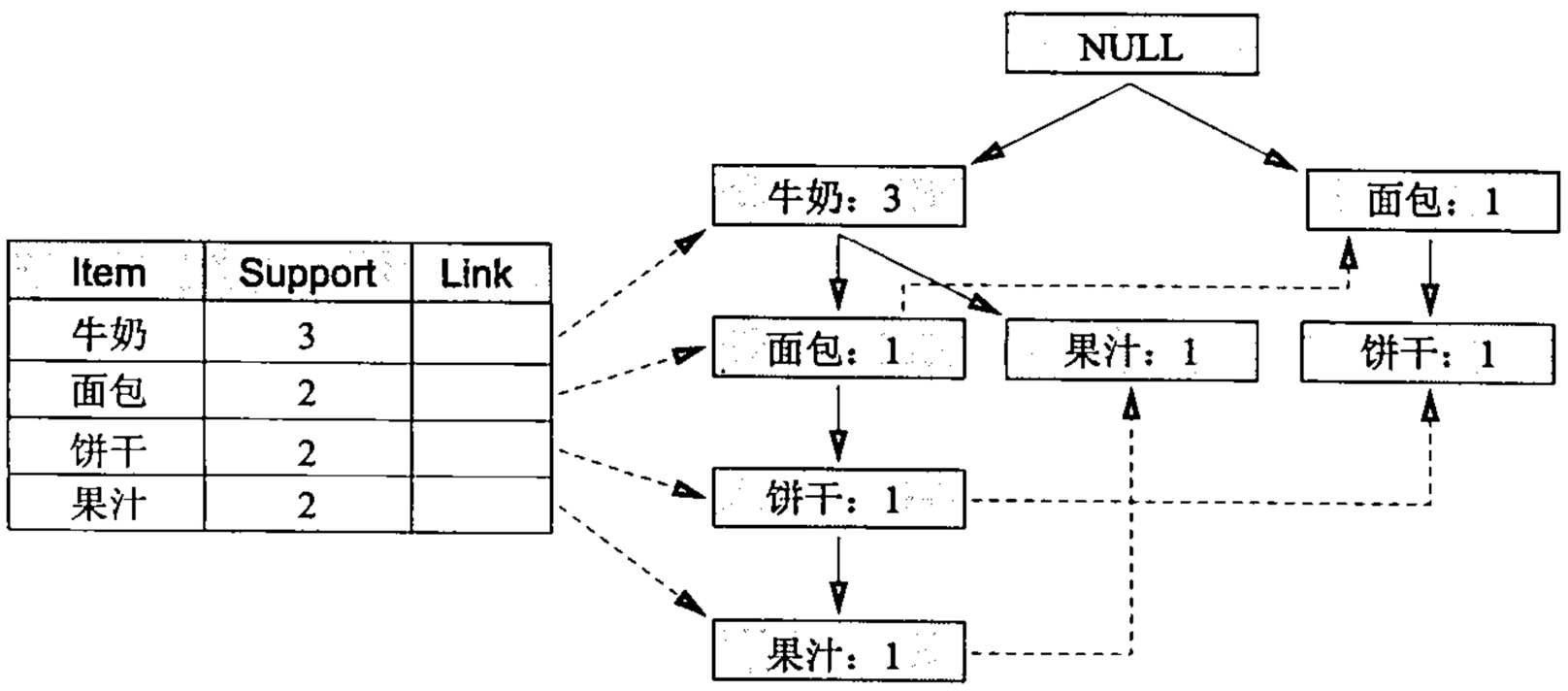


图12-2　FP树与项头表

算法12.2　发现频繁项集的FP增长算法

输入： FP树和最小支持度mins。
输出： 频繁模式（项集）。

```
procedure FP-growth （tree, alpha）；
Begin
  if 树包含一条路径P then
    对于该路径中结点的每个组合beta
        生成模式（beta∪alpha）
          支持度=beta中结点的最小支持度
    else
      对于树项头表中的每个项do
        begin
        生成模式beta=（i∪alpha），其support=i.support;
```

```
        构建beta的条件模式基;
        构建beta的条件FP树beta_tree;
        IF beta_tree非空 then
            FP-growth（beta_tree，beta）;
        end;
    End;
```

算法12.2用于挖掘频繁模式的FP树。利用FP树，对于一个给定频繁项，从对应此项的项头表开始，在FP树中遍历结点链，就可能找到包含该项的所有频繁模式。算法以频繁1项集（后缀模式）开始，构建它的条件模式基，然后构建它的条件FP树。条件模式基由一组前缀路径组成，而这里频繁项是后缀。例如，如果考虑项“果汁”，从图12-2可以看到，FP树中以“果汁”结束的有两条路径：（牛奶，面包，饼干，果汁）与（牛奶，果汁）。相关联的两条前缀路径是（牛奶，面包，饼干）与（牛奶）。条件FP树由该条件模式基中的模式构建，挖掘就是在这棵FP树上递归实现的。将后缀模式与从条件FP树生成的频繁模式连接起来，就可以形成频繁模式。

我们使用在图12-1中的数据和图12-2的树来说明这个算法。过程FP-growth带有两个参数被调用：最初的FP树和变量alpha的NULL值。由于最初的FP树中有多条路径，所以要执行第一个if语句的else部分。我们从频繁项“果汁”开始，按最低支持度的顺序检查频繁项（即，从表中的最后一个条目到第一个条目）。变量beta设置为“果汁”，支持度等于2。

沿着项头表中的结点链，构建条件模式基，它由两条路径组成（以“果汁”作为后缀）。两条路径分别为（牛奶，面包，饼干：1）与（牛奶：1）。条件FP树只有单个结点（牛奶：2）。这是由于结点面包和饼干的支持度只为1，它小于最小支持度2。使用仅有一个结点的FP树（即，牛奶：2）和beta值“果汁”递归地调用该算法。由于这个FP树仅有一条路径，所以会生成beta和该路径中结点的所有组合（即{牛奶，果汁}），其支持度等于2。

接着，要使用频繁项“饼干”。把变量beta设置为“饼干”，支持度为2。沿着项头表中的结点链，构建两条路径组成的条件模式基，两条路径分别为（牛奶，面包：1）和（面包：1）。条件FP树只有一个结点（面包：2）。使用仅有一个结点（即（面包：2））的FP树和beta值“饼干”递归调用该算法。由于FP树只有一个路径，所以生成beta和该路径中结点的所有组合，即{面包，饼干}，其支持度为2。接下来考虑频繁项“面包”，变量beta设置为“面包”，支持度为2。沿着项头表中的结点链，构建由一条路径组成的条件模式基，该路径为（牛奶：1）。由于其计数值小于最小支持度，所以条件FP树为空，因此不会产生频繁模式。

要考虑的最后一个频繁项是“牛奶”。这是项头表中的第一项，并且它的条件模式基和条件FP树都为空。因此，不增加频繁模式。执行该算法产生如下频繁模式（或项集）及其支持度：{{牛奶：3}，{面包：2}，{饼干：2}，{果汁：2}，{牛奶，果汁：2}，{面包，饼干：2}}。

12.2.5　分区算法

另一种算法，被称为**分区算法**（Partiton algorithm）[①]，以下是对此算法的概述。如果给定一个数据库，其中有少量潜在的大项集，譬如说，有几千个大项集，那么使用分区技术扫描一次就可以测试所有大项集的支持度。分区算法把数据库分成不相交的子集。这些子集都作为单独的数据库来加以考虑，并且该分区的所有大项集（称为*局部频繁项集*），都可以经过一次扫描产生。如果每个分区都能整个放在主存中，就可以在所有分区上高效地使用Apriori算法。这样一来，每次扫描只需要读取一次分区。使用分区方法唯一要注意的是：每个分区中所使用的最小支持度与初始值的含义稍有不同。最小支持度要基于分区的大小，而不是基于确定局部频繁（大）项集的数据库的大小。实际的支持度阈值与前面给定的阈值相同，但仅对分区计算支持度。

① 关于该算法的细节，实现该算法所使用的数据结构及其性能比较，见（Savasere et al.，1995）。

在每一遍扫描结束时，计算每个分区的所有频繁项集的并集。这形成了整个数据库的全局候选频繁项集。当合并这些列表时，可能会包含一些假情况（false positive）。也就是说，某些项集在一个分区中是频繁（大）项集，但是在其他多个分区中可能不是频繁项集，因此在考虑原始数据库时可能未达到最小支持度。注意，这里并没有假的反向情况（false negative），即不会漏掉大项集。第一遍扫描中识别出的候选全局大项集要在第二遍扫描中进行验证。也就是说，要在整个数据库中计算其真实的支持度。第二个阶段结束时，就会识别出所有的全局大项集。为了达到更高的效率，分区算法很自然地要并行或分布式实现。目前已经有人提出了该算法的改进算法[①]。

12.2.6　其他类型的关联规则

1．层次中的关联规则

有些关联类型，由于特殊原因人们对它们特别感兴趣。这些关联发生在项的层次之间。典型地，可以根据域的自然特性把项划分在不相交层次中。例如，超市中的食品、百货公司中的商品、或者体育用品商店的商品，都能够分成可以形成层次的类和子类。图12-3给出了超市商品的分类。图中展示了两个层次，饮料与甜点。整个组可能不会产生形如饮料 => 甜点，或者，甜点 => 饮料这样的关联。然而Healthy品牌冰冻酸乳酪 => 瓶装水，或者Richcream品牌冰淇淋 => 冰镇果酒之类的关联可能会满足可信度与支持度，成为人们感兴趣的有效关联规则。

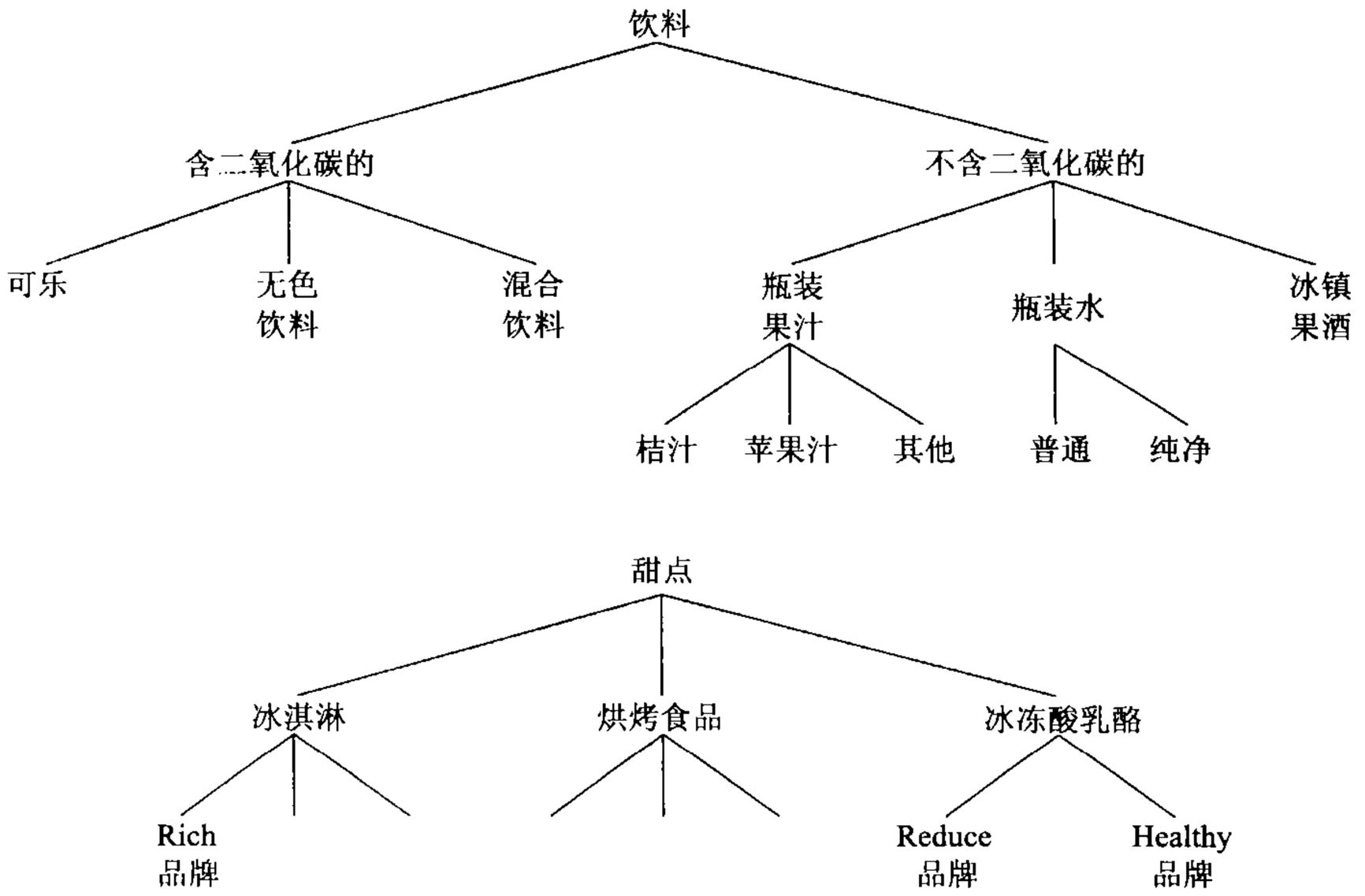


图12-3　超市中商品项的一种分类方法

因此，如果应用领域有一个项集可以自然分类成层次结构，人们对于在层次内部发现关联规则不一定特别感兴趣，特别感兴趣的是层次之间的关联。这种关联有可能发生在不同层次的项组之间。

2．多维关联

发现关联规则包含文件中的搜索模式。在数据挖掘这一章一开始就给出了一个顾客事务文件的示例，这个事务有3个维：事务id、时间和购买的商品项。然而，迄今为止所介绍的数据挖掘任务和算法只涉及其中的一个维：购买的商品项。如下规则是这样的一个示例，其中包含了单个维的标号：购买项(牛奶)=>购买项(果汁)。它对发现涉及多个维的规则可能是有意义的，举例来说，时间(6:30～

① 见（Cheung et al.，1996）以及（Lin & Dunham，1998）。

8:30）=>购买项（牛奶）。这样的规则称为多维关联规则。维表示文件记录的属性，从关系的角度讲，或表示一个关系的行的列，可以有分类属性或量化属性。分类属性有一个有限的值的集合，这些值并不显示有序联系。量化属性是数字，这些数字的值显示了有序联系，例如购买项就是一个分类属性，而事务id和时间是量化属性。

处理量化属性的一个方法是把量化属性的值划分为不相交的区间，并为这些区间指派标号。可以以基于域特殊知识的静态方式来实现这种方法。例如，在概念层次可以把工资值（salary）分为3个不同的类：低收入（0<Salary<29 999）、中等收入（30 000<Salary<74 999）和高收入（Salary>75 000）。这样一来，量化属性看起来就像是分类属性，所以可以使用典型的Apriori类型算法，或者其改进算法之一来进行规则挖掘。另一种分区方法是基于数据分布来对属性值进行分组，例如相等-深度划分，并为每个分区指派一个整数值。这个阶段的分区可能相当细，也就是说会划分出相当多的区间。因此，在挖掘过程中，这些分区可以与相邻的其他分区结合，条件是这些分区的支持度小于预定义的最大值。在此也可以使用Apriori类型的算法进行数据挖掘。

3．负向关联

发现负向关联（negative association）的问题比发现正向关联（positive assosiation）更加困难。负向关联的形式如：60%购买薯条的顾客不买瓶装水（这里，60%是指负向关联规则的可信度）。在包含10 000项的数据库中，存在$2^{10\,000}$种可能的项组合，但其中大多数在数据库中连一次都不会出现。如果某个不存在的项组合被用来表示一个负关联，则可能在RHS中有无数的负向关联规则是根本没有意义的。问题是如何去发现有意义的负向关联规则。一般来说，我们对两个特定项的集合几乎不在同一事务中出现的情况感兴趣。这里提出两个问题：

(1) 对于10 000件商品的存货，任意两件商品被同时购买的可能性是（1/10 000）×（1/10 000）=10^{-8}。如果我们发现这两件商品被一起购买的实际支持度为0，它不代表与预期的结果有显著的偏离，因此不是一个有意义（负向）的关联。

(2) 另一个问题更严重。我们用很低的支持度寻找项的组合，并且存在无数低的甚至为零的支持度。比如，包含1 000万个事务的数据集有10 000个项的25亿个配对组合中的大部分是未出现的。这将产生数十亿无用的规则。

因此，为了获得有意义的负向关联规则，必须利用项集先前的知识。一种方法是使用层次结构。假设我们使用图12-4中所示的软饮料与薯条的层次。

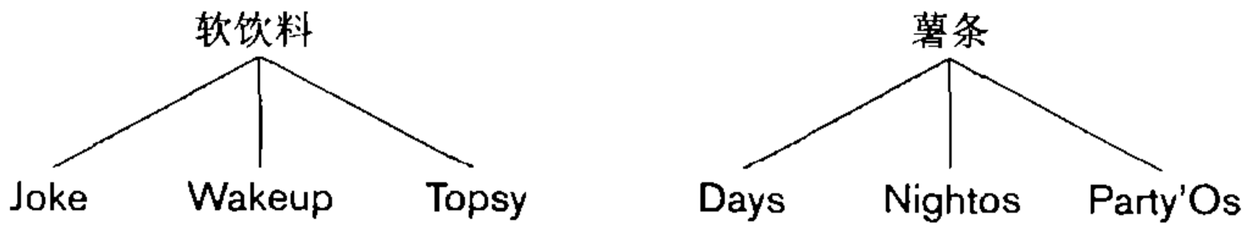


图12-4　软饮料与薯片的简单层次

软饮料和薯条之间展示了一个强正向关联，如果对于一个购买Days品牌薯条的顾客主要购买Topsy品牌而不是Joke品牌或Wakeup品牌的软饮料，这个事实存在很大的支持度，那说明这个关联是有意义的。因为我们通常期望如果在Days与Topsy之间存在这种强关联，那么也将在Days与Joke或Days与Wakeup之间存在这种强关联[①]。

在图12-3所示的冰冻酸乳酪与瓶装水的分组中，假设在各自的种类中，Reduce与Healthy之间的比为80:20，Plain与Clear之间的比为60∶40。就可能给出这样一种情况：在包含冰冻酸乳酪与瓶装水的交易中，Reduce与Plain一起出现的概率为48%。如果发现这个支持度只有20%，说明在Reduce与Plain之间有明显的负向关联，这也是一种有意义的发现。

在上面的情况中，以项的概括层次（也就是图12-3中给出的饮料和甜点的层次）存在的正向关联

① 简单地说，认为事务均衡地分布在层次结构的各个成员中。

（如冰冻酸乳酪和瓶装水分组之间）和项的分布（比如相关组之间的商标名称）的形式给定领域知识，发现负向关联的问题是很重要的。近期的工作已经由Georgia Tech的数据库小组做出报告（见本章的参考文献）。发现负向关联的范围受限于对项的层次与分布的了解，而负向关联的指数增长仍然是个挑战。

12.2.7　关联规则的附加考虑

在真实的数据库中挖掘关联规则由于如下因素将会变得更加复杂：

- 大多数情况下，项集的基数非常大，事务的数量也非常大。例如，一些零售业和通信业的操作型数据库，每天汇集上千万个事务。
- 在诸如地理位置和季节等因素方面，事务的表现各不相同，导致抽样困难。
- 项分类存在于多维中。因此，使用域知识驱动发现过程是很困难的，特别是对于负向关联规则。
- 数据质量是可变的。由于丢失、错误、冲突，以及在多个产业中的冗余数据等，而存在值得注意的问题。

12.3　分类

分类是学习一种模型的过程，该模型描述了数据的不同类别，这些类别是预先确定的。例如，在银行应用中，申请信用卡的客户可以按“高风险”、“中风险”或者“低风险”进行分类。因此，这类活动也称为**有监督的学习**（supervised learning）。一旦这个模型被建立，就可以使用它来对新数据进行分类。学习模型的第一步，是通过使用已经分类的训练数据集合来实现的。训练数据中的每个记录包含一个属性，该属性称为类标号，它表明记录所属的类。被产生的模型通常是以决策树或者规则集合的形式出现。与该模型以及产生这个模型的算法相关的主要问题包括：模型预测新数据正确类别的能力、与算法相关的计算代价以及算法的可伸缩性。

我们将以决策树形式的模型来检验这个方法。简单地说决策树不过是描述每个类的图形表示，换句话说，就是分类规则的表示。图12-5显示了一个决策树示例。从图中可以看到，如果一名顾客是已婚的（married），并且salary≥50k，那么其银行信用卡就是低风险（poor risk）的。这是描述低风险类的一个规则。该类以及另外两个类的其他规则，通过从根到每个叶子结点遍历整个决策树来形成。算法12.3展示了从训练数据集合构建决策树的过程。最初，所有的训练样本都在树的根上。这些样本基于所选属性递归划分。结点上用于划分样本的属性是具有最好分离准则的属性，例如，最大化信息得益度量的属性。

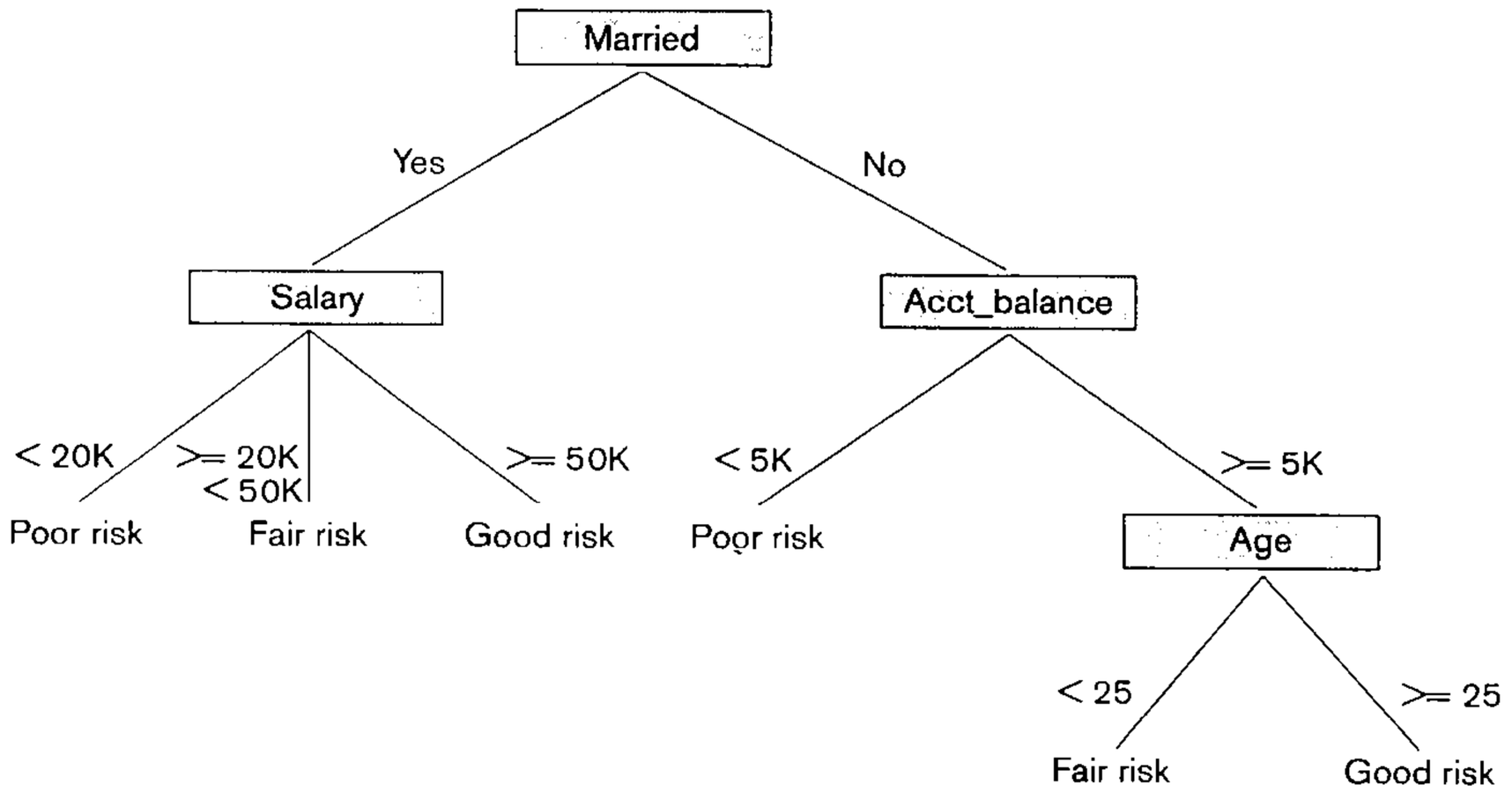


图12-5　信用卡应用的决策树示例

算法12.3　决策树归纳算法

输入：训练数据记录集合（$R_1, R_2, \cdots, R_m$）与属性集合（$A_1, A_2, \cdots, A_n$）。
输出：决策树。

```
Procedure Build_tree (records, attributes)
Begin
  创建一个结点N;
  if 所有的Records都属于同一类C then
    返回N，作为带有类标号为C的叶子结点;
  if attributes为空 then
    返回N，作为带有类标号为C的叶子结点，从而大部分records都属于它;
    从attributes中选出attribute Ai（具有最高的信息得益）;
  把N的标号设为Ai;
  对于Ai的每个已知值vj do
    begin
      对于条件Ai= vj从结点N增加一个分支;
      Sj=records的子集，这里Ai= vj;
      if Sj为空 then
        增加一个带有类标号为C的叶子结点L，这样大部分records都属于它，并返回L
      else
        增加由Build_tree(Sj, atrributes–Ai)返回的结点;
    end;
End;
```

在说明算法12.3之前，我们先详细解释一下**信息得益**（information gain）度量的概念。之所以采用**熵**（entropy）作为信息得益度量，是受到了以下目标的启发：最小化在结果分区中对样本数据进行分类所需的信息，并且最小化对新记录分类所需条件测试的期望次数。对s个样本的训练数据分类所需的期望信息由以下公式给出，其中类属性有n个值（$v_1, v_2, \cdots, v_n$），s_i是属于类标号v_i的样本数量：

$$I(S_1, S_2, \cdots, S_n) = -\sum_{i=1}^{n} p_i \log_2 p_i$$

其中p_i是标号为v_i的类的随机样本的概率，p_i的一个估值是s_i/s。把值为$\{v_1, \cdots, v_m\}$的属性A作为决策树分裂的测试属性。属性A把样本划分为子集$S_1, \cdots, S_m$，其中每个S_j中的样本对于属性A的值为v_j。每个S_j可以包含属于任何类的属性。S_j中属于类i的样本数量记为S_{ij}。与使用属性A作为测试属性相关联的熵定义为

$$E(A) = \sum_{j=1}^{m} \frac{S_{1j} + \cdots + S_{nj}}{S} \cdot I(S_{1j}, \cdots, S_{nj})$$

把p_i用p_{ij}代替，$p_{ij}=s_{ij}/s_j$，对$I(s_1, \cdots, s_n)$使用该公式，就可以定义$I(s_{1j}, s_{2j}, \cdots, s_{nj})$。现在，将属性$A$分区的信息得益Gain（$A$）定义为$I(s_1, \cdots, s_n)–E(A)$。我们可以使用图12-6中的样本训练数据来说明这一算法。

RID	Married	Salary	Acct_balance	Age	Loanworthy
1	no	>=50K	<5K	>=25	yes
2	yes	>=50K	>=5K	>=25	yes
3	yes	20K. . .50K	<5K	<25	no
4	no	<20K	>=5K	<25	no
5	no	<20K	<5K	>=25	no
6	yes	20K. . .50K	>=5K	>=25	yes

图12-6　分类算法的样本训练数据

属性RID表示用于标识单个记录的记录标识符，它是一个内部属性。我们使用RID标识示例中一个特定的记录。首先，计算把6个记录的训练数据分为$I(s_1, s_2)$所需的期望信息，其中第一个类标号值对应“*yes*”，第二个对应“*no*”。所以

$$I(3,3) = -0.5\log_2 0.5 - 0.5\log_2 0.5 = 1$$

现在，来计算这4个属性中的每一个的熵。Married = yes时，s_{11}=2，s_{21}=1，$I(s_{11}, s_{21})$=0.92。Married=no时，s_{12}=1，s_{22}=2，$I(s_{12}, s_{22})$=0.92。所以使用属性Married作为分区属性对样本分类需要的期望信息为

$$E(\text{Married}) = 3/6\ I(s_{11}, s_{21}) + 3/6\ \text{I}(s_{12}, s_{22}) = 0.92$$

信息中的得益Gain（Married）为1–0.92 = 0.08。采用类似的步骤来计算其他3个属性的得益，最后得到：

E(Salary) = 0.33	Gain(Salary) = 0.67
E(Acct_balance) = 0.92	Gain(Acct_balance) = 0.08
E(Age) = 0.54	Gain(Age) = 0.46

由于最大的增益是对属性Salary产生的，所以选择它作为分区属性。使用标号*Salary*创建树的根结点，它有3个分支，每个分支对应一个Salary值。对于小于20k和大于等于50k的两个值，所划分的所有相应的样本（小于20k，即RID为4和5的记录；大于等于50k，即RID为1和2的记录）都分别划分到对应这两个值的同一类*loanworthy no*和*loanworthy yes*中。因此要为这两个分区分别创建一个叶子结点。需要扩展的唯一分支是对于值为20k…50k的分支，对此有两个样本，即训练数据中RID为3和6的记录。使用这两个记录继续这个过程，将会发现：Gain(Married) = 0，Gain(Acct_balance) = 1，Gain(Age) =1。

因为Age和Acct_balance都有最大的得益，所以可以选择其中之一作为分区属性。假设选择Age为分区属性，用Age作为标号增加一个结点，这个结点有两个分支，一个小于25，一个大于或等于25。每个分支对其余的样本数据进行划分，这样样本记录分别属于各个分支，因此也属于一个类。最后创建两个叶子结点，到此结束。最终的决策树如图12-7所示。

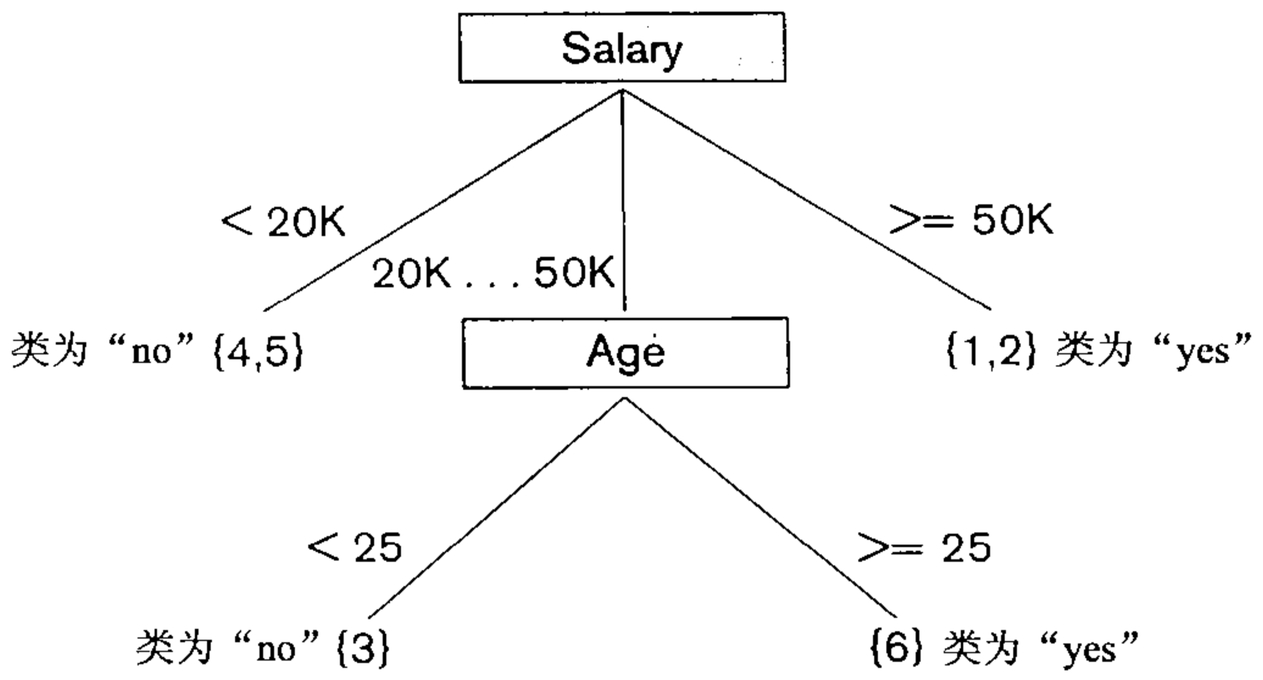


图12-7　基于样本训练数据的决策树，其中叶子结点由分区记录的RID集合表示

12.4　聚类

前面的分类数据挖掘任务处理基于使用预分类的训练样本分区数据。但是在没有训练样本时，这种方法对数据分区也是有用的，这被称为**无监督学习**（unsupervised learning）。例如，在商业业务中，确定有相似购买模式的客户群是相当重要的。或者，在医学中，确定对处方药品有相似反应的

患者群也是相当重要的。聚类的目标是把记录分组，这样在同一个组中的记录彼此都是相似的，并且同其他组中的记录是不相似的。通常这些组之间是不相交的。

聚类的一个重要因素是使用相似性函数。数据为数字型时，一般使用基于距离的相似性函数。例如，可以使用欧几里得距离来测量相似性。考虑两个n维的数据点（记录）r_j和r_k。对于这两个记录，认为第i维的值分别为r_{ji}和r_{ki}。n维空间的点r_j和r_k之间的欧几里得距离计算公式为

$$\mathrm{Distance}(r_j, r_k) = \sqrt{|r_{j1} - r_{k1}|^2 + |r_{j2} - r_{k2}|^2 + \cdots + |r_{jn} - r_{kn}|^2}$$

两点之间的距离越小，就认为它们之间的相似性越大。k-均值（k-Means）算法是一个经典的聚类算法，如算法12.4所示。

算法12.4　k均值聚类算法

输入：数据库D，该数据库的m条记录$r_1,\cdots, r_m$和期望的聚类个数k
输出：具有最小平方误差判别标准（squared-error criterion）的k个聚类集合
Begin
　随机选取k个记录作为k个聚类的形心（centroid）；
　repeat
　把每条记录r_i赋值给一个聚类，这样r_i与聚类形心（平均值）之间的距离是k个聚类中最小的；
　对于基于赋值给该聚类的记录，重新计算每个聚类的形心（平均值）；
　until 没有变化；
End;

这个算法首先随机选取k个记录来表示聚类$C_1,\cdots,C_k$的形心（平均值）$m_1,\cdots,m_k$。所有记录都被放在一个给定的聚类中，这个聚类基于该记录与聚类平均值之间的距离。如果m_i与记录r_j之间的距离在所有的聚类平均值中是最小的，则把记录r_j放在聚类C_i中。一旦所有的记录都放到了聚类中，则重新计算每个聚类的平均值。重复该过程，然后再次分析每个记录，并放置到平均值最接近的聚类中。这个过程可能需要几次迭代，虽然该算法可能会在一个局部最优点终止，但是该算法是收敛的。算法的终止条件一般为一个平方误差判别标准。对于平均值为$m_1,\cdots,m_k$的聚类$C_1,\cdots,C_k$，误差定义为

$$\mathrm{Error} = \sum_{i=1}^{k} \sum_{\forall r_j \in C_i} \mathrm{Distance}(r_j, m_i)^2$$

下面来分析如何把算法12.4应用到图12-8中的（二维）记录上。假设期望的聚类数量k为2，令算法为聚类C_1选择RID为3的记录，为聚类C_2选择RID为6的记录来作为初始聚类形心。其他的记录在repeat循环的第一次迭代中指派给这两个聚类之一。RID为1的记录到C_1的距离为22.4，到C_2的距离为32.0，所以把它加入到聚类C_1。RID为2的记录到C_1的距离为10.0，到C_2的距离为5.0，所以加入到聚类C_2。RID为4的记录到C_1的距离为25.5，到C_2的距离为36.6，所以加入到聚类C_1。RID为5的记录到C_1的距离为20.6，到C_2的距离为29.2，所以加入到聚类C_1。现在，计算两个聚类的新的平均值（形心）。聚类C_i有m维的n个记录，其平均值为向量

$$\overline{C_i} = \left(\frac{1}{n} \sum_{\forall r_j \in C_i} r_{ji}, \cdots, \frac{1}{n} \sum_{\forall r_j \in C_i} r_{jm} \right)$$

C_1的新平均值为(33.75, 8.75)，C_2的新平均值为(52.5, 25)。进行第二次迭代，6个记录在两个聚类之中的安排如下：RID为1、4和5的记录放到C_1中，RID为2、3和6的记录放到C_2中。C_1与C_2的平均值分别重新计算为(28.3, 6.7)和(51.7, 21.7)。在下一次迭代中，所有记录都保留在之前的一个聚类中，算法终止。

RID	Age	Years_of_service
1	30	5
2	50	25
3	50	15
4	25	5
5	30	10
6	55	25

图12-8　聚类示例的2维样本记录（未考虑RID列）

传统的做法是聚类算法都假设整个数据集合都可以放在主存中。最近，研究人员已经开发了用于大规模数据库的有效的和可伸缩的（scalable）算法。其中一个算法称为BIRCH。BIRCH是一个混合算法，它使用了层次聚类算法和附加的聚类方法，层次聚类算法建立数据的表示树，附加的聚类方法用于树的叶子结点。BIRCH算法有两个输入参数：一个参数指定可用的主存容量，另一个是聚类半径的初始阈值。主存用于存储描述性聚类信息，例如聚类的中心（平均值）以及聚类的半径（假设聚类是球形的）。半径阈值影响产生的聚类数量。例如，如果聚类半径阈值很大，虽然有多个记录也会被分成很少的几个聚类。该算法试图维持聚类的数量，从而使得其半径小于半径阈值。如果可用内存不足，则要加大半径阈值。

BIRCH算法要顺序读取数据记录，并把它们插入到内存中的一个树结构中，该结构用来保存数据的聚类结构。基于记录与聚类中心之间的距离，把记录插入到正确的叶子结点中（潜在聚类）。插入记录的叶子结点可能还要分裂，这取决于聚类更新的中心和半径以及半径阈值参数。另外，分裂时要存储额外的聚类信息，如果内存不足，则要加大半径阈值。加大半径阈值实际上可能会产生负面影响，那就是减少聚类数量，因为这样可能会合并一些结点。

总体上说，BIRCH是一个高效的聚类方法，从聚类的记录数量来说，它的计算复杂度是线性的。

12.5　其他数据挖掘问题的方法

12.5.1　序列模式的发现

序列模式的发现基于一个项集序列的概念。例如像前面所讨论的超市购物篮事务是按购买时间排序的，这样的顺序产生了一个项集序列。比如说{牛奶，面包，果汁}，{面包，鸡蛋}和{饼干，牛奶，咖啡}是一个顾客三次去超市中购物的**项集序列**（sequence of itemset）。项集序列S的**支持度**是在给定序列集合U中的百分比，其中S是U的一个子序列。在这个示例中，{牛奶，面包，果汁}，{面包，鸡蛋}，{面包，鸡蛋}与{饼干，牛奶，咖啡}都被认为是**子序列**（subsequence）。接下来，识别序列模式的问题是从给出的满足用户定义的最小支持度的序列集合中找出所有的子序列。序列S_1，S_2，S_3，…是“一个购买了商品集S_1的顾客可能也会购买商品集S_2，接着会买S_3，等等”这个事实的**预测器**（predictor）。这一预测的产生是基于该序列以前的支持度。已经研究了用于序列检测的各种算法。

12.5.2　时间序列中模式的发现

时间序列是事件的连续。每个事件可以是一个事务的一种给定类型。例如，对于每个股票或基金，收盘价是每个工作日都发生的事件。每个股票或基金的这些值序列组成了一个时间序列。对于一个时间序列，可以像以前所做的那样通过分析序列与子序列来寻找不同的模式。例如，我们可能发现股票连续上升或保持n天稳定的一段时期，或者发现股票与前一天收盘价波动不超过1%的最长的一段时期，或者发现股票涨幅或跌幅最大的一段时期。时间序列可以通过建立相似度（similarity）的度量进行比较，以便识别哪些公司的股票有类似的表现。时间序列的分析和挖掘是时态数据管理的一种扩展功能（见第8章）。

12.5.3　回归

回归（regression）是分类规则的一个特殊应用。如果把分类规则看作一个函数，它把其中的变量映射到目标类的变量，这个规则就被称为**回归规则**（regression rule）。当根据元组来预测变量的值，而不是将一个元组数据从关系映射到具体的类时，应该使用回归。例如，考虑一个关系：

LAB_TESTS (patient ID, test 1, test 2, ⋯, test n)

它包含一个病人n次化验结果的值。我们希望预测的目标变量是P，即病人存活的概率。回归规则的形式如下：

$$(\text{test 1 in range}_1)\text{and}(\text{test 2 in range}_2)\text{and}\ \cdots\ (\text{test } n \text{ in range}_n) \Rightarrow P = x,$$

或者$x < P \leqslant y$

这个选择依赖于我们可以预测出P的一个单值，还是P值的一个范围。如果把P看作一个函数：

$$P = f(\text{test 1, test 2, } \cdots, \text{test } n)$$

这个函数被称为预测P的**回归函数**（regression function）。通常，如果函数以如下形式出现：

$$Y = f(x_1, x_2, \cdots, x_n)$$

并且f在变量x_i的域中是线性的，那么从给定的元组$<X_1, X_2, \cdots, X_n, y>$的集合中推出$f$的过程被称为**线性回归**（linear regression）。线性回归是一种普遍使用的统计技术，用来拟合目标变量为y的n维中的一个观测值或点的集合。

回归分析是许多研究领域中十分常见的进行数据分析的工具。发现预测目标变量的函数就等价于一个数据挖掘操作。

12.5.4　神经网络

神经网络（neural network）是从人工智能研究中导出的一种技术，人工智能研究使用普通的回归并提供迭代的方法来实现。神经网络使用曲线拟合的方法从样本集合中推导出函数。这种技术提供一种“*学习方法*”，这种学习方法由作为初始推理和学习的测试样本驱动。使用这种学习方法，新的输入来自对已知样本的插值的响应。但是这种插值依赖于通过学习方法发展起来的宇宙模型（world model，问题域的内部表示）。

神经网络在广义上可以分为两类：有监督的网络和无监督的网络。**有监督的学习**是试图减少输出错误的适应方法，而那些不借助于样本输出来进行内部表示的方法称为**无监督学习**（unsupervised learning）。

神经网络是自适应的；也就是说，它可以从一个具体问题的信息中学习。神经网络在分类任务中表现出色，因此在数据挖掘中非常有用。然而它并不是没有问题，尽管能够自学习，但是它没有提供一种好的描述方法来表示所学到的东西。它输出的数量很大并且不容易理解。另一个局限是，神经网络开发的内部表示并不唯一。而且，通常神经网络对时间序列数据建立模型时是有困难的。但是，尽管有这些缺点，神经网络仍然是流行的，并且经常被一些商业化的软件提供商使用。

12.5.5　遗传算法

遗传算法（genetic algorithm，GA）是一类随机搜索程序，它能够在一个广泛的搜索空间拓扑范围中进行适应的和鲁棒的搜索。GA模拟自然界中生物物种的适者生存的进化机制，是由Holland[①]提出的。

GA成功地应用于多个的领域，如图像分析、调度和工程设计等。

---

① Holland在名为《自然和人工系统中的适应性》的著作（1975）介绍了遗传算法的基本思想。

遗传算法扩展了人类遗传学的思想，即人类DNA编码四字母的字母表（根据A、C、T、G四种核苷酸）。遗传算法的构建包括设计一个字母表，根据该字母表的字符串对决策问题的解决方案进行编码。字符串等价于个体。适应度函数（fitness function）定义哪些解决方案可以生存，哪些不能。解决方案进行组合的方法，是模仿从双亲字符串中切割与组合的交叉运算。给出一个容易改变的初始群体，进行一场能在字符串中发生突变的进化游戏。它们互相组合，生成新的一代个体。适应性最强的个体生存下来继续变异，直到生成一组成功的解决方案。

利用遗传算法产生解决方案与其他大部分搜索技术相比具有以下不同的特点：

- GA搜索在每一代中使用一个解决方案的集合，而不是单个解决方案；
- 字符串空间中的搜索，在编码解决方案空间中表现为一个更大的并行搜索；
- 搜索完成的记忆唯一地被表现为一代可用的解决方案的集合；
- 遗传算法是一个随机的算法，因为搜索机制使用了概率算子；
- 在从一代到下一代的过程中，GA通过操作编码解决方案，找到了在知识获取与知识利用之间几乎是最优的平衡。

遗传算法用于问题解答和聚类问题。遗传算法并行解决问题的能力为数据挖掘提供了有力的工具。GA的缺点包括个体解决方案的过量产生、搜索过程的随机性和对计算机处理的较高要求。在一般情况下，使用遗传算法完成有意义的工作需要强大的计算能力。

12.6　数据挖掘的应用

数据挖掘技术可以应用于大量的业务决策情况。特别是在预期回报比较高的应用领域，包括如下领域。

- 市场营销。应用包括基于购买模式的顾客行为的分析；有关广告、店址、投递目标等市场策略的确定；顾客、商店或产品的细分；价目表、商店布局、广告活动的设计。
- 金融。应用包括客户信用度的分析；应收账户的划分；诸如股票、证券、信托基金等金融投资的性能分析；金融选项的评估；欺骗行为的探测等。
- 制造。应用包括机器、人力、物料等资源的优化；制造过程、车间布局、产品设计的优化，例如，根据用户要求来设计汽车。
- 医疗保健。应用包括放射图像的模式发现；与症状和疾病相关的微阵列（基因芯片）实验数据分析；药品的副作用分析；某种治疗方法有效性的分析；医院内部流程的优化，病人健康数据与医生资格之间的联系分析等。

12.7　商业化数据挖掘工具

目前，商业化数据挖掘工具使用了几种常见的技术来获取知识。这些技术包括关联规则、聚类、神经网络、排序和统计分析，这些在前面已经讨论过。同时使用到的还有决策树，它被用来表示分类或聚类中的规则。统计分析包括回归分析和许多其他技术。还有些商业化产品使用遗传算法、案例推理、贝叶斯网络、非线性回归、组合优化、模式匹配、模糊逻辑等高级技术。本章已经讨论了其中的一些技术。

大部分数据挖掘工具都使用了开放的数据库连接（Open Database Connectivity，ODBC）接口。ODBC是与数据库相关的一个行业标准，它使得大部分流行的数据库程序可以访问数据，比如Access、dBASE、Informix、Oracle和SQL Server。其中有些软件包为专门的数据库程序提供接口，最常见的是Oracle、Access与SQL Server。大部分工具是在微软的Windows环境中工作，有少数是在UNIX操作系统上运行。所有产品的趋势是运行在微软的Windows环境中。有个名叫Data Surveyor的工具，遵循

ODMG标准，见第5章中关于ODMG面向对象标准的讨论。

一般来说，这些程序在单机上执行连续的处理。这些产品中的多数运行在客户-服务器模式中。一些产品在并行计算机体系结构中用于并行处理，并作为联机分析处理（OLAP）工具的一部分。

12.7.1　用户界面

大多数工具运行在图形用户界面（GUI）环境中。一些产品使用了成熟的可视化技术来观察数据与规则（例如，SGI的MineSet），并且可以通过这种方式交互地操纵数据。文本界面已经很少见了，并且主要在基于UNIX的工具中，比如IBM的Intelligent Miner。

12.7.2　应用编程接口

一般来说，应用编程接口（API）是一个可选的工具。大部分产品不允许使用它们的内部函数。但是有些允许应用程序员重用它们的代码。最常见的接口是C库与动态链接库（DLL）。一些工具包括专有的数据库命令语言。

表12-1中列出了11种最具代表性的数据挖掘工具。目前全球已有100多种商业化的数据挖掘产品。非美国的产品有来自荷兰的Data Surveyor和来自俄罗斯的Polyanalyst。

表12-1　一些有代表性的数据挖掘工具

公　司	产　品	技　术	平　台	接　口①
Acknosoft	Kate	决策树，案例推理	Win NT，UNIX	Microsoft Access
Angoss	Knowledge Seeker	决策树，统计学	Win NT	ODBC
Business Objects	Business Miner	神经网络，机器学习	Win NT	ODBC
CrossZ	QueryObject	统计分析的优化算法	Win NT，MVS，UNIX	ODBC
Data Distilleries	Data Surveyor	综合的，可混合不同类型的数据挖掘算法	UNIX	ODBC，ODMG-compliant
DBMiner Technology Inc.	DBMiner	OLAP分析，关联，分类，聚类算法	Win NT	Microsoft 7.0，OLAP
IBM	Intelligent Miner	分类，关联规则，预测模型	UNIX（AIX）	IBM，DB2
Megaputer Intelligence	Polyanalyst	符号知识获取，进化规划	Win NT，OS/2	ODBC，Oracle，DB2
NCR	Management Discovery Tool（MDT）	关联规则	Win NT	ODBC
SAS	Enterprise Miner	决策树，关联规则，神经网络，回归，聚类	UNIX（Solaris），Win NT，Macintosh	ODBC，Oracle，AS/400
Purple Insight	MineSet	决策树，关联规则	UNIX（Irix）	Oracle，Sybase，Informix

① ODBC：Open Data Base Connectivity（开放的数据库连接）
ODMG：Object Data Management Group（对象数据管理组织）

12.7.3　未来的方向

数据挖掘工具基于最新的科学研究思想，正在不断地发展。许多工具结合了人工智能（AI）、统计学和优化的最新算法。

目前，在客户-服务器体系结构、并行数据库和数据仓库中使用现代的数据库技术进行快速处理已经实现，例如分布式处理。未来的趋势则是更全面地转向互联网。另外，混合方法将变得普通；

使用所有可用资源进行处理将得到实现；数据的处理将同时利用并行和分布式的计算环境。这个转变非常重要，因为现代数据库包含大量的信息。不仅多媒体数据库在增长，而且图像的存储与检索都是很费时的操作。同时，二级存储设备的成本不断降低，因此大量信息的存储将变得可行，甚至对于小公司也是如此。这样，数据挖掘程序将不得不处理更多公司、更大的数据集合。

在不远的将来，微软的Windows NT和UNIX似乎将是标准平台，而NT占主导地位。大部分数据挖掘软件使用ODBC标准从商业数据库中抽取数据，专用的输入格式有望消失。实际中大量存在把非标准化数据作为数据挖掘的数据源的需求，例如图像和其他多媒体数据。但是，非标准化的数据挖掘算法还没有发展到足以商业化的成熟程度。

小结

在本章中我们纵览了数据挖掘这一重要的分支，它使用数据库技术发现数据中的额外知识或模式。我们给出了在数据库中知识发现的一个示例，知识发现的范畴比数据挖掘更宽。对于数据挖掘的各种技术中，我们重点介绍了关联规则挖掘、分类和聚类的细节。我们提出了各个领域中的算法，并通过示例说明了这些算法是如何工作的。

同时我们还简要讨论了其他多种技术，其中包括基于人工智能的神经网络和遗传算法。主动搜索技术也正在数据挖掘中得以应用，我们概要介绍了一些可能的研究方向。在未来的数据库技术产品市场中，有望产生大量的数据挖掘活动。对于近百种可用的数据挖掘工具，我们总结了其中的11种。在将来的研究中，数据挖掘工具无论从数量还是从功能上都会有显著扩展。

复习题

12.1　从数据库中进行知识发现有哪些不同阶段？描述一个从现存的事务数据库中挖掘新知识的完整应用方案。

12.2　数据挖掘的目标和任务是什么？

12.3　由数据挖掘产生的知识有哪5种类型？

12.4　作为知识的一种类型，什么是关联规则？给出支持度和可信度的定义，并用它们来定义一个关联规则。

12.5　什么是向下闭合性质？在开发查找关联规则的高效算法中，即对于查找大项集，此性质有什么帮助？

12.6　开发关联规则挖掘的FP树算法的动机是什么？

12.7　举例描述层次间的关联规则。

12.8　在图12-3所示的层次中，什么是一个负向关联规则？

12.9　从大型数据库中挖掘关联规则有什么困难？

12.10　什么是分类规则，如何把决策树与分类规则相关联？

12.11　什么是熵，如何将其用于建立决策树？

12.12　聚类和分类有哪些不同？

12.13　描述作为数据挖掘技术的神经网络和遗传算法。使用这些技术的主要困难是什么？

练习题

12.14　对下列数据集合应用Apriori算法。

Trans_id	Items_purchased
101	牛奶，面包，鸡蛋
102	牛奶，果汁

103	果汁，黄油
104	牛奶，面包，鸡蛋
105	咖啡，鸡蛋
106	咖啡
107	咖啡，果汁
108	牛奶，面包，饼干，鸡蛋
109	饼干，黄油
110	牛奶，面包

项集合为{牛奶，面包，饼干，鸡蛋，黄油，咖啡，果汁}。最小支持度为0.2。

12.15　对于练习题12.14中包含3个项的项集，给出两条可信度等于或大于0.7的规则。

12.16　对于分区算法，证明数据库中的任何频繁项集必然至少作为一个分区的局部频繁项集出现。

12.17　给出为练习题12.14中的数据所构造的FP树。

12.18　把FP-growth算法应用到练习题12.17所得到的FP树，并给出频繁项集。

12.19　在以下数据记录集合上应用分类算法。类属性为Repeat_customer。

RID	Age	City	Gender	Education	Repeat_customer
101	20…30	NY	F	college	YES
102	20…30	SF	M	graduate	YES
103	31…40	NY	F	college	YES
104	51…60	NY	F	college	NO
105	31…40	LA	M	high school	NO
106	41…50	NY	F	college	YES
107	41…50	NY	F	graduate	YES
108	20…30	LA	M	college	YES
109	20…30	NY	F	high school	NO
110	20…30	NY	F	college	YES

12.20　考虑下列二维记录集合。

RID	Dimension1	Dimension2
1	8	4
2	5	4
3	2	4
4	2	6
5	2	8
6	8	6

同时考虑两个不同的聚类模式：(1)$Cluster_1$包含记录{1,2,3}，$Cluster_2$包含记录{4,5,6}；(2)$Cluster_1$包含记录{1,6}，$Cluster_2$包含记录{2,3,4,5}。哪个模式更好，为什么？

12.21　使用*k*均值算法对练习题12.20中的数据聚类。以3作为*K*的值，并且假设RID为1，3和5的记录作为初始聚类形心（平均值）。

12.22　*k*均值算法使用的相似性度量为记录与聚类形心之间的距离。如果记录的属性不是定量的，但是在性质上可分类，例如值为{低，中，高}的Income_level，或值为{Yes,No}的Married，或值为{Alabama, Alaska,…,Wyoming}的State_of_residence，那么距离度量就没有意义了。定义一个更适合的相似性度量，从而可以用于包含分类数据的聚类数据记录。

参考文献

数据挖掘的文献来自多个领域，包括统计学、数学优化、机器学习和人工智能。数据挖掘是最近才成为数据库文献中的一个主题。因此我们只提到了与数据库相关的一些著作。（Chen et al.，1996）很好地总结了数据库在数据挖掘方面的观点。（Han & Kamber，2001）是一本非常好的著作，详细描述了数据挖掘领域中使用的不同的算法和技术。IBM Almaden研究中心的工作产生了大量的早期概念、算法以及一些性能研究成果。（Agrawal et al.，1993）首先进行了关于关联规则的主要研究。（Agrawal & Srikant，1994）的关于市场购物篮数据的Apriori算法被（Savasere et al.，1995）使用分区进行了改进。（Toivonen，1996）提出把抽样作为减少处理工作的一种方法。（Cheung et al.，1996）把分区扩展到分布式环境。（Lin & Dunham，1998）提出利用数据偏离求解问题的技术。（Agrawal et al.，1993b）讨论了关联规则的性能。（Mannila et al.，1994），（Park et al.，1995）和（Amir et al.，1997）提出了与关联规则相关的其他有效算法。（Han et al.，2000）提出了本章讨论的FP树算法。（Srikant，1995）提出了挖掘的普遍规则。（Savasere et al.，1998）描述了数据挖掘负关联的观点。（Agrawal et al.，1996）描述了IBM的Quest系统。（Sarawagi et al.，1998）描述了如何实现关联规则与关系数据库管理系统的集成。（Piatesky-Shapiro & Frawley，1992）发表了涉及知识发现领域广泛主题的论文。（Zhang et al.，1996）对于聚集大型数据库的问题提出了BIRCH算法。有关本章中涉及的决策树学习和分类的算法的信息可以在（Mitchell，1997）中找到。

（Adriaans & Zantinge，1996），（Fayyad et al.，1997），（Weiss & Indurkhya，1998）致力于研究数据挖掘的不同方面，以及它在预测中的用途。遗传算法的思想来自于（Holland，1975）。关于遗传算法的一个很好的综述可以在（Srinivas & Patnaik，1974）中看到。神经网络有很多文献，一个全面的介绍在（Lippman，1987）中可以得到。

在最近的一本书（Tan et al.，2006）中全面介绍了数据挖掘并详细列出了参考文献。建议读者也可以查阅每年一次的两个著名的数据挖掘会议的论文集：知识发现和数据挖掘会议（KDD），KDD始于1995年。SIAM数据挖掘国际会议（SDM）始于2001年。访问dblp.uni-trier.de可以查阅往届会议资料。

第13章

数据仓库和OLAP概述

数据处理能力的不断提高、数据分析工具和技术的不断成熟，促进了数据仓库（data warehouse）的迅速发展。数据仓库所提供的存储能力、功能以及对查询的响应速度都超过了面向事务的数据库。随着这些能力的不断增长，产生了提高数据库数据访问性能的强烈需求。纵观全书可以看到，传统的数据库试图在数据访问需求和数据完整性需求之间求得平衡。在现代组织中，使用数据的用户常常和数据源完全分离。很多人只需要读取数据，但要求能够非常快速地访问大量数据，这里所说的大量数据要比能够方便地下载到桌面的数据量要大很多，而且这些数据常常来自多数据库。由于许多分析是重复的、可预测的，软件商和系统支持公司已经开始设计能支持这些功能要求的系统。目前，在为来自中层管理的决策者们提供正确的细节层次信息以支持他们的决策制定方面，存在大量需求。数据仓库（data warehousing）技术、联机分析处理（OLAP）技术和数据挖掘技术提供了这些功能。我们已经在第12章介绍了数据挖掘技术。本章将对数据仓库与OLAP技术进行概要介绍。

13.1 概述、定义和术语

在《数据库系统基础：初级篇》的第1章中，我们把数据库定义为相关数据的集合，把数据库与数据库软件一起定义为数据库系统。数据仓库也是一个信息集合，同时也是一个支持系统。然而，两者之间存在着一个比较明显的区别：传统的数据库是事务数据库（关系数据库、面向对象数据库、网状数据库或者层次数据库）；而数据仓库的突出特征是主要用于决策支持应用。数据仓库的优化是为了数据检索，而不是为了常规事务处理。

由于数据仓库是众多公司为了满足各自特定需求而开发的，因此还没有一个单一的、规范的定义。专业杂志的文章和著名出版社出版的图书都用各种不同的方式阐述数据仓库的含义。各个厂商利用这个术语的流行性来销售各种相关产品，咨询顾问们也提供大量服务，这些都是在数据仓库的旗帜下进行的。然而在结构、功能、性能和目标上，数据仓库和传统数据库都截然不同。

W.H.Inmon[①]把**数据仓库**描述为面向主题的、集成的、非易变的、随时间变化的数据集合，用以支持管理中的决策。数据仓库为复杂分析、知识发现和决策制定提供数据访问。数据仓库支持在一个组织的数据和信息上进行高性能的查询，支持OLAP、DSS（决策支持系统）和数据挖掘等类型的应用。下面对这些应用逐一定义。

OLAP这一术语用来描述数据仓库中复杂数据的分析。在熟练的知识工作者的操作下，OLAP工具使用分布式计算能力进行分析，比起经济地、高效地置于个人台式机上进行分析，要求更大的存储能力和更强的处理能力。

DSS也称为**执行信息系统**（executive information systems，EIS）（不要与企业集成系统相混淆），用来支持一个公司的主要决策制定者利用更高级的数据进行复杂和重要的决策。数据挖掘（在第12章详细讨论过）用来进行知识发现，是为发现没有预料的新知识而搜索数据的一种过程。

---

① （Inmon，1992）首次使用术语“数据仓库”。

传统的数据库支持**联机事务处理**（online transaction processing，OLTP），包括插入、更新、删除操作，同时也支持信息查询。传统的关系数据库在处理只涉及数据库中一小部分数据的查询，以及处理对某个关系的少量元组的插入或者更新事务时，是优化的。然而，它们对于OLAP、DSS或者数据挖掘来说不是优化的。相反，数据仓库是以支持高效抽取、处理、展示分析和制定决策为目的而专门设计的。与传统数据库相比较，数据仓库通常包含来自多个数据源的海量数据，这些数据源包括不同数据模型的数据库，有时还包括来自独立系统和平台的文件。

13.2　数据仓库的特性

为了讨论数据仓库以及它们与事务数据库的区别，需要使用一个合适的数据模型。多维数据模型（13.3节将详细讨论）对于OLAP和决策支持技术就是非常合适的。数据仓库与多数据库相比，后者提供对分离的、并且通常是异构的数据库的访问，前者常常是来自多种数据源的集成数据，以多维模型方式存储。与大部分事务数据库不同，数据仓库典型地支持时间序列和趋势分析，这两者都需要比一般事务数据库提供更多的历史数据。

与事务数据库相比，数据仓库是非易失的。这是指数据仓库中的信息很少变化，可以认为是阶段性更新的非实时数据。在事务处理系统中，事务是数据库变化的单元和代理；相反地，数据仓库中的信息粒度要粗糙得多，它根据精心选择的更新原则进行更新，通常是增量更新。数据仓库的更新由数据仓库采集组件（acquisition component）来处理，它提供所有必需的预处理功能。

我们也可以把数据仓库更一般地描述为一个决策支持技术的集合，目标是使得知识工作者（高层主管（executive）、经理、分析员）做出更好、更快的决策[①]。图13-1给出了数据仓库的一般概念结构。它描述了整个数据仓库的处理过程，包括数据进入数据仓库之前可能的清洗和重新格式化，这一处理通常由行业中公认的抽取、转换和装载（extract，transformation和loading，ETL）工具来执行。在这个过程的后端，OLAP、数据挖掘和DSS可以生成新的关联信息，例如规则。在图中表示的这些信息又返回到数据仓库中。图中还显示了数据源可能包含文件。

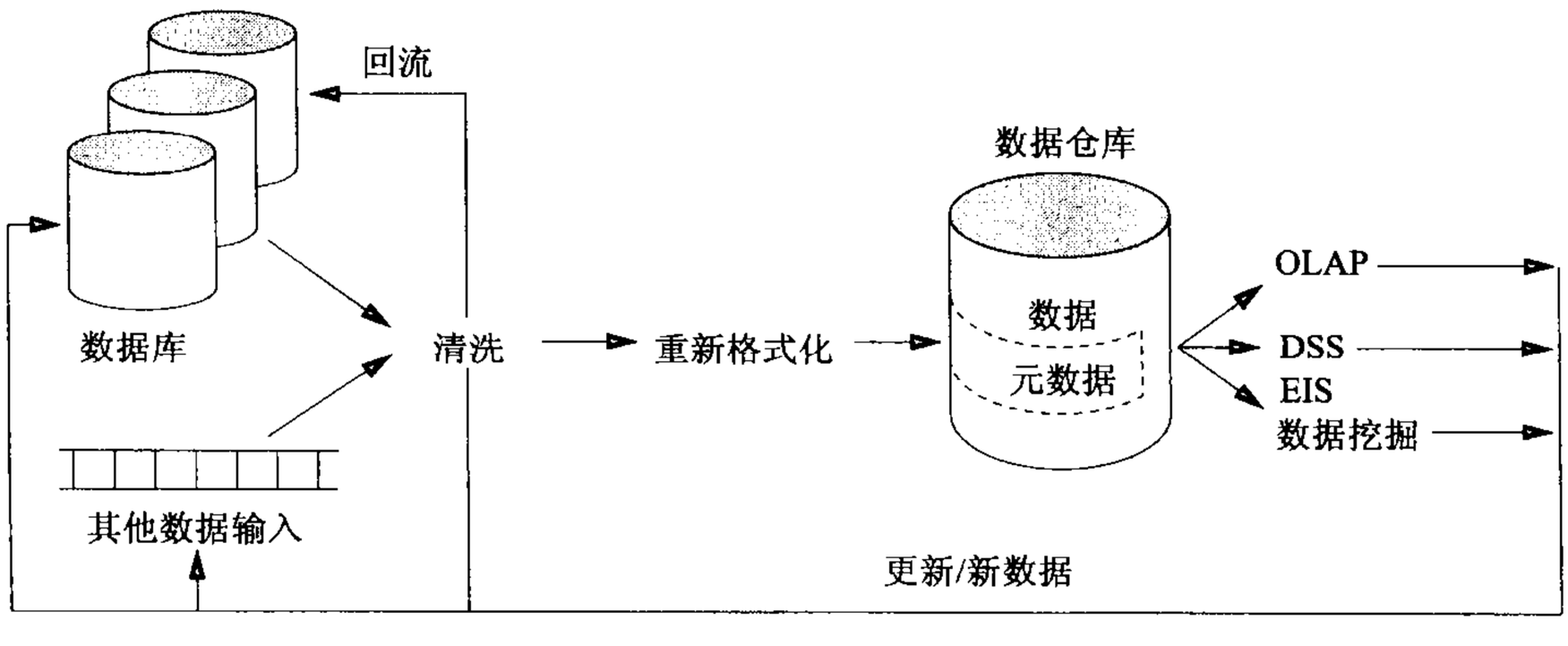


图13-1　数据仓库的处理过程

数据仓库有以下几个主要特性[②]：

- 多维概念视图。
- 总维数。
- 无限维和聚合层。

① （Chaudhuri & Dayal，1997）提供了一个有关这个主题的优秀教程，并用这个定义作为教程的开始。

② （Codd，1993）创造了术语OLAP，并且描述了这些特点。我们在这里引用了Codd的原作中的列表。

- 无限制跨维操作。
- 动态稀疏矩阵处理。
- 客户/服务器体系结构。
- 多用户支持。
- 可访问性。
- 透明性。
- 直观的数据操纵。
- 一致的报表性能。
- 灵活的报表。

由于数据仓库包含大量的数据，通常来说，数据仓库比源数据库要大一个数量级（有时大两个数量级）。因此，在企业范围的数据仓库、虚拟数据仓库、数据集市中，如何处理海量数据（可能是TB级）是个急需解决的问题。

- **企业范围的数据仓库**（Enterprise-wide data warehouses）是一项要求投入大量时间和资源的巨大工程。
- **虚拟数据仓库**（Virtual data warehouses）提供操作数据库的视图，它们被物化，以便对它们进行有效的访问。
- **数据集市**（Data marts）通常针对组织中的子集，例如一个部门，并且数据集中得更加紧密。

13.3　数据仓库的数据模型

多维模型利用了数据间内在的联系，把数据移入被称为数据立方体（data cube）（如果大于三维可以称为超立方体）的多维矩阵中。对于遵循维格式的数据，在多维矩阵上的查询性能比在关系数据模型上的查询性能要好得多。例如，一个公司的数据仓库可以包含3个示例维：财政周期维、产品维、地区维。

一个标准的电子数据表格是一个二维矩阵。例如一张关于特定时期各种产品的地区销售情况的电子表格。在表中，产品可以作为行来表示，各个地区的销售额组成列（图13-2展示了这个二维表）。加上时间维，例如公司的财政季度，从而产生三维矩阵，这样就可以用一个数据立方体来表示。

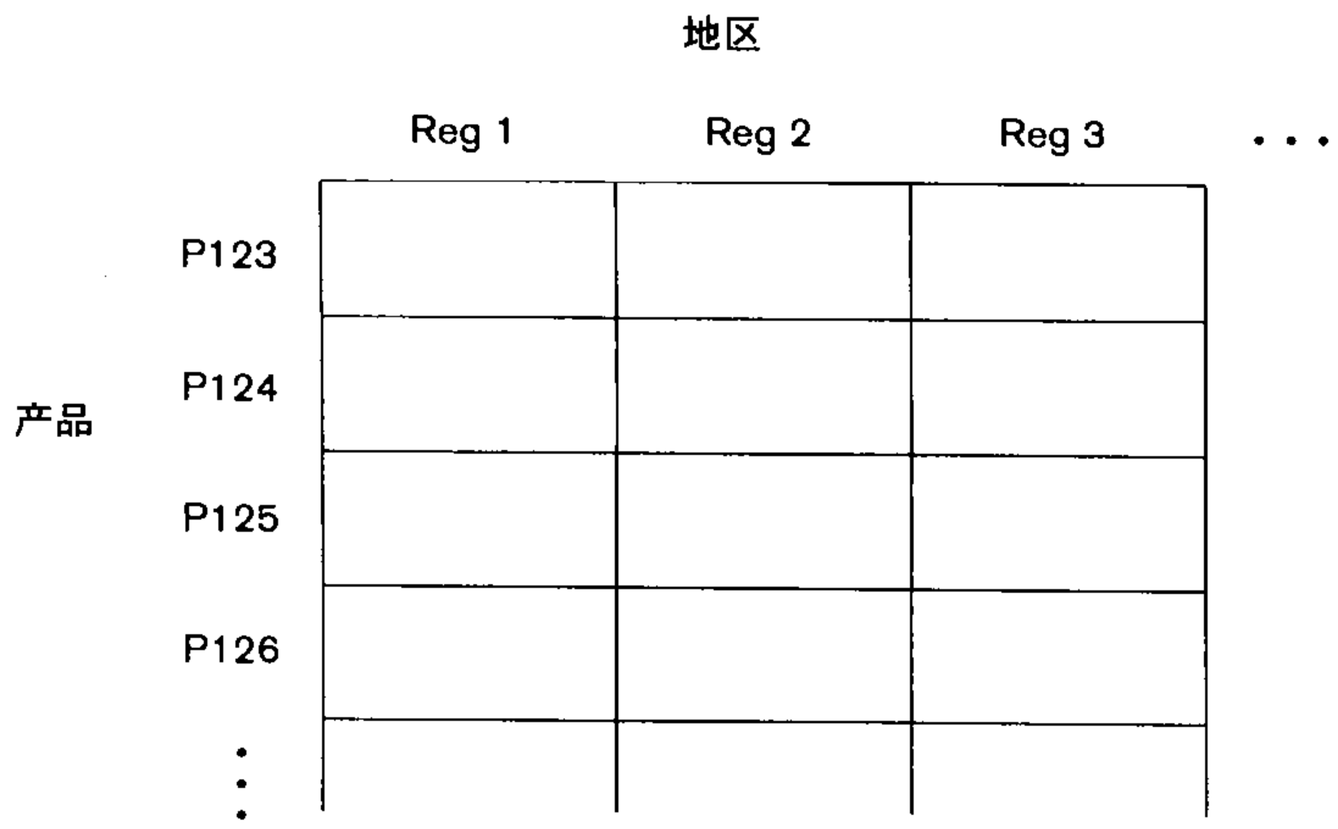


图13-2　二维矩阵模型

图13-3是一个通过财政季度和销售地区将产品销售额组织起来的三维数据立方体。每个单元格包含一个对应于特定产品、特定财政季度、特定地区的数据。通过增加其他的维，可以得到一个数

据超立方体，尽管超过三维将不是直观的，也不容易用图形方式表示。但数据可以从维的任意组合中直接查询，避免了复杂的数据库查询。现在已经有这样的工具，它可以根据用户选择的维来观察数据。

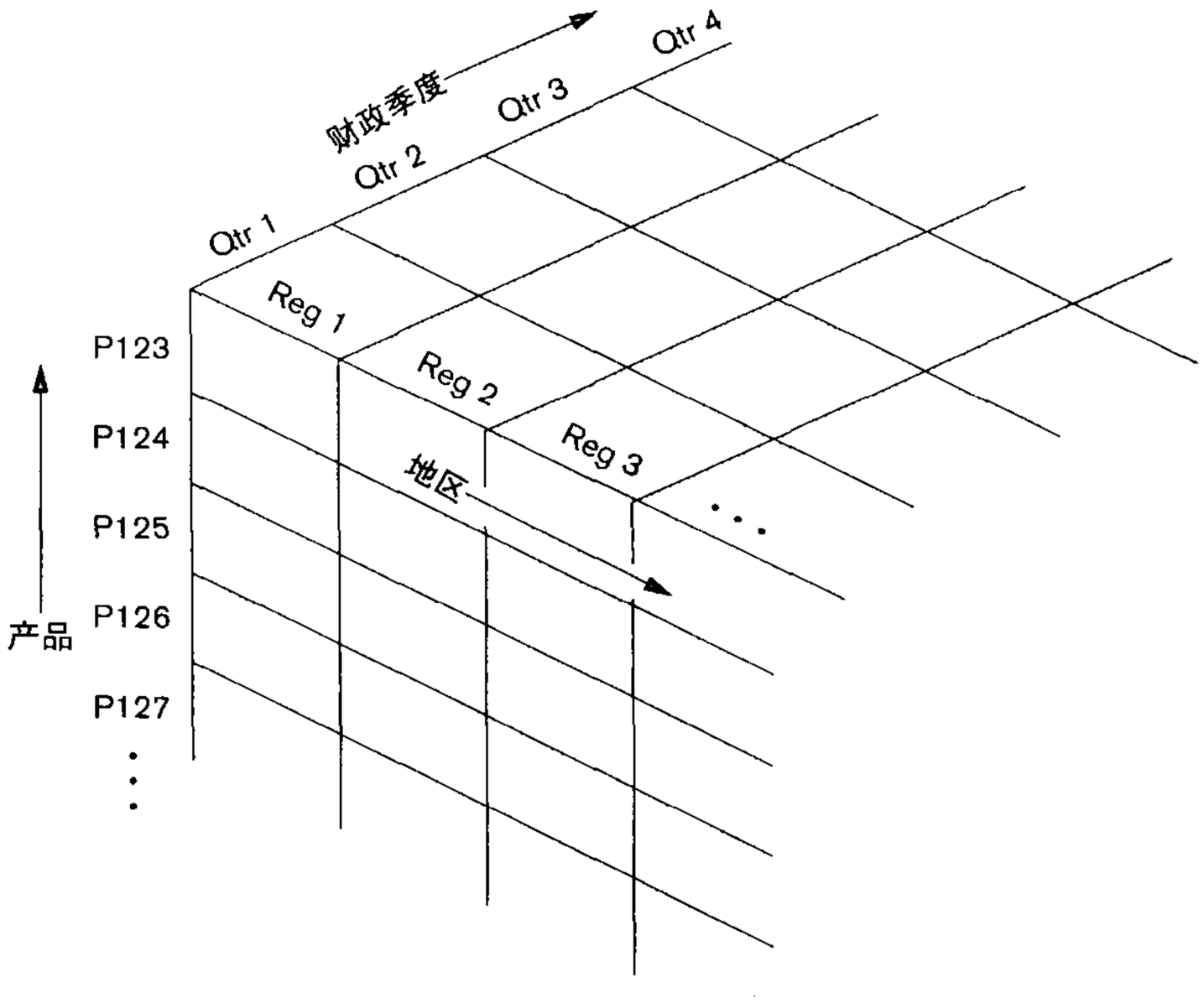


图13-3　三维数据立方体模型

在数据立方体中，使用称为**旋转**（pivoting或 rotation）的技术可以容易地实现从一个维层次（方位）转向另一个维层次。这种技术将数据立方体看成是通过旋转从轴的不同方向展示数据。例如，你可以旋转一个数据立方体使得地区销售额作为行，财政季度的销售额作为列，公司产品作为第3维（如图13-4所示）。因此，这种技术等价于每个产品分别对应一张地区销售表，每张表都是按地区展示该产品的季度销售额。

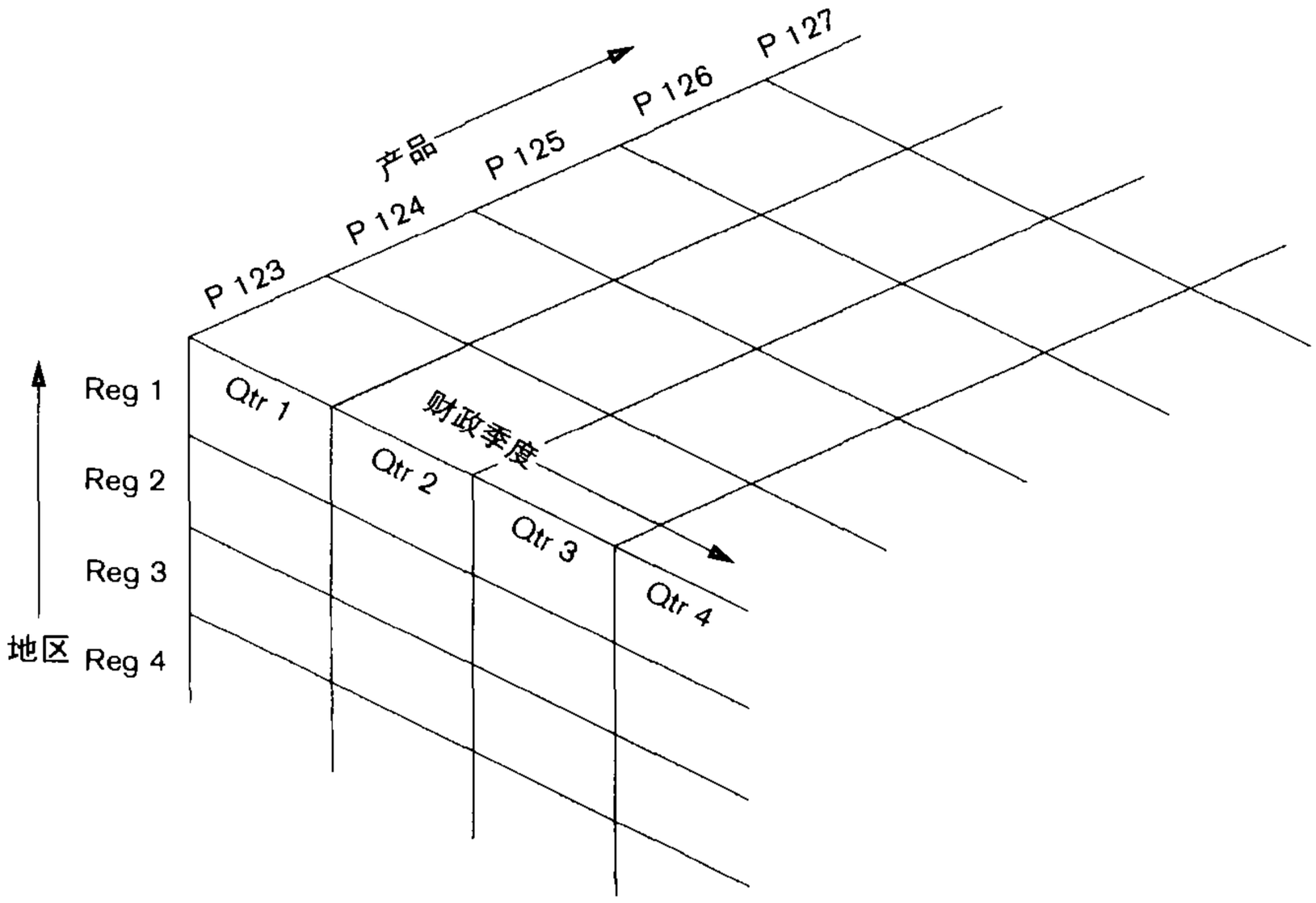


图13-4　图13-3中数据立方体的旋转版本

多维模型在向上归纳和向下钻取中很容易实现层次视图。向上归纳（roll-up）上升了一个层次，在某个维上组成较大的单元（例如，以季度或年份来累加每周的数据）。图13-5描述了一个向上归纳的例子，它把单一的产品组合成更大的粒度，即产品类。在图13-6中，向下钻取（drill-down）展示了一个相反的例子，提供一个更小粒度的视图，可以将一个国家的销售额分解为地区的、进而分解为子地区的，也可以把产品按样式进行分解。

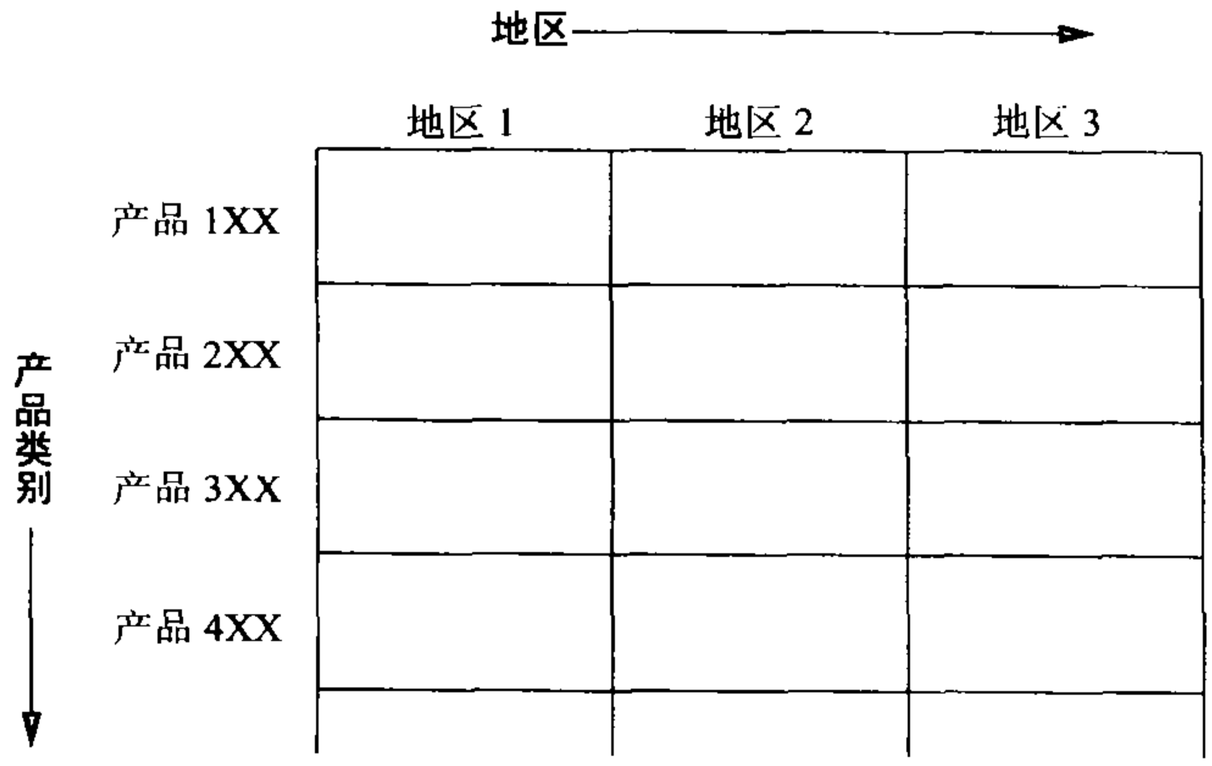


图13-5　向上归纳操作

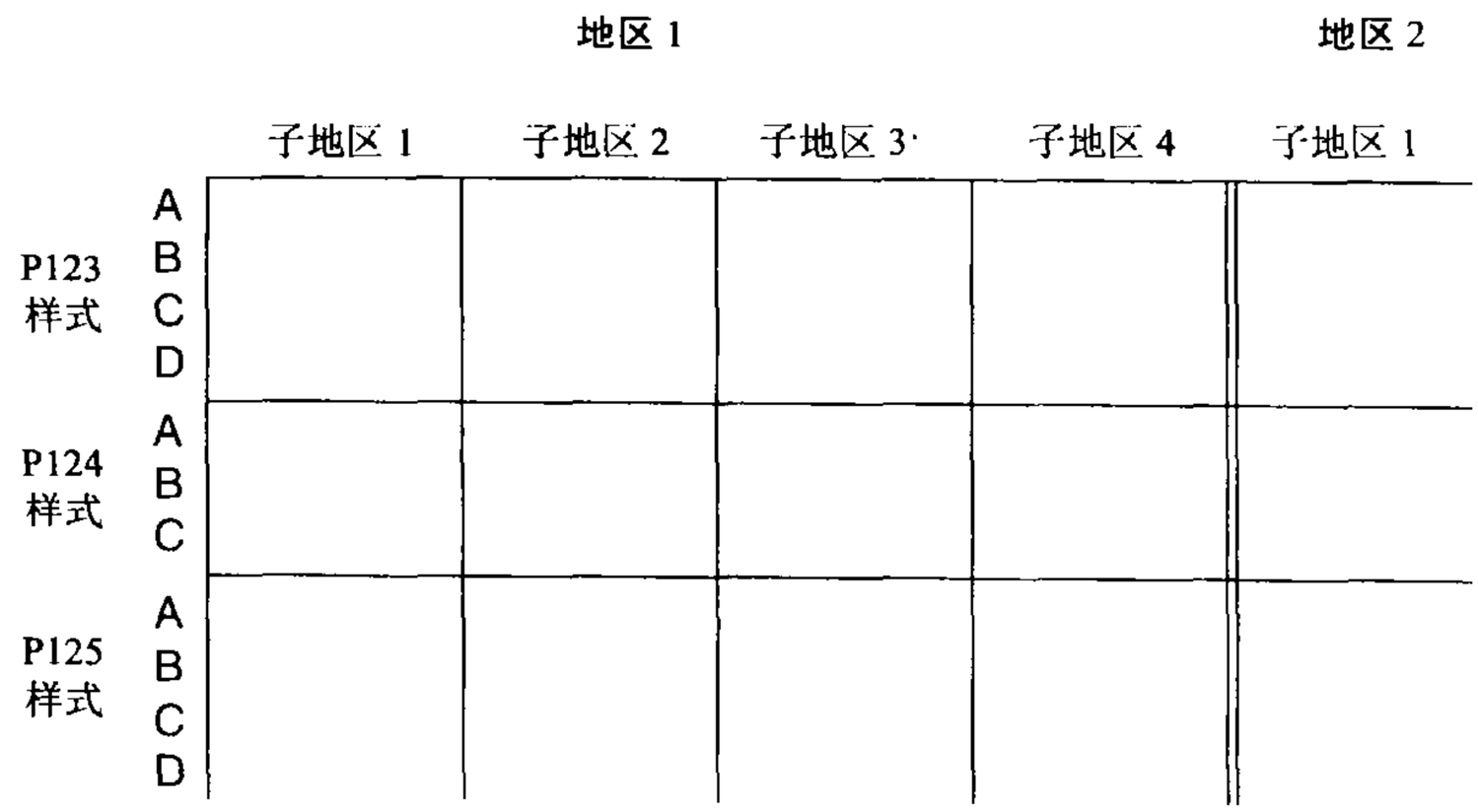


图13-6　向下钻取操作

多维存储模型涉及两类表：维表和事实表。**维表**由维的属性元组组成。**事实表**可以看成这样的元组集合，其中一个元组对应一个已记录的事实。这个事实包含一些可以度量的或者可以观察的变量，并且通过指向维表的指针来确定它（它们）。事实表中包含数据，而维识别那个数据中的每个元组。图13-7包含从多维表的角度来观察的事实表的一个示例。

两个常见的多维模式是星型模式和雪花模式。**星型模式**（star schema）由一个事实表和每个维对应的维表组成（参见图13-7）。**雪花模式**（snowflake schema）是星型模式的变形，它将星型模式中的维表通过规范化组织成层次结构（参见图13-8）。某些设置使得数据仓库被规范化到第三范式，以便可以访问数据仓库中最小的细节层次。**事实星座**（fact constellation）是共享某些维表的事实表的集合。图13-9展示了两个事实表组成的事实星座，即交易结果和交易预测组成的事实星座。它们共享的维表是产品表。事实星座限制了对数据仓库的可能的查询。

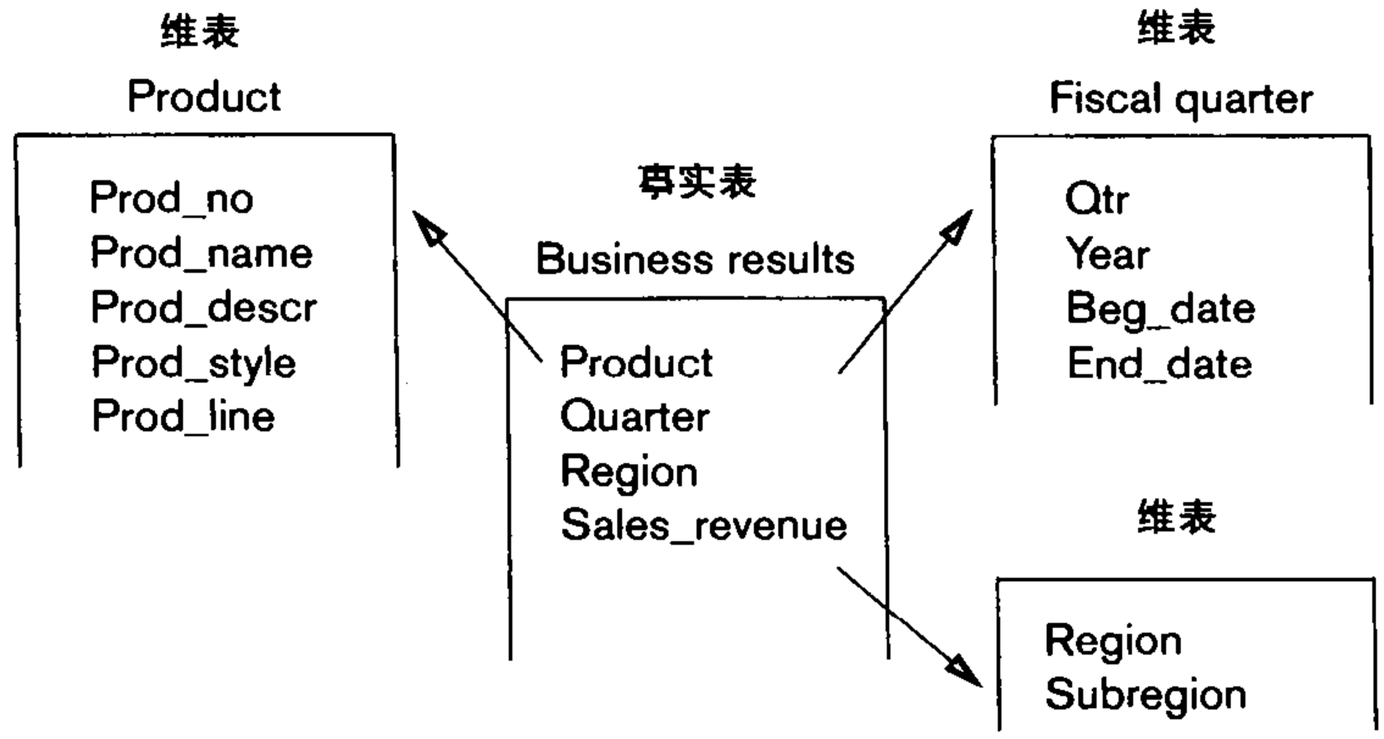


图13-7　带有事实表与维表的星型模式

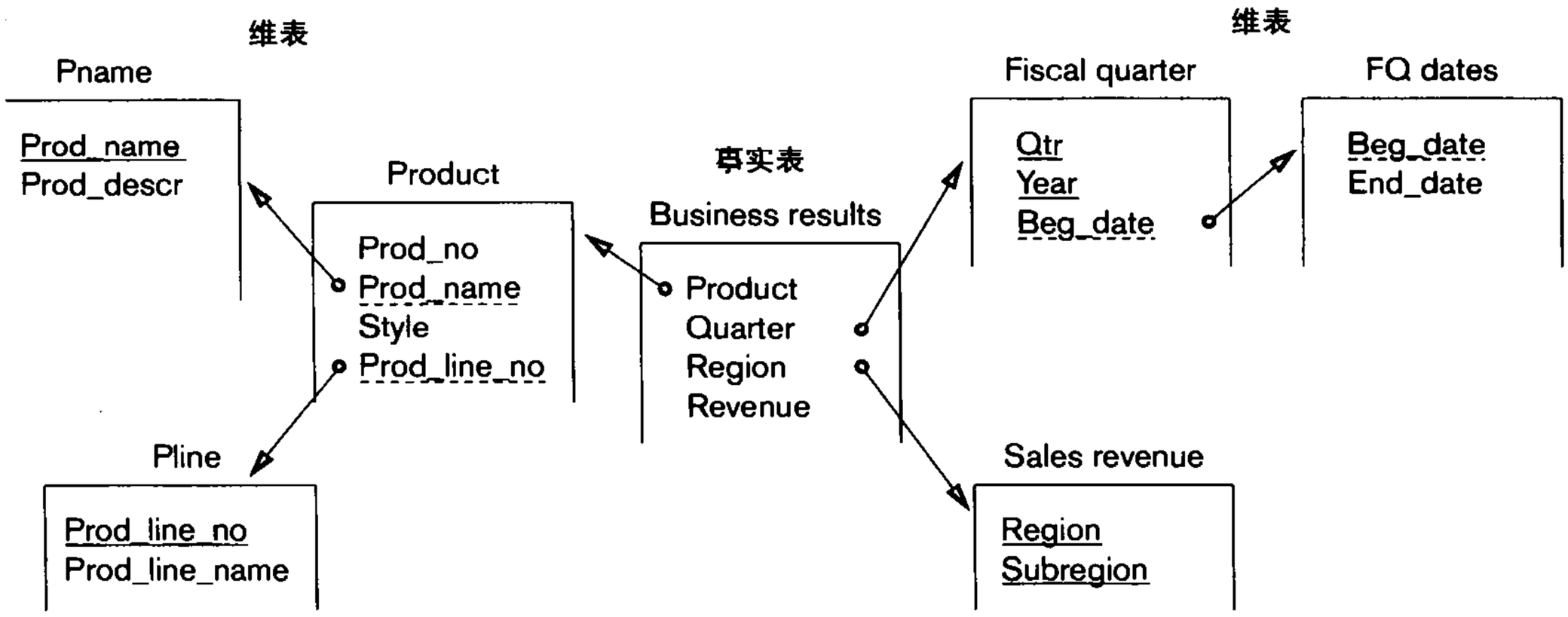


图13-8　雪花模式

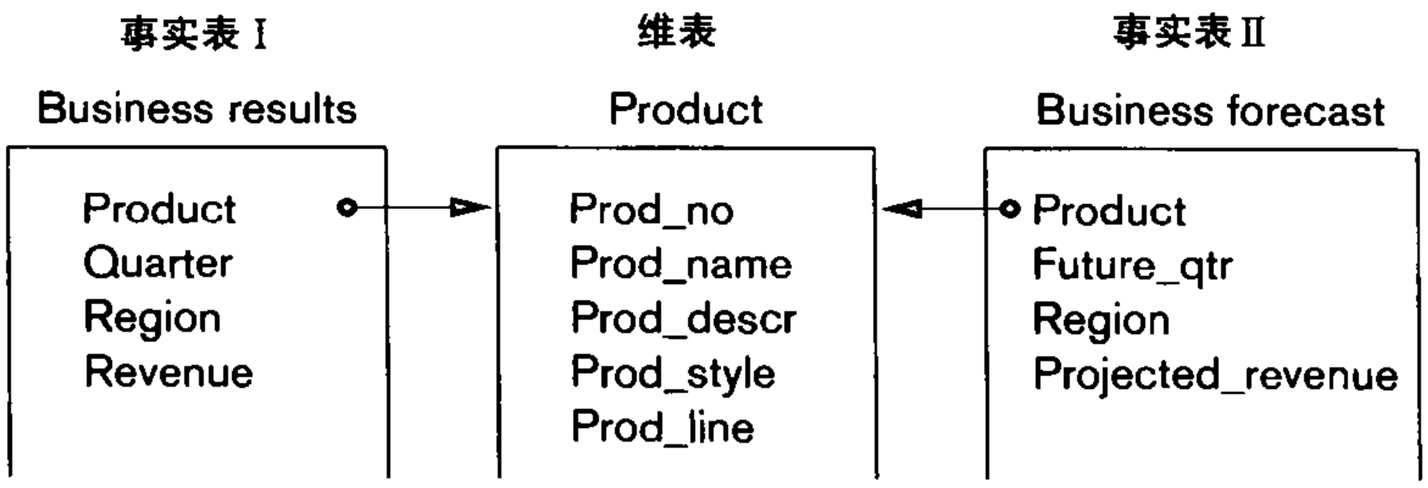


图13-9　事实星座

数据仓库存储也利用了索引技术来支持更高性能的访问（见《数据库系统基础：初级篇》中第14章对索引的讨论）。一种称为**位图索引**（bitmap indexing）的技术为索引域（列）中的每个值构造一个位向量。对于小基数（low-cardinality）的域它可以工作得很好。如果第*j*行包含被索引的值，则向量的第*j*位被置为1。例如，考虑一个包含100 000辆汽车的库存目录，在汽车型号上建立位图索引。如果有4种汽车型号：经济型、小型、中型、大型，就会有4个位向量，每个位向量包含100 000位（12.5K），整个索引的大小为50K。对于小基数的域，位图索引可以提供相当大的I/O和存储空间，这是它的优势。利用位向量，位图索引可以显著提高比较、聚集和连接方面的性能。

在星型模式中，维数据可以对事实表中的元组建立**连接索引**。连接索引是为了维护外码与主码之间的联系而使用的一种传统索引。它们关联了星型模式中维的值和事实表中的行。例如，考虑一个销售事实表，它以城市和财政季度作为维。如果在城市上有一个连接索引，则对于每个城市的连接索引都维护一个涉及该城市元组的元组ID。连接索引可以涉及多个维。

通过进一步利用数据仓库的非易变性，以及使用数据仓库进行分析具有一定程度的可预见性，数据仓库的存储可以对汇总数据的访问变得更加方便。目前使用两种方法：(1)包含汇总数据的较小的表，诸如生产线的季度销售额或销售收入；(2)在已经存在的表中将层次（例如星期、季度、年度）进行编码。通过比较，在易变的、面向事务的数据库中创建并维护这样的聚集计算的开销可能会相当大。

13.4　构建数据仓库

构建数据仓库时，创建者应当充分考虑数据仓库预期的用途。尽管在设计阶段没有办法预料所有可能的查询或分析，可是，设计应该明确地支持**特定查询**（ad-hoc querying），也就是说，对维表或事实表中属性的值的任何有意义的组合进行访问。例如，一个销售密集型的消费品公司和一个只关注基金增长的非盈利慈善机构相比，前者更需要以不同的方式来组织数据仓库。因此，应该选择一个能反映预期用途的合适模式。

为数据仓库获取数据，包括以下步骤：

1．数据必须从多个异构的数据源中抽取。例如数据库或者其他诸如包括金融市场信息或环境信息的各种数据源。

2．数据必须要经过格式化，以便与数据仓库内的数据相一致。使得来自不相关数据源的数据，在名称、含义和域上必须是相容的。例如，一个大企业的各个子公司可能有不同的财政日历，导致季度结束的日期各不相同，使得按照财政季度聚集财政数据变得很困难；不同类型的信用卡可能以不同方式上报事务，导致在计算所有的信用消费情况时变得相当困难。这些不一致的格式必须得到妥善的解决。

3．数据必须要经过清洗，以便保证数据的有效性。数据清洗是一个非常复杂的过程，被认为是构建数据仓库过程中最费力的一部分。对于输入数据，清洗必须在装载到数据仓库之前完成。关于清洗数据没有专门用于数据仓库的方法，现有的方法也可以应用到宿主数据库。但是，由于输入数据必须经过一致性检查并进行格式化，所以数据仓库的创建者应该利用这个机会检查数据的有效性和质量。显然，要求自动识别错误的数据和不完整的数据是困难的，因此，要求自动更正数据清洗中的错误可能更加棘手。在某些方面，例如对域的检查，是可以容易地编码到清洗数据的例行程序中，可是自动识别其他数据问题是很困难的（例如，查询City＝‘San Francisco’且State＝‘CT’时，要识别出这是一个错误的组合）。在对这些问题引起注意之后，对来自不同数据源的相似数据必须进行调整后才能装载到数据仓库中。当公司的数据经理发现他们的数据为了输入到数据仓库而被清洗了，他可能希望利用清洗后的数据来更新他们原有数据。把清洗后的数据写回到数据源的过程称为**回流**（backflushing）（见图13-1）。

4．数据必须要适合数据仓库的数据模型。对于不同来源的数据，必须以数据仓库数据模型的形式被装载。数据可能要从关系数据库、面向对象数据库或者其他遗留数据库（网状的或层次的）转换成多维模型。

5．数据必须要装载到数据仓库。数据仓库的海量数据使得装载数据成为一项可观的任务。需要一些装载的监控工具，以及从不完全或不正确的装载过程执行恢复的方法。对于数据仓库中的大量数据，增量更新方法通常是唯一可行的方法，而刷新原则可能是作为回答以下问题的一个折衷方案。

(1) 数据的即时性要求怎样？

(2) 数据仓库可以脱机运行吗？可以脱机运行多久？

(3) 数据的相互依赖性如何？

(4) 存储的可用性如何？

(5) 分布需求（例如复制和分区）如何？

(6) 装载时间（包括清洗、格式化、复制、传输，以及诸如重建索引等的开销）需要多久？

正如上述所言，数据库必须在事务处理的效率和支持查询要求（用户的特定查询）之间做出平衡，而数据仓库主要是对决策者需要的访问做出优化。数据仓库中的数据存储反映了这些特殊性，它包括以下处理：

- 按照数据仓库的数据模型存储数据。
- 创建并维护要求的数据结构。
- 创建并维护适合的存取路径。
- 提供时变数据作为新数据加入。
- 支持数据仓库中数据的更新。
- 刷新数据。
- 清理数据。

尽管为创建数据仓库的初始化工作已经投入了足够的时间，可是由于数据仓库中的海量数据，通常会导致以后不可能简单地将全部数据重新装载。可供选择的方法包括：有选择地（部分地）刷新数据，以及将数据仓库版本分别存放（这需要有双倍的数据仓库存储能力！）。当对数据仓库使用增量数据刷新机制时，数据可能需要周期性地清理。例如，一个要维护过去的12个商业季度交易数据的数据仓库，可以每年进行一次周期性的数据清理。

数据仓库的设计必须充分考虑它们驻留的环境。需要重点考虑的设计事项包括：

- 应用计划。
- 合适的数据模型。
- 可用数据源的特性。
- 元数据组件的设计。
- 模块化组件的设计。
- 易于管理和可变化的设计。
- 分布式体系结构和并行体系结构的考虑。

下面依次讨论这些内容。数据仓库设计最初是由应用计划驱动的。也就是说，预期谁将会使用数据仓库，以及他们将如何来使用它。支持这些应用计划的数据模型的选择是最初决定的关键。应用计划和数据仓库数据源的特征这两方面都需要考虑。模块化设计是允许数据仓库随着组织的信息环境演变而发展的可行且必要的条件。另外，一个好的数据仓库必须被设计为可维护的，使数据仓库管理员能够在提供给用户最佳支持的同时，有效地计划和管理数据仓库的变化。

回顾《数据库系统基础：初级篇》的第2章中有关元数据的概念。元数据定义为对数据库的描述，包括它的模式定义。**元数据资料库**（metadata repository）是数据仓库的一个关键部分。元数据资料库包括技术元数据和业务元数据。首先，技术元数据涵盖了请求处理、存储结构、数据描述、数据仓库操作与维护，以及访问支持功能等的细节。其次，业务元数据包括相关的业务规则和支持数据仓库的组织的细节。

组织的分布式计算环境的体系结构是设计数据仓库时的一个主要的决定性特征。

基本的分布式体系结构有两种：分布式数据仓库和联邦式数据仓库。对于**分布式数据仓库**（distributed warehouse），分布式数据库的所有问题是与分布式数据仓库相关的问题，例如，复制、分区、通信和一致性的有关问题。分布式体系结构可以在数据仓库的性能方面提供非常重要的益处，例如改善装载的平衡度、性能的可伸缩性以及更高的可用性。每个分布式站点上都驻留着一个元数

据资料库的独立副本。**联邦式数据仓库**（federated warehouse）的思想类似于联邦式数据库：是分散的自治数据仓库的联邦，每个都有它自己的元数据资料库。考虑数据仓库固有的巨大挑战性，这样的联邦很可能由较小规模的成分组成，例如数据集市。大型组织可以选择联邦式数据集市而不是建立巨大的数据仓库。

13.5　数据仓库的典型功能

数据仓库的存在是为了利于进行复杂的、数据密集的和频繁的特定查询。因此，数据仓库必须提供比传统数据库更强大、更有效的查询支持。数据仓库的存取组件支持增强的电子表格功能、有效的查询处理、结构化查询、特定查询、数据挖掘和物化视图。尤其是增强的电子表格功能，既支持目前的电子表格应用（例如微软的Excel），又支持OLAP应用程序。它提供如下的预编程（preprogramming）功能：

- 向上归纳。数据以渐增概括方式汇总（例如，周到季度，再到年度）。
- 向下钻取。展现渐增的细节层次（向上钻取的逆过程）。
- 旋转。执行表格的行列交换。
- 切片和切块。在维上执行投影操作。
- 排序。数据按序值有序排列。
- 选择。按值或范围选定数据的可用性。
- 导出（计算）属性。属性的值是通过对已经存储或已经导出的值的进行计算而获得的。

由于数据仓库不受事务环境的限制，在查询处理时能增进效率。使用的工具和技术有：查询转换、索引的交与并、专门的ROLAP（关系OLAP）和MOLAP（多维OLAP）功能、SQL扩展、高级连接方法和智能扫描（就像在结构化分段多级查询中那样）。

通过并行处理还可以改善性能。并行服务器体系结构包括对称多处理器（SMP）、聚类（cluster）、大规模并行处理（MPP）以及它们的组合。

知识工作者和决策者使用了很多工具，从参数查询到特定查询，再到数据挖掘。为此，数据仓库的存取组件必须支持结构化查询（包括参数的和特定的），从而共同构成一个可管理的查询环境。数据挖掘本身所使用的技术来自统计分析和人工智能。统计分析可以通过先进的电子表格、成熟的统计分析软件、或者用户自己编写的程序来实现。例如滞后（lagging）、移动平均（moving average）、回归分析（regression analysis）等技术都是通常被采用的。人工智能技术，包括遗传算法和神经网络，被用来进行分类，并且用于从数据仓库中发现知识，这些知识可能未能预料到，或者难以在查询中指定（第12章详细讨论了数据挖掘）。

13.6　数据仓库和视图

有些人认为数据仓库是对数据库视图的扩展。前面提到的物化视图是满足改善数据存取需求的一种方法（见《数据库系统基础：初级篇》中第8章关于视图的讨论），物化视图的性能增强问题已经进行了研究。然而，视图只提供了数据仓库功能和性能的一个子集。视图与数据仓库有着相似之处：它们都是从数据库中进行只读抽取，并且都是面向主题的。但是，数据仓库在以下一些方面与视图是不同的：

- 数据仓库是持久存储的，而不像视图在查询时才物化。
- 数据仓库通常是多维的，而不是关系的。关系数据库的视图是关系的。
- 数据仓库可以通过索引来优化性能。视图不能独立于底层的数据库来进行索引。
- 数据仓库的特征是可以提供特定的功能支持，而视图则不能。

- 数据仓库提供大量的集成数据，并且这些数据通常是时态数据，并且通常不是一个数据库所能容纳的；然而，视图只是对一个数据库中数据的抽取。

13.7　数据仓库中的难题和开放问题

13.7.1　数据仓库实现中的困难

数据仓库产生了一些重要的操作问题：构建、管理和质量控制。从项目管理，到数据仓库的设计、构建和实现，是一个重要且具挑战性的考虑事项，不应该被低估。在大型企业中建立一个企业范围的数据仓库是一项重大的工程，从概念化到实现可能需要数年时间。由于这些困难的存在，使得实现这项工程需要大量的时间。广泛的开发和配置数据集市提供了一个诱人的选择，尤其是对那些迫切需要OLAP、DSS和数据挖掘支持的组织。

数据仓库的管理是一项精深的工作，与数据仓库的规模和复杂度成正比。试图管理数据仓库的组织，必须对管理的复杂性有实际的了解。尽管设计为只读访问，但数据仓库像它的任何信息源一样都不是静态结构，因为可以预知源数据库是发展变化的。所以数据仓库的模式和数据的采集组件必须不断更新以便处理这些变化。

数据仓库中的另一个重要问题是数据质量的控制。数据的质量和一致性都是主要关注的事项。尽管数据在获取时已经通过清洗，但对于数据库管理员来说，数据的质量和一致性仍然是数据库管理中的重要问题。从异构的和完全不同的数据源中合并数据是一项非常具有挑战性的问题，包括：不同的命名、不同的域定义、不同的标识号以及类似的其他问题。源数据库每变化一次，数据仓库管理员就必须考虑它和数据仓库中其他元素可能产生的互相作用。

在构建数据仓库之前应保守地估算应用计划，并应不断地修正以反映当前需求。由于使用模式会随着时间的变化而不断地清晰起来，因此可以对存储方式和存取路径进行调整和持续优化，从而为组织使用数据仓库提供支持。这些活动应该在数据仓库的生命周期内延续，以便持续满足用户的需求。数据仓库还应该设计成为能够适应数据源的增加和减少，而无需进行大的重新设计。数据源和源数据将会演变，数据仓库必须适应这些变化。如何使可用的源数据适合数据仓库的数据模型，是一个持续存在的问题，也是一项既具有科学性又具有技巧性的任务。因为数据仓库技术在持续地、快速地变化，数据仓库的需求和性能也将会随着时间而发生显著的变化。另外，数据仓库技术本身经过若干时间也将会继续发展，因此组件的结构和功能也将不断地更新。这些必然的变化是使组件进行完全模块化设计的极好动力。

数据仓库的管理比传统数据库的管理需要更加广泛的技能。可能需要一个拥有交叉领域专长且具有高超技能的技术专家组，而不仅仅是一个人。如同数据库管理那样，数据仓库管理也只有部分是技术工作，大部分的职责是要求与组织中所有对数据仓库感兴趣的成员一起有效地工作。数据库管理员有时会遇到一些困难，对于数据仓库管理员来讲，由于他们的责任范围比数据库管理员更大，也就存在更大的挑战。

管理功能的设计和管理队伍的选择对于一个数据仓库来说是至关重要的。在一个大型组织里管理数据仓库无疑是一项重大的任务。许多商业化工具已经可以用来支持管理功能。有效的数据仓库管理将是一种团队活动，要求有一个拥有专业技能、密切合作和有效领导的大集体。正如我们必须为数据仓库的发展做好准备一样，也必须认识到管理团队的技能也将会随之发展。

13.7.2　数据仓库的开放问题

已经有许多围绕“数据仓库”这个术语的市场化的夸张描述。这些夸张描述预计可能会逐渐平息，但是数据仓库是支持专门分析和决策支持的集成数据汇集，这个概念无疑会持续下去。

作为一个活跃的研究领域，在不久的将来由于数据仓库和数据集市的迅速增长，关于数据仓库

技术的研究活动可能会不断地深入和扩大。老问题也将得到新的重视，例如，数据清洗、索引、分区以及视图可能会重新获得关注。

数据仓库技术的学术研究可能将集中在数据仓库的自动化方面，目前许多方面还需要大量的人工干预，例如，数据获取、数据质量管理、选择和构建合适的存取路径与结构、自身可维护性、功能以及性能优化。将主动数据库的功能（见7.1节）应用于数据仓库可能也将引起很大的关注。在数据仓库的创建与维护过程中，多领域技术的综合、业务规则的适当应用，可以使数据仓库更具智能化，更具相关性，也更具自治性。

数据仓库的商业化软件已经可用，可以从一些软件提供商那里得到，主要集中在数据仓库的管理和OLAP/DSS的应用方面。数据仓库的其他方面，例如设计、数据获取（尤其是清洗），主要由组织内部的IT经理和顾问们进行进一步的研究。

小结

在本章中我们对数据仓库领域进行了概要介绍。数据仓库可以看作一个过程，这个过程要求一系列前期活动。相比之下，数据挖掘（见第12章）可以看作是一个从已有的数据仓库中抽取知识的活动。在此还介绍了与数据仓库相关的重要概念，并讨论了与数据的多维视图相关的专门功能。本章还讨论了数据仓库为决策制定者提供准确层次的细节信息的有关方法，这些方法基于合适的组织与视角。

复习题

13.1　什么是数据仓库？它与数据库的区别是什么？

13.2　定义术语：OLAP（联机分析处理）、ROLAP（关系OLAP）、MOLAP（多维OLAP）、DSS（决策支持系统）。

13.3　描述数据仓库的特征。可从两个方面来考虑：数据仓库的功能和用户从中得到的好处。

13.4　什么是多维数据模型？在数据仓库中怎样使用它？

13.5　定义下列术语：星型模式、雪花模式、事实星座和数据集市。

13.6　对数据仓库建立了哪几种索引？举例说明每种索引的用处。

13.7　描述建立数据仓库的步骤。

13.8　在数据仓库的设计中哪些考虑事项起主要作用？

13.9　描述用户可以在数据仓库上实现的功能，并且列举一个多维数据仓库的例子来说明这些功能的结果。

13.10　如何将关系视图的概念与数据仓库、数据集市相关联？它们有哪些区别？

13.11　列举实现数据仓库过程中的困难。

13.12　列举数据仓库中的开放问题和研究难题。

参考文献

数据仓库已经成为一个非常流行的主题，并且已经出现在近几年的许多出版物中。这个术语的广泛应用应当归功于（Inmon，1992）。（Codd，1993）将这个术语推广到联机分析处理（OLAP），并且为了支持OLAP为数据仓库定义了一系列的特性。（Kimball，1996）因它在数据仓库领域的贡献而著名。（Mattison，1996）是若干关于数据仓库书籍的著作之一，在这本书中，对数据仓库中可用的技术和公司应该用来配置它们的多种策略给出了全面的分析。Ponniah概要介绍了从需求汇集到配置维护的整个数据仓库构建过程。（Bischoff & Alexander，1997）对数据仓库领域专家意见进行了编辑。（Chaudhuri & Dayal，1997）是针对这个主题的非常优秀的教程，同时（Widom，1995）指出了许多突出的研究问题。

第14章 新兴的数据库技术及应用

前面各章讨论了数据库的建模、设计和功能等相关问题，也谈到了与数据库管理系统相关的内部结构和性能问题。第11章讨论了Internet数据库，通过Internet数据库可以更普遍地访问数据，还讨论了促进不同的数据库与不同DBMS平台上应用开发的XML的用法。在前面两章中，讨论了数据库管理技术的变种，其中包括数据挖掘与数据仓库，这些技术为决策支持提供了庞大的数据库和相关工具。本章的重点是讨论在数据库领域中正在不断发展的两个方面：新兴的数据库技术及主要应用领域。但是，我们不打算详细讨论这些问题，而只是涉及一些主要的技术及其应用进展。首先，通过这些技术可以在DBMS中创建新功能，从而支持各种新应用，主要有：移动数据库，它允许用户在移动中广泛而灵活地访问数据；多媒体数据库，它提供对多媒体信息存储与处理的支持。移动数据库与多媒体数据库技术引发了诸多问题，14.1节和14.2节将简要介绍并讨论其中的一些问题及其解决方法。

接下来，我们考虑两个应用领域，这两个领域原来都是依靠人工处理文件系统或定制系统解决方案。14.3节将讨论地理信息系统，地理信息系统处理两种数据：独立的地理数据，以及空间数据与非空间数据的组合，例如人口普查数据。14.4节将讨论生物数据库及其应用，尤其是不同生物体的基因数据，其中包括人类基因组数据。这些应用的一个共同特征是：在每个特定应用领域中的数据都是该领域专用的自然数据。此外，这些数据都有静态特征，即最终用户只能从数据库中检索数据。数据库领域专家负责更新信息，并监管和分析新输入的数据。

14.1 移动数据库[①]

便携式和无线技术的最新进展引出了**移动计算**（mobile computing）的概念，这是一种数据通信与处理的新技术。便携式计算设备与无线电通信使得用户可以在任何地点、任何时间访问数据。这个特性对于地理上分散的组织尤其有用。典型的例子可能包括电子随从、新闻报道、代理服务和自动销售等。但是，还有许多硬件和软件问题需要在移动计算可以全面应用之前得到解决。

在分布式数据库系统中存在的一些软件问题，包括数据管理、事务管理和数据库恢复，在移动计算中，这些问题变得更加棘手了，其主要原因包括无线通信承受有限的、间歇性连接，为移动单元提供的电源（电池）的有限寿命，以及网络拓扑结构的随时变化。除此之外，移动计算还引发了新的体系结构问题和挑战。

14.1.1 移动计算体系结构

1. 基于基础设施的移动平台

图14-1[②]所示的是移动平台的通用体系结构。这是一个分布式结构，其中包括大量计算机——称为**固定主机**（fixed host，FS）和**基站**（base station，BS），它们通过高速的有线网络互相连接。固定主机是通用计算机，通常没有管理移动单元的装备，但是可以通过配置主机来管理移动单元。基站

① 在此感谢Waigan Yee与Wanxia Xie对本节作出的贡献。

② 改编来自（Dunham & Helal，1995）。

是**移动单元**（mobile unit，MU）到固定网络的网关。基站上装有无线接口，并提供网络访问服务，在这种服务中，移动单元是客户端。

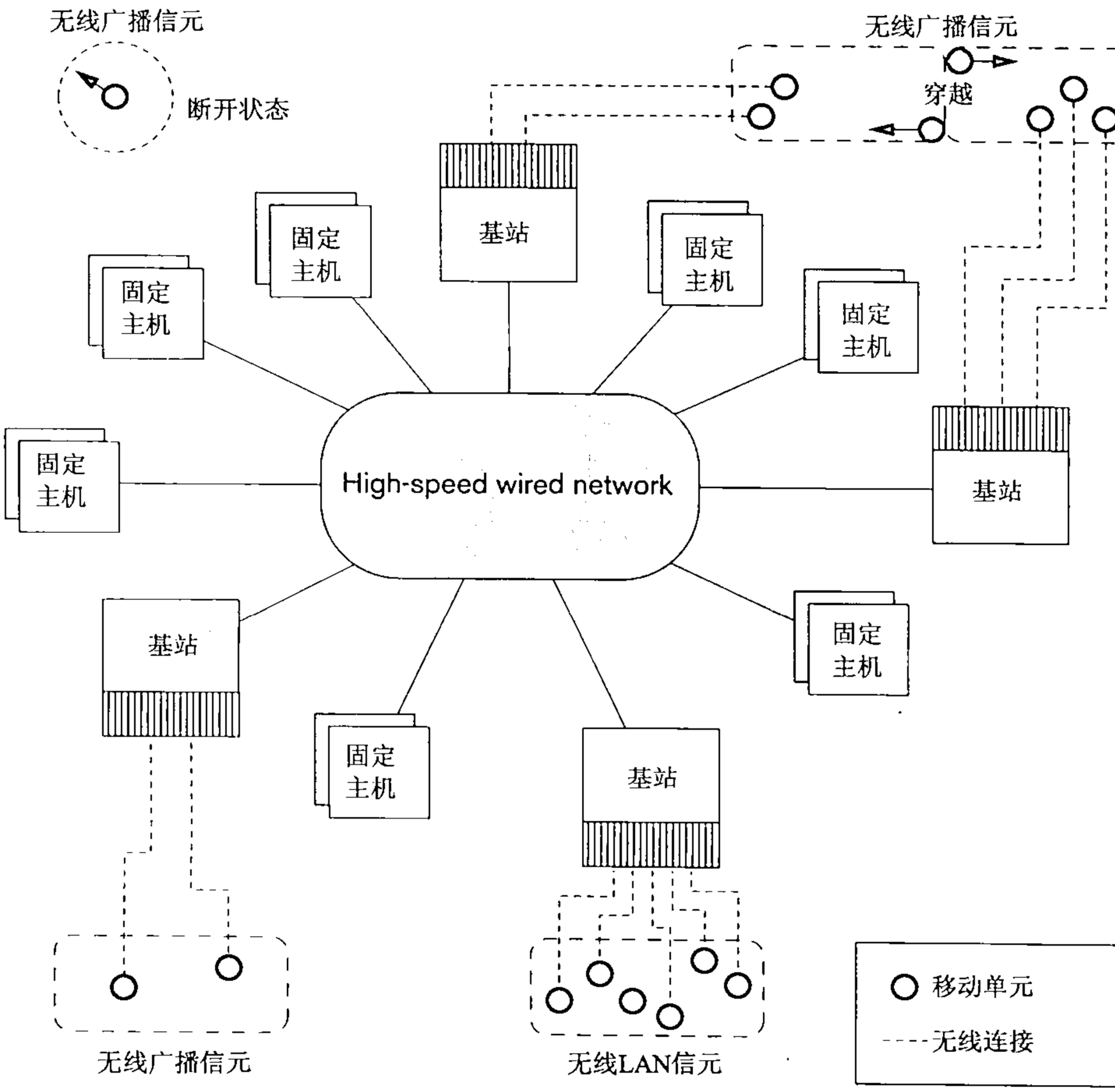


图14-1　基于基础设施的移动平台的通用体系结构

移动单元与基站通过无线介质进行通信，而无线介质的带宽明显低于有线网络。当前的无线技术的数据传输速率从几十kbit/s到几百kbit/s（2G移动电话）到几十Mbit/s（无线以太网，通常称为WiFi）。而现代（有线）以太网的数据传输速率能达到几百Mbit/s。

除数据传输速率之外，还可以用其他特征区分无线连接选项，其中包括范围、干扰、访问的本地性以及对包交换的支持。一些无线访问选项支持一个地理区域内的无缝漫游（例如蜂窝网络），而WiFi网络在其基站周围是局部的。某些无线网络（例如WiFi和蓝牙）使用无执照的频谱区域，这可能会与其他电器设备发生冲突，例如无绳电话。最后，现代无线网络能够以包为单元传输数据，为了保持带宽，这在有线网络中是常用的。无线应用在选择通信选项时，必须考虑这些特征。例如，物理对象会阻碍红外线信号。尽管这些阻碍会给一些应用带来不便，但在一个封闭房间内这种阻碍会使得无线通信更安全。在一个**地理移动区域**（geographic mobility domain）中，它是无线网络覆盖范围限定的一个区域，移动单元可以自由地移动。为了管理单元的移动性，可以把整个地理移动区域划分为一个或多个更小的区域，这些区域称为**信元**（cell），每个信元至少有一个基站支持。移动规范要求移动单元能够在信元内自由移动，同时要维持信息的**访问连续性**。也就是说，单元的移动不能对数据检索过程造成不好的影响，尤其是信元之间的移动。

2. 无基础设施的移动平台

上面描述的通信体系结构会给人这样的一个印象：移动单元是附加到固定网络上的，这种结构模仿了传统的客户-服务器体系结构。但是，无线通信也有可能采用其他的体系结构。一种可选的体系结构是无基础设施的移动平台，也称作移动特定网络（MANET），如图14-2所示。在MANET中，协同定位的移动单元不需要经由固定网络通信，而是采用性价比更好的技术形成自己的网络，例如蓝牙技术。在MANET中，移动单元负责自己数据的路由，起着作为基站的作用，同时也可以作为客户。而且，移动单元必须足够健壮来处理网络拓扑结构的变化，例如其他移动单元的到达或离开。所以，MANET应用在移动计算和移动数据管理的相关问题中又添加了一个附加的复杂层。

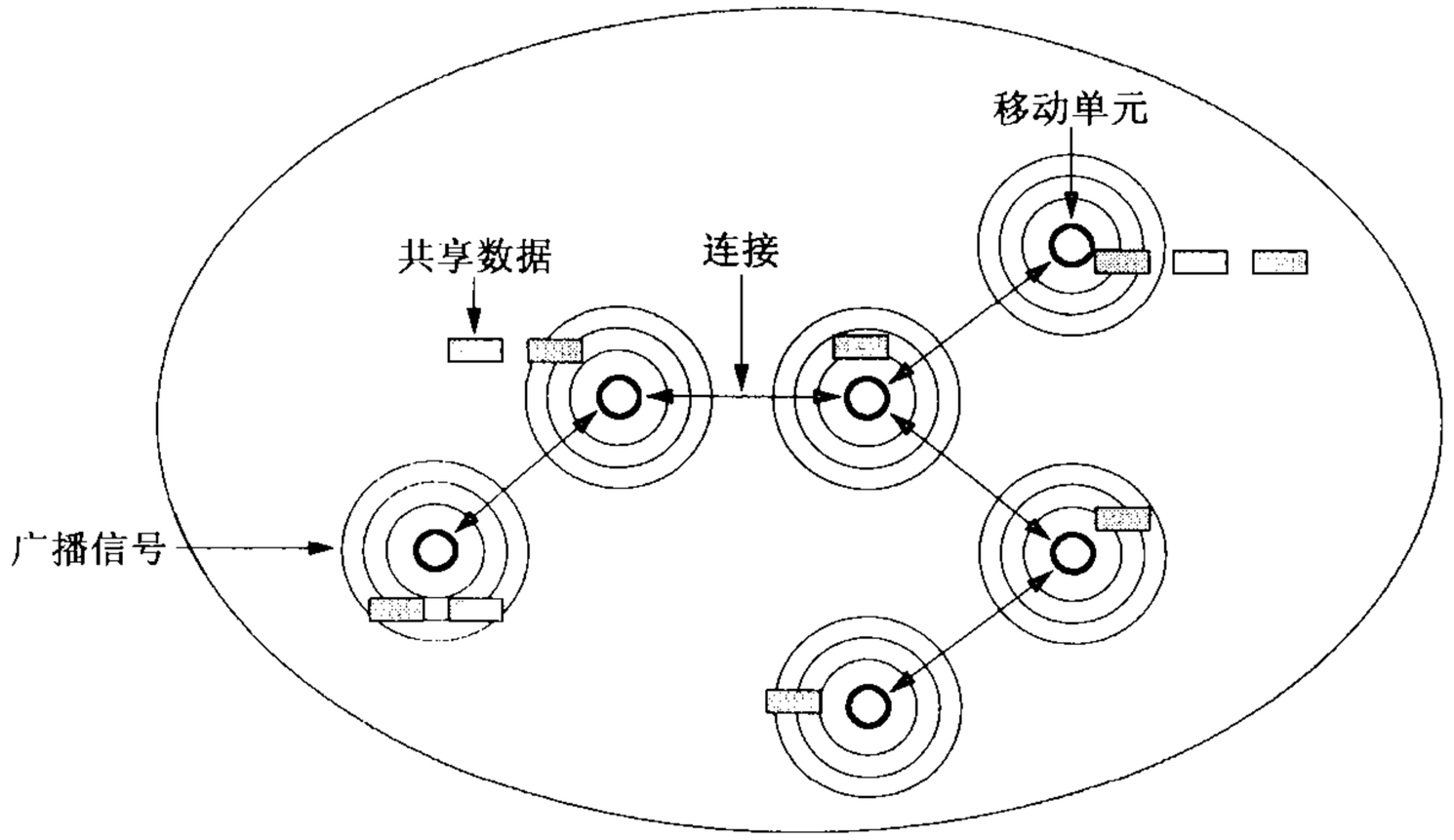


图14-2　MANET的体系结构：移动特定网络[①]

MANET应用可以归为点对点（peer-to-peer，P2P）的范例，这就意味着移动单元可以同时作为客户机和服务器。点对点体系结构也是一种网络类型，在这种网络类型中各点有着同样的责任。这与服务器致力于为客户服务的客户-服务器体系结构不同。有两种P2P网络：**混合P2P网络**和**完全P2P网络**。在混合P2P网络中，有一个中心目录服务器向所有点提供信息。通常各点在目录服务器中发布资源信息，并向目录服务器查询资源信息。Napster是一个典型的混合P2P网络的示例。在完全P2P网络中，不存在中心服务器。在第一代完全P2P网络（如Gnutella）中，查询和路由是通过广播完成的，这就导致了网络中的消息泛滥。在第二代完全P2P网络中（这就是结构化P2P网络，如Chord和CAN），查询和路由选择基于分布式散列表。结构化P2P网络确保能够连接到某些中继段（hop）中的各点。下面所讨论的MANET应用都是针对完全P2P网络的。

MANET应用的主要特征如下所示：

- **频繁的断开连接**。我们不能假定在协同应用的开发过程中，所有移动设备都总是保持连接的。我们需要特别考虑移动单元频繁的、有时是有意断开连接的情况。
- **频繁的网络分割**。移动设备的频繁断开连接和移动会导致频繁的网络分割。在开发MANET应用时，网络分割是一个需要非常慎重考虑的问题。
- **难以集中控制**。MANET中的中央服务器不总是实际的或可行的。例如，在网络分割情况下的中央服务器会使得某个分割之外的所有设施瘫痪。
- **点异构性**。自然地，当我们考虑到各个平台上包括操作系统、语言和数据库等软件和硬件的时候，各个点之间存在异构性。而且，在计算能力上，如CPU和存储、网络带宽以及电池功率，各点也是异构的。它的本质是基于各点的能力进行负载平衡。

① 这一体系结构是基于（IETF，1999）中的IETF提议，并伴有Carson和Macker的评论（1999）。

由于这种架构中可能出现网络分割，并且没有中央控制，所以事务处理和数据一致性控制变得非常困难。MANET通过移动单元进行资源发现与数据路由，这使得MANET中的计算更为复杂。这就需要开发新的模型、技术和算法，使得我们能够建立和部署MANET应用。MANET应用的示例包括：自然或非自然灾难的应急系统、战场信息共享系统、水路交通系统、多用户游戏、共享白板和分布式日历等。2001年9月11日的悲剧有力地证明了对这种应用的需求。尽管事实上纽约市有一个灾难防御计划，它们应对这种袭击的能力也是有限的。双塔下的灾难协调基地被摧毁了，一同被摧毁的还有附近的Verizon电话中继站。这些设施的损坏使得它们有效协调人员和资源的能力完全瘫痪，也带来了悲剧的结果。于是有了这样的期望，即在数年之后这些网络以及相关应用变成主导的应用。现在MANET是学术界和工业界的一个热点研究领域。这项研究还处于起步阶段，所以下面的讨论依然集中于之前描述的基本移动计算体系结构。

14.1.2　移动环境的特征

移动环境中的计算特征包括高通信等待时间、不连续的无线连接性、有限的电池寿命，当然还包括变化的客户机位置。等待时间是由无线介质独有的过程引起的，例如无线传输编码数据、接收器对无线信号的接收与过滤。电池寿命与电池大小直接相关，并与移动设备的性能和有效性有间接的关系。间歇的连通性则是有意识的或无意识的。无线信号无法到达的区域会发生无意识的断开，例如在电梯中或者在地铁隧道中。有意识的断开则是指用户的故意断开，例如飞机起飞过程中，或者移动设备关机。最后，期望客户机经常移动，这是它们的特权。所有的这些特征都会影响数据管理，在设计健壮的移动应用时必须考虑到这些特征。

为了弥补较长的等待时间和不可靠的连接性，客户机的高速缓存中复制了重要的、经常访问的数据，必要时可以脱机工作。访问数据需要通过无线传输进行，这些传输需要消耗较多的能量，使用高速缓存可以避免无线传输，所以除了提高数据的可用性和改善响应时间外，高速缓存还降低了客户的能耗。

另一方面，服务器可以不到达客户机。客户机也可以是不可到达的，它可能会处于**休眠**（dozing）状态，即处于一个能量保持状态，在这种状态中多个子系统被关闭，或者不在基站范围之内。还有一种情况：客户机与服务器都不能到达对方，所以必须修改上述体系结构来弥补这种情况。一种解决方案是在体系结构中增加不可达组件的**代理**（PROXY）。对于客户机（相应地对于服务器），代理能够高速缓存对于服务器的更新。当连接可用时，代理再把这些被高速缓存的更新自动传送到最终目的地。

正如上面所建议的那样，移动计算对服务器和客户机都提出了挑战。无线通信中的等待时间成为系统可伸缩性的障碍。因为无线通信引起的等待时间延长了每个客户请求服务的时间，这样服务器能够处理的客户数就更少了。服务器解决这一问题的方法之一是可能随时**广播**数据。广播利用了无线通信的自然特征：由于数据项的单一广播可以满足对它的所有请求，所以是可伸缩的。例如，服务器可以简单地定时广播天气信息，而不必向一个信元内的所有客户单独发送。广播也降低了服务器的负载，这是由于那些客户机不必都维持主动连接到服务器的状态。

客户机的移动性也给数据管理带来了挑战。首先，服务器必需跟踪客户机的位置才能有效地向它们传递信息。其次，客户机数据应该存储在网络站点上，以使得访问客户机所必需的流量最小化。如果客户机移动到距离站点“很远”的地方，那么在该固定站点上存储数据就会增加访问等待时间。最后，如同上面提到的情形，信元之间的移动行为对客户机必须是透明的。服务器必须能够适度地把传送的数据从一个基站转移到另一个基站，而客户机不会意识到这种转移。

客户机的移动性还支持基于位置的新应用。例如，考虑电子随从应用，它可以告诉用户最近的餐馆的位置。很明显，最近是相对于客户机的当前位置，而移动会使先前存储的所有响应都失效。在移动的基础上，客户机必须及时删除部分缓存内容，并从数据库请求更新的数据。

14.1.3　数据管理问题

从数据管理的角度来看，移动计算可以看作是分布式计算的一种变体。在下述两种可能的方案中，移动数据库可以看作是分布式的：

(1) 整个数据库主要分布在有线组件当中，每个组件可能拥有全部或部分的数据复制。基站或固定主机使用类似于DBMS的功能管理自己的数据库，同时提供附加功能以确定移动单元的位置，并支持附加的查询和事务管理，以满足移动环境下的应用需求。

(2) 数据库分布在有线和无线组件当中。数据管理的职责由基站或固定主机和移动单元共同负责。

因此，在第8章中所讨论的分布式数据管理问题，也可以应用于基于以下考虑和变化的移动数据库中。

- **数据分布和复制**。数据不均衡地分布在基站和移动单元中。一致性约束增加了缓存管理的难度。移动单元处理其自身的事务，并且可能长时间断开连接，而缓存试图为移动单元提供最频繁访问的数据或更新的数据。
- **事务模型**。在移动环境中，事务的容错和纠错变得更为棘手。一个移动事务在几个基站上顺序执行，由于移动设备的移动性，执行过程可能会涉及多个数据集合。此时缺乏对事务执行的集中协调，尤其是在上述第二种方案中。此外，由于移动单元断开连接，但仍被期望移动事务能够长时间存在。因此，需要对事务的传统ACID性质（见第3章）进行修正，并定义新的事务模型。
- **查询处理**。了解数据的存储位置非常重要，这影响到查询处理的性价比分析。由于移动单元的移动性和快速的资源变化，查询优化变得更加复杂了。当查询结果返回到移动单元时，这些移动单元可能正处于移动当中，或者正在穿越信元边界，但用户接收到的查询结果必须是正确而完整的。
- **恢复和容错**。移动数据库环境必须解决地址错误、介质失效、事务和通信失效所导致的问题。移动单元的地址错误常常是由电池电量不足引起的。如果移动单元自行关机，就不应被视为失效。移动设备在穿越信元时常会发生事务失效。移动单元失效会导致网络分割和影响路由算法。
- **移动数据库设计**。由于移动性以及频繁的关机，执行查询时全局名字解析问题变得更为复杂。移动数据库设计必须考虑到元数据的许多管理问题。例如，位置信息的不断更新。
- **基于位置的服务**。随着客户机的移动，依赖于位置的高速缓存信息也随之失效。在这种情况下，回收（eviction）技术非常重要。此外，要频繁更新依赖于位置的查询，然后应用这些（空间）查询以更新高速缓存，这也将会带来问题。
- **工作的划分**。移动环境特定的特征强制改变了查询处理中的分工。在某些情况下，客户机必须独立承担服务器的功能。但是，允许完全独立地访问副本数据的后果将是什么呢？客户机的能力与它们的责任之间的联系有待进一步研究。
- **安全**。移动数据的安全性不如固定位置数据的安全性。在这种环境下，管理与授权访问关键数据的适当技术就变得更加重要。移动数据是更易变的，相关技术必须能够弥补这个缺点。

14.1.4　应用：间歇同步数据库

随着移动计算方式逐渐普及起来，人们逐渐离开办公室和家进行工作，从而扩大了活动和功能的范围，这些活动包括各种类型的销售（特别是药品、消费品和工业零件的销售），法律实施，保险、金融咨询和规划，房地产及财产的管理活动，邮递与运输服务等。在这些应用中，由一个或一组服务器对中央数据库进行管理，大部分时间客户机携带装有DBMS软件的便携式电脑或掌上电脑来执行本地事务活动。客户机通过网络或拨号连接（可能甚至通过Internet）与服务器连接，这些连接通常是短期会话，例如30～60分钟。客户机把更新发送给服务器，服务器必须随之将更新数据输入到

中央数据库，中央数据库负责维护最新的数据，并为系统中的所有客户机准备适当的副本。因此，只要客户机通过一个称作客户机与服务器同步的过程连接到服务器，它们就可以接收到一组要安装到本地数据库的更新数据。这种情况的主要特征是客户机通常是与服务器断开连接的，服务器没有必要非得到达客户机。这个环境也存在着类似于分布式数据库和客户-服务器数据库中存在的问题，还有一些问题来自于移动数据库，但也提出了一些另外的研究问题。我们把这种环境称为**间歇同步数据库环境**（Intermittently Synchronized Database Environment，ISDBE），相应的数据库称为间歇同步数据库（Intermittently Synchronized Database，ISDB）。

总之，ISDB的下列特征使得它们不同于前面已经讨论过的移动数据库：

(1) 客户机在需要从服务器接收数据，或者向服务器发送更新数据，或者需要处理非本地数据的事务时，才与服务器进行连接。这种通信可以是单播（unicast）的，即服务器与客户机之间一对一通信，也可以是组播（multicast）的，即一个发送器或服务器可以周期性地与一个接收者集合通信，或者更新一组客户机。

(2) 服务器不能随意和客户机进行连接。

(3) 客户连接是无线连接还是有线连接，以及电源电量的问题并不重要。

(4) 客户机处于断开状态时，仍然可以方便地管理自己的数据和事务。在某种程度上客户机也可以执行它们各自的恢复操作。

(5) 客户机和服务器有多种连接方式，在多个服务器的情况下，基于临近原则、通讯结点和资源可用性等，客户机可选择与特定的服务器进行连接。

由于存在以上这些不同，有必要对ISDB的若干问题进行研究，这些问题不同于移动数据库系统中的典型问题。这些问题包括服务器数据库设计、客户机和服务器数据库事务与更新处理之间的一致性与同步管理、服务器带宽的有效利用以及ISDB环境下的可扩充性等。

14.2　多媒体数据库

未来多媒体信息系统有望主导我们的日常生活。我们的房子里会装上有线的宽带网络来处理交互式的多媒体应用。高清晰电视/计算机工作站将会访问多种数据库，其中包括数字图书馆、图像与视频数据库，这些数据库都将会分发大量的来自多个数据源的多媒体内容。

14.2.1　多媒体数据和应用的特征

在8.3节中，我们讨论了与多媒体数据相关的高级建模问题。第6章中讨论对象-关系数据库管理系统（ORDBMS）时，谈到如何对多种类型的数据进行处理。DBMS也不断扩展它们所支持的数据类型，在当前的系统中如下类型的多媒体数据是可用的。

- **文本**。包括格式化的和非格式化的文本。借助SGML和其变种如HTML，可以很容易地解析结构化文档。
- **图形**。示例包括使用一些描述性标准，例如CGM、PICT、postscript，进行编码的绘图和说明等。
- **图像**。使用标准格式，如位图、JPEG和MPEG，进行编码的图形、照片等。其中，JPEG和MPEG格式对图像进行了压缩。这些图像不能进一步细分为组件，因此，对其进行基于内容（例如，查找包含圆圈的所有图像）的查询绝非易事。
- **动画**。图像或图形数据的时态序列。
- **视频**。以特定速率输出的摄影数据的时态序列集，例如每秒30帧。
- **结构化音频**。包括音符、音调、持续时间等音频元素的序列。
- **音频**。从数字化位串表示的声音记录中得到的样本数据。存储之前一般把模拟记录转换为数字形式。

- **合成或混合的多媒体数据**。是多媒体数据类型的组合，例如音频和视频，它们既可以物理地混合以生成新的存储格式，也可以逻辑上混合保持原来的类型与形式。合成数据还包含附加的控制信息，它描述了如何呈现信息。

多媒体应用的特征

多媒体数据可以以多种不同的方式存储、传递和使用。根据多媒体数据的管理特征，可对多媒体应用进行如下分类。

- **资料库应用**（repository application）。大量的多媒体数据和元数据被存储以备检索之用。包含多媒体数据的中央资料库可以由DBMS进行维护，并且可组织成层次的存储结构，如本地磁盘、第三级磁盘和磁带、光盘等。这一类应用包括卫星图像资料库、工程制图与设计资料库、空间照片资料库以及X射线扫描照片资料库等。
- **表示应用**（presentation application）。大量的应用涉及多媒体数据的传递，这种传递受时态约束的控制，音频和视频数据就是用这种方式传递的。在这些应用中，为获得最优的视听效果，DBMS要在一定阈值之上以某个特定传输率传递数据以提供服务质量。数据在传送过程中被实时地进行处理，这和资料库应用不同，在资料库应用中数据的处理可以延后进行（如多媒体电子邮件）。例如，对视频数据的简单的多媒体观看需要一个可以模拟VCR功能的系统。复杂的和交互的多媒体展示包括控制串行和并行组件检索顺序的编曲方向。交互式环境必须有支持如实时编辑、分析或者注释视频音频数据的能力。
- **基于多媒体信息的协作**（collaborative work using multimedia information）。这是一个新的应用领域。在这类应用中，工程师可以执行复杂的设计任务，包括图形合并、根据设计约束进行主题剪裁、生成新文档、修改通知等。例如智能保健网络和远程医疗，要求医师互相协作，分析实时产生的多媒体病人数据和信息。

所有这些应用都对多媒体数据库系统的设计提出了很大的挑战。

14.2.2　数据管理问题

多媒体应用系统需要处理大量的图像、文档、音频和视频片段以及自由文本数据，这依赖于数据结构与内容的合适的建模，并随后对多媒体信息的存储和检索设计相应的数据库模式。多媒体信息系统非常复杂，涉及大量问题，包括如下几个主要的问题。

- **建模**。把数据库与信息检索技术应用到建模问题上还有很大的潜力。但在处理由各种类型的数据组成的复杂对象（见第4章）时存在着不少问题，这些数据类型包括数值、文本、图形（计算机生成的图像）、动画图像、音频流及视频序列等。另外，文档构成了特殊的领域，要给予特别的关注。
- **设计**。多媒体数据库的概念设计、逻辑设计和物理设计需要进一步研究，现在仍然是热门的研究领域。设计过程可以基于《数据库系统基础：初级篇》中的第12章所描述的通用方法学，但是每个层次的性能和优化问题会更加复杂。
- **存储**。把多媒体数据存储于标准的类磁盘设备上，会给信息的显示、压缩，到设备层次的映射、存档以及I/O操作时的缓冲带来困难。大部分多媒体产品生产商坚持采用JPEG或MPEG等标准来解决上述问题。在DBMS中，二进制大对象（Binary Large Object，BLOB）支持无类型位图的存储和检索。需要使用标准化的软件对同步和压缩/解压缩进行处理，这将涉及索引问题，多媒体数据的索引尚处于研究阶段。
- **查询和检索**。数据库方式的信息检索基于查询语言和内部索引结构。信息检索方式则严格依赖于关键字或预定义的索引项。对于图像、视频数据和音频数据的查询和检索带来了许多问题，例如有效的查询表达式、查询执行及优化。《数据库系统基础：初级篇》中的第16章所讨论的标准优化技术需要进行修改，才能应用于多媒体数据类型。

- **性能**。对于只涉及文档和文本的多媒体应用，性能约束取决于用户。对于视频回放和音频—视频同步的应用，性能表现则取决于物理限定。譬如，必须稳定地以30帧每秒的速率传输视频信息。查询优化技术可以在计算查询之前估算期望的响应时间。使用数据的并行处理技术可能会在一定程度上解决部分问题，但是这些努力当前是受限于进一步的实验。

上面所给出的问题导致了各种开放的研究问题，接下来我们讨论几个具有代表性的问题。

14.2.3　开放研究问题

1．多媒体数据库查询中的信息检索透视

数据库系统中的数据具有严格的结构，从数据模式可推知数据实例的意义，因此，数据库建模与数据库系统中的数据内容建模不是什么问题。但信息检索（information retrieval，IR）主要关注文本文档内容的建模（通过使用关键字、短语索引、语义网、词频、探测法编码等），而通常忽略多媒体对象的结构。借助内容建模，通过检查文档的内容描述符，系统就能确定出文档是否与当前查询相关。例如，把一个保险公司的事故声明报告看作多媒体对象，它包括事故现场图像、结构化的保单、事故当事人的音频记录、保险公司出示的文本报告及其他诸如此类的信息。在这种情况下应该采用哪种数据模型来表示多媒体信息呢？如何依赖这些数据构建查询？因此有效的查询执行成为一个复杂的问题。多媒体信息的语义异构性和表示复杂性又引发了多个新问题。

2．多媒体/超媒体数据建模和检索需求

为了充分利用多媒体数据建模的表达能力，多媒体系统应提供通用的结构，以支持用户在任意两个独立结点之间指定链接。**超媒体链接**（hypermedia link）或者说超链接具有一些与通常意义下的链接所不同的特性：

- 无论是否借助相关信息都可确定相应链接，这些链接可能具有大量关于它们的描述信息。
- 链接可以始于结点内的某一指定点，也可以始于整个结点。
- 链接可以是有方向性的，也可以是无方向性的，可以在任一方向遍历链接。

数据模型的链接能力要综合考虑上述问题。当执行基于内容的多媒体信息检索时，查询机制要访问链接以及与链接相关的信息。系统还应提供在所有链接上定义视图的工具，包括私有的和公共的。从结构化信息中可获得有用的上下文相关信息。与手工生成的超媒体链接相比，虽然不能提供关于两个结点的任何新信息，自动生成的超媒体链接却具有完全不同的意义。方便创建及使用链接，并且开发相应的导航式查询语言来使用这些链接，都是任何允许有效使用多媒体信息的系统的重要特征。该领域的研究对于Web上互联的数据库也很重要。

万维网提供了一个使得人们可以通过非结构化或者结构化互连的数据库阵列来访问海量信息的机会。Web的成功及增长使得很多问题，如发现、访问、维护这些信息，变得非常有挑战性。过去几年中，一些项目试图定义框架和语言，以便允许人们定义Web的语义内容，这样网络得到了良好定义并可使用机器进行处理。这一努力集中表现为**语义网**（Semantic Web）这一术语。资源描述框架（resource description framework，RDF）、可扩展超文本标记语言（Extensible Hypertext Markup Language，XHTML）、DARPA代理标记语言（DARPA Agent Markup Language，DAML）以及本体论推理层（Ontology Inference Layer，OIL）是语义网的部分主要组件[①]。更详细的讨论超出了本书我们讨论的范围。

3．图像索引

有两种建立图像索引的方法：通过图像处理技术自动识别对象和通过手工索引来指派索引项和短语。使用图像处理技术建立图像索引的一个重要问题涉及可伸缩性。目前的技术只能对图像的一些简单特征建立索引。随着可识别特征的增加，构建索引的复杂性会相应加大。另一个重要的问题

① 这些术语的概述见（Fensel，2000）。

是涉及查询的复杂性问题。可以采用不同的规则和推理机制从图像的简单特征中获取图像的高级信息。类似的，可以采用抽象方法来捕捉一些概念，不仅仅是用<属性，值>所定义的概念。这样，用户可被允许执行高级查询，譬如建筑学应用中的一个查询实例，“查找具有露天大厅并且前台区域可获得充足光照的旅馆”。

图像索引的信息检索方法基于下面3种索引策略。

(1) **分类系统**。按预定义的类别对图像进行层次结构分类。使用这个策略，索引编制人员和用户对于可用的类别要有充分的了解。但该策略不能获取复杂图像更详细的细节及图像中对象之间的联系。

(2) **基于关键字的系统**。使用类似于文本文档索引中采用的的索引词汇表。使用这个策略，可获取关于图像中表示的一些简单信息(如冰雪覆盖的地区)，以及经过人类的高级翻译所得到的信息(如终年未化的冰、最近的降雪量、极地地区的冰等)。

(3) **实体-属性-联系系统**。能够识别图像中所有的对象，对象间的联系以及对象的属性。

在文本文档的情形中，索引者可以从索引文档的可用词汇集合中选择关键字。但在处理视觉与视频数据时却不可能。

4. 文本检索中的问题

文本检索一直是商业应用和图书馆系统中所需的关键特征，虽然对以下问题已经做了一些工作，但是还有待改进，尤其是下列问题。

- **短语索引**。倘若短语描述符（与单词索引不同）是文档内容及所需信息的很好的标识符，那么，如果给文档内容和信息要求指定短语描述符，并应用于查询，则可实现显著改进。
- **使用主题词表**。目前的系统在文本检索方面不尽如人意，问题在于，用户词汇表不同于索引文档所使用的词汇表。一个解决办法是：使用主题词表来扩充用户查询的相关术语。这样，问题则转化为在感兴趣领域查找主题词表。在这个环境中的另一个资源是**本体论**（ontology）。本体论包含了与给定领域相关的一些关于外部世界的分类视图，这个世界视图通常包含了一组概念（例如，实体、属性和过程），它们的定义和联系描述了一个目标世界。本体论可以通过两种方法构造：即域相关本体论和一般本体论。一般本体论的目的是针对人类所遇到的所有（或大部分）类别构建一个通用框架。目前已经构造了各种域本体论，例如基因本体论（见13.4节）或者电子组件本体论[①]。
- **解决二义性**。在文本信息检索系统中，文本信息检索系统精度（检索到的相关项和所检索的所有项的比）较低的原因之一是词汇有多个上下文相关的含义。解决二义性问题的一种方法是使用在线字典或本体论；另一种方法是比较两个词出现的语境。

在DBMS发展的前30年（大概是1965年～1995年）其应用主要集中在大部分数值型的商业和工业数据的管理上。未来的几十年内，非数值型的文本信息可能会成为数据库系统管理的主要内容。文本检索问题正在变得与HTML文档和XML文档语境相关联。当前Web包含几十亿个这样的页面。搜索引擎会查找给定单词列表的相关文档，自由形式的自然语言查询正是这种情况。获取满足精度（相关检索到的文档的百分比）和检索率（检索到的所有相关文档的百分比）需求的正确结果仍然是个难题，在信息检索中它们是度量的标准。其结果会给DBMS增加各种功能，包括文档比较、文档概念化、文档理解、文档索引与文档汇总。多媒体信息系统促进了在历史上是两个独立领域学科的信息检索和数据库管理的融合。

14.2.4　多媒体数据库应用

大规模的多媒体数据库应用涉及多种学科，并增强现有的能力。以下是一些主要的应用实例。

- **文档和记录管理**。大量的工业、商业保存了详尽的记录与各种文档。这些数据可能包括工程

① (Uschold & Gruninger，1996）对本体论进行了很好的讨论。

设计与制造的数据、患者的医疗记录、出版材料和保险索赔记录等。

- **知识传播**。日益增长的电子图书、电子目录、电子手册、电子百科全书以及主题信息资料库等多媒体载体，将成为知识传播的有效途径。
- **教育培训**。可以根据多媒体资源来为不同的听众设计计算机辅助培训和交互学习的材料，从幼儿园的学生到设备操作者，再到专业人员。不久的将来，数字图书馆有望改变学生、研究人员以及其他用户访问大量教育资源库的方法。
- **市场、广告、零售、娱乐和旅游**。在这些应用领域，多媒体的应用无处不在，从有效的销售展示到虚拟城市旅游和艺术画廊等。电影工业已经显示了特效的力量，包括创建动画与合成设计动物、外星人和特殊效果。采用多媒体数据库中预先设计好的存储对象可以扩展这些应用的范围。
- **实时控制与监测**。利用主动数据库技术（见第8章），信息的多媒体表示可以有多种有效的监测与控制复杂任务的方法，这些复杂任务包括生产操作过程、核电站计划、特殊病人护理和运输系统等。

商用多媒体信息管理系统

目前，还没有DBMS专门是为了多媒体数据管理而设计的，因此没有哪个系统能够完全支持上述的所有多媒体信息管理的应用需求。然而已经有一些DBMS提供对多媒体数据类型的支持，这些DBMS包括Informix Dynamic Server、IBM的DB2 Universal database (UDB)、Oracle 9与10、CA-JASMINE、SYBASE以及ODB II。这些DBMS都支持对象技术，这对于各种复杂多媒体对象的建模是最基本的。这些系统的一个主要问题是需要以特定的方法设计处理多媒体数据的“刀片、盒式磁盘、扩展器”。提供的功能没有充分考虑到可伸缩性和性能。还有一些产品支持基于内容的图像检索，这些产品系统或者独立运行，或者和其他厂商的系统协同工作，这些产品包括Virage、Excalibur以及IBM的QBIC。多媒体操作尚需进一步标准化，MPEG-7及其他标准将会有助于问题的解决。

许多数据库系统已经被建立并原型化，且在不同程度上支持处理用户定义功能。作为一个典型的示例，IBM Almadeny研究中心开发的基于图像内容的查询（Query by Image Content，QBIC）工具使得用户可以依据诸如颜色、纹理、地块及对象位置等属性进行查询。在已构建的图像数据库中，该方法将对象识别为静止图片，将视频片断识别为称为快照的短序列，并计算与以上属性相关的特征。一些传感器数据库系统（例如，Cornell的COUGAR）将表示为关系的存储数据和表示为时间序列的传感器数据混合在一起。每个长期运行的传感器查询都会定义一个持久视图，该视图在给定时间间隔内保持不变。VDBMS视频数据库管理系统被设计用来支持关于视频的全方位功能，这里视频被看作是一种良好定义的抽象数据库数据类型。可视和语义描述（包括低层次可视特性，如色彩柱状图、纹理以及边界定位），表示和索引用于搜索的视频内容，会在视频处理过程中被抽取出来，并被存储在数据库中的独立表中，以便按视频内容进行搜索。

14.3　地理信息系统（GIS）[①]

地理信息系统（Geographic Information System，GIS[②]）通常被定义为用于收集、存储、显示、更新操纵以及分析空间数据的软件系统和硬件系统的集成。20世纪60年代和70年代，GIS是一种单机上应用的简单软件工具，用于解决相对简单的空间问题。随着时间的推移，GIS得到了扩展，它被看作是一种地理信息科学，而不仅仅是一种工具或系统。（Goodchild，1992）提出的一种概念是：用GIS技术处理空间信息，提出了一系列智能和科学上的挑战，它远不仅是“空间数据处理”，这意味着它实际上是地理信息科学。Goodchild讨论了解决GIS问题的过程中出现的一些不同的智能和科学

① 在此感谢Liora Sahar对本节的卓越贡献。

② 缩略语GIS可以根据上下文来表示一个系统或者多个系统。

问题：判定地图精确性、设计时间依赖的地理数据模型以及开发更好的转换工具和方法等问题。

GIS也可被视为综合了多个不同学术领域的交叉学科，这些领域包括测地学（Geodesy）（投影、测量、绘图法等）、遥感学、照相测量法、环境科学、城市规划、认知科学以及其他领域。作为结果，GIS依赖于以下领域的发展：计算机科学、数据库、统计学以及人工智能。所有这些由多学科综合引起的不同问题使得它不仅仅是一种简单的工具。

我们在8.3.1节中讨论了空间数据的特性。本节中我们的主题是描述作为数据库技术新的应用领域的GIS所带来的挑战。

处理地理信息带来一个基本问题：*为什么空间数据是与众不同的？*

地理/空间信息的特殊之处在于它描述的对象有着不同的位置，属性和相互间的联系是在一个已定义的空间中。地理的第一法则这样描述：*每个事物都与其他事物相关，但近处的事物比远处的事物相关性更强*。这一法则使得GIS系统能够进行基于对象间拓扑结构的分析。拓扑要求对地块之间的联系进行定义，如邻接、接近、连接以及其他联系。

地理信息还需要一种不同精度级别的数据源间集成的方法。由于该系统处理与我们日常生活相关方面的事务，它需要定期更新来保持与现实同步，并保持可靠性。GIS数据库中存储的用于实用的多数信息需要专门的检索和操作方法。GIS系统和应用处理的信息可被视为有确定意义和语境的数据，而不是简单的数据。

14.3.1　GIS 系统组件

GIS系统可视为3种组件的集成：硬件和软件、数据以及人。

- **硬件和软件**。硬件与终端用户使用的设备有关，如图形设备或绘图机和扫描仪。数据存储和操纵是通过使用一系列的处理器来完成的。随着Internet和Web应用的发展，Web服务器已经成为许多系统体系结构中的一部分，于是多数GIS遵循三层体系结构（见《数据库系统基础：初级篇》的图2-7）。软件部分与用于定义、存储、操纵数据的过程有关，因此与DBMS有些类似。为了提供存储、检索和操纵数据的有效方法，人们使用了不同的模型。
- **数据**。地理数据可以分成主要的两组：矢量数据和光栅数据。

  GIS的矢量层指的是用点、线、多边形表示的离散对象。线是由两个或多个点连接而成的，多边形是线的封闭集合。该层具有共享一个共同属性集的几何特性。层中的对象有着在它们之间定义的共有的拓扑（我们稍后会讨论拓扑）。将向量转化为光栅过程及其逆过程是许多GIS实现的一部分。

  光栅在二维中是一种连续单元格，等价于三维中的像素（立方单元）。光栅数据集按概念可分为离散型和连续型。在离散型光栅中，每个单元值被连接到某个单独的表的一个类。例子包括土壤类型、植被类型、土地适应性等。连续型光栅图像一般描述了某个空间中的连续现象，例如数字高度模型（DEM），其中每个像素就是一个高度值。与离散型光栅不同，连续型光栅没有相关的属性/分类表。

  矢量和光栅都有大量的数据源。矢量源包括数字地图、从图像测量中抽出的地块以及许多其他的源。典型的光栅源是航拍图像（扫描成数字的或本来就是数字的）、卫星图像（比如Landsat、Spot和Ikonos）以及扫描地图图像。
- **人**。人与开发GIS系统及收集数据的各个过程均相关。他们包括地图制作者和测量员（负责制作地图并测量地区和地理特征），以及系统用户（负责收集数据、上传数据到系统中、操纵系统和分析结果等）。

14.3.2　GIS 中数据的特征

地理数据有着独特的特征，使得数据建模比传统应用更为复杂。地理背景、拓扑关系以及其他

空间联系在概念上非常重要，因为它们与定义空间完整性规则相关。需要考虑的地理对象的一些方面如Friis-Christensen and Jensen（2001）所示。我们总结如下：

- **位置**。地块的空间位置是由确定参考系的坐标定义的。这些地块由点、线或多边形表示。这些地块的几何特征涉及空间的三维表示。
- **暂时性**。数据库模型应当考虑随时间变化的地块的存在与变化。这在动态数据（如土地分割）相关时尤为重要，因为我们需要表现实时、可靠的数据。
- **复杂空间特征**。地块由点、线、多边形和光栅等空间表示组成。复杂的表示允许联合，例如，三维对象与它的各个面上的不同多边形。
- **主题值**。对象的不同特征和质量可以被表示为属性。
- **模糊对象**。模糊用来处理对象位置和主题分类的不确定性。对象的位置用坐标表示，并且与误差度相关。主题方面的表达是通过将一个对象与一个有着确定性度量或确定性百分比的类相关联而进行的。县或地方自治机关总是处于一种动态的状态，包括变动的边界、道路、建筑物建设地点，所以没人能保证这些数据库在拓扑结构上百分之百地准确。
- **基于实体对基于域的数据**。世界可以被表示为一个离散实体集，如森林、河流、道路以及建筑物。这被称作基于实体的方法。基于域的方法将世界表示为连续函数，其中的属性在空间中变化。使用这种方法，某些自然现象如空气污染分布和地形可以得到最佳表达。
- **泛化**。泛化与对象尺度及细节的级别相关。对象可以是大尺度至小尺度的聚集，而相反的处理却非常有限。例如，县层可被聚集到州层，但是如果没有外部数据的话，相反的过程是无法实现的。像素辨析率为2米的图像可以用来生成像素辨析率为10米的图像，但是不加入任何细节用像素辨析率为10米的图像来创建像素辨析率为2米的图像，只能得到对原始图像的一个合成的分割而已。
- **角色**。数据模型内的对象根据会话域的不同，可能代表着不同角色。所以，角色是应用依赖的。
- **对象ID**。数据模型中对象应被唯一标识。此外，出于组织之间数据交换的需要，通用的对象ID是必要的。
- **数据质量**。数据质量指的是数据的可靠性和精确性，或者更一般地指数据有多好。定量的质量与空间精确性（对象位置误差）等可度量组件相关。定性的质量由不可度量组件组成，一般与整个数据集，而不是与某个确定对象相关。

GIS中的约束

约束是GIS中地理对象的一个重要方面。一般的完整性约束，如码、域、参照完整性和一般语义约束（见《数据库系统基础：初级篇》的5.2.4节和5.2.5节），不能获取地理信息的明显特征。可以用外码将一个县与某个州相关联，但是我们如何确定这个州边界内部的某个县的位置？专用于处理空间信息的约束可以划分为拓扑约束、语义约束及用户定义约束（Cockcroft，1997）。

- **拓扑完整性约束**。拓扑可用来处理地块以及它们之间的空间联系的行为。拓扑完整性约束在帮助减小误差和增进数据质量的插入及更新过程中使用。空间联系被定义为数据模型中的实体，并且是与特定的应用相关。当细分一块土地时，我们可以约束新的子块，使得它完全处于这块土地的边界内。
- **语义完整性约束**。语义处理空间地块的“意图”。语义完整性约束定义了数据库状态是否有效，依据应用于该地块的语义规则。例如，一块土地的边界不能跨越某建筑物的外廓线，两个建筑物不能共享同一条边。应用于道路（如单行线）的一些语义约束可能很难具体化。
- **用户定义完整性约束**。用户定义完整性约束是非空间DBMS的商业规则。例如，一幢房屋应当在消防栓的一定距离之内。
- **时态约束**。时态约束区分为准时的和持续的。准时的时态约束定义了在某个确定时间发生的事件，例如事故、地震、聚会等。持续的时态约束指的是一段时间内发生的变化的描述（Bradeur

et al.，2000）。例如，一座桥建成、维护、然后毁坏。于是，除了位置和地理变化外，数据还要有与时间相关的组件。

我们还需要考虑另一个约束，称作**泛化约束**（generalization constraint）。这个约束与系统内部的尺度层次和抽象层次相关。当一个特定层次中的数据上传至系统中时，它是满足不同约束的。当创建其他尺度层次时，这些约束如何变化或如何保持呢？例如，在大尺度（1:1 000）和小尺度（1:100K）中，一个建筑图形的显示会有所不同（甚至完全消失）。在某个层次中有效的约束，在其他层次中就未必是这样了。一些研究项目已开始着手解决这一问题，但是这依然是一项挑战，特别是当处理数据层次的自动创建和聚集时。

14.3.3　GIS 概念数据模型

本节主要讨论GIS中空间数据存储的一般概念模型。在地理系统中实现了不同的数据模型，GIS业界使用的流行模型有以下几种：

- **光栅数据模型**。光栅数据是一个单元阵列。其中每个单元值表示一个属性。元数据一般存储在文件头中，并且一般包括网格左上单元的地理坐标、单元大小以及行列数。这一模型一般用于分析应用，如建模、地图代数以及更为高级的地块抽取和聚类技术。
- **矢量数据模型**。矢量数据是一种离散对象表示方法，一般以点、线和多边形来表示。地块可以具有它们之间定义的一种联系（拓扑），并且可以包含一种行为。通过使用一个著名的已定义工具集来分析空间数据，并获取不同决策层次的信息，空间分析才得以完成。
- **网络模型**。网络是一种特殊类型的地块的拓扑模型，它允许流建模及资源和服务的传递。网络拓扑联系定义了线是如何通过点彼此相连的。这些规则存储在连通的表中。许多日常应用使用了网络模型，如优化校车路线、计划紧急路线、设计流水线系统以及创建运输模型等。
- **TIN数据模型**。三角不规则网络（Triangulated Irregular Network，TIN）用来创建和表达表面。在TIN结构中，表面由不重叠的三角形表示，三角形由不规则的空间点构成。点的密度由模型表面的地貌变化决定。TIN数据结构管理构成每个三角形及相邻三角形的点的信息。TIN模型有几个主要的优势。在DTM光栅模型中，每个像素都有一个高度值。一些值可以是原始测量值，而另一些可能是插值。在TIN模型中，由于没有对表达为特定像素或位置的表达值的限制，模型保留了原始测量点。点的密度不是固定的，而是随表面地貌的变化而变化。需要管理的点较少，使得存储、分析和管理更为有效。TIN模型被认为在计算高度、斜面以及点间的视线时更加容易。

14.3.4　GIS 的 DBMS 增强功能

直到20世纪90年代中期，地理信息系统还主要是基于私有的数据模型。这些模型一般是基于文件的，优化的，支持快速而有效地访问。这就带来了不同组织间共享数据的问题。传输标准还没有定义，这就在共享术语上限制了厂商。GIS模型演化到地理关系结构，而且属性数据存储于关系数据库中。空间特征存储于基于文件的系统中，并与属性相关联。尽管利用了RDBMS的优势，这个模型还是在可伸缩性上存在限定，并且不能充分发挥商业关系系统的优势（如备份和恢复、复制和安全等），因为大部分信息（空间数据）仍然存储于私有的基于文件的系统中。

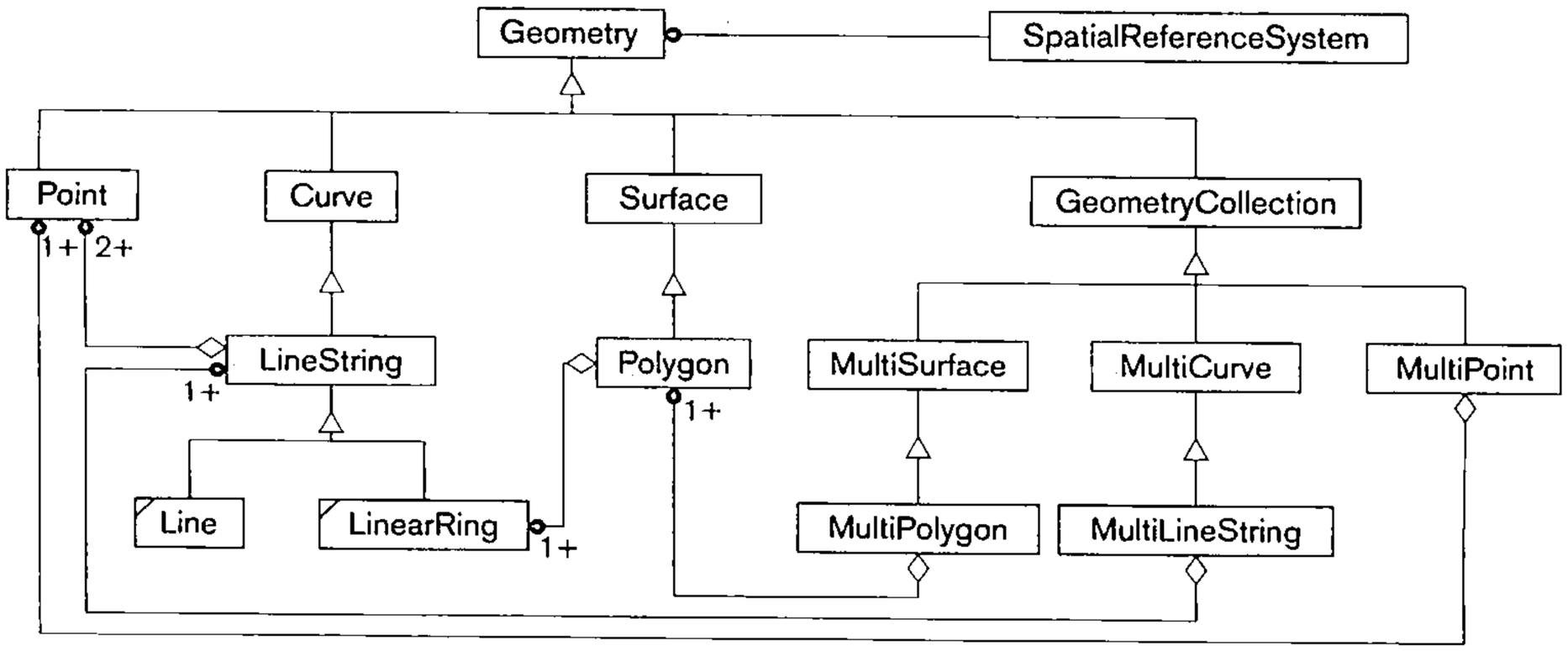
在20世纪90年代，基于对象的数据库出现了，这进一步凸显了RDBMS的缺点。简单的RDBMS不足以应对丰富的数据类型，例如地理类型，于是在处理GIS数据时经常会遇到困难。新的技术使得对象以及相关方法得以长期（或者临时）存储。OODBMS系统允许使用者定义复杂、自然的实体，并提供由OO方法（见第4章）派生出的概念和特征。OODBMS使得结构和非结构的数据[比如二进制大对象（BLOB）类型的卫星图像]得以定义和存储。可是尽管OODBMS有这么多的优势，它在市场上却并不成功。

下一步就涉及支持GIS应用的对象-关系数据库管理系统（ORDBMS）的设计。这些系统是适用于处理对象的关系数据库，允许存储同一系统中的对象的位置专门特征、它们的行为以及属性（一般定义为简单对象）。为了支持地理系统，这些ORDBMS必须创建一种查询语言，这一语言要支持确定的数据类型以及可以解释SQL查询的解析器。存储于GIS中的海量信息需要扩展的查询优化器、索引以及有效设计的、特别考虑了所需的复制服务的存储结构。查询时有时会需要很长的处理时间，需要处理大量地理记录的特殊事务服务。

在过去五年中，各提供商发布了若干数据库扩展版来扩展它们的ORDBMS[①]。Informix发布了Spatial and Geodetic Datablade（实现了R树索引），IBM发布了DB2 Spatial Extender、Oracle发布了OracleSpatial和OracleLocator选件。扩展版可以与GIS软件协同使用，实现空间分析、编缉和映射等复杂应用。ESRI的ArcSDE是一种GIS软件，可以用来访问存储在RDBMS中的大型多用户地理数据库。它增强了数据管理性能，扩展了数据库中的数据类型，实现了RDBMS间的模式移植。ArcSDE支持IBM的DB2、Informix、Oracle和Microsoft SQL。近期，一些提供商发布了应用服务器方案来支持更为先进的路由、映射和分析功能。Oracle的MapViewer就是一个这样的示例，它支持存储在OracleSpatial或OracleLocator中的信息映射。ESRI及IBM联合发布了一个空间解决方案，包括了DB2 Universal Database™ Spatial Extender，允许用户存储并操纵传统数据和空间数据。

14.3.5　GIS 标准和操作

设计数据库的一个基本步骤是理解系统需求。设计是基于系统的构建块（building block）的，构建块包括实体、方法和系统应当支持的约束。定义数据类型和方法的GIS组织内部对标准和策略的需求已经在几大组织中提到。为了使得空间数据共享，不同的标准化组织，如开放地球空间联盟（Open Geospatial Consortium，OGC™），美国联邦地理数据协会（Federal Geographic Data Committee，FGDC），以及国际标准化组织（International Organization for Standardization，ISO）已开始定义相关标准。1999年，OGC定义了GIS系统应当支持的核心数据类型（见图14-3）和方法。建议的模式被设计成可通过SQL进行访问。2003年，OGC定义了GML（Geography Markup Language，地理标识语言），这是一种用于地理信息的XML编码。它使得空间信息，包括地理地块的性质和几何学等，能够传送、建模和存储。GML中定义的对象支持地块的几何、拓扑、参考系统等的描述。



Courtesy Open GIS Consortium, 1999.

图14-3　几何学类层次

① 此处和其他处曾提到的一些产品都有商标——我们在本章的末尾列出了一些知名商标。

在OGC提供的几何分类层次（Geometry Class Hierarchy）模型中，Geometry是根类型。它包含一个已经定义了的空间参考系统（坐标系、投影），以及4个子类型：Point（点）、Curve（弧）、Surface（面）及GeometryCollection（几何集）。GeometryCollection是不同类型的几何图形的集合。GeometryCollection的子类型有MultiPoint、MultiLineString以及MultiPolygon，用于处理不同几何类型的聚合。Curve是一个一维几何对象，由多点构成的序列，LineString是其子类型。LineString指定了点之间的插值。图14-3中的符号表示法还指出了对于地块之间联系的最小基数约束(如，+1，+2)。

标准定义了一些用来测试几何对象间的空间关系的方法。

- **等价**（Equal）。一个几何图形是否在空间上等价于另一个？
- **分离**（Disjoint）。几何图形是否有公共点？
- **相交**（Intersect）。几何图形是否相交？
- **接触**（Touch）。几何图形是否空间接触（在边界线相交）？
- **交叉**（Cross）。几何图形是否空间交叉？是否重叠（可以有不同维度，例如线和多边形）？
- **内部**（Within）。一个几何图形是否在另一个内部空间？
- **包含**（Contain）。一个几何图形是否完全包含另一个？
- **重叠**（Overlap）。几何图形是否重叠（必须是同一维度的图形）？
- **相关**（Relate）。几何图形在空间上是否相关（通过测试两个几何图形的内部、边界、外部是否相交来检测）？

系统中几何图形空间分析由以下方法定义：

- **距离**（Distance）。返回两个几何图形中任意两点间的最短距离。
- **缓冲**（Buffer）。返回一个几何图形，它所有的点与给定几何图形的距离为小于或等于给定距离。
- **凸包**（ConvexHull）。返回一个几何图形，包括了原图形的凸包。凸包是一个最小的多边形，包含了一系列给定点的集合以及由这些点的集合构成的所有可能的线。
- **交**（Intersection）。返回一个几何图形，它包含了两个几何图形的相交点集。
- **并**（Union）。返回一个几何图形，它包含了两个几何图形的点的并集。
- **差**（Difference）。返回一个几何图形，它包含了两个几何图形的点的差集。
- **补差**（SymDifference）。返回一个几何图形，它包含了两个几何图形的点的补差集（分属于不同的输入几何图形，而并非公共点）。

标准还定义了不同几何图形的断言。例如，一个多边形被定义为一个平面，由一个外边界描述，也可能有几个内边界。每个内边界定义了多边形中的一个孔。对这样一个几何图形，断言应包括：

- 多边形是拓扑封闭的。
- 多边形的边界由一系列线环组成，分别定义了其内外边界。
- 边界内的任意两环不交叉。多边形内部的环可以在一点相交，但只能相切。

地块（feature）被定义为一个带有空间位置和几何属性的对象。地块在表中作为行存储，每个几何属性是参照一个几何表或视图的外码。地块间的联系定义为地块表间的一个外码参照。

标准中需要特别注意的一个重要约束是空间参照系统（或称为Datum，表示代表地球的椭圆体的大小、形状和原点）。数据库中的每个地块都在一行中存储，并必须有一列与它的空间参照系统相关联。空间参照系统确定了所有存储在列中的几何图形的坐标系。它定义了每个三维表面（经度/纬度）或一个特定的投影系统（投影是一种数学转换方法，将地球的三维形状转化为平面地图，并保持其固有的变形）中每个地块的特定位置。空间参照系统表存储了数据库中每个空间参照系统的信息。下面这个示例说明了一个states表的创建：

```
CREATE TABLE STATES
   (  Sname            VARCHAR(50)    NOT NULL,
      State_shape      POLYGON        NOT NULL,
      Country          VARCHAR(50),
   PRIMARY KEY (Sname),
   FOREIGN KEY (Country) REFERENCES COUNTRIES (Cname),
   );
```

这个语句定义了州名、州几何图形（多边形）以及国家。它定义主码为州名（不能为空），以及到国家表的一个外码。

下面是一个检索面积大于50 000的州的不同语句：

```
SELECT    Sname
FROM      STATES
WHERE     (AREA(State_shape) >50000);
```

Area是OGC标准（OGC1999）中定义的方法，它返回以坐标系中的单位度量的表面面积。下面的语句将会检索所有与Texas州共享一条边界的州。当几何图形空间接触时，Touches方法将返回1。

```
SELECT    S1.Sname
FROM      STATES S1, STATES S2
WHERE     ( (TOUCHES( S1.State_shape, S2.State_shape) == 1)
          AND
          (S2.State_name = 'Texas') )
```

空间数据传送标准（Spatial Data Transfer Standard，SDTS）已被美国采用并作为联邦信息处理标准（Federal Information Processing Standard，FIPS）出版，编号为173。SDTS是一种为空间数据的传输而设立的交换的、自包含的标准。SDTS支持矢量GIS地块、它们的属性、拓扑、元数据以及光栅数据集的交换。ESRI(ArcINFO)；LEICA(ErdasIMAGINE)，Intergraph以及MapInfo等公司已经在它们的产品中加入了支持SDTS标准的功能。

尽管在开发应用于存储、操纵和传送地理数据的标准方面已经投入了许多努力，GIS团体还没有完全采用GIS标准。不同的组织依然保存着它们旧的、有时还是基于文件的系统。在这种情况下，会发现在共享数据上很难实施规范。

14.3.6　GIS 应用和软件

由于GIS与我们周围的世界相关，因此有大量的基于GIS的应用。GIS最初起源于大学里的研究中心，被应用于一些专门组织及部门，如军方和政府。过去十年中，GIS扩展到房地产等其他领域，而在今天，GIS几乎与我们日常生活的方方面面相关。它们应用在大小公司的决策制定、健康部门、环境科学、城市规划、航海、运输、紧急服务、法律实施、人口普查跟踪等方面。普通人在网上查询驾驶方向或在车中使用基于GPS的导航系统时，也会与GIS技术打交道。

由于最终用户有着不同的需求，因此有各种各样的与GIS交互的产品和方法。专业需求可以通过软件来满足，如ESRI的ArcINFO，这个软件可以编辑数据，还有高级分析工具。简单的地图制作、报表和分析可以通过使用桌面GIS完成，如ESRI的ArcGIS。其他产品，如ESRI的ArcPAD，提供了地图制作、数据汇集、GIS以及为野外用户使用的移动轻型设备的GPS集成功能。如前所述，网络GIS使得可以进行显示、查询和地图制作。这一功能由ESRI ArcIMS、Intergraph GeoMedia Web Map以及MapInfo MapXtreme等软件提供。

GIS软件示例——ArcINFO和Geodatabase

ArcINFO和Geodatabase是ESRI（Environmental Systems Research Institute）公司的产品。ArcINFO最早是在70年代为微型计算机设计，后来发展到UNIX环境，今天它已经可以在多种平台上运行。ArcINFO中的数据存储在工作区中。每个工作区由多个相关的覆盖区、数据层以及INFO目录组成。

覆盖区（coverage）是ESRI用于存储矢量数据的专用格式。ArcINFO中实现的基本对象是点、线和多边形。INFO目录是GIS中应用的第一个健壮的RDBMS。INFO存储并操纵存储在基于文件系统中的不同几何图形的属性数据。

覆盖区格式允许用户在地块间建立拓扑关系。拓扑数据结构具有关于空间联系的内嵌信息。拓扑的定义继承于一些基本概念，例如，弧是两点间的一段曲线，曲线有中间点（顶点），而多边形是弧的集合。这些关系存储于以下属性表中：PAT（PointAttributeTable）、AAT（ArcAttributeTable）和PAT（PolygonAttributeTable）（PAT同时是多边形和点的缩写，因为多边形在系统中由一个标识点表示）。其他属性信息可以存储在不同的表中，也可以加入到这些表中。为了允许快速分析操作，层的拓扑可以用Clean和Build等命令创建。

ArcINFO由几个允许数据操纵的模块构成。ARC是主模块。利用ARC，用户可以使用Clean和Build命令操作覆盖区拓扑。Clean可以检查拓扑规则的错误，例如不确定的弧、邻近不重叠的点等。ARC提供了空间分析功能（矢量层的覆盖、交、并、拆分、缓冲等）。ARC还提供数据输入和输出功能，以及其他不同的功能，例如投影定义。AML（Arc Macro Languagc）模块允许用户编写结合其他arcINFO模块命令的脚本。ArcEdit模块用来进行数据的数字化和编辑，ArcPlot模块用来创建复杂的地图。GRID模块可用于光栅分析。这一分析是有限的，不允许进行如边缘检测、聚集等的高级图像处理。还有可以实现路由（找到最短路径……）、地理编码、分配等功能的Network模块，以及Image Integration模块。ArcINFO的DBMS驻留在INFO目录中，并且Table模块可用于查询数据。主要的查询函数有Select/Reselect/Aselect及Nselect。Select用来声明用户将要查询的表。Reselect等价于SQL中的Select。Aselect用于在前一个使用不同查询语句的选择中添加行，Nselect用于逆向选择。

Geodatabase是由ESRI定义的数据模型（对于arcGIS产品和应用），用来存储、检索及操纵地理数据。Geodatabase通过允许用户管理关系数据模型中的地理数据改进了用户的工作环境，它实施业务规则，并在数据类型之间建立拓扑和联系。这个模型使得用户可以在关系数据库中存储复杂对象。不同于ArcINFO将几何图形存储在基于文件的系统中，Geodatabase将几何图形存储在数据库表中。Geodatabase支持面向对象的矢量数据模型，并按照对象层次组织地理数据：对象类（非空间的）、地块类（空间的）以及地块数据集（使用相同空间参照的地块类）。在Geodatabase中的地块有一种几何形状并与其他地块相联系。地块的形状存储在称作geometry的列中，可用点（或多点）、多线和多边形来表示。属性和行为可附加到地块上，如子类型、完整性和拓扑规则。例如路层可以分解为高速公路、主路和州内公路，而且高速公路必须包含最小数目的车道。除此之外，还可以在地块间定义拓扑结构，而且拓扑规则也可以被执行。例如，建筑的地块线不能与道路相一致，州的多边形也不能重叠。

Geodatabase允许设计个人数据库，通过把Microsoft Office Access作为数据库使之支持多个浏览者和一个管理员。多用户数据库可以结合数据库一起使用，例如Oracle与ArcSDE软件结合可用于编辑和模式管理。

14.3.7　GIS 未来的工作

近十年来GIS领域发展很快。数据库技术的发展以及新的学科对GIS兴趣的增长，已产生了新的问题和困难。新的应用会继续提出新的挑战。下面就是其中的一些挑战。

- **数据源**。GIS领域引入汇集数据的新方法的频率越来越高。新的卫星、航拍以及GPS必须快速而有效地整合到GIS系统中。一个最近的示例是LIDAR卫星云点，它包含了要处理的海量数据。融合不同精度的新技术是GIS团体一直以来面临的挑战。许多由最近的数据源引入的大量信息是一个主要的数据库障碍。数据源的多样性导致了在集成方面的挑战。
- **数据模型**。两种经常使用的数据模型是矢量模型和光栅模型。一个主要的应用问题是现在的GIS系统能处理的模型只有一个。合并光栅和矢量数据受到了限制，完全整合这两种模型依然是一个难题。

- **标准**。GIS团体需要执行相关的全球标准或者至少是国家标准。推荐的标准中，ISO、OGC和FDGC应当得到私人和公共部门的认可、考虑及使用。这会使全世界的不同组织可以更好地共享信息。国际标准化组织（ISO TC 211）和欧洲标准化组织（CEN TC278）目前正在争论一些相关的问题，其中包括为实现快速有效的查询而进行的矢量数据和光栅数据的相互转换。
- **新的体系结构**。GIS应用需要新的客户-服务器体系结构，这得益于现有的RDBMS和ODBMS，以及扩展的关系数据库技术的发展。以前的系统将空间数据与非空间数据分离，并完全用DBMS来管理非空间数据。这个过程要求有适当的建模和集成技术，因为两种数据都在演变。Geodatabase似乎更为重视这个问题，表现在对arcINFO的改进。但目前还没有合适的工具来进行数据转换、变化管理及工作流管理。
- **版本和对象生命周期法**。由于不断变化的地理地块，GIS必须维护详尽的制图和地形数据。通过增量更新以及更新不同级别用户的授权模式，可以缓解由此带来的管理问题。对象生命周期法涉及对象的创建、注销，以及修改对象版本将其升级为永久对象等活动，这需要预定义一个完整的方法集来控制GIS对象的上述活动。
- **移动GIS**。携带含有地理信息的移动设备正变得越来越普遍。在路上获取在线更新和查询数据库会成为一个主要的应用领域。随着支持GPS的移动电话的出现，基于移动电话位置的应用也会很快出现。移动GIS会成为非常重要的研究课题，将会获得丰厚的回报。
- **时态GIS**。环绕我们的地理环境的改变总是会发生的。经常会建立人造地块，而其他地理地块可能会消失。需要记录许多这样的变化。不能保持最新的、当前的信息的GIS就会被淘汰。所有人造的变化，如公路、建筑、水坝以及水路，需要在最短的时间内更新。
- **GIS的多方面建模**。将来，具有许多功能的系统，从市场分析到车辆导航，将会需要面向边界（boundary-oriented）的数据和功能。从另一方面说，环境科学、水文学以及农业领域的应用需要更多面向区域的地形模型数据。这些功能是否可用一个简单而通用的GIS来实现还不是很清楚。GIS的特殊应用需求要求在支持完整的GIS应用之前，必须为普通的DBMS添加附加的数据类型和功能。
- **通用表示法**。GIS中的工具和系统应该是直观的，并且具有容易使用的表示法。由于ISO/TC 211和OGC采用UML为空间类建模，可以处理地理对象的UML的扩展也应当被考虑。
- **泛化**。GIS系统在不同尺度和精确度级别上存储数据。当用户从数据库中检索信息时，就需要一定级别的尺度和精确度，并且有可能与数据库中的原始存储尺度不同。例如更新某一层以及自动生成其他级别等问题应当引起重视。
- **面向GIS的专门DBMS**。数据库技术在不断发展，GIS团体必须整合DBMS提供的优势，例如复杂查询处理和优化、安全、备份、恢复以及在某些情况下的故障恢复机制。尽管几家GIS供应商试图紧跟数据库组织，用户可能在某些产品上工作了很长时间，并且不愿意在新的环境中学习和工作。数据库中的精确度级别如何保持、不同数据模型的分析过程如何结合等问题，都需要做更多的工作。缺乏了解GIS特殊需求的数据库管理员或许会成为一个障碍，使得许多公共和私人部门GIS组织无法实施使用标准DBMS的GIS。

14.4　基因组数据管理[①]

14.4.1　生物科学和遗传学

生物科学包含着庞大的多种多样的信息。环境科学主要研究自然环境中的生物是如何生存和互

① 感谢Ying Liu、Nalini Polavarapu及Saurav Sahay对本节的贡献。

动的。生物学和生态学研究特定的物种。解剖学集中研究生物的整体结构，记录单个个体的生理特征。传统的医药和生理学将生物体分解为系统和组织，并尽可能地汇集关于系统和组织工作的信息。组织学和细胞生物学在组织和细胞的层次上进行研究，提供关于细胞的内部结构和功能的知识。由此产生的信息经过分类、存储，历经数个世纪的积累，直到最近才成为数据库技术的一个主要应用领域。

遗传学（genetic）已成为信息技术应用的一个理想领域。广义上说，遗传学可以看作是一个数据模型构造，它基于基因（基因是遗传的基本单元）的信息，以及对信息间联系的确定。遗传学研究有3个分支：孟德尔（Mendelian）遗传学、分子遗传学及种群遗传学。孟德尔遗传学主要研究遗传特征在不同代之间的传递。分子遗传学在分子层次上研究基因的化学结构和功能。种群遗传学主要研究基因信息如何在种群间变化。

通过研究人员对基因的组成、结构和功能进行研究，分子遗传学提供了详细的关于基因的信息。分子遗传学的研究始于两个重要的发现。第一个是1869年Friedrich Miescher发现的核蛋白及其主要组成成分——脱氧核糖核酸（DNA）。在之后的研究中，人们又发现DNA和RNA是由核苷酸（包括糖、磷酸盐、碱基，它们合成核酸）组成的，核苷酸是通过糖和磷酸盐连接成的聚合体。第二个发现是在1944年，Oswald Avery研究发现DNA正是携带有遗传信息的分子物质。因此，基因是由线性排列在染色体中的核酸链组成的，主要有3个方面的功能：(1) 在两代之间复制遗传信息；(2) 提供生成多呔的蓝图；(3) 积累变化从而发生进化。1953年，Watson和Crick发现了DNA的双螺旋结构，这给分子遗传学研究提出了一个新的研究方向[①]。DNA及其结构的发现被认为是过去100年中最重要的生物学研究成果，它开创的新的研究领域也是未来100年的科学研究的前沿。1962年，Watson、Click和Wilkins也因此突破性的研究成果而获得了诺贝尔生理学/医学奖[②]。

14.4.2　生物数据的特征

生物数据表现出很多不同特征，这给生物信息的管理带来了很大困难。例如，现代实验技术提供了在一次实验中收集大量生物数据的能力。20世纪90年代后期的高密度cDNA或低核苷酸微阵列的开发，为大规模基因表达的监视带来了新的敏感度和种别级别。一个DNA微阵列（基因芯片）实验会有数以千计的基因同时产生数据，使研究人员很难解读数据并形成关于基因功能的结论。这些微阵列是两维矩阵，每个元素记载了某次实验中某个基因的表达级别。目标是以过度表达或者不足表达（overexpression or underexpression）的形式，寻找在样本和控制数据之间变化最明显的基因。

我们首先总结生物信息相关的特征，然后集中讨论一个新出现的称为**生物信息学**（bioinformatic）的交叉学科，现在生物信息学在一些大学中被作为研究生学位项目来研究。生物信息学主要研究关于DNA和蛋白质序列的分析，以实现基因信息的管理。管理所有类型的生物信息以及这些信息的建模、存储、检索和管理需要扩展到一个更大的范围。并且，生物信息学的应用包括药物靶位设计、变异及相关疾病研究、种族迁移模式的人类学调查及治疗处理等。

特征1。与其他大多数领域或应用的数据相比，生物数据具有高度复杂性。生物数据的定义必须能够表示复杂的数据子结构及其联系，并且要保证在生物数据建模过程中没有信息丢失。生物数据结构常常为信息的解释提供了附加的语境信息。生物信息系统必须能够描述任何数据模式、联系或模式子结构，而且不仅是层次的、二进制的或表数据中的任意级别上的复杂性。例如，MITOMAP是记录人类线粒体基因组的数据库[③]。

单个基因组是微小的圆形DNA片段，包含16 569个核苷基信息、52个基因位置编码信使RNA和

① 参见Naturc, 171:737 1953。

② http://www.pbs.org/wgbh/aso/databank/entries/do53dn.html。

③ MITOMAP的详细信息及其信息复杂性参见（Kogelnik et al.，1997，1998）、（Brandon et al.，2005）和http://www.mitomap.org。

核糖体RNA及转移RNA、1 500多个已知的种群变量、60多个已知的疾病结合体，以及一个关于产生氧化磷酸化路径的生物化学能量的复杂分子交互作用的有限知识集合。在这个方面有超过3 000篇的参考文献。正如预料的那样，对其进行管理存在大量的问题，我们不能使用传统的RDBMS或ODBMS方法来收集所有方面的信息。

特征2。数据变化的次数多，变化范围大。因此，生物信息系统必须能够灵活处理不同的数据类型及数据值。由于数据值具有如此大的变化范围，数据类型上的约束应该是有限的，因为这可能会把没有预料到的数据排除在外，例如溢出值，而这样的数据在生物学领域内却是常见的。排除这些数据将会导致有用信息的缺失。此外，生物数据结构频繁出现异常可能要求选择对给定的数据片段有效的数据类型。

特征3。生物数据库中的模式变化很快。因此，为了改善两代种群或数据库版本之间的信息流，必须支持模式的进化和数据对象的移植。大多数关系数据库和对象数据库系统并不支持模式扩展和生物设置中的频发事件。目前的系统，如GenBank，每年发布一到两次新模式的完整数据库，而不是在必须修改的时候才增加修改系统。这样经过改进的数据库会提供及时和有序的机制来跟踪生物数据库中随时间变化的单个数据实体。这类跟踪对于生物学研究人员访问、重现以前的结果是很重要的。

特征4。不同生物学家描述同一数据可能会采用不同的方法（甚至是在同一个系统内）。因此，系统应支持不同生物数据模式间，或模式的不同版本间的调整机制。由于生物数据的复杂性，任何实体都存在多种建模方式，结果会反映科学家的特殊兴趣。如果要求解释同一个实体，两个个体可能产生不同的数据模型，这两个模型可能会有大量的共同点。在这种情况下，能够在这些共同点之间进行查询对于生物调查人员是很有用的。通过连接模式网络中的数据元素可以实现上述查询。

特征5。大多数生物数据的用户并不需要写访问数据库，只读访问就足够了。只有称为监护员的特权用户才具有写访问的权限。例如，作为MITOMAP项目的一部分而创建的数据库，每月平均有15 000多位Internet用户访问。而每个月由非监护员提交的写访问任务不足20个。换句话说，需要写访问的用户数是很少的。用户产生大量的针对数据库的读访问模式，但是这些模式和传统关系数据库中看到的模式不同。用户请求的特定搜索要求对通常未预料的数据实例类组合建立索引。

特征6。大部分生物学家不可能有数据库的内部结构或模式设计的任何知识。生物数据库界面应该以如下形式把信息展示给用户：应用于用户关注的问题并以一种容易理解的形式反映底层的数据结构。生物数据库用户通常了解他们所需要的数据，但是他们没有数据结构或DBMS如何表示数据的技术知识。他们要依赖技术用户给他们提供数据库视图。关系模式不能为用户提供有关用户模式含义的提示或任何指导性的信息。Web界面通常提供了预设的搜索界面，这可能会限制对数据库的访问。然而，如果直接从数据库结构中生成这些用户界面，则可能产生更大可能范围的访问操作，尽管有可能并不能保证最大的可用性。

特征7。数据环境为数据在生物应用中的使用提供附加的信息。因此，必要时要对环境信息进行维护并提供给用户。此外，应该可以整合尽可能多的环境信息以充分解释生物数据的值。生物系统中独立的值并没有太大的意义。例如，在缺乏描述关于其组织、功能等信息的情况下，DNA序列并没有特别的意义。但是，在非致病DNA链环境中，DNA链上的单个核苷可能会成为导致镰状红细胞贫血的重要原因。

特征8。对复杂查询的定义及表示对生物学家而言非常重要。因此，生物数据库系统必须支持复杂的查询。在不了解任何数据结构的情况下（见特征6），普通用户无法通过他们自己的数据集合构造复杂查询。因此，为了真正使之有用，系统必须提供一些工具以支持构造复杂查询。如前所述，很多系统为用户提供了预定义的查询模板。

特征9。生物信息的用户经常需要访问“旧”的数据值，尤其是在对以前的报告结果进行验证的时候。因此，必须通过一个档案系统支持数据库中数据值的变化。在生物学领域内，对最新数据值

和以前数据值的访问同等重要。研究人员一般会查询最新数据，但是他们也必须能够重新构造先前的工作，并对先前和当前信息重新评估。所以，生物数据库中要更新的值不能只是简单地丢掉。

以上所有特征明确说明了一个事实，就是目前的DBMS还不能充分满足复杂的生物数据的需求。必须要开拓数据库管理系统的一个新方向[①]。

14.4.3　人类基因组项目和现有的生物数据库

基因组（genome）是指从一个生物体上得到的全部遗传信息。例如，人类基因组（human genome）通常是指创建人类生命所需的所有的基因集合，在23对染色体上分布着大约超过25 000个的基因，具有30亿~ 40亿个核苷。人类基因组项目（Human Genome Project，HGP）的目标是获得完整的碱基序列。完整的人类基因组序列草案已经于2000年6月公布，并于2001年印刷发行[②]。

人类基因组已经完成，而且有99.9%的正确率。单独来看，人类DNA序列并没有特别的用处。然而把序列和其他数据结合起来，它就是一个强有力的工具，可用来帮助人们解决遗传学、生物化学、医药学、人类学以及农业领域的很多问题。目前的基因组数据库，研究主要集中在监护（或进行一些初步观测和质量检验）以及对基因组序列数据信息进行分类上。除了人类基因组数据库外，人们还在研究一些生物体的基因组，如E.coli、Drosophila以及C.elegans等。下面简单讨论几个现有的数据库系统和信息系统，这些系统或者支持人类基因组项目或者就是从该项目中产生的。

1．GenBank

今天，世界上卓越的DNA序列数据库就是GenBank，它由美国国家医药图书馆（National Library of Medicine，NLM）的国家生物技术信息中心（National Center for Biotechnology Information，NCBI）负责维护，这个中心在1978年建立，作为DNA序列数据的中心资料库。从那时起，它就不断扩展，进而包括了表示序列标记数据、蛋白质序列数据、三维蛋白质结构、分类学，并与生物文献库（medline）连接。到2005年8月，该数据库中的序列数据已经超过了100GB，主要的数据包括超过16 500个物种的单独基因以及部分或完整基因组数据。通过与英国欧洲分子生物实验室（European Molecular Biology Laboratory，EMBL）以及日本DNA数据银行（DNA Data Bank of Japan，DDBJ）的国际合作，数据每天可在这3个站点之间进行交换。这3个站点上序列数据的镜像，可以满足世界各地生物领域的科学家们快速访问这些数据的需要。

虽然是一个复杂的综合性的数据库，但是GenBank的范围主要集中在人类及其他物种的基因序列以及相关文献的链接上。其他有限的数据源（例如，三维结构和OMIM，将在下面讨论）最近已经添加到GenBank中，这需要对现有的OMIM和PDB数据库重新格式化，并重新设计GenBank的结构以适应这些新的数据集合。

GenBank系统支持平面文件、关系数据库，以及包含**抽象语法表示法1**（Abstract Syntax Notation One，ASN.1）的文件的结合，ASN.1是电信行业中定义数据结构的一种语法。NCBI给每个GenBank条目指派了一个唯一的标识符。更新时要赋以新的标识符，原实体的标识符保持不变，以方便归档。因此，对实体旧的引用会有意指出新值和可能不合适的值。最新的概念还有另外一组唯一标识符（unique identifier，UID），它标志着概念的最新形式，并允许通过实体的原标识符访问它的旧版本。

虽然系统提供完整的数据库快照，并支持多种格式的数据库输出，包括ASN.1格式，但是一般数据库用户并不能直接访问数据的结构来执行查询或其他功能。系统提供查询工具Entrez（或其Web版本），它支持通过静态接口对关键字、序列以及GenBank的UID进行搜索。

2．基因组数据库（Genome Database，GDB）

GDB建于1989年，它是一个人类基因数据的映射目录，是把每条信息和人类基因组的某个特定

---

① 更详细的信息参见（Kogelinik et al.，1997，1998）和（Brandon et al.，2005）。

② 详见（Lander，E.S.et.al.，2001）。

位置联系起来的一个过程。映射的精确程度取决于数据的来源，但是一般不会精确到单个核苷基。GDB数据包括描述主要映射信息的数据（距离和置信度限定）、聚合酶链反应（Polymerase Chain Reaction，PCR）探针数据（实验条件、PCR引物以及所用试剂）。最近的成果是在GDB又添加了以下一些数据：与遗传轨迹相关的变异数据、实验细胞链数据、DNA探针库以及有限的多态和种群数据。

GDB的建立基于SYBASE，这是一个商用关系型DBMS，使用标准的实体-联系模型（见《数据库系统基础：初级篇》的第3章和第4章）进行数据建模。GDB的实现者已经注意到了采用这个模型获取稍微复杂的映射和探针数据的难度。为提高数据的完整性，简化应用程序员的程序开发工作，GDB发布了一个数据库访问工具包（Database Access Toolkit）。然而，大多数用户是通过Web界面对这十个互连的数据库管理器进行搜索。每个管理器对GDB系统内每个表的链接（联系）进行跟踪。如同GenBank那样，用户在搜索时只能得到数据的高层视图，并因此不能很轻松地从GDB表结构中获得任何知识。搜索方法只有当用户简单地查找映射或探针数据的索引时才非常有用。目前的接口不支持对数据库的探测性的特定搜索。GDB的数据库结构和OMIM（见下面）的集成还从来没有完全进行过。

3．在线人类孟德尔遗传数据库（Online Mendelian Inheritance in Man，OMIM）

OMIM是关于人类疾病遗传基本信息的电子摘要。Victor McCusick于1966年以硬复制（hard copy）的格式创建了该系统，开始时只有1 500个条目，1987年至1989年期间，GDB将其转换成全文本电子格式。1991年，Johns Hopkins大学把其管理权转交给了NCBI，整个数据库就转换成了NCBI的GenBank格式。目前，OMIM共包括16 000多个条目。

OMIM包含了基于器官与系统的5大疾病领域的资料。正在研究的形态学、生物化学、行为学以及其他特性被看作是个体（或细胞）的**显型**（phenotype）。Mendel发现基因可以以多种形态存在，称之为**等位基因**（allele）。**基因型**（genotype）是指个体的实际等位基因的组成。

显型和基因型条目的结构包括松散结构的文本数据，如一般描述、术语、遗传类型、变异、基因结构、映射和数值小分类等。当OMIM被移交给NCBI管理后，全文本的条目被转换为ASN.1结构化的形式。这大大提高了把OMIM数据链接到其他数据库系统的性能，同时给这些数据提供了精确的结构。然而，数据库的基本模式仍然难以进行修改。

4．大肠杆菌基因和代谢百科全书（Encyclopedia of Escherichia coli Genes and Metabolism，EcoCyc）

EcoCyc是最近开展的一项实验，它把关于基因组和E.coli K-12的新陈代谢的信息结合起来。EcoCyc数据库作为斯坦福研究所和海洋生物实验室的一项合作，创建于1996年。它主要对以下信息进行了分类和描述：已知的E.coli基因，这些基因编码的酶，以及每个酶催化的生化反应，还有它们的组织的代谢路径。通过这样的工作，EcoCyc扩展了基因信息的序列和功能范围。到2005年5月，EcoCyc中的信息已发布了10 747项。

实现EcoCyc系统最初使用的是面向对象的数据模型，数据存储在Ocelot中，Ocelot是一个框架知识表示系统。EcoCyc数据基于以下观察并根据对象类的层次结构进行排列：反应的特性独立于催化反应的酶，酶具有多种逻辑上区别于它的反应的性质。

EcoCyc提供两种查询方式：(1) 直接查询（通过预定义查询），(2) 间接查询（通过超文本导航）。直接查询通过使用菜单和对话框来启动大的但是有限的查询集合的执行，它不支持实际数据结构的导航。另外，也没有文档模式发展的机制。

表14-1总结了主要的与基因组相关的数据库，以及HGMDB和ACEDB数据库的特征。目前还有一些其他的蛋白质数据库，包含关于蛋白质结构的信息。比较著名的蛋白质数据库包括日内瓦大学的SWISS-PROT、Brookhaven国家实验室的蛋白质数据银行（Protein Data Bank，PDB），还有美国国家生物研究基金会的蛋白质识别资源库（Protein Identification Resource，PIR）。

表14-1　主要基因组相关数据库的总结

数据库名称	主要内容	初始技术	当前技术	DB问题范围	主要数据类型
GenBank	DNA/RNA序列、蛋白质	文本文件	平面文件/ASN.1	模式浏览、模式进化和其他数据库系统的链接	文本、数值和一些复杂类型
OMIM	疾病显型和基因显型等	索引卡/文本文件	平面文件/ASN.1	链接到其他数据库的非结构化、自由文本条目	文本
GDB	基因映射链接数据	平面文件	关系型的	模式扩展/进化、复杂对象和到其他数据库的链接	文本、数值
ACEDB	基因映射链接数据、序列数据（非人类）	面向对象	面向对象	模式扩展/进化，到其他数据库的链接	文本、数值
HGMDB	序列和序列变种	应用专用的平面文件	应用专用的平面文件	模式扩展/进化，到其他数据库的链接	文本
EcoCyc	生化反应和路径	面向对象	面向对象	类层次锁，模式进化	复杂类型、文本、数值

过去十年中，对于生物和医药领域数据库应用的关注正在不断增长。到目前为止，已经建立了GenBank、GDB、OMIM等作为特定生物数据类型的中心数据库。这些数据库虽然非常有用，但是还没有包括人类基因组项目数据的所有领域。不过，全世界都在努力设计新的工具和技术，以解决生物学家和医药研究人员所面临的数据管理难题。

5．基因本体论（Gene Ontology，GO）

我们已经在14.2.3节的多媒体信息建模语境中讨论了本体论的概念。基因本体论协会成立于1998年，它包括3个模型有机体数据库：FlyBase、Mouse Genome Informatics（MGI）和Saccharomyces or yeast Genome Database (SGD)。协会的目标是创建一个结构化的、精确定义的、公共的、描述基因角色的控制词汇表和任何有机体中的基因产物。随着多个物种基因组序列的完成，有机体之间的基因大片段显示出生物角色的相似性，生物学家普遍认为，大多数或所有的活细胞中保存的基因和蛋白质可能是单个整体（universe）。另一方面，基因组数据正在按指数增长，目前还没有一个统一的方法来解释和概念化共享的生物元素。通过使用基于共享生物属性和基因数据库之间的互操作性的公共词汇表，使得基因本体论对基因产物进行注释成为可能。

GO协会已经发展了3个组织原则：分子功能、生物过程和细胞成分，它们用来描述基因、基因产物或基因产物组的属性。分子功能定义为基因产物的生物化学活动；生物过程是指基因或基因产品中的生物对象；细胞成分是指基因产物在细胞中活动的位置。每个本体论包括一组良好定义的术语和联系词汇表。术语以有向的无环图（directed acyclic graph，DAG）形式来组织，其中每个术语结点可能有多个父亲和多个孩子。孩子可以是一个父亲的实例或者其父亲的一部分。在GO数据库的最新版本中，有20 000多个术语，术语之间有25 000多种联系。基因产物的注释由合作数据库中的每个数据库独立操作。注释的子集包含在GO数据库中，它包括1 386 000个基因产物和5 244 000个基因产物与GO术语之间的关联。

基因本体论是使用MySQL实现的，MySQL是一个开放源码的关系型DBMS。每月的数据库发布版本包括SQL和XML两种形式。还有一些用C、Java、Perl和XML等编写的工具和库集合，可用来进行数据库访问和应用开发。GO协会还提供一些基于Web的和基于控制台的GO浏览器。另一项在生物医药学和健康领域构建本体论的研究成果是美国国家医学图书馆的统一医学语言系统（Unified Medical Language System，UMLS）。UMLS包含3个组件：元词典或者词汇数据库，语义网络将词汇数据库中的概念分类并联接，专家词典包含了相关概念的专业词典信息。斯坦福大学的开放生物医学本体论计划（Open Biomedical Ontologies Project）是另一个大的贡献，它集成并提供访问跨不同生

物学和医学领域的共享词汇的能力。这些知识源将会被用于生物医学信息系统的语义标识。

6．基因表达选集(Gene Expression Omnibus，GEO)

GEO是一个公共存储，存储了科学协会提交的高产量基因表达数据。它由美国国家生物技术中心（NCBI）维护。GEO从基于微阵列的实验获取数据，这些实验测量了mRNA、染色体DNA以及蛋白质分子的相对级别。它的数据也可以来自基于非阵列的技术，如基因表达的序列分析（serial analysis of gene expression，SAGE）和大规模光谱分析蛋白体学技术（mass spectrometry proteomic technology）。到2006年1月止，已经有来自超过100个物种的大约十亿个基因表达测量结果的69 000多个样本数据，涉及了广泛的生物学问题。

参考文献

1．移动数据库参考文献

最近十年来，移动数据库的研究取得了很大进展，人们对于移动计算的关注与日俱增。在众多选读文献中，（Dhawan，1997）是一个很好的关于移动计算的文献。（Holtzman & Goodman，1993）讨论了无线网络和它的将来。（Jing et al.，1999）讨论了移动环境中的信息访问问题。（Imielinski & Badrinath，1994）提供了一个关于移动数据库的详细报告，并讨论了（Imielinski & Badrinath，1992）中提到的移动体系结构中的数据和元数据的分配问题。（Dunham & Helal，1995）探讨了移动数据库的查询处理、数据分布和事务管理问题。（Barbara，1999）是一个最新的关于移动数据管理的研究，涉及数据分发、一致性和位置依赖查询问题。（Pitoura & Samaras，1998）探讨了移动数据库存在的问题及其解决方案。（Bertino et al.，1998）讨论了移动数据库中容错及恢复的方法。（Pitoura et al.，1995）提议用聚合模型来处理传输处理中的断线问题。（Preguica et al.，2003）调查了用于移动数据库的基于预约的传输。（Milojicic et al.，2002）提出了一个有关点对点计算的指南。（Corson & Macker et.al.，1999）是对（IETF，1999）报告的回应，后者讨论了移动专用网络协议的性能问题。（Stoica et al.，2001）以及（Ratnasamy et al.，2001）讨论了基于分布散列表的结构化点对点网络。（Acharya et al.，1995）讨论了广播调度问题，这可以最小化平均查询等待时间，并把这些调度应用到最佳客户缓冲策略中。（Prasad et al.，2004）讨论了开发MANET应用的中间件平台。（Xie et al.，2003）以及（Xie，2005）提出了一种新的支持MANET应用中事务处理的事务模型和新技术。（Yee et al.，2002）讨论了广播（或推进）数据，它是一种大规模分发信息给用户的方法。（Chintalapati et al.，1997）提出了一种适应性的位置管理算法。（Jensen et al.，2001）讨论了属于基于位置服务的数据管理。（Wolfson，2001）描述了一种使用轨线而不是使用点来描述对象位置移动性的有效建模的新方法。（Mahajan et al.，1998）最初讨论了ISDB的扩展性问题，以及一种数据聚集和客户分组的方法。（Yee et al.，2001）讨论了ISDB应用中用于数据分组的特定集合算法。（Gray et al.，1993）讨论了ISDB同步中更新冲突及其解决方案技术。（Breibart et al.，1999）对应用于复制数据的延迟同步算法作了详细描述。

2．多媒体数据库参考文献

多媒体数据库管理正在成为一个重点研究领域，有一些工业项目正在按此途径进行。（Grosky，1994，1997）是两本优秀的多媒体数据库管理教程。（Pazandak & Srivastava，1995）对与多媒体数据库需求相关的数据库系统进行了评价。（Grosky et al.，1997）贡献了一些好文章，包括（Jagadish，1997）的基于内容的索引与检索概述。（Faloutsos et al.，1994）也讨论了基于内容的图像查询系统。（Li et al.，1998）介绍了图像建模，其中，图像被看作层次结构化的复杂对象，具有语义属性和可视属性。（Nwosu et al.，1996）和（Subramanian & Jajodia，1996）属于同样主题的书籍。（Flickner et al.，1995）描述了在IBM开发的QBIC系统。（Bonnet et al.，2001）描述了基于传感器的COUGAR系统。（Aref et al.，2004）描述了VDBMS视频数据库系统。（Lassila，1998）讨论了在Web上访问多媒体信

息的元数据需求。（Fensel，2000）总结了语义网方面的成果。（Khan，2000）讨论了基于本体论的信息检索。（Uschold & Gruninger，1996）提供了关于本体论的很多好资源。（Corcho et al.，2003）比较了各种本体论语言并讨论了建立本体论的方法学。（Dimitrova，1999）讨论了多媒体内容分析、索引与过滤。（Yoshitaka & Ichikawa，1999）对基于内容的多媒体检索进行了概述。更多信息可以参考以下站点。

- CA-JASMINE（多媒体ODBMS）：http://www.cai.com/products/jasmine.htm
- Excalibur技术：http://www.excalib.com
- Virage公司（基于内容的图像检索）：http://www.virage.com
- IBM的QBIC（按图像内容查询）产品：http://www.ibm.com

3. GIS参考文献

有很多关于GIS的书籍。（Adam & Gangopadhyay，1997）与（Laurini & Thompson，1992）主要讨论了GIS数据库和信息管理问题。（Kemp，1993）概括介绍了GIS以及数据源问题。（Huxhold，1991）介绍了Urban GIS。（Demers，2002）是另一本关于GIS基本原理的一本书。（Maguire et al.，1991）是一本非常好的关于GIS的论文集。（Antenucci，1998）对GIS技术进行了讨论。（Longley et al.，2001）讨论了GIS作为一门科学的地位，以及GIS作为交叉学科的方方面面。这本书回顾了与GIS处理的独立和特殊问题联系在一起的问题求解和决策支持GIS应用和工具。（Miller，2004）将空间数据和空间分析与Tobler提出的地理第一法则相关联。（Goodchild，1992）把GIS作为地理信息科学而不是一种工具或系统进行了讨论。（Goodchild，1992a）给出了一个地理数据建模的很好概述。（Shekhar & Chawla，2002）讨论了作为GIS核心的空间数据管理问题与方法。（Friis-Christensen & Jensen，2001）讨论了GIS建模的不同方面和需求，并评价了现行的GIS模型表示法。（Borges，Laender & Davis，1999）给出了GIS建模的空间完整性约束，以及提出使用OMT模型的扩展。（Cockcroft，1997）提出了GIS数据的约束集合。（Gordillo & Balaguer，1998）描述了GIS建模中的拓扑问题和领域数据，并提出了一个设计改进的可能方案。（Brodeur，Bedard & Proulx，2000）描述了当强调概念模型和数据字典时，结合国际标准，GIS数据库文档的重要性。（Indulska & Orlowska，2002）描述了GIS数据库中不同级别的聚合问题。（Worboys & Duckham，2004）探索了强调展现空间数据模型的GIS的计算机前景，他们强调了解释GIS数据时语境的重要性。（Maune，2001）提供了一个关于表面建模技术的回顾。（Bossomaier & Green，2002）是一个关于GIS操作、语言、元数据范例和标准的初级读本。（Peng & Tsou，2003）讨论了Internet GIS，它包括一整套新技术，旨在使得GIS移动性更强、功能更强大、更具灵活性，并能够更好地共享和传输地理信息。其他流行的GIS参考文献包括由美国商务部管理的有关美国道路的tiger文件（见（U.S.Department of Commerce，1993））。Laser_Scan的Web站点（http://www.1s1.co.uk/paper）是一个良好的信息源。环境系统研究所（ESRI）有一个优秀的各种级别的GIS文献图书馆，网址是http://www.esri.com。在http://www.esri.com/library/glossary/glossary.html可以查到关于GIS的术语。（Flynn & Pitts，2001）给出了关于arcINFO的技术回顾。（Zeilor，1999）提供了一个关于ESRI地理数据库的GIS数据建模指南，它包括Geodatabase中增强功能的回顾。爱丁堡大学维护了一个GIS Web资源列表，站点是http://www.geo.ed.ac.uk/home/giswww.html。美国地理调查（USGS）在站点http://www.usgs.gov/上有相关信息和数据。

4. 基因组数据库参考文献

近几年来，生物信息学逐渐成为热门的研究领域，围绕这个主题组织和召开了很多研讨会，包括CSB、ISMB、RECOMB和PSB。（Robbins，1993）概略介绍了人类基因组项目，而（Frenkel，1991）从人类基因组项目在生物信息学中的特殊地位详细介绍了该项目。（Cutticchia et al.，1993），（Benson et al.，2002）和（Pearson et al.，1994）是关于GDB、GenBank及OMIM的很好的参考文献。在美国的GeneBank、日本的DNA数据银行（DDBJ）（http://www.ddbj.nig.ac.jp/E-mail/homology.html）和欧洲分子生物学实验室（EBML）（参考（Stoesser G，2003））等组织的国际合作中，每天都在协作

数据库中交换数据以达到最优的同步。（Wheeler et al.，2000）讨论了当前允许用户访问和分析数据库中可用数据的各种工具。（Olken & Jagadish，2003）讨论了综合生物学的数据管理挑战。（Navathe & Patil，2004）概述了基因和蛋白质数据库发展，指出了最有挑战性的问题以及最具潜力的应用。

（Wallace，1995）是线性体基因研究的先驱，该项研究主要针对人类基因组的特定部分。（Anderson et al.，1981）也详细介绍了线性体基因的序列和组织的细节。（Kogelnik et al.，1997，1998）和（Kogelnik，1998）介绍了通过开发原型技术，来解决生物科学领域中遇到的数据管理问题。（Apweiler et al.，2003）对欧洲生物信息学研究所（European Bioinformatics Institute，EBI）维护的核心生物信息学资源进行了概述（例如SWISS-PROT+TrEMBL），并总结了这些资源重要的数据库管理问题。它们讨论了3种主要的数据库类型：序列数据库，如DDBJ/EMBL/GENEBANK核苷酸序列数据库；次数据库，例如PROSITE、PRINTS和PFAM；集成数据库，例如InterPro，它集成了6个主要的蛋白质签名数据库（PFAM、PRINTS、ProDom、PROSITE、SMART和TIGRFAM）中的数据。

欧洲生物信息学研究所的大分子结构数据库（Macromolecular Structure Database，E-MSD）是一个关系数据库（http://www.ebi.ac.uk/msd）（Boutselakis et al.，，2003），它设计为一个蛋白质和核酸结构，及其信息的单个访问点。这些数据库是从蛋白质数据银行（Protein Data Bank，PDB）中的条目导出的。搜索数据库包括扩展的导出性质集合，适合度标识符以及到其他EBI数据库的链接，包括InterPro、GO和SWISS-PROT，以及与SCOP、CATH、PFAM和PROSITE的链接。（Karp，1996）讨论了本节提到的各种数据库的互联问题，它定义了两类链接：集成数据的链接和有关数据库之间数据的链接。这在EcoCyc数据库的设计中得到了应用。

下面是一些重要的Web链接。人类基因组序列信息可以在http://www.ncbi.nlm.nih.gov/genome/seq/上找到。读者可以从以下网址：http://www.mitomap.org/ 访问MITOMAP数据库（1998年由Kogelnik创建）。（Brandon et al.，2005）描述了MITO-MASTER的最新更新。从网址http://expasy.hcuge.ch/sprot/可以访问到世界上最大的蛋白质数据库SWISS-PROT。http://probe.nalusda.gov:8080/acedocs/上提供关于ACEDB数据库的信息。基因本体论协会网址是http://www.godatabase.org。基因表达选集数据库在http://www.ncbi.nlm.nih.gov/geo上提供。（Barrett et al.，2005）描述了GEO数据库和相关工具的结构和发展。每年，*Nucleic Acid Research*的一月月刊专门用来介绍生物学应用方面的数据库。

致谢

本章包含对持有以下**注册商标**的公司和产品的引用：

- ESRI, ArcEdit, ArcPlot, AML, ArcINFO, ArcGIS, ArcPad, ArcIMS和ArcSDE是Environmental Systems Research Institute公司的注册商标。
- MapInfo和MapXtreme®是MapInfo公司的注册商标。
- Intergraph和GeoMedia Web Map是Intergraph公司的注册商标。
- Microsoft是Microsoft公司的注册商标。
- Informix和DataBlade是Informix公司的注册商标。
- Oracle是Oracle公司及其从属公司的注册商标。
- OpenGIS®是Open Geospatial Consortium公司的注册商标。
- IBM, DB2, DB2 Universal Database是IBM公司的注册商标。
- ERDAS IMAGINE是Leica Geosystems GIS & Mapping, LLC的注册商标和独有财产。
- 其他名称分别是它们所有者各自的注册商标。

附录 A

ER模型的可选图形表示法

图A-1展示了许多不同的用于描述ER和ERR模型概念的图形表示法。但遗憾的是，到目前为止还没有一种标准的图形表示法，不同的数据库设计者通常选用不同的图形表示法。同样，不同的**计算机辅助软件工程（CASE）**工具和**面向对象分析（OOA）**方法也采用不同的图形表示法。其中一些图形表示法与具有附加的概念和约束的模型相关联，这些概念和约束是在第3章中所描述的ER和ERR模型所不具有的，其他一些模型则具有相对较少的概念和约束。我们在第3章中采用的图形表示法与最初的ER图形表示法非常接近，这种图形表示法现在仍然广泛应用。在这里将要讨论一些其他可选择的图形表示法。

图A-1a展示了描述实体类型/类、属性和联系的各种图形表示法。在第3章中，我们使用的是图A-1a中（i）中的符号标记，被命名为矩形、椭圆形和菱形。需要注意的是符号（ii）中用于描述实体类型/类、属性和联系的三种表示法很相似，但是它们却被不同的方法所使用，用于表示三个不同的概念。一些工具和方法学使用符号（iii）中的直线表示法来表示联系。

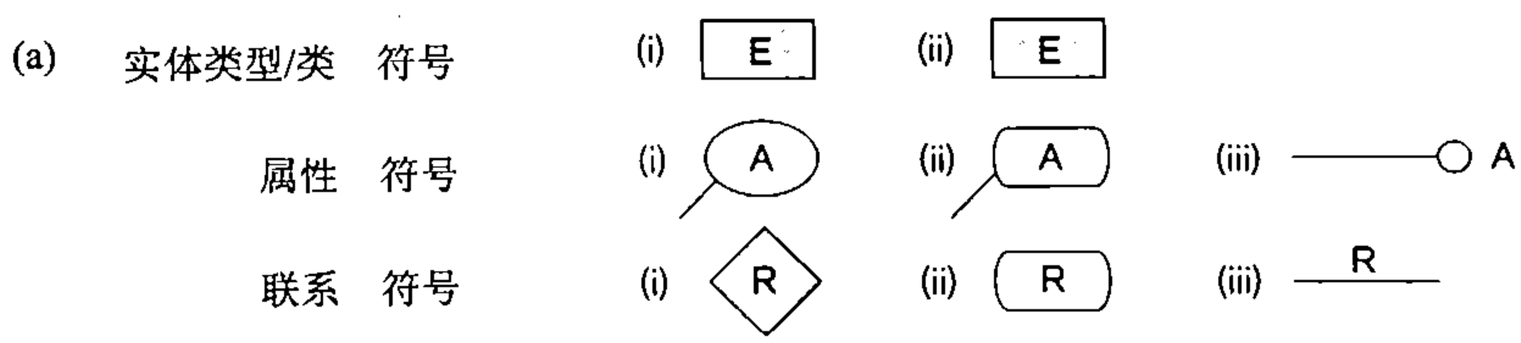
图A-1b展示了用于连接属性和实体类型的一些表示法。本书中采用的是（i）中的图形表示法。图形表示法（ii）中的属性用图A-1a中的（iii）来表示。最后两种图形表示法（iii）和（iv）在OOA方法和一些CASE工具中比较流行，尤其是最后一种，可以同时显示一个类的属性和方法，两者之间用一条水平直线隔开。

图A-1c展示了各种表示二元联系的**基数比例**（cardinality ratio）的图形表示法。本书第3章中采用的是图形表示法（i）。图形表示法（ii）（就是通常所说的**鸡爪**（chicken feet）表示法）现在非常流行。图形表示法（iv）使用箭头记号表示**函数引用**（从N端到1端），这个表示法类似于我们在关系模型中给出的用于标识外码的表示法（见图7-2）。图形表示法（v）（用于Bachman图中）采用相反方向的箭头（从1端到N端）。如果希望表示一对一联系（1:1），在图形表示法（ii）中使用一条没有鸡爪的直线表示，在（iii）中使用两半都是白色的菱形表示，而在（iv）中则在两边都放有箭头。如果表示多对多联系（M:N），在图形表示法（ii）中使用两端都放有鸡爪的直线来表示，在（iii）中使用两半都是黑色的菱形，而在（iv）中则使用无任何箭头的直线来表示。

图A-1d给出了描述（min，max）约束的几个表示法的变体，（min，max）约束用于描述基数比以及整体/部分的参与情况。图形表示法（ii）是图3-15中使用和在3.7.4节中讨论的表示法的可选表示法。回想一下我们的表示法，它规定了这样一种约束，每一个实体必须参与不低于最小值、不大于最大值数目的联系实例。因此，对于一对一联系（1:1），每个最大值都是1；对于多对多联系（M:N），每个最大值都是n。而一个大于零（zero）的最小值指定了总参与数（存在依赖）。在用直线表示联系的方法中，通常将（min，max）约束交换位置，如（iii）所示。在一些工具中（以及UML表示法中）常有一种变形，如(v)所示。还有另一种流行的技术，它的位置与（iii）相同，只是将min显示成o（“噢”的发音或圆，代表零）或显示成 |（垂直线，代表1）；将max显示成 |（垂直线，代表1）或显示成鸡爪符号（代表n），如（iv）所示。

图A-1e给出了一些显示特化/泛化的图形表示法。第4章中使用的是图形表示法（i），其中圆圈中的d用于指定不相交的子类（S1，S2，S3），而圆圈中的o指定重叠的子类。图形表示法（ii）使用G（代表泛化）来指定不相交，而使用Gs来指定重叠；其中一些表示法使用实心箭头而另一些则使用空心箭头（在图旁边显示）。图形表示法（iii）使用指向超类的三角形，而图形表示法（v）使用指向子类的三角形；也可以在同一种方法中同时使用这两种表示法，这时，图形表示法（iii）表示泛化而（v）表示特化。图形表示法（iv）使用一个方框符号表示超类，子类则位于这个方框符号之中。使用图形表示法（vi）时，有些

设计者使用单线箭头，有些则使用双线箭头（在图旁边显示）。



图A-1　可选的图形表示法：(a)实体类型/类、属性和联系的符号；(b) 显示属性；(c) 显示基数比；(d)各种（min，max）表示法；(e)显示特化/泛化的表示法

图A-1仅仅给出了一些曾经被使用或建议采用的用于描述数据库概念的图形表示法。其他图形表示法以及在上面所提到的各种表示法的不同组合形式也一直被使用着。建立一项每个人都遵守的标准是十分有用的，从而可以避免误解和减少混乱。

附录 B

磁盘参数

最重要的磁盘参数是定位一个任意的磁盘块、给定这个磁盘块的地址、然后在主存缓冲区和磁盘之间传输此块所花费的时间。这就是访问磁盘块的随机访问时间(random access time)。这里需要考虑三个时间元素。

(1) **寻道时间**(seek time，用s来表示)：在可移动磁头的磁盘中，为了使读/写磁头停留在正确的磁道上而进行的机械定位所花费的时间就是寻道时间（在采用固定磁头的磁盘中，这个时间是电子切换到相应的读/写磁头所花费的时间）。这个时间在采用可移动磁头的磁盘中是可变的，其值依赖于当前读/写磁头所处的磁道与块地址所指定的磁道之间的距离。通常情况下，磁盘制造厂商会提供一个毫秒级的平均寻道时间。一般的平均寻道时间范围介于10~60 ms之间。对于磁盘和内存之间存在的块传输延迟而言，它是主要的“罪人”。

(2) **旋转延迟**(rotational delay，用rd来表示)：当读/写磁头定位到正确的磁道后，用户必须等待指定磁盘块的起始位置旋转到读/写磁头下方。平均统计，它大约花费磁盘旋转一周所需时间的一半，但是实际花费的时间介于可以立即进行存取（如果在读/写磁头寻道完成后，指定磁盘块的起始位置正好处于读/写磁头之下）到磁盘完全旋转一周（如果在读/写磁头寻道完成后，指定磁盘块的起始位置刚刚越过磁头）的时间之间。如果采用每分钟旋转p圈（rpm）来定义磁盘转速，那么平均旋转延迟rd可以表示如下：

$$rd = (1/2)\times(1/p)\ \text{min} = (60\times1000)/(2\times p)\ \text{ms} = 30000/p\ \text{ms}$$

典型的p值为10 000 rpm，与此对应的rd值为 3 ms。在采用固定磁头的磁盘中，由于寻道时间可以忽略不计，所以对于传输磁盘块来说，这个时间元素是最主要的延迟。

(3) **块传输时间**（block transfer time，用btt来表示）：当读/写磁头定位到指定磁盘块的起始位置后，还需要花一些时间来传输这个块中的数据。块传输时间依赖于块的大小、磁道的大小和磁盘旋转速度。如果用tr字节/毫秒来表示磁盘的传输速率，块的大小为B字节，则块传输时间为

$$btt = B / tr\ \text{ms}$$

如果我们有一个磁道大小为50 KB、转速p为3600 rpm的磁盘，则它的传输速率为：

$$tr = (50\times1000) / (60\times1000/3600) = 3000\text{字节/ms}$$

因此，$btt = B / 3000$ ms，B是用字节表示的块的大小。

给定磁盘块的地址，寻找和传输一个磁盘块所需要的平均时间大约为 $(s + rd + btt)$ ms。这个公式同时适用于读或写一个磁盘块。因此，减少这个时间的最主要的方法是在同一个柱面的一个或多个磁道上，一次传输多个磁盘块，那么仅仅在传输第一个磁盘块时需要寻道时间。在同一个柱面上连续传输k个不相连的磁盘块，我们大约需要：

$$s + (\,k\,(\,rd + btt\,)\,)\ \text{ms}$$

因此，正如我们在第13章中所讨论的，因为要连续地读或写k个磁盘块，所以需要两个或更多主存储器中的缓冲区。当传输同一个磁道或柱面上的相连磁盘块时，每个磁盘块的传输时间则会大大地减少。同时，除了第一个磁盘块外，这种方法也去掉了其他所有磁盘块的旋转延迟，于是传输k个相连磁盘块的估计时间为：

$$s + rd + (\,k\,btt\,)\ \text{ms}$$

考虑**块间隙**（interblock gap）（见13.2.1节）可以更精确地计算传输连续磁盘块时的估计时间，块间隙中包括了某种信息，这种信息能够使读/写磁头确定将要读取的块。通常情况下，磁盘制造厂商会提供一个大容量传输速率（bulk transfer rate）参数，这个参数考虑了进行连续读取存储块时的间隙尺寸，用*btr*来表示。如果间隙尺寸为*G*字节，则有

$$btr = (B / (B + G)) tr \text{ 字节/ms}$$

大容量传输速率是在数据磁盘块中传输有用字节的速率。当磁盘旋转时，磁盘读/写磁头必须检查包括块间隙字节在内的磁道上的所有数据，而块间隙中存储的并不是真正的数据而是控制信息。当使用大容量传输速率时，传输若干连续块中的一个块内的有用字节所需的时间为*B*/*btr*。因此，读取存储在同一柱面中的连续*k*个磁盘块的估计时间为

$$s + rd + (k (B / btr)) \text{ ms}$$

磁盘的另一个参数是**重写时间**（rewrite time）。将一个磁盘块从磁盘读取到主存缓冲区，然后更新缓冲区中的内容，再将缓冲区的内容写回到同一个磁盘块中，在这种情况下就会使用这个参数。在很多情况下，更新主存缓冲区所花费的时间要少于磁盘旋转一周的时间。如果我们知道缓冲区已经为重写做好准备，那么系统就能够将磁头保持在同一个磁道上，并且在下一个磁盘旋转期间可以将更新后的缓冲区写回磁盘块中。因此，通常用磁盘旋转时间来估算重写时间（T_{rw}），

$$T_{rw} = 2\, rd \text{ ms} = 60000/\, p \text{ ms}$$

在这里，我们将所讨论的参数和所用到的符号总结一下。

寻道时间：　　s ms
旋转延迟：　　rd ms
块时间：　　btt ms
重写时间：　　T_{rw} ms
传输速率：　　tr 字节/ms
大容量传输速率：　　btr 字节/ms
块的大小：　　B 字节
块间隙的大小：　　G 字节
磁盘转速：　　p rpm（revolution per minute，圈数/分钟）

附录C

QBE语言概述

示例查询（Query-By-Example , QBE）语言的重要性在于：它是数据库系统中语法最简单的，并且也是最早的图形化查询语言之一。它是由IBM研究院开发的，现在作为IBM商业产品查询管理工具（Query Management Facility，QMF）与DB2接口选项的一部分。该语言还在Paradox DBMS中得到实现，与Microsoft Access DBMS（见第10章）中的指向和单击类型的接口相关。QBE不同于SQL，使用QBE用户不必按固定的语法显式地指定一个查询，查询是通过填充屏幕上显示的关系**模板**（template）建立的。图C-1显示了这些模板如何查找图7-1中的数据库。用户不必记忆属性或关系的名字，因为它们会作为模板的一部分显示出来。另外，用户也不必遵循严格的语法规则来说明查询，可以将常量和变量输入模板的列来构造一个与检索或更新请求相关的**示例**（example）。我们将看到，QBE与域关系演算相关，其初始规范说明在关系上是完备的。

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date

DEPT_LOCATIONS

Dnumber	Dlocation

PROJECT

Pname	Pnumber	Plocation	Dnum

WORKS_ON

Essn	Pno	Hours

图C-1　用QBE显示的图5-5的关系模式

C.1　QBE 中的基本检索

在QBE中检索查询是通过填充表的模板中一行或多行来指定的。对单个关系的查询，我们可能在关系模板的列中输入常量或者输入**示例元素**（example element, 一个QBE项）。一个示例元素表示一个域变量，指定为以下划线（_）为前缀的示例值。另外，在特定列中输入前缀P.（称为P点运算符）表示想要打印（或显示）结果中这些列的值。常量用来指定在这些列中必须完全匹配的值。

例如，考虑查询Q0：检索John B.Smith的出生日期和地址。在图C-2a到图C-2d中，展现了在QBE中如何用逐步简洁的方法来指定这个查询。图C-2a中一个雇员的示例表示为我们感兴趣的行类型。令John B.Smith在列Fname、Minit及Lname中为常量，我们在这些列中指定了一个精确的匹配。其他列的值都是以下划线作前缀，这表示它们是域变量（示例元素）。在Bdate和Address列中有P.前缀，这表明我们想要输出这些列的值。

(a) EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	_123456789	P._9/1/60	P._100 Main, Houston, TX	_M	_25000	_123456789	_3

(b) EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith		P._9/1/60	P._100 Main, Houston, TX				

(c) EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith		P._X	P._Y				

(d) EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith		P.	P.				

图C-2　在QBE中指定查询Q0的4种方法

Q0还可以简化为图C-2b表示。这里无需指定我们不感兴趣的列的示例值。而且，由于示例值完全是任意的，所以可以只指定它们的变量名，如图C-2c所示。最后，可以完全去掉示例值，如图C-2d所示，只是在要检索的列下指定P.。

为了看到QBE查询与域关系演算是何等的相似，比较图C-2d与域演算中的Q0（简化的），如下所示：

Q0 : { uv | EMPLOYEE(qrstuvwxyz) **and** *q*='John' **and** *r*='B' **and** *s*='Smith'}

可以把QBE模板的每一列认为是一个隐式域变量，这样，Fname对应域变量*q*，Minit对应*r*，……，Dno对应*z*。在QBE查询中，带有P. 的列对应于域演算中左部指定的变量，而带有常量值的列对应于带有相等性选择条件的元组变量。条件EMPLOYEE（qrstuvwxyz）及存在量词(existential quantifier)在QBE查询中是隐含的，因为使用了对应于EMPLOYEE关系的模板。

在QBE中，用户的界面首先通过显示所有关系名的列表来表示允许用户选择对于一个查询所需要的表（关系）。然后显示用户所选定关系的模板。用户移动到模板上合适的列上并指定该查询。系统提供专门的功能键，在模板之间移动并执行某些功能。

现在给出阐述QBE基本工具的一个示例。在输入常量值之前，可以在列中输入不是“=”的比较操作符（如>或≥）。例如，查询Q0A：列出在项目1上工作，且每周工作时间超过20小时的雇员的社会保险号，该查询可以如图C-3a所示那样来指定。对于更复杂的条件，用户可以使用**条件框**（condition box），它是通过按下特定的功能键来创建的。然后用户可以输入复杂的条件[①]。

例如，查询Q0B:列出在项目1或项目2 上工作，且每周工作时间超过20小时的雇员的社会保险号”，该查询可以如图C-3b所示那样来指定。

某些复杂条件可以不使用条件框来指定。其规则是：在关系模板的同一行上指定的所有条件，使用逻辑连接符and连接（选择的元组必须满足所有的条件），而在不同的行上指定的条件以or连接（必须至少满足一个条件）。因此，可以如图C-3c所示的那样来指定Q0B，在模板中输入不同的两行。

考虑查询Q0C：列出参加项目1并且也参加项目2的雇员的社会保险号，该查询不能像图C-4a所示的那样来指定，图C-4a表示的是参加项目1或者参加项目2的雇员。示例变量_ES将会把自己绑定到元组<-,1,->中的Essn值和元组<-,2,->中的Essn值。图C-4b说明了如何正确地指定Q0C，条件框中的条件（_EX = _EY）使得变量_EX和_EY只绑定到同一个Essn值上。

通常，一旦指定查询，就在模板中适当的列中显示结果值。如果结果值包括太多的行以至于不能在屏幕上完全显示时，大多数QBE实现都会有功能键来支持上下滚动。同样，如果模板太宽以至于不能在屏幕上完全显示时，则可以横向地滚动来查看所有模板。

① 条件框中不允许使用带有“¬”符号的否定运算符。

(a)

WORKS_ON

Essn	Pno	Hours
P.		> 20

(b)

WORKS_ON

Essn	Pno	Hours
P.	_PX	_HX

CONDITIONS

_HX > 20 and (PX = 1 or PX = 2)

(c)

WORKS_ON

Essn	Pno	Hours
P.	1	> 20
P.	2	> 20

图C-3　在QBE中指定复杂的条件：(a) 查询Q0A；(b) 带条件框的查询Q0B；(c) 不带条件框的查询Q0B

(a)

WORKS_ON

Essn	Pno	Hours
P._ES	1	
P._ES	2	

(b)

WORKS_ON

Essn	Pno	Hours
P._EX	1	
P._EY	2	

CONDITIONS

_EX = _EY

图C-4　指定参与两个项目的EMPLOYEES：(a) AND条件的不正确指定；(b) 正确的指定

在QBE中，指定连接操作的方式是通过在要被连接的列上使用相同变量[①]。例如，查询Q1：列出在'Research'部门工作的所有雇员的姓名和地址，该查询可以如图C-5a所示的那样来指定。可以在单个查询中指定任意数量的连接。我们还可以指定一个**结果表**（result table）用来显示连接查询的结果，如图C-5a所示，如果结果表包括来自两个或多个关系中的属性，就需要指定一个结果表。如果没有指定结果表，系统将提供在各个关系的列中的查询结果，这会使得结果难以解释。图C-5a还显示了QBE的另一个特性，即指定将被检索关系的所有属性，这是通过在关系模板中的关系名下放置一个P. 操作符来实现的。

(a)

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
_FN		_LN			_Addr				_DX

DEPARTMENT

Dname	Dnumber	Mgrssn	Mgr_start_date
Research	_DX		

RESULT			
P.	_FN	_LN	_Addr

(b)

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
_E1		_E2						_Xssn	
_S1		_S2	_Xssn						

RESULT				
P.	_E1	_E2	_S1	_S2

图C-5　显示QBE中的JOIN和结果关系：(a) 查询Q1；(b) 查询Q8

对于表的自身连接，需要指定不同的变量来表示对该表的不同引用。例如，查询Q8：对每个雇员，检索雇员的姓名及他或她的直接主管的姓名，该查询可以如图C- 5b所示的那样来指定。其中，以"E" 开头的变量表示雇员，以"S"开头的变量表示主管。

① 变量在QBE手册中称为**示例元素**（example clement）。

C.2　QBE中的分组、聚集和数据库修改

接下来，考虑需要分组或者聚集功能的查询类型。可以在列中指定分组操作符“*G.*”来表示元组将根据列中的值进行分组。可以指定一些常用的函数，如AVG.、SUM.、CNT.、MAX.和MIN.。在QBE中，函数AVG.、SUM.、CNT.在默认情况下是应用到一个组的不同值上。如果我们想把这些函数应用到所有的值上，我们必须使用前缀ALL[①]。这个约定不同于SQL中的约定，SQL中的默认函数是应用到所有值上的。

图C-6a表示查询Q23，它用于统计EMPLOYEE关系中的不同工资值。查询Q23A（见图C-6b）用于统计所有工资值，这与计算雇员的数量是一样的。图C-6c表示查询Q24，它检索部门号、雇员号以及每个部门的平均工资；因此，如同*G.* 函数所指示的那样，Dno列用于分组。可以在单个列中指定几个操作符如G.，P.，以及ALL.。图C-6d表示查询Q26，它显示了有超过两个雇员参加的项目的名称以及参加该项目的雇员数量。

(a) **EMPLOYEE**

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
							P.CNT.		

(b) **EMPLOYEE**

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
							P.CNT.ALL		

(c) **EMPLOYEE**

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
			P.CNT.ALL				P.AVG.ALL		P.G.

(d) **PROJECT**

Pname	Pnumber	Plocation	Dnum
P.	_PX		

WORKS_ON

Essn	Pno	Hours
P.CNT.EX	G._PX	

CONDITIONS

CNT._EX > 2

图C-6　QBE中的函数和分组：(a) 查询Q23；(b) 查询Q23A；(c) 查询Q24；(d) 查询Q26

QBE有一个否定符号¬，它的使用方式类似于SQL中的NOT EXISTS函数。图C-7展示了查询Q6，它列出了没有受赡养人的雇员姓名。否定符号¬表示：只从关系EMPLOYEE中选择这样的_SX变量的值，这些_SX变量的值在关系DEPENDENT中没有出现。在Essn列中放置¬_SX也可以达到同样的效果。

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
P.		P.	_SX						

DEPENDENT

	Essn	Dependent_name	Sex	Bdate	Relationship
¬	_SX				

图C-7　使用查询Q6说明否定运算符

① QBE中的ALL与**全称量词**（universal quantifier）无关。

尽管QBE语言最初提出时经证明支持SQL的EXISTS和NOT EXISTS函数的等价功能，但是在QMF（在DB2系统下）中的QBE实现并没有提供这种支持。因此，这里我们讨论的QBE的QMF版本在关系上是不完备的。例如，不能指定查询Q3：找出参加由部门5负责的所有项目的雇员。

QBE有3个运算符可以修改数据库：I.用于插入，D.用于删除，U.用于更新。插入和删除运算符在关系名下的模板列中指定，而更新运算符在要被更新的列下指定。图C-8a表示如何插入一个新的EMPLOYEE元组。对于删除，我们首先输入D.运算符，然后根据一个条件（见图C-8b）指定要被删除的元组。为了更新一个元组，要在属性名下指定U.运算符，后面跟着属性的新值。我们也可以以通常的方式来选择要被更新的元组。图C-8c表示了一个更新请求，即“John Smith”的工资增加10%，并且重新分配他到部门4。

(a)　EMPLOYEE

	Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
I.	Richard	K	Marini	653298653	30-Dec-52	98 Oak Forest, Katy, TX	M	37000	987654321	4

(b)　EMPLOYEE

	Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
D.				653298653						

(c)　EMPLOYEE

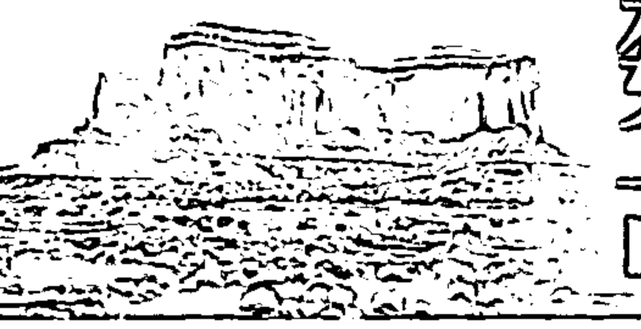
Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John		Smith					U._S*1.1		U.4

图C-8　在QBE中修改数据库：(a) 插入；(b) 删除；(c) 更新

QBE还有数据定义功能。数据库的表可以以交互方式指定，并且表的定义还可以通过增加、重新命名或删除列来进行更新。还可以对每一列指定各种特征，如该列是否是关系的码，其数据类型是什么，是否将在该字段上建立一个索引等。QBE还有视图定义、授权、以备以后使用的存储查询定义等功能。

QBE并没有采用SQL的线性风格；它是一种二维语言，因为用户要在整个屏幕上移动以指定一个查询。经过用户的测试表明QBE要比SQL更易于使用，尤其是对于非专业人员。在这个意义上说，QBE是第一个用户友好的可视化关系数据库语言。

最近，商业数据库系统开发了许多其他的用户友好的界面。菜单、图形以及窗体的使用正变得十分普遍。部分地填充窗体来设定一个检索请求与使用QBE是类似的。将来，商业关系数据库可能会提供目前还不常见的可视化查询语言。



数据库系统基础 高级篇（第5版）

“本书是数据库教材的典范，书中提供了丰富的内容供教师自由选择，可以根据授课需要对理论与实践、设计与编程、应用与实现，以及历史问题与现代观点等进行取舍。”

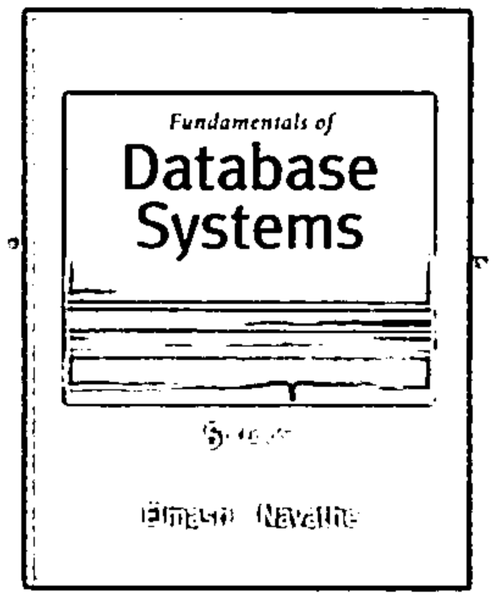
——Henry A. Etlinger，罗彻斯特理工学院

“这是一本杰出的、全新的数据库教科书，对本科生和研究生课程均很适用。书中提供了非常好的示例，清晰地描述了如何设计优良且可操作的数据库，以及如何从现存的数据库检索和操纵数据。”

——Peter Ng，得克萨斯大学泛美分校

《数据库系统基础》是世界众多高校广泛采用的数据库课程教科书。它清晰地阐述了数据库系统理论和设计问题，涵盖了数据库系统的设计、实现和管理方面的诸多专题，使用优秀的实例引入最新的现代数据库技术。第5版对书中内容进行了进一步完善和更新，反映了数据库技术和应用开发的最新发展趋势。书中新增了一些应用实例，而且章节后面提供了练习题和新的实验题，让读者既可以学到现代数据库技术，又可以获得构建数据库应用系统的实践经验。

《数据库系统基础》第5版的中译本分“初级篇”和“高级篇”两册出版。本书是其“高级篇”，主要介绍事务处理的概念，对象和对象-关系数据库，安全性，高级建模，分布式等高级主题和XML、数据挖掘、数据仓库等新兴技术。



Fundamentals of Database Systems
Fifth Edition



Ramez Elmasri



Shamkant B. Navathe

PEARSON Addison Wesley

www.PearsonEd.com

ISBN 978-7-115-17160-3

9 787115 171603 >

ISBN 978-7-115-17160-3/TP

定价：49.00 元

本书相关信息请访问：图灵网站 http://www.turingbook.com
读者/作者热线：(010) 88593802
反馈/投稿/推荐信箱：contact@turingbook.com

上架建议 计算机/计算机科学/数据库理论

人民邮电出版社网址 www.ptpress.com.cn

PEARSON Education